FARO DE ÁMBAR · DOCUMENTO COMPLETO DEL PROYECTO

Faro de Ámbar

Todo el juego, de la v0.1 a la v1.5

13 de noviembre de 2026 · Versión actual del prototipo web: **1.5** · Se vuelve a generar en cada versión

16 versiones
v0.1 y v0.2 (antes de los PRD) y 14 con su PRD, de la v0.3 a la v1.6.

14,867 líneas de juego
17,968 líneas con herramientas y documentos; 75 modelos 3D hechos con Blender.

190 pruebas aprobadas
Sumando los PRD; además 71 archivos de prueba en `faro/tests/`.

Qué hay en este documento

1. Qué es Faro de Ámbar y cómo se juega
2. Línea de tiempo de todas las versiones
3. Registro de mejoras (se amplía en cada versión)
4. Novedades detalladas de cada versión
5. Catálogo actual: tropas, edificios, modos y mapas
6. Cómo está hecho: archivos, recursos y forma de construirlo
7. Cómo se prueba
8. Plan para Roblox (PRD v1.6 del usuario)
9. Anexo: el código completo, archivo por archivo

> Regla del proyecto: cada versión nueva agrega sus mejoras al registro (sección 3) y vuelve a generar este documento. Los PRD y PDF de cada versión siguen siendo documentos aparte, en `faro/docs/prd/`.

01 · EL JUEGO

Qué es Faro de Ámbar y cómo se juega

Juego de estrategia en tiempo real sobre un mapa de hexágonos, hecho para el navegador. Funciona en iPad, celular y computadora (Safari o Chrome), sin instalar nada. Es el banco de pruebas de las reglas del PRD 1.5 antes de llevarlas a Roblox Studio.

Empiezas con tu Núcleo, una Granja y un Constructor: entrena más, levanta tu base, conquista islas y mantén el **Faro** (un 50 % más de monedas). Los rivales son bots que construyen, mejoran, capturan, cruzan el mar y atacan.

Versión actual: 1.5 · PRD y PDF en `docs/prd/`.

Cómo jugar

Abre `faro.html` en Safari o Chrome. En el iPad lo más cómodo es el enlace publicado.

Acción	Toque (iPad / celular)	Computadora
Mover el mapa	Arrastra con un dedo	Arrastra o WASD / flechas
Zoom y giro	Pellizca y gira con dos dedos	Rueda del ratón, Q y E
Seleccionar	Toca una tropa o un edificio	Clic
Todas las tropas de un tipo	Doble toque	Doble clic
Varias tropas	Botón «Recuadro» y arrastra	Mayús + arrastrar
Rueda de selección	Mantén pulsado el mapa y elige (Tierra, Mar, Aire, Todo, Constructores, grupos)	Tab
Patrullar / mantener posición	Botones «Patrullar» (y luego toca el otro punto) y «Mantener posición» en el panel de tropas	Igual
Grupos 1, 2 y 3	«Guardar en 1/2/3» y botones 1-2-3 junto al minimapa (dos toques: ir)	Número elige; Ctrl, Cmd o Mayús + número guarda
Avanzar atacando	Botón en el panel de tropas y luego toca el destino	F y clic
Constructores	Botón junto al minimapa (primero los que no trabajan)	C
Mover / atacar	Toca un destino o un enemigo	Clic izquierdo o derecho
Construir	Elige edificio → toca una casilla dorada → «Construir» (puedes poner varias seguidas) → «Terminar»	Igual; R gira, Enter construye
Mejorar	Toca un edificio terminado → «Mejorar»	Igual
Capturar	Lleva infantería junto a una Bandera o al Faro	Igual

Acción	Toque (iPad / celular)	Computadora
Pausa / salir	Botón de pausa → «Salir al menú» (la partida queda guardada)	Esc o P

Ganar: destruye todos los Núcleos enemigos (en «Rey del Faro», también manteniendo el Faro 3 minutos). **Perder:** si cae tu Núcleo.

02 · HISTORIA

Línea de tiempo de todas las versiones

Cada versión desde la 0.3 empezó con un PRD (el plan) y terminó con su PDF con los resultados reales de las pruebas.

Versión	Fecha	Qué trajo	Pruebas del PRD
0.1	6 nov 2026	Primera versión jugable, entonces llamada «HEXBOX».	—
0.2	7 nov 2026	Transporte, Pozo petrolero, Batería antiaérea y Lanzamisiles.	—
0.3	7 de noviembre de 2026	Web ahora · Roblox después · PRD + PDF	12 aprobadas, 2 pendientes
0.4	7 de noviembre de 2026	Bots con carácter · Interfaz que informa · Medido	16 aprobadas, 1 pendientes
0.5	7 de noviembre de 2026	Constructores · Niveles 1-3 · Blender	29 aprobadas, 2 pendientes
0.6	8 NOV 2026	Ritmo · Fluidez · Modelos y efectos	14 aprobadas
0.7	8 NOV 2026	Campaña · Controles · Tu iPad	13 aprobadas
0.8	8 NOV 2026	Interfaz · Bots (NPC) · Recursos	15 aprobadas, 1 fallidas, 1 pendientes
0.9	8 NOV 2026	Reglas · Ejército · Se ve mejor	20 aprobadas
1.0	8 NOV 2026	Construir y mejorar · Más poder · Más reto y se ve mejor	11 aprobadas
1.1	8 NOV 2026	Jugar con otros · Defensa · Más completo	10 aprobadas
1.2	10 NOV 2026	Controles y equipo · Colección · Mundo	11 aprobadas
1.3	11 NOV 2026	Arreglar · Seguir tu documento · Poner en orden	14 aprobadas
1.4	13 NOV 2026	Mandar mejor · Volver cada día · Tu iPad	13 aprobadas
1.5	13 NOV 2026	Portada nueva · Núcleo, edificios y tropas · Caza de errores	12 aprobadas
1.6	13 NOV 2026	Más profesional · Más claro al jugar · Todo documentado	0 aprobadas, 9 pendientes

03 · REGISTRO

Registro de mejoras

Una entrada por versión, de la más nueva a la más vieja. **Cada versión nueva agrega aquí sus mejoras y partes importantes** antes de publicarse (regla del proyecto). El detalle completo de cada una está en su PRD (`docs/prd/`).

v1.6 · en curso

- Documento completo del proyecto (este documento) con historia, archivos y todo el código; se vuelve a generar en cada versión.
- PRD Roblox v1.6 del usuario en PDF, con sus capturas y el esquema de la interfaz.
- Pruebas automáticas guardadas en `faro/tests/` para que el proyecto tenga todo su código.

v1.5 · 13 nov 2026

- Sin decoraciones de temporada (Calabazas e Invierno), a pedido del usuario. Las temporadas de la Liga y del Pase siguen.
- Portada nueva: JUGAR grande dentro de la imagen, modo con miniatura del mapa, iconos dibujados y fichas de números.
- Núcleo nuevo hecho con Blender: ciudadela con torre central que crece por nivel, color del jugador y aro de luz.
- Edificios con banderín del color del dueño, marcas de nivel, barra de mejora y aviso «zz» si están parados.
- Tropas con anillo de color, icono de tipo al alejar la cámara y ficha con daño, alcance, velocidad y contra qué son buenas.
- Caza de errores: 30 partidas largas con chequeos y 30 minutos de toques al azar. Arreglados: Dúos en Arrecife Doble, reparación sin camino, «Avanzar atacando» quieto, fila de jugadores partida y botón «¡A jugar!» escondido en el celular.
- Pruebas: 12 de 12 aprobadas; 96 pruebas anteriores bien.

v1.4 · 12-13 nov 2026

- Patrullar, Mantener posición, tropas veteranas (1 a 3 estrellas), reparación sola y producción continua («Siempre»).
- 12 logros, Top jugadores y desafío semanal.
- Opciones para el iPad: ahorro de batería, colores para daltonismo (probados con simulación) y letra grande.
- Arreglo: la Barcaza va a buscar a los soldados que no pueden llegar a su orilla.
- Pruebas: 13 de 13 aprobadas.

v1.3 · 10-11 nov 2026

- Los 29 puntos de la revisión «Bugs y mejoras» del usuario: «Avanzar atacando», abandono de la Liga, misil nuclear, Cúpula, agua, guardado más liviano, bots repartidos, panel compacto, botones de 44 px.
- Rangos de la Liga y tutorial de 3 pasos con Barcaza según el documento del usuario; copia del progreso para llevarlo a otro iPad.
- Pruebas: 14 de 14 aprobadas.

v1.2 · 9-10 nov 2026

- Rueda de selección, mensajes a los aliados, tabla de la Liga, aspectos y planos en el Taller.
- Mapas nuevos (Mesetas de Cobre y Dunas Rojas) con miniaturas, tutorial que señala qué tocar y temporadas.
- Pruebas: 11 de 11 aprobadas.

v1.1 · 9 nov 2026

- Alianzas, modo Dúos, Clasificatoria y órdenes a los aliados.
- Escudo antimisiles, música, modo espectador y resumen final con gráficas.

v1.0 · 9 nov 2026

- Botón «Construir», comunidad de 240, radiación del misil, «Mejorar todo» y mejoras automáticas, salva de 3 misiles.
- Dificultad Difícil más fuerte, Aeropuerto, Caza y Bombardero, portada 3D.

v0.9 · 8 nov 2026

- 30 mejoras pedidas: límites, población 180, Casas, escudos de 3 colores, construir de un toque, tierra fértil, daño a edificios.
- Ejército nuevo: Aeropuerto, cazas, bombardero, portaaviones, fragata, lancha, fábrica, médico, torreta 4, misil nuclear, helicópteros.
- Soldados realistas, AIR DEFENSE, Núcleo y portada; vuelta al suelo de colores lisos de la v0.6.

v0.8 · 8 nov 2026

- Bots más listos (defensa, línea, fuego concentrado, hostigamiento, aliados).
- Interfaz: avisos en el borde, barra de producción, iconos y ficha del enemigo.
- Caminar y correr de los soldados con captura de movimiento de CMU (Blender).

v0.7 · 8 nov 2026

- Campaña «La ruta del Faro» con 6 misiones y estrellas.

- Mejores controles: grupos 1-2-3, formación en línea, avanzar atacando.

v0.6 · 8 nov 2026

- Ritmo más rápido (Constructor inicial, bots que atacan antes), modo fluido y calidad automática.
- 15 modelos nuevos con Blender, efectos y sonidos.

v0.5 · 7 nov 2026

- Constructores, niveles 1 a 3, cinco energías, lugares de recursos y límites.
- Vestíbulo con progresión, tutorial, volver a la partida, guardado y modo creador.
- Primeros modelos 3D hechos con Blender y luz de ambiente de Poly Haven.

v0.4 · 7 nov 2026

- Bots con personalidad; interfaz con alertas, marcador y fichas.

v0.3 · 7 nov 2026

- Nace «Faro de Ámbar»: el Faro, Banderas, energía, escudos, Cúpula, construcción con vista previa y 6 modos.

v0.2 · 7 nov 2026

- Transporte, Pozo petrolero, Batería antiaérea y Lanzamisiles.

v0.1 · 6 nov 2026

- Primera versión jugable, entonces llamada «HEXBOX»: mapa de hexágonos, Núcleo, economía y bots.

04 · DETALLE

Novedades detalladas de cada versión

Novedades de la v1.5

- **Sin decoraciones de temporada:** ya no hay calabazas ni nieve por la fecha. Las temporadas de la Liga y del Pase siguen.
- **Portada nueva:** la imagen grande con el botón JUGAR adentro, el modo con la miniatura del mapa, iconos dibujados y tus números en fichas.
- **Núcleo nuevo (Blender):** una ciudadela con una torre central que crece en cada nivel, franjas y techos del color del jugador y un aro de luz que gira alrededor del cristal. El rayo del cielo tiene el color del jugador.
- **Edificios:** banderín del color del dueño, marcas doradas de nivel, barra celeste mientras se mejoran y aviso «zz» si un edificio que entrena se queda sin cola.
- **Fichas:** la cola dice cuánto falta y cada tropa tiene su «×» para quitarla (devuelve el dinero); lo que cambia al mejorar se ve siempre; las tarjetas de tropas ya no se cortan.
- **Tropas:** anillo del color del jugador debajo de cada una, icono de su tipo al alejar la cámara, ficha con vida, daño, alcance, velocidad y contra qué es fuerte o débil; con varias elegidas, tocar un tipo deja solo esas.
- **Errores arreglados (caza de errores):**
- Dúos en el mapa Arrecife Doble no se podía empezar.
- Un Constructor o Ingeniero mandado a reparar un edificio rodeado se quedaba esperando para siempre.
- «Avanzar atacando» podía quedar quieto viendo enemigos al otro lado del agua.
- La fila de jugadores de arriba se partía en dos líneas en el iPad.

Novedades de la v1.4

- **Mandar mejor a tu ejército:**
- **Patrullar:** van y vuelven entre dos puntos, peleando en el camino.
- **Mantener posición:** se quedan quietas y solo disparan a lo que tienen a tiro.
- **Tropas veteranas:** 1, 2 y 3 estrellas con 3, 8 y 15 eliminaciones. Cada estrella da +10 % de vida y +8 % de daño. Los bots también las ganan.
- **Reparación sola:** los Constructores e Ingenieros libres reparan lo dañado a 4 casillas. Se puede apagar.
- **Producción continua:** el botón «Siempre» sigue entrenando la última tropa mientras alcance el dinero y la comunidad.
- **Más motivos para volver:**
- **12 logros** con insignia y cristales; se ven en el Inventario.
- **Top jugadores** (documento, pág. 20) con pestañas Liga 3 vs 3, Victorias e Islas tomadas. Tu fila siempre se ve. Los demás son comandantes bot.
- **Desafío semanal:** mapa, semilla y regla fijos toda la semana. Cambia el lunes y la primera victoria da 150 cristales.

- **Para tu iPad** (en la pausa):
- **Ahorro de batería:** unos 30 cuadros por segundo y menos partículas.
- **Colores para daltonismo:** elegidos con simulación de los 3 tipos de daltonismo, con una forma por jugador en el minimapa.
- **Letra grande.**
- **Arreglo:** si los soldados no pueden llegar a la orilla junto a la Barcaza (lejos de la costa o con la orilla ocupada por edificios), la Barcaza va a buscarlos. Antes esperaban para siempre.

Novedades de la v1.3

Los 29 puntos de la revisión «Bugs y mejoras»:

- **Bugs arreglados:**
- «Avanzar atacando» ya no se congela con enemigos inalcanzables.
- Abandonar la Liga cuenta como derrota.
- El misil nuclear no daña lo tuyo ni a tus aliados, y la Cúpula lo frena.
- Ninguna tropa de tierra queda sobre el agua.
- «Construye 12 edificios» cuenta obras terminadas.
- Demoler devuelve la mejora en curso.
- El primer objetivo, la fila «aliado» y los textos de la Casa y del Cuartel ya están bien.
- **Más fluido:**
- El guardado es unas 6-8 veces más chico, se hace cada minuto y al pausar o cambiar de app.
- Los bots piensan por turnos (nunca dos en el mismo paso).
- **Interfaz:**
- Panel de selección compacto: los botones principales arriba y «Más detalles» para el resto.
- En el celular el panel ya no tapa la barra de arriba, y en el iPad deja libre el centro.
- Todos los botones del juego miden 44 × 44 px como mínimo.
- **Como el documento de juego v2:**
- 6 rangos (Recluta, Cabo, Sargento, Capitán, Comandante y Almirante del Faro), solo en la Liga 3 vs 3 y desde el nivel 10.
- Recién llegado a un rango, la primera derrota no te baja.
- Temporadas de 2 meses: el pase vuelve a empezar y la Liga baja 1 rango.
- Tutorial de 3 pasos con Puerto y Barcaza para tomar la isla del bot.
- Muro a 30.
- El modo «Relámpago» ahora se llama «Choque exprés».
- **Copia de tu progreso:** un código o un archivo para llevar tu nivel, cristales, aspectos y rango a otro iPad.

Novedades de la v1.2

- **Rueda de selección:** mantén pulsado el mapa (o la tecla Tab) y elige Tierra, Mar, Aire, Todo, Constructores o un grupo; puedes soltar el dedo sobre la opción.

- **Mensajes:** ventana con pestañas Todos / Aliados y mensajes rápidos («¡Gracias!», «¡Bien hecho!», «¡Cuidado!», «¡Ayuda!», «¡Ataquemos!»); los bots aliados contestan. Se abre desde el marcador o tocando los mensajes de la esquina.
- **Tabla de la Clasificatoria:** tú y 20 comandantes bot (marcados como bots), ordenados por puntos.
- **Colecciones nuevas:** «Frente 1940» y «Marea» (soldados, vehículos, barcos y Núcleo).
- **Planos, fragmentos y Taller:** un cofre da un aspecto, un plano o fragmentos; los repetidos dan fragmentos; en el Taller, plano + fragmentos = aspecto.
- **Mapas nuevos:** «Mesetas de Cobre» (desde una meseta se dispara y se ve 1 casilla más lejos) y «Dunas Rojas» (desierto con oasis). Todos los mapas muestran su miniatura al elegir.
- **Tutorial que señala:** un aro y un texto marcan qué tocar en cada paso; si el lugar está fuera de la pantalla o tapado por un panel, señala el punto en el minimapa.
- **Temporadas:** «Calabazas» (15 oct-5 nov) e «Invierno» (10 dic-6 ene, con nieve), solas según la fecha o elegidas en la pausa.
- **Toques más fáciles:** tocar el cuerpo o el techo de un edificio también lo elige.

Novedades de la v1.1

- **Alianzas** en los modos de todos contra todos: desde el marcador, «Aliarse» con un rival (el bot acepta o no, y dice por qué) y «Romper». Los aliados no se atacan y comparten lo que ven; si solo quedan aliados, la alianza se rompe: gana uno solo.
- **Dúos:** 3 parejas (tú y un bot aliado), cada pareja en su zona del mapa.
- **Clasificatoria:** 3 vs 3 por puntos con 10 rangos (Bronce I a Ámbar); el panel «Rangos» del vestíbulo muestra tu rango, tus puntos y tus últimas partidas.
- **Órdenes a aliados:** «¡Ayuda!» y «¡Ataquemos!» en el marcador.
- **Escudo antimisiles:** además del misil nuclear, derriba misiles y cohetes enemigos a 3 casillas.
- **Música propia** generada por el juego (vestíbulo y batalla, más intensa en combate) con volumen en la pausa.
- **Espectador:** al perder, «Seguir mirando» hasta el final.
- **Resumen final** con gráficas de monedas por minuto y tropas de cada imperio.

Novedades de la v1.0

- **Construir:** vuelve el botón «Construir» (tocar una casilla solo muestra la obra) y el constructor tiene que llegar y quedarse pegado a la obra para que avance; mientras trabaja, martilla.
- **Base grande:** comunidad hasta 240 (8 Casas). Con el Núcleo en nivel 3: «Mejorar todo» (un clic) y «Mejoras automáticas» hasta dejar todo al máximo, guardando una reserva de monedas.
- **Misil nuclear:** deja 1 minuto de radiación que quema a los enemigos de dentro (con cuenta regresiva), y «Lanzar los 3» dispara los tres AIR DEFENSE de nivel 4 a la vez (un Escudo antimisiles solo derriba uno).

- **Difícil de verdad:** más monedas, piensa y ataca antes, y tácticas propias (busca Constructores y economía, retira heridos antes, defensa antiaérea al ver aviones, mejora y usa el misil antes).
- **Aviones:** Caza, Bombardero y Aeropuerto rehechos en Blender; los aviones dan vueltas en el aire, se inclinan y dejan estela; fichas con misiles, bombas y si están ocultos, orden «Bombardear zona» y torre de control en el Aeropuerto.
- **Portada:** más grande, con cazas, Bombardero, Apache y Portaaviones, logo, versión y novedades.

Novedades de la v0.9

- **Ejército nuevo:** Aeropuerto militar con Caza furtivo y Bombardero furtivo; Portaaviones, Fragata y Lancha de desembarco en el Puerto; Tanque ligero, Tanque pesado, Camión de misiles, Transporte blindado y Lanzacohetes en la Fábrica; Médico en el Cuartel. El Jeep y el Transporte llevan soldados.
- **Helicópteros:** el Helicóptero tiene ahora el equipo y el precio del Apache viejo (con artilleros en las puertas); el Apache cuesta 10 000 y es más fuerte (cañón, misiles y bengalas). Sin límite propio, vuelven solos a repararse al Helipuerto o al Aeropuerto. Se ve cuándo disparan y lanzan misiles.
- **Defensas:** 20 Torretas; la Torreta nivel 4 tiene 3 miniguns, más daño y tiro antiaéreo. La AIR DEFENSE fija el blanco con el radar, prioriza aeronaves y lanza ráfagas de 2 misiles, con ficha de radar. Nivel 4 (solo 3): misil nuclear carísimo con 3 minutos de recarga; el Escudo antimisiles lo derriba.
- **Reglas:** comunidad hasta 180 (y más por eliminar tropas y Núcleos), Casa nivel 2 = Apartamento (+20), 6 Helipuertos, cada edificio con su escudo, escudo de las tropas solo dentro de la Cúpula, barras de escudo en azul, amarillo y rojo, tarjeta «Qué mejoró» en cada mejora, tierra fértil más grande, construir de un toque y todas las tropas armadas dañan edificios.
- **Modelos y portada:** soldados más realistas (chaleco, mochila, brazos que sostienen el arma), Núcleo con banderas, Apartamento, aviones, barcos y vehículos nuevos hechos en Blender, y una portada con una escena 3D del juego.
- **Calidad:** «Más fluido» usa materiales más livianos y «Alta» se ve más nítida en pantallas Retina.
- **Suelo de la v0.6:** vuelven los colores lisos por terreno; se quitaron las texturas de la v0.8.

Novedades de la v0.8

- **Interfaz:** suelo con texturas escaneadas de Poly Haven, flechas en el borde cuando atacan algo tuyo fuera de la pantalla, barra de producción (con los edificios parados en amarillo), iconos en los botones y ficha del enemigo (vida, nivel y qué usar contra él).
- **Bots más listos:** defensa en equipo con una Torreta del lado del ataque, se forman en línea antes de atacar, fuego concentrado en el enemigo más débil, hostigamiento a Granjas y Minas, aliados que atacan al mismo rival y se ayudan, y oleadas con todo el ejército pasados 15 minutos.

- **Soldados con movimiento real:** caminar y correr de la base de datos de captura de movimiento de CMU, procesados en Blender y aplicados a piernas con rodilla.

Novedades de la v0.7

- **Campaña «La ruta del Faro»:** 6 misiones con historia, meta, progreso en pantalla y hasta 3 estrellas cada una (Primera luz, Defender la costa, El Faro, Cruzar el mar, Asedio y La gran batalla). Cada misión se abre al terminar la anterior; premio de cristales la primera vez y por cada estrella nueva.
- **Mejores controles:** grupos 1-2-3, «Avanzar atacando», formación en bloque o en línea (de frente al destino), «Repetir» y «×5» en los edificios que entrenan, y botón Constructores.

Novedades de la v0.6

- **Ritmo más rápido:** cada imperio empieza con 1 Constructor y los bots atacan antes (primera oleada hacia el minuto 4-5 en Normal).
- **Modo fluido para iPad:** gráficos «Automática» (baja la calidad sola si el iPad no llega a 40 cuadros por segundo), «Alta calidad» y «Más fluido», y un contador de cuadros por segundo.
- **15 modelos más con Blender:** Cuartel, Fábrica, Puerto, Casa, las cinco energías, Submarino, Apache Guardián, Camión antiaéreo, Torreta chica, Puesto avanzado y Enfermería 1-3.
- **Efectos y sonidos:** explosiones con fuego, humo y restos, fogonazos, estelas, chispas, salpicaduras, humo en edificios dañados y sonidos sintetizados nuevos.

Contenido de la v0.5

- **Constructores:** las obras quedan «En construcción» hasta que un Constructor, un Ingeniero (el doble de rápido) o un Barco constructor (obras de mar) las termina.
- **Niveles 1-3** en todos los edificios (Casa hasta 4). Núcleo con láser en todos los niveles y doble láser en el 3; la Cúpula se compra con el Núcleo en nivel 3 y, en su nivel 3, da escudo a tus tropas. Torreta con 1 minigun en nivel 2 y 2 en nivel 3. Enfermería: clínica, mini hospital y hospital grande.
- **Cinco energías:** Solar, Eólica, Hidráulica, Mareomotriz y Undimotriz, cada una en su lugar.
- **Lugares de recursos:** Granjas en tierra fértil, Minas en vetas de oro, Bombas en petróleo de tierra y Plataformas en petróleo del mar (sin aviso de territorio). Cada edificio tiene límite.
- **20 unidades** y **AIR DEFENSE**. 5 mapas: Archipiélago Ámbar, Mar abierto, Continente, Bahía Gris y Arrecife Doble.
- **Vestíbulo:** PLAY, CAMBIAR modo, Personalizado, Tutorial, Tus números, Pase de 40 niveles, misiones, premio diario, códigos, Tienda (apariencias y cofres) e Inventario.
- **Volver a la partida:** salir al menú la deja en pausa y se guarda sola cada 15 s.
- **Modo creador** (solo para la cuenta dueña del juego): monedas y energía infinitas.

- **Modelos 3D hechos con Blender** (`blender/`), incluido el skin Brigada Tóxica, y luz de ambiente con un cielo de Poly Haven (CC0, «Kloofendal 48d Partly Cloudy», Greg Zaal).

05 · CATÁLOGO

Catálogo actual (v1.5)

Sacado directamente de `src/js/00-config.js`, así que siempre coincide con el juego. Daño/s = daño por disparo ÷ tiempo entre disparos (sin bonificaciones contra cada tipo).

Tropas (31)

Tropa	Tipo	Coste	Vida	Daño/s	Alcance	Velocidad	Comunidad	Se entrena en
Constructor	Infantería	200	100	—	—	1.6	1	Núcleo, Cuartel
Ingeniero	Infantería	500	140	—	—	1.6	1	Cuartel
Artillero	Infantería	400	260	23	3	1.5	1	Cuartel
Escopetero	Infantería	600	320	44	2	1.5	1	Cuartel
Ranger	Infantería	700	360	40	3	1.6	1	Cuartel
Francotirador	Infantería	900	170	34	5	1.3	1	Cuartel
Rompedor	Infantería	950	380	60	2	1.5	1	Cuartel
Cazador	Infantería	1300	200	50	6	1.2	1	Cuartel
Cohetero	Infantería	1500	230	37	4	1.3	1	Cuartel
Jeep	Vehículo	1600	750	50	3	2.6	2	Fábrica
Tanque	Vehículo	3400	2200	64	4	1.2	3	Fábrica
Camión antiaéreo	Vehículo	2400	900	78	5	1.8	2	Fábrica
Lanzamisiles	Vehículo	3800	700	63	7	1	2	Fábrica
Tanque ligero	Vehículo	2200	1300	64	3	2	2	Fábrica
Tanque pesado	Vehículo	5200	3600	95	3	0.9	4	Fábrica

Tropa	Tipo	Coste	Vida	Daño/s	Alcance	Velocidad	Comunidad	Se entrena en
Camión de misiles	Vehículo	4200	800	52	6	1.4	2	Fábrica
Transporte blindado	Vehículo	2000	1600	47	3	2	2	Fábrica
Lanzador de misiles	Vehículo	7000	1200	64	9	0.8	4	Fábrica
Médico	Infantería	900	220	—	—	1.6	1	Cuartel
Lancha rápida	Barco	1400	650	49	3	2.8	1	Puerto
Barco constructor	Barco	1600	500	—	—	2.2	1	Puerto
Barcaza	Barco	1800	900	—	—	1.8	1	Puerto
Lancha de desembarco	Barco	2600	1600	—	—	2.2	1	Puerto
Fragata antiaérea	Barco	3600	2200	73	6	1.7	3	Puerto
Portaaviones	Barco	15000	9000	117	3	1	8	Puerto
Destructor	Barco	4500	2800	71	5	1.3	4	Puerto
Submarino	Submarino	3800	1400	80	3	1.6	3	Puerto
Helicóptero	Aéreo	7500	2400	107	4	2.3	4	Helipuerto
Apache Guardián	Aéreo	10000	3200	167	4	2.1	5	Helipuerto
Caza furtivo	Aéreo	6000	1400	117	5	3.6	3	Aeropuerto militar
Bombardero furtivo	Aéreo	9000	2600	108	1	2	5	Aeropuerto militar

Edificios (23)

Edificio	Coste	Vida	Obra (s)	Nivel máx.	Qué hace
Núcleo	0	6000	20	3	Tu base. Genera monedas, entrena Constructores, dispara un láser y cura a las tropas cercanas. Si cae, pierdes.
Granja	300	350	6	3	+240 monedas/min. Solo en tierra fértil (parcelas marcadas).
Bomba de petróleo	1400	700	10	3	+650 monedas/min. Solo en petróleo en tierra (manchas oscuras).
Mina de oro	1000	650	10	3	+600 monedas/min. Solo en vetas de oro.
Plataforma petrolera	2000	900	14	3	+1100 monedas/min. En el petróleo del mar. La construye el Barco constructor.
Casa	500	400	6	2	+10 de comunidad. Se mejora una vez a Apartamento: +20 más (+30 en total).
Solar	200	300	5	3	+3 de energía cada 10 s. En cualquier tierra.
Eólica	300	400	6	3	+5 de energía cada 10 s. En cualquier tierra.
Hidráulica	700	600	9	3	+8 de energía cada 10 s. En tierra junto al agua.
Mareomotriz	1000	700	11	3	+11 de energía cada 10 s. En agua poco profunda junto a la costa. La construye el Barco constructor.
Undimotriz	1500	800	13	3	+15 de energía cada 10 s. En mar profundo. La construye el Barco constructor.
Cuartel	1000	900	10	3	Entrena los 10 soldados: del Constructor al Cohetero, y el Médico.
Fábrica	3000	1500	16	3	Fabrica tanques, camiones antiaéreos, jeeps y lanzamisiles.
Puerto	2200	1300	14	3	En agua junto a la orilla. Construye los 5 barcos y repara los cercanos.
Helipuerto	4000	1000	18	3	Produce el Helicóptero y el Apache Guardián, y repara a los helicópteros cercanos. Hasta 6.
Torreta	1500	1100	10	4	Cañón contra tropas, barcos y helicópteros. Nivel 2: 1 minigun · nivel 3: 2 · nivel 4: 3 miniguns, más daño y antiaéreo. Hasta 20.
Torreta chica	400	500	5	3	Torreta pequeña y barata. Solo la puede construir un Ingeniero.
AIR DEFENSE	2000	1000	12	4	Misiles de largo alcance contra aeronaves. Nivel 4 (solo 3 de tus 6): lanza el misil nuclear.

Edificio	Coste	Vida	Obra (s)	Nivel máx.	Qué hace
Muro	30	1500	3	3	Muralla que bloquea a las tropas enemigas y se une con los muros vecinos. Las tuyas pasan.
Enfermería	2000	700	12	3	Cura infantería y vehículos cercanos. Nivel 1 clínica (3 %/s), nivel 2 mini hospital (5 %/s), nivel 3 hospital grande (8 %/s).
Aeropuerto militar	6000	1800	22	3	Pista y hangares: fabrica el Caza furtivo y el Bombardero furtivo, y repara aviones cercanos.
Escudo antimisiles	9000	1400	20	1	Derriba el misil nuclear enemigo si cae a 7 casillas o menos (una carga cada 2 minutos), y misiles y cohetes enemigos a 3 casillas (uno cada 1,5 s).
Puesto avanzado	800	800	8	1	Amplía tu territorio. Lo fundan los ingenieros y los barcos constructores.

Modos (13)

Modo	Jugadores	Reglas
Conquista de islas	6	6 imperios. Gana el último Núcleo en pie. Puedes aliarte desde el marcador.
Islas en la niebla	6	Solo ves lo que está cerca de tus tropas y edificios.
Turbo ×2	6	Monedas y tropas salen el doble de rápido.
Dúos	6	3 parejas: tú y un aliado bot aparecen juntos. Gana la última pareja en pie.
Liga 3 vs 3	6	Dos equipos de 3 con aliados bots. Da puntos de rango.
Liga 3 vs 3	6	Dos equipos de 3 con aliados bots. Da puntos de rango: ganar +25, perder −15.
Rey del Faro	4	Mantén el Faro 3 min en total.
Choque exprés	4	6.000 monedas y ritmo ×2.
Duelo 1 vs 1	2	Tú contra 1 rival
Todos contra todos	4	4 imperios, gana el último
Equipos 2 vs 2	4	Con Rojo como aliado
Gran batalla	6	6 imperios, mapa lleno
Tutorial	2	Aprende a jugar contra un bot lento en otra isla.

Mapas (7)

Mapa	Cómo es
Archipiélago Ámbar	Islas en círculo unidas por puentes de arena.
Mar abierto	Sin puentes: necesitas barcos o helicópteros.
Continente	Una gran isla con lagos.
Bahía Gris	Islas irregulares; solo algunas tienen puente.
Arrecife Doble	Dos grupos de islas separados por agua navegable.
Mesetas de Cobre	Un continente con mesetas altas: desde lo alto se dispara y se ve 1 casilla más lejos.
Dunas Rojas	Desierto de islas con puentes y oasis fértiles.

06 · CÓMO ESTÁ HECHO

Archivos, recursos y forma de construirlo

El juego es una sola página web (`faro/faro.html`, 2.50 MB) que `build.mjs` arma uniendo el código de `src/`, los modelos 3D, la animación de caminar y el cielo. Usa three.js r128 para el 3D. Funciona en Safari y Chrome, también en el iPad, sin instalar nada.

Archivos de código (31)

Archivo	Líneas	Qué contiene
`faro/src/index.template.html`	1,079	Estructura de la página y todos los estilos de la interfaz.
`faro/src/js/00-config.js`	572	Faro de Ámbar: datos del juego (edificios, unidades, terreno, jugadores).
`faro/src/js/01-util.js`	173	Utilidades: azar con semilla, matemática hexagonal, ruido y montículo binario.
`faro/src/js/02-map.js`	473	Generación del mapa hexagonal, pasabilidad y búsqueda de caminos (A*).
`faro/src/js/03-models.js`	873	Modelos 3D de bajo polígono hechos con formas simples y fusionados en una sola
`faro/src/js/04-world.js`	2,630	Estado y simulación de la partida: economía, construcción, producción,
`faro/src/js/045-save.js`	250	Guardar y cargar una partida (punto 16: si sales sin querer, puedes volver).
`faro/src/js/046-campaign.js`	291	Campaña «La ruta del Faro» (v0.7): prepara cada misión, sigue su meta y calcula las estrellas.
`faro/src/js/05-ai.js`	1,279	Bots (NPC): cada uno tiene una personalidad. Construyen economía, se expanden, planifican un
`faro/src/js/06-render.js`	1,829	Escena 3D: terreno instanciado, decoración, fronteras, entidades, efectos,
`faro/src/js/061-fx.js`	232	Efectos con partículas (v0.6): fuego, humo, restos, chispas, fogonazos, estelas y salpicaduras.
`faro/src/js/07-input.js`	616	Controles: toque (iPad/celular), ratón y teclado.
`faro/src/js/08-ui.js`	2,024	Interfaz HTML: menú, barra superior, panel de construcción, panel de selección,
`faro/src/js/085-meta.js`	1,761	Vestíbulo y progresión (documento págs. 14-27): jugar, modos, personalizado, tutorial, pase,
`faro/src/js/086-music.js`	190	Música propia (v1.1): melodías generadas por el juego con Web Audio, sin archivos.
`faro/src/js/09-main.js`	595	Arranque y bucle principal.

Archivo	Líneas	Qué contiene
`faro/build.mjs`	62	Une el código, los modelos y los recursos en un solo archivo: faro.html.
`faro/blender/cmu_gait.py`	125	Saca un ciclo de paso (caminar y correr) de la base de datos de captura de movimiento de CMU.
`faro/blender/gen_core15.py`	17	Exporta solo el Núcleo nuevo de la v1.5 (core_1, core_2 y core_3).
`faro/blender/generate_models.py`	155	Genera los modelos 3D de Faro de Ámbar con Blender y los exporta como GLB.
`faro/blender/lib.py`	215	Herramientas para modelar Faro de Ámbar con Python dentro de Blender.
`faro/blender/models_ref.py`	781	Modelos rehechos según las referencias del 7 de noviembre (diseño propio, estilo del juego).
`faro/blender/models_ref2.py`	436	Modelos de la v0.6 (diseño propio, estilo de las referencias del 7 de noviembre).
`faro/blender/models_ref3.py`	428	Modelos de la v0.9 de Faro de Ámbar: vehículos y barcos nuevos, aviones, Aeropuerto, defensas y
`faro/blender/models_v15.py`	110	Modelos de la v1.5: Núcleo nuevo (ciudadela con torre central que crece en cada nivel).
`faro/docs/pdf.cjs`	43	Convierte un documento HTML en PDF A4 con encabezado y número de página.
`faro/docs/doc_style.py`	57	Estilo común de los documentos.
`faro/docs/roblox/build_prd_roblox.py`	95	Arma el PDF del PRD Roblox v1.6 con sus imágenes.
`faro/docs/documento/build_documento.py`	276	Arma este documento.
`faro/README.md`	256	Presentación del prototipo, controles, novedades de cada versión y créditos.
`faro/AGENTS.md`	45	Reglas de trabajo del proyecto (PRD y PDF en cada versión, documento completo).

Recursos

- **75 modelos 3D** (GLB, 1398 KB) generados con Blender por los scripts de `faro/blender/`: aa, aadef, aadef_4, airport, antinuke, apache, apc, arty, barge, barracks, boatEng, bomber, breaker, breaker@toxico, builder, builder@toxico, carrier, core_1, core_2, core_3, destroyer, engineer, engineer@toxico, factory, farm, fighter, frigate, harbor, heavytank, heli, helipad, hospital_1, hospital_2, hospital_3, house, house_2, hunter, hunter@toxico, hydro, jeep, lander, launcher, lighttank, medic, medic@toxico, mine, miniturret, missiletruck, outpost, pump, ranger, ranger@toxico, rifle, rifle@toxico, rig, rocket, rocket@toxico, scaffold, shotgun, shotgun@toxico, sniper, sniper@toxico, solar,

speedboat, sub, tank, tidal, turret_1, turret_2, turret_3, turret_4, wall_post, wall_seg, wave, wind.

- **Animación de caminar y correr** (`assets/anim/cmu_gait.json`) sacada de la base de captura de movimiento de CMU.
- **Cielo de ambiente** (`assets/env/sky_env.jpg`) de Poly Haven, licencia CC0.

Cómo se construye

```
node faro/build.mjs                                          # genera faro/faro.html
python3 faro/blender/generate_models.py faro/assets/models  # modelos 3D (módulo bpy de Blender)
node faro/docs/pdf.cjs faro/docs/prd/PRD_web_vX.Y.html       # PDF de un PRD
python3 faro/docs/documento/build_documento.py               # este documento (y luego pdf.cjs)
```

07 · PRUEBAS

Cómo se prueba

Cada versión se prueba con partidas simuladas (la lógica del juego corre en Node, sin gráficos) y en Chromium con tamaño de iPad y de celular, tocando la pantalla como lo haría una persona. Las pruebas de versiones anteriores se vuelven a correr en cada versión. Chromium dibuja sin tarjeta gráfica: sus cuadros por segundo solo sirven para comparar versiones, no para saber cómo va en un iPad.

En los PRD hay **190 pruebas aprobadas** en total. Los 71 archivos de prueba (11,083 líneas) están en `faro/tests/`; por su tamaño, aquí se listan y su código queda en esa carpeta.

Archivo	Líneas	Qué prueba
`README.md`	26	Pruebas de Faro de Ámbar
`camp-auto.cjs`	33	D04: las 6 misiones se pueden ganar con órdenes automáticas (un piloto bot juega por ti).
`camp-m3.cjs`	38	Misión 3 con órdenes automáticas como las de un jugador: tropas al Faro, Cuartel, Granjas y refuerzos.
`camp-m4.cjs`	57	Misión 4 con órdenes automáticas como las de un jugador: subir a la Barcaza, cruzar, desembarcar y capturar.
`camp-save.cjs`	27	
`camp-setup.cjs`	12	D02: cada misión arranca con su mapa, rivales y preparación.
`duel-diff.cjs`	26	Punto 5 (v1.0): piloto automático con nivel de jugador medio (tácticas y ritmo de Normal, ingresos de jugador)
`fill-prd.py`	80	Completa la tabla de pruebas del PRD v0.5 con los resultados reales (JSON de las pruebas).
`fill-prd10.py`	146	Completa el PRD v1.0 con los resultados reales.
`fill-prd11.py`	133	Completa el PRD v1.1 con los resultados reales.
`fill-prd12.py`	148	Completa el PRD v1.2 con los resultados reales.
`fill-prd13.py`	204	Completa el PRD v1.3 con los resultados reales y la tabla de contenido y precios.
`fill-prd14.py`	208	Completa el PRD v1.4 con los resultados reales.
`fill-prd15.py`	188	Completa el PRD v1.5 con los resultados reales.
`fill-prd6.py`	41	Completa la tabla de pruebas del PRD v0.6 con los resultados reales.
`fill-prd7.py`	86	Completa el PRD v0.7 con los resultados reales y los ajustes hechos durante la versión.
`fill-prd8.py`	73	Completa el PRD v0.8 con los resultados reales y lo que cambió durante la versión.

Archivo	Líneas	Qué prueba
`fill-prd9.py`	174	Completa el PRD v0.9 con los resultados reales y lo que cambió durante la versión.
`helpers.cjs`	30	Ayudas para correr las pruebas de v0.1-v0.4 en la v0.5 (vestíbulo nuevo y sin tropas iniciales).
`sim.cjs`	43	Ejecuta la lógica del juego (sin gráficos) en Node para probar a los bots.
`soak15.cjs`	94	L08: partidas largas con chequeos automáticos cada segundo de juego.
`test-fight.cjs`	51	
`test-hexbox.cjs`	53	
`test-input.cjs`	165	
`test-layout.cjs`	31	
`test-new.cjs`	102	
`test-portrait.cjs`	19	
`test-scene.cjs`	54	
`test-v03.cjs`	477	
`test-v04-bots.cjs`	253	Pruebas de los bots del PRD v0.4 (A01, A03–A09) en Node, con azar repetible.
`test-v04-ui.cjs`	236	
`test-v05-maps-bots.cjs`	90	B23 (mapas) y B24 (bots en los 5 mapas) del PRD v0.5.
`test-v05-meta.cjs`	187	
`test-v05-ref.cjs`	84	B27-B30: modelos según referencias, skin Brigada Tóxica, luz de Poly Haven y piernas al caminar.
`test-v05-rules.cjs`	357	Pruebas de reglas del PRD v0.5 (B01-B12, B22) con la lógica del juego en Node.
`test-v05-sizes.cjs`	54	B25: vestíbulo, tienda, ventanas y pantalla final sin desbordes en 5 tamaños.
`test-v05-visual.cjs`	81	Pruebas visuales del PRD v0.5 (B13-B15) en Chromium.
`test-v06-pace.cjs`	45	C01-C03 del PRD v0.6: Constructor inicial y ritmo de ataque de los bots.
`test-v06.cjs`	202	C05-C08 y C10-C14 del PRD v0.6 en Chromium.
`test-v07-sizes.cjs`	141	D13: nada tapado ni fuera de pantalla en 5 tamaños con los botones nuevos y la pantalla de misión.

Archivo	Líneas	Qué prueba
`test-v07.cjs`	345	D01-D03 y D05-D11 del PRD v0.7 en Chromium (iPad horizontal, toques simulados).
`test-v08-bots-b.cjs`	190	E06-E10 del PRD v0.8: comportamiento de los bots en partidas simuladas.
`test-v08-bots.cjs`	190	E06-E10 del PRD v0.8: comportamiento de los bots en partidas simuladas.
`test-v08.cjs`	176	E01-E05 y E12 del PRD v0.8 en Chromium (iPad horizontal).
`test-v08b.cjs`	158	E01-E05 y E12 del PRD v0.8 en Chromium (iPad horizontal).
`test-v09-b.cjs`	216	Pruebas de la v0.9 en Chromium (iPad horizontal): F01, F04 (colores), F06 (tarjeta), F07, F14 (iconos),
`test-v09-sim-b.cjs`	353	Pruebas de lógica de la v0.9 (F02-F05, F06, F08-F14, F17) con el juego sin gráficos.
`test-v09-sim.cjs`	353	Pruebas de lógica de la v0.9 (F02-F05, F06, F08-F14, F17) con el juego sin gráficos.
`test-v09-sizes.cjs`	142	F19: nada tapado ni fuera de pantalla en 5 tamaños con las pantallas nuevas de la v0.9.
`test-v09.cjs`	213	Pruebas de la v0.9 en Chromium (iPad horizontal): F01, F04 (colores), F06 (tarjeta), F07, F14 (iconos),
`test-v10-sim.cjs`	231	Pruebas de lógica de la v1.0 (G02, G03, G04, G05 tácticas, G07, G08) con el juego sin gráficos.
`test-v10-sizes.cjs`	166	G09/G10: nada tapado ni fuera de pantalla en 5 tamaños con las pantallas nuevas de la v1.0.
`test-v10.cjs`	207	Pruebas de la v1.0 en Chromium (iPad horizontal): G01, G03 (vista), G04 (botones), G06, G08 (botón), G09 (portada).
`test-v11-sim.cjs`	161	Pruebas de lógica de la v1.1 (H01, H02, H04, H06, H07, H08) con el juego sin gráficos.
`test-v11-sizes.cjs`	208	H09: nada tapado ni fuera de pantalla en 5 tamaños con las pantallas nuevas de la v1.1.
`test-v11.cjs`	155	Pruebas de la v1.1 en Chromium (iPad horizontal): H01, H03, H05, H06, H07, H08 en pantalla.
`test-v11.orig.cjs`	155	Pruebas de la v1.1 en Chromium (iPad horizontal): H01, H03, H05, H06, H07, H08 en pantalla.
`test-v12-sim.cjs`	80	Pruebas de lógica de la v1.2 (I06 mapas y ventaja de altura, I09 temporadas por fecha).
`test-v12-sizes.cjs`	317	I10: nada tapado ni fuera de pantalla en 5 tamaños, con las pantallas de la v1.1 y las nuevas de la v1.2.

Archivo	Líneas	Qué prueba
`test-v12.cjs`	488	Pruebas de la v1.2 en Chromium (iPad horizontal, toques simulados): I01-I05, I07-I09.
`test-v13-sim.cjs`	212	Pruebas de lógica de la v1.3 (bugs de tu revisión). Con FARO_JS=<código v1.2> se reproduce cada bug.
`test-v13.cjs`	366	Pruebas de la v1.3 en Chromium: J02, J06 (fila «aliado» y ayuda), J07 (guardado), J08 (interfaz), J09, J10, J11.
`test-v14-board.cjs`	60	K13 (arreglo hallado en las pruebas): tropas que suben a una Barcaza lejos de su orilla.
`test-v14-cvd.cjs`	63	K10 (parte de colores): ¿se distinguen los 6 colores de jugador con daltonismo?
`test-v14-sim.cjs`	191	Pruebas de lógica de la v1.4: K01-K05, reglas del desafío (K08) y estrellas de los bots (K11).
`test-v14.cjs`	282	Pruebas de la v1.4 en Chromium (iPad, toques simulados): K01-K10 en pantalla.
`test-v15-monkey-v1.cjs`	122	L07: toques al azar en iPad y celular. Busca errores de consola, congelamientos y textos «undefined»/«NaN».
`test-v15-monkey.cjs`	142	L07: toques al azar en iPad y celular. Busca errores de consola, congelamientos y textos «undefined»/«NaN».
`test-v15-sim.cjs`	162	Pruebas de lógica de la v1.5: L09 (errores encontrados), reglas sin cambios (L03/L11) y alturas del Núcleo (L03).
`test-v15.cjs`	381	Pruebas de la v1.5 en Chromium: L01 (sin temporadas), L02 (portada), L03 (Núcleo), L04 (edificios), L05 (ficha),
`test-visual03.cjs`	49	

08 · ROBLOX

Plan para Roblox (PRD v1.6 del usuario)

El usuario escribió el **PRD y plan de inicio v1.6** para llevar Faro de Ámbar a **Roblox Studio + Luau**. Está completo en `faro/docs/roblox/` (Markdown original y PDF con sus imágenes). El prototipo web sigue siendo el banco de pruebas de reglas y controles; el desarrollo en Roblox empieza cuando el usuario lo indique.

Etapa	Herramientas	Para qué
1. Empezar	Roblox Studio + Luau + Rojo + Git	Base del juego, mecánicas, código organizado y versiones.
2. Intermedio	Blender + Animation Editor + SurfaceAppearance	Personajes, vehículos, edificios, animaciones y materiales.
3. Completar	Roblox Studio + Luau + DataStoreService	Menús, inventario, misiones, controles y guardado del progreso.
4. Pulir	MicroProfiler + Developer Console + simuladores	Errores, rendimiento, multijugador y móvil.
5. Publicar	Roblox Studio + Creator Hub + Analytics	Publicar, revisar y mejorar con los resultados.

> Lo que el prototipo web ya probó (reglas, controles en iPad, bots, campaña, Barcaza, Cúpula, progreso) sirve de referencia para las pruebas CA-01 a CA-16 del plan de Roblox. Los modelos GLB de `faro/assets/models/` se pueden importar en Roblox Studio (3D Importer).

ANEXO

El código completo, archivo por archivo

31 archivos, 17,968 líneas, en el orden en que se arma el juego: la página, la lógica y el dibujo (`src/js/`, que se ejecutan en orden de nombre), el armado, los modelos de Blender, las herramientas de documentos y las reglas del proyecto. El archivo final `faro.html` no se imprime porque es este mismo código unido a los modelos.

- `faro/src/index.template.html` · 1,079 líneas
- `faro/src/js/00-config.js` · 572 líneas
- `faro/src/js/01-util.js` · 173 líneas
- `faro/src/js/02-map.js` · 473 líneas
- `faro/src/js/03-models.js` · 873 líneas
- `faro/src/js/04-world.js` · 2,630 líneas
- `faro/src/js/045-save.js` · 250 líneas
- `faro/src/js/046-campaign.js` · 291 líneas
- `faro/src/js/05-ai.js` · 1,279 líneas
- `faro/src/js/06-render.js` · 1,829 líneas
- `faro/src/js/061-fx.js` · 232 líneas
- `faro/src/js/07-input.js` · 616 líneas
- `faro/src/js/08-ui.js` · 2,024 líneas
- `faro/src/js/085-meta.js` · 1,761 líneas
- `faro/src/js/086-music.js` · 190 líneas
- `faro/src/js/09-main.js` · 595 líneas
- `faro/build.mjs` · 62 líneas
- `faro/blender/cmu_gait.py` · 125 líneas
- `faro/blender/gen_core15.py` · 17 líneas
- `faro/blender/generate_models.py` · 155 líneas
- `faro/blender/lib.py` · 215 líneas
- `faro/blender/models_ref.py` · 781 líneas
- `faro/blender/models_ref2.py` · 436 líneas
- `faro/blender/models_ref3.py` · 428 líneas
- `faro/blender/models_v15.py` · 110 líneas
- `faro/docs/pdf.cjs` · 43 líneas
- `faro/docs/doc_style.py` · 57 líneas
- `faro/docs/roblox/build_prd_roblox.py` · 95 líneas
- `faro/docs/documento/build_documento.py` · 276 líneas
- `faro/README.md` · 256 líneas
- `faro/AGENTS.md` · 45 líneas

faro/src/index.template.html · 1,079 líneas

```
<title>Faro de Ámbar</title>
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link rel="stylesheet" href="https://fonts.googleapis.com/css2?family=Barlow:wght@400;500;600&family=Chakra+Petch:wght@500;600;700&display=swap">
<style>
/* Concepto: interfaz de mesa de mando sobre el mapa. Paneles de pizarra translúcida en los
   bordes, el mapa siempre libre en el centro; oro para monedas y acciones principales. */
:root {
  --bg: #0c141b;
  --panel: rgba(12, 21, 29, 0.84);
  --panel-solid: #111c25;
  --raise: #1a2834;
  --raise-2: #233544;
  --line: rgba(205, 226, 240, 0.13);
  --text: #edf3f7;
  --muted: #9aaebd;
  --gold: #f2ab38;
  --gold-ink: #2a1902;
  --energy: #7fd4ff;
  --sea: #3aa6c2;
  --good: #5ee07a;
  --bad: #f0585a;
  --font-display: 'Chakra Petch', 'Rajdhani', 'Segoe UI', system-ui, sans-serif;
  --font-body: 'Barlow', 'Segoe UI', system-ui, -apple-system, sans-serif;
  --r: 10px;
  --gap: 10px;
  --top: calc(10px + env(safe-area-inset-top, 0px));
  --left: calc(10px + env(safe-area-inset-left, 0px));
  --right: calc(10px + env(safe-area-inset-right, 0px));
  --bottom: calc(10px + env(safe-area-inset-bottom, 0px));
  color-scheme: dark;
}
* { box-sizing: border-box; }
[hidden] { display: none !important; }
html, body { height: 100%; }
body {
  margin: 0; background: var(--bg); color: var(--text); font-family: var(--font-body);
  font-size: 15px; overflow: hidden; overscroll-behavior: none; -webkit-user-select: none; user-select: none;
  -webkit-tap-highlight-color: transparent; touch-action: none;
}
button { font: inherit; color: inherit; }
button:focus-visible, input:focus-visible + span, .opt input:focus-visible + span { outline: 2px solid var(--gold); outline-offset: 2px; }
#gl, #overlay { position: fixed; top: 0; right: 0; bottom: 0; left: 0; inset: 0; width: 100%; height: 100%; display: block; }
#gl { touch-action: none; }
#overlay { pointer-events: none; }
.num { font-family: var(--font-display); font-variant-numeric: tabular-nums; }
.panel {
  background: var(--panel); border: 1px solid var(--line); border-radius: var(--r);
  -webkit-backdrop-filter: blur(10px); backdrop-filter: blur(10px);
  box-shadow: 0 8px 24px rgba(0, 0, 0, 0.28);
}
.btn {
  display: inline-flex; align-items: center; justify-content: center; gap: 6px; min-height: 40px;
  padding: 0 14px; border-radius: 8px; border: 1px solid var(--line); background: var(--raise);
  cursor: pointer; font-weight: 600; font-size: 14px; white-space: nowrap;
}
.btn:hover { background: var(--raise-2); }
.btn:active { transform: translateY(1px); }
.btn.gold { background: var(--gold); color: var(--gold-ink); border-color: transparent; }
.btn.gold:hover { filter: brightness(1.06); }
.btn.danger { background: rgba(240, 88, 90, 0.14); color: #ffb4b5; border-color: rgba(240, 88, 90, 0.35); }
.btn.icon { width: 42px; padding: 0; }
.btn svg, .qbtn svg { width: 20px; height: 20px; flex: none; }
.lbl { font-size: 11px; letter-spacing: 0.08em; text-transform: uppercase; color: var(--muted); }

/* ---------- barra superior ---------- */
.topbar {
  position: fixed; top: var(--top); left: var(--left); right: var(--right); display: flex;
  align-items: flex-start; gap: var(--gap); pointer-events: none; z-index: 5;
}
.topbar > * { pointer-events: auto; }
.res { display: flex; align-items: stretch; padding: 6px 4px; }
.stat { display: grid; grid-template-columns: auto auto; column-gap: 8px; align-items: center; padding: 0 12px; }
.stat + .stat { border-left: 1px solid var(--line); }
.stat svg { grid-row: span 2; width: 22px; height: 22px; color: var(--muted); }
.stat.coins svg { color: var(--gold); }
.stat.energy svg { color: var(--energy); }
.stat.energy small { color: var(--energy); }
#king { position: fixed; top: calc(var(--top) + 64px); left: 50%; transform: translateX(-50%); z-index: 4; display: flex; gap: 8px; align-items: center; padding: 6px 10px; font-size: 12.5px; font-weight: 600; white-space: nowrap; }
#king .bar { width: 90px; height: 6px; border-radius: 3px; background: rgba(255,255,255,0.12); overflow: hidden; }
```

```
#king .bar i { display: block; height: 100%; }
.stat b { font-family: var(--font-display); font-size: 21px; line-height: 1.1; font-weight: 600; font-variant-numeric: tabular-nums; }
.stat .lbl { line-height: 1.2; }
.stat small { font-family: var(--font-display); color: var(--good); font-size: 12px; margin-left: 4px; font-weight: 500; }
.stat.full b { color: var(--bad); }
.players { display: flex; flex-wrap: wrap; gap: 6px; padding: 6px; margin-inline: auto; justify-content: center; }
.pchip {
  display: flex; align-items: center; gap: 6px; padding: 4px 9px 4px 5px; border-radius: 7px;
  background: rgba(255, 255, 255, 0.04); font-size: 13px; font-weight: 600; white-space: nowrap;
}
.pchip i { width: 14px; height: 14px; border-radius: 4px; display: block; flex: none; }
.pchip em { font-style: normal; font-size: 11px; color: var(--muted); font-weight: 500; }
.pchip.dead { opacity: 0.45; text-decoration: line-through; }
.sys { display: flex; gap: 6px; padding: 5px; }

/* ---------- objetivos y avisos ---------- */
#leftcol { position: fixed; top: calc(var(--top) + 64px); left: var(--left); width: 236px; bottom: calc(var(--bottom) + 284px); display: flex; flex-direction: column; gap: 8px; z-index: 4; pointer-events: none; }
#leftcol > * { pointer-events: auto; }
#objectives { flex: none; padding: 10px 12px; }
#feed { flex: 1 1 auto; min-height: 0; overflow: hidden; display: flex; flex-direction: column; justify-content: flex-end; gap: 5px; pointer-events: none; }
.msg { pointer-events: auto; display: grid; grid-template-columns: auto 1fr; gap: 2px 8px; align-items: start; padding: 7px 10px; border-radius: 8px; background: var(--panel); border: 1px solid var(--line); font-size: 12.5px; line-height: 1.35; animation: tin 0.18s ease-out; transition: opacity 0.6s; }
.msg i { width: 10px; height: 10px; border-radius: 3px; margin-top: 3px; display: block; }
.msg b { font-weight: 600; }
.msg.bad { border-color: rgba(240, 88, 90, 0.5); }
.msg.old { opacity: 0.45; }
.msg button { grid-column: 2; justify-self: start; }
.mini-btn { background: none; border: 1px solid var(--line); color: var(--gold); border-radius: 6px; padding: 2px 8px; font-size: 12px; font-weight: 600; cursor: pointer; min-height: 26px; }
.stat[role="button"] { cursor: pointer; border-radius: 6px; }
.stat[role="button"]:hover { background: rgba(255, 255, 255, 0.05); }
#players { cursor: pointer; }
.pop { position: fixed; z-index: 11; padding: 12px 14px; display: grid; gap: 8px; }
.pop header { display: flex; justify-content: space-between; align-items: center; gap: 12px; }
.pop h4 { margin: 0; font-family: var(--font-display); font-size: 14px; letter-spacing: 0.08em; text-transform: uppercase; }
.pop table { border-collapse: collapse; font-size: 13px; width: 100%; }
.pop td, .pop th { padding: 4px 6px; text-align: left; white-space: nowrap; }
.pop th { color: var(--muted); font-weight: 600; font-size: 11px; letter-spacing: 0.06em; text-transform: uppercase; border-bottom: 1px solid var(--line); }
.pop td.n { text-align: right; font-family: var(--font-display); font-variant-numeric: tabular-nums; }
.pop tr.total td { border-top: 1px solid var(--line); font-weight: 600; }
#econ { top: calc(var(--top) + 64px); left: var(--left); width: min(300px, calc(100% - 20px)); }
#score { top: calc(var(--top) + 64px); left: 50%; transform: translateX(-50%); width: min(640px, calc(100% - 20px)); max-height: calc(100% - var(--top) - 76px); overflow-y: auto; }
#econ { max-height: calc(100% - var(--top) - 76px); overflow-y: auto; }
#score .wrap { overflow-x: auto; }
#score i.dot { display: inline-block; width: 10px; height: 10px; border-radius: 3px; margin-right: 6px; vertical-align: middle; }
#score tr.dead td { opacity: 0.45; }
.cardinfo { display: grid; grid-template-columns: repeat(auto-fit, minmax(110px, 1fr)); gap: 4px 12px; padding: 8px 10px; border-radius: 8px; background: rgba(255, 255, 255, 0.04); font-size: 12.5px; }
.cardinfo .title { grid-column: 1 / -1; font-weight: 600; }
.cardinfo span { color: var(--muted); }
.cardinfo b { font-family: var(--font-display); font-weight: 600; color: var(--text); }
.cardinfo .wide { grid-column: 1 / -1; color: var(--muted); }
.cardinfo .wide b { font-family: var(--font-body); }
.hint-press { font-size: 11.5px; color: var(--muted); }
#objectives header { display: flex; align-items: center; justify-content: space-between; gap: 8px; }
#objectives button { background: none; border: 0; color: var(--muted); cursor: pointer; font-size: 12px; padding: 4px; }
#objectives ol { margin: 8px 0 0; padding: 0; list-style: none; display: grid; gap: 5px; font-size: 13px; }
#objectives li { display: flex; gap: 8px; align-items: flex-start; color: var(--text); line-height: 1.3; }
#objectives li::before { content: ''; flex: none; width: 13px; height: 13px; margin-top: 2px; border-radius: 4px; border: 1.5px solid var(--muted); }
#objectives li.done { color: var(--muted); text-decoration: line-through; }
#objectives li.done::before { background: var(--good); border-color: var(--good); }
#objectives.collapsed ol { display: none; }
#toasts { position: fixed; top: calc(var(--top) + 64px); left: 50%; transform: translateX(-50%); display: grid; gap: 6px; justify-items: center; z-index: 9; pointer-events: none; width: min(440px, calc(100% - 32px)); }
.toast { padding: 8px 14px; border-radius: 8px; background: var(--panel-solid); border: 1px solid var(--line); font-weight: 600; font-size: 14px; animation: tin 0.18s ease-out; text-align: center; display: flex; gap: 10px; align-items: center; justify-content: center; }
.toast.has-btn { pointer-events: auto; }
.toast.bad { border-color: rgba(240, 88, 90, 0.55); color: #ffc2c3; }
.toast.good { border-color: rgba(94, 224, 122, 0.5); color: #c6f7d2; }
@keyframes tin { from { opacity: 0; transform: translateY(-6px); } }

/* ---------- panel de construcción ---------- */
#buildpanel { position: fixed; top: calc(var(--top) + 64px); right: var(--right); width: 262px; max-height: calc(100% - var(--top) - var(--bottom) - 64px); display: flex; flex-direction: column; z-index: 4; overflow: hidden; }
#buildpanel > header { display: flex; align-items: center; justify-content: space-between; padding: 10px 12px 6px; }
#buildpanel h2 { margin: 0; font-family: var(--font-display); font-size: 15px; letter-spacing: 0.12em; text-transform:
```

```
uppercase; }
 151 #btn-build-close { width: 34px; min-height: 34px; }
 152 #btn-build-close svg { transition: transform 0.15s; }
 153 #buildpanel.min .tabs, #buildpanel.min .blist { display: none; }
 154 #buildpanel.min #btn-build-close svg { transform: rotate(180deg); }
 155 .tabs { display: grid; grid-template-columns: repeat(2, 1fr); gap: 4px; padding: 0 8px 8px; }
 156 .tab { font-size: 12px !important; }
 157 .lvtag { font-family: var(--font-display); font-size: 12px; color: var(--gold); border: 1px solid rgba(242,171,56,0.4);
border-radius: 5px; padding: 0 5px; margin-left: 6px; vertical-align: middle; }
 158 .tab { min-height: 34px; border-radius: 7px; border: 1px solid transparent; background: transparent; color: var(--
muted); font-weight: 600; font-size: 13px; cursor: pointer; }
 159 .tab[aria-selected="true"] { background: var(--raise-2); color: var(--text); border-color: var(--line); }
 160 .blist { overflow-y: auto; padding: 0 8px 8px; display: grid; gap: 6px; -webkit-overflow-scrolling: touch; touch-
action: pan-y; }
 161 .bitem { display: grid; grid-template-columns: 52px 1fr; gap: 2px 10px; align-items: center; padding: 6px 8px 6px 6px;
border-radius: 8px; background: var(--raise); border: 1px solid transparent; text-align: left; cursor: pointer; }
 162 .bitem:hover { border-color: var(--line); }
 163 .bitem img, .thumb { width: 52px; height: 52px; border-radius: 7px; background: rgba(255, 255, 255, 0.05); grid-row:
span 2; display: block; }
 164 .bitem .top { display: flex; justify-content: space-between; gap: 6px; align-items: baseline; }
 165 .bitem b { font-size: 14px; }
 166 .cost { font-family: var(--font-display); font-weight: 600; color: var(--gold); font-size: 14px; font-variant-numeric:
tabular-nums; white-space: nowrap; }
 167 .bitem .desc { font-size: 12px; color: var(--muted); line-height: 1.3; }
 168 .bitem.poor .cost { color: var(--bad); }
 169 .bitem.maxed { opacity: 0.45; }
 170 .bitem.active { border-color: var(--gold); }
 171
 172 /* ---------- dock izquierdo ---------- */
 173 #leftdock { position: fixed; left: var(--left); bottom: var(--bottom); display: flex; gap: 8px; align-items: flex-end;
z-index: 4; pointer-events: none; }
 174 #leftdock > * { pointer-events: auto; }
 175 #minimap { width: 172px; height: 172px; border-radius: var(--r); border: 1px solid var(--line); display: block;
background: #16405a; cursor: pointer; touch-action: none; }
 176 .quick { display: grid; gap: 6px; }
 177 .qbtn { width: 50px; min-height: 50px; display: grid; justify-items: center; align-content: center; gap: 2px; border-
radius: 9px; border: 1px solid var(--line); background: var(--panel); cursor: pointer; font-size: 10.5px; font-weight: 600;
color: var(--muted); padding: 4px 0; }
 178 .qbtn.on { background: var(--gold); color: var(--gold-ink); border-color: transparent; }
 179 .qbtn:hover { color: var(--text); }
 180
 181 /* ---------- selección ---------- */
 182 #selpanel, #placebar { position: fixed; bottom: var(--bottom); left: calc(var(--left) + 248px); right: calc(var(--
right) + 274px); max-width: 640px; margin-inline: auto; z-index: 6; }
 183 #selpanel { padding: 10px; display: grid; gap: 8px; }
 184 .selhead { display: grid; grid-template-columns: auto 1fr auto; gap: 10px; align-items: center; }
 185 .selhead h3 { margin: 0; font-family: var(--font-display); font-size: 17px; font-weight: 600; }
 186 .selhead p { margin: 2px 0 0; font-size: 12.5px; color: var(--muted); line-height: 1.35; }
 187 .hp { height: 7px; border-radius: 4px; background: rgba(255, 255, 255, 0.09); overflow: hidden; margin-top: 6px; }
 188 .hp i { display: block; height: 100%; background: var(--good); border-radius: 4px; }
 189 .hp.build i { background: var(--gold); }
 190 .hptext { font-family: var(--font-display); font-size: 12px; color: var(--muted); margin-top: 3px; font-variant-
numeric: tabular-nums; }
 191 .cards { display: flex; gap: 6px; overflow-x: auto; padding-bottom: 2px; touch-action: pan-x; }
 192 .card { flex: 0 0 auto; width: 92px; display: grid; justify-items: center; gap: 1px; padding: 6px 4px; border-radius:
8px; background: var(--raise); border: 1px solid transparent; cursor: pointer; text-align: center; }
 193 .card:hover { border-color: var(--line); }
 194 .card img { width: 54px; height: 54px; }
 195 .card b { font-size: 12.5px; line-height: 1.15; }
 196 .card .meta { display: flex; gap: 6px; font-size: 12px; align-items: center; }
 197 .card .meta span { color: var(--muted); font-family: var(--font-display); }
 198 .card.poor .cost { color: var(--bad); }
 199 .queue { display: flex; gap: 6px; align-items: center; flex-wrap: wrap; font-size: 12px; color: var(--muted); }
 200 .qitem { position: relative; width: 40px; height: 40px; border-radius: 7px; background: var(--raise); border: 1px solid
var(--line); cursor: pointer; overflow: hidden; padding: 0; }
 201 .qitem img { width: 100%; height: 100%; display: block; }
 202 .qitem i { position: absolute; left: 0; bottom: 0; height: 3px; background: var(--gold); }
 203 .actions { display: flex; gap: 6px; flex-wrap: wrap; }
 204 .chips { display: flex; gap: 6px; flex-wrap: wrap; }
 205 .chip { display: flex; align-items: center; gap: 4px; padding: 2px 8px 2px 2px; border-radius: 7px; background: var(--
raise); font-size: 13px; font-weight: 600; }
 206 .chip img { width: 30px; height: 30px; }
 207 #placebar { padding: 8px 8px 8px 14px; display: flex; align-items: center; gap: 10px; justify-content: space-between;
flex-wrap: wrap; }
 208 #placebar p { flex: 1 1 240px; min-width: 0; }
 209 #placebar .err { color: #ffb4b5; }
 210 .btn:disabled { opacity: 0.45; cursor: not-allowed; }
 211 .ecost { font-family: var(--font-display); font-weight: 600; color: var(--energy); font-size: 13px; white-space:
nowrap; }
 212 .costs { display: flex; gap: 6px; align-items: baseline; }
 213 .bitem.poorE .ecost { color: var(--bad); }
 214 .levels { display: inline-flex; gap: 3px; margin-left: 6px; vertical-align: middle; }
 215 .levels i { width: 8px; height: 8px; border-radius: 2px; background: rgba(255,255,255,0.15); display: block; }
 216 .levels i.on { background: var(--gold); }
 217 .minimap-wrap { position: relative; }
 218 #btn-mini { position: absolute; top: 4px; right: 4px; width: 30px; min-height: 30px; padding: 0; background:
rgba(12,21,29,0.8); }
 219 .minimap-wrap.min #minimap { width: 40px; height: 40px; opacity: 0; pointer-events: none; }
```

```
 220 .minimap-wrap.min #btn-mini { position: static; width: 44px; min-height: 44px; }
 221 #placebar p { margin: 0; font-size: 14px; line-height: 1.3; }
 222 #boxsel { position: fixed; border: 1.5px dashed #fff; background: rgba(255, 255, 255, 0.08); pointer-events: none; z-
index: 8; border-radius: 3px; }
 223 #btn-build { display: none; }
 224
 225 /* ---------- pantallas ---------- */
 226 .screen { position: fixed; top: 0; right: 0; bottom: 0; left: 0; inset: 0; z-index: 20; display: flex; padding:
max(16px, env(safe-area-inset-top, 0px)) max(16px, env(safe-area-inset-right, 0px)) max(16px, env(safe-area-inset-bottom,
0px)) max(16px, env(safe-area-inset-left, 0px)); overflow-y: auto; touch-action: pan-y; -webkit-user-select: none; }
 227 .menu { align-items: center; justify-content: space-between; gap: 32px; background: linear-gradient(90deg, rgba(8, 15,
21, 0.86) 0%, rgba(8, 15, 21, 0.5) 46%, rgba(8, 15, 21, 0.12) 70%); }
 228 .menu-hero { max-width: 520px; padding-left: clamp(0px, 3vw, 40px); }
 229 .brand { display: flex; align-items: center; gap: 14px; }
 230 .brand svg { width: 64px; height: 64px; flex: none; }
 231 .brand h1 { margin: 0; font-family: var(--font-display); font-weight: 700; font-size: clamp(40px, 6.6vw, 76px); letter-
spacing: 0.01em; line-height: 0.95; text-wrap: balance; }
 232 .version { display: inline-block; margin-top: 10px; font-size: 12px; color: var(--muted); border: 1px solid var(--
line); border-radius: 6px; padding: 2px 8px; }
 233 .brand h1 span { color: var(--gold); }
 234 .tagline { font-size: clamp(16px, 2vw, 19px); line-height: 1.5; color: #d3e0e8; max-width: 44ch; margin: 18px 0 20px;
text-wrap: pretty; }
 235 .facts { list-style: none; display: flex; flex-wrap: wrap; gap: 8px; margin: 0 0 18px; padding: 0; }
 236 .facts li { padding: 7px 12px; border-radius: 8px; border: 1px solid var(--line); background: rgba(12, 21, 29, 0.6);
font-size: 14px; color: var(--muted); }
 237 .facts b { font-family: var(--font-display); color: var(--text); margin-right: 4px; }
 238 .hint { font-size: 13px; color: var(--muted); margin: 0; line-height: 1.5; }
 239 .menu-card { width: min(470px, 100%); padding: 18px; display: grid; gap: 14px; margin-right: clamp(0px, 3vw, 40px); }
 240 .menu-card fieldset { border: 0; margin: 0; padding: 0; display: grid; gap: 7px; min-width: 0; }
 241 .menu-card legend { padding: 0; margin-bottom: 7px; font-size: 11px; letter-spacing: 0.1em; text-transform: uppercase;
color: var(--muted); font-weight: 600; }
 242 .legend-row { display: flex; justify-content: space-between; align-items: center; }
 243 .opts { display: grid; grid-template-columns: 1fr 1fr; gap: 7px; }
 244 .opts.three { grid-template-columns: repeat(3, 1fr); }
 245 .opt { position: relative; display: block; cursor: pointer; }
 246 .opt input { position: absolute; opacity: 0; pointer-events: none; }
 247 .opt span { display: grid; gap: 2px; height: 100%; padding: 9px 11px; border-radius: 8px; background: var(--raise);
border: 1px solid transparent; }
 248 .opt b { font-size: 14px; }
 249 .opt small { font-size: 12px; color: var(--muted); line-height: 1.3; }
 250 .opt input:checked + span { border-color: var(--gold); background: rgba(245, 187, 60, 0.1); }
 251 .toggles { grid-template-columns: 1fr 1fr; }
 252 .check { display: flex; align-items: center; gap: 9px; padding: 9px 11px; border-radius: 8px; background: var(--raise);
cursor: pointer; font-weight: 600; font-size: 14px; }
 253 .check input { width: 18px; height: 18px; accent-color: var(--gold); margin: 0; }
 254 .cta { min-height: 52px; font-size: 17px; letter-spacing: 0.06em; text-transform: uppercase; font-family: var(--font-
display); }
 255 .linkbtn { background: none; border: 0; color: var(--gold); cursor: pointer; font-weight: 600; font-size: 13px;
padding: 4px 0; }
 256 .dialog { margin: auto; width: min(460px, 100%); padding: 22px; display: grid; gap: 14px; }
 257 .dialog h2 { margin: 0; font-family: var(--font-display); font-size: 30px; letter-spacing: 0.04em; }
 258 .dialog p { margin: 0; color: var(--muted); line-height: 1.5; }
 259 .dialog .row { display: flex; gap: 8px; flex-wrap: wrap; }
 260 .dialog .row .btn { flex: 1 1 140px; }
 261 .overlay-dim { background: rgba(6, 11, 16, 0.6); }
 262 .stats { display: grid; grid-template-columns: 1fr 1fr; gap: 8px; }
 263 .stats div { padding: 10px 12px; border-radius: 8px; background: var(--raise); }
 264 .stats b { display: block; font-family: var(--font-display); font-size: 22px; font-variant-numeric: tabular-nums; }
 265 .stats span { font-size: 12px; color: var(--muted); }
 266 .result-victory h2 { color: var(--gold); }
 267 .result-defeat h2 { color: var(--bad); }
 268 .help { margin: 0; padding: 0; list-style: none; display: grid; gap: 6px; font-size: 14px; line-height: 1.4; }
 269 .help b { color: var(--text); }
 270 .help li { color: var(--muted); }
 271 .seg2 { display: grid; grid-template-columns: 1fr 1fr; gap: 6px; }
 272 .seg3 { display: grid; grid-template-columns: repeat(3, 1fr); gap: 6px; }
 273 .seg3 .btn { padding: 0 8px; white-space: normal; line-height: 1.15; }
 274 .small-note { margin: 6px 0 0 !important; font-size: 12.5px; color: var(--muted); min-height: 1em; }
 275 .fpschip { display: inline-flex; align-items: center; padding: 0 8px; border-radius: 7px; background: rgba(255, 255,
255, 0.06); font-size: 13px; font-weight: 600; color: var(--good); white-space: nowrap; }
 276
 277 /* ---------- pantallas estrechas (iPad vertical, celular) ---------- */
 278 @media (max-width: 860px) {
 279   .stat { padding: 0 8px; }
 280   .stat b { font-size: 17px; }
 281   .stat svg { width: 18px; height: 18px; }
 282   .pchip span, .pchip em { display: none; }
 283   .pchip { padding: 4px; }
 284   #objectives, #leftcol { display: none; }
 285   #buildpanel { display: none; top: calc(var(--top) + 60px); width: min(300px, calc(100% - 20px)); z-index: 12; }
 286   #buildpanel.open { display: flex; }
 287   #buildpanel.min .tabs, #buildpanel.min .blist { display: grid; }
 288   #btn-build { display: inline-flex; position: fixed; right: var(--right); top: 50%; transform: translateY(-50%); z-
index: 6; flex-direction: column; min-height: 64px; width: 64px; font-size: 11px; padding: 0; }
 289   #leftdock { top: calc(var(--top) + 60px); bottom: auto; flex-direction: column; align-items: flex-start; }
 290   #minimap { width: 112px; height: 112px; }
 291   .quick { grid-template-columns: repeat(5, 44px); }
 292   .qbtn { width: 44px; min-height: 44px; font-size: 0; gap: 0; }
```

```
  #selpanel, #placebar { left: var(--left); right: var(--right); }
  .card { width: 84px; }
  .menu { flex-direction: column; justify-content: flex-start; align-items: stretch; background: rgba(8, 15, 21, 0.72);
}
  .menu-hero, .menu-card { margin: 0 auto; padding-left: 0; }
  .menu-card { padding: 14px; }
  .facts, .hint { display: none; }
  .tagline { margin: 10px 0 0; }
}
@media (max-width: 480px) {
  .stat .lbl { display: none; }
  .stat { grid-template-columns: auto auto; }
  .stat small { display: none; }
  .opts.three { grid-template-columns: 1fr 1fr 1fr; }
  .opt small { display: none; }
}
@media (max-width: 480px) {
  .stat { padding: 0 6px; column-gap: 5px; }
  .stat b { font-size: 15px; }
  .players { padding: 6px 5px; gap: 3px; flex-wrap: nowrap; }
  .pchip i { width: 11px; height: 11px; }
  #btn-sound { display: none; }
}
@media (max-width: 420px) {
  .stat.clock { display: none; }
  .players { display: none; }
  #king { left: auto; right: var(--right); transform: none; top: calc(var(--top) + 56px); }
  #king .bar { width: 48px; }
}
/* Pantallas bajas: celular en horizontal. */
@media (max-height: 520px) {
  .topbar { gap: 6px; }
  .res { padding: 4px 2px; }
  .stat b { font-size: 16px; }
  .stat .lbl { display: none; }
  #objectives, #leftcol { display: none; }
  #leftdock { top: calc(var(--top) + 52px); bottom: auto; flex-direction: column; align-items: flex-start; }
  #minimap { width: 96px; height: 96px; }
  .quick { grid-template-columns: repeat(2, 44px); }
  .qbtn { width: 44px; min-height: 40px; font-size: 0; gap: 0; }
  #selpanel, #placebar { left: calc(var(--left) + 108px); right: calc(var(--right) + 76px); }
  #selpanel { padding: 8px; gap: 6px; }
  .selhead { gap: 8px; }
  .selhead .thumb { width: 40px; height: 40px; }
  .selhead p, .chips, .hptext { display: none; }
  .selhead h3 { font-size: 15px; }
  .hp { margin-top: 4px; }
  .card { width: 74px; padding: 4px; }
  .card img { width: 40px; height: 40px; }
  .card b { font-size: 11.5px; }
  .btn { min-height: 36px; }
  #buildpanel { top: calc(var(--top) + 52px); max-height: calc(100% - var(--top) - var(--bottom) - 52px); }
  .bitem .desc { display: none; }
  .bitem img { width: 40px; height: 40px; }
  .bitem { grid-template-columns: 40px 1fr; }
  .menu { flex-direction: row; align-items: flex-start; background: rgba(8, 15, 21, 0.72); }
  .menu-hero { flex: 0 1 36%; }
  .brand svg { width: 40px; height: 40px; }
  .brand h1 { font-size: 40px; }
  .tagline { font-size: 14px; margin: 8px 0 0; }
  .menu-card { flex: 1 1 auto; padding: 12px; gap: 9px; }
  .opt span { padding: 7px 9px; }
  .opt small { display: none; }
  .menu-card legend { margin-bottom: 4px; }
  .cta { min-height: 44px; }
}
/* Celular vertical: el panel de selección no sube más allá del botón de construir. */
@media (max-width: 860px) and (orientation: portrait) {
  #selpanel { max-height: calc(50vh - 44px - var(--bottom)); overflow-y: auto; overscroll-behavior: contain; touch-
action: pan-y; -webkit-overflow-scrolling: touch; }
}
/* ---------- vestíbulo ---------- */
.lobby { flex-direction: column; gap: 12px; background: linear-gradient(180deg, rgba(8,15,21,0.82), rgba(8,15,21,0.55)
60%, rgba(8,15,21,0.7)); }
.lobby-top { display: flex; align-items: center; gap: 12px; flex-wrap: wrap; }
.brand-sm { display: flex; align-items: center; gap: 8px; font-family: var(--font-display); font-size: 20px; }
.brand-sm svg { width: 34px; height: 34px; }
.brand-sm b span { color: var(--gold); }
.brand-sm .version { margin: 0; }
.lobby-tabs { display: flex; gap: 4px; margin-inline: auto; background: var(--panel); border: 1px solid var(--line);
border-radius: 10px; padding: 4px; }
.lobby-tabs button { min-height: 38px; padding: 0 16px; border-radius: 7px; border: 0; background: none; color: var(--
muted); font-family: var(--font-display); font-weight: 600; letter-spacing: 0.06em; text-transform: uppercase; cursor:
pointer; }
.lobby-tabs button[aria-selected="true"] { background: var(--raise-2); color: var(--text); }
.lobby-res { display: flex; gap: 8px; align-items: center; }
.crystals, .lvl, .creator-badge { display: inline-flex; align-items: center; gap: 6px; padding: 6px 10px; border-
radius: 8px; background: var(--panel); border: 1px solid var(--line); font-family: var(--font-display); font-weight: 600; }
```

```
 374 .crystals i { width: 12px; height: 12px; background: #6fd3ff; transform: rotate(45deg); border-radius: 2px; display:
block; }
 375 .creator-badge { background: var(--gold); color: var(--gold-ink); border-color: transparent; }
 376 .lobby-body { flex: 1 1 auto; min-height: 0; display: grid; grid-template-columns: 190px minmax(0, 1fr) 230px; gap:
12px; align-items: start; }
 377 .lobby-body.wide { grid-template-columns: minmax(0, 1fr); }
 378 .lmenu { display: grid; gap: 4px; padding: 8px; }
 379 .lmenu button { text-align: left; min-height: 40px; padding: 0 12px; border-radius: 8px; border: 0; background: none;
color: var(--text); font-weight: 600; cursor: pointer; display: flex; align-items: center; justify-content: space-between;
gap: 6px; }
 380 .lmenu button.on { background: var(--raise-2); box-shadow: inset 3px 0 0 var(--gold); }
 381 .lmenu button:disabled { color: var(--muted); cursor: not-allowed; }
 382 .lmenu small { font-size: 10.5px; color: var(--muted); font-weight: 500; }
 383 .dot { width: 8px; height: 8px; border-radius: 50%; background: var(--bad); display: inline-block; }
 384 .lmain { display: grid; gap: 12px; min-width: 0; }
 385 .lcard { padding: 16px; display: grid; gap: 12px; }
 386 .ranked .rankhead { display: flex; gap: 14px; align-items: center; }
 387 .rankbadge { width: 64px; height: 64px; flex: 0 0 64px; border-radius: 18px; display: grid; place-items: center;
background: radial-gradient(circle at 30% 25%, #fff6, transparent 55%), var(--rk); box-shadow: 0 4px 14px rgba(0,0,0,0.35),
inset 0 0 0 3px rgba(255,255,255,0.25); transform: rotate(45deg); }
 388 .rankbadge b { transform: rotate(-45deg); font-family: var(--font-display); font-size: 26px; color: #1a1206; }
 389 .ranked h2 { margin: 2px 0; }
 390 .ranked small { font-size: 11px; letter-spacing: 0.1em; text-transform: uppercase; color: var(--muted); }
 391 .rankbar { height: 10px; border-radius: 999px; background: rgba(255,255,255,0.08); overflow: hidden; }
 392 .rankbar i { display: block; height: 100%; background: linear-gradient(90deg, #f2ab38, #ffd27a); }
 393 .ranklist { list-style: none; margin: 0; padding: 0; display: grid; grid-template-columns: repeat(auto-fit,
minmax(120px, 1fr)); gap: 6px; }
 394 .ranklist li { display: flex; align-items: center; gap: 8px; padding: 6px 8px; border-radius: 8px; background:
rgba(8,15,21,0.35); font-size: 13px; opacity: 0.6; }
 395 .ranklist li.past { opacity: 0.85; }
 396 .ranklist li.on { opacity: 1; outline: 2px solid #f2ab38; font-weight: 700; }
 397 .ranklist li i { width: 12px; height: 12px; border-radius: 3px; transform: rotate(45deg); }
 398 .ranklist li small { margin-left: auto; color: var(--muted); }
 399 .rankhist { list-style: none; margin: 0; padding: 0; display: grid; gap: 4px; }
 400 .rankhist li { display: grid; grid-template-columns: 90px 50px 1fr; gap: 8px; align-items: center; font-size: 13px;
padding: 4px 8px; border-radius: 6px; background: rgba(8,15,21,0.3); }
 401 .rankhist li.win span { color: #8ff0a4; }
 402 .rankhist li.loss span { color: #ffb4b5; }
 403 #wheel { position: fixed; inset: 0; z-index: 30; background: rgba(6,10,14,0.35); }
 404 .wheel-hub { position: absolute; transform: translate(-50%, -50%); width: 64px; height: 64px; border-radius: 50%;
display: grid; place-items: center; font-size: 12px; font-weight: 700; color: var(--muted); background: rgba(12,21,29,0.92);
border: 1px solid var(--line); }
 405 .wheel-opt { position: absolute; transform: translate(-50%, -50%); width: 84px; min-height: 52px; border-radius: 14px;
border: 1px solid var(--line); background: rgba(16,26,36,0.95); color: var(--text); display: grid; place-items: center; gap:
0; padding: 4px; cursor: pointer; font: inherit; }
 406 .wheel-opt b { font-size: 12px; white-space: nowrap; }
 407 .wheel-opt small { font-size: 11px; color: var(--muted); font-variant-numeric: tabular-nums; }
 408 .wheel-opt.hot, .wheel-opt:hover:not(:disabled) { border-color: #f2ab38; background: rgba(242,171,56,0.2); }
 409 .wheel-opt:disabled { opacity: 0.4; cursor: default; }
 410 #chatpop { top: calc(var(--top) + 64px); left: 50%; transform: translateX(-50%); width: min(520px, calc(100% - 20px));
max-height: calc(100% - var(--top) - 76px); overflow-y: auto; }
 411 .chattabs { display: flex; gap: 6px; }
 412 .chattabs button[aria-selected="true"] { border-color: #f2ab38; color: var(--text); }
 413 .chatlog { display: grid; gap: 4px; max-height: 260px; overflow-y: auto; font-size: 13px; }
 414 .chatlog p { margin: 0; padding: 4px 8px; border-radius: 8px; background: rgba(8,15,21,0.35); }
 415 .chatlog p.mine { background: rgba(47,127,224,0.16); }
 416 .chatlog p i { display: inline-block; width: 8px; height: 8px; border-radius: 2px; margin-right: 6px; vertical-align:
middle; }
 417 .chatlog small { color: var(--muted); margin-left: 6px; }
 418 .quickmsgs { display: flex; flex-wrap: wrap; gap: 6px; }
 419 .ladder { list-style: none; margin: 0; padding: 0; display: grid; gap: 3px; max-height: 320px; overflow-y: auto; }
 420 .ladder li { display: grid; grid-template-columns: 38px 1fr 12px 70px 46px; gap: 8px; align-items: center; font-size:
13px; padding: 4px 8px; border-radius: 6px; background: rgba(8,15,21,0.3); }
 421 .ladder li.me { outline: 2px solid #f2ab38; background: rgba(242,171,56,0.12); font-weight: 700; }
 422 .ladder li i { width: 10px; height: 10px; border-radius: 2px; transform: rotate(45deg); }
 423 .ladder li small { font-size: 10px; color: var(--muted); border: 1px solid var(--line); border-radius: 4px; padding: 0
4px; margin-left: 4px; }
 424 .ladder li em { font-style: normal; color: var(--muted); font-size: 12px; }
 425 .ladder li strong { text-align: right; font-variant-numeric: tabular-nums; }
 426 .item.plan .btn { white-space: normal; line-height: 1.2; max-width: 100%; }
 427 .item.plan img { filter: grayscale(0.6) brightness(0.9); outline: 2px dashed rgba(127,200,255,0.6); outline-offset:
-4px; border-radius: 8px; }
 428 .fragicon { width: 64px; height: 64px; margin: 4px auto; background: conic-gradient(from 30deg, #7fc8ff, #b48cff,
#7fc8ff 60%, #ffe08a, #7fc8ff); clip-path: polygon(50% 0, 95% 30%, 80% 100%, 20% 100%, 5% 30%); }
 429 .mapcard { display: grid; gap: 4px; justify-items: start; }
 430 .mapthumb { width: 100%; max-width: 132px; aspect-ratio: 1; border-radius: 10px; border: 1px solid var(--line); }
 431 .mapthumbs { display: grid; grid-template-columns: repeat(auto-fill, minmax(96px, 1fr)); gap: 6px; }
 432 .mapopt { display: grid; gap: 4px; padding: 6px; border-radius: 10px; border: 1px solid var(--line); background:
rgba(8,15,21,0.35); color: var(--text); font: inherit; font-size: 12px; cursor: pointer; text-align: center; }
 433 .mapopt img { width: 100%; aspect-ratio: 1; border-radius: 8px; }
 434 .mapopt.on { border-color: #f2ab38; background: rgba(242,171,56,0.14); }
 435 #coach { position: fixed; inset: 0; pointer-events: none; z-index: 25; }
 436 .coach-ring { position: fixed; border: 3px solid #f2ab38; box-shadow: 0 0 0 4px rgba(242,171,56,0.25), 0 0 18px
rgba(242,171,56,0.6); animation: coachpulse 1.2s ease-in-out infinite; }
 437 @keyframes coachpulse { 50% { box-shadow: 0 0 0 9px rgba(242,171,56,0.12), 0 0 26px rgba(242,171,56,0.8); } }
 438 @media (prefers-reduced-motion: reduce) { .coach-ring { animation: none; } }
 439 .coach-tip { position: fixed; width: 220px; padding: 8px 10px; border-radius: 10px; background: #f2ab38; color:
#1a1206; font-size: 13px; font-weight: 700; box-shadow: 0 6px 18px rgba(0,0,0,0.35); }
```

```
.seg4 { grid-template-columns: repeat(4, 1fr) !important; }
.musicbtn.off { opacity: 0.5; }
.musicbtn.off svg { stroke-dasharray: 2 2; }
#specbar { position: fixed; top: calc(var(--top) + 64px); left: 50%; transform: translateX(-50%); z-index: 7; display: flex; gap: 10px; align-items: center; padding: 8px 12px; max-width: calc(100% - 32px); flex-wrap: wrap; justify-content: center; font-size: 13px; }
body.spectating #buildpanel, body.spectating #btn-build, body.spectating #selpanel, body.spectating #prodbar, body.spectating #groups, body.spectating #quick, body.spectating #objectives { display: none !important; }
.histcharts { display: grid; grid-template-columns: repeat(auto-fit, minmax(260px, 1fr)); gap: 10px; }
.histchart { margin: 0; padding: 8px; border-radius: 10px; background: rgba(8,15,21,0.35); border: 1px solid var(--line); }
.histchart figcaption { font-size: 12px; font-weight: 700; color: var(--muted); letter-spacing: 0.04em; margin-bottom: 4px; }
.histwrap { position: relative; }
.histchart svg { width: 100%; height: auto; display: block; touch-action: none; }
.histchart .hgrid { stroke: rgba(255,255,255,0.08); stroke-width: 1; }
.histchart .haxis { fill: var(--muted); font-size: 10px; }
.histchart .hend { fill: var(--text); font-size: 10px; }
.histchart .hcross { stroke: rgba(255,255,255,0.45); stroke-width: 1; stroke-dasharray: 3 3; }
.histtip { position: absolute; top: 4px; min-width: 140px; padding: 6px 8px; border-radius: 8px; background: rgba(8,12,18,0.94); border: 1px solid var(--line); font-size: 11.5px; display: grid; gap: 2px; pointer-events: none; }
.histtip span { display: flex; align-items: center; gap: 6px; }
.histtip i { width: 8px; height: 8px; border-radius: 2px; }
.histtip em { margin-left: auto; font-style: normal; font-variant-numeric: tabular-nums; }
.histlegend { display: flex; flex-wrap: wrap; gap: 6px 14px; font-size: 12px; color: var(--text); margin: 6px 0; }
.histlegend span { display: inline-flex; align-items: center; gap: 6px; }
.histtable summary { cursor: pointer; font-size: 12.5px; color: var(--muted); margin: 4px 0; }
.histtable table { width: 100%; font-size: 12px; border-collapse: collapse; }
.histtable td, .histtable th { padding: 3px 6px; text-align: left; border-bottom: 1px solid var(--line); }
.btn.mini { min-height: 30px; padding: 0 10px; font-size: 12px; margin-left: 8px; }
.allyorders { display: flex; flex-wrap: wrap; align-items: center; gap: 6px; padding: 6px 10px; font-size: 12.5px; color: var(--muted); }
.allyorders .btn.mini { margin-left: 0; min-height: 36px; }
.allyhint { margin: 0; padding: 0 10px 6px; font-size: 12px; color: var(--muted); }
.planecard { display: flex; gap: 10px; align-items: center; padding: 6px 8px; border: 1px solid var(--line); border-radius: 10px; background: rgba(8,15,21,0.35); }
.planecard img { width: 46px; height: 46px; flex: 0 0 46px; }
.planecard .numbers { grid-template-columns: auto auto; gap: 2px 12px; font-size: 12px; }
.radar.airport .pads { display: flex; flex-wrap: wrap; gap: 6px; }
.padslot { display: inline-flex; align-items: center; gap: 4px; padding: 2px 6px 2px 2px; border-radius: 8px; background: rgba(8,15,21,0.45); font-size: 12px; }
.padslot img { width: 26px; height: 26px; }
.upall { display: grid; gap: 6px; padding: 8px; border-radius: 10px; border: 1px solid rgba(242,171,56,0.35); background: rgba(242,171,56,0.05); }
.upall .actions { margin: 0; }
.btn.good { border-color: rgba(94,224,122,0.7); color: #8ff0a4; background: rgba(94,224,122,0.12); }
.coretiles { display: grid; grid-template-columns: repeat(auto-fit, minmax(104px, 1fr)); gap: 6px; }
.ctile { padding: 6px 8px; border-radius: 8px; background: rgba(8,15,21,0.35); border: 1px solid var(--line); min-width: 0; }
.ctile small { display: block; font-size: 10.5px; letter-spacing: 0.06em; text-transform: uppercase; color: var(--muted); white-space: nowrap; overflow: hidden; text-overflow: ellipsis; }
.ctile b { font-family: var(--font-display); font-size: 14px; font-variant-numeric: tabular-nums; }
.ctile.good b { color: var(--good, #5ee07a); }
.ctile.gold b { color: var(--gold, #f2ab38); }
.ctile.bad b { color: #ef5350; }
.ctile.muted b { color: var(--muted); }
.nextup { display: flex; flex-wrap: wrap; gap: 4px 6px; align-items: center; padding: 6px 8px; border-radius: 8px; border: 1px dashed rgba(242,171,56,0.5); background: rgba(242,171,56,0.06); }
.nextup small { width: 100%; font-size: 10.5px; letter-spacing: 0.06em; text-transform: uppercase; color: var(--gold, #f2ab38); font-weight: 700; }
.nextup span { font-size: 12px; padding: 2px 7px; border-radius: 999px; background: rgba(8,15,21,0.4); }
.radar { display: flex; gap: 12px; align-items: center; padding: 8px 10px; border: 1px solid var(--line); border-radius: 10px; background: rgba(8,15,21,0.35); }
.radar-scope { flex: 0 0 52px; height: 52px; border-radius: 50%; position: relative; overflow: hidden;
  background: radial-gradient(circle, rgba(94,224,122,0.18) 0 30%, transparent 31% 32%, rgba(94,224,122,0.12) 33% 62%, transparent 63% 64%, rgba(94,224,122,0.08) 65%);
  border: 1px solid rgba(94,224,122,0.55); }
.radar-scope i { position: absolute; inset: 0; background: conic-gradient(from 0deg, rgba(94,224,122,0.65), rgba(94,224,122,0) 70deg); animation: radarspin 2.4s linear infinite; }
@keyframes radarspin { to { transform: rotate(360deg); } }
.radar-body { flex: 1 1 auto; min-width: 0; display: grid; gap: 4px; }
.radar-status { font-size: 13px; }
.radar .numbers { grid-template-columns: auto auto; gap: 2px 12px; font-size: 12px; }
.radar-idle { color: var(--muted); }
.radar-track { color: #f2c94c; }
.radar-lock { color: #ef5350; }
@media (prefers-reduced-motion: reduce) { .radar-scope i { animation: none; } }
.cover { position: relative; min-height: 230px; padding: 0; overflow: hidden; display: flex; align-items: flex-end;
  background: linear-gradient(180deg, #16283a, #d9874a 60%, #f6c46a); }
.cover-img { position: absolute; inset: -6%; background: var(--cover, none) 70% 40% / cover no-repeat; animation: coverpan 28s ease-in-out infinite alternate; }
@keyframes coverpan { from { transform: scale(1.02) translate(1.5%, 0); } to { transform: scale(1.1) translate(-2.5%, -1.5%); } }
@media (prefers-reduced-motion: reduce) { .cover-img { animation: none; } }
.cover::before { content: ''; position: absolute; inset: 0; z-index: 1; background: linear-gradient(90deg, rgba(8,15,21,0.82), rgba(8,15,21,0.42) 46%, rgba(8,15,21,0) 70%), linear-gradient(0deg, rgba(8,15,21,0.55), rgba(8,15,21,0) 45%); }
.cover-text { position: relative; z-index: 2; padding: 18px 20px; max-width: 62%; }
```

```
 507 .cover-brand { display: flex; align-items: center; gap: 8px; }
 508 .cover-brand svg { width: 26px; height: 26px; }
 509 .cover-text small { font-size: 11px; letter-spacing: 0.12em; text-transform: uppercase; color: #f2ab38; font-weight:
700; }
 510 .cover-text h2 { margin: 4px 0 4px; font-family: var(--font-display); font-size: 36px; line-height: 1.02; letter-
spacing: 0.02em; color: #fff; text-shadow: 0 2px 10px rgba(0,0,0,0.55); }
 511 .cover-text h2 span { color: #f2ab38; }
 512 .cover-text p { margin: 0; font-size: 13.5px; line-height: 1.35; color: #e8edf1; text-shadow: 0 1px 4px
rgba(0,0,0,0.6); }
 513 .cover-news { list-style: none; margin: 10px 0 0; padding: 0; display: flex; flex-wrap: wrap; gap: 5px; }
 514 .cover-news li { font-size: 11px; font-weight: 700; padding: 3px 8px; border-radius: 999px; color: #1a1206; background:
rgba(242,171,56,0.92); }
 515 .lcard h2 { margin: 0; font-family: var(--font-display); font-size: 22px; letter-spacing: 0.03em; }
 516 .lcard h3 { margin: 0; font-size: 12px; letter-spacing: 0.1em; text-transform: uppercase; color: var(--muted); }
 517 .lcard p { margin: 0; color: var(--muted); line-height: 1.45; }
 518 .mode-now { display: grid; gap: 4px; justify-items: center; text-align: center; padding: 10px; border-radius: 10px;
background: rgba(255,255,255,0.04); }
 519 .mode-now small { font-size: 11px; letter-spacing: 0.1em; color: var(--muted); text-transform: uppercase; }
 520 .mode-now b { font-family: var(--font-display); font-size: 22px; }
 521 .play-row { display: grid; grid-template-columns: 1fr 1.6fr 1fr; gap: 8px; }
 522 .play-row .btn { min-height: 52px; font-family: var(--font-display); font-size: 16px; letter-spacing: 0.06em; text-
transform: uppercase; }
 523 .play-row .btn.gold { font-size: 20px; }
 524 .resume { display: flex; gap: 10px; align-items: center; justify-content: space-between; padding: 10px 12px; border-
radius: 10px; border: 1px solid rgba(242,171,56,0.5); background: rgba(242,171,56,0.08); flex-wrap: wrap; }
 525 .resume p { color: var(--text); }
 526 .opts.four { grid-template-columns: repeat(auto-fit, minmax(120px, 1fr)); }
 527 .numbers { display: grid; grid-template-columns: 1fr auto; gap: 6px 10px; font-size: 13.5px; }
 528 .numbers span { color: var(--muted); }
 529 .numbers b { font-family: var(--font-display); text-align: right; font-variant-numeric: tabular-nums; }
 530 .lside { display: grid; gap: 8px; }
 531 .sidebtn { display: grid; grid-template-columns: 38px 1fr auto; align-items: center; gap: 10px; padding: 10px; border-
radius: 10px; background: var(--panel); border: 1px solid var(--line); cursor: pointer; text-align: left; color: var(--
text); }
 532 .sidebtn:hover { background: var(--raise); }
 533 .sidebtn .ic { width: 38px; height: 38px; border-radius: 9px; display: grid; place-items: center; background: var(--
raise-2); font-family: var(--font-display); font-weight: 700; color: var(--gold); }
 534 .sidebtn b { display: block; font-size: 14px; }
 535 .sidebtn small { color: var(--muted); font-size: 12px; }
 536 .badge { min-width: 20px; height: 20px; border-radius: 10px; background: var(--bad); color: #fff; font-size: 11px;
font-weight: 700; display: grid; place-items: center; padding: 0 6px; }
 537 .xpbar { height: 8px; border-radius: 4px; background: rgba(255,255,255,0.1); overflow: hidden; }
 538 .xpbar i { display: block; height: 100%; background: linear-gradient(90deg, #5ee07a, #9be15d); }
 539 .dialog.wide { width: min(760px, 100%); max-height: calc(100vh - 40px); overflow: auto; }
 540 .dialog header { display: flex; justify-content: space-between; align-items: center; gap: 10px; }
 541 .mode-cards { display: grid; grid-template-columns: repeat(auto-fit, minmax(150px, 1fr)); gap: 8px; }
 542 .mode-card { position: relative; display: grid; gap: 4px; padding: 12px; border-radius: 10px; background: var(--raise);
border: 1px solid transparent; text-align: left; cursor: pointer; color: var(--text); }
 543 .mode-card.on { border-color: var(--gold); background: rgba(242,171,56,0.1); }
 544 .mode-card:disabled { opacity: 0.5; cursor: not-allowed; }
 545 .mode-card b { font-family: var(--font-display); font-size: 15px; letter-spacing: 0.04em; text-transform: uppercase; }
 546 .mode-card small { color: var(--muted); font-size: 12px; line-height: 1.35; }
 547 .mode-card em { font-style: normal; font-size: 11px; color: var(--gold); font-weight: 600; }
 548 .daygrid { display: grid; grid-template-columns: repeat(7, 1fr); gap: 6px; }
 549 .day { display: grid; gap: 4px; justify-items: center; padding: 10px 4px; border-radius: 10px; background: var(--
raise); border: 1px solid transparent; font-size: 12px; }
 550 .day b { font-family: var(--font-display); font-size: 16px; color: #6fd3ff; }
 551 .day.done { opacity: 0.55; }
 552 .day.today { border-color: var(--gold); }
 553 .day.big b { color: var(--gold); }
 554 .mission { display: grid; grid-template-columns: 1fr auto; gap: 4px 10px; align-items: center; padding: 10px; border-
radius: 10px; background: var(--raise); }
 555 .mission .xpbar { grid-column: 1 / -1; }
 556 .mission.done { border: 1px solid rgba(94,224,122,0.5); }
 557 .tiers { display: grid; grid-template-columns: repeat(auto-fill, minmax(92px, 1fr)); gap: 6px; }
 558 .tier { display: grid; gap: 3px; justify-items: center; padding: 8px 4px; border-radius: 8px; background: var(--raise);
font-size: 11.5px; text-align: center; border: 1px solid transparent; }
 559 .tier.reached { border-color: rgba(94,224,122,0.5); }
 560 .tier.claimed { opacity: 0.5; }
 561 .tier img { width: 46px; height: 46px; }
 562 .shopgrid { display: grid; grid-template-columns: repeat(auto-fill, minmax(130px, 1fr)); gap: 8px; }
 563 .item { display: grid; gap: 4px; justify-items: center; padding: 10px 6px; border-radius: 10px; background: var(--
raise); border: 1px solid transparent; text-align: center; cursor: pointer; color: var(--text); }
 564 .item img { width: 76px; height: 76px; }
 565 .item b { font-size: 13px; }
 566 .item.owned { border-color: rgba(94,224,122,0.45); }
 567 .item.equipped { border-color: var(--gold); }
 568 .rar { font-size: 10.5px; font-weight: 700; letter-spacing: 0.06em; text-transform: uppercase; border-radius: 5px;
padding: 1px 6px; }
 569 .rar.comun { background: #3a434b; color: #cfd6dc; } .rar.raro { background: #1d4f8c; color: #cfe3ff; }
 570 .rar.epico { background: #5b2d8c; color: #e6d4ff; } .rar.legendario { background: #8c5a12; color: #ffe4b0; }
 571 .chest { display: grid; gap: 8px; padding: 14px; border-radius: 12px; background: var(--raise); }
 572 .probs { display: grid; grid-template-columns: auto 1fr auto; gap: 4px 8px; align-items: center; font-size: 12.5px; }
 573 .probs .xpbar { height: 6px; }
 574 .subtabs { display: flex; gap: 6px; flex-wrap: wrap; }
 575 .subtabs button { min-height: 36px; padding: 0 14px; border-radius: 8px; border: 1px solid var(--line); background:
var(--raise); color: var(--muted); font-weight: 600; cursor: pointer; }
 576 .subtabs button.on { background: var(--gold); color: var(--gold-ink); border-color: transparent; }
```

```
.endhero { display: flex; gap: 14px; align-items: center; }
.endhero h2 { font-size: 40px !important; }
.endstats { display: grid; grid-template-columns: repeat(4, 1fr); gap: 8px; }
.endstats div { padding: 10px; border-radius: 8px; background: var(--raise); text-align: center; }
.endstats b { display: block; font-family: var(--font-display); font-size: 24px; }
.endstats span { font-size: 12px; color: var(--muted); }
.positions { display: grid; gap: 4px; }
.positions div { display: flex; gap: 10px; align-items: center; padding: 6px 10px; border-radius: 8px; background: rgba(255,255,255,0.04); font-size: 14px; }
.positions div.me { border: 1px solid var(--gold); }
.positions i { width: 10px; height: 10px; border-radius: 3px; display: block; }
.prizes { display: grid; grid-template-columns: repeat(3, 1fr); gap: 8px; }
.prizes div { padding: 10px; border-radius: 8px; background: var(--raise); }
.prizes b { display: block; font-family: var(--font-display); color: var(--gold); font-size: 17px; }
.prizes span { font-size: 12px; color: var(--muted); }
.advice { padding: 10px 12px; border-radius: 8px; border: 1px solid rgba(111,211,255,0.4); background: rgba(111,211,255,0.07); font-size: 13.5px; }
.field { display: grid; gap: 6px; }
.field label { font-size: 11px; letter-spacing: 0.1em; text-transform: uppercase; color: var(--muted); font-weight: 600; }
.field input[type="text"] { min-height: 44px; border-radius: 8px; border: 1px solid var(--line); background: var(--raise); color: var(--text); padding: 0 12px; font: inherit; font-family: var(--font-display); letter-spacing: 0.08em; text-transform: uppercase; }
.seg { display: flex; gap: 4px; flex-wrap: wrap; }
.seg button { min-height: 36px; padding: 0 12px; border-radius: 8px; border: 1px solid var(--line); background: var(--raise); color: var(--text); font-weight: 600; cursor: pointer; }
.seg button.on { border-color: var(--gold); background: rgba(242,171,56,0.12); }
.rowx { display: flex; justify-content: space-between; align-items: center; gap: 10px; }
.steps { margin: 0; padding-left: 20px; display: grid; gap: 6px; line-height: 1.4; }
.dialog .btn.icon { width: 38px; min-height: 38px; padding: 0; display: grid; place-items: center; }
.dialog .btn.icon svg { width: 18px; height: 18px; }
#code-form { grid-template-columns: 1fr auto; align-items: end; }
#code-form label { grid-column: 1 / -1; }
.positions small { margin-left: auto; color: var(--muted); font-size: 12px; }
.positions b { font-family: var(--font-display); min-width: 28px; }
@media (max-width: 980px) {
  .lobby-body { grid-template-columns: minmax(0, 1fr); }
  .lmenu { display: flex; flex-wrap: wrap; }
  .lmenu button { flex: 1 1 auto; }
  .lside { grid-template-columns: repeat(auto-fit, minmax(160px, 1fr)); }
}
@media (max-width: 520px) {
  .lobby-tabs { order: 3; width: 100%; justify-content: stretch; }
  .lobby-tabs button { flex: 1; padding: 0 8px; }
  .play-row { grid-template-columns: 1fr 1fr; }
  .play-row .gold { grid-column: 1 / -1; order: -1; }
  .endstats { grid-template-columns: repeat(2, 1fr); }
  .prizes { grid-template-columns: 1fr; }
  .daygrid { grid-template-columns: repeat(4, 1fr); }
  .cover { min-height: 200px; }
  .cover-text { max-width: 92%; padding: 14px; }
  .cover-text h2 { font-size: 26px; }
  .cover-news li { font-size: 10px; padding: 2px 7px; }
}
/* ---------- v0.7: grupos, campaña y misión ---------- */
.mapcol { display: flex; flex-direction: column; align-items: flex-start; gap: 6px; }
.groups { display: grid; grid-template-columns: repeat(4, 1fr); gap: 4px; width: 172px; }
.groups .qbtn { width: auto; min-height: 40px; padding: 2px 0; gap: 0; }
.qbtn.grp b { font-family: var(--font-display); font-size: 16px; line-height: 1; color: var(--text); }
.qbtn.grp small { font-size: 10px; line-height: 1.1; font-family: var(--font-display); }
.qbtn.grp svg { width: 18px; height: 18px; }
.qbtn.grp.has small { color: var(--gold); }
.qbtn.grp.alert small { color: var(--good); }
.actions.groupsave .btn { min-width: 44px; padding: 0 10px; }
.actions.repeat .queue { margin-right: auto; }
#leftcol.camp #objectives li.star::before { content: '★'; border: 0; width: auto; height: auto; margin: 0; color: var(--gold); font-size: 13px; line-height: 1.3; }
#leftcol.camp #objectives li.star span { display: grid; }
#leftcol.camp #objectives li.star small { color: var(--muted); font-size: 11.5px; }
#leftcol.camp #objectives li.star.lost { color: var(--muted); }
#leftcol.camp #objectives li.star.lost::before { color: var(--muted); }
#leftcol.camp #objectives { border-color: rgba(242,171,56,0.45); }
.missions { display: grid; gap: 8px; }
.mcard { display: grid; grid-template-columns: 44px minmax(0, 1fr) auto; gap: 12px; align-items: center; padding: 10px 12px; border-radius: 10px; background: var(--raise); border: 1px solid var(--line); }
.mcard.done { border-color: rgba(242,171,56,0.45); }
.mcard.locked { opacity: 0.55; }
.mnum { width: 44px; height: 44px; display: grid; place-items: center; border-radius: 10px; background: rgba(242,171,56,0.14); color: var(--gold); font-family: var(--font-display); font-size: 22px; font-weight: 700; }
.mcard.locked .mnum { background: rgba(255,255,255,0.06); color: var(--muted); }
.mbody { display: grid; gap: 2px; min-width: 0; }
.mbody b { font-size: 15px; }
.mbody span { font-size: 13px; color: var(--text); line-height: 1.35; }
.mbody small { font-size: 12px; color: var(--muted); line-height: 1.35; }
.mside { display: grid; gap: 6px; justify-items: end; }
.mside .btn { min-width: 96px; }
.mstars i, .bigstars i { font-style: normal; color: rgba(255,255,255,0.18); }
.mstars i.on, .bigstars i.on { color: var(--gold); }
```

```
 656 .mstars { font-size: 16px; letter-spacing: 2px; }
 657 .mlock { font-size: 12px; color: var(--muted); text-align: right; max-width: 120px; }
 658 .bigstars { margin-left: auto; font-size: 44px; letter-spacing: 4px; line-height: 1; white-space: nowrap; }
 659 .starlist { margin: 0; padding: 0; list-style: none; display: grid; gap: 6px; }
 660 .starlist li { display: flex; justify-content: space-between; gap: 10px; flex-wrap: wrap; padding: 8px 10px; border-
radius: 8px; background: var(--raise); font-size: 14px; }
 661 .starlist li span { color: var(--muted); font-size: 13px; }
 662 .starlist li.ok b { color: var(--gold); }
 663 .starlist li.ok b::before { content: '★ '; }
 664 .starlist li.no b::before { content: '☆ '; color: var(--muted); }
 665 .brief .story { color: var(--text) !important; font-size: 15.5px; }
 666 .brief-grid { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; }
 667 .brief-grid h3 { margin: 0 0 6px; }
 668 .brief .tip { padding: 10px 12px; border-radius: 8px; border: 1px solid rgba(111,211,255,0.4); background:
rgba(111,211,255,0.07); color: var(--text) !important; font-size: 13.5px; }
 669 @media (max-width: 860px), (max-height: 520px) {
 670   /* En pantallas chicas el panel de la misión se ve (plegado) arriba a la derecha. */
 671   #leftcol.camp { display: flex; left: auto; right: var(--right); top: calc(var(--top) + 60px); bottom: auto; width:
min(250px, calc(100% - 290px)); min-width: 180px; }
 672   #leftcol.camp #feed { display: none; }
 673   #leftcol.camp #objectives { display: block; padding: 8px 10px; }
 674   #leftcol.camp #objectives ol { font-size: 12.5px; }
 675   .groups { width: auto; grid-template-columns: repeat(4, 44px); gap: 6px; }
 676   .groups .qbtn { min-height: 44px; }
 677   .brief-grid { grid-template-columns: 1fr; }
 678   .bigstars { font-size: 34px; }
 679 }
 680 @media (max-height: 520px) {
 681   .groups { grid-template-columns: repeat(4, 40px); gap: 4px; }
 682   .groups .qbtn { min-height: 38px; }
 683   #leftcol.camp { top: calc(var(--top) + 52px); }
 684 }
 685 @media (max-width: 480px) and (min-height: 521px) {
 686   /* Celular vertical: el panel de la misión va debajo del minimapa y los botones rápidos. */
 687   #leftcol.camp { right: auto; left: var(--left); top: calc(var(--top) + 282px); width: min(300px, calc(100% - 96px));
}
 688 }
 689 @media (max-width: 520px) {
 690   .mcard { grid-template-columns: 36px minmax(0, 1fr); }
 691   .mnum { width: 36px; height: 36px; font-size: 18px; }
 692   .mside { grid-column: 1 / -1; grid-auto-flow: column; justify-content: space-between; align-items: center; }
 693   .endhero { flex-wrap: wrap; }
 694   .bigstars { margin-left: 0; }
 695 }
 696 /* ---------- v0.8: barra de producción, avisos en el borde, iconos y ficha del enemigo ---------- */
 697 #prodbar { position: fixed; top: calc(var(--top) + 64px); left: 50%; transform: translateX(-50%); z-index: 5; display:
flex; gap: 4px; align-items: center; padding: 4px 6px; max-width: min(560px, calc(100% - 540px)); overflow-x: auto;
scrollbar-width: none; }
 698 #prodbar .lbl { padding: 0 4px; white-space: nowrap; }
 699 .pitem { position: relative; flex: none; width: 40px; height: 40px; padding: 2px; border-radius: 8px; border: 1px solid
var(--line); background: rgba(255,255,255,0.04); cursor: pointer; display: grid; place-items: center; }
 700 .pitem img { width: 32px; height: 32px; display: block; }
 701 .pitem em { position: absolute; top: -3px; right: -3px; min-width: 16px; height: 16px; padding: 0 3px; border-radius:
8px; background: var(--gold); color: var(--gold-ink); font-style: normal; font-size: 10.5px; font-weight: 700; line-height:
16px; text-align: center; }
 702 .pitem i { position: absolute; left: 4px; right: 4px; bottom: 3px; height: 3px; border-radius: 2px; background:
rgba(255,255,255,0.12); overflow: hidden; }
 703 .pitem i s { display: block; height: 100%; width: 0; background: var(--good); text-decoration: none; }
 704 .pitem.idle { border-color: #f2c94c; box-shadow: inset 0 0 0 1px rgba(242,201,76,0.5); }
 705 body.has-prod #toasts { top: calc(var(--top) + 116px); }
 706 #edges { position: fixed; inset: 0; pointer-events: none; z-index: 7; }
 707 .edge { position: absolute; left: 0; top: 0; pointer-events: auto; display: flex; align-items: center; gap: 4px;
padding: 4px 8px 4px 4px; min-height: 36px; border-radius: 18px; border: 1px solid rgba(240,88,90,0.7); background:
rgba(60,12,14,0.85); color: #ffd3d3; font-size: 12px; font-weight: 600; cursor: pointer; white-space: nowrap; animation:
edgein 0.18s ease-out; }
 708 /* Solo transparencia: la posición de la flecha va en transform y una animación de transform la taparía. */
 709 @keyframes edgein { from { opacity: 0; } to { opacity: 1; } }
 710 .edge i { width: 26px; height: 26px; display: grid; place-items: center; border-radius: 50%; background: var(--bad);
color: #fff; }
 711 .edge i svg { width: 16px; height: 16px; }
 712 .btn .bi { display: inline-flex; margin-right: 6px; }
 713 .btn .bi svg { width: 16px; height: 16px; }
 714 .queue .bi { display: inline-flex; margin-right: 4px; }
 715 .queue .bi svg { width: 14px; height: 14px; }
 716 .enemycard { grid-template-columns: auto minmax(0, 1fr); border: 1px solid rgba(240,88,90,0.35); }
 717 .enemycard b { font-family: var(--font-body); }
 718 @media (max-width: 860px) {
 719   #prodbar { left: calc(var(--left) + 258px); right: var(--right); transform: none; max-width: none; justify-content:
flex-start; }
 720   body.camp-on #prodbar { right: calc(var(--right) + 262px); }
 721 }
 722 @media (max-height: 520px) {
 723   #prodbar { top: calc(var(--top) + 52px); left: calc(var(--left) + 186px); right: calc(var(--right) + 10px);
transform: none; max-width: none; }
 724   body.camp-on #prodbar { right: calc(var(--right) + 262px); }
 725 }
 726 @media (max-width: 600px) and (min-height: 521px) {
 727   #prodbar { display: none !important; }
```

```
  body.has-prod #toasts { top: calc(var(--top) + 64px); }
}
/* ---------- v0.9: tarjeta «Qué mejoró» ---------- */
.toast.upcard { flex-direction: column; align-items: stretch; text-align: left; gap: 4px; border-color: rgba(94,224,122,0.6); cursor: pointer; }
.toast.upcard ul { margin: 0; padding-left: 18px; font-weight: 500; font-size: 13px; display: grid; gap: 2px; }
@media (prefers-reduced-motion: reduce) {
  .toast { animation: none; }
}
/* ---------- v1.3: panel de selección compacto y botones de 44 × 44 px (PRD 1.5) ---------- */
/* Arriba los botones principales; las partes largas, plegadas en «Más detalles». */
#selpanel { max-height: min(44vh, 380px); overflow-y: auto; overscroll-behavior: contain; touch-action: pan-y; -webkit-overflow-scrolling: touch; }
#selpanel > .selhead { order: -5; }
#selpanel > .mainacts { order: -4; }
/* v1.5: la cola y «Repetir / Siempre» justo debajo de los botones; después las tarjetas (ahora en filas). */
#selpanel > #sel-queue { order: -3; }
#selpanel > .actions.repeat, #selpanel > #loop-wait { order: -2; }
#selpanel:not(.more) .sec { display: none !important; }
.selhead .headbtns { display: flex; gap: 6px; align-items: center; }
.btn.seltoggle { min-width: 44px; padding: 0 10px; font-size: 12.5px; }
@media (max-height: 520px) {
  /* Celular horizontal: más bajo y sin pasar por encima de la barra de arriba. */
  #selpanel { max-height: min(56vh, calc(100vh - var(--top) - 66px - var(--bottom))); }
}
/* Botones táctiles: 44 × 44 px como mínimo. */
.btn, .btn.mini, .btn.icon { min-height: 44px !important; }
.btn.icon { min-width: 44px; width: 44px; }
.btn.mini { padding: 0 12px; }
.tab { min-height: 44px !important; }
.qbtn, .groups .qbtn { min-height: 44px !important; min-width: 44px; }
#btn-build-close { width: 44px !important; min-height: 44px !important; }
#btn-mini { width: 44px !important; min-height: 44px !important; top: 0; right: 0; background: rgba(12,21,29,0.55); }
#btn-obj { min-height: 44px; min-width: 44px; padding: 0 10px; border-radius: 8px; }
.players { min-height: 44px; align-items: center; }
.pchip { min-height: 32px; }
.stat[data-econ] { min-height: 44px; }
.wheel-opt { min-height: 52px; }
/* v1.4: estrellas de las tropas veteranas en la ficha. */
.vetstars { color: #ffd36a; font-size: 0.85em; letter-spacing: 1px; }
/* v1.4: logros, Top jugadores y desafío semanal. */
.achs { display: grid; grid-template-columns: repeat(auto-fill, minmax(230px, 1fr)); gap: 8px; }
.ach { display: grid; grid-template-columns: 48px 1fr; gap: 10px; align-items: start; padding: 10px; border-radius: 10px; background: var(--raise); opacity: 0.7; }
.ach.on { opacity: 1; box-shadow: inset 0 0 0 1px rgba(242,171,56,0.45); }
.ach b { display: block; font-size: 14px; }
.ach small { display: block; color: var(--muted); font-size: 12px; line-height: 1.35; }
.ach em { display: block; font-style: normal; font-size: 12px; color: var(--gold); margin-top: 4px; }
.ach .xpbar { margin-top: 6px; }
.achbadge { width: 48px; height: 48px; border-radius: 50%; display: grid; place-items: center; font-family: var(--font-display); font-weight: 700; font-size: 15px; color: #10161d; background: var(--ac); border: 3px solid rgba(255,255,255,0.25); filter: grayscale(1) brightness(0.6); }
.ach.on .achbadge { filter: none; }
.toptable { width: 100%; border-collapse: collapse; font-size: 14px; }
.toptable th, .toptable td { text-align: left; padding: 8px 6px; border-bottom: 1px solid var(--line); }
.toptable th { font-size: 12px; color: var(--muted); font-weight: 600; }
.toptable td small { color: var(--muted); font-size: 11px; }
.toptable tr.me td { background: rgba(242,171,56,0.14); font-weight: 700; }
.toptable tr.gap td { text-align: center; color: var(--muted); }
.medal { display: inline-grid; place-items: center; width: 26px; height: 26px; border-radius: 50%; font-weight: 700; color: #10161d; }
.medal.m1 { background: #f2c94c; } .medal.m2 { background: #c0c7cf; } .medal.m3 { background: #c98a4b; }
.weeklybox { display: grid; grid-template-columns: 160px 1fr; gap: 14px; align-items: start; }
.weeklybox .mapthumb { width: 160px; height: auto; border-radius: 10px; }
@media (max-width: 560px) { .weeklybox { grid-template-columns: 1fr; } .weeklybox .mapthumb { width: 100%; max-width: 240px; } .toptable { font-size: 12.5px; } .toptable th, .toptable td { padding: 7px 4px; } }
/* v1.4: letra grande en los paneles del juego (el panel conserva su tamaño y se desplaza). */
body.bigtext #selpanel > *, body.bigtext #buildpanel .blist, body.bigtext #buildpanel .tabs, body.bigtext #objlist,
body.bigtext #chatpop > *, body.bigtext #score > *, body.bigtext #econ > *, body.bigtext #pause .panel > *,
body.bigtext .toast { zoom: 1.15; }
body.bigtext .coach-tip { font-size: 15px; width: 250px; }
/* Con letra grande, los paneles de la pausa pueden achicarse al ancho de la pantalla. */
body.bigtext #pause .panel > * { min-width: 0; }
body.bigtext .seg3 { grid-template-columns: repeat(3, minmax(0, 1fr)); }
@media (max-width: 520px) { body.bigtext .seg4 { grid-template-columns: repeat(2, minmax(0, 1fr)) !important; } }
.seg3.a11y, body.bigtext .seg3.a11y { grid-template-columns: minmax(0, 1fr); }
/* «+0/10 s» no se parte en dos líneas. */
#energyRate, #income { white-space: nowrap; }
/* v1.5: la fila de jugadores no se parte en dos líneas; si falta lugar, los nombres se acortan. */
.players { flex-wrap: nowrap; min-width: 0; flex: 0 1 auto; gap: 4px; }
.pchip { min-width: 42px; flex: 0 1 auto; padding: 4px 7px 4px 5px; gap: 5px; }
.pchip span { min-width: 0; overflow: hidden; text-overflow: ellipsis; }
/* ---------- v1.5: fichas de edificios y tropas ---------- */
.cards { display: grid; grid-template-columns: repeat(auto-fill, minmax(86px, 1fr)); overflow-x: visible; touch-action: auto; }
.card { width: auto; min-width: 0; }
.card b { overflow: hidden; text-overflow: ellipsis; white-space: nowrap; max-width: 100%; }
#sel-queue { display: flex; flex-wrap: wrap; align-items: center; gap: 6px; }
```

```
 807 .qhead { width: 100%; }
 808 .qitem { width: 44px; height: 44px; }
 809 .qx { position: absolute; top: 1px; right: 2px; width: 15px; height: 15px; border-radius: 50%; background:
rgba(12,16,22,0.85); color: #ffb4b5; font-size: 12px; line-height: 15px; font-weight: 700; text-align: center; }
 810 .qnote { font-size: 11.5px; color: var(--muted); flex: 1 1 140px; }
 811 .unittiles { grid-template-columns: repeat(auto-fill, minmax(98px, 1fr)); }
 812 .vsline { display: flex; flex-wrap: wrap; gap: 6px; }
 813 .vs { font-size: 12px; padding: 3px 8px; border-radius: 999px; background: rgba(255,255,255,0.05); border: 1px solid
var(--line); }
 814 .vs.good { border-color: rgba(94,224,122,0.45); }
 815 .vs.good b { color: #8ff0a4; }
 816 .vs.bad { border-color: rgba(240,88,90,0.45); }
 817 .vs.bad b { color: #ffb4b5; }
 818 .vs.muted b { color: var(--muted); }
 819 button.chip { cursor: pointer; font: inherit; color: inherit; min-height: 44px; }
 820 button.chip:hover { background: var(--raise-2); }
 821 /* v1.5: en pantallas bajas, «¡A jugar!» queda fijo al pie de la ficha de la misión (antes quedaba escondido abajo). */
 822 .brief .brief-go { position: sticky; bottom: -22px; margin: 0 -22px -22px; padding: 12px 22px 14px; background: var(--
panel-solid); border-top: 1px solid var(--line); z-index: 1; }
 823 /* ---------- v1.5: portada nueva ---------- */
 824 .lobby-tabs button { min-height: 44px; }
 825 .lobby .seg button, .lmenu button { min-height: 44px; }
 826 .lobby-body { grid-template-columns: 212px minmax(0, 1fr) 236px; }
 827 .lmenu button { gap: 8px; }
 828 .lmenu .lm-label { white-space: nowrap; overflow: hidden; text-overflow: ellipsis; min-width: 0; }
 829 .lmenu small { white-space: nowrap; flex: none; }
 830 .cover.hero { min-height: 300px; display: grid; grid-template-columns: minmax(0, 1fr) auto; align-items: end; gap:
12px; }
 831 .cover.hero .cover-text { max-width: none; padding: 20px 22px; }
 832 .cover.hero .cover-text h2 { font-size: clamp(28px, 3vw, 42px); white-space: nowrap; }
 833 .hero-play { position: relative; z-index: 2; display: grid; gap: 8px; justify-items: end; padding: 18px 20px; }
 834 .hero-btn { min-height: 64px !important; min-width: 210px; padding: 0 26px; font-family: var(--font-display); font-
size: 26px; letter-spacing: 0.08em; text-transform: uppercase; display: flex; gap: 10px; align-items: center; justify-
content: center; box-shadow: 0 8px 24px rgba(242,171,56,0.35), 0 2px 0 rgba(0,0,0,0.25); }
 835 .hero-btn svg { width: 22px; height: 22px; flex: none; }
 836 .hero-mode { font-size: 12.5px; font-weight: 600; color: #e8edf1; background: rgba(8,15,21,0.62); padding: 4px 10px;
border-radius: 999px; max-width: 100%; text-align: center; }
 837 .modecard { grid-template-columns: 132px minmax(0, 1fr); align-items: center; }
 838 .modethumb { width: 132px; height: 132px; border-radius: 10px; object-fit: cover; background: rgba(255,255,255,0.04);
grid-row: span 2; align-self: start; }
 839 .modecard .mode-now { justify-items: start; text-align: left; background: none; padding: 0; }
 840 .modebtns { display: flex; flex-wrap: wrap; gap: 8px; }
 841 .modebtns .btn { flex: 1 1 130px; min-height: 46px; display: flex; gap: 8px; align-items: center; justify-content:
center; }
 842 .modebtns .btn svg { width: 18px; height: 18px; flex: none; }
 843 .modefoot { grid-column: 1 / -1; }
 844 .stattiles.numbers { grid-template-columns: repeat(3, minmax(0, 1fr)); gap: 8px; }
 845 .stattiles.numbers div { display: grid; gap: 2px; padding: 10px 12px; border-radius: 10px; background:
rgba(255,255,255,0.04); border: 1px solid var(--line); min-width: 0; }
 846 .stattiles.numbers b { font-size: 22px; text-align: left; overflow: hidden; text-overflow: ellipsis; white-space:
nowrap; }
 847 .stattiles.numbers span { font-size: 11.5px; }
 848 .sidebtn .ic { color: var(--gold); }
 849 .sidebtn .ic svg { width: 22px; height: 22px; }
 850 @media (max-width: 980px) {
 851   .lobby-body { grid-template-columns: minmax(0, 1fr); }
 852 }
 853 @media (max-width: 640px) {
 854   .cover.hero { grid-template-columns: minmax(0, 1fr); min-height: 270px; gap: 4px; }
 855   .hero-play { justify-items: stretch; padding: 0 14px 14px; }
 856   .hero-btn { min-width: 0; width: 100%; min-height: 56px !important; font-size: 22px; }
 857   .cover.hero .cover-text { padding: 14px 14px 0; }
 858   .cover.hero .cover-text h2 { font-size: 30px; }
 859   .modecard { grid-template-columns: 84px minmax(0, 1fr); }
 860   .modethumb { width: 84px; height: 84px; grid-row: auto; }
 861   .modebtns { grid-column: 1 / -1; }
 862   .stattiles.numbers { grid-template-columns: repeat(2, minmax(0, 1fr)); }
 863 }
 864 </style>
 865
 866 <canvas id="gl"></canvas>
 867 <canvas id="overlay"></canvas>
 868
 869 <div id="hud" hidden>
 870   <header class="topbar">
 871     <div class="res panel">
 872       <div class="stat coins" title="Monedas: toca para ver de dónde salen" role="button" tabindex="0" data-
econ="coins">
 873         <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 2.5 20.2 7.2v9.6L12
21.5 3.8 16.8V7.2z"/><path d="M9 9.5h4.2a1.8 1.8 0 0 1 0 3.6H10.8a1.8 1.8 0 0 0 0 3.6H15M12 7.5v1.8M12 16.7v1.8"/></svg>
 874         <span class="lbl">Monedas</span>
 875         <div><b id="coins">0</b><small id="income">+0/min</small></div>
 876       </div>
 877       <div class="stat" id="stat-pop" title="Población">
 878         <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="9" cy="8" r="3.2"/>
<path d="M3 20c0-3.4 2.7-6 6-6s6 2.6 6 6"/><circle cx="17" cy="9" r="2.4"/><path d="M15.5 14.2c3 .2 5.5 2.4 5.5 5.8"/></svg>
 879         <span class="lbl">Tropas</span>
 880         <b id="pop">0/0</b>
```

```
      </div>
      <div class="stat energy" title="Energía: toca para ver de dónde sale" role="button" tabindex="0" data-
econ="energy">
        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linejoin="round"><path
d="M13 2 4.5 13.5H11L10 22l8.5-11.5H12z"/></svg>
        <span class="lbl">Energía</span>
        <div><b id="energy">0</b><small id="energyRate">+0/10 s</small></div>
      </div>
      <div class="stat clock" title="Tiempo">
        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="12" cy="13" r="8"/>
<path d="M12 9v4l2.5 2.5M9.5 2.5h5"/></svg>
        <span class="lbl">Tiempo</span>
        <b id="clock">00:00</b>
      </div>
    </div>
    <div id="players" class="players panel" role="button" tabindex="0" title="Toca para ver el marcador"></div>
    <div class="sys panel">
      <span class="fpschip num" id="fps" hidden>-- fps</span>
      <button class="btn icon" id="btn-sound" type="button" aria-label="Sonido"></button>
      <button class="btn icon" id="btn-pause" type="button" aria-label="Pausa">
        <svg viewBox="0 0 24 24" fill="currentColor"><rect x="6" y="5" width="4" height="14" rx="1"/><rect x="14" y="5"
width="4" height="14" rx="1"/></svg>
      </button>
    </div>
  </header>

  <div id="leftcol">
  <aside id="objectives" class="panel">
    <header><span class="lbl" id="objtitle">Primeros pasos</span><button type="button" id="btn-obj">Ocultar</button>
</header>
    <ol id="objlist"></ol>
  </aside>
  <div id="feed" aria-live="polite"></div>
  </div>
  <section id="econ" class="pop panel" hidden></section>
  <section id="score" class="pop panel" hidden></section>
  <section id="chatpop" class="pop panel" hidden aria-label="Mensajes"></section>

  <div id="toasts" aria-live="polite"></div>
  <div id="prodbar" class="panel" hidden aria-label="Producción"></div>
  <div id="edges"></div>
  <div id="king" class="panel" hidden></div>

  <section id="buildpanel" class="panel" aria-label="Construir">
    <header>
      <h2>Construir</h2>
      <button class="btn icon" id="btn-build-close" type="button" aria-label="Plegar panel">
        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.2"><path d="M6 15l6-6 6 6"/></svg>
      </button>
    </header>
    <div class="tabs" role="tablist" id="btabs"></div>
    <div class="blist" id="blist"></div>
  </section>
  <button class="btn" id="btn-build" type="button">
    <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M3 21h18M5 21V10l7-5 7 5v11M9
21v-6h6v6"/></svg>
    Construir
  </button>

  <div id="leftdock">
    <div class="mapcol">
    <div class="groups" id="groups" aria-label="Grupos y constructores">
      <button class="qbtn grp" type="button" data-group="1" title="Grupo 1 (tecla 1 · dos toques: ir)"><b>1</b><small
id="grp1">—</small></button>
      <button class="qbtn grp" type="button" data-group="2" title="Grupo 2 (tecla 2 · dos toques: ir)"><b>2</b><small
id="grp2">—</small></button>
      <button class="qbtn grp" type="button" data-group="3" title="Grupo 3 (tecla 3 · dos toques: ir)"><b>3</b><small
id="grp3">—</small></button>
      <button class="qbtn grp" type="button" id="btn-builders" title="Constructores: primero los que no trabajan (C)">
        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M14.5 5.5l4 4M4 20l8.5-
8.5M12 6l6 6-2.5 2.5-6-6z"/><path d="M3 21h6"/></svg><small id="bld-idle">0</small></button>
    </div>
    <div class="minimap-wrap" id="miniwrap">
      <canvas id="minimap" width="344" height="344" aria-label="Minimapa: toca para mover la cámara"></canvas>
      <button class="btn icon" id="btn-mini" type="button" aria-label="Plegar minimapa">
        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M3 6l6-3 6 3 6-3v15l-6 3-
6-3-6 3z"/><path d="M9 3v15M15 6v15"/></svg>
      </button>
    </div>
    </div>
    <div class="quick" id="quick">
      <button class="qbtn" type="button" data-quick="army" title="Seleccionar ejército (T)">
        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><circle cx="12" cy="6" r="3"/><path
d="M6 21v-5a6 6 0 0 1 12 0v5M12 13v8"/></svg>Ejército</button>
      <button class="qbtn" type="button" data-quick="sea" title="Seleccionar barcos">
        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M3 15h18l-3 5H6zM7
15V9h10v6M12 9V4"/></svg>Barcos</button>
      <button class="qbtn" type="button" data-quick="air" title="Seleccionar aéreos">
```

```
956         <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M3 6h18M12 6v4"/><ellipse
cx="11" cy="13" rx="5" ry="3"/><path d="M16 13h5l-1 3M8 16l-1 3h8"/></svg>Aéreos</button>
957       <button class="qbtn" type="button" id="btn-box" title="Selección por recuadro (Mayús + arrastrar)">
958         <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-dasharray="3 2.5"><rect
x="4" y="4" width="16" height="16" rx="2"/></svg>Recuadro</button>
959       <button class="qbtn" type="button" id="btn-home" title="Centrar en el Núcleo (Espacio)">
960         <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M12 3 20 7.5v9L12 21l-8-
4.5v-9z"/><rect x="9.5" y="9.5" width="5" height="5" rx="0.5"/></svg>Núcleo</button>
961     </div>
962   </div>
963
964   <section id="selpanel" class="panel" hidden></section>
965   <div id="placebar" class="panel" hidden>
966     <p id="placetext"></p>
967     <div class="actions">
968       <button class="btn" id="btn-place-rot" type="button" title="Girar (R)">
969         <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M20 12a8 8 0 0 1 1-2.4-
5.7M20 4v5h-5"/></svg>Girar</button>
970       <button class="btn gold" id="btn-place-ok" type="button" title="Construir aquí (Enter)">Construir</button>
971       <button class="btn" id="btn-place-cancel" type="button">Cancelar</button>
972     </div>
973   </div>
974   <div id="boxsel" hidden></div>
975   <div id="specbar" class="panel" hidden><span><b>Modo espectador</b> · ves el resto de la partida hasta que haya
ganador</span>
976     <button class="btn" type="button" id="btn-spec-exit">Salir al vestíbulo</button></div>
977 </div>
978
979 <div id="menu" class="screen lobby">
980   <header class="lobby-top">
981     <div class="brand-sm">
982       <svg viewBox="0 0 64 64" aria-hidden="true"><path d="M32 3 57 17.5v29L32 61 7 46.5v-29z" fill="#1a2834"
stroke="#f2ab38" stroke-width="3"/><path d="M26 50h12l-2-24h-8z" fill="#eef1f3"/><path d="M27.2 40h9.6l.5 5h-10.6zM28
31h8l.4 4h-8.8z" fill="#c0392b"/><rect x="27" y="20" width="10" height="6" rx="1" fill="#f2ab38"/><path d="M26 20l6-6 6 6z"
fill="#eef1f3"/><path d="M37 21l15-6v10z" fill="#f2ab38" opacity=".55"/><path d="M27 21 12 15v10z" fill="#f2ab38"
opacity=".35"/></svg>
983       <b>Faro de <span>Ámbar</span></b>
984       <span class="version">v1.5</span>
985     </div>
986     <nav class="lobby-tabs" id="lobby-tabs" role="tablist">
987       <button type="button" role="tab" data-view="play" aria-selected="true">Jugar</button>
988       <button type="button" role="tab" data-view="shop" aria-selected="false">Tienda</button>
989       <button type="button" role="tab" data-view="inv" aria-selected="false">Inventario</button>
990     </nav>
991     <div class="lobby-res">
992       <button class="btn icon musicbtn" id="btn-music-lobby" type="button" aria-label="Música">
993         <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M9 18V5l11-2v13"/><circle
cx="6" cy="18" r="3"/><circle cx="17" cy="16" r="3"/></svg></button>
994       <span class="creator-badge" id="creator-badge" hidden>Creador</span>
995       <span class="crystals" title="Cristales: se ganan jugando"><i></i><b id="crystals">0</b></span>
996       <span class="lvl" id="lvl-badge">Nivel 1</span>
997     </div>
998   </header>
999   <div class="lobby-body" id="lobby-body"></div>
1000 </div>
1001 <div id="modal" class="screen overlay-dim" hidden>
1002   <div class="dialog panel wide" id="modal-body"></div>
1003 </div>
1004
1005 <div id="pause" class="screen overlay-dim" hidden>
1006   <div class="dialog panel">
1007     <h2>Pausa</h2>
1008     <div class="row">
1009       <button class="btn gold" id="btn-resume" type="button">Continuar</button>
1010       <button class="btn" id="btn-restart" type="button">Reiniciar</button>
1011     </div>
1012     <div>
1013       <div class="lbl" style="margin-bottom:6px">Gráficos</div>
1014       <div class="seg3">
1015         <button class="btn" type="button" data-quality="auto">Automática</button>
1016         <button class="btn" type="button" data-quality="high">Alta calidad</button>
1017         <button class="btn" type="button" data-quality="low">Más fluido</button>
1018       </div>
1019       <p class="small-note" id="quality-note"></p>
1020     </div>
1021     <div>
1022       <div class="lbl" style="margin-bottom:6px">Para tu iPad</div>
1023       <div class="seg3 a11y">
1024         <button class="btn" type="button" data-battery="toggle" id="btn-battery" aria-pressed="false">Ahorro de
batería: no</button>
1025         <button class="btn" type="button" data-cvd="toggle" id="btn-cvd" aria-pressed="false">Colores para daltonismo:
no</button>
1026         <button class="btn" type="button" data-bigtext="toggle" id="btn-bigtext" aria-pressed="false">Letra grande:
no</button>
1027       </div>
1028       <p class="small-note">Ahorro de batería: unos 30 cuadros por segundo y menos efectos. Daltonismo: colores que se
distinguen y una forma por jugador en el minimapa.</p>
1029     </div>
```

```
    <div>
      <div class="lbl" style="margin-bottom:6px">Música</div>
      <div class="seg3 seg4">
        <button class="btn" type="button" data-music="0">Apagada</button>
        <button class="btn" type="button" data-music="0.3">Baja</button>
        <button class="btn" type="button" data-music="0.6">Media</button>
        <button class="btn" type="button" data-music="1">Alta</button>
      </div>
    </div>
    <ul class="help">
      <li><b>Mover el mapa:</b> arrastra con un dedo. Zoom y giro con dos dedos (o rueda, Q y E).</li>
      <li><b>Ordenar:</b> toca una tropa y luego un destino o un enemigo. Doble toque elige todas las del mismo tipo.
</li>
      <li><b>Varias tropas:</b> botón Recuadro y arrastra (o Mayús + arrastrar).</li>
      <li><b>Grupos 1, 2 y 3:</b> con tropas elegidas, «Guardar en 1/2/3». Toca 1, 2 o 3 junto al minimapa para
elegirlas; dos toques llevan la cámara. Teclado: número elige; Ctrl o Mayús + número guarda.</li>
      <li><b>Avanzar atacando:</b> en el panel de tropas, y luego toca el destino: pelean con lo que encuentren en el
camino (tecla F).</li>
      <li><b>Formación:</b> «bloque» las junta; «línea» las pone una al lado de otra, de frente al destino.</li>
      <li><b>Repetir tropa:</b> en el Cuartel, la Fábrica, el Puerto o el Helipuerto, «Repetir» y «×5» ponen otra vez
la última tropa.</li>
      <li><b>Botón Constructores</b> (junto al minimapa o tecla C): elige primero a los que no trabajan.</li>
      <li><b>Campaña:</b> 6 misiones en el vestíbulo. Cada una da hasta 3 estrellas.</li>
      <li><b>Constructores:</b> empiezas con uno; entrena más en el Núcleo. Levantan y mejoran los edificios; el
Ingeniero va el doble de rápido y el Barco constructor hace las obras de mar.</li>
      <li><b>Gráficos:</b> «Automática» baja la calidad sola si el juego va lento. «Contador de cuadros» muestra
cuántos cuadros por segundo da tu iPad.</li>
      <li><b>Construir:</b> elige un edificio, toca una casilla dorada, revisa la vista previa y «Construir». El
constructor libre más cercano va solo; también puedes tocar la obra con él seleccionado.</li>
      <li><b>Lugares:</b> Granjas en tierra fértil, Minas en vetas de oro, Bombas en petróleo de tierra y Plataformas
en petróleo del mar.</li>
      <li><b>Energía:</b> Solar, Eólica, Hidráulica, Mareomotriz y Undimotriz. La gastan vehículos, aeronaves, buques,
la Fábrica y el Helipuerto.</li>
      <li><b>Mejorar:</b> toca un edificio terminado y «Mejorar»: niveles 1 a 3 (la Casa llega a nivel 2: Apartamento).
Con el Núcleo en nivel 3 se compra la Cúpula.</li>
      <li><b>Capturar:</b> lleva infantería junto a una Bandera o al Faro y despeja la zona de enemigos durante 12 s.
</li>
      <li><b>Reparar:</b> selecciona un Constructor o Ingeniero y toca un edificio tuyo dañado.</li>
      <li><b>Demoler:</b> devuelve el 75 % si la obra no estaba terminada y el 40 % si ya lo estaba, más las tropas en
cola y la mejora en curso.</li>
      <li><b>Cruzar el mar:</b> selecciona tropas, toca tu Barcaza y luego «Desembarcar» junto a otra isla.</li>
      <li><b>Salir:</b> «Salir al menú» deja la partida en pausa; vuelves con «Volver a la partida». También se guarda
sola cada minuto y al pausar o cambiar de app.</li>
      <li><b>Ganar:</b> destruye todos los Núcleos enemigos.</li>
    </ul>
    <div class="row">
      <button class="btn" id="btn-sound2" type="button">Sonido: sí</button>
      <button class="btn" id="btn-fps" type="button">Contador de cuadros: no</button>
      <button class="btn danger" id="btn-quit" type="button">Salir al menú</button>
    </div>
  </div>
</div>

<div id="brief" class="screen overlay-dim" hidden></div>
<div id="end" class="screen overlay-dim" hidden>
  <div class="dialog panel" id="enddialog"></div>
</div>

<script src="https://cdnjs.cloudflare.com/ajax/libs/three.js/r128/three.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/three@0.128.0/examples/js/loaders/GLTFLoader.js"></script>
<script>
/*__SCRIPTS__*/
</script>
```

faro/src/js/00-config.js · 572 líneas

```
'use strict';
// Faro de Ámbar: datos del juego (edificios, unidades, terreno, jugadores).
// Todo el balance vive aquí para poder ajustarlo sin tocar la lógica.

const SQ3 = Math.sqrt(3);
// v1.2: color de un terreno según el mapa (Dunas Rojas tiene su propia paleta).
function terrainColor(mapKind, t) {
  const m = typeof MAPS !== 'undefined' && MAPS[mapKind];
  const pal = m && m.tint && typeof PALETTES !== 'undefined' ? PALETTES[m.tint] : null;
  return pal && pal[t] !== undefined ? pal[t] : TERRAIN[t].color;
}
const GAME_NAME = 'Faro de Ámbar';
const GAME_VERSION = '1.5';
const START_COINS = 2000;
const START_ENERGY = 60;
// Tropas iniciales de cada imperio (v0.6: ritmo más rápido).
const START_UNITS = ['builder'];
// Comunidad (población) máxima: 240 desde la v1.0 (180 en la v0.9).
const MAX_POP = 240;
// v1.5: desde esta distancia de cámara, cada tropa muestra un icono de su tipo.
const UNIT_ICON_DIST = 34;
// v1.2: tiempo que hay que mantener pulsado el mapa para abrir la rueda de selección.
const WHEEL_HOLD_MS = 450;
// v1.0: distancia máxima (centro a centro) a la que el constructor trabaja en una obra: pegado a ella.
const BUILD_REACH = 1.2;
// v1.0: «Mejoras automáticas» (Núcleo nivel 3): revisa cada 2 s y guarda una reserva de monedas para tropas.
const AUTO_UP = { every: 2, reserve: 1500, coreLevel: 3 };

// Puntos de captura (PRD 1.5 RF-07 y RF-12).
const CAPTURE = { time: 12, near: 1, clear: 2 };
const POINTS = {
  faro: { name: 'Faro', claim: 3, bonus: 0.5, income: 0, desc: '+50 % de ingresos para quien lo controle.' },
  flag: { name: 'Bandera', claim: 2, bonus: 0, income: 200, desc: '+200 monedas/min y territorio en la isla.' },
};
const KING_TIME = 180;

// Escudos del Núcleo (PRD 1.5 RF-11).
const EMERGENCY = { at: 0.25, time: 20 };
const DOME = { cost: 6000, hp: 3000, range: 3, recover: 60 };
// Misil nuclear (v0.9): lo lanza un AIR DEFENSE de nivel 4. Muy caro, recarga larga y aviso al rival.
// v1.4: ahorro de batería (unos 30 cuadros por segundo y menos partículas).
const BATTERY = { minFrameMs: 31, fx: 0.3 };
// v1.4: tropas veteranas. Estrellas por eliminaciones; cada estrella da +10 % de vida máxima y +8 % de daño.
const VETERAN = { kills: [3, 8, 15], hp: 0.10, dmg: 0.08 };
// v1.4: reparación sola (Constructores e Ingenieros libres) y producción continua.
const AUTO_REPAIR = { range: 4, every: 1.5 };
const LOOP_TRAIN = { every: 0.5 };
const NUKE = { cost: 20000, cooldown: 180, range: 40, radius: 3, dmg: 4000, flight: 9,
  // v1.0: 1 minuto de radiación donde cae; quema por segundo esta parte de la vida máxima de los enemigos.
  // El Núcleo se quema más despacio, para que un solo misil no decida la partida.
  rad: { time: 60, unit: 0.04, bld: 0.005, core: 0.0025 } };
// Casa: nivel 1 (+10 de comunidad) y nivel 2 = Apartamento (+20 más). Precio de la mejora.
const HOUSE_LEVELS = [0, 1500];
// Reparación del Ingeniero: vida por segundo y pausa tras recibir daño.
const REPAIR = { rate: 25, pause: 4 };
const HOUSE_LIMIT = 8;

// h = altura de la cara superior de la casilla.
const TERRAIN = {
  deep: { water: true, color: 0x175a85, h: -0.06, name: 'Mar profundo' },
  shallow: { water: true, color: 0x2a88a8, h: -0.03, name: 'Agua poco profunda' },
  oil: { water: true, color: 0x2a88a8, h: -0.03, name: 'Mancha de petróleo' },
  sand: { land: true, color: 0xd6b877, h: 0.26, name: 'Arena' },
  grass: { land: true, color: 0x6fae45, h: 0.42, name: 'Pasto' },
  forest: { land: true, color: 0x548f37, h: 0.42, slow: 1.45, cover: true, name: 'Bosque' },
  gold: { land: true, color: 0x9fb04a, h: 0.42, name: 'Veta de oro' },
  fertile: { land: true, color: 0x93a24b, h: 0.42, name: 'Tierra fértil' },
  oilland: { land: true, color: 0x6f7253, h: 0.42, name: 'Petróleo en tierra' },
  mount: { color: 0x9a958b, h: 0.6, cover: true, name: 'Montaña' },
  // v1.2: meseta alta (ventaja de altura) y duna del desierto.
  high: { land: true, color: 0xa8794e, h: 0.82, name: 'Meseta', highGround: true },
  dune: { land: true, color: 0xd9a066, h: 0.3, name: 'Duna' },
};

const PLAYER_COLORS = [0x2f7fe0, 0xe0484d, 0xf0a030, 0x9b5de5, 0xe85d9f, 0x1fb5a3];
const PLAYER_NAMES = ['Tú', 'Rojo', 'Naranja', 'Violeta', 'Rosa', 'Turquesa'];
// v1.4: colores para daltonismo. Elegidos con una simulación de protanopía, deuteranopía y
// tritanopía para que cualquier par se distinga (diferencia CIEDE2000 de 19 o más), todos claros
// para verse sobre el fondo oscuro; además, una forma distinta por jugador en el minimapa.
const BASE_COLORS = PLAYER_COLORS.slice();
const BASE_NAMES = PLAYER_NAMES.slice();
const CVD_COLORS = [0x3b6cce, 0xb82e2e, 0xfcfc88, 0x9da9e7, 0xef6c82, 0x88fce9];
const CVD_NAMES = ['Tú', 'Rojo', 'Amarillo', 'Lavanda', 'Rosa', 'Aguamarina'];
const PLAYER_SHAPES = ['circle', 'square', 'triangle', 'diamond', 'star', 'cross'];
```

```
function applyPalette(cvd) {
  const cols = cvd ? CVD_COLORS : BASE_COLORS;
  const names = cvd ? CVD_NAMES : BASE_NAMES;
  for (let i = 0; i < PLAYER_COLORS.length; i++) {
    PLAYER_COLORS[i] = cols[i];
    PLAYER_NAMES[i] = names[i];
  }
}

// Multiplicadores de daño por tipo de objetivo:
// inf = infantería, veh = vehículo, ship = barco, sub = submarino, air = aéreo, bld = edificio.
// Lugares (on): 'land' tierra libre · 'landcoast' tierra junto al agua · 'coast' agua junto a la orilla ·
// 'shallowcoast' agua poco profunda junto a la costa · 'deep' mar profundo · lista = solo esos terrenos.
// builtBy: 'land' Constructor/Ingeniero · 'sea' Barco constructor · 'engineer' solo Ingeniero.
const LEVEL_UP_COST = [0, 0.5, 1.0, 1.5]; // mejorar a nivel 2 = 50 % del precio; a nivel 3 = 100 %; a nivel 4 = 150 %
const lvlMult = (lv) => 1 + 0.6 * ((lv || 1) - 1); // v0.9: +60 % de vida, daño e ingresos por nivel
// Comunidad extra por pelear (v0.9): +2 por cada 10 tropas enemigas eliminadas y +15 por Núcleo destruido.
const POP_BONUS = { perKills: 10, killPop: 2, corePop: 15 };
// Escudo propio de cada edificio (v0.9): 20 % de su vida; se recarga si no lo atacan durante 8 s.
const BLD_SHIELD = { pct: 0.2, delay: 8, regen: 0.1 };
// Aeronaves con poca vida vuelven solas a repararse al Helipuerto o al Aeropuerto.
const AIR_RETREAT = 0.3;
const BUILDINGS = {
  core: {
    name: 'Núcleo', cat: null, cost: 0, hp: 6000, build: 20, income: 900, pop: 20, maxLevel: 3,
    claim: 4, vision: 6, size: 1.25, heal: { cls: ['inf', 'veh', 'air', 'ship', 'sub'], pct: 0.02, range: 2 },
    produces: ['builder'],
    levels: [null, { income: 900, pop: 20, hp: 6000, laser: { dmg: 45, range: 4, beams: 1 } },
      { cost: 2500, income: 1200, pop: 25, hp: 8000, laser: { dmg: 70, range: 5, beams: 1 } },
      { cost: 5000, income: 1500, pop: 30, hp: 10000, laser: { dmg: 70, range: 5, beams: 2 } }],
    weapon: { dmg: 45, cd: 1.0, range: 4, proj: 'laser', mult: { inf: 1.3, veh: 1, ship: 1, air: 1, bld: 0, sub: 0 } },
    desc: 'Tu base. Genera monedas, entrena Constructores, dispara un láser y cura a las tropas cercanas. Si cae,
pierdes.',
  },
  farm: {
    fem: true, name: 'Granja', cat: 'eco', cost: 300, hp: 350, build: 6, income: 240, on: ['fertile'], limit: 14,
    builtBy: 'land', desc: '+240 monedas/min. Solo en tierra fértil (parcelas marcadas).',
  },
  pump: {
    fem: true, name: 'Bomba de petróleo', cat: 'eco', cost: 1400, hp: 700, build: 10, income: 650, on: ['oilland'],
limit: 6,
    builtBy: 'land', desc: '+650 monedas/min. Solo en petróleo en tierra (manchas oscuras).',
  },
  mine: {
    fem: true, name: 'Mina de oro', cat: 'eco', cost: 1000, hp: 650, build: 10, income: 600, on: ['gold'], limit: 8,
    builtBy: 'land', desc: '+600 monedas/min. Solo en vetas de oro.',
  },
  rig: {
    fem: true, name: 'Plataforma petrolera', cat: 'eco', cost: 2000, hp: 900, build: 14, income: 1100, on: ['oil'],
limit: 6,
    builtBy: 'sea', sea: true, desc: '+1100 monedas/min. En el petróleo del mar. La construye el Barco constructor.',
  },
  house: {
    fem: true, name: 'Casa', cat: 'eco', cost: 500, hp: 400, build: 6, pop: 10, on: 'land', limit: HOUSE_LIMIT,
maxLevel: 2,
    popLevels: [0, 10, 30], levelNames: [null, 'Casa', 'Apartamento'],
    builtBy: 'land', desc: '+10 de comunidad. Se mejora una vez a Apartamento: +20 más (+30 en total).',
  },
  solar: {
    fem: true, name: 'Solar', cat: 'energy', cost: 200, hp: 300, build: 5, energy: 3, on: 'land', limit: 8,
    builtBy: 'land', desc: '+3 de energía cada 10 s. En cualquier tierra.',
  },
  wind: {
    fem: true, name: 'Eólica', cat: 'energy', cost: 300, hp: 400, build: 6, energy: 5, on: 'land', limit: 8,
    builtBy: 'land', desc: '+5 de energía cada 10 s. En cualquier tierra.',
  },
  hydro: {
    fem: true, name: 'Hidráulica', cat: 'energy', cost: 700, hp: 600, build: 9, energy: 8, on: 'landcoast', limit: 4,
    builtBy: 'land', desc: '+8 de energía cada 10 s. En tierra junto al agua.',
  },
  tidal: {
    fem: true, name: 'Mareomotriz', cat: 'energy', cost: 1000, hp: 700, build: 11, energy: 11, on: 'shallowcoast',
limit: 4,
    builtBy: 'sea', sea: true, desc: '+11 de energía cada 10 s. En agua poco profunda junto a la costa. La construye el
Barco constructor.',
  },
  wave: {
    fem: true, name: 'Undimotriz', cat: 'energy', cost: 1500, hp: 800, build: 13, energy: 15, on: 'deep', limit: 3,
    builtBy: 'sea', sea: true, desc: '+15 de energía cada 10 s. En mar profundo. La construye el Barco constructor.',
  },
  barracks: {
    name: 'Cuartel', cat: 'prod', cost: 1000, hp: 900, build: 10, on: 'land', limit: 3, builtBy: 'land',
    produces: ['builder', 'engineer', 'rifle', 'shotgun', 'ranger', 'sniper', 'breaker', 'hunter', 'rocket', 'medic'],
    desc: 'Entrena los 10 soldados: del Constructor al Cohetero, y el Médico.',
  },
  factory: {
    fem: true, name: 'Fábrica', cat: 'prod', cost: 3000, energyCost: 50, hp: 1500, build: 16, on: 'land', limit: 2,
builtBy: 'land',
```

```
    produces: ['jeep', 'apc', 'lighttank', 'tank', 'heavytank', 'aa', 'arty', 'missiletruck', 'launcher'],
    desc: 'Fabrica tanques, camiones antiaéreos, jeeps y lanzamisiles.',
  },
  harbor: {
    name: 'Puerto', cat: 'prod', cost: 2200, hp: 1300, build: 14, on: 'coast', limit: 2, builtBy: 'land',
    produces: ['speedboat', 'boatEng', 'barge', 'lander', 'frigate', 'destroyer', 'sub', 'carrier'], heal: { cls:
['ship', 'sub'], rate: 25, range: 2 },
    desc: 'En agua junto a la orilla. Construye los 5 barcos y repara los cercanos.',
  },
  helipad: {
    name: 'Helipuerto', cat: 'prod', cost: 4000, energyCost: 80, hp: 1000, build: 18, on: 'land', limit: 6, builtBy:
'land',
    produces: ['heli', 'apache'], heal: { cls: ['air'], rate: 40, range: 2 },
    desc: 'Produce el Helicóptero y el Apache Guardián, y repara a los helicópteros cercanos. Hasta 6.',
  },
  turret: {
    fem: true, name: 'Torreta', cat: 'def', cost: 1500, hp: 1100, build: 10, on: 'land', limit: 20, vision: 5, builtBy:
'land', maxLevel: 4,
    weapon: { dmg: 40, cd: 0.9, range: 4, proj: 'bullet', mult: { inf: 1.5, veh: 1, ship: 1, air: 0.8, bld: 0.6, sub: 0
} },
    minigun: { dmg: 10, cd: 0.15, range: 3, mult: { inf: 1.2, veh: 0.8, ship: 0.5, air: 0.7, bld: 0, sub: 0 } },
    // Nivel 4: 3 miniguns con más daño y tiro antiaéreo (ayuda al AIR DEFENSE).
    aaLevel: 4, aaMult: { air: 1.8 }, aaMiniMult: { air: 1.6 },
    desc: 'Cañón contra tropas, barcos y helicópteros. Nivel 2: 1 minigun · nivel 3: 2 · nivel 4: 3 miniguns, más daño y
antiaéreo. Hasta 20.',
  },
  miniturret: {
    fem: true, name: 'Torreta chica', cat: 'def', cost: 400, hp: 500, build: 5, on: 'land', limit: 4, vision: 4,
builtBy: 'engineer',
    weapon: { dmg: 20, cd: 0.6, range: 3, proj: 'bullet', mult: { inf: 1.3, veh: 0.7, ship: 0.6, air: 0.5, bld: 0, sub:
0 } },
    desc: 'Torreta pequeña y barata. Solo la puede construir un Ingeniero.',
  },
  aadef: {
    name: 'AIR DEFENSE', cat: 'def', cost: 2000, hp: 1000, build: 12, on: 'land', limit: 6, vision: 6, builtBy: 'land',
maxLevel: 4,
    // v0.9: el radar fija el blanco (0,5 s), prioriza aeronaves y lanza ráfagas de 2 misiles.
    weapon: { dmg: 90, cd: 2.4, range: 6, proj: 'missile', mult: { air: 2.2 }, salvo: 2, lock: 0.5 },
    prefersAir: true,
    // Nivel 4 (solo 3 por imperio): lanza el misil nuclear (ver NUKE).
    l4Limit: 3,
    desc: 'Misiles de largo alcance contra aeronaves. Nivel 4 (solo 3 de tus 6): lanza el misil nuclear.',
  },
  wall: {
    name: 'Muro', cat: 'def', cost: 30, hp: 1500, build: 3, on: 'land', limit: 40, builtBy: 'land',
    desc: 'Muralla que bloquea a las tropas enemigas y se une con los muros vecinos. Las tuyas pasan.',
  },
  hospital: {
    fem: true, name: 'Enfermería', cat: 'def', cost: 2000, hp: 700, build: 12, on: 'land', limit: 2, builtBy: 'land',
    heal: { cls: ['inf', 'veh'], pct: 0.03, range: 2 },
    healLevels: [null, { pct: 0.03, range: 2, name: 'Clínica' }, { pct: 0.05, range: 2, name: 'Mini hospital' }, { pct:
0.08, range: 3, name: 'Hospital grande' }],
    desc: 'Cura infantería y vehículos cercanos. Nivel 1 clínica (3 %/s), nivel 2 mini hospital (5 %/s), nivel 3
hospital grande (8 %/s).',
  },
  // v0.9: Aeropuerto militar (caza y bombardero furtivos) y Escudo antimisiles (derriba el misil nuclear).
  airport: {
    name: 'Aeropuerto militar', cat: 'prod', cost: 6000, energyCost: 100, hp: 1800, build: 22, on: 'land', limit: 2,
builtBy: 'land',
    produces: ['fighter', 'bomber'], heal: { cls: ['air'], rate: 40, range: 2 },
    desc: 'Pista y hangares: fabrica el Caza furtivo y el Bombardero furtivo, y repara aviones cercanos.',
  },
  antinuke: {
    name: 'Escudo antimisiles', cat: 'def', cost: 9000, energyCost: 120, hp: 1400, build: 20, on: 'land', limit: 2,
builtBy: 'land',
    maxLevel: 1, intercept: { range: 7, recharge: 120 },
    // v1.1: defensa de punto: también derriba misiles y cohetes enemigos que pasen cerca.
    pointDefense: { range: 3, cd: 1.5 },
    desc: 'Derriba el misil nuclear enemigo si cae a 7 casillas o menos (una carga cada 2 minutos), y misiles y cohetes
enemigos a 3 casillas (uno cada 1,5 s).',
  },
  outpost: {
    name: 'Puesto avanzado', cat: null, cost: 800, hp: 800, build: 8, claim: 2, vision: 4, limit: 6, maxLevel: 1,
    desc: 'Amplía tu territorio. Lo fundan los ingenieros y los barcos constructores.',
  },
};

const BUILD_TABS = [
  { id: 'eco', name: 'Economía', items: ['farm', 'pump', 'mine', 'rig', 'house'] },
  { id: 'energy', name: 'Energía', items: ['solar', 'wind', 'hydro', 'tidal', 'wave'] },
  { id: 'prod', name: 'Producción', items: ['barracks', 'factory', 'harbor', 'helipad', 'airport'] },
  { id: 'def', name: 'Defensa', items: ['turret', 'aadef', 'antinuke', 'wall', 'hospital', 'miniturret'] },
];

// Cúpula: se compra con el Núcleo en nivel 3; nivel 3 da escudo personal a tus tropas.
const DOME_LEVELS = [null,
  { cost: 6000, hp: 3000, range: 3 },
  { cost: 4000, hp: 4500, range: 4 },
```

```
  { cost: 6000, hp: 6000, range: 4, unitShield: 0.25 }];
const UNIT_SHIELD = { regenDelay: 5, regen: 0.1 };

const UNITS = {
  builder: {
    name: 'Constructor', cls: 'inf', mover: 'land', cost: 200, hp: 100, speed: 1.6, pop: 1, build: 4, vision: 3,
    builds: 'land', buildRate: 1, repairs: true,
    desc: 'Construye y repara edificios. Casi no pelea.',
  },
  engineer: {
    name: 'Ingeniero', cls: 'inf', mover: 'land', cost: 500, hp: 140, speed: 1.6, pop: 1, build: 6, vision: 3,
    builds: 'land', buildRate: 2, repairs: true, founds: 'land',
    desc: 'Construye el doble de rápido, repara, funda puestos avanzados y pone la Torreta chica.',
  },
  rifle: {
    name: 'Artillero', cls: 'inf', mover: 'land', cost: 400, hp: 260, speed: 1.5, pop: 1, build: 5,
    weapon: { dmg: 16, cd: 0.7, range: 3, proj: 'bullet', mult: { inf: 1, veh: 0.6, ship: 0.5, air: 0.35, bld: 0.5, sub:
0 } },
    desc: 'Ametralladora. Barato, bueno en grupo y para tomar islas.',
  },
  shotgun: {
    name: 'Escopetero', cls: 'inf', mover: 'land', cost: 600, hp: 320, speed: 1.5, pop: 1, build: 6,
    weapon: { dmg: 48, cd: 1.1, range: 2, proj: 'bullet', mult: { inf: 1.3, veh: 0.6, ship: 0.5, air: 0.3, bld: 0.5,
sub: 0 } },
    desc: 'Mucho daño de cerca, poco alcance.',
  },
  ranger: {
    name: 'Ranger', cls: 'inf', mover: 'land', cost: 700, hp: 360, speed: 1.6, pop: 1, build: 7,
    weapon: { dmg: 24, cd: 0.6, range: 3, proj: 'bullet', mult: { inf: 1.15, veh: 0.7, ship: 0.6, air: 0.4, bld: 0.5,
sub: 0 } },
    desc: 'Artillero de élite: casco, chaleco y rifle.',
  },
  sniper: {
    name: 'Francotirador', cls: 'inf', mover: 'land', cost: 900, hp: 170, speed: 1.3, pop: 1, build: 7,
    weapon: { dmg: 75, cd: 2.2, range: 5, proj: 'bullet', mult: { inf: 1.6, veh: 0.4, ship: 0.4, air: 0, bld: 0.3, sub:
0 } },
    desc: 'Rifle de largo alcance. Muy fuerte contra infantería, poca vida.',
  },
  breaker: {
    name: 'Rompedor', cls: 'inf', mover: 'land', cost: 950, hp: 380, speed: 1.5, pop: 1, build: 7,
    weapon: { dmg: 15, cd: 0.25, range: 2, proj: 'bullet', mult: { inf: 1.4, veh: 0.5, ship: 0.4, air: 0.3, bld: 0.4,
sub: 0 } },
    desc: 'Escopeta automática. Dispara muy rápido de cerca.',
  },
  hunter: {
    name: 'Cazador', cls: 'inf', mover: 'land', cost: 1300, hp: 200, speed: 1.2, pop: 1, build: 9,
    weapon: { dmg: 130, cd: 2.6, range: 6, proj: 'bullet', mult: { inf: 1.4, veh: 0.9, ship: 0.6, air: 0, bld: 0.4, sub:
0 } },
    desc: 'Rifle pesado: dispara más lejos y pega más fuerte que el Francotirador.',
  },
  rocket: {
    name: 'Cohetero', cls: 'inf', mover: 'land', cost: 1500, hp: 230, speed: 1.3, pop: 1, build: 8,
    weapon: { dmg: 95, cd: 2.6, range: 4, proj: 'rocket', mult: { inf: 0.6, veh: 1.8, ship: 1.4, air: 0, bld: 1.6, sub:
0 } },
    desc: 'Cohetes que rompen tanques, barcos y edificios.',
  },
  jeep: {
    name: 'Jeep', cls: 'veh', mover: 'land', cost: 1600, energy: 10, hp: 750, speed: 2.6, pop: 2, build: 9, capacity: 4,
carries: 'inf',
    weapon: { dmg: 20, cd: 0.4, range: 3, proj: 'bullet', mult: { inf: 1.3, veh: 0.6, ship: 0.5, air: 0.5, bld: 0.5,
sub: 0 } },
    desc: 'Rápido. Ideal para cazar infantería.',
  },
  tank: {
    name: 'Tanque', cls: 'veh', mover: 'land', cost: 3400, energy: 30, hp: 2200, speed: 1.2, pop: 3, build: 14,
    weapon: { dmg: 140, cd: 2.2, range: 4, proj: 'shell', mult: { inf: 0.9, veh: 1.3, ship: 1, air: 0, bld: 1.5, sub: 0
} },
    desc: 'Blindado y potente contra soldados y edificios. No dispara al aire.',
  },
  aa: {
    name: 'Camión antiaéreo', cls: 'veh', mover: 'land', cost: 2400, energy: 20, hp: 900, speed: 1.8, pop: 2, build: 10,
    weapon: { dmg: 70, cd: 0.9, range: 5, proj: 'flak', mult: { inf: 0, veh: 0, ship: 0, air: 2.4, bld: 0, sub: 0 } },
    desc: 'Protege a tus tropas desde el cielo: solo dispara a unidades aéreas.',
  },
  arty: {
    name: 'Lanzamisiles', cls: 'veh', mover: 'land', cost: 3800, energy: 40, hp: 700, speed: 1.0, pop: 2, build: 14,
    weapon: { dmg: 380, cd: 6, range: 7, proj: 'missile', mult: { bld: 1 } },
    desc: 'Artillería de muy largo alcance. Solo ataca edificios.',
  },
  lighttank: {
    name: 'Tanque ligero', cls: 'veh', mover: 'land', cost: 2200, energy: 15, hp: 1300, speed: 2.0, pop: 2, build: 10,
    weapon: { dmg: 70, cd: 1.1, range: 3, proj: 'shell', mult: { inf: 1.1, veh: 1.1, ship: 0.8, air: 0, bld: 0.9, sub: 0
} },
    desc: 'Rápido y barato: bueno para flanquear y apoyar a la infantería.',
  },
  heavytank: {
    name: 'Tanque pesado', cls: 'veh', mover: 'land', cost: 5200, energy: 45, hp: 3600, speed: 0.9, pop: 4, build: 20,
    weapon: { dmg: 210, cd: 2.2, range: 3, proj: 'shell', mult: { inf: 0.9, veh: 1.5, ship: 1, air: 0, bld: 1.4, sub: 0
```

```
} },
    desc: 'Lento y muy blindado: rompe tanques y defensas.',
  },
  missiletruck: {
    name: 'Camión de misiles', cls: 'veh', mover: 'land', cost: 4200, energy: 40, hp: 800, speed: 1.4, pop: 2, build:
16,
    weapon: { dmg: 420, cd: 8, range: 6, proj: 'missile', mult: { bld: 1.8 } },
    desc: 'Lanza misiles de largo alcance solo contra edificios. Necesita escolta.',
  },
  apc: {
    name: 'Transporte blindado', cls: 'veh', mover: 'land', cost: 2000, energy: 15, hp: 1600, speed: 2.0, pop: 2, build:
10, capacity: 6, carries: 'inf',
    weapon: { dmg: 14, cd: 0.3, range: 3, proj: 'bullet', mult: { inf: 1.2, veh: 0.4, ship: 0.3, air: 0.3, bld: 0.3,
sub: 0 } },
    desc: 'Lleva 6 soldados protegidos y tiene una ametralladora.',
  },
  launcher: {
    name: 'Lanzador de misiles', cls: 'veh', mover: 'land', cost: 7000, energy: 70, hp: 1200, speed: 0.8, pop: 4, build:
24,
    weapon: { dmg: 900, cd: 14, range: 9, proj: 'missile', mult: { inf: 0.3, veh: 0.6, ship: 0.6, air: 0, bld: 1.6, sub:
0 } },
    desc: 'Camión gigante con un misil de muy largo alcance contra bases. Lento y caro.',
  },
  medic: {
    name: 'Médico', cls: 'inf', mover: 'land', cost: 900, hp: 220, speed: 1.6, pop: 1, build: 8, vision: 3,
    healAura: { cls: ['inf'], pct: 0.05, range: 1 },
    desc: 'Cura a los soldados cercanos (5 % de su vida por segundo). No pelea.',
  },
  speedboat: {
    name: 'Lancha rápida', cls: 'ship', mover: 'sea', cost: 1400, hp: 650, speed: 2.8, pop: 1, build: 8,
    weapon: { dmg: 22, cd: 0.45, range: 3, proj: 'bullet', mult: { inf: 1.2, veh: 0.6, ship: 0.8, air: 0.4, bld: 0.5,
sub: 0 } },
    desc: 'La más rápida. Explora y ataca la costa.',
  },
  boatEng: {
    name: 'Barco constructor', cls: 'ship', mover: 'sea', cost: 1600, hp: 500, speed: 2.2, pop: 1, build: 8,
    builds: 'sea', buildRate: 1, repairs: true, founds: 'sea',
    desc: 'Construye Plataformas, Mareomotrices y Undimotrices, y funda puestos en otras islas.',
  },
  barge: {
    name: 'Barcaza', cls: 'ship', mover: 'sea', cost: 1800, hp: 900, speed: 1.8, pop: 1, build: 10, capacity: 6,
    desc: 'Lleva 6 tropas de tierra a otras islas (un vehículo ocupa 2).',
  },
  lander: {
    name: 'Lancha de desembarco', cls: 'ship', mover: 'sea', cost: 2600, hp: 1600, speed: 2.2, pop: 1, build: 12,
capacity: 10,
    desc: 'Más rápida y resistente que la Barcaza: lleva 10 lugares (un vehículo ocupa 2) y desembarca en la playa.',
  },
  frigate: {
    name: 'Fragata antiaérea', cls: 'ship', mover: 'sea', cost: 3600, energy: 30, detects: true, hp: 2200, speed: 1.7,
pop: 3, build: 14,
    weapon: { dmg: 95, cd: 1.3, range: 6, proj: 'missile', mult: { inf: 0.3, veh: 0.3, ship: 0.6, air: 2.1, bld: 0.3,
sub: 0.4 } },
    desc: 'Barco con misiles antiaéreos de largo alcance. Protege a la flota de aviones y helicópteros.',
  },
  carrier: {
    name: 'Portaaviones', cls: 'ship', mover: 'sea', cost: 15000, energy: 120, detects: true, hp: 9000, speed: 1.0, pop:
8, build: 40,
    hangar: 4, heal: { cls: ['air'], rate: 40, range: 2 },
    weapon: { dmg: 14, cd: 0.12, range: 3, proj: 'bullet', mult: { inf: 0.6, veh: 0.5, ship: 0.5, air: 1.6, bld: 0.3,
sub: 0 } },
    desc: 'El buque más grande: lleva hasta 4 Cazas en su hangar, repara aviones cercanos y tiene defensas de corto
alcance.',
  },
  destroyer: {
    name: 'Destructor', cls: 'ship', mover: 'sea', cost: 4500, energy: 30, detects: true, hp: 2800, speed: 1.3, pop: 4,
build: 16,
    weapon: { dmg: 120, cd: 1.7, range: 5, proj: 'shell', mult: { inf: 1, veh: 1, ship: 1.2, air: 0.3, bld: 1.3, sub:
1.4 } },
    desc: 'Mucha vida. Dispara lejos a barcos, submarinos y la costa.',
  },
  sub: {
    name: 'Submarino', cls: 'sub', mover: 'sea', cost: 3800, energy: 30, stealth: true, hp: 1400, speed: 1.6, pop: 3,
build: 14,
    weapon: { dmg: 240, cd: 3, range: 3, proj: 'torpedo', waterOnly: true, mult: { ship: 1.5, sub: 1, bld: 1.5 } },
    desc: 'Va escondido bajo el agua y rompe barcos. Solo lo ven Destructores, Helicópteros y Puertos cercanos.',
  },
  // v0.9: el Helicóptero tiene lo que tenía el Apache (y su precio); el Apache sube de nivel.
  heli: {
    name: 'Helicóptero', cls: 'air', mover: 'air', cost: 7500, energy: 60, detects: true, hp: 2400, speed: 2.3, pop: 4,
build: 20,
    losBlock: true, doorGun: true,
    weapon: { dmg: 75, cd: 0.7, range: 4, proj: 'rocket', mult: { inf: 1.2, veh: 1.4, ship: 1.2, air: 1.1, bld: 1.1,
sub: 0.8 } },
    gun: { dmg: 12, cd: 0.12, range: 3, mult: { inf: 1.3, veh: 0.6, ship: 0.5, air: 0.8, bld: 0.3, sub: 0 } },
    desc: 'Cohetes, cañón y minigun de puerta. Vuela sobre todo; montañas y bosques le reducen el alcance a 2.',
  },
  apache: {
```

```
    name: 'Apache Guardián', cls: 'air', mover: 'air', cost: 10000, energy: 80, detects: true, hp: 3200, speed: 2.1, pop: 5, build: 26,
    losBlock: true, flares: 20,
    weapon: { dmg: 100, cd: 0.6, range: 4, proj: 'rocket', mult: { inf: 1.2, veh: 1.5, ship: 1.3, air: 1.2, bld: 1.2, sub: 0.9 } },
    gun: { dmg: 16, cd: 0.1, range: 3, mult: { inf: 1.4, veh: 0.7, ship: 0.6, air: 0.9, bld: 0.4, sub: 0 } },
    // Misiles guiados contra tanques y edificios, de largo alcance.
    missile: { dmg: 320, cd: 6, range: 6, proj: 'missile', mult: { inf: 0.4, veh: 1.6, ship: 1.4, air: 0, bld: 1.3, sub: 0.6 } },
    desc: 'Cañón de 30 mm, cohetes y misiles guiados de largo alcance contra tanques. Bengalas: esquiva un misil antiaéreo cada 20 s.',
  },
  fighter: {
    name: 'Caza furtivo', cls: 'air', mover: 'air', cost: 6000, energy: 60, hp: 1400, speed: 3.6, pop: 3, build: 18, plane: true,
    weapon: { dmg: 140, cd: 1.2, range: 5, proj: 'missile', mult: { inf: 0.1, veh: 0.2, ship: 0.2, air: 2.3, bld: 0.3, sub: 0 } },
    desc: 'Caza de quinta generación: muy rápido, misiles aire-aire. Limpia el cielo de aviones y helicópteros.',
  },
  bomber: {
    name: 'Bombardero furtivo', cls: 'air', mover: 'air', cost: 9000, energy: 90, hp: 2600, speed: 2.0, pop: 5, build: 28, plane: true,
    stealth: true, prefersBld: true,
    weapon: { dmg: 650, cd: 6, range: 1, proj: 'bomb', mult: { inf: 0.5, veh: 0.8, ship: 1, air: 0, bld: 2, sub: 0 } },
    desc: 'Ala volante invisible al radar: las defensas solo lo ven de muy cerca. Suelta bombas pesadas sobre edificios.',
  },
};

for (const [k, d] of Object.entries(BUILDINGS)) {
  d.key = k;
  if (!d.maxLevel) d.maxLevel = 3;
}
// v0.9: toda tropa armada puede dañar edificios y Núcleos (las antiaéreas, poco).
for (const d of Object.values(UNITS)) {
  if (d.weapon && !(d.weapon.mult.bld > 0)) d.weapon.mult.bld = 0.3;
}
for (const [k, d] of Object.entries(UNITS)) d.key = k;

// Modos del vestíbulo (lobby: true) y modos internos para Personalizado y pruebas.
const MODES = {
  // v1.1: alliances = se pueden formar y romper alianzas durante la partida (todos contra todos).
  conquest: { name: 'Conquista de islas', desc: '6 imperios. Gana el último Núcleo en pie. Puedes aliarte desde el marcador.', players: 6, teams: [0, 1, 2, 3, 4, 5], lobby: true, alliances: true },
  fog: { name: 'Islas en la niebla', desc: 'Solo ves lo que está cerca de tus tropas y edificios.', players: 6, teams: [0, 1, 2, 3, 4, 5], fog: true, lobby: true, alliances: true },
  turbo: { name: 'Turbo ×2', desc: 'Monedas y tropas salen el doble de rápido.', players: 6, teams: [0, 1, 2, 3, 4, 5], speed: 2, lobby: true, alliances: true },
  duos: { name: 'Dúos', desc: '3 parejas: tú y un aliado bot aparecen juntos. Gana la última pareja en pie.', players: 6, teams: [0, 0, 1, 1, 2, 2], lobby: true, duos: true },
  // v1.3: la Clasificatoria aparte (v1.1) ya no se elige; queda para abrir partidas guardadas antes.
  ranked: { name: 'Liga 3 vs 3', desc: 'Dos equipos de 3 con aliados bots. Da puntos de rango.', players: 6, teams: [0, 0, 0, 1, 1, 1], ranked: true },
  league: { name: 'Liga 3 vs 3', desc: 'Dos equipos de 3 con aliados bots. Da puntos de rango: ganar +25, perder −15.', players: 6, teams: [0, 0, 0, 1, 1, 1], lobby: true, minLevel: 10, ranked: true },
  king: { name: 'Rey del Faro', desc: 'Mantén el Faro 3 min en total.', players: 4, teams: [0, 1, 2, 3], king: true, lobby: true },
  blitz: { name: 'Choque exprés', desc: '6.000 monedas y ritmo ×2.', players: 4, teams: [0, 1, 2, 3], coins: 6000, energy: 150, speed: 2, lobby: true, alliances: true },
  duel: { name: 'Duelo 1 vs 1', desc: 'Tú contra 1 rival', players: 2, teams: [0, 1] },
  ffa: { name: 'Todos contra todos', desc: '4 imperios, gana el último', players: 4, teams: [0, 1, 2, 3], alliances: true },
  teams: { name: 'Equipos 2 vs 2', desc: 'Con Rojo como aliado', players: 4, teams: [0, 0, 1, 1] },
  big: { name: 'Gran batalla', desc: '6 imperios, mapa lleno', players: 6, teams: [0, 1, 2, 3, 4, 5], alliances: true },
  tutorial: { name: 'Tutorial', desc: 'Aprende a jugar contra un bot lento en otra isla.', players: 2, teams: [0, 1], tutorial: true, coins: 6000 },
};

const MAPS = {
  archipelago: { name: 'Archipiélago Ámbar', desc: 'Islas en círculo unidas por puentes de arena.' },
  open: { name: 'Mar abierto', desc: 'Sin puentes: necesitas barcos o helicópteros.' },
  continent: { name: 'Continente', desc: 'Una gran isla con lagos.' },
  bay: { name: 'Bahía Gris', desc: 'Islas irregulares; solo algunas tienen puente.', palette: 'cold' },
  reef: { name: 'Arrecife Doble', desc: 'Dos grupos de islas separados por agua navegable.' },
  // v1.2: variantes con nombres propios.
  mesa: { name: 'Mesetas de Cobre', desc: 'Un continente con mesetas altas: desde lo alto se dispara y se ve 1 casilla más lejos.', base: 'continent' },
  dunes: { name: 'Dunas Rojas', desc: 'Desierto de islas con puentes y oasis fértiles.', base: 'archipelago', tint: 'desert' },
};

// Colores alternativos por mapa (Bahía Gris).
const PALETTES = {
  // v1.2: Dunas Rojas.
  desert: { deep: 0x1f5f7a, shallow: 0x3aa0a8, oil: 0x3aa0a8, sand: 0xe3bf7e, grass: 0xbfa75c, forest: 0x7f8f3e,
    gold: 0xc9b04a, mount: 0xb07a52, fertile: 0x93a84a, oilland: 0x7a6a4e, dune: 0xd9a066 },
  cold: { deep: 0x1d4a63, shallow: 0x3a7891, oil: 0x3a7891, sand: 0xb9b6a6, grass: 0x6f9a74, forest: 0x4f7a5d,
    gold: 0x95a36a, mount: 0x8d949a, fertile: 0x8a9a62, oilland: 0x5e6458 },
```

```
};

// Tácticas por dificultad: agrupar oleadas, retirarse, contrarrestar y atacar defensas primero.
const DIFFICULTY = {
  // v0.8: focus = fuego concentrado, harass = hostigar la economía, turret = Torreta del lado del
  // ataque, teamwork = aliados atacan juntos y se ayudan.
  // v1.0, solo Difícil: snipe = busca primero Constructores y economía; medevac = retira antes a los
  // heridos; earlyAA = defensa antiaérea en cuanto ve aviones; fastUp = mejora antes; earlyNuke = misil antes.
  easy: { name: 'Fácil', income: 0.75, think: 3.0, firstWave: 230, wave: 8,
    tactics: { group: false, retreat: false, counter: false, focusDefense: false, focus: false, harass: false, turret:
false, teamwork: true } },
  normal: { name: 'Normal', income: 1.0, think: 1.8, firstWave: 150, wave: 7,
    tactics: { group: true, retreat: true, counter: true, focusDefense: false, focus: true, harass: true, turret: true,
teamwork: true } },
  hard: { name: 'Difícil', income: 1.7, think: 0.8, firstWave: 80, wave: 6,
    tactics: { group: true, retreat: true, counter: true, focusDefense: true, focus: true, harass: true, turret: true,
teamwork: true,
      snipe: true, medevac: true, earlyAA: true, fastUp: true, earlyNuke: true } },
};

// Personalidades de los bots. plan = peso de cada tipo de tropa en su ejército.
const PERSONALITIES = {
  aggro: {
    name: 'Agresivo', firstWave: 0.6, waveEvery: 0.7, waveSize: 0.8, milShare: 0.12, turrets: 0.5, capture: 0,
    plan: { inf: 3, veh: 3, sea: 1, air: 1 },
    lines: { attack: ['¡Prepárate, voy por ti!', 'Tu base será mía.', '¡A la carga!'], faro: ['El Faro es mío. ¡Ven a
quitármelo!'],
      fall: ['Esto no ha terminado…'], provoked: ['¿Me atacas? Te arrepentirás.'] },
  },
  builder: {
    name: 'Constructor', firstWave: 1.5, waveEvery: 1.3, waveSize: 1.4, milShare: -0.08, turrets: 1.6, capture: 0,
    plan: { inf: 3, veh: 3, sea: 1, air: 2 },
    lines: { attack: ['Mi ejército está listo. Allá voy.', 'He preparado algo grande para ti.'], faro: ['El Faro
alimentará mi imperio.'],
      fall: ['Mis murallas no bastaron.'], provoked: ['Reforzaré mis defensas… y luego iré por ti.'] },
  },
  admiral: {
    name: 'Almirante', firstWave: 1.1, waveEvery: 1.0, waveSize: 1.0, milShare: 0, turrets: 1, capture: 0, harborFirst:
true,
    plan: { inf: 3, veh: 1, sea: 4, air: 1 },
    lines: { attack: ['Mi flota zarpa hacia tu costa.', 'Rumbo a tu isla. ¡Zafarrancho!'], faro: ['Ese Faro guiará a mi
flota.'],
      fall: ['Abandonen el barco…'], provoked: ['¿Disparas a mis barcos? Error.'] },
  },
  keeper: {
    name: 'Guardián del Faro', firstWave: 1.0, waveEvery: 1.0, waveSize: 1.0, milShare: 0, turrets: 1, capture: 2,
    plan: { inf: 3, veh: 2, sea: 1, air: 1 },
    lines: { attack: ['Despejaré el camino al Faro, empezando por ti.', 'Nadie amenaza el Faro.'], faro: ['La luz del
Faro es mía.'],
      fall: ['Que otro cuide el Faro…'], provoked: ['Aléjate de mis Banderas.'] },
  },
};
const PERSONALITY_ORDER = ['aggro', 'builder', 'admiral', 'keeper'];
const ALLY_LINES = { attack: 'Voy a atacar a {t}. ¡Cúbreme!', faro: 'Tomé el Faro para el equipo.', flag: 'Tomé una
Bandera para el equipo.' };
const CAT_NAMES = { inf: 'infantería', veh: 'vehículos', ship: 'buques', sub: 'submarinos', air: 'aeronaves', bld:
'edificios' };

// Campaña «La ruta del Faro» (v0.7). goal = lo que hay que lograr para ganar; stars = las 2 estrellas
// extra (la primera estrella es terminar la misión). personas: personalidad de cada bot por número.
// Tipos de meta: eco (ingresos y tropas), survive (aguantar), hold (Faro en tu poder), flags (Banderas
// de otras islas), destroy (todos los Núcleos rivales). Estrellas: time (menos de X s), lostBld y
// lostUnits (perder X o menos), coreEnd (Núcleo sobre X al final), coreMin (Núcleo nunca bajo X).
const CAMPAIGN = [
  {
    id: 1, name: 'Primera luz', mapKind: 'continent', seed: 7101, players: 2, difficulty: 'easy',
    personas: [null, 'builder'], passive: true, crystals: 50,
    story: 'Tu colonia acaba de llegar a una isla tranquila. El Faro de Ámbar lleva años apagado y nadie vigila la
costa. Antes de pensar en batallas, pon en marcha la economía y arma una pequeña guardia.',
    tip: 'Las Granjas van en la tierra fértil. El Cuartel entrena Artilleros y Escopeteros. Tu vecino de esta isla no
atacará.',
    goal: { kind: 'eco', income: 1600, army: 5 },
    stars: [{ kind: 'time', max: 360 }, { kind: 'lostBld', max: 0 }],
  },
  {
    id: 2, name: 'Defender la costa', mapKind: 'archipelago', seed: 7202, players: 2, difficulty: 'normal',
    personas: [null, 'admiral'], coins: 4000, crystals: 75, firstWave: 100, waveEvery: 0.9,
    enemyStart: { buildings: ['harbor'], units: ['speedboat', 'speedboat', 'rifle', 'rifle', 'shotgun'] },
    myStart: { buildings: ['turret'] },
    story: 'Un Almirante quiere tu isla para su flota y sus barcos ya zarparon. Levanta defensas en la costa y aguanta
hasta que llegue la ayuda.',
    tip: 'Las Torretas disparan a barcos. Pon algunas entre el agua y tu Núcleo, y repara con Constructores.',
    goal: { kind: 'survive', time: 360 },
    stars: [{ kind: 'coreEnd', min: 0.5 }, { kind: 'lostBld', max: 3 }],
  },
  {
    id: 3, name: 'El Faro', mapKind: 'archipelago', seed: 7303, players: 2, difficulty: 'normal',
    personas: [null, 'keeper'], crystals: 100, myStart: { units: ['rifle', 'rifle', 'rifle'] },
```

```
    guardFaro: { units: ['rifle', 'rifle', 'rifle', 'shotgun'] },
    story: 'El Faro está en la isla del centro y un Guardián del Faro ya lo tomó con 4 guardias. Tienes 3 Artilleros: junta más tropas, quítale el Faro y mantenlo encendido.',
    tip: 'Solo la infantería captura: quédate junto al Faro sin enemigos cerca. El tiempo se suma aunque lo pierdas un rato. Usa «Avanzar atacando».',
    goal: { kind: 'hold', time: 60 },
    stars: [{ kind: 'time', max: 480 }, { kind: 'lostUnits', max: 6 }],
  },
  {
    id: 4, name: 'Cruzar el mar', mapKind: 'open', seed: 7404, players: 2, difficulty: 'normal',
    personas: [null, 'keeper'], crystals: 125, myStart: { buildings: ['harbor'], units: ['barge'] },
    guardFlags: { count: 2, units: ['rifle', 'rifle', 'shotgun'] },
    story: 'Más allá del mar hay islas con Banderas. Tu Puerto y una Barcaza ya están listos, pero un Guardián del Faro puso guardias en las Banderas más cercanas. Entrena infantería, cruza el mar y quítaselas.',
    tip: 'Construye un Cuartel y entrena Artilleros o Escopeteros. Selecciónalos y toca la Barcaza para subirlos; muévela junto a otra isla y toca «Desembarcar».',
    goal: { kind: 'flags', count: 2 },
    stars: [{ kind: 'time', max: 600 }, { kind: 'lostUnits', max: 8 }],
  },
  {
    id: 5, name: 'Asedio', mapKind: 'continent', seed: 7505, players: 2, difficulty: 'hard',
    personas: [null, 'builder'], coins: 8000, crystals: 150, fortress: true,
    story: 'Un imperio se encerró tras murallas, Torretas de nivel 2 y 3 y AIR DEFENSE. Tienes 8.000 monedas para preparar el asedio. Rompe sus defensas y destruye su Núcleo.',
    tip: 'Los Tanques y los Lanzamisiles pegan fuerte a los edificios. La AIR DEFENSE derriba helicópteros: déjalos para el final.',
    goal: { kind: 'destroy' },
    stars: [{ kind: 'time', max: 720 }, { kind: 'lostUnits', max: 15 }],
  },
  {
    id: 6, name: 'La gran batalla', mapKind: 'archipelago', seed: 7606, players: 4, difficulty: 'normal',
    personas: [null, 'builder', 'aggro', 'admiral'], coins: 4000, crystals: 250,
    story: 'Tres imperios quieren la luz del Faro de Ámbar. Solo uno quedará en pie. Que sea el tuyo.',
    tip: 'Empiezas con 4.000 monedas. Los rivales también pelean entre ellos: defiende tu Núcleo (la estrella pide que nunca baje del 50 %) y ataca al más débil. Ahora tienen aviones y misiles: ten AIR DEFENSE.',
    goal: { kind: 'destroy' },
    stars: [{ kind: 'time', max: 1500 }, { kind: 'coreMin', min: 0.5 }],
  },
];
const CAMPAIGN_STAR_CRYSTALS = 20;

const OBJECTIVES = [
  { id: 'builder', text: 'Entrena un Constructor más en el Núcleo' },
  { id: 'energy', text: 'Construye una energía (Solar o Eólica)' },
  { id: 'barracks', text: 'Construye un Cuartel' },
  { id: 'army', text: 'Ten 5 tropas de combate' },
  { id: 'capture', text: 'Captura una Bandera o el Faro con infantería' },
  { id: 'kill', text: 'Destruye un Núcleo enemigo' },
];
```

faro/src/js/01-util.js · 173 líneas

```
'use strict';
// Utilidades: azar con semilla, matemática hexagonal, ruido y montículo binario.

function mulberry32(seed) {
  let a = seed >>> 0;
  return function rand() {
    a = (a + 0x6d2b79f5) | 0;
    let t = Math.imul(a ^ (a >>> 15), 1 | a);
    t = (t + Math.imul(t ^ (t >>> 7), 61 | t)) ^ t;
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}

// Hexágonos de cara plana con coordenadas axiales (q, r). Radio de casilla = 1.
const HEX_DIRS = [[1, 0], [1, -1], [0, -1], [-1, 0], [-1, 1], [0, 1]];

function hexDist(aq, ar, bq, br) {
  const dq = aq - bq;
  const dr = ar - br;
  return (Math.abs(dq) + Math.abs(dr) + Math.abs(dq + dr)) / 2;
}

function hexToWorld(q, r) {
  return { x: 1.5 * q, z: SQ3 * (r + q / 2) };
}

function hexRound(q, r) {
  const x = q;
  const z = r;
  const y = -x - z;
  let rx = Math.round(x);
  let ry = Math.round(y);
  let rz = Math.round(z);
  const dx = Math.abs(rx - x);
  const dy = Math.abs(ry - y);
  const dz = Math.abs(rz - z);
  if (dx > dy && dx > dz) rx = -ry - rz;
  else if (dy > dz) ry = -rx - rz;
  else rz = -rx - ry;
  return { q: rx, r: rz };
}

function worldToHex(x, z) {
  return hexRound((2 / 3) * x, (-1 / 3) * x + (SQ3 / 3) * z);
}

// Desplazamientos ordenados por distancia, para buscar casillas libres en espiral.
const SPIRAL = (() => {
  const out = [];
  const R = 9;
  for (let q = -R; q <= R; q++) {
    for (let r = -R; r <= R; r++) {
      const d = hexDist(0, 0, q, r);
      if (d <= R) out.push({ q, r, d });
    }
  }
  out.sort((a, b) => a.d - b.d || a.q - b.q || a.r - b.r);
  return out;
})();

function hash2(ix, iz, seed) {
  let h = Math.imul(ix, 374761393) ^ Math.imul(iz, 668265263) ^ Math.imul(seed, 1442695041);
  h = Math.imul(h ^ (h >>> 13), 1274126177);
  h ^= h >>> 16;
  return (h >>> 0) / 4294967296;
}

function valueNoise(x, z, seed) {
  const ix = Math.floor(x);
  const iz = Math.floor(z);
  const fx = x - ix;
  const fz = z - iz;
  const sx = fx * fx * (3 - 2 * fx);
  const sz = fz * fz * (3 - 2 * fz);
  const a = hash2(ix, iz, seed);
  const b = hash2(ix + 1, iz, seed);
  const c = hash2(ix, iz + 1, seed);
  const d = hash2(ix + 1, iz + 1, seed);
  return a + (b - a) * sx + (c - a) * sz + (a - b - c + d) * sx * sz;
}

function fbm(x, z, seed) {
  return valueNoise(x, z, seed) * 0.6 + valueNoise(x * 2.1, z * 2.1, seed + 7) * 0.3 +
    valueNoise(x * 4.3, z * 4.3, seed + 13) * 0.1;
```

```
}

class MinHeap {
  constructor() {
    this.items = [];
    this.prio = [];
  }

  get size() {
    return this.items.length;
  }

  push(item, p) {
    const it = this.items;
    const pr = this.prio;
    let i = it.length;
    it.push(item);
    pr.push(p);
    while (i > 0) {
      const parent = (i - 1) >> 1;
      if (pr[parent] <= p) break;
      it[i] = it[parent];
      pr[i] = pr[parent];
      i = parent;
    }
    it[i] = item;
    pr[i] = p;
  }

  pop() {
    const it = this.items;
    const pr = this.prio;
    const top = it[0];
    const lastI = it.pop();
    const lastP = pr.pop();
    const n = it.length;
    if (n > 0) {
      let i = 0;
      for (;;) {
        const l = i * 2 + 1;
        const r = l + 1;
        let m = i;
        let mp = lastP;
        if (l < n && pr[l] < mp) { m = l; mp = pr[l]; }
        if (r < n && pr[r] < mp) { m = r; mp = pr[r]; }
        if (m === i) break;
        it[i] = it[m];
        pr[i] = pr[m];
        i = m;
      }
      it[i] = lastI;
      pr[i] = lastP;
    }
    return top;
  }
}

function clamp(v, lo, hi) {
  return v < lo ? lo : v > hi ? hi : v;
}

function lerpAngle(a, b, t) {
  let d = b - a;
  while (d > Math.PI) d -= Math.PI * 2;
  while (d < -Math.PI) d += Math.PI * 2;
  return a + d * t;
}

function fmt(n) {
  return Math.floor(n).toLocaleString('es-ES');
}

function fmtTime(s) {
  const m = Math.floor(s / 60);
  const ss = Math.floor(s % 60);
  return `${String(m).padStart(2, '0')}:${String(ss).padStart(2, '0')}`;
}

function hexColor(n) {
  return '#' + n.toString(16).padStart(6, '0');
}

// Escudos en 3 etapas (v0.9): azul = bien, amarillo = a medias, rojo = débil.
function SHIELD_CSS(k) {
  return k > 0.66 ? '#4aa8ff' : k > 0.33 ? '#f2c94c' : '#ef5350';
}
function SHIELD_HEX(k) {
```

```
  return k > 0.66 ? 0x4aa8ff : k > 0.33 ? 0xf2c94c : 0xef5350;
}
```

faro/src/js/02-map.js · 473 líneas

```
'use strict';
// Generación del mapa hexagonal, pasabilidad y búsqueda de caminos (A*).

class GameMap {
  constructor(seed, kind, nPlayers, pairs = false) {
    this.seed = seed;
    this.kind = kind;
    this.n = nPlayers;
    // v1.1: Dúos: las bases de cada pareja (0-1, 2-3, 4-5) quedan juntas y las parejas, separadas.
    this.pairs = pairs;
    this.R = nPlayers > 4 ? 22 : 21;
    this.cells = [];
    this.lookup = new Map();
    this.homes = [];
    this.points = [];
    this.startFarms = [];
    this.hubs = [];
    this.generate();
  }

  key(q, r) {
    return (q + 50) * 128 + (r + 50);
  }

  get(q, r) {
    const i = this.lookup.get(this.key(q, r));
    return i === undefined ? null : this.cells[i];
  }

  atWorld(x, z) {
    const h = worldToHex(x, z);
    return this.get(h.q, h.r);
  }

  dist(a, b) {
    return hexDist(a.q, a.r, b.q, b.r);
  }

  generate() {
    const rng = mulberry32(this.seed);
    const seed = this.seed;
    const R = this.R;
    const n = this.n;
    // v1.2: las variantes se generan como su mapa base y después se les cambia el terreno.
    const kind = MAPS[this.kind] && MAPS[this.kind].base ? MAPS[this.kind].base : this.kind;

    for (let q = -R; q <= R; q++) {
      for (let r = -R; r <= R; r++) {
        if (hexDist(0, 0, q, r) > R) continue;
        const w = hexToWorld(q, r);
        const cell = {
          i: this.cells.length, q, r, x: w.x, z: w.z, t: 'deep', h: 0, land: false,
          owner: -1, ownerD: 99, ownerId: 0, bld: null, nb: [], comp: -1, bridge: false,
        };
        this.lookup.set(this.key(q, r), cell.i);
        this.cells.push(cell);
      }
    }
    for (const c of this.cells) {
      for (const [dq, dr] of HEX_DIRS) {
        const nb = this.get(c.q + dq, c.r + dr);
        if (nb) c.nb.push(nb.i);
      }
    }

    // Islas: una por jugador en un anillo, una central y varias pequeñas.
    const ringHex = n > 4 ? 15 : 14;
    const ring = ringHex * SQ3;
    const a0 = rng() * Math.PI * 2;
    const blobs = [];
    const homePos = [];
    const homeXZ = [];
    if (this.pairs && n % 2 === 0) {
      // Dúos: cada pareja en su zona del anillo, separadas 20° a cada lado del centro de la zona.
      const zones = n / 2;
      for (let i = 0; i < n; i++) {
        const z = Math.floor(i / 2);
        const ang = a0 + (z * Math.PI * 2) / zones + (i % 2 ? 1 : -1) * (Math.PI / 9);
        homeXZ[i] = [Math.cos(ang) * ring, Math.sin(ang) * ring];
      }
    } else if (kind === 'reef') {
      // Arrecife Doble: dos grupos de islas, uno a cada lado, separados por agua navegable.
      const side = [[], []];
      for (let i = 0; i < n; i++) side[i % 2].push(i);
```

```
    for (let g = 0; g < 2; g++) {
      const sx = g === 0 ? -1 : 1;
      const k = side[g].length;
      side[g].forEach((pid, j) => {
        const zz = k === 1 ? 0 : (j - (k - 1) / 2) * (ring * (k === 2 ? 1.1 : 0.72));
        homeXZ[pid] = [sx * ring * 0.82, zz];
      });
      this.hubs.push({ x: sx * ring * 0.42, z: 0 });
    }
  } else {
    for (let i = 0; i < n; i++) {
      let ang = a0 + (i * Math.PI * 2) / n;
      let rad = ring;
      if (kind === 'bay') {
        // Bahía Gris: posiciones irregulares.
        ang += (rng() - 0.5) * 0.55;
        rad *= 0.86 + rng() * 0.2;
      }
      homeXZ[i] = [Math.cos(ang) * rad, Math.sin(ang) * rad];
    }
  }
  // v1.5: en Dúos el Arrecife usa el anillo de parejas, pero igual necesita sus dos islas centro.
  if (kind === 'reef' && !this.hubs.length) for (const sx of [-1, 1]) this.hubs.push({ x: sx * ring * 0.42, z: 0 });
  for (let i = 0; i < n; i++) {
    const cell = this.atWorld(homeXZ[i][0], homeXZ[i][1]);
    this.homes.push(cell.i);
    homePos.push({ x: cell.x, z: cell.z, ang: Math.atan2(cell.z, cell.x) });
    blobs.push({ x: cell.x, z: cell.z, r: 4.8 * SQ3 });
  }
  const centerR = kind === 'open' || kind === 'reef' ? 3.2 : 4.3;
  blobs.push({ x: 0, z: 0, r: centerR * SQ3 });
  const small = [];
  const addSmall = (ang, dist, rad) => {
    const c = this.atWorld(Math.cos(ang) * dist * SQ3, Math.sin(ang) * dist * SQ3);
    if (!c) return;
    small.push({ x: c.x, z: c.z, r: rad * SQ3, i: c.i });
    blobs.push(small[small.length - 1]);
  };
  if (kind === 'reef') {
    for (const h of this.hubs) {
      const c = this.atWorld(h.x, h.z);
      small.push({ x: c.x, z: c.z, r: 2.6 * SQ3, i: c.i, hub: true });
      blobs.push(small[small.length - 1]);
    }
    addSmall(Math.PI / 2, ringHex * 0.78, 1.8);
    addSmall(-Math.PI / 2, ringHex * 0.78, 1.8);
  } else if (n === 2) {
    addSmall(a0 + Math.PI / 2, 10, 2.4);
    addSmall(a0 - Math.PI / 2, 10, 2.4);
    addSmall(a0 + Math.PI / 2, 17, 1.7);
    addSmall(a0 - Math.PI / 2, 17, 1.7);
  } else {
    for (let i = 0; i < n; i++) {
      const ang = a0 + ((i + 0.5) * Math.PI * 2) / n;
      addSmall(ang, ringHex + 1, 1.9);
      if (kind !== 'continent') addSmall(ang, 8.6, 1.5);
    }
    if (kind === 'bay') {
      for (let k = 0; k < 3; k++) addSmall(rng() * Math.PI * 2, 6 + rng() * 10, 1.3 + rng() * 0.6);
    }
  }

  // Valor de tierra por casilla.
  const lakes = [];
  if (kind === 'continent') {
    for (let i = 0; i < n; i++) {
      const ang = a0 + ((i + 0.5) * Math.PI * 2) / n;
      lakes.push({ x: Math.cos(ang) * ring * 0.92, z: Math.sin(ang) * ring * 0.92, r: 2.7 * SQ3 });
    }
  }
  const bigR = (ringHex + 5.5) * SQ3;
  for (const c of this.cells) {
    const noise = (fbm(c.x * 0.17 + 100, c.z * 0.17, seed) - 0.5) * 0.6;
    let v;
    if (kind === 'continent') {
      v = 1 - Math.hypot(c.x, c.z) / bigR + noise;
      for (const l of lakes) {
        const d = Math.hypot(c.x - l.x, c.z - l.z) / l.r;
        if (d + noise * 0.8 < 1) v = -1;
      }
    } else {
      v = -1;
      for (const b of blobs) v = Math.max(v, 1 - Math.hypot(c.x - b.x, c.z - b.z) / b.r);
      v += noise;
    }
    c.land = v > 0.08;
    if (hexDist(0, 0, c.q, c.r) >= R - 1) c.land = false;
  }
```

```
    // El centro de cada isla de inicio siempre es tierra firme.
    for (const hi of this.homes) {
      const h = this.cells[hi];
      for (const c of this.cells) if (this.dist(c, h) <= 2) c.land = true;
    }
    if (kind !== 'continent') {
      for (const s of small) {
        const sc = this.cells[s.i];
        sc.land = true;
      }
      const cc = this.get(0, 0);
      for (const c of this.cells) if (this.dist(c, cc) <= 1) c.land = true;
    }

    // Puentes de arena: hacia la isla central (Archipiélago; en Bahía Gris solo algunos)
    // o hacia el centro de cada grupo (Arrecife Doble).
    const bridge = (ax, az, bx, bz) => {
      const dx = bx - ax;
      const dz = bz - az;
      const len2 = dx * dx + dz * dz || 1;
      for (const c of this.cells) {
        const t = clamp(((c.x - ax) * dx + (c.z - az) * dz) / len2, 0, 1);
        const d = Math.hypot(c.x - (ax + dx * t), c.z - (az + dz * t));
        if (d <= 0.95 && !c.land) {
          c.land = true;
          c.bridge = true;
        }
      }
    };
    if (kind === 'archipelago') for (const hp of homePos) bridge(hp.x, hp.z, 0, 0);
    if (kind === 'bay') homePos.forEach((hp, i) => { if (i % 2 === 0) bridge(hp.x, hp.z, 0, 0); });
    if (kind === 'reef') homePos.forEach((hp, i) => { const h = this.hubFor(i); bridge(hp.x, hp.z, h.x, h.z); });

    // Bahía: agua dentro del territorio inicial para poder construir un puerto.
    homePos.forEach((hp, idx) => {
      const home = this.cells[this.homes[idx]];
      const ox = Math.cos(hp.ang);
      const oz = Math.sin(hp.ang);
      let best = null;
      let bestD = Infinity;
      for (const c of this.cells) {
        if (this.dist(c, home) !== 4) continue;
        const d = Math.hypot(c.x - (home.x + ox * 4 * SQ3), c.z - (home.z + oz * 4 * SQ3));
        if (d < bestD) { bestD = d; best = c; }
      }
      if (best) {
        best.land = false;
        best.bridge = false;
        for (const ni of best.nb) {
          const nc = this.cells[ni];
          if (this.dist(nc, home) >= 4) { nc.land = false; nc.bridge = false; }
        }
      }
      hp.bay = best;
    });

    // Agua: poco profunda cerca de la costa, profunda lejos.
    const waterDist = this.bfsDistance((c) => c.land);
    for (const c of this.cells) {
      if (!c.land) c.t = waterDist[c.i] <= 2 ? 'shallow' : 'deep';
      else {
        const coastal = c.nb.some((ni) => !this.cells[ni].land);
        c.t = c.bridge || coastal ? 'sand' : 'grass';
      }
    }

    const nearHome = (c, d) => this.homes.some((hi) => this.dist(c, this.cells[hi]) <= d);
    const cc0 = this.get(0, 0);
    for (const c of this.cells) {
      if (c.t !== 'grass' || nearHome(c, 2)) continue;
      if (fbm(c.x * 0.3 + 40, c.z * 0.3 - 20, seed + 11) > 0.66 && this.dist(c, cc0) > 2) c.t = 'mount';
      else if (fbm(c.x * 0.36 - 60, c.z * 0.36 + 10, seed + 3) > 0.58) c.t = 'forest';
    }

    // Vetas de oro.
    const pick = (list, k) => {
      const out = [];
      const pool = list.slice();
      while (out.length < k && pool.length) {
        const i = Math.floor(rng() * pool.length);
        const c = pool.splice(i, 1)[0];
        if (out.some((o) => this.dist(o, c) <= 1)) continue;
        out.push(c);
      }
      return out;
    };
    for (const hi of this.homes) {
      const h = this.cells[hi];
```

```
    const cand = this.cells.filter((c) => c.land && !c.bridge && this.dist(c, h) === 3 && c.t !== 'mount');
    for (const c of pick(cand, 2)) c.t = 'gold';
  }
  if (kind !== 'continent') {
    const cand = this.cells.filter((c) => c.land && !c.bridge && this.dist(c, cc0) <= 2 && this.dist(c, cc0) > 0);
    for (const c of pick(cand, 3)) c.t = 'gold';
  } else {
    for (let i = 0; i < n; i++) {
      const ang = a0 + ((i + 0.5) * Math.PI * 2) / n;
      const c = this.atWorld(Math.cos(ang) * ring * 0.55, Math.sin(ang) * ring * 0.55);
      if (c && c.land) c.t = 'gold';
    }
    const cand = this.cells.filter((c) => c.land && this.dist(c, cc0) <= 2);
    for (const c of pick(cand, 3)) c.t = 'gold';
  }
  // Islas pequeñas: bandera en el centro y una veta de oro al lado.
  for (const s of small) {
    const sc = this.cells[s.i];
    if (!sc.land || this.points.some((pt) => this.dist(this.cells[pt.cell], sc) < 4)) continue;
    if (this.homes.some((hi) => this.dist(this.cells[hi], sc) < 6)) continue;
    sc.t = 'grass';
    this.points.push({ type: 'flag', cell: sc.i });
    const nb = sc.nb.map((ni) => this.cells[ni]).find((c) => c.land && !c.bridge);
    if (nb) nb.t = 'gold';
  }

  // Lugares de recursos: tierra fértil (Granjas) y petróleo en tierra (Bombas).
  const spot = (center, dmin, dmax, k, type, loose = false) => {
    const cand = this.cells.filter((c) => (c.t === 'grass' || (loose && (c.t === 'forest' || c.t === 'sand'))) &&
      !c.bridge && this.dist(c, center) >= dmin &&
      this.dist(c, center) <= dmax && !this.points.some((pt) => pt.cell === c.i) && c.i !== center.i);
    const got = pick(cand, k);
    for (const c of got) c.t = type;
    return got;
  };
  // Tierra fértil en zonas grandes (v0.9): dos manchas de hasta 4 casillas juntas por base.
  const patch = (center, dmin, dmax) => {
    const seeds = spot(center, dmin, dmax, 1, 'fertile');
    for (const sc of seeds) {
      const nb = sc.nb.map((ni) => this.cells[ni]).filter((c) => c.t === 'grass' && !c.bridge && c.i !== center.i &&
        !this.points.some((pt) => pt.cell === c.i) && !this.homes.includes(c.i));
      for (const c of nb.slice(0, 3)) c.t = 'fertile';
    }
  };
  for (const hi of this.homes) {
    const h = this.cells[hi];
    const first = spot(h, 2, 2, 1, 'fertile');
    this.startFarms.push(first.length ? first[0].i : -1);
    spot(h, 2, 4, 3, 'fertile');
    patch(h, 3, 4);
    patch(h, 3, 5);
    // Al menos 8 casillas fértiles a 4 de cada Núcleo (si el agua tapó una mancha, se completa al lado).
    const fert = () => this.cells.filter((c) => c.t === 'fertile' && this.dist(c, h) <= 4).length;
    for (let tries = 0; tries < 3 && fert() < 8; tries++) patch(h, 2, 4);
    if (fert() < 8) spot(h, 2, 4, 8 - fert(), 'fertile', true);
    if (!spot(h, 3, 4, 1, 'oilland').length) spot(h, 2, 4, 1, 'oilland', true);
  }
  const hubCells = small.filter((s) => s.hub).map((s) => this.cells[s.i]);
  for (const hc of hubCells) {
    spot(hc, 1, 2, 2, 'fertile');
    spot(hc, 1, 2, 1, 'oilland');
  }
  if (kind !== 'continent') {
    spot(cc0, 1, 3, 3, 'fertile');
    spot(cc0, 1, 3, 2, 'oilland');
  } else {
    for (let i = 0; i < n; i++) {
      const ang = a0 + ((i + 0.5) * Math.PI * 2) / n;
      const c = this.atWorld(Math.cos(ang) * ring * 0.45, Math.sin(ang) * ring * 0.45);
      if (c) { spot(c, 0, 2, 2, 'fertile'); spot(c, 0, 2, 1, 'oilland'); }
    }
  }

  // Manchas de petróleo junto a islas pequeñas, la isla central y fuera de cada bahía.
  const waterNear = (center, dmin, dmax) => this.cells.filter((c) => !c.land && c.t !== 'oil' &&
    this.dist(c, center) >= dmin && this.dist(c, center) <= dmax && waterDist[c.i] <= 2);
  for (const s of small) pick(waterNear(this.cells[s.i], 2, 3), 1).forEach((c) => { c.t = 'oil'; });
  if (kind !== 'continent') pick(waterNear(cc0, centerR + 1, centerR + 3), 2).forEach((c) => { c.t = 'oil'; });
  homePos.forEach((hp) => {
    if (!hp.bay) return;
    const target = this.atWorld(hp.bay.x + Math.cos(hp.ang) * 1.6 * SQ3, hp.bay.z + Math.sin(hp.ang) * 1.6 * SQ3);
    if (target && !target.land) target.t = 'oil';
  });
  for (const l of lakes) {
    const c = this.atWorld(l.x, l.z);
    if (c && !c.land) c.t = 'oil';
  }

```

```
    // Banderas: tierra fértil y petróleo cerca, para que valga la pena tomarlas.
    for (const pt of this.points) {
      const pc = this.cells[pt.cell];
      spot(pc, 1, 2, 2, 'fertile');
      spot(pc, 1, 2, 1, 'oilland');
    }

    // Continente: banderas entre las bases, a medio camino del centro.
    if (kind === 'continent') {
      for (let i = 0; i < n; i++) {
        const ang = a0 + ((i + 0.5) * Math.PI * 2) / n;
        for (const k of [0.7, 0.62, 0.78]) {
          const c = this.atWorld(Math.cos(ang) * ring * k, Math.sin(ang) * ring * k);
          if (!c || !c.land || c.t === 'gold' || c.t === 'mount') continue;
          if (this.points.some((pt) => this.dist(this.cells[pt.cell], c) < 4)) continue;
          if (this.homes.some((hi) => this.dist(this.cells[hi], c) < 6)) continue;
          c.t = 'grass';
          this.points.push({ type: 'flag', cell: c.i });
          break;
        }
      }
    }

    // El Faro ocupa el centro del mapa.
    const center = this.get(0, 0);
    center.land = true;
    center.bridge = false;
    center.t = 'grass';
    for (const ni of center.nb) {
      const c = this.cells[ni];
      if (c.t === 'mount' || c.t === 'gold' || c.t === 'fertile' || c.t === 'oilland') c.t = 'grass';
    }
    this.points.unshift({ type: 'faro', cell: center.i });

    if (this.kind === 'mesa') this.makeMesas(rng, a0, ring, n);
    if (this.kind === 'dunes') this.makeDunes();
    for (const c of this.cells) c.h = TERRAIN[c.t].h;
    this.computeComponents();

    // Garantiza los caminos a pie que el mapa promete (al centro, o al centro de cada grupo).
    if (kind === 'archipelago' || kind === 'continent' || kind === 'reef' || kind === 'bay') {
      const target = this.get(0, 0).comp;
      const bad = kind === 'reef'
        ? this.homes.some((hi, i) => this.cells[hi].comp !== this.atWorld(this.hubFor(i).x, this.hubFor(i).z).comp)
        : kind === 'bay'
          ? this.homes.some((hi, i) => i % 2 === 0 && this.cells[hi].comp !== target)
          : this.homes.some((hi) => this.cells[hi].comp !== target);
      if (bad) {
        for (const c of this.cells) if (c.t === 'mount') { c.t = 'forest'; c.h = TERRAIN.forest.h; }
        this.computeComponents();
      }
    }
  }

  // Mesetas de Cobre (v1.2): mesetas altas entre las bases y alrededor del centro.
  makeMesas(rng, a0, ring, n) {
    const free = (c) => (c.t === 'grass' || c.t === 'forest' || c.t === 'sand') && !c.bridge &&
      !this.points.some((pt) => this.dist(this.cells[pt.cell], c) <= 1) && !this.homes.some((hi) =>
this.dist(this.cells[hi], c) <= 3);
    const plateau = (x, z, rad) => {
      for (const c of this.cells) if (free(c) && Math.hypot(c.x - x, c.z - z) <= rad * SQ3) c.t = 'high';
    };
    for (let i = 0; i < n; i++) {
      const ang = a0 + ((i + 0.5) * Math.PI * 2) / n;
      plateau(Math.cos(ang) * ring * 0.55, Math.sin(ang) * ring * 0.55, 2.6);
    }
    for (let k = 0; k < 3; k++) {
      const ang = a0 + rng() * Math.PI * 2;
      plateau(Math.cos(ang) * ring * 0.25, Math.sin(ang) * ring * 0.25, 1.6);
    }
  }

  // Dunas Rojas (v1.2): lejos de las bases, el pasto y los bosques son dunas.
  makeDunes() {
    for (const c of this.cells) {
      if ((c.t === 'grass' || c.t === 'forest') && !c.bridge && !this.points.some((pt) => pt.cell === c.i) &&
        !this.homes.some((hi) => this.dist(this.cells[hi], c) <= 3)) c.t = 'dune';
    }
  }

  bfsDistance(isSource) {
    const dist = new Int32Array(this.cells.length).fill(999);
    const queue = [];
    for (const c of this.cells) if (isSource(c)) { dist[c.i] = 0; queue.push(c.i); }
    for (let h = 0; h < queue.length; h++) {
      const ci = queue[h];
      for (const ni of this.cells[ci].nb) {
        if (dist[ni] > dist[ci] + 1) { dist[ni] = dist[ci] + 1; queue.push(ni); }
```

```
      }
    }
    return dist;
  }

  // v1.5: isla centro de grupo del Arrecife para el jugador i (en Dúos, la del lado donde está su casa).
  hubFor(i) {
    if (this.pairs) return this.cells[this.homes[i]].x < 0 ? this.hubs[0] : this.hubs[1];
    return this.hubs[i % 2];
  }

  // Componentes conexas: tierra (sin montañas) y agua por separado.
  computeComponents() {
    for (const c of this.cells) c.comp = -1;
    let id = 0;
    for (const c of this.cells) {
      if (c.comp !== -1 || c.t === 'mount') continue;
      const isLand = TERRAIN[c.t].land === true;
      const stack = [c.i];
      c.comp = id;
      while (stack.length) {
        const ci = stack.pop();
        for (const ni of this.cells[ci].nb) {
          const nc = this.cells[ni];
          if (nc.comp !== -1 || nc.t === 'mount') continue;
          if ((TERRAIN[nc.t].land === true) !== isLand) continue;
          nc.comp = id;
          stack.push(ni);
        }
      }
      id++;
    }
  }

  isCoast(c) {
    return TERRAIN[c.t].water && c.nb.some((ni) => TERRAIN[this.cells[ni].t].land);
  }
}
```

faro/src/js/03-models.js · 873 líneas

```
'use strict';
// Modelos 3D de bajo polígono hechos con formas simples y fusionados en una sola
// geometría por tipo y color de jugador (menos llamadas de dibujo en el iPad).

const PAL = {
  stone: 0x8f8b82, dark: 0x3b4048, metal: 0x6e7782, wood: 0x8b5d3c, white: 0xefece4,
  olive: 0x5d6d3b, skin: 0xe0b089, yellow: 0xf1c232, field: 0xc9a54b, crop: 0x8dbf3d,
  brick: 0xb3653b, red: 0xd94141, gray: 0x8b939c, black: 0x24272c, orange: 0xf08a24,
  concrete: 0xb5b2aa, glass: 0x9fd6ee,
};

const GEO = {
  box: new THREE.BoxGeometry(1, 1, 1),
  cyl6: new THREE.CylinderGeometry(1, 1, 1, 6),
  cyl8: new THREE.CylinderGeometry(1, 1, 1, 8),
  cyl12: new THREE.CylinderGeometry(1, 1, 1, 12),
  cone4: new THREE.ConeGeometry(1, 1, 4),
  cone6: new THREE.ConeGeometry(1, 1, 6),
  cone8: new THREE.ConeGeometry(1, 1, 8),
  prism: new THREE.CylinderGeometry(1, 1, 1, 3),
  sphere: new THREE.SphereGeometry(1, 8, 6),
  half: new THREE.CylinderGeometry(1, 1, 1, 10, 1, false, 0, Math.PI),
  dodeca: new THREE.DodecahedronGeometry(1, 0),
  octa: new THREE.OctahedronGeometry(1, 0),
};

const _v = new THREE.Vector3();
const _q = new THREE.Quaternion();
const _e = new THREE.Euler();
const _s = new THREE.Vector3();

class PartBuilder {
  constructor() {
    this.parts = [];
  }

  add(geo, color, x, y, z, sx, sy, sz, rx = 0, ry = 0, rz = 0) {
    const m = new THREE.Matrix4();
    m.compose(_v.set(x, y, z), _q.setFromEuler(_e.set(rx, ry, rz)), _s.set(sx, sy, sz));
    this.parts.push({ g: geo, c: color, m });
    return this;
  }

  // Caja apoyada en y (la base queda en y).
  box(w, h, d, color, x = 0, y = 0, z = 0, ry = 0) {
    return this.add(GEO.box, color, x, y + h / 2, z, w, h, d, 0, ry, 0);
  }

  cyl(r, h, color, x = 0, y = 0, z = 0, geo = GEO.cyl8) {
    // Los prismas de 6 lados se giran 30° para alinearse con las casillas.
    const ry = geo === GEO.cyl6 ? Math.PI / 6 : 0;
    return this.add(geo, color, x, y + h / 2, z, r, h, r, 0, ry, 0);
  }

  // Tejado a dos aguas a lo largo del eje x, con la base en yBase.
  roof(len, halfDepth, height, color, x, yBase, z) {
    return this.add(GEO.prism, color, x, yBase + height * 0.5, z, halfDepth / 0.866, len, height,
      -Math.PI / 2, 0, Math.PI / 2);
  }

  build() {
    const pos = [];
    const nor = [];
    const col = [];
    const tmp = new THREE.Color();
    for (const p of this.parts) {
      const g = (p.g.index ? p.g.toNonIndexed() : p.g.clone()).applyMatrix4(p.m);
      const pa = g.attributes.position;
      const na = g.attributes.normal;
      tmp.set(p.c);
      for (let i = 0; i < pa.count; i++) {
        pos.push(pa.getX(i), pa.getY(i), pa.getZ(i));
        nor.push(na.getX(i), na.getY(i), na.getZ(i));
        col.push(tmp.r, tmp.g, tmp.b);
      }
      g.dispose();
    }
    const geo = new THREE.BufferGeometry();
    geo.setAttribute('position', new THREE.Float32BufferAttribute(pos, 3));
    geo.setAttribute('normal', new THREE.Float32BufferAttribute(nor, 3));
    geo.setAttribute('color', new THREE.Float32BufferAttribute(col, 3));
    geo.computeBoundingSphere();
    return geo;
  }
```

```
}

function shade(color, f) {
  const c = new THREE.Color(color);
  if (f >= 0) c.lerp(new THREE.Color(0xffffff), f);
  else c.multiplyScalar(1 + f);
  return c.getHex();
}

const MODEL_MAT = new THREE.MeshStandardMaterial({ vertexColors: true, flatShading: true, roughness: 0.85, metalness: 0.05 });

// Cada constructor devuelve { main, turret?, turretY?, rotor?, rotorY?, crystal? } en geometrías.
const MODEL_BUILDERS = {
  core(tc) {
    const b = new PartBuilder();
    b.cyl(0.98, 0.22, PAL.stone, 0, 0, 0, GEO.cyl6);
    b.cyl(0.78, 0.2, PAL.dark, 0, 0.22, 0, GEO.cyl6);
    b.cyl(0.36, 0.5, shade(tc, -0.25), 0, 0.42, 0, GEO.cyl6);
    for (let i = 0; i < 6; i++) {
      const a = (i / 6) * Math.PI * 2 + Math.PI / 6;
      b.box(0.16, 0.55, 0.16, i % 2 ? tc : shade(tc, 0.35), Math.cos(a) * 0.72, 0.4, Math.sin(a) * 0.72, -a);
    }
    const c = new PartBuilder();
    c.add(GEO.box, tc, 0, 0, 0, 0.5, 0.5, 0.5, Math.PI / 4, 0, Math.PI / 4);
    return { main: b.build(), crystal: c.build(), crystalY: 1.35 };
  },
  farm(tc) {
    const b = new PartBuilder();
    b.box(1.35, 0.05, 1.05, PAL.field, 0.05, 0, 0.1);
    for (let i = 0; i < 5; i++) b.box(1.2, 0.09, 0.09, PAL.crop, 0.1, 0.05, -0.25 + i * 0.17);
    b.box(0.42, 0.34, 0.4, PAL.red, -0.48, 0, -0.42);
    b.roof(0.48, 0.24, 0.2, tc, -0.48, 0.34, -0.42);
    b.cyl(0.11, 0.55, PAL.white, -0.12, 0, -0.5);
    b.add(GEO.cone8, PAL.white, -0.12, 0.62, -0.5, 0.12, 0.14, 0.12);
    return { main: b.build() };
  },
  pump(tc) {
    const b = new PartBuilder();
    b.box(1.2, 0.06, 0.95, PAL.field, 0, 0, 0.05);
    b.box(0.5, 0.08, 0.3, PAL.dark, 0.05, 0.06, -0.05);
    b.add(GEO.box, PAL.metal, 0.05, 0.42, -0.17, 0.06, 0.75, 0.06, 0.3, 0, 0);
    b.add(GEO.box, PAL.metal, 0.05, 0.42, 0.07, 0.06, 0.75, 0.06, -0.3, 0, 0);
    b.box(0.3, 0.08, 0.06, PAL.metal, 0.05, 0.74, -0.05);
    b.cyl(0.2, 0.42, tc, -0.42, 0, -0.32);
    b.cyl(0.21, 0.04, PAL.white, -0.42, 0.42, -0.32);
    b.cyl(0.05, 0.3, PAL.dark, 0.05, 0.06, 0.42);
    b.box(0.04, 0.85, 0.04, PAL.dark, 0.5, 0, -0.38);
    b.box(0.26, 0.15, 0.02, tc, 0.64, 0.66, -0.38);
    const r = new PartBuilder();
    r.box(0.08, 0.08, 0.95, PAL.yellow, 0, -0.04, 0.1);
    r.add(GEO.box, PAL.yellow, 0, -0.12, 0.6, 0.1, 0.26, 0.16, 0.35, 0, 0);
    r.box(0.14, 0.14, 0.18, PAL.dark, 0, -0.07, -0.42);
    return { main: b.build(), rocker: r.build(), rockerY: 0.82, rockerZ: -0.05 };
  },
  mill(tc) {
    const b = new PartBuilder();
    b.cyl(0.42, 0.12, PAL.stone, 0, 0, 0, GEO.cyl6);
    b.add(GEO.cyl8, PAL.white, 0, 0.62, 0, 0.26, 1.0, 0.26);
    b.add(GEO.cone8, tc, 0, 1.28, 0, 0.32, 0.32, 0.32);
    b.box(0.16, 0.24, 0.02, PAL.wood, 0, 0.12, 0.27);
    b.box(0.1, 0.1, 0.02, PAL.glass, 0, 0.62, 0.27);
    const r = new PartBuilder();
    for (let i = 0; i < 4; i++) {
      const a = (i / 4) * Math.PI * 2;
      r.add(GEO.box, PAL.wood, Math.cos(a) * 0.32, Math.sin(a) * 0.32, 0, 0.62, 0.05, 0.02, 0, 0, a);
      r.add(GEO.box, PAL.white, Math.cos(a) * 0.38, Math.sin(a) * 0.38, 0.015, 0.46, 0.14, 0.01, 0, 0, a);
    }
    r.add(GEO.cyl8, PAL.dark, 0, 0, 0, 0.06, 0.08, 0.06, Math.PI / 2, 0, 0);
    return { main: b.build(), blades: r.build(), bladesY: 1.0, bladesZ: 0.3 };
  },
  aadef(tc) {
    const b = new PartBuilder();
    b.cyl(0.5, 0.25, PAL.stone, 0, 0, 0, GEO.cyl6);
    b.cyl(0.24, 0.2, PAL.dark, 0, 0.25, 0, GEO.cyl8);
    const t = new PartBuilder();
    t.box(0.5, 0.1, 0.36, PAL.olive, 0, 0, 0);
    t.add(GEO.box, tc, 0, 0.3, 0.02, 0.48, 0.3, 0.5, -0.5, 0, 0);
    for (let i = 0; i < 3; i++) {
      for (let j = 0; j < 2; j++) {
        t.add(GEO.cyl8, PAL.white, -0.15 + i * 0.15, 0.3 + j * 0.13 - 0.04, 0.28 + j * 0.06, 0.045, 0.06, 0.045, 1.07, 0, 0);
      }
    }
    return { main: b.build(), turret: t.build(), turretY: 0.45 };
  },
  mine(tc) {
    const b = new PartBuilder();
```

```
    b.add(GEO.dodeca, PAL.stone, 0, 0.2, 0.05, 0.75, 0.5, 0.65);
    b.box(0.32, 0.3, 0.2, PAL.black, 0, 0.02, 0.55);
    b.box(0.42, 0.06, 0.06, PAL.wood, 0, 0.32, 0.62);
    b.box(0.3, 0.16, 0.22, PAL.yellow, 0.55, 0, 0.45);
    for (let i = 0; i < 4; i++) b.add(GEO.octa, 0xffd23f, -0.35 + i * 0.22, 0.45 + (i % 2) * 0.12, -0.1, 0.08, 0.11,
0.08);
    b.box(0.03, 0.7, 0.03, PAL.dark, -0.6, 0, 0.35);
    b.box(0.25, 0.15, 0.02, tc, -0.48, 0.52, 0.35);
    return { main: b.build() };
  },
  rig(tc) {
    const b = new PartBuilder();
    for (const [x, z] of [[-0.45, -0.45], [0.45, -0.45], [-0.45, 0.45], [0.45, 0.45]]) b.cyl(0.07, 0.75, PAL.yellow, x,
-0.3, z);
    b.box(1.25, 0.12, 1.25, PAL.metal, 0, 0.4, 0);
    b.add(GEO.cone4, PAL.gray, -0.2, 1.0, -0.15, 0.32, 1.1, 0.32, 0, Math.PI / 4, 0);
    b.box(0.45, 0.3, 0.35, tc, 0.32, 0.52, 0.3);
    b.box(0.5, 0.04, 0.5, PAL.orange, 0.3, 0.52, -0.35);
    b.cyl(0.04, 0.5, PAL.dark, 0.55, 0.52, -0.55);
    return { main: b.build() };
  },
  house(tc) {
    const b = new PartBuilder();
    b.box(0.62, 0.38, 0.55, PAL.white, 0, 0, 0);
    b.roof(0.7, 0.32, 0.3, tc, 0, 0.38, 0);
    b.box(0.14, 0.22, 0.02, PAL.wood, 0, 0, 0.28);
    b.box(0.12, 0.1, 0.02, PAL.glass, -0.18, 0.15, 0.28);
    b.box(0.12, 0.1, 0.02, PAL.glass, 0.18, 0.15, 0.28);
    b.box(0.32, 0.3, 0.3, shade(PAL.white, -0.08), 0.48, 0, -0.2);
    return { main: b.build() };
  },
  barracks(tc) {
    const b = new PartBuilder();
    b.add(GEO.half, PAL.olive, 0, 0, 0, 0.45, 1.15, 0.45, 0, 0, Math.PI / 2);
    b.box(0.04, 0.3, 0.26, PAL.dark, 0.58, 0, 0);
    b.box(0.04, 0.9, 0.04, PAL.dark, -0.2, 0, 0.6);
    b.box(0.32, 0.2, 0.02, tc, -0.03, 0.66, 0.6);
    for (let i = 0; i < 3; i++) b.box(0.2, 0.1, 0.12, PAL.field, -0.55 + i * 0.2, 0, 0.62);
    b.box(0.2, 0.18, 0.2, PAL.olive, -0.62, 0, -0.5);
    return { main: b.build() };
  },
  factory(tc) {
    const b = new PartBuilder();
    b.box(1.15, 0.55, 0.85, PAL.brick, 0, 0, 0);
    b.box(1.16, 0.12, 0.86, tc, 0, 0.42, 0);
    b.box(0.5, 0.36, 0.04, PAL.dark, 0.1, 0, 0.43);
    b.cyl(0.1, 0.9, PAL.gray, -0.4, 0.3, -0.25);
    b.cyl(0.1, 0.75, PAL.gray, -0.15, 0.3, -0.25);
    for (let i = 0; i < 4; i++) b.box(0.16, 0.08, 0.02, 0xffe27a, -0.4 + i * 0.27, 0.3, -0.43);
    return { main: b.build() };
  },
  harbor(tc) {
    const b = new PartBuilder();
    b.box(1.35, 0.22, 1.1, PAL.concrete, 0, -0.08, 0);
    b.box(0.75, 0.42, 0.55, PAL.dark, -0.18, 0.14, -0.18);
    b.box(0.77, 0.08, 0.57, tc, -0.18, 0.56, -0.18);
    for (let i = 0; i < 3; i++) b.box(0.14, 0.06, 0.02, 0xffe27a, -0.4 + i * 0.22, 0.38, 0.1);
    b.box(0.06, 1.0, 0.06, PAL.yellow, 0.48, 0.14, 0.3);
    b.box(0.7, 0.06, 0.06, PAL.yellow, 0.3, 1.1, 0.3);
    b.box(0.08, 0.3, 0.02, PAL.dark, 0.0, 0.84, 0.3);
    return { main: b.build() };
  },
  helipad(tc) {
    const b = new PartBuilder();
    b.cyl(0.88, 0.12, PAL.dark, 0, 0, 0, GEO.cyl12);
    b.box(0.08, 0.02, 0.5, PAL.white, -0.18, 0.12, 0);
    b.box(0.08, 0.02, 0.5, PAL.white, 0.18, 0.12, 0);
    b.box(0.36, 0.02, 0.08, PAL.white, 0, 0.12, 0);
    b.box(0.26, 0.5, 0.26, tc, 0.66, 0, -0.55);
    b.box(0.3, 0.12, 0.3, PAL.glass, 0.66, 0.5, -0.55);
    return { main: b.build() };
  },
  turret(tc) {
    const b = new PartBuilder();
    b.cyl(0.46, 0.28, PAL.stone, 0, 0, 0, GEO.cyl6);
    b.cyl(0.3, 0.22, PAL.dark, 0, 0.28, 0, GEO.cyl8);
    const t = new PartBuilder();
    t.box(0.42, 0.24, 0.42, tc, 0, 0, 0);
    t.add(GEO.cyl8, PAL.metal, -0.08, 0.12, 0.38, 0.05, 0.45, 0.05, Math.PI / 2, 0, 0);
    t.add(GEO.cyl8, PAL.metal, 0.08, 0.12, 0.38, 0.05, 0.45, 0.05, Math.PI / 2, 0, 0);
    return { main: b.build(), turret: t.build(), turretY: 0.5 };
  },
  wall(tc) {
    const b = new PartBuilder();
    b.cyl(0.9, 0.5, PAL.stone, 0, 0, 0, GEO.cyl6);
    b.cyl(0.91, 0.07, tc, 0, 0.5, 0, GEO.cyl6);
    for (let i = 0; i < 6; i++) {
      const a = (i / 6) * Math.PI * 2;
```

```
      b.box(0.22, 0.16, 0.22, shade(PAL.stone, 0.1), Math.cos(a) * 0.62, 0.56, Math.sin(a) * 0.62, -a);
    }
    return { main: b.build() };
  },
  hospital(tc) {
    const b = new PartBuilder();
    b.box(0.85, 0.4, 0.62, PAL.white, 0, 0, 0);
    b.box(0.3, 0.03, 0.09, PAL.red, 0, 0.4, 0);
    b.box(0.09, 0.03, 0.3, PAL.red, 0, 0.4, 0);
    b.box(0.2, 0.24, 0.02, PAL.dark, 0, 0, 0.32);
    b.box(0.04, 0.85, 0.04, PAL.dark, 0.58, 0, 0.4);
    b.box(0.26, 0.16, 0.02, tc, 0.72, 0.64, 0.4);
    return { main: b.build() };
  },
  outpost(tc) {
    const b = new PartBuilder();
    b.add(GEO.cone4, tc, -0.15, 0.3, 0, 0.5, 0.6, 0.5, 0, Math.PI / 4, 0);
    b.box(0.04, 1.0, 0.04, PAL.dark, 0.45, 0, -0.2);
    b.box(0.32, 0.18, 0.02, tc, 0.62, 0.78, -0.2);
    b.box(0.22, 0.2, 0.22, PAL.wood, 0.4, 0, 0.35);
    b.box(0.16, 0.14, 0.16, PAL.wood, 0.2, 0, 0.5);
    return { main: b.build() };
  },

  // --- Unidades (miran hacia +z) ---
  soldier(tc, extra) {
    const b = new PartBuilder();
    b.box(0.08, 0.13, 0.08, PAL.dark, -0.05, 0, 0);
    b.box(0.08, 0.13, 0.08, PAL.dark, 0.05, 0, 0);
    b.box(0.2, 0.2, 0.13, tc, 0, 0.13, 0);
    b.add(GEO.sphere, PAL.skin, 0, 0.4, 0, 0.075, 0.075, 0.075);
    const helmet = extra === 'engineer' ? PAL.yellow : extra === 'sniper' ? 0x4f6b2e : PAL.olive;
    b.add(GEO.sphere, helmet, 0, 0.43, 0, 0.085, 0.05, 0.085);
    if (extra === 'engineer') {
      b.box(0.14, 0.1, 0.08, PAL.orange, 0.13, 0.08, 0.05);
    } else if (extra === 'rocket') {
      b.add(GEO.cyl8, PAL.olive, 0.12, 0.33, 0.02, 0.05, 0.42, 0.05, Math.PI / 2, 0, 0);
    } else {
      const len = extra === 'sniper' ? 0.42 : 0.28;
      b.box(0.04, 0.04, len, PAL.black, 0.09, 0.22, 0.1 + len / 3);
    }
    return { main: b.build(), scale: 1.35 };
  },
  jeep(tc) {
    const b = new PartBuilder();
    for (const [x, z] of [[-0.2, -0.22], [0.2, -0.22], [-0.2, 0.22], [0.2, 0.22]]) {
      b.add(GEO.cyl8, PAL.black, x, 0.08, z, 0.08, 0.07, 0.08, 0, 0, Math.PI / 2);
    }
    b.box(0.42, 0.14, 0.7, tc, 0, 0.08, 0);
    b.box(0.38, 0.12, 0.22, PAL.glass, 0, 0.22, 0.05);
    b.box(0.06, 0.12, 0.06, PAL.dark, 0, 0.22, -0.18);
    b.box(0.04, 0.04, 0.3, PAL.black, 0, 0.33, -0.08);
    return { main: b.build(), scale: 1.15 };
  },
  tank(tc) {
    const b = new PartBuilder();
    b.box(0.14, 0.14, 0.82, PAL.black, -0.27, 0, 0);
    b.box(0.14, 0.14, 0.82, PAL.black, 0.27, 0, 0);
    b.box(0.5, 0.16, 0.78, PAL.olive, 0, 0.08, 0);
    b.box(0.52, 0.04, 0.2, tc, 0, 0.24, -0.25);
    const t = new PartBuilder();
    t.box(0.36, 0.15, 0.4, tc, 0, 0, -0.03);
    t.add(GEO.cyl8, PAL.dark, 0, 0.08, 0.38, 0.04, 0.5, 0.04, Math.PI / 2, 0, 0);
    return { main: b.build(), turret: t.build(), turretY: 0.24, scale: 1.15 };
  },
  aa(tc) {
    const b = new PartBuilder();
    for (const [x, z] of [[-0.21, -0.25], [0.21, -0.25], [-0.21, 0.25], [0.21, 0.25]]) {
      b.add(GEO.cyl8, PAL.black, x, 0.08, z, 0.08, 0.07, 0.08, 0, 0, Math.PI / 2);
    }
    b.box(0.44, 0.16, 0.8, PAL.olive, 0, 0.08, 0);
    b.box(0.4, 0.18, 0.22, tc, 0, 0.24, 0.26);
    const t = new PartBuilder();
    t.box(0.26, 0.12, 0.26, tc, 0, 0, 0);
    t.add(GEO.cyl8, PAL.dark, -0.07, 0.18, 0.14, 0.03, 0.4, 0.03, 0.9, 0, 0);
    t.add(GEO.cyl8, PAL.dark, 0.07, 0.18, 0.14, 0.03, 0.4, 0.03, 0.9, 0, 0);
    return { main: b.build(), turret: t.build(), turretY: 0.24, scale: 1.15 };
  },
  arty(tc) {
    const b = new PartBuilder();
    for (const z of [-0.28, 0, 0.28]) {
      for (const x of [-0.22, 0.22]) b.add(GEO.cyl8, PAL.black, x, 0.08, z, 0.08, 0.07, 0.08, 0, 0, Math.PI / 2);
    }
    b.box(0.44, 0.14, 0.86, PAL.olive, 0, 0.08, 0);
    b.box(0.42, 0.2, 0.22, tc, 0, 0.22, 0.32);
    b.box(0.36, 0.08, 0.14, PAL.glass, 0, 0.3, 0.38);
    const t = new PartBuilder();
    t.add(GEO.box, PAL.olive, 0, 0.12, -0.05, 0.38, 0.22, 0.5, -0.35, 0, 0);
```

```
    t.add(GEO.box, tc, 0, 0.12, 0.2, 0.39, 0.05, 0.08, -0.35, 0, 0);
    return { main: b.build(), turret: t.build(), turretY: 0.22, turretZ: -0.12, scale: 1.15 };
  },
  barge(tc) {
    const b = new PartBuilder();
    b.box(0.72, 0.16, 1.25, PAL.gray, 0, -0.02, 0);
    b.box(0.08, 0.2, 1.2, shade(PAL.gray, -0.15), -0.34, 0.12, 0);
    b.box(0.08, 0.2, 1.2, shade(PAL.gray, -0.15), 0.34, 0.12, 0);
    b.box(0.64, 0.06, 0.12, PAL.dark, 0, 0.12, 0.6);
    b.box(0.3, 0.26, 0.24, tc, 0.18, 0.12, -0.48);
    b.box(0.2, 0.08, 0.14, PAL.glass, 0.18, 0.3, -0.42);
    b.cyl(0.035, 0.32, PAL.dark, -0.2, 0.12, -0.52);
    return { main: b.build(), scale: 1.2 };
  },
  speedboat(tc) {
    const b = new PartBuilder();
    b.box(0.38, 0.14, 0.75, PAL.white, 0, 0, -0.05);
    b.add(GEO.cone4, PAL.white, 0, 0.07, 0.48, 0.27, 0.36, 0.1, Math.PI / 2, 0, 0);
    b.box(0.4, 0.04, 0.6, tc, 0, 0.1, -0.05);
    b.box(0.22, 0.12, 0.18, PAL.glass, 0, 0.14, 0.05);
    b.box(0.04, 0.04, 0.26, PAL.black, 0, 0.26, 0.15);
    return { main: b.build(), scale: 1.2 };
  },
  boatEng(tc) {
    const b = new PartBuilder();
    b.box(0.5, 0.18, 0.85, PAL.orange, 0, 0, -0.05);
    b.add(GEO.cone4, PAL.orange, 0, 0.09, 0.5, 0.36, 0.3, 0.12, Math.PI / 2, 0, 0);
    b.box(0.3, 0.22, 0.25, tc, 0, 0.18, -0.25);
    b.box(0.05, 0.45, 0.05, PAL.yellow, 0.12, 0.18, 0.18);
    b.box(0.05, 0.05, 0.4, PAL.yellow, 0.12, 0.6, 0.3);
    return { main: b.build(), scale: 1.2 };
  },
  destroyer(tc) {
    const b = new PartBuilder();
    b.box(0.5, 0.2, 1.4, PAL.gray, 0, 0, -0.1);
    b.add(GEO.cone4, PAL.gray, 0, 0.1, 0.78, 0.36, 0.45, 0.14, Math.PI / 2, 0, 0);
    b.box(0.52, 0.05, 1.2, tc, 0, 0.15, -0.15);
    b.box(0.3, 0.26, 0.42, shade(PAL.gray, 0.15), 0, 0.2, -0.3);
    b.box(0.2, 0.18, 0.2, shade(PAL.gray, 0.25), 0, 0.46, -0.32);
    b.cyl(0.07, 0.3, PAL.dark, 0, 0.46, -0.55);
    const t = new PartBuilder();
    t.box(0.24, 0.12, 0.26, shade(PAL.gray, -0.1), 0, 0, 0);
    t.add(GEO.cyl8, PAL.dark, 0, 0.06, 0.26, 0.035, 0.36, 0.035, Math.PI / 2, 0, 0);
    return { main: b.build(), turret: t.build(), turretY: 0.2, turretZ: 0.35, scale: 1.25 };
  },
  sub(tc) {
    const b = new PartBuilder();
    b.add(GEO.cyl8, PAL.black, 0, 0, 0, 0.16, 1.0, 0.16, Math.PI / 2, 0, 0);
    b.add(GEO.sphere, PAL.black, 0, 0, 0.5, 0.16, 0.16, 0.16);
    b.add(GEO.sphere, PAL.black, 0, 0, -0.5, 0.16, 0.16, 0.2);
    b.box(0.12, 0.2, 0.26, tc, 0, 0.1, 0.05);
    b.box(0.3, 0.02, 0.08, PAL.dark, 0, 0.0, -0.5);
    return { main: b.build(), scale: 1.2 };
  },
  heli(tc) {
    const b = new PartBuilder();
    b.add(GEO.sphere, tc, 0, 0.18, 0.05, 0.22, 0.2, 0.36);
    b.add(GEO.sphere, PAL.glass, 0, 0.22, 0.28, 0.15, 0.12, 0.14);
    b.box(0.07, 0.07, 0.6, shade(tc, -0.2), 0, 0.2, -0.45);
    b.box(0.03, 0.18, 0.12, shade(tc, -0.2), 0, 0.26, -0.72);
    b.box(0.03, 0.03, 0.5, PAL.dark, -0.16, -0.02, 0.05);
    b.box(0.03, 0.03, 0.5, PAL.dark, 0.16, -0.02, 0.05);
    b.box(0.5, 0.05, 0.06, PAL.dark, 0, 0.08, 0.05);
    b.box(0.06, 0.05, 0.12, PAL.olive, -0.26, 0.08, 0.05);
    b.box(0.06, 0.05, 0.12, PAL.olive, 0.26, 0.08, 0.05);
    const r = new PartBuilder();
    r.box(1.3, 0.02, 0.07, PAL.dark, 0, 0, 0);
    r.box(0.07, 0.02, 1.3, PAL.dark, 0, 0, 0);
    r.cyl(0.04, 0.08, PAL.dark, 0, -0.06, 0);
    return { main: b.build(), rotor: r.build(), rotorY: 0.42, scale: 1.1 };
  },
};

const UNIT_MODEL = {
  engineer: (tc) => MODEL_BUILDERS.soldier(tc, 'engineer'),
  rifle: (tc) => MODEL_BUILDERS.soldier(tc, 'rifle'),
  sniper: (tc) => MODEL_BUILDERS.soldier(tc, 'sniper'),
  rocket: (tc) => MODEL_BUILDERS.soldier(tc, 'rocket'),
  jeep: (tc) => MODEL_BUILDERS.jeep(tc),
  tank: (tc) => MODEL_BUILDERS.tank(tc),
  aa: (tc) => MODEL_BUILDERS.aa(tc),
  speedboat: (tc) => MODEL_BUILDERS.speedboat(tc),
  boatEng: (tc) => MODEL_BUILDERS.boatEng(tc),
  destroyer: (tc) => MODEL_BUILDERS.destroyer(tc),
  sub: (tc) => MODEL_BUILDERS.sub(tc),
  heli: (tc) => MODEL_BUILDERS.heli(tc),
  arty: (tc) => MODEL_BUILDERS.arty(tc),
  barge: (tc) => MODEL_BUILDERS.barge(tc),
```

```
};

// ---------- Modelos de Blender (GLB incrustados) ----------
// Cada GLB trae partes ('main', 'turret', 'rotor', 'blades', 'rocker', 'crystal'); los materiales
// EQUIPO y EQUIPO_OSC se pintan con el color del jugador.
const GLB = {};
const GLB_MAT = new THREE.MeshStandardMaterial({ vertexColors: true, roughness: 0.72, metalness: 0.06 });
// v0.9 «Más fluido»: materiales Lambert, mucho más livianos para la tarjeta gráfica del iPad.
const GLB_MAT_FAST = new THREE.MeshLambertMaterial({ vertexColors: true });
const MODEL_MAT_FAST = new THREE.MeshLambertMaterial({ vertexColors: true });
let FAST_MATS = false;
function fastMat(mat, fast) {
  if (fast) return mat === GLB_MAT ? GLB_MAT_FAST : mat === MODEL_MAT ? MODEL_MAT_FAST : mat;
  return mat === GLB_MAT_FAST ? GLB_MAT : mat === MODEL_MAT_FAST ? MODEL_MAT : mat;
}
const PART_NAMES = ['main', 'turret', 'rotor', 'blades', 'rocker', 'crystal', 'legl', 'legr', 'shinl', 'shinr'];
// Caminar y correr de CMU (v0.8): ángulos de cadera y rodilla, inclinación y rebote por ciclo.
const CMU_GAIT = typeof FARO_ANIM !== 'undefined' && FARO_ANIM && FARO_ANIM.walk && FARO_ANIM.run ? FARO_ANIM : null;

function _b64ToBuffer(b64) {
  const bin = atob(b64);
  const out = new Uint8Array(bin.length);
  for (let i = 0; i < bin.length; i++) out[i] = bin.charCodeAt(i);
  return out.buffer;
}

function _readGLB(name, gltf) {
  const model = { parts: {} };
  gltf.scene.updateMatrixWorld(true);
  gltf.scene.traverse((node) => {
    if (!node.isMesh) return;
    // El objeto de Blender se llama como la parte; los meshes con varios materiales cuelgan de él.
    let holder = node;
    while (holder && !PART_NAMES.includes(holder.name.replace(/[._]\d+$/, '')) && holder.parent) holder = holder.parent;
    const part = PART_NAMES.includes((holder.name || '').replace(/[._]\d+$/, '')) ? holder.name.replace(/[._]\d+$/, '')
: 'main';
    const g = node.geometry.index ? node.geometry.toNonIndexed() : node.geometry.clone();
    // Geometría relativa al pivote de la parte.
    const rel = new THREE.Matrix4().copy(holder.matrixWorld).invert().multiply(node.matrixWorld);
    g.applyMatrix4(rel);
    const matName = (node.material && node.material.name) || '';
    const team = matName.startsWith('EQUIPO_OSC') ? 2 : matName.startsWith('EQUIPO') ? 1 : 0;
    const base = node.material && node.material.color ? node.material.color.clone() : new THREE.Color(0xaaaaaa);
    const entry = model.parts[part] || (model.parts[part] = { chunks: [], pos: holder.position.clone() });
    entry.chunks.push({ g, team, base });
  });
  GLB[name] = model;
}

// Carga todos los modelos. Si falta el cargador o algo falla, el juego usa los modelos de código.
function loadGLBModels() {
  if (typeof FARO_GLB === 'undefined' || !THREE.GLTFLoader) return Promise.resolve(0);
  const loader = new THREE.GLTFLoader();
  const names = Object.keys(FARO_GLB);
  return Promise.all(names.map((name) => new Promise((resolve) => {
    try {
      loader.parse(_b64ToBuffer(FARO_GLB[name]), '', (gltf) => {
        try { _readGLB(name, gltf); } catch (err) { console.warn('[Modelos] No se pudo leer', name, err); }
        resolve();
      }, (err) => {
        console.warn('[Modelos] Error en', name, err);
        resolve();
      });
    } catch (err) {
      console.warn('[Modelos] Error en', name, err);
      resolve();
    }
  }))).then(() => Object.keys(GLB).length);
}

// Luz de ambiente (IBL) a partir del cielo de Poly Haven. Se calcula una vez por WebGLRenderer.
let ENV_MAP = null;
function loadEnvMap(gl) {
  if (typeof FARO_ENV === 'undefined' || !FARO_ENV || !gl) return Promise.resolve(null);
  return new Promise((resolve) => {
    new THREE.TextureLoader().load(FARO_ENV, (tex) => {
      try {
        tex.mapping = THREE.EquirectangularReflectionMapping;
        tex.encoding = THREE.sRGBEncoding;
        const pmrem = new THREE.PMREMGenerator(gl);
        ENV_MAP = pmrem.fromEquirectangular(tex).texture;
        pmrem.dispose();
        tex.dispose();
        GLB_MAT.envMapIntensity = 0.55;
        resolve(ENV_MAP);
      } catch (err) {
        console.warn('[Modelos] Sin luz de ambiente:', err);
        resolve(null);
```

```
      }
    }, undefined, () => resolve(null));
  });
}

function glbKey(type) {
  if (GLB[type]) return type;
  if (GLB[type + '_1']) return type + '_1';
  return null;
}

// Aspectos de la Tienda: solo cambian los colores del modelo (vida, daño y velocidad iguales).
const SKIN_LOOKS = {
  selva: { name: 'Selva', tints: [0x4e6b34, 0x6b5a3a, 0x3d5530], mix: 0.55 },
  artico: { name: 'Ártico', tints: [0xeef4f7, 0xc9dde8], mix: 0.62, teamDark: 0xdfe9ef },
  castillo: { name: 'Castillo', tints: [0x8c7b63, 0x6e6252, 0xa39478], mix: 0.6 },
  neon: { name: 'Neón', tints: [0x1b1830, 0x231d3d], mix: 0.8, glow: true },
  dorado: { name: 'Dorado', tints: [0xf2c14e, 0xd9a638], mix: 0.72, teamDark: 0xb8862a },
  // v1.2: colecciones nuevas.
  frente: { name: 'Frente 1940', tints: [0x5b5a3a, 0x8a7f55, 0x5a4632], mix: 0.62 },
  marea: { name: 'Marea', tints: [0x2a6f9e, 0xf08a7a, 0x7fd0d8], mix: 0.58 },
  // Brigada Tóxica: cambia la forma (capucha y máscara de gas), no solo el color.
  toxico: { name: 'Tóxico', tints: [], mix: 0, geo: true },
};

function _skinColor(base, look, i) {
  const L = SKIN_LOOKS[look];
  if (!L || !L.tints.length) return base;
  return base.clone().lerp(new THREE.Color(L.tints[i % L.tints.length]), L.mix);
}

function _buildPart(entry, color, look) {
  const pos = [];
  const nor = [];
  const col = [];
  const L = SKIN_LOOKS[look];
  const team = new THREE.Color(color);
  const teamDark = new THREE.Color(shade(color, -0.3));
  if (L && L.glow) {
    team.lerp(new THREE.Color(0xffffff), 0.25);
    teamDark.copy(team);
  } else if (L && L.teamDark) teamDark.set(L.teamDark);
  entry.chunks.forEach((ch, idx) => {
    const pa = ch.g.attributes.position;
    const na = ch.g.attributes.normal;
    const c = ch.team === 1 ? team : ch.team === 2 ? teamDark : L ? _skinColor(ch.base, look, idx) : ch.base;
    for (let i = 0; i < pa.count; i++) {
      pos.push(pa.getX(i), pa.getY(i), pa.getZ(i));
      if (na) nor.push(na.getX(i), na.getY(i), na.getZ(i));
      col.push(c.r, c.g, c.b);
    }
  });
  const geo = new THREE.BufferGeometry();
  geo.setAttribute('position', new THREE.Float32BufferAttribute(pos, 3));
  if (nor.length) geo.setAttribute('normal', new THREE.Float32BufferAttribute(nor, 3));
  else geo.computeVertexNormals();
  geo.setAttribute('color', new THREE.Float32BufferAttribute(col, 3));
  geo.computeBoundingSphere();
  return geo;
}

const _modelCache = new Map();
function getModel(type, color, look) {
  const key = type + ':' + color + ':' + (look || '');
  let m = _modelCache.get(key);
  if (m) return m;
  // Algunos aspectos traen su propio modelo (por ejemplo «rifle@toxico»).
  const gk = look && GLB[type + '@' + look] ? type + '@' + look : glbKey(type);
  if (gk) {
    const src = GLB[gk];
    m = { glb: true, partPos: {} };
    for (const [part, entry] of Object.entries(src.parts)) {
      m[part] = _buildPart(entry, color, look);
      m.partPos[part] = entry.pos;
    }
  } else {
    const base = type.replace(/_\d+$/, '');
    const fn = UNIT_MODEL[type] || MODEL_BUILDERS[type] || UNIT_MODEL[base] || MODEL_BUILDERS[base];
    m = fn ? fn(color) : MODEL_BUILDERS.house(color);
  }
  _modelCache.set(key, m);
  return m;
}

function _partMesh(geo, mat, out, group, name, pos) {
  const mesh = new THREE.Mesh(geo, mat);
  mesh.castShadow = true;
  if (pos) mesh.position.copy(pos);
```

```
  group.add(mesh);
  out[name] = mesh;
  return mesh;
}

// Crea el grupo 3D de una entidad. Devuelve { group, main, turret, rotor, blades, rocker, crystal }.
function createVisual(type, color, look) {
  const m = getModel(type, color, look);
  const group = new THREE.Group();
  const out = { group, main: null, turret: null, rotor: null, crystal: null, rocker: null, blades: null, type };
  const mat = fastMat(m.glb ? GLB_MAT : MODEL_MAT, FAST_MATS);
  _partMesh(m.main, mat, out, group, 'main', null).receiveShadow = true;
  const pp = m.partPos || {};
  if (m.blades) _partMesh(m.blades, mat, out, group, 'blades', pp.blades || new THREE.Vector3(0, m.bladesY, m.bladesZ));
  if (m.rocker) _partMesh(m.rocker, mat, out, group, 'rocker', pp.rocker || new THREE.Vector3(0, m.rockerY, m.rockerZ ||
0));
  if (m.turret) {
    _partMesh(m.turret, mat, out, group, 'turret', pp.turret || new THREE.Vector3(0, m.turretY || 0, m.turretZ || 0));
    out.turretBaseY = out.turret.position.y;
  }
  if (m.rotor) _partMesh(m.rotor, mat, out, group, 'rotor', pp.rotor || new THREE.Vector3(0, m.rotorY, 0));
  // Piernas de los soldados (se mueven al caminar).
  if (m.legl) _partMesh(m.legl, mat, out, group, 'legl', pp.legl);
  if (m.legr) _partMesh(m.legr, mat, out, group, 'legr', pp.legr);
  // La pierna cuelga del muslo: gira en la rodilla y sigue a la cadera.
  if (m.shinl && out.legl) _partMesh(m.shinl, mat, out, out.legl, 'shinl', pp.shinl.clone().sub(pp.legl));
  if (m.shinr && out.legr) _partMesh(m.shinr, mat, out, out.legr, 'shinr', pp.shinr.clone().sub(pp.legr));
  if (m.crystal) {
    const cmat = new THREE.MeshStandardMaterial({
      color, emissive: color, emissiveIntensity: 0.55, flatShading: true, roughness: 0.4,
    });
    _partMesh(m.crystal, cmat, out, group, 'crystal', pp.crystal || new THREE.Vector3(0, m.crystalY, 0));
    out.crystalBaseY = out.crystal.position.y;
  }
  if (m.scale) group.scale.setScalar(m.scale);
  return out;
}

// El Faro y las Banderas: partes neutras + piezas que cambian de color según el dueño.
const NEUTRAL = 0xd9dcdf;
function createPointVisual(type) {
  const group = new THREE.Group();
  const own = new THREE.MeshStandardMaterial({ color: NEUTRAL, flatShading: true, roughness: 0.7 });
  const out = { group, own, beam: null, lamp: null };
  const b = new PartBuilder();
  if (type === 'faro') {
    b.cyl(0.95, 0.18, PAL.stone, 0, 0, 0, GEO.cyl6);
    b.cyl(0.62, 0.22, shade(PAL.stone, 0.12), 0, 0.18, 0, GEO.cyl6);
    const levels = 5;
    for (let i = 0; i < levels; i++) {
      const r0 = 0.42 - i * 0.035;
      b.add(GEO.cyl12, i % 2 ? 0xc0392b : PAL.white, 0, 0.4 + i * 0.36 + 0.18, 0, r0, 0.36, r0);
    }
    b.cyl(0.34, 0.06, PAL.dark, 0, 2.2, 0, GEO.cyl12);
    for (let i = 0; i < 8; i++) {
      const a = (i / 8) * Math.PI * 2;
      b.box(0.03, 0.34, 0.03, PAL.dark, Math.cos(a) * 0.24, 2.26, Math.sin(a) * 0.24);
    }
    b.add(GEO.cone8, PAL.dark, 0, 2.78, 0, 0.3, 0.32, 0.3);
    b.box(0.2, 0.3, 0.02, PAL.wood, 0, 0.4, 0.44);
    const main = new THREE.Mesh(b.build(), fastMat(MODEL_MAT, FAST_MATS));
    main.castShadow = true;
    group.add(main);
    const lamp = new THREE.Mesh(new THREE.SphereGeometry(0.17, 12, 10),
      new THREE.MeshStandardMaterial({ color: 0xffc35a, emissive: 0xffa21f, emissiveIntensity: 1.4 }));
    lamp.position.y = 2.42;
    group.add(lamp);
    const beamGeo = new THREE.ConeGeometry(0.7, 5, 16, 1, true);
    beamGeo.translate(0, -2.5, 0);
    beamGeo.rotateZ(Math.PI / 2);
    const beam = new THREE.Mesh(beamGeo, new THREE.MeshBasicMaterial({
      color: 0xffc35a, transparent: true, opacity: 0.22, depthWrite: false, side: THREE.DoubleSide,
      blending: THREE.AdditiveBlending,
    }));
    const pivot = new THREE.Group();
    pivot.position.y = 2.42;
    pivot.add(beam);
    group.add(pivot);
    const band = new THREE.Mesh(new THREE.CylinderGeometry(0.64, 0.64, 0.08, 6, 1, false, Math.PI / 6), own);
    band.position.y = 0.42;
    group.add(band);
    out.beam = pivot;
    out.lamp = lamp;
  } else {
    b.cyl(0.32, 0.1, PAL.stone, 0, 0, 0, GEO.cyl6);
    b.cyl(0.035, 1.25, PAL.dark, 0, 0.1, 0);
    b.add(GEO.sphere, PAL.yellow, 0, 1.38, 0, 0.05, 0.05, 0.05);
    const main = new THREE.Mesh(b.build(), fastMat(MODEL_MAT, FAST_MATS));
```

```
    main.castShadow = true;
    group.add(main);
    const cloth = new THREE.Mesh(new THREE.BoxGeometry(0.5, 0.3, 0.02), own);
    cloth.position.set(0.27, 1.16, 0);
    cloth.castShadow = true;
    group.add(cloth);
    out.cloth = cloth;
  }
  return out;
}

// Miniaturas para los menús: se renderizan una vez con un renderizador pequeño.
const Thumbs = {
  cache: new Map(),
  renderer: null,
  scene: null,
  camera: null,

  init() {
    try {
      const canvas = document.createElement('canvas');
      this.renderer = new THREE.WebGLRenderer({ canvas, antialias: true, alpha: true, preserveDrawingBuffer: true });
      this.renderer.setPixelRatio(1);
      this.renderer.setSize(112, 112);
      this.scene = new THREE.Scene();
      this.scene.add(new THREE.HemisphereLight(0xffffff, 0x445566, 0.9));
      const sun = new THREE.DirectionalLight(0xffffff, 0.8);
      sun.position.set(3, 5, 4);
      this.scene.add(sun);
      this.camera = new THREE.PerspectiveCamera(30, 1, 0.1, 50);
    } catch (err) {
      console.warn('[Thumbs] Sin miniaturas:', err);
      this.renderer = null;
    }
  },

  get(type, color, look) {
    const key = type + ':' + color + ':' + (look || '');
    if (this.cache.has(key)) return this.cache.get(key);
    let url = '';
    if (this.renderer) {
      const isPoint = type === 'faro' || type === 'flag';
      const v = isPoint ? createPointVisual(type) : createVisual(type, color, look);
      if (isPoint) {
        v.own.color.set(color);
        if (v.beam) v.beam.visible = false;
      }
      v.group.scale.setScalar(1);
      this.scene.add(v.group);
      const box = new THREE.Box3().setFromObject(v.group);
      const size = box.getSize(new THREE.Vector3()).length();
      const center = box.getCenter(new THREE.Vector3());
      const d = size * 1.75;
      this.camera.position.set(center.x + d * 0.62, center.y + d * 0.55, center.z + d * 0.62);
      this.camera.lookAt(center);
      this.renderer.render(this.scene, this.camera);
      url = this.renderer.domElement.toDataURL('image/png');
      this.scene.remove(v.group);
    }
    this.cache.set(key, url);
    return url;
  },

  // Portada (v0.9): una escena con los modelos del juego, dibujada una sola vez como imagen.
  coverUrl: '',
  cover(w = 1400, h = 440) {
    if (this.coverUrl || !this.renderer) return this.coverUrl;
    try {
      const r = this.renderer;
      const scene = new THREE.Scene();
      const sky = document.createElement('canvas');
      sky.width = 4;
      sky.height = 256;
      const sctx = sky.getContext('2d');
      const grad = sctx.createLinearGradient(0, 0, 0, 256);
      grad.addColorStop(0, '#16283a');
      grad.addColorStop(0.55, '#d9874a');
      grad.addColorStop(1, '#f6c46a');
      sctx.fillStyle = grad;
      sctx.fillRect(0, 0, 4, 256);
      scene.background = new THREE.CanvasTexture(sky);
      scene.fog = new THREE.Fog(0xe0a060, 12, 30);
      if (ENV_MAP) scene.environment = ENV_MAP;
      scene.add(new THREE.HemisphereLight(0xffe2b8, 0x2d4a5a, 0.75));
      const sun = new THREE.DirectionalLight(0xffc98a, 1.05);
      sun.position.set(-6, 5, 3);
      scene.add(sun);
      const water = new THREE.Mesh(new THREE.PlaneGeometry(80, 40),
```

```
        new THREE.MeshStandardMaterial({ color: 0x2b6f93, roughness: 0.25, metalness: 0.1 }));
      water.rotation.x = -Math.PI / 2;
      water.position.y = -0.12;
      scene.add(water);
      // Isla de casillas hexagonales.
      const hexGeo = new THREE.CylinderGeometry(0.98, 0.98, 0.3, 6);
      const grass = new THREE.MeshStandardMaterial({ color: 0x6f9c55, roughness: 0.95, flatShading: true });
      const sand = new THREE.MeshStandardMaterial({ color: 0xd8c38a, roughness: 0.95, flatShading: true });
      const rock = new THREE.MeshStandardMaterial({ color: 0x8b8f87, roughness: 0.95, flatShading: true });
      for (let q = -6; q <= 14; q++) {
        for (let rr = -5; rr <= 4; rr++) {
          const x = q * 1.5;
          const z = rr * 1.732 + (q & 1) * 0.866;
          const d = Math.hypot((x - 4.5) * 0.48, z);
          if (d > 5.6 || z > 3.6) continue;
          const edge = d > 4.6 || z > 2.6;
          const hill = q >= 6 && q <= 7 && rr <= -1;
          const t = new THREE.Mesh(hexGeo, edge ? sand : hill ? rock : grass);
          t.position.set(x, hill ? 0.18 : 0, z);
          scene.add(t);
        }
      }
      const blue = PLAYER_COLORS[0];
      const red = PLAYER_COLORS[1];
      const put = (type, x, y, z, yaw = 0, s = 1, color = blue, look) => {
        const v = createVisual(type, color, look);
        v.group.position.set(x, y, z);
        v.group.rotation.y = yaw;
        if (s !== 1) v.group.scale.multiplyScalar(s);
        scene.add(v.group);
        return v;
      };
      const faro = createPointVisual('faro');
      faro.own.color.set(0xf2ab38);
      faro.group.position.set(9.9, 0.33, -3.4);
      faro.group.scale.setScalar(0.85);
      scene.add(faro.group);
      put('core_3', 0.8, 0.15, -2.4, 0.3);
      put('aadef_4', 3.0, 0.15, -3.2, 0.8);
      put('turret_4', 1.8, 0.15, 0.4, 1.0);
      put('apc', 3.2, 0.15, -0.6, 1.2, 1.15);
      put('heavytank', 4.4, 0.15, 0.9, 1.15, 1.15);
      put('tank', 6.0, 0.15, -0.2, 1.25, 1.15);
      put('launcher', 7.6, 0.33, -2.2, 1.3, 1.1);
      ['rifle', 'rocket', 'rifle', 'medic', 'sniper', 'rifle'].forEach((k, i) =>
        put(k, 4.6 + i * 0.6, 0.15, 2.2 - (i % 2) * 0.4, 1.4, 1.5));
      const heli = put('heli', 3.6, 1.8, 0.8, 1.3, 1.2);
      if (heli.rotor) heli.rotor.rotation.y = 0.5;
      const apache = put('apache', 6.8, 2.2, -0.6, 1.25, 1.2);
      if (apache.rotor) apache.rotor.rotation.y = 1.1;
      const jet = put('fighter', 8.6, 2.85, 0.8, 1.9, 1.3);
      jet.group.rotation.z = -0.35;
      put('frigate', 11.2, -0.1, 2.6, -0.4, 1.2, red);
      // v1.0: Bombardero y segundo Caza en vuelo con estela, y el Portaaviones en el mar.
      const bomb = put('bomber', 4.6, 3.35, -2.2, 1.75, 1.3);
      bomb.group.rotation.z = 0.12;
      const jet2 = put('fighter', 1.8, 2.9, -0.2, 1.95, 1.1);
      jet2.group.rotation.z = 0.3;
      put('carrier', 13.6, -0.1, -0.4, 1.2, 1.25);
      const trailMat = new THREE.MeshBasicMaterial({ color: 0xffffff, transparent: true, opacity: 0.55, depthWrite:
false });
      const trailGeo = new THREE.BoxGeometry(0.05, 0.05, 1);
      const trail = (v, len) => {
        const yaw = v.group.rotation.y;
        const m = new THREE.Mesh(trailGeo, trailMat);
        m.scale.z = len;
        m.rotation.y = yaw;
        m.position.set(v.group.position.x - Math.sin(yaw) * (len / 2 + 0.4), v.group.position.y + 0.18,
v.group.position.z - Math.cos(yaw) * (len / 2 + 0.4));
        scene.add(m);
      };
      trail(jet, 4);
      trail(jet2, 3);
      trail(bomb, 3.5);
      const cam = new THREE.PerspectiveCamera(30, w / h, 0.1, 80);
      cam.position.set(5.2, 3.4, 10.4);
      cam.lookAt(5.6, 1.3, 0);
      r.setSize(w, h);
      r.render(scene, cam);
      this.coverUrl = r.domElement.toDataURL('image/jpeg', 0.86);
      hexGeo.dispose();
      [grass, sand, rock, water.material, trailMat].forEach((m) => m.dispose());
      trailGeo.dispose();
      water.geometry.dispose();
      scene.background.dispose();
      r.setSize(112, 112);
    } catch (err) {
      console.warn('[Thumbs] Sin portada:', err);
```

```
      this.coverUrl = '';
    }
    return this.coverUrl;
  },
};
```

faro/src/js/04-world.js · 2,630 líneas

```
'use strict';
// Estado y simulación de la partida: economía, construcción, producción,
// movimiento, combate, territorio y condiciones de victoria.

let _nextId = 1;

// Partida personalizada: jugadores 2-6, equipos opcionales, monedas iniciales y reglas.
function customMode(c) {
  const n = clamp(c.players || 4, 2, 6);
  const teams = [];
  for (let i = 0; i < n; i++) teams.push(c.teams ? (i < Math.ceil(n / 2) ? 0 : 1) : i);
  return {
    name: 'Personalizado', players: n, teams, coins: c.coins || START_COINS, energy: c.energy || START_ENERGY,
    fog: !!c.fog, speed: c.turbo ? 2 : 1, king: !!c.king, custom: true,
  };
}

class Game {
  constructor(opts) {
    this.opts = opts;
    const mode = opts.mission ? Campaign.mode(opts.mission) : opts.custom ? customMode(opts.custom) : MODES[opts.mode];
    this.mode = mode;
    this.diff = DIFFICULTY[opts.difficulty];
    this.map = new GameMap(opts.seed, opts.mapKind, mode.players, !!mode.duos);
    this.cells = this.map.cells;
    this.speed = mode.speed || (opts.turbo ? 2 : 1);
    this.fog = !!(opts.fog || mode.fog);
    this.creator = !!opts.creator;
    // v1.4: regla especial del desafío semanal ('nofactory', 'noair'...).
    this.rule = opts.rule || null;
    this.elimOrder = [];
    this._assignT = 0;
    this.time = 0;
    this.over = false;
    this.result = null;
    this.units = [];
    this.buildings = [];
    this.projectiles = [];
    // Zonas de radiación del misil nuclear (v1.0).
    this.radZones = [];
    // Historia de la partida para las gráficas del final (v1.1): cada 10 s, monedas/min y ejército.
    this.history = [];
    this._histT = 0;
    this.removed = [];
    this.fx = [];
    this.events = [];
    this.territoryDirty = true;
    this.territoryVersion = 0;
    this.objDone = {};
    this._coreWarn = -99;
    this._slowT = 0;
    this._fogT = 0;

    this.kingTime = {};
    this.players = [];
    for (let i = 0; i < mode.players; i++) {
      this.players.push({
        id: i, team: mode.teams[i], team0: mode.teams[i], name: PLAYER_NAMES[i], color: PLAYER_COLORS[i],
        coins: (i === 0 && opts.startCoins) || mode.coins || START_COINS, energy: mode.energy || START_ENERGY,
energyRate: 0,
        human: i === 0, alive: true, income: 0, popUsed: 0, popCap: 0, coresKilled: 0,
        emergencyUsed: false, dome: null,
        stats: { built: 0, trained: 0, lost: 0, kills: 0, earned: 0, captures: 0, upgrades: 0, hitBy: {} },
      });
    }
    // Puntos de captura: el Faro y las Banderas.
    this.points = this.map.points.map((pt, i) => {
      const c = this.cells[pt.cell];
      const point = {
        id: i, type: pt.type, def: POINTS[pt.type], cell: pt.cell, x: c.x, z: c.z, y: c.h, alive: true, kind: 'point',
        owner: -1, prog: 0, capTeam: -1, contested: false,
      };
      c.point = point;
      return point;
    });
    this._pointT = 0;
    this.chat = [];
    this._alertT = {};
    this.fights = [];
    this._fightCell = new Map();
    // Inicio: Núcleo, 1 Granja, 2.000 monedas y (desde la v0.6) 1 Constructor para arrancar rápido.
    for (const p of this.players) {
      const home = this.cells[this.map.homes[p.id]];
      p.core = this.addBuilding(p.id, 'core', home.i, true);
```

```
      const sf = this.map.startFarms[p.id];
      if (sf >= 0 && !this.cells[sf].bld) this.addBuilding(p.id, 'farm', sf, true);
      const spawn = this.freeCellsNear(home.i, 'land', p.id, START_UNITS.length + 1);
      START_UNITS.forEach((t, k) => { if (spawn[k] !== undefined) this.addUnit(p.id, t, spawn[k]); });
      if (this.mode.tutorial && p.id !== 0) {
        // Tutorial: un bot lento con un Núcleo débil para practicar.
        p.core.maxHp = 3000;
        p.core.hp = 3000;
        p.core._w = { ...BUILDINGS.core.weapon, dmg: 8, range: 3 };
        p.core._wLv = 1;
      }
    }
    this.recomputeTerritory();
    this.refreshEconomy();
    // Personalidades repartidas según la semilla del mapa: cada bot es distinto.
    const order = PERSONALITY_ORDER.slice();
    const rng = mulberry32(opts.seed + 77);
    for (let i = order.length - 1; i > 0; i--) {
      const j = Math.floor(rng() * (i + 1));
      [order[i], order[j]] = [order[j], order[i]];
    }
    this.bots = this.players.filter((p) => !p.human).map((p, i) => {
      p.persona = opts.personas && opts.personas[p.id] ? opts.personas[p.id] : order[i % order.length];
      return new Bot(this, p.id);
    });
    // Campaña (v0.7): preparación de la misión y seguimiento de su meta.
    this.camp = null;
    if (opts.mission) Campaign.setup(this);
  }

  // ---------- consultas básicas ----------
  isEnemy(a, b) {
    return this.players[a].team !== this.players[b].team;
  }

  // ---------- alianzas (v1.1, modos de todos contra todos) ----------
  alliancesAllowed() {
    return !!this.mode.alliances && !this.camp && !this.mode.tutorial;
  }

  // Fuerza aproximada de un imperio: vida de su ejército y de sus edificios.
  strengthOf(pid) {
    let s = 0;
    for (const u of this.units) if (u.alive && u.owner === pid && u.def.weapon) s += u.hp;
    for (const b of this.buildings) if (b.alive && b.owner === pid) s += b.hp * 0.25;
    return s;
  }

  // El bot decide si acepta la alianza; devuelve { ok, text } con su respuesta.
  proposeAlliance(from, to) {
    const A = this.players[from];
    const B = this.players[to];
    if (!this.alliancesAllowed()) return { ok: false, text: 'En este modo no hay alianzas' };
    if (!B || !B.alive || !A.alive || from === to) return { ok: false, text: 'Ese imperio ya no está' };
    if (A.team === B.team) return { ok: false, text: `${B.name} ya es tu aliado` };
    A.allyAsk = A.allyAsk || {};
    if (this.time - (A.allyAsk[to] ?? -999) < 60) return { ok: false, text: `${B.name}: «Ya te contesté. Pregúntame más tarde.»` };
    A.allyAsk[to] = this.time;
    const teams = new Set(this.players.filter((p) => p.alive).map((p) => p.team));
    if (teams.size <= 2) return this.allyNo(to, 'Solo quedamos nosotros: uno tiene que ganar.');
    if (this.time - ((B.hurtBy || {})[from] ?? -999) < 90) return this.allyNo(to, 'Hace poco me atacaste. No confío en ti.');
    // Ganas de aliarse: según su forma de ser, si tienen un rival en común más fuerte y si está en apuros.
    const base = { aggro: 0.3, admiral: 0.45, builder: 0.65, keeper: 0.55 }[B.persona] ?? 0.5;
    const sA = this.strengthOf(from);
    const sB = this.strengthOf(to);
    let score = base;
    const strongest = this.players.filter((p) => p.alive && p.team !== A.team && p.team !== B.team)
      .reduce((m, p) => Math.max(m, this.strengthOf(p.id)), 0);
    if (strongest > Math.max(sA, sB)) score += 0.25;
    if (this.time - (B.underAttackAt ?? -999) < 45) score += 0.2;
    if (sA > sB * 2.2) score -= 0.25;
    if (score < 0.6) return this.allyNo(to, B.persona === 'aggro' ? 'Yo peleo solo.' : 'Todavía no. Muéstrame que vale la pena.');
    this.formAlliance(from, to);
    this.say(to, 'Trato hecho: ahora somos aliados. ¡Vamos por los demás!', { force: true, toMe: true });
    return { ok: true, text: `${B.name} aceptó la alianza` };
  }

  allyNo(pid, text) {
    this.say(pid, text, { force: true, toMe: true });
    return { ok: false, text: `${this.players[pid].name}: «${text}»` };
  }

  // Une a «b» (y a sus aliados) al equipo de «a».
  formAlliance(a, b) {
    const ta = this.players[a].team;
```

```
    const tb = this.players[b].team;
    for (const p of this.players) if (p.team === tb) p.team = ta;
    this.allianceChanged();
    this.emit('alliance', { a, b, ok: true });
  }

  // «pid» sale de su alianza y vuelve a su equipo de siempre.
  breakAlliance(pid, by = pid) {
    const p = this.players[pid];
    if (!this.alliancesAllowed()) return 'En este modo no hay alianzas';
    const mates = this.players.filter((q) => q.id !== pid && q.team === p.team);
    if (!mates.length) return 'No tienes aliados';
    p.team = p.team0;
    // Si su equipo era el de la alianza, los demás toman el equipo del primero de ellos.
    if (mates.some((q) => q.team === p.team0)) {
      const t = mates[0].team0;
      for (const q of mates) q.team = t;
    }
    this.allianceChanged();
    this.emit('alliance', { a: pid, ok: false, by });
    return null;
  }

  allianceChanged() {
    // Nadie sigue atacando a un nuevo aliado; la vista compartida se recalcula.
    for (const u of this.units) {
      if (u.target && u.target.alive !== undefined && !this.isEnemy(u.owner, u.target.owner)) { u.target = null; if
(u.state === 'attack') u.state = 'idle'; }
      if (u.autoTarget && !this.isEnemy(u.owner, u.autoTarget.owner)) u.autoTarget = null;
    }
    for (const b of this.buildings) if (b.autoTarget && !this.isEnemy(b.owner, b.autoTarget.owner)) b.autoTarget =
null;
    this.territoryDirty = true;
    this.allianceVersion = (this.allianceVersion || 0) + 1;
  }

  emit(type, data) {
    this.events.push({ type, ...data });
  }

  toast(pid, text, kind = 'info') {
    if (pid === 0) this.emit('toast', { text, kind });
  }

  // Mensaje de un bot. Se guarda siempre; la interfaz muestra los que te afectan.
  say(pid, text, opts = {}) {
    const p = this.players[pid];
    if (!p || p.human) return false;
    if (!opts.force && this.time - (p.lastSay || -99) < 20) {
      // Guarda el mensaje más importante y lo dice en cuanto pueda.
      if (opts.queue && (!p.pendingSay || p.pendingSay.opts.queue <= opts.queue)) p.pendingSay = { text, opts };
      return false;
    }
    p.lastSay = this.time;
    const me = this.players[0];
    const msg = {
      pid, text, time: this.time, x: opts.x, z: opts.z, kind: opts.kind || 'info',
      ally: p.team === me.team, toMe: !!opts.toMe, public: !!opts.public,
    };
    this.chat.push(msg);
    if (this.chat.length > 60) this.chat.shift();
    this.emit('chat', { msg });
    return true;
  }

  // v1.2: mensaje tuyo al chat de aliados; los bots aliados contestan.
  playerSay(text) {
    const me = this.players[0];
    const msg = { pid: 0, text, time: this.time, kind: 'info', ally: true, mine: true, toMe: false, public: false };
    this.chat.push(msg);
    if (this.chat.length > 60) this.chat.shift();
    const replies = {
      '¡Gracias!': ['¡De nada!', '¡Para eso estamos!', 'Cuando quieras.'],
      '¡Bien hecho!': ['¡Gracias, compañero!', '¡Seguimos así!', '¡Juntos somos fuertes!'],
      '¡Cuidado!': ['¡Atento! Pongo defensas.', '¡Visto! Vigilo mi lado.', '¡Gracias por avisar!'],
    }[text];
    let n = 0;
    if (replies) {
      for (const p of this.players) {
        if (p.id === 0 || !p.alive || p.human || p.team !== me.team) continue;
        this.say(p.id, replies[(n + Math.floor(this.time)) % replies.length], { force: true, toMe: true });
        n++;
      }
    }
    return n;
  }

  // Alerta para el jugador con ubicación (para el botón «Ver» y el pulso del minimapa).
```

```
  alert(key, text, kind, x, z, every = 15) {
    if (this.time - (this._alertT[key] || -99) < every) return;
    this._alertT[key] = this.time;
    this.emit('toast', { text, kind, x, z });
    this.emit('ping', { x, z, kind });
  }

  passable(cell, mover, owner) {
    if (mover === 'air') return true;
    const T = TERRAIN[cell.t];
    if (mover === 'land' ? !T.land : !T.water) return false;
    const b = cell.bld;
    if (b) return b.type === 'wall' && !this.isEnemy(b.owner, owner);
    return true;
  }

  groundY(u, cell) {
    if (u.mover === 'air') return 2.3;
    if (u.mover === 'sea') return u.cls === 'sub' ? -0.1 : 0.02;
    return cell.h;
  }

  freeCellsNear(ci, mover, owner, count) {
    const c0 = this.cells[ci];
    const out = [];
    for (const off of SPIRAL) {
      if (off.d === 0 && mover !== 'air') continue;
      const c = this.map.get(c0.q + off.q, c0.r + off.r);
      if (!c || !this.passable(c, mover, owner)) continue;
      out.push(c.i);
      if (out.length >= count) break;
    }
    while (out.length < count) out.push(out[0] ?? ci);
    return out;
  }

  // ---------- A* ----------
  findPath(start, goalI, mover, owner) {
    const cells = this.cells;
    const N = cells.length;
    if (!this._g) {
      this._g = new Float32Array(N);
      this._from = new Int32Array(N);
      this._stamp = new Uint32Array(N);
      this._closed = new Uint32Array(N);
      this._run = 0;
    }
    const run = ++this._run;
    const G = this._g;
    const F = this._from;
    const S = this._stamp;
    const C = this._closed;
    const goal = cells[goalI];
    const goalOk = this.passable(goal, mover, owner);
    const heap = new MinHeap();
    S[start] = run;
    G[start] = 0;
    F[start] = -1;
    heap.push(start, 0);
    let best = start;
    let bestH = this.map.dist(cells[start], goal);
    let iter = 0;
    while (heap.size) {
      const cur = heap.pop();
      if (C[cur] === run) continue;
      C[cur] = run;
      const hc = this.map.dist(cells[cur], goal);
      if (hc < bestH) { bestH = hc; best = cur; }
      if (cur === goalI || (!goalOk && hc <= 1)) { best = cur; break; }
      if (++iter > 5000) break;
      for (const ni of cells[cur].nb) {
        if (C[ni] === run) continue;
        const nc = cells[ni];
        if (!this.passable(nc, mover, owner)) continue;
        const slow = mover === 'land' ? TERRAIN[nc.t].slow || 1 : 1;
        const ng = G[cur] + slow;
        if (S[ni] !== run || ng < G[ni]) {
          S[ni] = run;
          G[ni] = ng;
          F[ni] = cur;
          heap.push(ni, ng + this.map.dist(nc, goal) * 1.01);
        }
      }
    }
    const path = [];
    let c = best;
    while (c !== -1 && c !== start) {
      path.push(c);
```

```
      c = F[c];
    }
    path.reverse();
    return path;
  }

  // ---------- niveles ----------
  bMaxHp(def, lv) {
    return def.levels ? def.levels[lv].hp : Math.round(def.hp * lvlMult(lv));
  }

  bIncome(b) {
    const d = b.def;
    if (d.levels) return d.levels[b.level].income;
    return Math.round((d.income || 0) * lvlMult(b.level));
  }

  bEnergy(b) {
    return Math.round((b.def.energy || 0) * lvlMult(b.level));
  }

  bPop(b) {
    const d = b.def;
    if (d.levels) return d.levels[b.level].pop;
    if (d.popLevels) return d.popLevels[b.level] || 0;
    return (d.pop || 0) * b.level;
  }

  // Comunidad extra por pelear (v0.9): por tropas enemigas eliminadas y Núcleos destruidos.
  popBonus(p) {
    return Math.floor(p.stats.kills / POP_BONUS.perKills) * POP_BONUS.killPop + p.coresKilled * POP_BONUS.corePop;
  }

  // Arma según el nivel: láser del Núcleo, Air Defense más fuerte, etc.
  // Escudo propio de cada edificio (v0.9): 20 % de su vida máxima.
  resetShield(b) {
    b.shieldMax = Math.round(b.maxHp * BLD_SHIELD.pct);
    b.shield = b.shieldMax;
  }

  weaponOf(b) {
    const d = b.def;
    if (!d.weapon) return null;
    if (b._wLv === b.level && b._w) return b._w;
    let w = d.weapon;
    if (d.aaLevel && b.level >= d.aaLevel) {
      // Torreta nivel 4: también es antiaérea.
      w = { ...w, mult: { ...w.mult, ...d.aaMult } };
    }
    if (d.levels && d.levels[b.level].laser) {
      const L = d.levels[b.level].laser;
      w = { ...w, dmg: L.dmg, range: L.range, beams: L.beams };
    } else if (b.level > 1) {
      w = { ...w, dmg: w.dmg * lvlMult(b.level), range: w.range + (b.type === 'aadef' ? 1 : 0) };
    }
    b._w = w;
    b._wLv = b.level;
    return w;
  }

  healOf(b) {
    const d = b.def;
    if (!d.heal) return null;
    if (d.healLevels) return { cls: d.heal.cls, pct: d.healLevels[b.level].pct, range: d.healLevels[b.level].range };
    if (d.heal.rate) return { ...d.heal, rate: d.heal.rate * lvlMult(b.level) };
    return d.heal;
  }

  // v0.9: cada nivel entrena un 40 % más rápido.
  prodRate(b) {
    return 1 + 0.4 * (b.level - 1);
  }

  // ---------- creación de entidades ----------
  addBuilding(owner, type, ci, instant = false) {
    const def = BUILDINGS[type];
    const cell = this.cells[ci];
    const built = instant || def.build === 0;
    const hp0 = this.bMaxHp(def, 1);
    const b = {
      id: _nextId++, kind: 'bld', type, def, owner, cell: ci, x: cell.x, z: cell.z, y: cell.h, level: 1, upgrading:
null,
      hp: built ? hp0 : hp0 * 0.1, maxHp: hp0, built, progress: built ? 1 : 0, workAcc: 0, lastWork: -99,
      miniCd: [0, 0, 0], shield: 0, shieldMax: 0,
      coastal: !!TERRAIN[cell.t].water || cell.nb.some((ni) => TERRAIN[this.cells[ni].t].water),
      queue: [], rally: -1, alive: true, cd: 0, scanT: Math.random() * 0.3, autoTarget: null,
      onWater: !!TERRAIN[cell.t].water, healT: 0, aimYaw: 0, lastHit: -99, visible: true,
      rot: Math.floor(Math.random() * 6) * (Math.PI / 3),
```

```
    };
    if (type === 'core' || type === 'harbor') b.rot = 0;
    if (built) this.resetShield(b);
    cell.bld = b;
    this.buildings.push(b);
    if (def.claim) this.territoryDirty = true;
    // Aparta a las unidades que estaban sobre la casilla.
    for (const u of this.units) {
      if (!u.alive || u.cell !== ci || u.mover === 'air') continue;
      const free = this.freeCellsNear(ci, u.mover, u.owner, 1)[0];
      const fc = this.cells[free];
      u.x = fc.x + u.offX;
      u.z = fc.z + u.offZ;
      u.cell = free;
      if (u.state !== 'idle') this.setPath(u, u.dest >= 0 ? u.dest : free);
    }
    return b;
  }

  addUnit(owner, type, ci) {
    const def = UNITS[type];
    const cell = this.cells[ci];
    const a = Math.random() * Math.PI * 2;
    const rr = Math.sqrt(Math.random()) * 0.33;
    const u = {
      id: _nextId++, kind: 'unit', type, def, owner, cls: def.cls, mover: def.mover, cell: ci,
      offX: Math.cos(a) * rr, offZ: Math.sin(a) * rr, x: 0, z: 0, y: 0, hp: def.hp, maxHp: def.hp,
      alive: true, state: 'idle', path: [], pathI: 0, dest: -1, target: null, autoTarget: null,
      aggroTarget: null, cd: Math.random() * 0.5, scanT: Math.random() * 0.3, yaw: Math.random() * 6,
      aimYaw: 0, moveYaw: 0, moving: false, repathT: 0, resume: -1, foundCell: -1, role: 'home',
      lastHit: -99, visible: true, inside: null, cargo: def.capacity || def.hangar ? [] : null, boardTarget: null,
      buildTarget: null, shield: 0, shieldMax: 0,
    };
    u.x = cell.x + u.offX;
    u.z = cell.z + u.offZ;
    u.y = this.groundY(u, cell);
    this.units.push(u);
    return u;
  }

  // ---------- territorio ----------
  recomputeTerritory() {
    for (const c of this.cells) {
      c.owner = -1;
      c.ownerD = 99;
      c.ownerId = 0;
    }
    const claimers = this.buildings.filter((b) => b.alive && b.def.claim)
      .map((b) => ({ owner: b.owner, cell: b.cell, claim: b.def.claim, id: b.id }));
    for (const pt of this.points) {
      if (pt.owner >= 0) claimers.push({ owner: pt.owner, cell: pt.cell, claim: pt.def.claim, id: 1e6 + pt.id });
    }
    for (const b of claimers) {
      const bc = this.cells[b.cell];
      for (const off of SPIRAL) {
        if (off.d > b.claim) break;
        const c = this.map.get(bc.q + off.q, bc.r + off.r);
        if (!c) continue;
        if (off.d < c.ownerD || (off.d === c.ownerD && b.id < c.ownerId)) {
          c.owner = b.owner;
          c.ownerD = off.d;
          c.ownerId = b.id;
        }
      }
    }
    this.territoryDirty = false;
    this.territoryVersion++;
  }

  // ---------- economía ----------
  refreshEconomy() {
    for (const p of this.players) {
      p.income = 0;
      p.popCap = 0;
      p.popUsed = 0;
      p.energyRate = 0;
      p.bonus = 0;
    }
    for (const b of this.buildings) {
      if (!b.alive) continue;
      const p = this.players[b.owner];
      if (b.built) {
        p.income += this.bIncome(b);
        p.popCap += this.bPop(b);
        p.energyRate += this.bEnergy(b);
      }
      for (const q of b.queue) p.popUsed += UNITS[q.type].pop;
    }
```

```
    for (const pt of this.points) {
      if (pt.owner < 0) continue;
      const p = this.players[pt.owner];
      p.income += pt.def.income;
      p.bonus += pt.def.bonus;
    }
    for (const u of this.units) if (u.alive) this.players[u.owner].popUsed += u.def.pop;
    for (const p of this.players) {
      p.popCap += this.popBonus(p);
      if (this.creator && p.id === 0) p.popCap = MAX_POP;
      p.popCap = Math.min(MAX_POP, p.popCap);
      p.baseIncome = p.income;
      p.income = Math.round(p.income * (1 + p.bonus));
    }
  }

  incomeMult(p) {
    return p.human ? 1 : this.diff.income;
  }

  count(pid, type) {
    let n = 0;
    for (const b of this.buildings) if (b.alive && b.owner === pid && b.type === type) n++;
    return n;
  }

  limitFor(pid, type) {
    const def = BUILDINGS[type];
    if (type === 'house') return HOUSE_LIMIT + 2 * this.players[pid].coresKilled;
    return def.limit || 0;
  }

  placeInvalid(pid, type, ci, ignoreCost = false) {
    const def = BUILDINGS[type];
    const c = this.cells[ci];
    const p = this.players[pid];
    if (!c) return 'Casilla no válida';
    // v1.4: reglas del desafío semanal (solo para ti).
    if (pid === 0 && this.rule) {
      if (this.rule === 'nofactory' && type === 'factory') return 'Desafío: sin Fábrica';
      if (this.rule === 'noair' && (type === 'helipad' || type === 'airport')) return 'Desafío: sin Helipuerto ni
Aeropuerto';
    }
    // Las estructuras de mar se ponen en su lugar del mar aunque esté fuera de tu territorio (punto 15).
    if (def.sea) {
      if (c.owner >= 0 && c.owner !== pid && this.isEnemy(c.owner, pid)) return 'Es territorio enemigo';
    } else if (c.owner !== pid) {
      return 'Fuera de tu territorio';
    }
    if (c.point) return `Aquí está ${c.point.type === 'faro' ? 'el Faro' : 'una Bandera'}`;
    if (c.bld) return 'Casilla ocupada';
    const reserved = { gold: 'Las vetas de oro son para Minas', fertile: 'La tierra fértil es para Granjas',
      oilland: 'El petróleo en tierra es para Bombas de petróleo' };
    const nearWater = c.nb.some((ni) => TERRAIN[this.cells[ni].t].water);
    if (Array.isArray(def.on)) {
      if (!def.on.includes(c.t)) return 'Solo en ' + def.on.map((t) => TERRAIN[t].name.toLowerCase()).join(' o ');
    } else if (def.on === 'land' || def.on === 'landcoast') {
      if (!TERRAIN[c.t].land) return 'Debe ir sobre tierra';
      if (reserved[c.t]) return reserved[c.t];
      if (def.on === 'landcoast' && !nearWater) return 'Debe ir en tierra junto al agua';
    } else if (def.on === 'coast') {
      if (c.t !== 'shallow' || !this.map.isCoast(c)) return 'Debe ir en el agua, junto a la orilla';
    } else if (def.on === 'shallowcoast') {
      if (c.t !== 'shallow' || !this.map.isCoast(c)) return 'Debe ir en agua poco profunda junto a la costa';
    } else if (def.on === 'deep') {
      if (c.t !== 'deep') return 'Debe ir en mar profundo';
    }
    const limit = this.limitFor(pid, type);
    if (limit && this.count(pid, type) >= limit) return `Límite alcanzado (${limit})`;
    if (!ignoreCost && p.coins < def.cost) return 'Monedas insuficientes';
    if (!ignoreCost && def.energyCost && p.energy < def.energyCost) return `Energía insuficiente (${def.energyCost}):
construye energías (Solar, Eólica...)`;
    return null;
  }

  place(pid, type, ci, rot) {
    const err = this.placeInvalid(pid, type, ci);
    if (err) return err;
    const p = this.players[pid];
    const def = BUILDINGS[type];
    p.coins -= def.cost;
    p.energy -= def.energyCost || 0;
    const b = this.addBuilding(pid, type, ci);
    if (rot !== undefined) b.rot = rot;
    // v1.3: el edificio cuenta para las misiones cuando se termina la obra (no al colocarlo).
    if (b.built) p.stats.built++;
    this.emit('sound', { name: 'place', pid });
    if (!this.assignSite(b) && pid === 0) this.warnNoBuilder(b);
```

```
    return null;
  }

  // ---------- obras: constructores, ingenieros y barcos constructores ----------
  needsWork(b) {
    return b.alive && ((!b.built && !b.def.selfBuild && b.type !== 'outpost') || !!b.upgrading);
  }

  canBuildSite(u, b) {
    const by = b.def.builtBy || 'land';
    if (!u.def.builds) return false;
    if (by === 'engineer') return u.type === 'engineer';
    return u.def.builds === by;
  }

  // ¿Puede la unidad llegar a una casilla junto a la obra?
  reachesSite(u, b) {
    const bc = this.cells[b.cell];
    const comp = this.cells[u.cell].comp;
    if (bc.comp === comp && TERRAIN[bc.t].water === (u.mover === 'sea')) return true;
    return bc.nb.some((ni) => this.cells[ni].comp === comp);
  }

  builderFor(b, onlyIdle = true) {
    let best = null;
    let bd = Infinity;
    const bc = this.cells[b.cell];
    for (const u of this.units) {
      if (!u.alive || u.inside || u.owner !== b.owner || !this.canBuildSite(u, b)) continue;
      if (onlyIdle && u.state !== 'idle') continue;
      if (u.role === 'cap' || u.role === 'wave') continue;
      if (!this.reachesSite(u, b)) continue;
      const d = this.map.dist(this.cells[u.cell], bc);
      if (d < bd) { bd = d; best = u; }
    }
    return best;
  }

  assignSite(b) {
    const u = this.builderFor(b);
    if (!u) return false;
    this.orderBuild([u], b);
    return true;
  }

  warnNoBuilder(b) {
    const by = b.def.builtBy || 'land';
    const has = this.units.some((u) => u.alive && u.owner === b.owner && this.canBuildSite(u, b));
    const who = by === 'sea' ? 'un Barco constructor (en el Puerto)' : by === 'engineer' ? 'un Ingeniero (en el
Cuartel)'
      : 'un Constructor (en el Núcleo o el Cuartel)';
    const text = has ? 'Obra en espera: tu constructor está ocupado; irá al terminar.' : `Obra en espera: necesitas
${who}.`;
    this.alert('nobuilder' + by, text, 'bad', b.x, b.z, 8);
  }

  orderBuild(units, b) {
    let n = 0;
    for (const u of units) {
      if (!u.alive || u.inside || !this.canBuildSite(u, b)) continue;
      u.state = 'build';
      u.buildTarget = b;
      u.target = null;
      u.repairTarget = null;
      u.repathT = 0;
      n++;
    }
    return n;
  }

  // Cada segundo: las obras sin nadie reciben al constructor libre más cercano.
  assignBuilders() {
    const busy = new Set();
    for (const u of this.units) if (u.alive && u.state === 'build' && u.buildTarget) busy.add(u.buildTarget);
    for (const b of this.buildings) {
      if (!this.needsWork(b) || busy.has(b)) continue;
      if (this.assignSite(b)) busy.add(b);
    }
  }

  // Trabajo de un constructor junto a la obra (se aplica en el siguiente paso del edificio).
  workOn(u, b, dt) {
    b.workAcc += (u.def.buildRate || 1) * dt;
    b.lastWork = this.time;
    u.workingAt = this.time;
    u.aimYaw = Math.atan2(b.x - u.x, b.z - u.z);
    // Martilleo de obra (solo tus constructores suenan).
    if (u.owner === 0 && Math.random() < dt * 1.3) this.emit('sound', { name: 'build', x: b.x, z: b.z });
```

```
    if (Math.random() < dt * 1.5) this.fx.push({ k: 'spark', x: b.x + (Math.random() - 0.5) * 0.7, y: b.y + 0.4 + Math.random() * 0.4, z: b.z + (Math.random() - 0.5) * 0.7, s: 0.18 });
  }

  demolish(b) {
    if (!b.alive || b.type === 'core') return;
    const p = this.players[b.owner];
    const back = b.built ? 0.4 : 0.75;
    p.coins += Math.floor(b.def.cost * back);
    p.energy += Math.floor((b.def.energyCost || 0) * back);
    for (const q of b.queue) {
      p.coins += UNITS[q.type].cost;
      p.energy += UNITS[q.type].energy || 0;
    }
    b.queue = [];
    // v1.3: la mejora en curso se devuelve entera (se pagó y no llegó a terminarse).
    if (b.upgrading && b.upgrading.paid) p.coins += b.upgrading.paid;
    b.upgrading = null;
    b.alive = false;
    const c = this.cells[b.cell];
    if (c.bld === b) c.bld = null;
    if (b.def.claim) this.territoryDirty = true;
    this.fx.push({ k: 'boom', x: b.x, y: b.y + 0.3, z: b.z, s: 0.9 });
  }

  foundInvalid(pid, ci, how) {
    const c = this.cells[ci];
    if (!c || !TERRAIN[c.t].land) return 'Elige una casilla de tierra';
    if (c.t === 'gold') return 'No sobre vetas de oro: ponlo al lado';
    if (c.point) return 'Ese punto se captura con infantería';
    if (c.bld) return 'Casilla ocupada';
    if (c.owner >= 0 && this.isEnemy(c.owner, pid)) return 'Es territorio enemigo';
    if (how === 'sea' && !c.nb.some((ni) => TERRAIN[this.cells[ni].t].water)) return 'Debe estar en la costa';
    return null;
  }

  enqueue(b, type) {
    const def = UNITS[type];
    const p = this.players[b.owner];
    if (!b.built) return 'Aún en construcción';
    if (!b.def.produces || !b.def.produces.includes(type)) return 'No se produce aquí';
    if (b.queue.length >= 5) return 'Cola llena (5)';
    if (def.limit) {
      const have = this.units.filter((u) => u.alive && u.owner === b.owner && u.type === type).length +
        this.buildings.reduce((s, x) => s + (x.owner === b.owner ? x.queue.filter((q) => q.type === type).length : 0), 0);
      if (have >= def.limit) return `Límite alcanzado (${def.limit})`;
    }
    if (p.coins < def.cost) return 'Monedas insuficientes';
    if (def.energy && p.energy < def.energy) return `Energía insuficiente (${def.energy}): construye energías (Solar, Eólica...)`;
    this.refreshEconomy();
    if (p.popUsed + def.pop > p.popCap) return 'Población llena: construye o amplía Casas';
    p.coins -= def.cost;
    p.energy -= def.energy || 0;
    b.queue.push({ type, t: 0 });
    b.lastType = type;
    p.popUsed += def.pop;
    return null;
  }

  cancelQueue(b, idx) {
    const q = b.queue[idx];
    if (!q) return;
    b.queue.splice(idx, 1);
    const p = this.players[b.owner];
    p.coins += UNITS[q.type].cost;
    p.energy += UNITS[q.type].energy || 0;
  }

  // Mejoras (punto 6): todos los edificios suben a nivel 3; la Casa a 4. Las hace un constructor.
  upgradeCost(b) {
    const d = b.def;
    const lv = b.level;
    if (lv >= d.maxLevel) return 0;
    if (d.levels) return d.levels[lv + 1].cost;
    if (b.type === 'house') return HOUSE_LEVELS[lv];
    return Math.max(50, Math.round(d.cost * LEVEL_UP_COST[lv]));
  }

  upgradeInvalid(b) {
    if (!b.built) return 'Aún en construcción';
    if (b.upgrading) return 'Ya se está mejorando';
    if (b.level >= b.def.maxLevel) return b.def.maxLevel === 1 ? 'Este edificio no se mejora' : `${b.def.name}: nivel máximo`;
    if (b.def.l4Limit && b.level === 3) {
      const n = this.buildings.filter((x) => x.alive && x.owner === b.owner && x.type === b.type && (x.level >= 4 || (x.upgrading && x.upgrading.level >= 4))).length;
```

```
      if (n >= b.def.l4Limit) return `Solo ${b.def.l4Limit} ${b.def.name} pueden llegar a nivel 4`;
    }
    if (!this.creator || b.owner !== 0) {
      if (this.players[b.owner].coins < this.upgradeCost(b)) return 'Monedas insuficientes';
    }
    return null;
  }

  // v1.0: «Mejorar todo» y «Mejoras automáticas», solo con el Núcleo en nivel 3.
  massUpgradeInvalid(pid) {
    const p = this.players[pid];
    if (!p.core || !p.core.alive || p.core.level < AUTO_UP.coreLevel) return `Necesita el Núcleo en nivel ${AUTO_UP.coreLevel}`;
    return null;
  }

  // Edificios que todavía se pueden mejorar (sin contar el dinero), del más barato al más caro.
  upgradeCandidates(pid) {
    const out = [];
    for (const b of this.buildings) {
      if (!b.alive || b.owner !== pid || !b.built || b.upgrading || b.level >= (b.def.maxLevel || 1)) continue;
      const why = this.upgradeInvalid(b);
      if (why && why !== 'Monedas insuficientes') continue;
      out.push(b);
    }
    return out.sort((a, c) => this.upgradeCost(a) - this.upgradeCost(c));
  }

  // Un clic: pone a mejorar todo lo que alcanza el dinero (del más barato al más caro).
  upgradeAll(pid, reserve = 0) {
    const why = this.massUpgradeInvalid(pid);
    if (why) return why;
    const p = this.players[pid];
    let n = 0;
    let spent = 0;
    for (const b of this.upgradeCandidates(pid)) {
      const cost = this.upgradeCost(b);
      if (p.coins - cost < reserve) continue;
      if (typeof this.upgrade(b) === 'string') continue;
      n++;
      spent += cost;
    }
    return { n, spent };
  }

  // Mejoras automáticas: siguen mejorando hasta dejar todo al máximo, como mucho tantos edificios
  // a la vez como Constructores tenga el imperio, y sin bajar de la reserva.
  updateAutoUpgrade(dt) {
    this._autoUpT = (this._autoUpT || 0) - dt;
    if (this._autoUpT > 0) return;
    this._autoUpT = AUTO_UP.every;
    for (const p of this.players) {
      if (!p.alive || !p.autoUpgrade || this.massUpgradeInvalid(p.id)) continue;
      const busy = this.buildings.filter((b) => b.alive && b.owner === p.id && b.upgrading).length;
      const builders = this.units.filter((u) => u.alive && u.owner === p.id && u.def.builds).length;
      if (busy >= Math.max(1, builders)) continue;
      const b = this.upgradeCandidates(p.id).find((x) => p.coins - this.upgradeCost(x) >= AUTO_UP.reserve);
      if (b && typeof this.upgrade(b) !== 'string') {
        p.autoUps = (p.autoUps || 0) + 1;
        if (p.id === 0) this.emit('toast', { text: `Mejora automática: ${b.def.name} a nivel ${b.level + 1}`, kind: 'info', x: b.x, z: b.z });
      }
    }
  }

  upgrade(b) {
    const err = this.upgradeInvalid(b);
    if (err) return err;
    const p = this.players[b.owner];
    const paid = this.upgradeCost(b);
    p.coins -= paid;
    b.upgrading = { level: b.level + 1, progress: 0, paid };
    p.stats.upgrades++;
    this.emit('sound', { name: 'place', pid: b.owner });
    if (!this.assignSite(b) && b.owner === 0) this.warnNoBuilder(b);
    return b;
  }

  // Lo que da un edificio en su nivel actual (para la tarjeta «Qué mejoró»).
  snapshot(b) {
    const w = this.weaponOf(b);
    const heal = this.healOf(b);
    return {
      hp: b.maxHp, income: this.bIncome(b), pop: this.bPop(b), energy: this.bEnergy(b), prod: b.def.produces ? this.prodRate(b) : 0,
      dmg: w ? Math.round(w.dmg) : 0, range: w ? w.range : 0, beams: w && w.beams ? w.beams : 1,
      guns: b.def.minigun ? Math.max(0, b.level - 1) : 0, aa: !!(b.def.aaLevel && b.level >= b.def.aaLevel),
      heal: heal ? Math.round((heal.pct || 0) * 100) : 0, shield: b.shieldMax,
```

```
    };
  }

  upgradeLines(b, before, after) {
    const out = [];
    const n = (v) => fmt(Math.round(v));
    if (after.hp > before.hp) out.push(`Vida ${n(before.hp)} → ${n(after.hp)}`);
    if (after.shield > before.shield) out.push(`Escudo ${n(before.shield)} → ${n(after.shield)}`);
    if (after.income > before.income) out.push(`Monedas ${n(before.income)} → ${n(after.income)} por min`);
    if (after.pop > before.pop) out.push(`Comunidad +${after.pop - before.pop}`);
    if (after.energy > before.energy) out.push(`Energía ${before.energy} → ${after.energy} cada 10 s`);
    if (after.prod > before.prod) out.push(`Entrena ${Math.round((after.prod / before.prod - 1) * 100)} % más rápido`);
    if (after.dmg > before.dmg) out.push(`Daño ${before.dmg} → ${after.dmg}`);
    if (after.range > before.range) out.push(`Alcance ${before.range} → ${after.range}`);
    if (after.beams > before.beams) out.push('Doble láser');
    if (after.guns > before.guns) out.push(`${after.guns} minigun${after.guns > 1 ? 's' : ''}`);
    if (after.aa && !before.aa) out.push('Ahora dispara a aeronaves');
    if (after.heal > before.heal) out.push(`Cura ${after.heal} % por segundo`);
    if (b.type === 'core' && b.level === 3) out.push('Ya puedes comprar la Cúpula');
    if (b.type === 'aadef' && b.level === 4) out.push('Puede lanzar el misil nuclear');
    if (b.def.levelNames) out.unshift(`Ahora es ${b.def.levelNames[b.level]}`);
    return out;
  }

  // v0.9: qué daría la próxima mejora (se muestra antes de pagar).
  upgradePreview(b) {
    if (b.level >= (b.def.maxLevel || 1)) return [];
    const before = this.snapshot(b);
    const keep = { level: b.level, maxHp: b.maxHp, shieldMax: b.shieldMax };
    b.level++;
    b.maxHp = this.bMaxHp(b.def, b.level);
    b.shieldMax = Math.round(b.maxHp * BLD_SHIELD.pct);
    const after = this.snapshot(b);
    const lines = this.upgradeLines(b, before, after);
    Object.assign(b, keep);
    return lines;
  }

  finishUpgrade(b) {
    const lv = b.upgrading.level;
    const before = this.snapshot(b);
    b.upgrading = null;
    const ratio = b.hp / b.maxHp;
    b.level = lv;
    b.maxHp = this.bMaxHp(b.def, lv);
    b.hp = Math.min(b.maxHp, Math.max(b.hp, b.maxHp * ratio));
    this.resetShield(b);
    this.refreshEconomy();
    this.emit('built', { b, upgraded: true });
    if (b.owner === 0) {
      const lines = this.upgradeLines(b, before, this.snapshot(b));
      this.emit('upgraded', { b, level: lv, lines });
      this.emit('toast', { text: `${b.def.name}: nivel ${lv}`, kind: 'good' });
      this.emit('sound', { name: 'done' });
    }
  }

  cargoSlots(t) {
    let n = 0;
    for (const u of t.cargo) n += u.cls === 'veh' ? 2 : 1;
    return n;
  }

  orderRepair(units, b) {
    let n = 0;
    for (const u of units) {
      if (!u.alive || !u.def.repairs || u.inside) continue;
      u.state = 'repair';
      u.repairTarget = b;
      u.target = null;
      u.repathT = 0;
      n++;
    }
    return n;
  }

  repairBuilding(u, b, dt) {
    u.aimYaw = Math.atan2(b.x - u.x, b.z - u.z);
    if (this.time - b.lastHit < REPAIR.pause) return;
    b.hp = Math.min(b.maxHp, b.hp + REPAIR.rate * dt);
    if (Math.random() < dt * 2) this.fx.push({ k: 'heal', x: b.x + (Math.random() - 0.5) * 0.6, y: b.y + 0.6, z: b.z +
(Math.random() - 0.5) * 0.6 });
  }

  repairAround(u, dt) {
    const uc = this.cells[u.cell];
    for (const ci of [uc.i, ...uc.nb]) {
      const b = this.cells[ci].bld;
```

```
      if (b && b.alive && b.built && b.owner === u.owner && b.hp < b.maxHp) {
        this.repairBuilding(u, b, dt);
        return;
      }
    }
  }

  // ¿Puede esta tropa subir a ese transporte? Jeep y Transporte blindado: solo soldados.
  // Portaaviones: solo Cazas (en su hangar).
  canBoard(u, t) {
    if (!u.alive || u.inside || u === t || !t.alive || this.isEnemy(u.owner, t.owner)) return false;
    if (t.def.hangar) return !!u.def.plane && u.type === 'fighter';
    if (!t.def.capacity || u.mover !== 'land') return false;
    if (t.def.carries === 'inf' && u.cls !== 'inf') return false;
    if (u.def.capacity) return false;
    return true;
  }

  orderBoard(units, t) {
    for (const u of units) {
      if (!this.canBoard(u, t)) continue;
      u.state = 'board';
      u.boardTarget = t;
      u.target = null;
      u.repathT = 0;
    }
  }

  // v1.5: recuerda por unos segundos que no puede llegar a ese objetivo.
  markUnreach(u, t, secs = 6) {
    const un = u.unreach || (u.unreach = {});
    for (const k of Object.keys(un)) if (un[k] <= this.time) delete un[k];
    un[t.id] = this.time + secs;
  }

  isUnreach(u, t) {
    return !!(u.unreach && u.unreach[t.id] > this.time);
  }

  // v1.4: la tropa que va a subir no llega al transporte (la orilla junto a él está tapada por
  // edificios, o el transporte está lejos de la costa): si el transporte está quieto, va a buscarla
  // a la orilla libre más cercana a la tropa.
  callTransport(u, t) {
    if (t.owner !== u.owner || t.inside || t.mover === 'air' || !t.alive) return false;
    if (t.state !== 'idle' || (t.path && t.pathI < t.path.length)) return false;
    if ((t.pickupT || 0) > this.time) return false;
    t.pickupT = this.time + 5;
    const uc = this.cells[u.cell];
    const tc = this.cells[t.cell];
    let best = -1;
    let bd = Infinity;
    for (const off of SPIRAL) {
      if (off.d > 6 || off.d * 4 > bd) break;
      const n = this.map.get(uc.q + off.q, uc.r + off.r);
      if (!n || n.comp !== uc.comp || !this.passable(n, u.mover, u.owner)) continue;
      for (const si of n.nb) {
        const s = this.cells[si];
        if (s.comp !== tc.comp || (si !== t.cell && !this.passable(s, t.mover, t.owner))) continue;
        const d = off.d * 4 + this.map.dist(s, tc) * 0.01;
        if (d < bd) { bd = d; best = si; }
      }
    }
    if (best < 0 || best === t.cell) return false;
    this.orderMove([t], best);
    return true;
  }

  board(u, t) {
    const need = u.cls === 'veh' ? 2 : 1;
    if (this.cargoSlots(t) + need > (t.def.capacity || t.def.hangar)) {
      this.toast(u.owner, `${t.def.name}: no queda lugar`, 'bad');
      u.state = 'idle';
      return false;
    }
    t.cargo.push(u);
    u.inside = t;
    u.state = 'idle';
    u.path = [];
    u.boardTarget = null;
    u.target = null;
    u.autoTarget = null;
    if (u.owner === 0) this.emit('sound', { name: 'ready' });
    return true;
  }

  unload(t) {
    if (!t.cargo || !t.cargo.length) return `${t.def.name} va vacío`;
    const c0 = this.cells[t.cell];
```

```
    const spots = [];
    const mover = t.def.hangar ? 'air' : 'land';
    for (const off of SPIRAL) {
      if (off.d > 2) break;
      const c = this.map.get(c0.q + off.q, c0.r + off.r);
      if (c && this.passable(c, mover, t.owner)) spots.push(c.i);
    }
    if (!spots.length) return t.mover === 'sea' ? 'Acércate a la orilla para desembarcar' : 'No hay lugar para bajar';
    t.cargo.forEach((u, i) => {
      const ci = spots[i % spots.length];
      const c = this.cells[ci];
      u.inside = null;
      u.cell = ci;
      u.x = c.x + u.offX;
      u.z = c.z + u.offZ;
      u.y = c.h;
      u.state = 'idle';
      u.path = [];
    });
    const out = t.cargo;
    t.cargo = [];
    if (t.owner === 0) this.emit('sound', { name: 'order' });
    // v1.4: desembarco en otra isla (para el logro «Lobo de mar»).
    const home = this.players[t.owner].core && this.cells[this.players[t.owner].core.cell];
    if (t.mover === 'sea' && home && out.some((u) => u.mover === 'land' && this.cells[u.cell].comp !== home.comp)) {
      const st = this.players[t.owner].stats;
      st.landings = (st.landings || 0) + 1;
    }
    return out;
  }

  // ---------- órdenes ----------
  setPath(u, dest) {
    u.dest = dest;
    u.path = u.mover === 'air' ? [dest] : this.findPath(u.cell, dest, u.mover, u.owner);
    u.pathI = 0;
  }

  spreadDest(u, center, used) {
    for (const off of SPIRAL) {
      if (off.d > 7) break;
      const c = this.map.get(center.q + off.q, center.r + off.r);
      if (!c || used.has(c.i) || !this.passable(c, u.mover, u.owner)) continue;
      return c.i;
    }
    return center.i;
  }

  // Formación en línea (v0.7): una fila de frente al destino, perpendicular a la dirección de marcha.
  // Devuelve una casilla por tropa (ordenadas de izquierda a derecha) y hacia dónde miran.
  lineSlots(units, center) {
    let cx = 0;
    let cz = 0;
    for (const u of units) { cx += u.x; cz += u.z; }
    cx /= units.length;
    cz /= units.length;
    let dx = center.x - cx;
    let dz = center.z - cz;
    const len = Math.hypot(dx, dz);
    if (len < 0.5) { dx = 0; dz = 1; } else { dx /= len; dz /= len; }
    const px = -dz;
    const pz = dx;
    const perRow = 10;
    const order = [...units].sort((a, b) => (a.x * px + a.z * pz) - (b.x * px + b.z * pz));
    const slots = new Map();
    const used = new Set();
    order.forEach((u, k) => {
      const row = Math.floor(k / perRow);
      const inRow = Math.min(perRow, order.length - row * perRow);
      const off = (k % perRow) - (inRow - 1) / 2;
      const x = center.x + px * off * SQ3 - dx * row * SQ3;
      const z = center.z + pz * off * SQ3 - dz * row * SQ3;
      const c = this.map.atWorld(x, z) || center;
      const dest = this.spreadDest(u, c, used);
      used.add(dest);
      slots.set(u, dest);
      // Dentro de su casilla, se para justo sobre la línea (sin salirse de la casilla).
      const dc = this.cells[dest];
      let ox = x - dc.x;
      let oz = z - dc.z;
      const ol = Math.hypot(ox, oz);
      if (ol > 0.7) { ox *= 0.7 / ol; oz *= 0.7 / ol; }
      // v1.3: si con ese desplazamiento la tropa quedaría sobre una casilla que no puede pisar
      // (agua para las de tierra, tierra para los barcos), se queda más cerca del centro.
      if (u.mover !== 'air') {
        const over = this.map.atWorld(dc.x + ox, dc.z + oz);
        if (over && over.i !== dest && !this.passable(over, u.mover, u.owner)) {
          const k = Math.min(1, 0.33 / (Math.hypot(ox, oz) || 1));
```

```
          ox *= k;
          oz *= k;
        }
        u.offX = ox;
        u.offZ = oz;
      }
    });
    return { slots, face: Math.atan2(dx, dz) };
  }

  orderMove(units, ci, attackMove = false, formation = 'block') {
    const used = new Set();
    const center = this.cells[ci];
    const alive = units.filter((u) => u.alive);
    const line = formation === 'line' && alive.length > 1 ? this.lineSlots(alive, center) : null;
    for (const u of alive) {
      const dest = line ? line.slots.get(u) : this.spreadDest(u, center, used);
      used.add(dest);
      u.faceYaw = line ? line.face : null;
      u.target = null;
      u.autoTarget = null;
      u.aggroTarget = null;
      u.state = attackMove ? 'amove' : 'move';
      u.resume = attackMove ? dest : -1;
      u.leashCell = -1;
      u.patrol = null;
      this.setPath(u, dest);
    }
  }

  // v1.4: patrullar entre donde están y el punto elegido.
  orderPatrol(units, ci) {
    const alive = units.filter((u) => u.alive && !u.inside && u.def.weapon);
    this.orderMove(alive, ci, true);
    for (const u of alive) {
      const dest = u.path.length ? u.path[u.path.length - 1] : ci;
      u.patrol = { a: u.cell, b: dest, leg: 1, laps: 0 };
      u.state = 'patrol';
      u.resume = -1;
    }
    return alive.length;
  }

  // v1.4: «Mantener posición»: se quedan quietas y solo disparan a lo que tienen a tiro.
  setHold(units, on) {
    for (const u of units) {
      if (!u.alive || !u.def.weapon) continue;
      u.hold = !!on;
      if (on) {
        u.patrol = null;
        if (u.state === 'attack' || u.state === 'amove') {
          u.state = 'idle';
          u.path = [];
          u.target = null;
          u.resume = -1;
        }
      }
    }
  }

  orderAttack(units, target) {
    for (const u of units) {
      if (!u.alive) continue;
      u.faceYaw = null;
      u.patrol = null;
      const w = u.def.weapon;
      if (!w || !this.canAttack(u, w, target)) continue;
      u.state = 'attack';
      u.target = target;
      u.resume = -1;
      u.repathT = 0;
      u.leashCell = -1;
    }
  }

  orderFound(u, ci) {
    const why = this.foundInvalid(u.owner, ci, u.def.founds);
    if (why) return why;
    u.state = 'found';
    u.foundCell = ci;
    u.target = null;
    this.setPath(u, ci);
    return null;
  }

  orderStop(units) {
    for (const u of units) {
      u.patrol = null;
```

```
      u.state = 'idle';
      u.path = [];
      u.target = null;
      u.resume = -1;
    }
  }

  // ---------- combate ----------
  canAttack(a, w, t) {
    if (!t.alive || t.inside || !this.isEnemy(a.owner, t.owner)) return false;
    if (t.def.stealth && !(t.seenBy & (1 << this.players[a.owner].team))) return false;
    if (t.kind === 'bld') return w.waterOnly ? !!(t.onWater || t.coastal) : (w.mult.bld || 0) > 0;
    if (w.waterOnly) return t.cls === 'ship' || t.cls === 'sub';
    return (w.mult[t.cls] || 0) > 0;
  }

  coverBetween(a, t) {
    const dx = t.x - a.x;
    const dz = t.z - a.z;
    const d = Math.hypot(dx, dz);
    const steps = Math.ceil(d / 0.8);
    for (let i = 1; i < steps; i++) {
      const c = this.map.atWorld(a.x + (dx * i) / steps, a.z + (dz * i) / steps);
      if (c && TERRAIN[c.t].cover && c.i !== t.cell) return true;
    }
    return false;
  }

  // v1.2: ventaja de altura: desde una meseta se dispara 1 casilla más lejos a lo que está abajo.
  heightBonus(a, t) {
    if (a.mover === 'air' || a.cell === undefined || !this.cells[a.cell]) return 0;
    if (!TERRAIN[this.cells[a.cell].t].highGround) return 0;
    if (t && t.cell !== undefined && this.cells[t.cell] && TERRAIN[this.cells[t.cell].t].highGround) return 0;
    return 1;
  }

  inRange(a, w, t, extraTiles = 0) {
    let r = w.range + extraTiles + (w.range >= 2 ? this.heightBonus(a, t) : 0);
    const d = Math.hypot(t.x - a.x, t.z - a.z);
    const reach = (rr) => rr * SQ3 + (t.kind === 'bld' ? 0.75 : 0.35);
    if (d > reach(r)) return false;
    if (a.def.losBlock && d > reach(2) && this.coverBetween(a, t)) r = Math.min(r, 2);
    // Ocultación: una tropa en bosque solo se ataca de cerca, salvo desde el aire.
    if (t.kind === 'unit' && t.mover === 'land' && a.mover !== 'air' && this.cells[t.cell].t === 'forest') r =
Math.min(r, 2);
    return d <= reach(r);
  }

  acquire(a, w, extraTiles, exclude = null) {
    const maxD = (w.range + extraTiles + (w.range >= 2 ? this.heightBonus(a, null) : 0)) * SQ3 + 0.8;
    // Fuego concentrado (bots desde Normal, v0.8): entre lo que tienen a tiro, primero la tropa
    // armada con menos vida; así la eliminan antes en vez de repartir el daño.
    const focus = extraTiles === 0 && this.players[a.owner].focusFire;
    // Difícil (v1.0): entre lo que tiene a tiro, primero los Constructores y la economía.
    const snipe = extraTiles === 0 && this.players[a.owner].snipe;
    let best = null;
    let bs = Infinity;
    // v1.3: enemigos a los que esta tropa no encontró camino hace poco: no los vuelve a perseguir
    // (si no, en «Avanzar atacando» cambiaba de estado en cada paso y no se movía).
    const unreach = extraTiles > 0 && a.unreach ? a.unreach : null;
    const check = (t) => {
      if (!t.alive || t.owner === a.owner || t === exclude) return;
      if (unreach && (unreach[t.id] || 0) > this.time && !this.inRange(a, w, t, 0)) return;
      const dx = t.x - a.x;
      const dz = t.z - a.z;
      if (Math.abs(dx) > maxD || Math.abs(dz) > maxD) return;
      if (!this.canAttack(a, w, t)) return;
      const d = Math.hypot(dx, dz);
      if (d > maxD || !this.inRange(a, w, t, extraTiles)) return;
      const s = a.def && a.def.prefersBld
        ? d + (t.kind === 'bld' ? 0 : 8)
        : a.def && a.def.prefersAir
          ? d + (t.cls === 'air' ? 0 : 20)
        : focus
          ? (t.kind === 'bld' ? 1e6 : 0) + (t.def.weapon ? 0 : 5000) + t.hp + (t.shield || 0) + d * 4
          : d + (t.kind === 'bld' ? 3 : 0) + (t.def.weapon ? 0 : 1.5);
      let sc = s;
      if (snipe && !(a.def && (a.def.prefersBld || a.def.prefersAir))) {
        if (t.kind === 'unit' && t.def.builds) sc -= focus ? 6000 : 6;
        else if (t.kind === 'bld' && t.def.income) sc -= focus ? 5e5 : 1;
      }
      if (sc < bs) { bs = sc; best = t; }
    };
    for (const t of this.units) check(t);
    for (const t of this.buildings) check(t);
    return best;
  }

```

```
  muzzleY(a) {
    if (a.kind === 'bld') return a.y + (a.type === 'core' ? 1.3 : 0.65);
    if (a.mover === 'air') return a.y + 0.2;
    return a.y + (a.cls === 'inf' ? 0.35 : 0.35);
  }

  fire(a, w, t, gun = 0) {
    if (a.def.stealth) a.lastShot = this.time;
    const mult = t.kind === 'bld' ? w.mult.bld || 0 : w.mult[t.cls] || 0;
    const dmg = w.dmg * mult * (1 + (a.vet || 0) * VETERAN.dmg);
    let sx = a.x;
    const sy = this.muzzleY(a) + (gun && a.type === 'turret' ? 0.05 : 0);
    let sz = a.z;
    if (gun && a.kind === 'bld') {
      // Cañones laterales (miniguns o segundo láser) salen un poco a cada lado.
      const side = gun % 2 === 1 ? 1 : -1;
      const yaw = a.aimYaw || 0;
      sx += Math.cos(yaw) * 0.32 * side;
      sz -= Math.sin(yaw) * 0.32 * side;
    }
    const ty = t.kind === 'bld' ? t.y + 0.4 : t.y + (t.cls === 'inf' ? 0.25 : 0.2);
    const color = PLAYER_COLORS[a.owner];
    if (w.proj === 'laser') {
      this.fx.push({ k: 'laser', x1: sx, y1: sy, z1: sz, x2: t.x, y2: ty, z2: t.z });
      this.damage(t, dmg, a.owner, a);
      this.fx.push({ k: 'spark', x: t.x, y: ty, z: t.z, s: 0.4 });
    } else if (w.proj === 'bullet' || w.proj === 'bolt' || w.proj === 'flak') {
      this.fx.push({ k: 'tracer', x1: sx, y1: sy, z1: sz, x2: t.x, y2: ty, z2: t.z, color, heavy: w.proj !== 'bullet'
});
      this.damage(t, dmg, a.owner, a);
      if (w.proj === 'flak' || w.proj === 'bolt') this.fx.push({ k: 'spark', x: t.x, y: ty, z: t.z, s: 0.35 });
    } else {
      const speed = w.proj === 'rocket' ? 10 : w.proj === 'shell' ? 12 : w.proj === 'missile' ? 7 : w.proj === 'bomb' ?
3 : 5;
      const d = Math.hypot(t.x - sx, t.z - sz);
      // Bengalas del Apache: esquiva un misil cada 20 s.
      let miss = false;
      if (w.proj === 'missile' && t.def.flares && this.time >= (t.flareAt || 0)) {
        miss = true;
        t.flareAt = this.time + t.def.flares;
        this.fx.push({ k: 'flare', x: t.x, y: t.y + 0.4, z: t.z });
        if (t.owner === 0) this.emit('toast', { text: `${t.def.name}: bengalas, esquivó un misil`, kind: 'good' });
      }
      this.projectiles.push({
        kind: w.proj, t, dmg, owner: a.owner, src: a, sx, sy, sz, tx: t.x, ty, tz: t.z,
        time: 0, dur: Math.max(w.proj === 'bomb' ? 0.6 : 0.12, d / speed), color, miss,
      });
    }
    // Sonido: la minigun de la Torreta suena distinto al cañón.
    this.emit('sound', { name: gun && a.type === 'turret' ? 'minigun' : w.proj, x: sx, z: sz });
  }

  domeCovers(owner, t) {
    const p = this.players[owner];
    if (!p.dome || p.dome.hp <= 0 || !p.core.alive) return false;
    return Math.hypot(t.x - p.core.x, t.z - p.core.z) <= DOME_LEVELS[p.dome.level].range * SQ3 + 0.5;
  }

  unitShieldActive(pid) {
    const p = this.players[pid];
    return !!(p && p.dome && p.dome.level >= 3 && p.dome.hp > 0 && p.core && p.core.alive);
  }

  // v0.9: el escudo personal (Cúpula nivel 3) solo funciona dentro de la Cúpula.
  unitShielded(u) {
    return this.unitShieldActive(u.owner) && this.domeCovers(u.owner, u);
  }

  // Lugares con pelea reciente: la interfaz dibuja un icono de combate (documento pág. 13).
  noteFight(t) {
    const last = this._fightCell.get(t.cell) || -99;
    if (this.time - last < 1) return;
    this._fightCell.set(t.cell, this.time);
    this.fights.push({ x: t.x, z: t.z, y: t.y, t: this.time, cell: t.cell });
    if (this.fights.length > 40) this.fights.shift();
  }

  damage(t, dmg, src, srcEnt, nuke = false) {
    if (!t.alive || dmg <= 0) return;
    if (srcEnt) this.noteFight(t);
    // v1.1: quién atacó a quién (para las alianzas).
    if (src >= 0 && src !== t.owner && this.players[t.owner]) {
      const tp = this.players[t.owner];
      tp.hurtBy = tp.hurtBy || {};
      tp.hurtBy[src] = this.time;
      tp.underAttackAt = this.time;
    }
    if (t.owner === 0 && srcEnt) {
```

```
      const k = srcEnt.kind === 'bld' ? 'bld' : srcEnt.type;
      const hb = this.players[0].stats.hitBy;
      hb[k] = (hb[k] || 0) + dmg;
    }
    // Escudo personal de las tropas (Cúpula nivel 3).
    if (t.kind === 'unit' && t.shield > 0) {
      const a = Math.min(t.shield, dmg);
      t.shield -= a;
      dmg -= a;
      t.lastHit = this.time;
      if (dmg <= 0) return;
    }
    // Cúpula: absorbe el daño de barcos, aeronaves y lanzamisiles cerca del Núcleo.
    // v1.3: también el del misil nuclear; lo que pase de la vida de la Cúpula sí llega.
    if (((srcEnt && (srcEnt.cls === 'ship' || srcEnt.cls === 'sub' || srcEnt.cls === 'air' || srcEnt.type === 'arty'))
|| nuke) &&
      this.domeCovers(t.owner, t)) {
      const dome = this.players[t.owner].dome;
      const took = nuke ? Math.min(dome.hp, dmg) : dmg;
      dome.hp -= took;
      dome.lastHit = this.time;
      if (dome.hp <= 0) {
        dome.hp = 0;
        dome.brokenAt = this.time;
        this.fx.push({ k: 'boom', x: this.players[t.owner].core.x, y: 2, z: this.players[t.owner].core.z, s: 1.2 });
        if (t.owner === 0) this.emit('toast', { text: `La Cúpula se rompió. Vuelve en ${DOME.recover} s`, kind: 'bad'
});
        for (const u of this.units) if (u.owner === t.owner) u.shield = 0;
      }
      dmg -= took;
      if (dmg <= 0) return;
    }
    // Escudo propio del edificio (v0.9).
    if (t.kind === 'bld' && t.shield > 0) {
      const a = Math.min(t.shield, dmg);
      t.shield -= a;
      dmg -= a;
      t.lastHit = this.time;
      if (dmg <= 0) return;
    }
    if (t.type === 'core') {
      if (t.shieldUntil > this.time) return;
      const p = this.players[t.owner];
      if (!p.emergencyUsed && t.hp - dmg <= t.maxHp * EMERGENCY.at) {
        p.emergencyUsed = true;
        t.hp = Math.max(1, Math.min(t.hp, t.maxHp * EMERGENCY.at));
        t.shieldUntil = this.time + EMERGENCY.time;
        this.emit('toast', { text: t.owner === 0 ? `¡Escudo de emergencia! ${EMERGENCY.time} s de protección`
          : `${p.name} activó su escudo de emergencia`, kind: t.owner === 0 ? 'good' : 'info' });
        this.emit('sound', { name: 'alarm' });
        return;
      }
    }
    t.hp -= dmg;
    t.lastHit = this.time;
    t.lastAttacker = src;
    const victim = this.players[t.owner];
    if (t.kind === 'bld' && srcEnt && (srcEnt.cls === 'ship' || srcEnt.cls === 'sub' || srcEnt.cls === 'air' ||
srcEnt.type === 'arty')) {
      victim.navalHitAt = this.time;
    }
    if (t.owner === 0) {
      if (t.type === 'core' && this.time - this._coreWarn > 15) {
        this._coreWarn = this.time;
        this.alert('core', '¡Tu Núcleo está bajo ataque!', 'bad', t.x, t.z, 0);
        this.emit('sound', { name: 'alarm' });
      } else if (t.kind === 'bld') {
        this.alert('bld', `Atacan tu ${t.def.name}`, 'bad', t.x, t.z, 20);
      } else if (this.cells[t.cell].owner === 0) {
        this.alert('units', 'Tus tropas pelean en tu territorio', 'bad', t.x, t.z, 25);
      }
    } else if (src === 0 && !victim.human && !victim.provoked && t.kind === 'bld') {
      victim.provoked = true;
      const lines = PERSONALITIES[victim.persona].lines.provoked;
      this.say(t.owner, lines[Math.floor(Math.random() * lines.length)], { toMe: true, force: true });
    }
    if (t.hp <= 0) {
      if (srcEnt && srcEnt.kind === 'unit' && srcEnt.alive && this.isEnemy(srcEnt.owner, t.owner))
this.creditKill(srcEnt, t);
      this.kill(t, src);
    }
  }

  // v1.4: eliminaciones de cada tropa y sus estrellas (veteranas).
  creditKill(u, t) {
    u.kills = (u.kills || 0) + 1;
    const st = this.players[u.owner].stats;
    if (t.kind === 'unit' && t.mover === 'air') st.airKills = (st.airKills || 0) + 1;
```

```
    const stars = VETERAN.kills.filter((k) => u.kills >= k).length;
    if (stars > (u.vet || 0)) {
      const before = 1 + (u.vet || 0) * VETERAN.hp;
      const after = 1 + stars * VETERAN.hp;
      const add = u.maxHp * (after / before - 1);
      u.maxHp += add;
      u.hp += add;
      u.vet = stars;
      st.bestVet = Math.max(st.bestVet || 0, stars);
      this.fx.push({ k: 'heal', x: u.x, y: u.y + 0.6, z: u.z });
      if (u.owner === 0) this.emit('toast', { text: `${u.def.name}: ${'★'.repeat(stars)} veterano (${u.kills}
eliminaciones)`, kind: 'good', x: u.x, z: u.z });
    }
  }

  // v1.4: un Constructor o Ingeniero libre repara solo los edificios dañados cercanos.
  autoRepairScan(u, dt) {
    const p = this.players[u.owner];
    if (!p.human || p.autoRepair === false) return;
    u.repScanT = (u.repScanT || 0) - dt;
    if (u.repScanT > 0) return;
    u.repScanT = AUTO_REPAIR.every;
    const uc = this.cells[u.cell];
    let best = null;
    let bd = Infinity;
    for (const b of this.buildings) {
      if (!b.alive || !b.built || b.owner !== u.owner || b.hp >= b.maxHp * 0.98 || b.upgrading) continue;
      const d = this.map.dist(uc, this.cells[b.cell]);
      if (d > AUTO_REPAIR.range) continue;
      if (this.isUnreach(u, b)) continue;
      if (this.units.some((x) => x !== u && x.alive && x.state === 'repair' && x.repairTarget === b)) continue;
      if (d < bd) { bd = d; best = b; }
    }
    if (best && bd > 1) {
      this.orderRepair([u], best);
      u.autoRep = true;
      p.stats.autoRepairs = (p.stats.autoRepairs || 0) + 1;
    }
  }

  kill(e, killer) {
    if (!e.alive) return;
    e.alive = false;
    const big = e.kind === 'bld' ? 1.5 : e.cls === 'inf' ? 0.45 : 0.95;
    this.fx.push({ k: 'boom', x: e.x, y: e.y + 0.3, z: e.z, s: big, death: true, bld: e.kind === 'bld' });
    this.emit('sound', { name: e.kind === 'bld' ? 'bigboom' : e.cls !== 'inf' ? 'boom' : 'hit', x: e.x, z: e.z });
    if (killer >= 0 && killer !== e.owner) this.players[killer].stats.kills++;
    if (e.kind === 'unit') this.players[e.owner].stats.lost++;
    if (e.kind === 'bld') this.players[e.owner].stats.lostBld = (this.players[e.owner].stats.lostBld || 0) + 1;
    if (e.kind === 'bld' && e.owner === 0 && this.camp && !this.over) this.camp.lostBld++;
    if (e.cargo) for (const c of e.cargo) this.kill(c, killer);
    if (e.kind === 'bld') {
      const c = this.cells[e.cell];
      if (c.bld === e) c.bld = null;
      if (e.def.claim) this.territoryDirty = true;
      if (e.type === 'core') this.eliminate(e.owner, killer);
      else if (e.owner === 0) {
        this.emit('toast', { text: `Perdiste: ${e.def.name}`, kind: 'bad', x: e.x, z: e.z });
        this.emit('ping', { x: e.x, z: e.z, kind: 'bad' });
      }
    }
  }

  eliminate(pid, killer) {
    const p = this.players[pid];
    if (!p.alive) return;
    p.alive = false;
    this.elimOrder.push({ pid, time: this.time, killer });
    if (killer >= 0 && killer !== pid) this.players[killer].coresKilled++;
    if (!p.human && p.persona) {
      const lines = PERSONALITIES[p.persona].lines.fall;
      this.say(pid, lines[Math.floor(Math.random() * lines.length)], { force: true, public: true });
    }
    for (const pt of this.points) {
      if (pt.owner === pid) {
        pt.owner = -1;
        pt.prog = 0;
        pt.capTeam = -1;
        this.territoryDirty = true;
      }
    }
    for (const u of this.units) if (u.alive && u.owner === pid) this.kill(u, -1);
    for (const b of this.buildings) if (b.alive && b.owner === pid) this.kill(b, -1);
    this.emit('toast', { text: pid === 0 ? 'Tu Núcleo ha caído' : `${p.name} ha sido eliminado`, kind: pid === 0 ?
'bad' : 'good' });
    if (killer === 0 && pid !== 0) this.completeObjective('kill');
    this.checkEnd();
  }
```

```
1552
1553  endKing(team) {
1554    if (this.over) return;
1555    this.over = true;
1556    this.result = team === this.players[0].team ? 'victory' : 'defeat';
1557    this.kingWinner = team;
1558    this.emit('over', { result: this.result, king: true });
1559  }
1560
1561  checkEnd() {
1562    if (this.over) return;
1563    const me = this.players[0];
1564    const alive = this.players.filter((p) => p.alive);
1565    const teams = new Set(alive.map((p) => p.team));
1566    // v1.1: si solo quedan aliados de una alianza hecha en la partida, la alianza se rompe: gana uno solo.
1567    if (alive.length > 1 && teams.size <= 1 && this.alliancesAllowed() && new Set(alive.map((p) => p.team0)).size > 1)
{
1568      for (const p of alive) p.team = p.team0;
1569      this.allianceChanged();
1570      const bot = alive.find((p) => !p.human);
1571      if (bot) this.say(bot.id, 'Solo quedamos nosotros: se acabó la alianza. ¡Que gane el mejor!', { force: true,
public: true });
1572      this.emit('toast', { text: 'La alianza se rompió: solo puede quedar un imperio', kind: 'bad' });
1573      return;
1574    }
1575    // v1.1: espectador. Ya perdiste; la partida sigue hasta que queda un solo equipo.
1576    if (this.spectating) {
1577      if (teams.size <= 1) {
1578        this.over = true;
1579        this.spectating = false;
1580        this.spectateWinner = alive.length ? alive[0].id : -1;
1581        this.emit('over', { result: this.result, spectated: true });
1582      }
1583      return;
1584    }
1585    if (!me.alive) {
1586      this.over = true;
1587      this.result = 'defeat';
1588    } else if (teams.size <= 1) {
1589      this.over = true;
1590      this.result = 'victory';
1591    }
1592    if (this.over) this.emit('over', { result: this.result });
1593  }
1594
1595  // Al perder, se puede seguir mirando si todavía pelean dos equipos o más.
1596  canSpectate() {
1597    if (this.result !== 'defeat' || this.camp || this.mode.tutorial) return false;
1598    return new Set(this.players.filter((p) => p.alive).map((p) => p.team)).size > 1;
1599  }
1600
1601  startSpectating() {
1602    if (!this.canSpectate()) return false;
1603    this.spectating = true;
1604    this.over = false;
1605    return true;
1606  }
1607
1608  completeObjective(id) {
1609    if (this.objDone[id]) return;
1610    this.objDone[id] = true;
1611    this.emit('objective', { id });
1612  }
1613
1614  // ---------- simulación ----------
1615  update(rawDt) {
1616    const dt = Math.min(rawDt, 0.1) * this.speed;
1617    if (dt <= 0) return;
1618    this.time += dt;
1619
1620    this._slowT -= dt;
1621    if (this._slowT <= 0) {
1622      this._slowT = 0.5;
1623      this.refreshEconomy();
1624      this.checkObjectives();
1625      for (const p of this.players) {
1626        if (p.pendingSay && p.alive && this.time - (p.lastSay || -99) >= 20) {
1627          const { text, opts } = p.pendingSay;
1628          p.pendingSay = null;
1629          this.say(p.id, text, { ...opts, queue: 0 });
1630        }
1631      }
1632    }
1633    if (this.creator) {
1634      // Modo creador: monedas y energía infinitas para el jugador.
1635      this.players[0].coins = 999999;
1636      this.players[0].energy = 99999;
1637    }
```

```
    this._assignT -= dt;
    if (this._assignT <= 0) {
      this._assignT = 1;
      this.assignBuilders();
    }
    for (const p of this.players) {
      if (!p.alive) continue;
      const inc = (p.income / 60) * dt * this.incomeMult(p);
      p.coins += inc;
      p.stats.earned += inc;
      p.energy += (p.energyRate / 10) * dt;
      if (p.dome && p.dome.hp <= 0 && this.time - p.dome.brokenAt >= DOME.recover) {
        p.dome.hp = p.dome.maxHp;
        if (p.id === 0) this.emit('toast', { text: 'La Cúpula volvió a levantarse', kind: 'good' });
      }
    }
    this.updatePoints(dt);

    for (let i = 0; i < this.buildings.length; i++) {
      const b = this.buildings[i];
      if (b.alive) this.updateBuilding(b, dt);
    }
    for (let i = 0; i < this.units.length; i++) {
      const u = this.units[i];
      if (u.alive) this.updateUnit(u, dt);
    }
    this.updateProjectiles(dt);
    this.updateRadiation(dt);
    this.updateAutoUpgrade(dt);
    this.recordHistory(dt);
    if (!this.over && this.bots.length) {
      // Como mucho un bot piensa por paso; el turno rota para que ninguno se quede esperando.
      const n = this.bots.length;
      this._botTurn = ((this._botTurn || 0) + 1) % n;
      let busy = false;
      for (let k = 0; k < n; k++) busy = this.bots[(this._botTurn + k) % n].update(dt, busy) || busy;
    }
    if (this.camp) Campaign.update(this, dt);

    if (this.units.some((u) => !u.alive)) {
      for (const u of this.units) if (!u.alive) this.removed.push(u);
      this.units = this.units.filter((u) => u.alive);
    }
    if (this.buildings.some((b) => !b.alive)) {
      for (const b of this.buildings) if (!b.alive) this.removed.push(b);
      this.buildings = this.buildings.filter((b) => b.alive);
    }
    if (this.territoryDirty) this.recomputeTerritory();

    this._fogT -= dt;
    if (this._fogT <= 0) {
      this._fogT = 0.3;
      this.updateVisibility();
    }
  }

  checkObjectives() {
    if (this.count(0, 'farm') && this.buildings.some((b) => b.owner === 0 && b.type === 'farm' && b.built))
this.completeObjective('farm');
    if (this.buildings.some((b) => b.owner === 0 && b.type === 'barracks' && b.built))
this.completeObjective('barracks');
    const army = this.units.filter((u) => u.owner === 0 && u.def.weapon).length;
    if (army >= 5) this.completeObjective('army');
    // v1.3: ya empiezas con un Constructor; el objetivo es entrenar otro.
    if ((this.players[0].stats.builders || 0) > 0) this.completeObjective('builder');
    if (this.buildings.some((b) => b.owner === 0 && b.def.cat === 'energy' && b.built))
this.completeObjective('energy');
    if (this.points.some((pt) => pt.owner === 0)) this.completeObjective('capture');
    // v1.3: pasos del tutorial de tu documento.
    if (this.mode.tutorial) {
      const done = (t) => this.buildings.filter((b) => b.owner === 0 && b.alive && b.built && t(b)).length;
      if (done((b) => b.type === 'farm') >= 2 && done((b) => b.def.cat === 'energy') >= 1)
this.completeObjective('tut_build');
      const soldiers = this.units.filter((u) => u.owner === 0 && u.alive && u.cls === 'inf' && u.def.weapon).length;
      if (done((b) => b.type === 'barracks') >= 1 && soldiers >= 3) this.completeObjective('tut_army');
    }
  }

  // Cúpula (puntos 5 y 6): pide Núcleo nivel 3; sube de nivel 1 a 3.
  domeCost(pid) {
    const p = this.players[pid];
    const next = p.dome ? p.dome.level + 1 : 1;
    return next <= 3 ? DOME_LEVELS[next].cost : 0;
  }

  domeInvalid(pid) {
    const p = this.players[pid];
    if (!p.core.alive || p.core.level < 3) return 'Mejora el Núcleo a nivel 3 para comprar la Cúpula';
```

```
    if (p.dome && p.dome.level >= 3) return 'La Cúpula ya está en nivel 3';
    if (!(this.creator && pid === 0) && p.coins < this.domeCost(pid)) return 'Monedas insuficientes';
    return null;
  }

  buyDome(pid) {
    const err = this.domeInvalid(pid);
    if (err) return err;
    const p = this.players[pid];
    p.coins -= this.domeCost(pid);
    const lv = p.dome ? p.dome.level + 1 : 1;
    const L = DOME_LEVELS[lv];
    p.dome = { level: lv, hp: L.hp, maxHp: L.hp, brokenAt: -99, lastHit: -99 };
    if (L.unitShield) {
      for (const u of this.units) {
        if (u.alive && u.owner === pid) {
          u.shieldMax = u.maxHp * L.unitShield;
          u.shield = u.shieldMax;
        }
      }
    }
    this.emit('sound', { name: 'done', pid });
    return null;
  }

  // ---------- captura de puntos ----------
  updatePoints(dt) {
    this._pointT -= dt;
    if (this._pointT > 0) return;
    const step = 0.25 - this._pointT;
    this._pointT = 0.25;
    const nearD = CAPTURE.near * SQ3 + 0.5;
    const clearD = CAPTURE.clear * SQ3 + 0.5;
    for (const pt of this.points) {
      const near = new Map();
      const close = new Set();
      for (const u of this.units) {
        if (!u.alive || u.inside) continue;
        const d = Math.hypot(u.x - pt.x, u.z - pt.z);
        if (d > clearD) continue;
        const team = this.players[u.owner].team;
        close.add(team);
        if (d <= nearD && u.cls === 'inf') {
          if (!near.has(team)) near.set(team, u.owner);
        }
      }
      const ownerTeam = pt.owner >= 0 ? this.players[pt.owner].team : -1;
      pt.contested = close.size > 1;
      if (pt.contested) continue;
      if (near.size === 1) {
        const [team, pid] = [...near.entries()][0];
        const k = step / CAPTURE.time;
        if (team === ownerTeam) {
          pt.prog = Math.min(1, pt.prog + k);
        } else if (pt.owner >= 0) {
          pt.prog -= k;
          if (pt.prog <= 0) {
            const lost = pt.owner;
            pt.owner = -1;
            pt.prog = 0;
            pt.capTeam = team;
            this.territoryDirty = true;
            if (this.players[lost].team === this.players[0].team) {
              this.alert('pt' + pt.id, `Perdiste ${pt.type === 'faro' ? 'el Faro' : 'una Bandera'}`, 'bad', pt.x, pt.z,
0);
            }
          }
        } else {
          if (pt.capTeam !== team) {
            pt.capTeam = team;
            pt.prog = 0;
          }
          pt.prog += k;
          if (pt.prog >= 1) {
            pt.prog = 1;
            pt.owner = pid;
            this.players[pid].stats.captures++;
            if (pt.type === 'faro') this.players[pid].stats.faro = (this.players[pid].stats.faro || 0) + 1;
            this.territoryDirty = true;
            this.refreshEconomy();
            const what = pt.type === 'faro' ? 'el Faro' : 'una Bandera';
            const capP = this.players[pid];
            if (!capP.human && capP.persona) {
              if (capP.team === this.players[0].team) {
                this.say(pid, pt.type === 'faro' ? ALLY_LINES.faro : ALLY_LINES.flag, { x: pt.x, z: pt.z });
              } else if (pt.type === 'faro') {
                const lines = PERSONALITIES[capP.persona].lines.faro;
                this.say(pid, lines[Math.floor(Math.random() * lines.length)], { x: pt.x, z: pt.z, public: true, force:
```

```
true });
              }
            }
            if (this.players[pid].team === this.players[0].team) {
              this.emit('toast', { text: pid === 0 ? `¡Capturaste ${what}!` : `${this.players[pid].name} capturó
${what}`, kind: 'good' });
              if (pid === 0) this.emit('sound', { name: 'capture' });
              this.emit('sound', { name: 'done' });
            } else if (pt.type === 'faro') {
              this.emit('toast', { text: `${this.players[pid].name} capturó el Faro`, kind: 'bad', x: pt.x, z: pt.z });
              this.emit('ping', { x: pt.x, z: pt.z, kind: 'bad' });
            }
          }
        }
      } else if (pt.owner < 0 && pt.prog > 0 && near.size === 0) {
        pt.prog = Math.max(0, pt.prog - step / (CAPTURE.time * 2));
      }
    }
    if (this.mode.king && !this.over) {
      const faro = this.points.find((pt) => pt.type === 'faro');
      if (faro && faro.owner >= 0) {
        const team = this.players[faro.owner].team;
        this.kingTime[team] = (this.kingTime[team] || 0) + step;
        if (this.kingTime[team] >= KING_TIME) this.endKing(team);
      }
    }
  }

  updateStealth() {
    const detR = (3 * SQ3 + 0.5) ** 2;
    for (const s of this.units) {
      if (!s.alive || !s.def.stealth) continue;
      if (this.time - (s.lastShot || -99) < 3) {
        s.seenBy = 0xff;
        continue;
      }
      let mask = 1 << this.players[s.owner].team;
      const check = (e) => {
        if (!e.alive || !this.isEnemy(e.owner, s.owner)) return;
        const dx = e.x - s.x;
        const dz = e.z - s.z;
        if (dx * dx + dz * dz <= detR) mask |= 1 << this.players[e.owner].team;
      };
      for (const u of this.units) if (u.def.detects) check(u);
      for (const b of this.buildings) {
        if (!b.built) continue;
        // Puertos ven submarinos; AIR DEFENSE y Torretas de nivel 4 ven al bombardero furtivo de cerca.
        if (b.type === 'harbor' && s.mover === 'sea') check(b);
        else if (s.mover === 'air' && (b.type === 'aadef' || (b.def.aaLevel && b.level >= b.def.aaLevel))) check(b);
      }
      s.seenBy = mask;
    }
  }

  updateVisibility() {
    const fog = this.fog;
    const myTeam = this.players[0].team;
    this.updateStealth();
    const stealthy = (u) => u.def.stealth && !(u.seenBy & (1 << myTeam));
    if (!fog) {
      for (const u of this.units) u.visible = !u.inside && !stealthy(u);
      return;
    }
    const eyes = [];
    for (const u of this.units) {
      if (this.players[u.owner].team === myTeam) eyes.push([u.x, u.z, (((u.def.vision || (u.def.weapon ?
u.def.weapon.range + 1.5 : 3)) + this.heightBonus(u, null)) * SQ3) ** 2]);
    }
    for (const b of this.buildings) {
      if (this.players[b.owner].team === myTeam) eyes.push([b.x, b.z, ((b.def.vision || 3) * SQ3) ** 2]);
    }
    for (const u of this.units) {
      if (u.inside) { u.visible = false; continue; }
      if (this.players[u.owner].team === myTeam) { u.visible = true; continue; }
      if (stealthy(u)) { u.visible = false; continue; }
      const inForest = u.mover === 'land' && this.cells[u.cell].t === 'forest';
      const forestR = (2 * SQ3 + 0.4) ** 2;
      let vis = false;
      for (const e of eyes) {
        const dx = u.x - e[0];
        const dz = u.z - e[1];
        if (dx * dx + dz * dz <= (inForest ? Math.min(e[2], forestR) : e[2])) { vis = true; break; }
      }
      u.visible = vis;
    }
  }

  updateBuilding(b, dt) {
```

```
    const def = b.def;
    // El trabajo de los constructores llega en workAcc; el Puesto avanzado se levanta solo.
    const work = b.type === 'outpost' || this.creator && b.owner === 0 && this.opts.instantBuild ? dt * 3 : b.workAcc;
    b.workAcc = 0;
    b.helped = work > 0;
    if (b.upgrading) {
      b.upgrading.progress += work / (def.build * 0.5);
      if (b.upgrading.progress >= 1) this.finishUpgrade(b);
    }
    if (!b.built) {
      b.progress += work / def.build;
      b.hp = Math.min(b.maxHp, b.hp + (b.maxHp * 0.9 * work) / def.build);
      if (b.progress >= 1) {
        b.progress = 1;
        b.built = true;
        this.players[b.owner].stats.built++;
        this.resetShield(b);
        this.emit('built', { b });
        if (b.owner === 0) {
          this.emit('toast', { text: `${def.name} ${def.fem ? 'terminada' : 'terminado'}`, kind: 'good' });
          this.emit('sound', { name: 'done' });
        }
      }
      return;
    }
    // Escudo propio: se recarga si nadie lo ataca durante 8 s.
    if (b.shield < b.shieldMax && this.time - b.lastHit > BLD_SHIELD.delay) {
      b.shield = Math.min(b.shieldMax, b.shield + b.shieldMax * BLD_SHIELD.regen * dt);
    }
    if (b.queue.length) {
      const q = b.queue[0];
      q.t += dt * this.prodRate(b);
      if (q.t >= UNITS[q.type].build) {
        b.queue.shift();
        this.spawnFrom(b, q.type);
      }
    }
    // v1.4: producción continua: con la cola vacía vuelve a poner la última tropa si alcanza el dinero,
    // la energía y la comunidad; si no, espera sin gastar.
    if (b.loop && b.lastType && !b.queue.length) {
      b.loopT = (b.loopT || 0) - dt;
      if (b.loopT <= 0) {
        b.loopT = LOOP_TRAIN.every;
        const err = this.enqueue(b, b.lastType);
        b.loopWait = err || null;
        if (!err) this.players[b.owner].stats.looped = (this.players[b.owner].stats.looped || 0) + 1;
      }
    }
    const w = this.weaponOf(b);
    if (w) {
      b.cd -= dt;
      b.scanT -= dt;
      if (b.scanT <= 0) {
        b.scanT = 0.3;
        b.autoTarget = this.acquire(b, w, 0);
        b.autoTarget2 = w.beams > 1 ? this.acquire(b, w, 0, b.autoTarget) : null;
      }
      const t = b.autoTarget;
      // AIR DEFENSE (v0.9): el radar fija cada blanco nuevo antes de lanzar.
      if (w.lock) {
        if (t !== b.lockOn) {
          b.lockOn = t && t.alive ? t : null;
          b.lockT = w.lock;
        }
        b.lockT -= dt;
      }
      if (b.salvoLeft > 0) {
        b.salvoT -= dt;
        const st = b.salvoTarget;
        if (b.salvoT <= 0) {
          if (st && st.alive && this.inRange(b, w, st)) this.fire(b, w, st, b.salvoLeft);
          b.salvoLeft--;
          b.salvoT = 0.22;
        }
      }
      if (t && t.alive && this.inRange(b, w, t)) {
        b.aimYaw = Math.atan2(t.x - b.x, t.z - b.z);
        if (b.cd <= 0 && !(w.lock && b.lockT > 0)) {
          if (w.salvo > 1) {
            b.salvoLeft = w.salvo - 1;
            b.salvoT = 0.22;
            b.salvoTarget = t;
            b.lastLaunch = this.time;
          }
          this.fire(b, w, t);
          // Núcleo nivel 3: segundo láser (a otro enemigo si lo hay).
          if (w.beams > 1) {
            const t2 = b.autoTarget2 && b.autoTarget2.alive && this.inRange(b, w, b.autoTarget2) ? b.autoTarget2 : t;
```

```
            if (t2.alive) this.fire(b, w, t2, 1);
          }
          b.cd = w.cd;
        }
      }
    }
    // Torreta nivel 2: 1 minigun; nivel 3: 2; nivel 4: 3 con más daño y tiro antiaéreo.
    const mg = def.minigun;
    if (mg && b.level > 1) {
      const guns = b.level - 1;
      const aa = def.aaLevel && b.level >= def.aaLevel;
      const mw = aa ? { ...mg, dmg: mg.dmg * 1.5, mult: { ...mg.mult, ...def.aaMiniMult }, proj: 'bullet' } : { ...mg,
proj: 'bullet' };
      for (let gi = 0; gi < guns; gi++) {
        b.miniCd[gi] = (b.miniCd[gi] || 0) - dt;
        if (b.miniCd[gi] > 0) continue;
        const mt = b.autoTarget && b.autoTarget.alive && this.canAttack(b, mw, b.autoTarget) && this.inRange(b, mw,
b.autoTarget)
          ? b.autoTarget : this.acquire(b, mw, 0);
        if (!mt) continue;
        this.fire(b, mw, mt, gi + 1);
        b.miniCd[gi] = mg.cd;
      }
    }
    const heal = this.healOf(b);
    if (heal) {
      b.healT -= dt;
      if (b.healT <= 0) {
        b.healT = 0.5;
        const rr = (heal.range * SQ3 + 0.4) ** 2;
        for (const u of this.units) {
          if (!u.alive || u.inside || this.isEnemy(u.owner, b.owner) || !heal.cls.includes(u.cls) || u.hp >= u.maxHp)
continue;
          const dx = u.x - b.x;
          const dz = u.z - b.z;
          if (dx * dx + dz * dz > rr) continue;
          u.hp = Math.min(u.maxHp, u.hp + (heal.rate ? heal.rate * 0.5 : heal.pct * u.maxHp * 0.5));
          if (Math.random() < 0.25) this.fx.push({ k: 'heal', x: u.x, y: u.y + 0.5, z: u.z });
        }
      }
    }
  }

  spawnFrom(b, type) {
    const def = UNITS[type];
    const ci = this.freeCellsNear(b.cell, def.mover, b.owner, 1)[0];
    const u = this.addUnit(b.owner, type, ci);
    const st = this.players[b.owner].stats;
    st.trained++;
    if (def.builds === 'land') st.builders = (st.builders || 0) + 1;
    if (b.rally >= 0) this.orderMove([u], b.rally, !!def.weapon);
    if (b.owner === 0) this.emit('sound', { name: 'ready' });
    this.emit('spawned', { u, b });
    return u;
  }

  followPath(u, dt) {
    if (u.pathI >= u.path.length) return true;
    const cells = this.cells;
    const c = cells[u.path[u.pathI]];
    if (u.mover !== 'air' && !this.passable(c, u.mover, u.owner)) {
      const d = u.dest;
      this.setPath(u, d);
      if (!u.path.length) return true;
      return false;
    }
    const last = u.pathI === u.path.length - 1;
    const tx = c.x + (last ? u.offX : u.offX * 0.5);
    const tz = c.z + (last ? u.offZ : u.offZ * 0.5);
    const dx = tx - u.x;
    const dz = tz - u.z;
    const d = Math.hypot(dx, dz);
    let sp = u.def.speed * SQ3;
    if (u.mover === 'land') sp /= TERRAIN[cells[u.cell].t].slow || 1;
    const step = sp * dt;
    if (d <= step) {
      u.x = tx;
      u.z = tz;
      u.pathI++;
    } else {
      u.x += (dx / d) * step;
      u.z += (dz / d) * step;
    }
    if (d > 0.01) u.moveYaw = Math.atan2(dx, dz);
    u.moving = true;
    const nc = this.map.atWorld(u.x, u.z);
    // v1.3: una tropa de tierra no pasa a «estar» en una casilla de agua (ni un barco en tierra)
    // aunque corte camino por el borde.
```

```
    if (nc && (u.mover === 'air' || TERRAIN[nc.t][u.mover === 'land' ? 'land' : 'water'])) u.cell = nc.i;
    return u.pathI >= u.path.length;
  }

  updateUnit(u, dt) {
    // Escudo personal: dentro de la Cúpula se recarga; al salir se pierde.
    if (this.unitShielded(u)) {
      if (!u.shieldMax) {
        u.shieldMax = u.maxHp * DOME_LEVELS[3].unitShield;
        u.shield = u.shieldMax;
      } else if (this.time - u.lastHit > UNIT_SHIELD.regenDelay) {
        u.shield = Math.min(u.shieldMax, u.shield + u.shieldMax * UNIT_SHIELD.regen * dt);
      }
    } else if (u.shield > 0 || u.shieldMax > 0) {
      u.shield = 0;
      u.shieldMax = 0;
    }
    if (u.inside) {
      u.x = u.inside.x;
      u.z = u.inside.z;
      u.cell = u.inside.cell;
      return;
    }
    u.moving = false;
    if (u.mover === 'air' && u.owner === 0) this.airAutoRepair(u, dt);
    const w = u.def.weapon;
    let ft = null;
    if (w) {
      u.cd -= dt;
      u.scanT -= dt;
      if (u.scanT <= 0) {
        u.scanT = 0.25 + Math.random() * 0.1;
        u.autoTarget = this.acquire(u, w, 0);
        // v1.4: «Mantener posición» no sale a perseguir; «Patrullar» pelea como «Avanzar atacando».
        // v1.5: después de no encontrar camino a un enemigo, sigue su camino unos segundos antes de buscar otro.
        u.aggroTarget = !u.autoTarget && !u.hold && !(u.aggroBlock > this.time) && (u.state === 'idle' || u.state ===
'amove' || u.state === 'patrol') ? this.acquire(u, w, 2) : null;
      }
      if (u.target && u.target.alive && this.inRange(u, w, u.target)) ft = u.target;
      else if (u.autoTarget && u.autoTarget.alive && this.inRange(u, w, u.autoTarget)) ft = u.autoTarget;
      if (ft) {
        u.aimYaw = Math.atan2(ft.x - u.x, ft.z - u.z);
        if (u.cd <= 0) {
          this.fire(u, w, ft);
          u.cd = w.cd;
          u.lastShot = this.time;
          u.lastShotKind = w.proj;
        }
      }
    }

    // Armas extra (v0.9): minigun de puerta del Helicóptero y del Apache, y misiles guiados del Apache.
    if (u.def.gun) this.fireExtra(u, u.def.gun, 'gunCd', dt, 'bullet');
    if (u.def.missile) this.fireExtra(u, u.def.missile, 'missileCd', dt, 'missile');
    // Médico y Portaaviones: curan a los suyos cerca (el Portaaviones también a los Cazas del hangar).
    const aura = u.def.healAura || (u.def.heal && u.kind === 'unit' ? u.def.heal : null);
    if (aura) this.healAround(u, aura, dt);

    switch (u.state) {
      case 'idle':
        if (u.def.repairs) {
          this.repairAround(u, dt);
          this.autoRepairScan(u, dt);
        }
        if (u.aggroTarget && u.aggroTarget.alive && u.def.weapon) {
          u.state = 'attack';
          u.target = u.aggroTarget;
          u.resume = -1;
          u.leashCell = u.cell;
          u.repathT = 0;
        }
        break;
      case 'move':
        if (this.followPath(u, dt)) u.state = 'idle';
        break;
      case 'patrol': {
        // v1.4: va y vuelve entre dos puntos, peleando con lo que encuentra.
        if (ft) break;
        if (u.aggroTarget && u.aggroTarget.alive) {
          u.state = 'attack';
          u.target = u.aggroTarget;
          u.repathT = 0;
          u.leashCell = -1;
          break;
        }
        const P = u.patrol;
        if (!P) { u.state = 'idle'; break; }
        if (this.followPath(u, dt)) {
```

```
          P.leg = 1 - P.leg;
          P.laps = (P.laps || 0) + 1;
          this.setPath(u, P.leg ? P.b : P.a);
          if (!u.path.length) { u.state = 'idle'; u.patrol = null; }
        }
        break;
      }
      case 'amove':
        if (ft) break;
        if (u.aggroTarget && u.aggroTarget.alive) {
          u.state = 'attack';
          u.target = u.aggroTarget;
          u.repathT = 0;
          u.leashCell = -1;
          break;
        }
        if (this.followPath(u, dt)) {
          u.state = 'idle';
          u.resume = -1;
        }
        break;
      case 'attack': {
        const t = u.target;
        const lost = !t || !t.alive || !w || !this.canAttack(u, w, t) ||
          (u.leashCell >= 0 && this.map.dist(this.cells[u.cell], this.cells[u.leashCell]) > 5);
        if (lost) {
          u.target = null;
          if (u.patrol) {
            u.state = 'patrol';
            this.setPath(u, u.patrol.leg ? u.patrol.b : u.patrol.a);
          } else if (u.resume >= 0) {
            u.state = 'amove';
            this.setPath(u, u.resume);
          } else {
            u.state = 'idle';
            u.path = [];
          }
          u.leashCell = -1;
          break;
        }
        if (this.inRange(u, w, t)) {
          u.path = [];
          u.pathI = 0;
          break;
        }
        u.repathT -= dt;
        if (u.repathT <= 0) {
          u.repathT = 1.0;
          this.setPath(u, t.cell);
          if (!u.path.length && u.mover !== 'air') {
            // Sin camino: se recuerda 6 s para no volver a elegirlo enseguida.
            const un = u.unreach || (u.unreach = {});
            for (const k of Object.keys(un)) if (un[k] <= this.time) delete un[k];
            un[t.id] = this.time + 6;
            if (u.aggroTarget === t) u.aggroTarget = null;
            if (u.patrol || u.resume >= 0) u.aggroBlock = this.time + 4;
            u.target = null;
            if (u.patrol) {
              u.state = 'patrol';
              this.setPath(u, u.patrol.leg ? u.patrol.b : u.patrol.a);
              break;
            }
            u.state = u.resume >= 0 ? 'amove' : 'idle';
            if (u.resume >= 0) this.setPath(u, u.resume);
            break;
          }
        }
        this.followPath(u, dt);
        break;
      }
      case 'build': {
        const b = u.buildTarget;
        if (!b || !this.needsWork(b)) {
          u.state = 'idle';
          u.buildTarget = null;
          u.path = [];
          break;
        }
        const bc = this.cells[b.cell];
        const uc = this.cells[u.cell];
        if (hexDist(uc.q, uc.r, bc.q, bc.r) <= 1) {
          // v1.0: la obra solo avanza con el constructor quieto y pegado a ella. En la casilla de al
          // lado, primero termina de acercarse (sin salir de su casilla); mientras camina, no avanza.
          u.path = [];
          const dx = b.x - u.x;
          const dz = b.z - u.z;
          const d = Math.hypot(dx, dz);
          if (d > BUILD_REACH) {
```

```
            const step = Math.min(d - BUILD_REACH + 0.01, u.def.speed * SQ3 * dt);
            u.x += (dx / d) * step;
            u.z += (dz / d) * step;
            u.moveYaw = Math.atan2(dx, dz);
            u.moving = true;
            break;
          }
          this.workOn(u, b, dt);
          break;
        }
        u.repathT -= dt;
        if (u.repathT <= 0) {
          u.repathT = 1.2;
          this.setPath(u, b.cell);
          if (!u.path.length && hexDist(uc.q, uc.r, bc.q, bc.r) > 1) {
            u.state = 'idle';
            u.buildTarget = null;
            break;
          }
        }
        this.followPath(u, dt);
        break;
      }
      case 'repair': {
        const b = u.repairTarget;
        if (!b || !b.alive || b.hp >= b.maxHp) {
          u.state = 'idle';
          u.repairTarget = null;
          u.path = [];
          break;
        }
        const bc = this.cells[b.cell];
        const uc = this.cells[u.cell];
        if (hexDist(uc.q, uc.r, bc.q, bc.r) <= 1) {
          u.path = [];
          this.repairBuilding(u, b, dt);
          break;
        }
        u.repathT -= dt;
        if (u.repathT <= 0) {
          u.repathT = 1;
          this.setPath(u, b.cell);
          // v1.5: sin camino hasta el edificio (rodeado o en otra isla): lo deja y no lo vuelve a intentar enseguida.
          if (!u.path.length) {
            this.markUnreach(u, b, 15);
            if (u.owner === 0 && !u.autoRep) this.toast(0, `${u.def.name}: no puede llegar a ${b.def.name} para
repararlo`, 'bad');
            u.state = 'idle';
            u.repairTarget = null;
            u.autoRep = false;
            break;
          }
        }
        this.followPath(u, dt);
        break;
      }
      case 'board': {
        const t = u.boardTarget;
        if (!t || !t.alive) {
          u.state = 'idle';
          break;
        }
        if (this.map.dist(this.cells[u.cell], this.cells[t.cell]) <= 1) {
          this.board(u, t);
          break;
        }
        u.repathT -= dt;
        if (u.repathT <= 0) {
          u.repathT = 1;
          this.setPath(u, t.cell);
          // Ya está lo más cerca que puede llegar a pie: que el transporte venga a buscarla.
          if (!u.path.length) this.callTransport(u, t);
        }
        this.followPath(u, dt);
        break;
      }
      case 'found': {
        const fc = this.cells[u.foundCell];
        const near = this.map.dist(this.cells[u.cell], fc) <= (u.mover === 'land' ? 0 : 1);
        const done = this.followPath(u, dt);
        if (near || done) {
          if (near || this.map.dist(this.cells[u.cell], fc) <= 1) this.tryFound(u);
          else {
            this.toast(u.owner, 'El ingeniero no puede llegar ahí', 'bad');
            u.state = 'idle';
          }
        }
        break;
```

```
      }
      default:
        break;
    }

    const cell = this.cells[u.cell];
    const gy = this.groundY(u, cell);
    u.y += (gy - u.y) * Math.min(1, dt * 10);
    // Quieta en formación en línea: mira hacia donde se la mandó.
    const face = u.faceYaw !== null && u.faceYaw !== undefined && u.state === 'idle' ? u.faceYaw : u.yaw;
    const want = u.moving ? u.moveYaw : ft && !u.def.turretOnly ? u.aimYaw : face;
    u.yaw = lerpAngle(u.yaw, want, Math.min(1, dt * 8));
  }

  fireExtra(u, w, key, dt, proj) {
    u[key] = (u[key] || 0) - dt;
    if (u[key] > 0) return;
    const ww = { ...w, proj: w.proj || proj };
    const t = u.autoTarget && u.autoTarget.alive && this.canAttack(u, ww, u.autoTarget) && this.inRange(u, ww,
u.autoTarget)
      ? u.autoTarget : this.acquire(u, ww, 0);
    if (!t) { u[key] = 0.25; return; }
    this.fire(u, ww, t, 1);
    u.lastShot = this.time;
    u.lastShotKind = proj;
    u[key] = w.cd;
  }

  healAround(u, aura, dt) {
    u.healT = (u.healT || 0) - dt;
    if (u.healT > 0) return;
    u.healT = 0.5;
    const rr = (aura.range * SQ3 + 0.4) ** 2;
    const heal = (t) => {
      const add = aura.rate ? aura.rate * 0.5 : aura.pct * t.maxHp * 0.5;
      t.hp = Math.min(t.maxHp, t.hp + add);
      if (Math.random() < 0.2) this.fx.push({ k: 'heal', x: t.x, y: t.y + 0.5, z: t.z });
    };
    for (const t of this.units) {
      if (t === u || !t.alive || t.inside || this.isEnemy(t.owner, u.owner) || !aura.cls.includes(t.cls) || t.hp >=
t.maxHp) continue;
      const dx = t.x - u.x;
      const dz = t.z - u.z;
      if (dx * dx + dz * dz <= rr) heal(t);
    }
    if (u.cargo) for (const t of u.cargo) if (t.alive && t.hp < t.maxHp && aura.cls.includes(t.cls)) heal(t);
  }

  // Tus aeronaves con menos del 30 % de vida vuelven solas a repararse (Helipuerto o Aeropuerto).
  airAutoRepair(u, dt) {
    u.repT = (u.repT || 0) - dt;
    if (u.repT > 0) return;
    u.repT = 1;
    if (u.repairing) {
      if (u.hp >= u.maxHp * 0.95) u.repairing = false;
      return;
    }
    if (u.hp >= u.maxHp * AIR_RETREAT) return;
    const pad = this.repairPadFor(u);
    if (!pad) return;
    u.repairing = true;
    this.orderMove([u], pad.cell);
    this.emit('toast', { text: `${u.def.name} vuelve a repararse`, kind: 'info', x: pad.x, z: pad.z });
  }

  repairPadFor(u) {
    let best = null;
    let bd = Infinity;
    for (const b of this.buildings) {
      if (!b.alive || !b.built || b.owner !== u.owner) continue;
      const h = b.def.heal;
      if (!h || !h.cls.includes(u.cls)) continue;
      const d = Math.hypot(b.x - u.x, b.z - u.z);
      if (d < bd) { bd = d; best = b; }
    }
    return best;
  }

  tryFound(u) {
    const why = this.foundInvalid(u.owner, u.foundCell, u.def.founds);
    const p = this.players[u.owner];
    if (why) {
      this.toast(u.owner, why, 'bad');
      u.state = 'idle';
      return false;
    }
    if (p.coins < BUILDINGS.outpost.cost) {
      // Espera en el sitio hasta reunir el dinero.
```

```
      if (!u.warnedCoins) {
        u.warnedCoins = true;
        this.toast(u.owner, 'El ingeniero espera: faltan monedas para el puesto (800)', 'bad');
      }
      return false;
    }
    p.coins -= BUILDINGS.outpost.cost;
    u.alive = false;
    this.addBuilding(u.owner, 'outpost', u.foundCell);
    if (u.owner === 0) {
      this.emit('toast', { text: 'Puesto avanzado fundado', kind: 'good' });
      this.emit('sound', { name: 'place' });
    }
    return true;
  }

  updateProjectiles(dt) {
    if (!this.projectiles.length) return;
    // v1.1: Escudos antimisiles con su defensa de punto lista.
    const pds = this.buildings.filter((b) => b.alive && b.built && b.def.pointDefense);
    for (const p of this.projectiles) {
      p.time += dt;
      if (p.kind === 'nuke') {
        this.updateNuke(p);
        continue;
      }
      if (pds.length && (p.kind === 'missile' || p.kind === 'rocket') && this.pointDefend(p, pds)) continue;
      if (p.t.alive && !p.miss) {
        p.tx = p.t.x;
        p.tz = p.t.z;
      }
      if (p.time >= p.dur) {
        p.done = true;
        if (p.t.alive && !p.miss) this.damage(p.t, p.dmg, p.owner, p.src);
        this.fx.push({ k: 'boom', x: p.tx, y: p.ty, z: p.tz, s: p.kind === 'torpedo' ? 0.7 : p.kind === 'bomb' ? 1.1 :
0.5, small: p.kind !== 'bomb' });
      }
    }
    this.projectiles = this.projectiles.filter((p) => !p.done);
  }

  // Defensa de punto (v1.1): derriba en el aire un misil o cohete enemigo cerca del Escudo antimisiles.
  pointDefend(p, pds) {
    const k = Math.min(1, p.time / p.dur);
    const x = p.sx + (p.tx - p.sx) * k;
    const z = p.sz + (p.tz - p.sz) * k;
    for (const b of pds) {
      if (!this.isEnemy(b.owner, p.owner) || this.time < (b.pdReadyAt || 0)) continue;
      const pd = b.def.pointDefense;
      if (Math.hypot(x - b.x, z - b.z) > pd.range * SQ3 + 0.5) continue;
      p.done = true;
      b.pdReadyAt = this.time + pd.cd;
      b.pdKills = (b.pdKills || 0) + 1;
      this.pdKills = (this.pdKills || 0) + 1;
      const y = (p.sy || 0.5) + ((p.ty || 0) - (p.sy || 0.5)) * k + 0.4;
      this.fx.push({ k: 'tracer', x1: b.x, y1: b.y + 1.3, z1: b.z, x2: x, y2: y, z2: z, color: 0xffe08a, heavy: true
});
      this.fx.push({ k: 'boom', x, y, z, s: 0.45, small: true });
      return true;
    }
    return false;
  }

  // ---------- misil nuclear (v0.9) ----------
  nukeInvalid(b, ci) {
    if (!b || !b.alive || b.type !== 'aadef' || b.level < 4 || !b.built) return 'Necesita un AIR DEFENSE de nivel 4';
    if (b.upgrading) return 'Termina primero la mejora';
    if (this.time < (b.nukeReadyAt || 0)) return `Recargando: faltan ${Math.ceil(b.nukeReadyAt - this.time)} s`;
    const p = this.players[b.owner];
    if (!(this.creator && b.owner === 0) && p.coins < NUKE.cost) return `Monedas insuficientes (${fmt(NUKE.cost)})`;
    if (ci !== undefined) {
      const c = this.cells[ci];
      if (!c) return 'Casilla no válida';
      if (this.map.dist(c, this.cells[b.cell]) > NUKE.range) return 'Demasiado lejos';
      if (this.map.dist(c, this.cells[p.core.cell]) <= NUKE.radius + 2) return 'Muy cerca de tu Núcleo';
    }
    return null;
  }

  // v1.0: salva de 3 misiles (los 3 AIR DEFENSE de nivel 4 a la vez).
  readySilos(pid) {
    return this.buildings.filter((b) => b.alive && b.owner === pid && b.type === 'aadef' && b.built && b.level >= 4 &&
      !b.upgrading && this.time >= (b.nukeReadyAt || 0));
  }

  salvoInvalid(pid, ci) {
    const silos = this.readySilos(pid);
    if (silos.length < 3) return `Necesitas 3 AIR DEFENSE de nivel 4 listos (tienes ${silos.length})`;
```

```
    const p = this.players[pid];
    if (!(this.creator && pid === 0) && p.coins < NUKE.cost * 3) return `Monedas insuficientes (${fmt(NUKE.cost * 3)})`;
    if (ci !== undefined) {
      for (const b of silos.slice(0, 3)) {
        const why = this.nukeInvalid(b, ci);
        if (why) return why;
      }
    }
    return null;
  }

  launchSalvo(pid, ci) {
    const why = this.salvoInvalid(pid, ci);
    if (why) return why;
    const silos = this.readySilos(pid).slice(0, 3);
    for (const b of silos) this.launchNuke(b, ci);
    this.salvos = (this.salvos || 0) + 1;
    return silos.length;
  }

  launchNuke(b, ci) {
    const why = this.nukeInvalid(b, ci);
    if (why) return why;
    const p = this.players[b.owner];
    if (!(this.creator && b.owner === 0)) p.coins -= NUKE.cost;
    b.nukeReadyAt = this.time + NUKE.cooldown;
    const c = this.cells[ci];
    this.projectiles.push({
      kind: 'nuke', t: null, dmg: NUKE.dmg, owner: b.owner, src: b, sx: b.x, sy: b.y + 1, sz: b.z, tx: c.x, ty: c.h, tz: c.z,
      time: 0, dur: NUKE.flight, color: PLAYER_COLORS[b.owner], cell: ci, checked: false,
    });
    this.nukesLaunched = (this.nukesLaunched || 0) + 1;
    this.emit('sound', { name: 'alarm' });
    // Aviso a quien tenga algo cerca del blanco (y siempre a quien lo lanza).
    const R = NUKE.radius + 2;
    for (const q of this.players) {
      if (!q.alive) continue;
      const mine = q.id === b.owner;
      const near = this.buildings.some((x) => x.alive && x.owner === q.id && this.map.dist(this.cells[x.cell], c) <= R) ||
        this.units.some((u) => u.alive && u.owner === q.id && this.map.dist(this.cells[u.cell], c) <= R);
      if (q.id === 0 && (mine || near)) {
        this.emit('toast', { text: mine ? `Misil nuclear lanzado: llega en ${NUKE.flight} s` : `¡Misil nuclear en camino! Llega en ${NUKE.flight} s`, kind: mine ? 'info' : 'bad', x: c.x, z: c.z });
        this.emit('ping', { x: c.x, z: c.z, kind: 'bad' });
      }
      // Una sola frase aunque lleguen varios a la vez (salva de 3, v1.0).
      if (!q.human && near && !mine && q.persona && this.time - (q.nukeSaidAt ?? -99) > 2) {
        q.nukeSaidAt = this.time;
        this.say(q.id, '¡Un misil nuclear! ¡Todos a cubierto!', { force: true, public: true });
      }
    }
    return null;
  }

  updateNuke(p) {
    const k = p.time / p.dur;
    // Escudo antimisiles enemigo cerca del blanco: lo derriba a mitad de camino.
    if (!p.checked && k >= 0.6) {
      p.checked = true;
      const tc = this.cells[p.cell];
      const shield = this.buildings.find((b) => b.alive && b.built && b.def.intercept && this.isEnemy(b.owner, p.owner) &&
        this.time >= (b.chargeAt || 0) && this.map.dist(this.cells[b.cell], tc) <= b.def.intercept.range);
      if (shield) {
        shield.chargeAt = this.time + shield.def.intercept.recharge;
        p.done = true;
        const x = p.sx + (p.tx - p.sx) * k;
        const z = p.sz + (p.tz - p.sz) * k;
        this.fx.push({ k: 'boom', x, y: 4, z, s: 1.4 });
        this.nukesIntercepted = (this.nukesIntercepted || 0) + 1;
        if (shield.owner === 0 || p.owner === 0) {
          this.emit('toast', { text: shield.owner === 0 ? '¡Tu Escudo antimisiles derribó el misil nuclear!' : 'El Escudo antimisiles enemigo derribó tu misil', kind: shield.owner === 0 ? 'good' : 'bad' });
        }
        return;
      }
    }
    if (p.time < p.dur) return;
    p.done = true;
    const R = NUKE.radius * SQ3 + 0.6;
    this.fx.push({ k: 'nuke', x: p.tx, y: p.ty, z: p.tz });
    this.emit('sound', { name: 'bigboom', x: p.tx, z: p.tz });
    const hits = [...this.units.filter((u) => u.alive && !u.inside), ...this.buildings.filter((b) => b.alive)];
    for (const t of hits) {
      // v1.3: solo daña a los enemigos de quien lo lanzó (igual que la radiación).
```

```
      if (!this.isEnemy(p.owner, t.owner)) continue;
      const d = Math.hypot(t.x - p.tx, t.z - p.tz);
      if (d > R) continue;
      this.damage(t, p.dmg * (1 - (d / R) * 0.6), p.owner, null, true);
    }
    // v1.0: queda 1 minuto de radiación que sigue quemando a los enemigos de quien lo lanzó.
    if (!this.radZones) this.radZones = [];
    this.radZones.push({ x: p.tx, y: p.ty, z: p.tz, cell: p.cell, owner: p.owner, start: this.time, until: this.time +
NUKE.rad.time, tick: 0 });
    const zc = this.cells[p.cell];
    const near0 = this.buildings.some((b) => b.alive && b.owner === 0 && this.map.dist(this.cells[b.cell], zc) <=
NUKE.radius + 2);
    if (p.owner === 0 || near0) {
      this.emit('toast', { text: `Radiación por ${NUKE.rad.time} s donde cayó el misil`, kind: p.owner === 0 ? 'info' :
'bad', x: p.tx, z: p.tz });
    }
  }

  // Gráficas del final (v1.1): una muestra cada 10 s (a lo sumo 240; si sobran, se juntan de a dos).
  recordHistory(dt) {
    if (!this.history) this.history = [];
    this._histT = (this._histT || 0) - dt;
    if (this._histT > 0) return;
    this._histT = 10;
    const army = this.players.map(() => 0);
    for (const u of this.units) if (u.alive && u.def.weapon) army[u.owner]++;
    this.history.push({ t: Math.round(this.time), inc: this.players.map((p) => (p.alive ? p.income : 0)), army });
    if (this.history.length > 240) this.history = this.history.filter((_, i) => i % 2 === 0);
  }

  // Radiación (v1.0): cada medio segundo quema a tropas y edificios enemigos dentro de la zona.
  updateRadiation(dt) {
    if (!this.radZones || !this.radZones.length) return;
    const R = NUKE.radius * SQ3 + 0.6;
    for (const z of this.radZones) {
      z.tick += dt;
      if (z.tick < 0.5) continue;
      const step = z.tick;
      z.tick = 0;
      const burn = (t, pct) => {
        if (!t.alive || t.inside || !this.isEnemy(z.owner, t.owner)) return;
        if (Math.hypot(t.x - z.x, t.z - z.z) > R) return;
        this.damage(t, t.maxHp * pct * step, z.owner, null);
      };
      for (const u of this.units) burn(u, NUKE.rad.unit);
      for (const b of this.buildings) burn(b, b.type === 'core' ? NUKE.rad.core : NUKE.rad.bld);
      this.fx.push({ k: 'rad', x: z.x, y: z.y, z: z.z, r: R });
    }
    this.radZones = this.radZones.filter((z) => this.time < z.until);
  }
}
```

faro/src/js/045-save.js · 250 líneas

```
'use strict';
// Guardar y cargar una partida (punto 16: si sales sin querer, puedes volver).
// Convierte todo el estado del juego en JSON: las referencias entre objetos (un edificio que
// apunta a su objetivo, el Núcleo de cada jugador...) se guardan como números, y los datos de
// configuración (BUILDINGS, UNITS...) como su nombre, para no copiarlos.

const SAVE_KEY = 'faro-save-v1';
const SAVE_VERSION = 1;

const Save = {
  _paths: null,
  _byPath: null,

  // Recorre la configuración una vez y anota la ruta de cada objeto (por ejemplo «BUILDINGS.core»).
  _consts() {
    if (this._paths) return this._paths;
    const paths = new Map();
    const byPath = new Map();
    const roots = {
      BUILDINGS, UNITS, MODES, MAPS, DIFFICULTY, POINTS, PERSONALITIES, TERRAIN, DOME_LEVELS, UNIT_SHIELD,
      HOUSE_LEVELS, DOME, EMERGENCY, REPAIR, CAPTURE, PALETTES, BUILD_TABS, OBJECTIVES, PLAYER_COLORS,
      PLAYER_NAMES, LEVEL_UP_COST, PERSONALITY_ORDER, ALLY_LINES, CAT_NAMES, CAMPAIGN,
    };
    const walk = (o, path, depth) => {
      if (!o || typeof o !== 'object' || paths.has(o) || depth > 6) return;
      paths.set(o, path);
      byPath.set(path, o);
      for (const k of Object.keys(o)) walk(o[k], path + '.' + k, depth + 1);
    };
    for (const [name, obj] of Object.entries(roots)) walk(obj, name, 0);
    this._paths = paths;
    this._byPath = byPath;
    return paths;
  },

  // v1.3: guardado más chico y rápido (cada 60 s). El mapa no se guarda: se vuelve a crear con su
  // semilla al cargar (una suma de control comprueba que sea el mismo). Las casillas solo guardan lo
  // que no se puede recalcular; los efectos visuales y avisos ya mostrados tampoco se guardan.
  MAP_STATIC: ['i', 'q', 'r', 'x', 'z', 't', 'h', 'land', 'nb', 'comp', 'bridge'],
  // Datos de las casillas que se vuelven a poner al cargar: territorio (se recalcula), edificio y punto.
  CELL_REBUILT: ['owner', 'ownerD', 'ownerId', 'bld', 'point'],
  GAME_TRANSIENT: ['fx', 'events', 'removed'],

  mapCheck(cells) {
    let h = 2166136261;
    for (const c of cells) {
      const str = `${c.q},${c.r},${c.t},${Math.round(c.h * 100)},${c.land ? 1 : 0},${c.comp},${c.bridge ? 1 : 0};`;
      for (let k = 0; k < str.length; k++) {
        h ^= str.charCodeAt(k);
        h = Math.imul(h, 16777619);
      }
    }
    return h >>> 0;
  },

  serialize(game) {
    const consts = this._consts();
    const ids = new Map();
    const objs = [];
    const map = game.map;
    const compact = map instanceof GameMap && map.cells === game.cells;
    const cellIdx = compact ? new Map(map.cells.map((c, i) => [c, i])) : null;
    const skipGame = new Set(this.GAME_TRANSIENT);
    if (compact) skipGame.add('cells');
    const skipMap = new Set(['cells', 'lookup']);
    const classes = new Map([[Game.prototype, 'Game'], [Bot.prototype, 'Bot'], [GameMap.prototype, 'GameMap'],
[MinHeap.prototype, 'MinHeap']]);
    const enc = (v) => {
      if (typeof v === 'function' || v === undefined) return undefined;
      if (typeof v === 'number' && !Number.isFinite(v)) return { $n: String(v) };
      if (v === null || typeof v !== 'object') return v;
      const cp = consts.get(v);
      if (cp) return { $c: cp };
      const known = ids.get(v);
      if (known !== undefined) return { $r: known };
      if (cellIdx && cellIdx.has(v)) {
        // Referencia a una casilla: al cargar apunta a la casilla recreada.
        const cid = objs.length;
        ids.set(v, cid);
        objs.push({ cell: cellIdx.get(v) });
        return { $r: cid };
      }
      const id = objs.length;
      ids.set(v, id);
```

```
    objs.push(null);
    let rec;
    if (Array.isArray(v)) rec = { a: v.map((x) => { const e = enc(x); return e === undefined ? null : e; }) };
    else if (v instanceof Map) rec = { m: [...v].map(([k, x]) => [enc(k), enc(x)]) };
    else if (v instanceof Set) rec = { s: [...v].map(enc) };
    else if (ArrayBuffer.isView(v)) rec = { t: v.constructor.name, n: v.length };
    else {
      const o = {};
      const skip = v === game ? skipGame : compact && v === map ? skipMap : null;
      for (const k of Object.keys(v)) {
        if (skip && skip.has(k)) continue;
        const e = enc(v[k]);
        if (e !== undefined) o[k] = e;
      }
      rec = { o };
      const cls = classes.get(Object.getPrototypeOf(v));
      if (cls) rec.k = cls;
      if (compact && v === map) {
        // Datos de casillas que no se recalculan solos (si los hay): se guardan aparte.
        const fixed = new Set([...this.MAP_STATIC, ...this.CELL_REBUILT]);
        const extra = [];
        map.cells.forEach((c, i) => {
          const ks = Object.keys(c).filter((k) => !fixed.has(k));
          if (!ks.length) return;
          const eo = {};
          for (const k of ks) {
            const e = enc(c[k]);
            if (e !== undefined) eo[k] = e;
          }
          extra.push([i, eo]);
        });
        // La suma de control se calcula una vez por mapa (sus casillas fijas no cambian en la partida).
        if (!this._checks) this._checks = new WeakMap();
        let check = this._checks.get(map);
        if (check === undefined) {
          check = this.mapCheck(map.cells);
          this._checks.set(map, check);
        }
        rec.gen = { seed: map.seed, kind: map.kind, n: map.n, pairs: !!map.pairs, check, extra };
      }
    }
    objs[id] = rec;
    return { $r: id };
  };
  enc(game);
  let maxId = 0;
  for (const e of game.units) maxId = Math.max(maxId, e.id || 0);
  for (const e of game.buildings) maxId = Math.max(maxId, e.id || 0);
  return { v: SAVE_VERSION, nextId: Math.max(maxId + 1, _nextId), objs };
},

deserialize(data) {
  if (!data || data.v !== SAVE_VERSION || !Array.isArray(data.objs)) throw new Error('Guardado no válido');
  this._consts();
  const byPath = this._byPath;
  const classes = { Game, Bot, GameMap, MinHeap };
  const typed = { Float32Array, Int32Array, Uint32Array, Uint8Array, Int16Array, Uint16Array, Float64Array };
  let fresh = null;
  const genRec = data.objs.find((r) => r && r.gen);
  if (genRec) {
    const g0 = genRec.gen;
    fresh = new GameMap(g0.seed, g0.kind, g0.n, g0.pairs);
    if (this.mapCheck(fresh.cells) !== g0.check) throw new Error('El mapa no coincide con esta versión del juego');
  }
  const made = data.objs.map((r) => {
    if (r.cell !== undefined) return fresh.cells[r.cell];
    if (r.a) return [];
    if (r.m) return new Map();
    if (r.s) return new Set();
    // Los arreglos numéricos son memoria de trabajo del buscador de caminos: se crean vacíos.
    if (r.t) return new (typed[r.t] || Float32Array)(r.n);
    return r.k && classes[r.k] ? Object.create(classes[r.k].prototype) : {};
  });
  const dec = (v) => {
    if (v === null || typeof v !== 'object') return v;
    if (v.$r !== undefined) return made[v.$r];
    if (v.$c !== undefined) {
      if (!byPath.has(v.$c)) throw new Error('Dato de configuración desconocido: ' + v.$c);
      return byPath.get(v.$c);
    }
    if (v.$n !== undefined) return Number(v.$n);
    return v;
  };
  data.objs.forEach((r, i) => {
    if (r.cell !== undefined) return;
    const t = made[i];
    if (r.a) r.a.forEach((x) => t.push(dec(x)));
    else if (r.m) r.m.forEach(([k, x]) => t.set(dec(k), dec(x)));
```

```
      else if (r.s) r.s.forEach((x) => t.add(dec(x)));
      else if (r.o) for (const k of Object.keys(r.o)) t[k] = dec(r.o[k]);
    });
    const game = made[0];
    if (!(game instanceof Game)) throw new Error('Guardado no válido');
    _nextId = Math.max(_nextId, data.nextId || 1);
    game.events = [];
    if (!game.fx) game.fx = [];
    if (!game.removed) game.removed = [];
    if (genRec) {
      // Mapa recreado: casillas nuevas, con sus edificios, puntos y datos extra.
      game.map.cells = fresh.cells;
      game.map.lookup = fresh.lookup;
      game.cells = fresh.cells;
      for (const [ci, eo] of genRec.gen.extra || []) for (const k of Object.keys(eo)) fresh.cells[ci][k] = dec(eo[k]);
      for (const c of fresh.cells) { c.bld = null; delete c.point; }
      for (const b of game.buildings) if (b.alive) fresh.cells[b.cell].bld = b;
      for (const pt of game.points || []) fresh.cells[pt.cell].point = pt;
    }
    // Los datos de cada edificio y tropa vuelven a apuntar a la configuración actual.
    for (const b of game.buildings) if (BUILDINGS[b.type]) b.def = BUILDINGS[b.type];
    for (const u of game.units) if (UNITS[u.type]) u.def = UNITS[u.type];
    // Partidas guardadas antes de la v1.1: equipo de origen para las alianzas.
    for (const p of game.players) if (p.team0 === undefined) p.team0 = p.team;
    if (!game.radZones) game.radZones = [];
    game.territoryDirty = true;
    return game;
  },

  // Datos para mostrar en el vestíbulo sin cargar toda la partida.
  summary(game) {
    const me = game.players[0];
    return {
      mode: game.mode.name, map: (MAPS[game.opts.mapKind] || {}).name || '', time: Math.round(game.time),
      alive: game.players.filter((p) => p.alive).length, players: game.players.length,
      units: game.units.filter((u) => u.owner === 0 && u.alive).length, coins: Math.round(me.coins),
      savedAt: Date.now(), creator: !!game.creator,
      ranked: !!(game.mode.ranked && !game.opts.custom),
    };
  },

  write(game) {
    if (!game || game.over || game.preview) return false;
    try {
      const data = this.serialize(game);
      data.summary = this.summary(game);
      localStorage.setItem(SAVE_KEY, JSON.stringify(data));
      return true;
    } catch (err) {
      console.warn('[Guardado] No se pudo guardar la partida:', err && err.message);
      return false;
    }
  },

  peek() {
    try {
      const raw = localStorage.getItem(SAVE_KEY);
      if (!raw) return null;
      const data = JSON.parse(raw);
      return data && data.v === SAVE_VERSION ? data.summary || {} : null;
    } catch (err) {
      return null;
    }
  },

  read() {
    try {
      const raw = localStorage.getItem(SAVE_KEY);
      return raw ? this.deserialize(JSON.parse(raw)) : null;
    } catch (err) {
      console.warn('[Guardado] No se pudo cargar la partida:', err && err.message);
      return null;
    }
  },

  clear() {
    try { localStorage.removeItem(SAVE_KEY); } catch (err) { /* sin almacenamiento */ }
  },
};
```

faro/src/js/046-campaign.js · 291 líneas

```
'use strict';
// Campaña «La ruta del Faro» (v0.7): prepara cada misión, sigue su meta y calcula las estrellas.
// El estado de la misión vive en game.camp (solo datos simples, para que se guarde con la partida).

const Campaign = {
  def(id) {
    return CAMPAIGN.find((m) => m.id === id) || null;
  },

  // Modo de juego de una misión: todos contra todos. Las monedas extra son solo para ti.
  mode(id) {
    const m = this.def(id);
    const teams = [];
    for (let i = 0; i < m.players; i++) teams.push(i);
    return { name: `Misión ${m.id} · ${m.name}`, players: m.players, teams, campaign: m.id };
  },

  // Opciones para empezar una misión: mapa y semilla fijos, así siempre es la misma isla.
  opts(id) {
    const m = this.def(id);
    return { mode: 'campaign', mission: id, mapKind: m.mapKind, difficulty: m.difficulty, seed: m.seed, personas:
m.personas.slice() };
  },

  // Una misión está abierta si es la primera, si terminaste la anterior o en modo creador.
  unlocked(id, P, creator) {
    if (creator || id === 1) return true;
    return !!(P && P.campaign && P.campaign.done[id - 1]);
  },

  goalText(m) {
    const g = m.goal;
    if (g.kind === 'eco') return `Llega a ${fmt(g.income)} monedas por minuto y ten ${g.army} tropas de combate.`;
    if (g.kind === 'survive') return `Aguanta ${Math.round(g.time / 60)} minutos con tu Núcleo en pie.`;
    if (g.kind === 'hold') return `Toma el Faro y mantenlo ${g.time} s en total.`;
    if (g.kind === 'flags') return `Captura ${g.count} Banderas de otras islas.`;
    return m.players > 2 ? 'Sé el último Núcleo en pie.' : 'Destruye el Núcleo enemigo.';
  },

  starText(s) {
    if (s.kind === 'time') return `En menos de ${Math.round(s.max / 60)} min`;
    if (s.kind === 'lostBld') return s.max === 0 ? 'Sin perder edificios' : `Pierdes ${s.max} edificios o menos`;
    if (s.kind === 'lostUnits') return `Pierdes ${s.max} tropas o menos`;
    if (s.kind === 'coreEnd') return `Núcleo sobre el ${Math.round(s.min * 100)} % al terminar`;
    return `Tu Núcleo nunca baja del ${Math.round(s.min * 100)} %`;
  },

  // ---------- preparación ----------
  setup(g) {
    const m = this.def(g.opts.mission);
    const me = g.players[0];
    g.camp = {
      id: m.id, lostBld: 0, minCore: 1, faroHold: 0, flags: [], won: false, checkT: 0,
      homeComp: g.cells[me.core.cell].comp,
    };
    if (m.coins) me.coins = m.coins;
    for (const bot of g.bots) {
      if (m.passive) bot.passive = true;
      if (m.firstWave) bot.nextWaveAt = m.firstWave + Math.random() * 15;
      if (m.waveEvery) bot.waveMult = m.waveEvery;
      if (m.enemyStart) this.giveStart(g, bot.pid, m.enemyStart);
      if (m.fortress) this.fortify(g, bot.pid);
      if (m.guardFlags) this.guardFlags(g, bot.pid, m.guardFlags);
      if (m.guardFaro) this.guardPoints(g, bot.pid, g.points.filter((pt) => pt.type === 'faro'), m.guardFaro.units);
    }
    if (m.myStart) this.giveStart(g, 0, m.myStart);
    g.recomputeTerritory();
    g.refreshEconomy();
  },

  // Casilla libre más cercana al Núcleo donde cabe ese edificio (en su territorio o en agua libre).
  spotFor(g, pid, type) {
    const core = g.cells[g.players[pid].core.cell];
    let best = -1;
    let bd = Infinity;
    for (const c of g.cells) {
      if (c.bld || c.point || (c.owner >= 0 && c.owner !== pid)) continue;
      const d = g.map.dist(c, core);
      if (d > 8 || d < 2) continue;
      const why = g.placeInvalid(pid, type, c.i, true);
      if (why && !(why === 'Fuera de tu territorio' && c.owner < 0 && type === 'harbor')) continue;
      if (type === 'harbor' && (c.t !== 'shallow' || !g.map.isCoast(c))) continue;
      if (d < bd) { bd = d; best = c.i; }
    }
```

```
    return best;
  },

  giveStart(g, pid, spec) {
    const p = g.players[pid];
    let harbor = null;
    for (const type of spec.buildings || []) {
      const ci = this.spotFor(g, pid, type);
      if (ci < 0) continue;
      const b = g.addBuilding(pid, type, ci, true);
      if (type === 'harbor') harbor = b;
      g.recomputeTerritory();
    }
    const sea = (spec.units || []).filter((t) => UNITS[t].mover === 'sea');
    const land = (spec.units || []).filter((t) => UNITS[t].mover !== 'sea');
    if (sea.length && harbor) {
      const cells = g.freeCellsNear(harbor.cell, 'sea', pid, sea.length);
      sea.forEach((t, k) => { if (cells[k] !== undefined) g.addUnit(pid, t, cells[k]); });
    }
    const cells = g.freeCellsNear(p.core.cell, 'land', pid, land.length + 2);
    land.forEach((t, k) => { if (cells[k + 2] !== undefined) g.addUnit(pid, t, cells[k + 2]); });
  },

  // Cruzar el mar: el rival ya tiene las Banderas de otras islas más cercanas a ti, con guardias.
  guardFlags(g, pid, spec) {
    const mine = g.cells[g.players[0].core.cell];
    const flags = g.points.filter((pt) => pt.type === 'flag' && g.cells[pt.cell].comp !== g.camp.homeComp)
      .sort((a, b) => g.map.dist(g.cells[a.cell], mine) - g.map.dist(g.cells[b.cell], mine)).slice(0, spec.count);
    this.guardPoints(g, pid, flags, spec.units);
  },

  // El rival empieza dueño de esos puntos, con guardias a su lado.
  guardPoints(g, pid, points, units) {
    for (const pt of points) {
      pt.owner = pid;
      pt.prog = 1;
      pt.capTeam = g.players[pid].team;
      const cells = g.freeCellsNear(pt.cell, 'land', pid, units.length);
      units.forEach((t, k) => {
        const u = g.addUnit(pid, t, cells[k]);
        u.role = 'guard';
      });
    }
    g.territoryDirty = true;
  },

  // Asedio: Núcleo en nivel 2, arco de murallas, Torretas de nivel 3 y 2 y AIR DEFENSE detrás.
  fortify(g, pid) {
    const core = g.players[pid].core;
    const cc = g.cells[core.cell];
    const mine = g.cells[g.players[0].core.cell];
    const dir = Math.atan2(mine.z - cc.z, mine.x - cc.x);
    const off = (c) => {
      let a = Math.atan2(c.z - cc.z, c.x - cc.x) - dir;
      while (a > Math.PI) a -= Math.PI * 2;
      while (a < -Math.PI) a += Math.PI * 2;
      return Math.abs(a);
    };
    const free = (c) => !c.bld && !c.point && TERRAIN[c.t].land && !['gold', 'fertile', 'oilland'].includes(c.t);
    const ring = (d) => g.cells.filter((c) => g.map.dist(c, cc) === d && free(c)).sort((a, b) => off(a) - off(b));
    const level = (b, lv) => {
      if (lv <= 1) return;
      b.upgrading = { level: lv, progress: 1 };
      g.finishUpgrade(b);
    };
    level(core, 2);
    // Muralla en arco (v0.9: también sobre tierra fértil, que ahora ocupa más casillas).
    const wallRing = g.cells.filter((c) => g.map.dist(c, cc) === 3 && !c.bld && !c.point && TERRAIN[c.t].land && !
['gold', 'oilland'].includes(c.t));
    // Los 5 tramos más de frente, como en la v0.7.
    wallRing.filter((c) => off(c) < 1.3).sort((a, b) => off(a) - off(b)).slice(0, 5).forEach((c) => g.addBuilding(pid,
'wall', c.i, true));
    const r2 = ring(2);
    const picks = [];
    const take = (ok) => {
      const c = r2.find((x) => !picks.includes(x) && ok(x) && picks.every((p) => g.map.dist(p, x) >= 2));
      if (c) picks.push(c);
      return c;
    };
    const front = take(() => true);
    if (front) level(g.addBuilding(pid, 'turret', front.i, true), 3);
    for (let k = 0; k < 2; k++) {
      const c = take((x) => off(x) > 0.45 && off(x) < 2);
      if (c) level(g.addBuilding(pid, 'turret', c.i, true), 2);
    }
    const back = [...r2].reverse().find((x) => !picks.includes(x) && off(x) > 1.8);
    if (back) g.addBuilding(pid, 'aadef', back.i, true);
  },
```

```
170
171   // ---------- durante la partida ----------
172   update(g, dt) {
173     const c = g.camp;
174     if (!c || g.over || g.preview) return;
175     const core = g.players[0].core;
176     if (core.alive) c.minCore = Math.min(c.minCore, core.hp / core.maxHp);
177     c.checkT -= dt;
178     if (c.checkT > 0) return;
179     const step = 0.25 - c.checkT;
180     c.checkT = 0.25;
181     const m = this.def(c.id);
182     if (m.goal.kind === 'hold') {
183       const faro = g.points.find((pt) => pt.type === 'faro');
184       if (faro && faro.owner === 0) c.faroHold += step;
185     } else if (m.goal.kind === 'flags') {
186       for (const pt of g.points) {
187         if (pt.type === 'flag' && pt.owner === 0 && g.cells[pt.cell].comp !== c.homeComp && !c.flags.includes(pt.id)) {
188           c.flags.push(pt.id);
189           g.emit('toast', { text: `Bandera de otra isla: ${c.flags.length} de ${m.goal.count}`, kind: 'good' });
190         }
191       }
192     }
193     if (this.goalMet(g)) this.win(g);
194   },
195
196   army(g) {
197     return g.units.filter((u) => u.alive && u.owner === 0 && u.def.weapon).length;
198   },
199
200   goalMet(g) {
201     const c = g.camp;
202     const goal = this.def(c.id).goal;
203     const me = g.players[0];
204     if (!me.alive) return false;
205     if (goal.kind === 'eco') return me.income >= goal.income && this.army(g) >= goal.army;
206     if (goal.kind === 'survive') return g.time >= goal.time;
207     if (goal.kind === 'hold') return c.faroHold >= goal.time;
208     if (goal.kind === 'flags') return c.flags.length >= goal.count;
209     return false; // destroy: lo decide checkEnd cuando cae el último Núcleo rival
210   },
211
212   win(g) {
213     if (g.over) return;
214     g.over = true;
215     g.result = 'victory';
216     g.camp.won = true;
217     g.emit('over', { result: 'victory', mission: true });
218   },
219
220   // Líneas de la meta con su progreso (para el panel de objetivos).
221   goalLines(g) {
222     const c = g.camp;
223     const m = this.def(c.id);
224     const goal = m.goal;
225     const me = g.players[0];
226     if (goal.kind === 'eco') {
227       const army = this.army(g);
228       return [
229         { text: `Ingresos: ${fmt(me.income)} / ${fmt(goal.income)} por min`, done: me.income >= goal.income },
230         { text: `Tropas de combate: ${Math.min(army, goal.army)} / ${goal.army}`, done: army >= goal.army },
231       ];
232     }
233     if (goal.kind === 'survive') {
234       return [{ text: `Aguanta: ${fmtTime(Math.min(g.time, goal.time))} / ${fmtTime(goal.time)}`, done: g.time >=
goal.time }];
235     }
236     if (goal.kind === 'hold') {
237       const faro = g.points.find((pt) => pt.type === 'faro');
238       const now = faro && faro.owner === 0 ? ' · ¡es tuyo!' : '';
239       return [{ text: `Faro en tu poder: ${Math.floor(Math.min(c.faroHold, goal.time))} / ${goal.time} s${now}`, done:
c.faroHold >= goal.time }];
240     }
241     if (goal.kind === 'flags') {
242       return [{ text: `Banderas de otras islas: ${Math.min(c.flags.length, goal.count)} / ${goal.count}`, done:
c.flags.length >= goal.count }];
243     }
244     const rivals = g.players.filter((p) => !p.human);
245     const down = rivals.filter((p) => !p.alive).length;
246     if (rivals.length === 1) return [{ text: down ? 'Núcleo enemigo destruido' : 'Destruye el Núcleo enemigo', done:
down > 0 }];
247     return [{ text: `Núcleos rivales caídos: ${down} / ${rivals.length}`, done: down >= rivals.length }];
248   },
249
250   // Estrellas: la primera por terminar y dos por condiciones. final = al acabar la partida.
251   stars(g) {
252     const c = g.camp;
253     const m = this.def(c.id);
```

```
    const me = g.players[0];
    const won = c.won || (g.over && g.result === 'victory');
    const core = me.core;
    const coreK = core && core.alive ? core.hp / core.maxHp : 0;
    const out = [{ text: 'Termina la misión', ok: won, live: true, why: won ? '' : 'No se cumplió la meta' }];
    for (const s of m.stars) {
      let ok;
      let live;
      let why;
      let now;
      if (s.kind === 'time') {
        live = g.time < s.max;
        ok = won && live;
        now = live ? `quedan ${fmtTime(s.max - g.time)}` : 'se acabó el tiempo';
        why = `Tardaste ${fmtTime(g.time)}`;
      } else if (s.kind === 'lostBld' || s.kind === 'lostUnits') {
        const n = s.kind === 'lostBld' ? c.lostBld : me.stats.lost;
        live = n <= s.max;
        ok = won && live;
        now = `llevas ${n}`;
        why = `Perdiste ${n} ${s.kind === 'lostBld' ? (n === 1 ? 'edificio' : 'edificios') : (n === 1 ? 'tropa' :
'tropas')}`;
      } else if (s.kind === 'coreEnd') {
        live = coreK >= s.min;
        ok = won && live;
        now = `ahora ${Math.round(coreK * 100)} %`;
        why = `Tu Núcleo terminó al ${Math.round(coreK * 100)} %`;
      } else {
        live = c.minCore >= s.min;
        ok = won && live;
        now = `mínimo ${Math.round(c.minCore * 100)} %`;
        why = `Tu Núcleo bajó al ${Math.round(c.minCore * 100)} %`;
      }
      if (!won && live) why = 'Hay que terminar la misión';
      out.push({ text: this.starText(s), ok, live, now, why: ok ? '' : why });
    }
    return out;
  },
};
```

faro/src/js/05-ai.js · 1,279 líneas

```
'use strict';
// Bots (NPC): cada uno tiene una personalidad. Construyen economía, se expanden, planifican un
// ejército mixto que contrarresta al enemigo, agrupan sus oleadas, se retiran heridos, reparan,
// capturan puntos (también cruzando el mar) y hablan con el jugador.

const unitCat = (u) => (u.cls === 'ship' || u.cls === 'sub' ? 'sea' : u.cls);
// Tropas rápidas para hostigar la economía enemiga (v0.8).
// Orden de preferencia: Jeep, Lancha, Ranger (sin Fábrica ni Puerto) y Helicóptero.
const FAST_UNITS = { jeep: 'veh', speedboat: 'sea', ranger: 'inf', heli: 'air' };
const ECO_TYPES = ['farm', 'mine', 'pump', 'rig'];

class Bot {
  constructor(game, pid) {
    this.g = game;
    this.pid = pid;
    const p = game.players[pid];
    this.pkey = p.persona || 'aggro';
    this.persona = PERSONALITIES[this.pkey];
    this.tac = game.diff.tactics;
    this.t = 1 + Math.random() * 2;
    this.wave = Math.max(4, Math.round(game.diff.wave * this.persona.waveSize));
    this.nextWaveAt = game.diff.firstWave * this.persona.firstWave + Math.random() * 30;
    this.claimed = new Set();
    this.defendT = 0;
    // Dos bolsas: economía (edificios, expansión) y ejército (tropas).
    this.ecoBank = p.coins * 0.75;
    this.milBank = p.coins * 0.25;
    this.lastT = 0;
    this.saveFor = null;
    this.raid = null;
    this.threatAt = -99;
    this.siegeAt = -99;
    this.ferryTrips = 0;
    // v0.8: fuego concentrado, hostigamiento, Torreta de defensa y ayuda a aliados.
    p.focusFire = !!this.tac.focus;
    p.snipe = !!this.tac.snipe;
    this.nextHarassAt = 200 + Math.random() * 60;
    this.wantFast = -99;
    this.turretAt = -99;
    this.helpAt = {};
    this.lastTarget = null;
    this.harassSent = 0;
    this.firstHarassAt = -1;
    this.defTurrets = 0;
    this.helpSent = 0;
    this.lastDefense = null;
    // Métricas para las pruebas.
    this.wavesSent = 0;
    this.waveSizes = [];
    this.retreats = 0;
    this.repairs = 0;
    this.firstWaveAt = -1;
  }

  get p() {
    return this.g.players[this.pid];
  }

  // v1.3: si otro bot ya pensó en este paso (busy), espera al siguiente: así nunca piensan dos
  // a la vez y el paso más lento de la simulación baja.
  update(dt, busy = false) {
    if (!this.p.alive) return false;
    this.t -= dt;
    this.defendT -= dt;
    if (this.t > 0 || busy) return false;
    this.t = this.g.diff.think * (0.8 + Math.random() * 0.4);
    this.think();
    return true;
  }

  targetArmy() {
    return 3 + Math.floor(this.g.time / 22);
  }

  think() {
    const g = this.g;
    const p = this.p;
    g.refreshEconomy();
    const mine = g.buildings.filter((b) => b.alive && b.owner === this.pid);
    const units = g.units.filter((u) => u.alive && u.owner === this.pid);
    const army = units.filter((u) => u.def.weapon).length;

    // Reparte lo ganado desde la última decisión entre las dos bolsas.
    const gained = (p.income * g.incomeMult(p) * (g.time - this.lastT)) / 60;
```

```
    this.lastT = g.time;
    const T = g.time;
    const base = T < 50 ? 0.2 : army < this.targetArmy() ? (T < 240 ? 0.45 : 0.55) : 0.38;
    const overdue = T > this.nextWaveAt + 60 ? 0.15 : 0;
    const milShare = clamp(base + overdue + (T < 50 ? 0 : this.persona.milShare), 0.1, 0.8);
    this.milBank += gained * milShare;
    this.ecoBank += gained * (1 - milShare);
    const total = this.milBank + this.ecoBank;
    if (total > p.coins) {
      // Gastos no anotados (puestos, barcos ingenieros): salen primero de la economía.
      let excess = total - p.coins;
      const fromEco = Math.min(this.ecoBank, excess);
      this.ecoBank -= fromEco;
      excess -= fromEco;
      this.milBank = Math.max(0, this.milBank - excess);
    } else if (total < p.coins) {
      this.ecoBank += p.coins - total;
    }

    this.defend(units);
    if (g.mode.tutorial) return;
    this.staffBuilders(mine, units);
    this.build(mine);
    this.expand(mine, units);
    this.produce(mine, units);
    if (this.tac.retreat) this.retreat(units, mine);
    this.repairBase(units, mine);
    // Campaña: un vecino tranquilo no ataca ni sale a capturar.
    if (this.passive) return;
    this.attack(units);
    this.ferry(mine, units);
    if (this.tac.harass) this.harass(units);
    if (this.tac.teamwork) this.helpAllies(units);
    if (this.tac.focus) this.useNuke(mine);
  }

  // Misil nuclear (v0.9, desde Normal): solo pasados 20 min, contra el Núcleo enemigo más cercano.
  useNuke(mine) {
    const g = this.g;
    if (g.time < (this.tac.earlyNuke ? 900 : 1200)) return;
    const target = this.chooseEnemyCore();
    if (!target) return;
    // Difícil (v1.0): con los 3 listos y dinero, la salva de 3 (un Escudo antimisiles solo derriba uno).
    if (this.tac.earlyNuke && this.p.coins >= NUKE.cost * 3 + 3000 && !g.salvoInvalid(this.pid, target.cell)) {
      const n = g.launchSalvo(this.pid, target.cell);
      if (typeof n === 'number') {
        this.nukes = (this.nukes || 0) + n;
        this.ecoBank = Math.max(0, this.ecoBank - NUKE.cost * n);
        return;
      }
    }
    for (const b of mine) {
      if (b.type !== 'aadef' || b.level < 4) continue;
      if (this.p.coins < NUKE.cost + 3000) return;
      if (g.nukeInvalid(b, target.cell)) continue;
      if (!g.launchNuke(b, target.cell)) {
        this.nukes = (this.nukes || 0) + 1;
        this.ecoBank = Math.max(0, this.ecoBank - NUKE.cost);
        return;
      }
    }
  }

  coreCell() {
    return this.g.cells[this.p.core.cell];
  }

  // ---------- construcción ----------
  build(mine) {
    const g = this.g;
    const p = this.p;
    const T = g.time;
    const n = (t) => mine.filter((b) => b.type === t).length;
    const ownFree = (terrain) => g.cells.some((c) => c.owner === this.pid && c.t === terrain && !c.bld);
    const coast = g.cells.some((c) => c.owner === this.pid && c.t === 'shallow' && !c.bld && g.map.isCoast(c));
    const easy = g.opts.difficulty === 'easy';
    const plan = this.persona.plan;
    const core = p.core;
    const want = [];
    const open = g.opts.mapKind === 'open';
    // Sin constructores no se levanta nada: obras pendientes frente a constructores disponibles.
    const units = g.units.filter((u) => u.alive && u.owner === this.pid);
    const builders = units.filter((u) => u.def.builds === 'land').length;
    const sites = mine.filter((b) => g.needsWork(b) && (b.def.builtBy || 'land') !== 'sea').length;
    const tooBusy = sites >= builders * 2 + 1;
    // Base castigada por buques, aeronaves o artillería (o Núcleo muy dañado): Cúpula primero.
    const shelled = p.navalHitAt !== undefined && T - p.navalHitAt < 90;
    if (core.level >= 3 && (!p.dome || (p.dome.level < 3 && T > 700)) && (shelled || core.hp < core.maxHp * 0.8 || T >
```

```
650)) want.push('dome');
    if (n('farm') < 2) want.push('farm');
    if (n('barracks') < 1 && T > 10) want.push('barracks');
    // Sin camino por tierra a ningún Núcleo enemigo: el Puerto (y la Barcaza) van primero.
    const landReach = g.players.some((q) => q.alive && g.isEnemy(q.id, this.pid) && q.core.alive &&
      g.cells[q.core.cell].comp === this.coreCell().comp);
    if (!landReach && coast && n('harbor') < 1 && T > 45) want.push('harbor');
    if (this.persona.harborFirst && coast && n('harbor') < 1 && T > 60) want.push('harbor');
    if (this.milBank > 2500) {
      if (!open && n('factory') < 2 && T > 240) want.push('factory');
      else if (n('barracks') < 3) want.push('barracks');
    }
    if (ownFree('fertile') && n('farm') < 4) want.push('farm');
    const factoryAt = plan.veh >= 3 ? 120 : 170;
    if (n('factory') < 1 && T > (open ? 300 : factoryAt)) want.push('factory');
    const energyB = mine.filter((b) => b.def.cat === 'energy').length;
    const heavyNow = n('factory') + n('harbor') + n('helipad');
    if (energyB < 1 && (heavyNow > 0 || T > 200)) want.push(n('wind') < 8 ? 'wind' : 'solar');
    if (p.popCap - p.popUsed < 6 && p.popCap < MAX_POP) {
      const houseUp = mine.find((b) => b.type === 'house' && b.built && !b.upgrading && b.level < b.def.maxLevel);
      if (n('house') < g.limitFor(this.pid, 'house')) want.push('house');
      else if (houseUp) want.push('up:house');
    }
    if (heavyNow && p.energy < 60 && energyB < Math.min(5, 1 + n('factory') + n('helipad') + n('harbor'))) {
      want.push(ownFree('shallow') && n('hydro') < 2 ? 'hydro' : n('wind') < 8 ? 'wind' : 'solar');
    }
    if (T > 240 && core.level < 2 && n('barracks') >= 1 && energyB >= 1) want.push('up:core');
    if (ownFree('gold') && n('mine') < 2) want.push('mine');
    if (ownFree('oilland') && n('pump') < 2) want.push('pump');
    if (n('harbor') < 1 && coast && open) want.push('harbor');
    if (ownFree('gold') && n('mine') < 3 + Math.floor(T / 300)) want.push('mine');
    if (n('harbor') < 1 && coast && T > (plan.sea >= 3 ? 90 : 240)) want.push('harbor');
    const enemyAir = g.units.some((u) => u.cls === 'air' && g.isEnemy(u.owner, this.pid));
    if (enemyAir && n('aadef') < 2 && T > 180) want.push('aadef');
    // Difícil (v1.0): en cuanto ve aviones enemigos, hasta 3 AIR DEFENSE y las mejora.
    if (enemyAir && this.tac.earlyAA) {
      if (n('aadef') < 3) want.unshift('aadef');
      else want.push('up:aadef');
    }
    // Mejoras: Núcleo primero (nivel 3 abre la Cúpula), luego economía y defensas.
    const fast = this.tac.fastUp;
    if (T > (fast ? 200 : 260) && core.level < 2) want.push('up:core');
    if (T > (fast ? 400 : 520) && core.level < 3) want.push('up:core');
    if (T > 300) want.push('up:farm', 'up:mine', 'up:pump', 'up:barracks');
    if (T > 420) want.push('up:turret', 'up:barracks');
    const turretCap = Math.round(4 * this.persona.turrets);
    if (n('turret') < Math.min(turretCap, Math.floor(((T - 60) * this.persona.turrets) / 120))) want.push('turret');
    if (n('harbor') && (ownFree('oil') || this.seaSpot('rig') >= 0)) want.push('rig');
    if (n('harbor') && heavyNow > 1 && p.energy < 80 && n('tidal') < 2 && this.seaSpot('tidal') >= 0)
want.push('tidal');
    if (!easy && plan.air >= 1 && n('helipad') < 1 && T > (plan.air >= 2 ? 300 : 380)) want.push('helipad');
    if (ownFree('fertile') && n('farm') < Math.min(10, 3 + Math.floor(T / 70))) want.push('farm');
    if (ownFree('oilland') && n('pump') < 2 + Math.floor(T / 300)) want.push('pump');
    if (n('barracks') < 2 && T > 280) want.push('barracks');
    if (n('hospital') < 1 && T > (this.tac.retreat ? 300 : 420)) want.push('hospital');
    if (n('factory') < 2 && T > 480 && !open) want.push('factory');
    if (n('harbor') < 2 && coast && T > (plan.sea >= 3 ? 300 : 420)) want.push('harbor');
    // v0.9: Aeropuerto, AIR DEFENSE de nivel 4 (misil nuclear) y Escudo antimisiles si el rival tiene misiles.
    if (!easy && plan.air >= 1 && n('airport') < 1 && T > (plan.air >= 2 ? 540 : 720)) want.push('airport');
    const nukeT = this.tac.earlyNuke ? 800 : 1000;
    if (this.tac.focus && T > nukeT && n('aadef') >= 1) want.push('up:aadef');
    if (this.tac.focus && T > nukeT && n('aadef') < 1) want.push('aadef');
    const enemyNuke = g.buildings.some((b) => b.alive && b.type === 'aadef' && b.level >= 4 && g.isEnemy(b.owner,
this.pid));
    // v0.9: con más de 15 000 monedas guardadas (pasados 15 min) invierte en aire y defensa antiaérea:
    // Aeropuerto, un segundo Helipuerto y AIR DEFENSE hasta el nivel 4 (misil nuclear).
    if (!easy && T > 900 && p.coins > 15000) {
      if (this.milBank > 8000) {
        this.ecoBank += this.milBank * 0.5;
        this.milBank *= 0.5;
      }
      const rich = [];
      if (n('airport') < 1) rich.push('airport');
      if (n('aadef') < 2) rich.push('aadef');
      if (n('aadef') >= 1) rich.push('up:aadef');
      if (n('helipad') < 2) rich.push('helipad');
      want.unshift(...rich);
    }
    if (enemyNuke && n('antinuke') < 1) want.unshift('antinuke');

    // Una mejora barata cada ~minuto (Granja, Mina, Bomba o Casa) aunque haya otras compras esperando.
    if (T > 300 && T - (this.cheapUpAt || 0) > (fast ? 35 : 60) && !tooBusy) {
      const cand = mine.filter((x) => ['farm', 'mine', 'pump', 'house'].includes(x.type) && x.built && !x.upgrading &&
x.level < x.def.maxLevel)
        .sort((x, y) => g.upgradeCost(x) - g.upgradeCost(y))[0];
      if (cand && this.ecoBank >= g.upgradeCost(cand)) {
        const uc = g.upgradeCost(cand);
        if (typeof g.upgrade(cand) !== 'string') {
```

```
          this.ecoBank -= uc;
          this.cheapUpAt = T;
        }
      }
    }
    if (!want.length) {
      this.milBank += this.ecoBank * 0.5;
      this.ecoBank *= 0.5;
      return;
    }
    let actions = 0;
    for (const type of want) {
      if (actions >= 2) break;
      if (type.startsWith('up:')) {
        const t = type.slice(3);
        const b = mine.find((x) => x.type === t && x.built && !x.upgrading && x.level < x.def.maxLevel);
        if (!b || tooBusy) continue;
        const uc = g.upgradeCost(b);
        if (this.ecoBank < uc) {
          if (t === 'core') break;
          continue;
        }
        if (typeof g.upgrade(b) !== 'string') {
          this.ecoBank -= uc;
          actions++;
        }
        continue;
      }
      if (type === 'dome') {
        const dc = g.domeCost(this.pid);
        if (this.ecoBank + this.milBank < dc) break;
        if (!g.buyDome(this.pid)) {
          const fromEco = Math.min(this.ecoBank, dc);
          this.ecoBank -= fromEco;
          this.milBank = Math.max(0, this.milBank - (dc - fromEco));
          actions++;
        }
        continue;
      }
      const def = BUILDINGS[type];
      if (!def.sea && tooBusy) continue;
      const cost = def.cost;
      if (this.ecoBank < cost || p.coins < cost) break;
      if (def.energyCost && p.energy < def.energyCost) continue;
      const ci = def.sea ? this.seaSpot(type) : this.pickCell(type);
      if (ci < 0) continue;
      if (!g.place(this.pid, type, ci)) {
        this.ecoBank -= cost;
        actions++;
      }
    }
  }

  // Estructuras de mar: el lugar libre más cercano a la base (puede estar fuera del territorio).
  seaSpot(type) {
    const g = this.g;
    const core = this.coreCell();
    let best = -1;
    let bd = Infinity;
    for (const c of g.cells) {
      if (c.bld || (c.owner >= 0 && c.owner !== this.pid)) continue;
      const d = g.map.dist(c, core);
      if (d > 9) continue;
      if (g.placeInvalid(this.pid, type, c.i, true)) continue;
      if (d < bd) { bd = d; best = c.i; }
    }
    return best;
  }

  // Constructores: 2 al empezar y más a medida que crece la base (máximo 5).
  // Barco constructor si hay estructuras de mar esperando.
  staffBuilders(mine, units) {
    const g = this.g;
    const want = Math.min(5, 2 + Math.floor(mine.length / 6));
    const have = units.filter((u) => u.def.builds === 'land').length +
      mine.reduce((s, b) => s + b.queue.filter((q) => UNITS[q.type].builds === 'land').length, 0);
    const sieged = g.time - this.siegeAt < 15;
    if (have < want && (!sieged || have === 0)) {
      const prod = mine.filter((b) => b.built && b.def.produces && b.def.produces.includes('builder') && b.queue.length
< 2)
        .sort((a, b) => a.queue.length - b.queue.length)[0];
      if (prod && this.p.coins >= UNITS.builder.cost && !g.enqueue(prod, 'builder')) {
        this.ecoBank = Math.max(0, this.ecoBank - UNITS.builder.cost);
      }
    }
    const seaWork = mine.some((b) => g.needsWork(b) && b.def.builtBy === 'sea');
    if (seaWork) this.queueOnce(mine, 'harbor', 'boatEng');
  }
```

```
344
345   pickCell(type) {
346     const g = this.g;
347     const core = this.coreCell();
348     let best = -1;
349     let bestS = Infinity;
350     const toCenter = Math.hypot(core.x, core.z) || 1;
351     for (const c of g.cells) {
352       if (c.owner !== this.pid || c.bld || c.bridge) continue;
353       if (g.placeInvalid(this.pid, type, c.i)) continue;
354       const d = g.map.dist(c, core);
355       if (d < 2 && type !== 'turret') continue;
356       let s = Math.random() * 1.5;
357       if (type === 'turret') {
358         const dot = -(c.x * core.x + c.z * core.z) / toCenter;
359         s += Math.abs(d - 2.5) * 2 - dot * 0.3;
360       } else if (type === 'farm' || type === 'house') {
361         s += Math.abs(d - 3);
362       } else if (type === 'mine' || type === 'rig' || type === 'harbor') {
363         s += d * 0.3;
364       } else {
365         s += d * 1.2;
366       }
367       // Evita tapar el paso: penaliza casillas con pocas vecinas libres.
368       let free = 0;
369       for (const ni of c.nb) {
370         const nc = g.cells[ni];
371         if (TERRAIN[nc.t].land && !nc.bld) free++;
372       }
373       if (TERRAIN[c.t].land && free < 3) s += 4;
374       if (s < bestS) { bestS = s; best = c.i; }
375     }
376     return best;
377   }
378
379   // ---------- expansión ----------
380   expand(mine, units) {
381     const g = this.g;
382     const T = g.time;
383     if (T < 25) return;
384     const outposts = mine.filter((b) => b.type === 'outpost').length;
385     const maxOut = Math.min(6, 2 + Math.floor(T / 200));
386     const busy = units.filter((u) => u.def.founds && u.state === 'found');
387     if (outposts + busy.length >= maxOut) return;
388     this.claimed = new Set(busy.map((u) => u.foundCell));
389
390     const landEng = units.filter((u) => u.type === 'engineer' && u.state !== 'found' && u.state !== 'repair' && u.state
!== 'build');
391     const target = this.findExpansion('land');
392     if (target >= 0) {
393       if (landEng.length) {
394         if (this.p.coins >= BUILDINGS.outpost.cost) g.orderFound(landEng[0], target);
395       } else {
396         this.queueOnce(mine, 'barracks', 'engineer');
397       }
398     }
399     const harbors = mine.filter((b) => b.type === 'harbor' && b.built);
400     if (harbors.length && T > 140) {
401       const boats = units.filter((u) => u.type === 'boatEng' && u.state !== 'found');
402       const st = this.findExpansion('sea');
403       if (st >= 0) {
404         if (boats.length) {
405           if (this.p.coins >= BUILDINGS.outpost.cost) g.orderFound(boats[0], st);
406         } else if ((this.boatEngs || 0) < 3 && this.ecoBank >= UNITS.boatEng.cost + BUILDINGS.outpost.cost) {
407           if (this.queueOnce(mine, 'harbor', 'boatEng')) this.boatEngs = (this.boatEngs || 0) + 1;
408         }
409       }
410     }
411   }
412
413   queueOnce(mine, btype, utype) {
414     const g = this.g;
415     const builders = mine.filter((b) => b.type === btype && b.built);
416     if (!builders.length) return false;
417     if (builders.some((b) => b.queue.some((q) => q.type === utype))) return false;
418     if (g.units.some((u) => u.owner === this.pid && u.type === utype)) return false;
419     if (g.enqueue(builders[0], utype)) return false;
420     // Barcaza: gasto militar. Ingenieros: gasto de expansión.
421     if (utype === 'barge' || utype === 'lander') this.milBank = Math.max(0, this.milBank - UNITS[utype].cost);
422     else this.ecoBank = Math.max(0, this.ecoBank - UNITS[utype].cost);
423     return true;
424   }
425
426   findExpansion(how) {
427     const g = this.g;
428     const core = this.coreCell();
429     let best = -1;
430     let bestS = Infinity;
```

```
    for (const r of g.cells) {
      if ((r.t !== 'gold' && r.t !== 'oil') || r.owner !== -1 || r.bld) continue;
      if (how === 'land' && r.t === 'oil') continue;
      for (const off of SPIRAL) {
        if (off.d > 2) break;
        const c = g.map.get(r.q + off.q, r.r + off.r);
        if (!c || this.claimed.has(c.i) || c.owner !== -1) continue;
        if (g.foundInvalid(this.pid, c.i, how)) continue;
        if (how === 'land' && c.comp !== core.comp) continue;
        const s = g.map.dist(c, core) + off.d * 0.5;
        if (s < bestS) { bestS = s; best = c.i; }
      }
    }
    return best;
  }

  // ---------- producción: plan de ejército y contrarrestos ----------
  enemyMix() {
    const g = this.g;
    const mix = { inf: 0, veh: 0, sea: 0, air: 0, total: 0 };
    for (const u of g.units) {
      if (!u.alive || !u.def.weapon || !g.isEnemy(u.owner, this.pid)) continue;
      mix[unitCat(u)]++;
      mix.total++;
    }
    for (const k of ['inf', 'veh', 'sea', 'air']) mix[k] = mix.total ? mix[k] / mix.total : 0;
    return mix;
  }

  pickUnit(producers, units) {
    const g = this.g;
    const T = g.time;
    const open = g.opts.mapKind === 'open';
    const cats = {};
    for (const b of producers) {
      if (b.type === 'barracks') cats.inf = 'barracks';
      else if (b.type === 'factory') cats.veh = 'factory';
      else if (b.type === 'harbor') cats.sea = 'harbor';
      else if (b.type === 'helipad') cats.air = 'helipad';
      else if (b.type === 'airport') cats.plane = 'airport';
    }
    const have = { inf: 0, veh: 0, sea: 0, air: 0, plane: 0 };
    const catOf = (d) => (d.plane ? 'plane' : unitCat(d));
    for (const u of units) if (u.def.weapon) have[catOf(u.def.cls ? { ...u.def, cls: u.cls } : u.def)]++;
    for (const b of producers) {
      for (const q of b.queue) if (UNITS[q.type].weapon) have[catOf(UNITS[q.type])]++;
    }
    const plan = { ...this.persona.plan, plane: Math.max(1, this.persona.plan.air) };
    if (open) {
      plan.sea += 2;
      plan.inf = Math.max(1, plan.inf - 1);
    }
    // v0.9: con mucho dinero guardado, más aviones (Cazas y Bombarderos).
    if (T > 900 && this.p.coins > 15000) plan.plane += 2;
    // Energía acumulada: prioriza tropas que la usan.
    if (this.p.energy > 120) {
      plan.veh += 2;
      plan.sea += 1;
      plan.air += 1;
    }
    const mix = this.enemyMix();
    if (this.tac.counter && mix.total >= 3) {
      if (mix.air >= 0.12) plan.veh += 2;
      if (mix.sea >= 0.25) plan.sea += 2;
      if (mix.veh >= 0.3) plan.inf += 1;
    }
    // Hostigamiento: si faltan tropas rápidas, entrena una (Jeep, Lancha o Helicóptero).
    if (this.tac.harass && T - this.wantFast < 40) {
      const fastHave = units.filter((u) => FAST_UNITS[u.type]).length +
        producers.reduce((s, b) => s + b.queue.filter((q) => FAST_UNITS[q.type]).length, 0);
      const opt = Object.entries(FAST_UNITS).find(([t, c]) => cats[c] && (t !== 'heli' || this.p.energy >=
UNITS.heli.energy));
      if (opt && fastHave < 3) return { type: opt[0], btype: cats[opt[1]], since: T };
    }
    // Recién empezada la partida, o bajo ataque: infantería rápida.
    const urgent = T < 120 || g.time - this.threatAt < 12;
    let avail = Object.keys(cats);
    if (urgent && cats.inf) avail = ['inf'];
    let best = null;
    let bestGap = -Infinity;
    const planSum = avail.reduce((s, c) => s + plan[c], 0) || 1;
    const haveSum = avail.reduce((s, c) => s + have[c], 0) || 1;
    for (const c of avail) {
      const gap = plan[c] / planSum - have[c] / haveSum + Math.random() * 0.05;
      if (gap > bestGap) { bestGap = gap; best = c; }
    }
    if (!best) return null;
    return { type: this.unitFor(best, mix, have, units), btype: cats[best], since: T };
```

```
  }

  unitFor(cat, mix, have, units) {
    const T = this.g.time;
    const r = Math.random();
    const counter = this.tac.counter && mix.total >= 3;
    if (cat === 'plane') {
      // Aeropuerto: Cazas si el rival tiene aire; si no, sobre todo Bombarderos.
      return (mix.air >= 0.1 && r < 0.75) || r < 0.35 ? 'fighter' : 'bomber';
    }
    if (cat === 'inf') {
      if (T < 90) return r < 0.7 ? 'rifle' : 'shotgun';
      const infN = units.filter((u) => u.cls === 'inf' && u.def.weapon).length;
      if (infN >= 6 && !units.some((u) => u.type === 'medic') && r < 0.25) return 'medic';
      if (counter && mix.veh >= 0.3) return r < 0.65 ? 'rocket' : 'rifle';
      if (counter && mix.inf >= 0.6) return r < 0.3 ? 'sniper' : r < 0.55 ? 'breaker' : r < 0.75 ? 'hunter' : 'rifle';
      return r < 0.3 ? 'rifle' : r < 0.45 ? 'ranger' : r < 0.58 ? 'shotgun' : r < 0.78 ? 'rocket' : r < 0.88 ? 'sniper'
: r < 0.95 ? 'breaker' : 'hunter';
    }
    if (cat === 'veh') {
      const aaHave = units.filter((u) => u.type === 'aa').length;
      if (counter && mix.air >= 0.12 && aaHave < 3) return 'aa';
      if (counter && mix.veh >= 0.3) return r < 0.7 ? 'tank' : 'jeep';
      if (counter && mix.inf >= 0.6) return r < 0.55 ? 'jeep' : 'tank';
      if (mix.air > 0 && aaHave < 1) return 'aa';
      if (T < 300) return r < 0.5 ? 'jeep' : 'lighttank';
      return r < 0.16 ? 'lighttank' : r < 0.34 ? 'tank' : r < 0.5 ? 'heavytank' : r < 0.6 ? 'jeep' : r < 0.7 ? 'arty'
        : r < 0.8 ? 'missiletruck' : r < 0.9 ? 'launcher' : 'apc';
    }
    if (cat === 'sea') {
      if (counter && mix.air >= 0.15 && r < 0.4) return 'frigate';
      if (counter && mix.sea >= 0.25) return r < 0.5 ? 'destroyer' : 'sub';
      if (T > 900 && this.p.coins > UNITS.carrier.cost * 1.2 && !units.some((u) => u.type === 'carrier') && r < 0.5)
return 'carrier';
      return r < 0.45 ? 'destroyer' : r < 0.7 ? 'speedboat' : r < 0.85 ? 'frigate' : 'sub';
    }
    // El Apache no tiene límite (v0.9): con dinero de sobra, el bot lleva tantos Apaches como Helicópteros.
    const nApache = units.filter((u) => u.type === 'apache').length;
    const nHeli = units.filter((u) => u.type === 'heli').length;
    return nApache <= nHeli && this.p.coins > UNITS.apache.cost * 1.2 ? 'apache' : 'heli';
  }

  produce(mine, units) {
    const g = this.g;
    const p = this.p;
    const producers = mine.filter((b) => b.built && b.def.produces);
    if (!producers.length) {
      this.ecoBank += this.milBank;
      this.milBank = 0;
      return;
    }
    let budget = Math.min(p.coins, this.milBank);
    for (let k = 0; k < 3; k++) {
      if (!this.saveFor || !producers.some((b) => b.type === this.saveFor.btype)) this.saveFor =
this.pickUnit(producers, units);
      const s = this.saveFor;
      if (!s) return;
      const def = UNITS[s.type];
      const b = producers.filter((x) => x.type === s.btype && x.queue.length < 2)
        .sort((a, c) => a.queue.length - c.queue.length)[0];
      if (!b) return;
      if (def.cost > budget || p.energy < (def.energy || 0)) {
        // Ahorra para esta tropa; si tarda demasiado, vuelve a planificar.
        if (g.time - s.since > 70) this.saveFor = null;
        return;
      }
      const err = g.enqueue(b, s.type);
      this.saveFor = null;
      if (err) return;
      budget -= def.cost;
      this.milBank -= def.cost;
    }
  }

  // ---------- combate ----------
  defend(units) {
    const g = this.g;
    if (this.defendT > 0) return;
    this.defendT = 2;
    const core = this.coreCell();
    let threat = null;
    let td = Infinity;
    for (const e of g.units) {
      if (!e.alive || e.inside || !g.isEnemy(e.owner, this.pid)) continue;
      const c = g.cells[e.cell];
      const d = g.map.dist(c, core);
      if (c.owner === this.pid || d <= 5) {
        if (d < td) { td = d; threat = e; }
```

```
      }
    }
    if (!threat) return;
    // Urgencia (comprar infantería rápida) solo si el peligro está junto al Núcleo y el ejército es pequeño.
    const armySize = units.filter((u) => u.def.weapon).length;
    if (td <= 4 && armySize < 8) this.threatAt = g.time;
    if (td <= 3) this.siegeAt = g.time;
    // Una oleada que se estaba reuniendo vuelve a defender si el peligro está cerca del Núcleo.
    if (this.raid && td <= 4) {
      for (const u of this.raid.units) if (u.alive && u.role === 'raid') u.role = 'home';
      this.raid = null;
    }
    // Defensa en equipo (v0.8): van todas las tropas de casa, también las que salen de la producción,
    // y los hostigadores que estén cerca. Los guardias se quedan cuidando sus Banderas.
    const tc = g.cells[threat.cell];
    const near = (u) => g.map.dist(g.cells[u.cell], tc) <= 10;
    const guards = units.filter((u) => u.def.weapon && !u.inside && g.canAttack(u, u.def.weapon, threat) &&
      (u.role === 'home' || (u.role === 'harass' && near(u))) &&
      (u.state === 'idle' || u.state === 'move' || (u.state === 'amove' && u.role !== 'raid')));
    for (const u of guards) if (u.role !== 'home') u.role = 'home';
    const nearby = units.filter((u) => u.def.weapon && !u.inside && u.role !== 'retreat' && near(u));
    this.lastDefense = { t: g.time, cell: tc.i, near: nearby.length, sent: guards.filter(near).length,
      fighting: nearby.filter((u) => u.state === 'attack' || guards.includes(u)).length };
    if (this.tac.turret && td <= 7) this.defenseTurret(tc);
    const byMover = { land: [], sea: [], air: [] };
    for (const u of guards) byMover[u.mover].push(u);
    if (byMover.land.length && TERRAIN[tc.t].land) g.orderMove(byMover.land, tc.i, true);
    if (byMover.air.length) g.orderMove(byMover.air, tc.i, true);
    if (byMover.sea.length) {
      const wc = this.waterNear(tc, byMover.sea[0]);
      if (wc >= 0) g.orderMove(byMover.sea, wc, true);
    }
  }

  // Tropas con menos del 30 % de vida vuelven a curarse y luego regresan.
  retreat(units, mine) {
    const g = this.g;
    const core = this.p.core;
    for (const u of units) {
      if (!u.def.weapon || u.inside) continue;
      if (u.role === 'retreat') {
        if (u.hp >= u.maxHp * 0.85) {
          u.role = 'home';
          u.state = 'idle';
        } else if (u.state === 'idle' && u.retreatTo >= 0 && g.map.dist(g.cells[u.cell], g.cells[u.retreatTo]) > 2) {
          g.orderMove([u], u.retreatTo, false);
        }
        continue;
      }
      // Difícil (v1.0): retira a los heridos antes (45 % de vida) para curarlos.
      if (u.hp >= u.maxHp * (this.tac.medevac ? 0.45 : 0.3)) continue;
      const comp = g.cells[u.cell].comp;
      let spot = null;
      let sd = Infinity;
      const consider = (b) => {
        if (!b.alive || !b.built) return;
        const bc = g.cells[b.cell];
        if (u.mover === 'land' && bc.comp !== comp) return;
        if (u.mover === 'sea' && !bc.nb.some((ni) => g.cells[ni].comp === comp) && bc.comp !== comp) return;
        const d = g.map.dist(bc, g.cells[u.cell]);
        if (d < sd) { sd = d; spot = b; }
      };
      if (u.mover === 'sea') {
        for (const b of mine) if (b.type === 'harbor') consider(b);
      } else if (u.mover === 'air' && g.repairPadFor(u)) {
        consider(g.repairPadFor(u));
      } else {
        if (u.mover === 'land') for (const b of mine) if (b.type === 'hospital') consider(b);
        if (!spot || sd > 8) consider(core);
      }
      if (!spot) continue;
      const dest = g.freeCellsNear(spot.cell, u.mover, this.pid, 1)[0];
      u.role = 'retreat';
      u.retreatTo = dest;
      u.target = null;
      if (this.raid) this.raid.units = this.raid.units.filter((x) => x !== u);
      g.orderMove([u], dest, false);
      this.retreats++;
    }
  }

  // Ingenieros libres reparan los edificios dañados de la base.
  repairBase(units, mine) {
    const g = this.g;
    const engs = units.filter((u) => u.def.repairs && u.state === 'idle' && !u.inside);
    if (!engs.length) return;
    const damaged = mine.filter((b) => b.built && b.hp < b.maxHp * 0.9 && g.time - b.lastHit > REPAIR.pause)
      .sort((a, b) => a.hp / a.maxHp - b.hp / b.maxHp);
```

```
    for (const b of damaged) {
      const bc = g.cells[b.cell];
      const e = engs.find((x) => !g.isUnreach(x, b) && (g.cells[x.cell].comp === bc.comp || bc.nb.some((ni) =>
g.cells[ni].comp === g.cells[x.cell].comp)));
      if (!e) continue;
      g.orderRepair([e], b);
      engs.splice(engs.indexOf(e), 1);
      this.repairs++;
      if (!engs.length) return;
    }
  }

  waterNear(target, ship) {
    const g = this.g;
    const comp = g.cells[ship.cell].comp;
    for (const off of SPIRAL) {
      if (off.d > 5) break;
      const c = g.map.get(target.q + off.q, target.r + off.r);
      if (c && TERRAIN[c.t].water && !c.bld && c.comp === comp) return c.i;
    }
    return -1;
  }

  coastNear(target, ship) {
    const g = this.g;
    const comp = g.cells[ship.cell].comp;
    for (const off of SPIRAL) {
      if (off.d > 6) break;
      const c = g.map.get(target.q + off.q, target.r + off.r);
      if (c && TERRAIN[c.t].water && !c.bld && c.comp === comp && g.map.isCoast(c)) return c.i;
    }
    return -1;
  }

  // Banderas sin dueño de tu equipo a las que no se llega por tierra.
  farFlags() {
    const g = this.g;
    const core = this.coreCell();
    const team = this.p.team;
    return g.points.filter((pt) => pt.type === 'flag' && (pt.owner < 0 || g.players[pt.owner].team !== team) &&
      g.cells[pt.cell].comp !== core.comp)
      .sort((a, b) => g.map.dist(g.cells[a.cell], core) - g.map.dist(g.cells[b.cell], core));
  }

  // Barcaza: lleva tropas a islas sin camino por tierra (Núcleos enemigos, el Faro o Banderas).
  ferry(mine, units) {
    const g = this.g;
    if (g.time < 200 || !mine.some((b) => b.type === 'harbor' && b.built)) return;
    const core = this.coreCell();
    const landReach = g.players.some((p) => p.alive && g.isEnemy(p.id, this.pid) && p.core.alive &&
      g.cells[p.core.cell].comp === core.comp);
    const faro = g.points.find((pt) => pt.type === 'faro');
    const faroByLand = !g.mode.king || !faro || g.cells[faro.cell].comp === core.comp;
    const flagsWanted = this.farFlags().length > 0 &&
      (!landReach || this.pkey === 'admiral' || this.pkey === 'keeper' || g.time > 420);
    if (landReach && faroByLand && !flagsWanted) return;
    // v0.9: pasados 10 minutos y con dinero, usa la Lancha de desembarco (más rápida, 10 lugares).
    const t = units.find((u) => u.type === 'barge' || u.type === 'lander');
    if (!t) {
      this.queueOnce(mine, 'harbor', g.time > 600 && this.p.coins > 4000 ? 'lander' : 'barge');
      return;
    }
    t.botMode = t.botMode || 'load';
    if (t.botMode === 'load') {
      if (t.state !== 'idle') return;
      if (g.map.dist(g.cells[t.cell], core) > 7) {
        const wc = this.coastNear(core, t);
        if (wc >= 0) g.orderMove([t], wc);
        return;
      }
      const boarding = units.filter((u) => u.state === 'board' && u.boardTarget === t);
      let slots = t.def.capacity - g.cargoSlots(t) - boarding.reduce((s, u) => s + (u.cls === 'veh' ? 2 : 1), 0);
      const pick = [];
      for (const u of units) {
        if (u.mover !== 'land' || !u.def.weapon || u.inside || u.role !== 'home' || u.state !== 'idle') continue;
        const need = u.cls === 'veh' ? 2 : 1;
        if (need <= slots) {
          pick.push(u);
          slots -= need;
        }
      }
      if (pick.length) g.orderBoard(pick, t);
      t.loadT = (t.loadT || 0) + g.diff.think;
      const full = g.cargoSlots(t) >= t.def.capacity - 1;
      if (full || (t.cargo.length >= 3 && t.loadT > 60)) {
        const target = this.ferryTarget(landReach);
        if (!target) return;
        const wc = this.coastNear(g.cells[target.cell], t);
```

```
        if (wc < 0) return;
        for (const u of boarding) u.state = 'idle';
        t.botMode = 'sail';
        t.botTarget = target;
        t.loadT = 0;
        this.ferryTrips++;
        if (target.kind === 'bld') {
          this.coreRaids = (this.coreRaids || 0) + 1;
          if (this.firstRaidAt === undefined) this.firstRaidAt = g.time;
        }
        g.orderMove([t], wc);
      }
    } else if (t.botMode === 'sail') {
      if (t.state !== 'idle') return;
      const res = g.unload(t);
      if (typeof res !== 'string') {
        let target = t.botTarget && t.botTarget.alive ? t.botTarget : this.ferryTarget(landReach);
        if (target && target.kind === 'point') {
          const team = this.p.team;
          if (target.owner >= 0 && g.players[target.owner].team === team) target = null;
        }
        if (target && target.kind === 'point') {
          for (const u of res) {
            u.role = 'cap';
            u.capPoint = target;
            u.capSince = g.time;
          }
        } else {
          for (const u of res) u.role = 'wave';
        }
        if (target) g.orderMove(res, target.cell, true);
      }
      t.botMode = 'return';
      const wc = this.coastNear(core, t);
      if (wc >= 0) g.orderMove([t], wc);
    } else if (t.botMode === 'return' && t.state === 'idle') {
      t.botMode = 'load';
    }
  }

  ferryTarget(landReach = true) {
    const g = this.g;
    if (g.mode.king) {
      const faro = g.points.find((pt) => pt.type === 'faro');
      if (faro && (faro.owner < 0 || g.players[faro.owner].team !== this.p.team)) return faro;
    }
    const flags = this.farFlags();
    const flagFirst = this.pkey === 'keeper' || this.pkey === 'admiral' || this.ferryTrips % 2 === 1 || landReach;
    if (flags.length && flagFirst) return flags[0];
    if (landReach) return flags[0] || null;
    return this.chooseEnemyCore() || flags[0] || null;
  }

  // ---------- órdenes de un aliado humano (v1.1) ----------
  // «¡Ayuda!»: manda hasta la mitad de su ejército (máx. 8) a defender la base del aliado.
  answerHelp(pid) {
    const g = this.g;
    const ally = g.players[pid];
    if (!this.p.alive || !ally || !ally.core || !ally.core.alive || g.isEnemy(pid, this.pid)) return 0;
    const home = g.units.filter((u) => u.alive && u.owner === this.pid && u.def.weapon && !u.inside && u.role !==
'guard');
    if (!home.length) {
      g.say(this.pid, 'No tengo tropas libres ahora, ¡aguanta!', { force: true, toMe: true });
      return 0;
    }
    const go = home.slice(0, Math.min(8, Math.max(1, Math.ceil(home.length / 2))));
    const sent = this.sendTo(go, { cell: ally.core.cell });
    for (const u of sent) u.role = 'help';
    this.helpAt[pid] = g.time;
    if (sent.length) this.helpSent++;
    g.say(this.pid, sent.length ? `¡Voy a ayudarte con ${sent.length} tropas!` : 'No llego a tu base desde aquí.', {
force: true, toMe: true });
    return sent.length;
  }

  // «¡Ataquemos!»: ataca ya al rival que elegiste y lo prefiere en sus oleadas por 3 minutos.
  answerAttack(targetPid) {
    const g = this.g;
    const foe = g.players[targetPid];
    if (!this.p.alive || !foe || !foe.alive || !g.isEnemy(targetPid, this.pid)) return 0;
    this.focusPid = targetPid;
    this.focusUntil = g.time + 180;
    const target = foe.core && foe.core.alive ? foe.core : g.buildings.find((b) => b.alive && b.owner === targetPid);
    if (!target) return 0;
    const army = g.units.filter((u) => u.alive && u.owner === this.pid && u.def.weapon && !u.inside && u.role !==
'guard');
    const go = army.slice(0, Math.max(1, Math.ceil(army.length * 0.7)));
    const sent = go.length ? this.sendTo(go, target) : [];
```

```
    for (const u of sent) u.role = 'help';
    this.lastTarget = target;
    g.say(this.pid, sent.length ? `¡Vamos por ${foe.name}! Mando ${sent.length} tropas.` : `Atacaré a ${foe.name} con la próxima oleada.`, { force: true, toMe: true });
    return sent.length;
  }

  chooseEnemyCore() {
    const g = this.g;
    const core = this.coreCell();
    // v1.1: el rival que pidió el aliado humano con «¡Ataquemos!».
    if (this.focusPid !== undefined && g.time < this.focusUntil) {
      const f = g.players[this.focusPid];
      if (f && f.alive && f.core && f.core.alive && g.isEnemy(f.id, this.pid)) return f.core;
    }
    let cores = g.players
      .filter((p) => p.alive && g.isEnemy(p.id, this.pid) && p.core.alive)
      .map((p) => ({ p, d: g.map.dist(g.cells[p.core.cell], core) }))
      .sort((a, b) => a.d - b.d);
    if (!cores.length) return null;
    // Primero los Núcleos a los que el ejército llega a pie.
    const byLand = cores.filter((c) => g.cells[c.p.core.cell].comp === core.comp);
    if (byLand.length) cores = byLand;
    // Aliados coordinados (v0.8): atacan al mismo rival que un aliado bot.
    if (this.tac.teamwork) {
      for (const b of g.bots) {
        if (b === this || b.p.team !== this.p.team || !b.p.alive) continue;
        const t = b.lastTarget;
        if (t && t.alive && cores.some((c) => c.p.core === t) && Math.random() < 0.8) {
          this.lastTarget = t;
          return t;
        }
      }
    }
    const pickI = cores.length > 1 && Math.random() < 0.3 ? 1 : 0;
    this.lastTarget = cores[pickI].p.core;
    return this.lastTarget;
  }

  sendTo(units, targetB) {
    const g = this.g;
    const tc = g.cells[targetB.cell];
    const land = [];
    const air = [];
    const sea = [];
    for (const u of units) {
      if (u.mover === 'air') air.push(u);
      else if (u.mover === 'sea') sea.push(u);
      else if (g.cells[u.cell].comp === tc.comp || tc.nb.some((ni) => g.cells[ni].comp === g.cells[u.cell].comp)) land.push(u);
    }
    // Avanzan atacando, juntas (en bloque) para llegar a la vez; la línea se usa al reunirse.
    if (land.length) g.orderMove(land, tc.i, true);
    if (air.length) g.orderMove(air, tc.i, true);
    if (sea.length) {
      const wc = this.waterNear(tc, sea[0]);
      if (wc >= 0) g.orderMove(sea, wc, true);
    }
    return [...land, ...air, ...(sea.length && this.waterNear(tc, sea[0]) >= 0 ? sea : [])];
  }

  // ---------- v0.8: Torreta de defensa, hostigamiento y aliados ----------
  // Levanta una Torreta entre el Núcleo y el lado por donde vino el ataque (cada 45 s como mucho).
  defenseTurret(threatCell) {
    const g = this.g;
    if (g.time - this.turretAt < 45) return false;
    const core = this.coreCell();
    const p = this.p;
    if (p.coins < BUILDINGS.turret.cost + 200) return false;
    const dir = Math.atan2(threatCell.z - core.z, threatCell.x - core.x);
    const off = (c) => {
      let a = Math.atan2(c.z - core.z, c.x - core.x) - dir;
      while (a > Math.PI) a -= Math.PI * 2;
      while (a < -Math.PI) a += Math.PI * 2;
      return Math.abs(a);
    };
    const own = g.buildings.filter((b) => b.alive && b.owner === this.pid && b.type === 'turret');
    const cand = g.cells.filter((c) => {
      const d = g.map.dist(c, core);
      return d >= 2 && d <= 3 && off(c) < 0.9 && !own.some((t) => g.map.dist(g.cells[t.cell], c) < 2) && !g.placeInvalid(this.pid, 'turret', c.i);
    }).sort((a, b) => off(a) - off(b));
    if (!cand.length) return false;
    if (g.place(this.pid, 'turret', cand[0].i)) return false;
    this.turretAt = g.time;
    this.defTurrets++;
    return true;
  }
```

```
 948
 949   // Granja, Mina, Bomba o Plataforma enemiga lejos de sus defensas y alcanzable por esa tropa.
 950   ecoTarget(u, maxDist = Infinity) {
 951     const g = this.g;
 952     const uc = g.cells[u.cell];
 953     const defs = g.buildings.filter((b) => b.alive && g.isEnemy(b.owner, this.pid) && b.def.weapon && g.canAttack(b,
b.def.weapon, u));
 954     let best = null;
 955     let bs = Infinity;
 956     for (const b of g.buildings) {
 957       if (!b.alive || !g.isEnemy(b.owner, this.pid) || !ECO_TYPES.includes(b.type)) continue;
 958       const c = g.cells[b.cell];
 959       const d = g.map.dist(c, uc);
 960       if (d > maxDist) continue;
 961       if (u.mover === 'land' && c.comp !== uc.comp && !c.nb.some((ni) => g.cells[ni].comp === uc.comp)) continue;
 962       if (u.mover === 'sea' && this.waterNear(c, u) < 0) continue;
 963       let safe = 99;
 964       for (const t of defs) safe = Math.min(safe, g.map.dist(g.cells[t.cell], c) - (t.def.weapon.range || 4));
 965       if (safe < -1) continue;
 966       const s = d / 3 - Math.min(safe, 6) * 3;
 967       if (s < bs) { bs = s; best = b; }
 968     }
 969     return best;
 970   }
 971
 972   // Hostigamiento: 2-3 tropas rápidas atacan la economía enemiga lejos de sus defensas.
 973   harass(units) {
 974     const g = this.g;
 975     const team = units.filter((u) => u.role === 'harass' && !u.inside);
 976     for (const u of team) {
 977       if (u.state !== 'idle') continue;
 978       const t = this.ecoTarget(u, 7);
 979       if (t) this.sendTo([u], t);
 980       else {
 981         u.role = 'home';
 982         g.orderMove([u], this.p.core.cell, false);
 983       }
 984     }
 985     if (team.length || g.time < this.nextHarassAt) return;
 986     const fast = units.filter((u) => FAST_UNITS[u.type] && u.role === 'home' && !u.inside && u.state === 'idle');
 987     if (fast.length < 2) {
 988       this.wantFast = g.time;
 989       return;
 990     }
 991     const group = fast.slice(0, 3);
 992     const target = this.ecoTarget(group[0]);
 993     if (!target) {
 994       this.nextHarassAt = g.time + 30;
 995       return;
 996     }
 997     const sent = this.sendTo(group, target);
 998     for (const u of sent) u.role = 'harass';
 999     if (sent.length) {
1000       this.harassSent++;
1001       if (this.firstHarassAt < 0) this.firstHarassAt = g.time;
1002       this.lastHarass = { t: g.time, target: target.type, owner: target.owner, units: sent.map((u) => u.type) };
1003       this.nextHarassAt = g.time + 80 + Math.random() * 40;
1004     } else this.nextHarassAt = g.time + 30;
1005   }
1006
1007   // Aliados (Liga y Equipos): si atacan la base de un aliado, va la mitad del ejército de casa.
1008   helpAllies(units) {
1009     const g = this.g;
1010     if (g.time - (this._helpScan || -99) < 3) return;
1011     this._helpScan = g.time;
1012     // Las tropas que ya ayudaron vuelven a casa.
1013     for (const u of units) {
1014       if (u.role === 'help' && u.state === 'idle') {
1015         u.role = 'home';
1016         g.orderMove([u], this.p.core.cell, false);
1017       }
1018     }
1019     const me = this.p;
1020     for (const ally of g.players) {
1021       if (ally.id === this.pid || !ally.alive || ally.team !== me.team || !ally.core || !ally.core.alive) continue;
1022       if (g.time - (this.helpAt[ally.id] || -99) < 40) continue;
1023       const ac = g.cells[ally.core.cell];
1024       let threat = null;
1025       let n = 0;
1026       for (const e of g.units) {
1027         if (!e.alive || e.inside || !g.isEnemy(e.owner, this.pid) || !e.def.weapon) continue;
1028         if (g.map.dist(g.cells[e.cell], ac) <= 6) { n++; if (!threat) threat = e; }
1029       }
1030       if (!threat || n < 2) continue;
1031       const tc = g.cells[threat.cell];
1032       const home = units.filter((u) => u.def.weapon && u.role === 'home' && !u.inside && (u.state === 'idle' || u.state
=== 'move') && g.canAttack(u, u.def.weapon, threat) &&
1033         (u.mover !== 'land' || g.cells[u.cell].comp === tc.comp));
```

```
      if (home.length < 3) continue;
      const go = home.slice(0, Math.min(8, Math.ceil(home.length / 2)));
      const sent = this.sendTo(go, threat.kind === 'bld' ? threat : { cell: tc.i });
      for (const u of sent) u.role = 'help';
      if (sent.length) {
        this.helpAt[ally.id] = g.time;
        this.helpSent++;
        this.lastHelp = { t: g.time, ally: ally.id, units: sent.length };
        if (ally.human) g.say(this.pid, '¡Voy a ayudarte!', { x: tc.x, z: tc.z, queue: 2 });
      }
    }
  }

  // Siguiente objetivo de una tropa en oleada. En Difícil, primero las defensas cercanas.
  nextTarget(from, comp) {
    const g = this.g;
    let best = null;
    let bd = Infinity;
    for (const b of g.buildings) {
      if (!b.alive || !g.isEnemy(b.owner, this.pid)) continue;
      const c = g.cells[b.cell];
      if (comp !== undefined && c.comp !== comp && !c.nb.some((ni) => g.cells[ni].comp === comp)) continue;
      let d = g.map.dist(c, from);
      if (this.tac.focusDefense && b.def.weapon && b.type !== 'core' && d <= 6) d -= 10;
      if (d < bd) { bd = d; best = b; }
    }
    return best;
  }

  // Envía escuadras de infantería a capturar Banderas y el Faro; deja un guardia en cada punto.
  capture(units) {
    const g = this.g;
    const T = g.time;
    if (T < 45) return;
    const myTeam = this.p.team;
    const core = this.coreCell();
    const king = !!g.mode.king;
    const squads = new Map();
    const guards = new Map();
    for (const u of units) {
      if (u.role === 'guard') {
        const pt = u.capPoint;
        if (!pt || pt.owner < 0 || g.players[pt.owner].team !== myTeam) {
          u.role = 'cap';
          u.capSince = T;
        } else {
          guards.set(pt, u);
          if (u.state === 'idle' && g.map.dist(g.cells[u.cell], g.cells[pt.cell]) > 1) g.orderMove([u], pt.cell, true);
          continue;
        }
      }
      if (u.role !== 'cap') continue;
      const pt = u.capPoint;
      const held = pt && pt.owner >= 0 && g.players[pt.owner].team === myTeam;
      if (!pt || (held && T - (u.capSince || 0) > (king && pt.type === 'faro' ? 9999 : 90))) {
        if (pt && held && !guards.has(pt) && guards.size < 3) {
          u.role = 'guard';
          guards.set(pt, u);
        } else {
          u.role = 'home';
          u.capPoint = null;
        }
        continue;
      }
      squads.set(pt, (squads.get(pt) || 0) + 1);
      if (u.state === 'idle' && g.map.dist(g.cells[u.cell], g.cells[pt.cell]) > 1) g.orderMove([u], pt.cell, true);
    }
    const maxSquads = (king ? 3 : 2) + this.persona.capture;
    if (squads.size >= maxSquads) return;
    const faroBias = (king ? 30 : 6) + this.persona.capture * 6;
    const targets = g.points
      .filter((pt) => !squads.has(pt) && (pt.owner < 0 || g.players[pt.owner].team !== myTeam))
      .filter((pt) => g.cells[pt.cell].comp === core.comp)
      .map((pt) => ({ pt, d: g.map.dist(g.cells[pt.cell], core) - (pt.type === 'faro' ? faroBias : 0) }))
      .sort((a, b) => a.d - b.d);
    if (!targets.length) return;
    const pt = targets[0].pt;
    const size = pt.type === 'faro' ? (king ? 6 : 4) : 3;
    const free = units.filter((u) => u.cls === 'inf' && u.def.weapon && u.role === 'home' && !u.inside &&
      (u.state === 'idle' || u.state === 'move'));
    if (free.length < size + (T < 200 ? 0 : 2)) return;
    const squad = free.slice(0, size);
    for (const u of squad) {
      u.role = 'cap';
      u.capPoint = pt;
      u.capSince = T;
    }
    g.orderMove(squad, pt.cell, true);
```

```
  }

  // Casilla propia más cercana al objetivo: allí se reúne la oleada antes de salir.
  stagingCell(target) {
    const g = this.g;
    const core = this.coreCell();
    const tc = g.cells[target.cell];
    let best = -1;
    let bd = Infinity;
    for (const c of g.cells) {
      if (c.owner !== this.pid || c.comp !== core.comp || c.bld || !g.passable(c, 'land', this.pid)) continue;
      const d = g.map.dist(c, tc);
      if (d < bd) { bd = d; best = c.i; }
    }
    return best;
  }

  announce(units, target) {
    const g = this.g;
    if (!units.length) return;
    let x = 0;
    let z = 0;
    for (const u of units) { x += u.x; z += u.z; }
    x /= units.length;
    z /= units.length;
    const me = g.players[0];
    const owner = g.players[target.owner];
    if (target.owner === 0 && g.isEnemy(this.pid, 0)) {
      const lines = this.persona.lines.attack;
      g.say(this.pid, lines[Math.floor(Math.random() * lines.length)], { x, z, toMe: true, kind: 'bad', queue: 3 });
    } else if (owner && this.p.team === me.team) {
      g.say(this.pid, ALLY_LINES.attack.replace('{t}', owner.name), { x, z, queue: 1 });
    }
  }

  launch(units, target, announced = false) {
    const g = this.g;
    if (!target || !target.alive) target = this.chooseEnemyCore();
    if (!target) return [];
    let sent = this.sendTo(units, target);
    if (!sent.length) {
      // Sin camino por tierra: ataca el edificio enemigo alcanzable más cercano.
      const alt = this.nextTarget(this.coreCell(), this.coreCell().comp);
      if (alt) sent = this.sendTo(units, alt);
    }
    if (this.tac.focusDefense && sent.length) {
      const tc = g.cells[target.cell];
      const defs = g.buildings.filter((b) => b.alive && g.isEnemy(b.owner, this.pid) && b.def.weapon && b.type !==
'core' &&
        g.map.dist(g.cells[b.cell], tc) <= 5).sort((a, b) => g.map.dist(g.cells[a.cell], tc) -
g.map.dist(g.cells[b.cell], tc));
      if (defs.length) {
        const able = sent.filter((u) => g.canAttack(u, u.def.weapon, defs[0]));
        g.orderAttack(able, defs[0]);
      }
    }
    for (const u of sent) u.role = 'wave';
    if (sent.length) {
      if (!announced) this.announce(sent, target);
      this.wavesSent++;
      this.waveSizes.push(sent.length);
      if (this.firstWaveAt < 0) this.firstWaveAt = g.time;
      this.wave = Math.min(this.wave + 3, 26);
      this.nextWaveAt = g.time + (50 + Math.random() * 40) * this.persona.waveEvery * (this.waveMult || 1);
    }
    return sent;
  }

  attack(units) {
    const g = this.g;
    this.capture(units);
    const army = units.filter((u) => u.def.weapon && !u.inside);
    // Las tropas que terminaron su ataque buscan el siguiente objetivo.
    for (const u of army) {
      if (u.role !== 'wave' || u.state !== 'idle') continue;
      const c = g.cells[u.cell];
      const comp = u.mover === 'land' ? c.comp : undefined;
      const next = this.nextTarget(c, comp);
      if (next) this.sendTo([u], next);
      else {
        u.role = 'home';
        g.orderMove([u], this.p.core.cell, false);
      }
    }
    // Oleada reuniéndose en la frontera: sale cuando está junta o tras 25 s.
    if (this.raid) {
      const r = this.raid;
      r.units = r.units.filter((u) => u.alive && u.role === 'raid');
```

```
      if (!r.units.length) {
        this.raid = null;
        return;
      }
      const st = g.cells[r.stage];
      const ready = r.units.filter((u) => u.mover === 'sea' || g.map.dist(g.cells[u.cell], st) <= 3).length;
      // No sale mientras su aviso de ataque espera turno (el límite de mensajes es de 20 s).
      const warning = this.p.pendingSay && this.p.pendingSay.opts.queue === 3;
      if ((ready >= r.units.length * 0.8 && !warning) || g.time - r.start > 25) {
        // Las rezagadas (a más de 5 casillas) se quedan en casa: nadie llega solo al combate.
        const go = r.units.filter((u) => u.mover !== 'land' || g.map.dist(g.cells[u.cell], st) <= 5);
        for (const u of r.units) if (!go.includes(u)) u.role = 'home';
        this.raid = null;
        if (go.length < 3) {
          for (const u of go) u.role = 'home';
          this.nextWaveAt = g.time + 30;
          return;
        }
        const land = go.filter((u) => u.mover === 'land');
        const near = land.filter((u) => g.map.dist(g.cells[u.cell], st) <= 4).length;
        this.lastLaunch = { together: land.length ? near / land.length : 1, size: go.length, gathered: g.time - r.start
};
        this.launch(go, r.target, true);
      }
      return;
    }
    // Romper empates (v0.8, desde Normal): pasados 15 min, si la oleada está lista sale con todo lo
    // que hay en casa (los guardias siguen en sus Banderas), dejando 3 defensores.
    if (this.tac.focus && g.time > 900 && g.time >= this.nextWaveAt) {
      const home = army.filter((u) => u.role === 'home');
      if (home.length >= 12) {
        const target = this.chooseEnemyCore();
        if (target) {
          const keep = home.filter((u) => u.mover === 'land').slice(0, 3);
          if (this.launch(home.filter((u) => !keep.includes(u)), target).length) {
            this.allIn = (this.allIn || 0) + 1;
            return;
          }
        }
      }
    }
    if (g.time < this.nextWaveAt) return;
    // v0.5 empieza sin tropas: la primera oleada pide menos y crece con el tiempo.
    // Si se atrasa más de 1 min, también salen los guardias de las Banderas.
    const late = g.time > this.nextWaveAt + 60;
    let need = Math.min(this.wave, Math.max(5, Math.floor(this.p.popCap * 0.5)), 3 + Math.floor(g.time / 150));
    if (late) need = Math.max(3, need - 2);
    if (army.length < need) return;
    const target = this.chooseEnemyCore();
    if (!target) return;
    // Solo van las tropas que pueden llegar: las de tierra, si hay camino por tierra hasta el Núcleo.
    const tc = g.cells[target.cell];
    const reach = (u) => u.mover !== 'land' || g.cells[u.cell].comp === tc.comp || tc.nb.some((ni) => g.cells[ni].comp
=== g.cells[u.cell].comp);
    const home = army.filter((u) => (u.role === 'home' || (late && u.role === 'guard')) && reach(u));
    if (home.length < need) return;
    if (this.tac.group) {
      // Solo se reúnen las tropas de tierra de la isla principal (más aéreas y navales).
      const coreComp = this.coreCell().comp;
      const raiders = home.filter((u) => u.mover !== 'land' || g.cells[u.cell].comp === coreComp);
      let stage = this.stagingCell(target);
      if (stage < 0) stage = this.p.core.cell;
      const landers = raiders.filter((u) => u.mover !== 'sea');
      if (raiders.length >= Math.min(need, 5)) {
        for (const u of raiders) u.role = 'raid';
        if (landers.length) g.orderMove(landers, stage, true, this.tac.line !== false ? 'line' : 'block');
        this.raid = { units: raiders, target, stage, start: g.time };
        this.announce(raiders, target);
        return;
      }
    }
    this.launch(home, target);
  }
}
```

faro/src/js/06-render.js · 1,829 líneas

```
'use strict';
// Escena 3D: terreno instanciado, decoración, fronteras, entidades, efectos,
// barras de vida (capa 2D), minimapa y cámara.

class Renderer {
  constructor(gl, canvas, overlay, game, quality) {
    this.game = game;
    this.canvas = canvas;
    this.overlay = overlay;
    this.octx = overlay.getContext('2d');
    this.quality = quality;
    this.renderer = gl;
    this.renderer.shadowMap.type = THREE.PCFSoftShadowMap;
    this.applyQuality(quality);

    const scene = new THREE.Scene();
    this.scene = scene;
    const sky = 0x8fcbe0;
    scene.background = new THREE.Color(sky);
    scene.fog = new THREE.Fog(sky, 70, 140);
    this.camera = new THREE.PerspectiveCamera(42, 1, 0.5, 400);

    scene.add(new THREE.HemisphereLight(0xe8f6ff, 0x4d6a45, 0.62));
    // Luz de ambiente del cielo de Poly Haven para los modelos de Blender.
    if (ENV_MAP) scene.environment = ENV_MAP;
    const sun = new THREE.DirectionalLight(0xfff1dc, 0.88);
    sun.castShadow = true;
    sun.shadow.mapSize.set(2048, 2048);
    const sc = sun.shadow.camera;
    sc.left = -30; sc.right = 30; sc.top = 30; sc.bottom = -30; sc.near = 1; sc.far = 120;
    sun.shadow.bias = -0.0008;
    scene.add(sun);
    scene.add(sun.target);
    this.sun = sun;

    const home = game.cells[game.map.homes[0]];
    this.cam = { tx: home.x, tz: home.z, dist: 24, yaw: Math.atan2(home.x, home.z), pitch: 0.95 };
    this.vis = new Map();
    this.raycaster = new THREE.Raycaster();
    this._ndc = new THREE.Vector2();
    this._plane = new THREE.Plane(new THREE.Vector3(0, 1, 0), 0);
    this._p = new THREE.Vector3();
    this.selection = [];
    this.placement = null;
    this.hoverCell = -1;
    this.markers = [];
    this.pings = [];
    this.lastTerritory = -1;
    this.lastBuildCount = -1;
    this.miniDirty = true;

    this.buildTerrain();
    this.buildDecor();
    this.buildBorders();
    this.buildFx();
    this.buildPoints();
    this.buildShields();
    this.resize();
  }

  // «Alta calidad» o «Más fluido» (la «Automática» elige entre las dos según los cuadros por segundo).
  applyQuality(q) {
    this.quality = q;
    const dpr = window.devicePixelRatio || 1;
    // v0.9: Alta llega a 2× en pantallas Retina (un poco más nítida).
    this.renderer.setPixelRatio(q === 'high' ? Math.min(dpr, 2) : 1);
    this.renderer.shadowMap.enabled = q === 'high';
    this.renderer.shadowMap.needsUpdate = true;
    // Más fluido: sin la luz de ambiente del cielo y la mitad de partículas.
    if (this.scene) this.scene.environment = q === 'high' && ENV_MAP ? ENV_MAP : null;
    if (this.fxs) this.fxs.scale = q === 'high' ? 1 : 0.5;
    // v0.9: Más fluido usa materiales Lambert (más livianos) y Alta un reflejo del cielo algo más fuerte.
    FAST_MATS = q !== 'high';
    GLB_MAT.envMapIntensity = 0.68;
    if (this.scene) {
      this.scene.traverse((o) => {
        if (!o.material) return;
        if (o.isMesh) o.material = fastMat(o.material, FAST_MATS);
        o.material.needsUpdate = true;
      });
    }
    this.resize();
  }

```

```
  resize() {
    const w = this.canvas.clientWidth || window.innerWidth;
    const h = this.canvas.clientHeight || window.innerHeight;
    this.width = w;
    this.height = h;
    this.renderer.setSize(w, h, false);
    if (this.camera) {
      this.camera.aspect = w / h;
      this.camera.fov = w < h ? 55 : 42;
      this.camera.updateProjectionMatrix();
    }
    const dpr = Math.min(window.devicePixelRatio || 1, 2);
    this.overlay.width = Math.round(w * dpr);
    this.overlay.height = Math.round(h * dpr);
    this.octx.setTransform(dpr, 0, 0, dpr, 0, 0);
    if (this.fxs && this.camera) {
      // Tamaño de las partículas en píxeles reales del dibujo.
      const hpx = h * this.renderer.getPixelRatio();
      this.fxs.setScale(hpx / (2 * Math.tan((this.camera.fov * Math.PI) / 360)));
    }
  }

  // ---------- terreno ----------
  // v0.9: vuelve el suelo de la v0.6 (colores lisos por terreno, sin texturas).
  buildTerrain() {
    const cells = this.game.cells;
    const geo = new THREE.CylinderGeometry(0.985, 0.985, 1, 6, 1, false, Math.PI / 6);
    geo.translate(0, -0.5, 0);
    const mat = new THREE.MeshStandardMaterial({ flatShading: true, roughness: 0.95, metalness: 0 });
    const mesh = new THREE.InstancedMesh(geo, mat, cells.length);
    mesh.receiveShadow = true;
    const m = new THREE.Matrix4();
    const rng = mulberry32(this.game.opts.seed + 5);
    this.baseColors = [];
    for (const c of cells) {
      const h = c.h;
      m.makeScale(1, h + 1.4, 1);
      m.setPosition(c.x, h, c.z);
      mesh.setMatrixAt(c.i, m);
      const col = new THREE.Color(terrainColor(this.game.map.kind, c.t));
      const v = (rng() - 0.5) * (TERRAIN[c.t].water ? 0.04 : 0.09);
      col.offsetHSL(0, 0, v);
      this.baseColors.push(col);
      mesh.setColorAt(c.i, col);
    }
    mesh.instanceMatrix.needsUpdate = true;
    this.terrain = mesh;
    this.scene.add(mesh);

    const ocean = new THREE.Mesh(
      new THREE.PlaneGeometry(600, 600),
      new THREE.MeshStandardMaterial({ color: 0x175a85, roughness: 0.6 }),
    );
    ocean.rotation.x = -Math.PI / 2;
    ocean.position.y = -0.09;
    ocean.receiveShadow = true;
    this.scene.add(ocean);
  }

  tintTerritory() {
    const g = this.game;
    const tmp = new THREE.Color();
    for (const c of g.cells) {
      tmp.copy(this.baseColors[c.i]);
      if (c.owner >= 0) tmp.lerp(new THREE.Color(PLAYER_COLORS[c.owner]), TERRAIN[c.t].water ? 0.1 : 0.16);
      this.terrain.setColorAt(c.i, tmp);
    }
    this.terrain.instanceColor.needsUpdate = true;
  }

  buildDecor() {
    const g = this.game;
    const rng = mulberry32(g.opts.seed + 9);
    const trees = [];
    const peaks = [];
    const golds = [];
    const oils = [];
    const rocks = [];
    for (const c of g.cells) {
      if (c.t === 'forest') {
        const k = 2 + Math.floor(rng() * 2);
        c.trees = [];
        for (let i = 0; i < k; i++) {
          const a = rng() * Math.PI * 2;
          const r = 0.2 + rng() * 0.45;
          c.trees.push(trees.length);
          trees.push({ x: c.x + Math.cos(a) * r, y: c.h, z: c.z + Math.sin(a) * r, s: 0.75 + rng() * 0.5, c: c.i });
        }
```

```
      } else if (c.t === 'mount') {
        peaks.push({ x: c.x + (rng() - 0.5) * 0.2, y: c.h, z: c.z + (rng() - 0.5) * 0.2, s: 0.85 + rng() * 0.35, c: c.i
});
      } else if (c.t === 'gold') {
        c.golds = [];
        for (let i = 0; i < 4; i++) {
          const a = (i / 4) * Math.PI * 2 + rng();
          c.golds.push(golds.length);
          golds.push({ x: c.x + Math.cos(a) * 0.4, y: c.h + 0.1, z: c.z + Math.sin(a) * 0.4, s: 0.12 + rng() * 0.06, c:
c.i });
        }
      } else if (c.t === 'oil') {
        c.oil = oils.length;
        oils.push({ x: c.x, y: c.h + 0.015, z: c.z, s: 0.62, c: c.i });
      } else if ((c.t === 'grass' || c.t === 'sand') && rng() < 0.07) {
        rocks.push({ x: c.x + (rng() - 0.5) * 1.1, y: c.h, z: c.z + (rng() - 0.5) * 1.1, s: 0.09 + rng() * 0.08, c: c.i
});
      }
    }
    const inst = (geo, mat, list, place, shadow = true) => {
      const mesh = new THREE.InstancedMesh(geo, mat, Math.max(1, list.length));
      mesh.count = list.length;
      const m = new THREE.Matrix4();
      list.forEach((it, i) => mesh.setMatrixAt(i, place(it, m)));
      mesh.castShadow = shadow;
      mesh.receiveShadow = true;
      this.scene.add(mesh);
      return mesh;
    };
    const flat = (color, extra = {}) => new THREE.MeshStandardMaterial({ color, flatShading: true, roughness: 0.9,
...extra });
    const trunkGeo = new THREE.CylinderGeometry(0.05, 0.06, 0.25, 5);
    trunkGeo.translate(0, 0.125, 0);
    const crownGeo = new THREE.ConeGeometry(0.26, 0.62, 6);
    crownGeo.translate(0, 0.52, 0);
    const placeTree = (it, m) => m.makeScale(it.s, it.s, it.s).setPosition(it.x, it.y, it.z);
    this.trees = trees;
    this.treeTrunks = inst(trunkGeo, flat(0x7a5232), trees, placeTree);
    this.treeCrowns = inst(crownGeo, flat(0x3f8a3a), trees, placeTree);
    const tc = new THREE.Color();
    trees.forEach((t, i) => {
      tc.set(0x3f8a3a).offsetHSL((rng() - 0.5) * 0.04, 0, (rng() - 0.5) * 0.12);
      this.treeCrowns.setColorAt(i, tc);
    });

    const peakGeo = new THREE.ConeGeometry(0.88, 1.5, 6);
    peakGeo.translate(0, 0.75, 0);
    const capGeo = new THREE.ConeGeometry(0.36, 0.6, 6);
    capGeo.translate(0, 1.21, 0);
    const placePeak = (it, m) => m.makeScale(it.s, it.s, it.s).setPosition(it.x, it.y, it.z);
    inst(peakGeo, flat(0x8f8a80), peaks, placePeak);
    inst(capGeo, flat(0xf4f6f8), peaks, placePeak);

    this.golds = golds;
    this.goldMesh = inst(new THREE.OctahedronGeometry(1, 0),
      flat(0xffcf3a, { emissive: 0x8a6400, emissiveIntensity: 0.5, roughness: 0.4 }), golds,
      (it, m) => m.makeScale(it.s, it.s * 1.5, it.s).setPosition(it.x, it.y + it.s, it.z));
    this.oils = oils;
    const oilGeo = new THREE.CylinderGeometry(1, 1, 0.02, 10);
    this.oilMesh = inst(oilGeo, flat(0x161a20, { roughness: 0.25, metalness: 0.4 }), oils,
      (it, m) => m.makeScale(it.s, 1, it.s * 0.8).setPosition(it.x, it.y, it.z), false);
    inst(new THREE.DodecahedronGeometry(1, 0), flat(0x9a978f), rocks,
      (it, m) => m.makeScale(it.s, it.s * 0.7, it.s).setPosition(it.x, it.y + it.s * 0.4, it.z));
  }

  // Oculta árboles, oro y petróleo bajo los edificios.
  refreshDecor() {
    const g = this.game;
    const m = new THREE.Matrix4();
    const zero = new THREE.Matrix4().makeScale(0, 0, 0);
    this.trees.forEach((t, i) => {
      const hidden = !!g.cells[t.c].bld;
      const mat = hidden ? zero : m.makeScale(t.s, t.s, t.s).setPosition(t.x, t.y, t.z);
      this.treeTrunks.setMatrixAt(i, mat);
      this.treeCrowns.setMatrixAt(i, mat);
    });
    this.treeTrunks.instanceMatrix.needsUpdate = true;
    this.treeCrowns.instanceMatrix.needsUpdate = true;
    this.golds.forEach((it, i) => {
      const hidden = !!g.cells[it.c].bld;
      this.goldMesh.setMatrixAt(i, hidden ? zero : m.makeScale(it.s, it.s * 1.5, it.s).setPosition(it.x, it.y + it.s,
it.z));
    });
    this.goldMesh.instanceMatrix.needsUpdate = true;
    this.oils.forEach((it, i) => {
      const hidden = !!g.cells[it.c].bld;
      this.oilMesh.setMatrixAt(i, hidden ? zero : m.makeScale(it.s, 1, it.s * 0.8).setPosition(it.x, it.y, it.z));
    });
```

```
    this.oilMesh.instanceMatrix.needsUpdate = true;
  }

  // ---------- fronteras de territorio ----------
  buildBorders() {
    this.borderMat = new THREE.MeshBasicMaterial({ vertexColors: true, transparent: true, opacity: 0.92, depthWrite:
false });
    this.borderMesh = new THREE.Mesh(new THREE.BufferGeometry(), this.borderMat);
    this.borderMesh.renderOrder = 2;
    this.scene.add(this.borderMesh);
  }

  rebuildBorders() {
    const g = this.game;
    const pos = [];
    const col = [];
    const tmp = new THREE.Color();
    for (const c of g.cells) {
      if (c.owner < 0) continue;
      tmp.set(PLAYER_COLORS[c.owner]);
      const y = Math.max(c.h, 0) + 0.035;
      for (const [dq, dr] of HEX_DIRS) {
        const n = g.map.get(c.q + dq, c.r + dr);
        if (n && n.owner === c.owner) continue;
        const w = hexToWorld(dq, dr);
        const a = Math.atan2(w.z, w.x);
        const a1 = a - Math.PI / 6;
        const a2 = a + Math.PI / 6;
        const o = 0.97;
        const i = 0.8;
        const p1 = [c.x + Math.cos(a1) * o, y, c.z + Math.sin(a1) * o];
        const p2 = [c.x + Math.cos(a2) * o, y, c.z + Math.sin(a2) * o];
        const p3 = [c.x + Math.cos(a2) * i, y, c.z + Math.sin(a2) * i];
        const p4 = [c.x + Math.cos(a1) * i, y, c.z + Math.sin(a1) * i];
        pos.push(...p1, ...p3, ...p2, ...p1, ...p4, ...p3);
        for (let k = 0; k < 6; k++) col.push(tmp.r, tmp.g, tmp.b);
      }
    }
    const geo = new THREE.BufferGeometry();
    geo.setAttribute('position', new THREE.Float32BufferAttribute(pos, 3));
    geo.setAttribute('color', new THREE.Float32BufferAttribute(col, 3));
    this.borderMesh.geometry.dispose();
    this.borderMesh.geometry = geo;
  }

  // ---------- efectos ----------
  buildFx() {
    const MAXT = 400;
    const tg = new THREE.BufferGeometry();
    this.tracerPos = new Float32Array(MAXT * 6);
    this.tracerCol = new Float32Array(MAXT * 6);
    tg.setAttribute('position', new THREE.BufferAttribute(this.tracerPos, 3));
    tg.setAttribute('color', new THREE.BufferAttribute(this.tracerCol, 3));
    this.tracerLines = new THREE.LineSegments(tg, new THREE.LineBasicMaterial({ vertexColors: true, transparent: true,
opacity: 0.95 }));
    this.tracerLines.frustumCulled = false;
    this.scene.add(this.tracerLines);
    this.tracers = [];
    this.MAXT = MAXT;

    // Partículas (v0.6): fuego, humo, restos, chispas, fogonazos, estelas y agua.
    this.fxs = new FxSystem(this.scene);
    this.fxs.scale = this.quality === 'high' ? 1 : 0.5;

    this.projMeshes = { rocket: [], shell: [], torpedo: [], missile: [], bomb: [], nuke: [] };
    this.projGeo = {
      bomb: new THREE.CylinderGeometry(0.06, 0.045, 0.3, 8).rotateX(Math.PI / 2),
      nuke: new THREE.CylinderGeometry(0.1, 0.13, 0.95, 10).rotateX(Math.PI / 2),
      rocket: new THREE.CylinderGeometry(0.035, 0.035, 0.28, 6).rotateX(Math.PI / 2),
      missile: new THREE.CylinderGeometry(0.05, 0.05, 0.42, 6).rotateX(Math.PI / 2),
      shell: new THREE.SphereGeometry(0.07, 6, 5),
      torpedo: new THREE.CylinderGeometry(0.05, 0.05, 0.35, 6).rotateX(Math.PI / 2),
    };
    this.projMat = {
      bomb: new THREE.MeshBasicMaterial({ color: 0x3a3f36 }),
      nuke: new THREE.MeshBasicMaterial({ color: 0xf4f4f0 }),
      rocket: new THREE.MeshBasicMaterial({ color: 0xffe08a }),
      missile: new THREE.MeshBasicMaterial({ color: 0xfff6e6 }),
      shell: new THREE.MeshBasicMaterial({ color: 0x2b2b2b }),
      torpedo: new THREE.MeshBasicMaterial({ color: 0xd8f3ff }),
    };

    const ringGeo = new THREE.RingGeometry(0.46, 0.56, 28).rotateX(-Math.PI / 2);
    this.ringGeo = ringGeo;
    this.rings = [];
    this.ringMatOwn = new THREE.MeshBasicMaterial({ color: 0xffffff, transparent: true, opacity: 0.9, depthWrite: false
});
    this.ringMatEnemy = new THREE.MeshBasicMaterial({ color: 0xff5050, transparent: true, opacity: 0.9, depthWrite:
```

```
false });
    this.markerMat = new THREE.MeshBasicMaterial({ color: 0x7dffa8, transparent: true, opacity: 1, depthWrite: false
});

    this.rangeRing = new THREE.Mesh(new THREE.RingGeometry(0.985, 1, 72).rotateX(-Math.PI / 2),
      new THREE.MeshBasicMaterial({ color: 0xffffff, transparent: true, opacity: 0.45, depthWrite: false }));
    this.rangeRing.renderOrder = 5;
    this.rangeRing.visible = false;
    this.scene.add(this.rangeRing);

    const hexGeo = new THREE.CircleGeometry(0.9, 6).rotateX(-Math.PI / 2);
    this.placeMesh = new THREE.InstancedMesh(hexGeo, new THREE.MeshBasicMaterial({ color: 0xffd75e, transparent: true,
opacity: 0.45, depthWrite: false }), this.game.cells.length);
    this.placeMesh.count = 0;
    this.placeMesh.renderOrder = 3;
    this.scene.add(this.placeMesh);
    this.hoverMesh = new THREE.Mesh(hexGeo, new THREE.MeshBasicMaterial({ color: 0xffffff, transparent: true, opacity:
0.55, depthWrite: false }));
    this.hoverMesh.visible = false;
    this.hoverMesh.renderOrder = 4;
    this.scene.add(this.hoverMesh);

    const flag = new PartBuilder();
    flag.box(0.04, 0.9, 0.04, PAL.dark, 0, 0, 0);
    flag.box(0.34, 0.22, 0.02, 0xffffff, 0.17, 0.62, 0);
    this.rallyFlag = new THREE.Mesh(flag.build(), fastMat(MODEL_MAT, FAST_MATS));
    this.rallyFlag.visible = false;
    this.scene.add(this.rallyFlag);
  }

  // ¿Cae en el agua? Entonces salpica.
  overWater(x, z) {
    const h = worldToHex(x, z);
    const c = this.game.map.get(h.q, h.r);
    return !!(c && TERRAIN[c.t].water && !c.bld);
  }

  spawnBoom(x, y, z, s, small, bld) {
    if (small) {
      if (this.overWater(x, z)) this.fxs.splash(x, z, s);
      else this.fxs.impact(x, y, z, s);
    } else {
      this.fxs.explosion(x, y, z, s, bld);
      if (this.overWater(x, z)) this.fxs.splash(x, z, s * 1.2);
    }
  }

  // Láser del Núcleo (punto 17): un rayo ámbar brillante que dura un instante.
  spawnLaser(f) {
    if (!this.lasers) {
      this.lasers = [];
      this.laserGeo = new THREE.CylinderGeometry(1, 1, 1, 8, 1, true).rotateX(Math.PI / 2);
    }
    let L = this.lasers.find((l) => l.t >= l.dur);
    if (!L) {
      if (this.lasers.length >= 12) return;
      const outer = new THREE.Mesh(this.laserGeo, new THREE.MeshBasicMaterial({ color: 0xffb63a, transparent: true,
opacity: 0.6, depthWrite: false, blending: THREE.AdditiveBlending }));
      const inner = new THREE.Mesh(this.laserGeo, new THREE.MeshBasicMaterial({ color: 0xfff4d6, transparent: true,
opacity: 0.95, depthWrite: false }));
      const grp = new THREE.Group();
      grp.add(outer);
      grp.add(inner);
      this.scene.add(grp);
      L = { grp, outer, inner, t: 1, dur: 0.18 };
      this.lasers.push(L);
    }
    const a = new THREE.Vector3(f.x1, f.y1, f.z1);
    const b = new THREE.Vector3(f.x2, f.y2, f.z2);
    const len = a.distanceTo(b);
    L.grp.position.copy(a).add(b).multiplyScalar(0.5);
    L.grp.lookAt(b);
    L.outer.scale.set(0.09, 0.09, len);
    L.inner.scale.set(0.035, 0.035, len);
    L.grp.visible = true;
    L.t = 0;
  }

  updateLasers(dt) {
    if (!this.lasers) return;
    for (const L of this.lasers) {
      if (L.t >= L.dur) { L.grp.visible = false; continue; }
      L.t += dt;
      const k = 1 - L.t / L.dur;
      L.outer.material.opacity = 0.6 * k;
      L.inner.material.opacity = 0.95 * k;
    }
    this.laserCount = this.lasers.filter((L) => L.t < L.dur).length;
```

```
  }

  spawnHeal(x, y, z) {
    this.fxs.heal(x, y, z);
  }

  // Explosión del misil nuclear: destello enorme, bola de fuego y columna de humo (sin nada violento).
  spawnNuke(x, y, z) {
    this.fxs.flash(x, y + 1, z, 9);
    this.fxs.explosion(x, y + 0.3, z, 4, true);
    for (let i = 0; i < 10; i++) {
      const a = (i / 10) * Math.PI * 2;
      this.fxs.explosion(x + Math.cos(a) * 3.5, y + 0.2, z + Math.sin(a) * 3.5, 1.6, true);
    }
    for (let i = 0; i < 26; i++) this.fxs.smoke(x + (Math.random() - 0.5) * 1.5, y + i * 0.35, z + (Math.random() -
0.5) * 1.5, 2.6);
    this.nukeFlashT = 1.2;
  }

  // Radiación (v1.0): fuego y humo sueltos dentro de la zona mientras dura.
  spawnRadBurn(f) {
    const n = this.quality === 'high' ? 3 : 1;
    for (let i = 0; i < n; i++) {
      const a = Math.random() * Math.PI * 2;
      const d = Math.sqrt(Math.random()) * f.r * 0.85;
      const x = f.x + Math.cos(a) * d;
      const z = f.z + Math.sin(a) * d;
      if (Math.random() < 0.45) this.fxs.explosion(x, f.y + 0.15, z, 0.45, false);
      this.fxs.smoke(x, f.y + 0.4, z, 1.2);
    }
  }

  // Disco verde de la zona de radiación, que late y se apaga al final.
  updateRadZones(time) {
    const g = this.game;
    const zones = g.radZones || [];
    if (!this.radMeshes) this.radMeshes = new Map();
    for (const [z, m] of this.radMeshes) {
      if (!zones.includes(z)) {
        this.scene.remove(m);
        m.material.dispose();
        this.radMeshes.delete(z);
      }
    }
    const R = NUKE.radius * SQ3 + 0.6;
    for (const z of zones) {
      let m = this.radMeshes.get(z);
      if (!m) {
        if (!this.radGeo) {
          this.radGeo = new THREE.CircleGeometry(1, 40);
          this.radGeo.rotateX(-Math.PI / 2);
        }
        m = new THREE.Mesh(this.radGeo, new THREE.MeshBasicMaterial({ color: 0x7cff4a, transparent: true, opacity: 0.3,
depthWrite: false, blending: THREE.AdditiveBlending }));
        m.scale.setScalar(R);
        m.position.set(z.x, z.y + 0.08, z.z);
        m.renderOrder = 3;
        this.scene.add(m);
        this.radMeshes.set(z, m);
      }
      const left = Math.max(0, z.until - g.time);
      const fade = Math.min(1, left / 8);
      m.material.opacity = (0.16 + 0.1 * Math.sin(time * 3)) * fade;
    }
  }

  consumeFx() {
    const g = this.game;
    for (const f of g.fx) {
      if (f.k === 'laser') this.spawnLaser(f);
      else if (f.k === 'tracer') {
        if (this.tracers.length < this.MAXT) this.tracers.push({ ...f, t: 0, dur: f.heavy ? 0.12 : 0.07 });
        // Fogonazo en el cañón y chispa donde pega.
        if (f.heavy || Math.random() < 0.6) this.fxs.flash(f.x1, f.y1, f.z1, f.heavy ? 0.36 : 0.24);
        if (Math.random() < 0.35) this.fxs.sparks(f.x2, f.y2, f.z2, 0.4, 1);
      } else if (f.k === 'boom') this.spawnBoom(f.x, f.y, f.z, f.s, f.small, f.bld);
      else if (f.k === 'spark') this.fxs.sparks(f.x, f.y, f.z, f.s, 2);
      else if (f.k === 'heal') this.spawnHeal(f.x, f.y, f.z);
      else if (f.k === 'flare') {
        // Bengalas del Apache: chispas brillantes hacia los lados.
        for (let i = 0; i < 3; i++) this.fxs.sparks(f.x, f.y, f.z, 1.2, 4);
        this.fxs.flash(f.x, f.y, f.z, 0.6);
      } else if (f.k === 'nuke') this.spawnNuke(f.x, f.y, f.z);
      else if (f.k === 'rad') this.spawnRadBurn(f);
    }
    g.fx.length = 0;
  }

```

```
  updateFx(dt) {
    // Trazadoras
    let n = 0;
    const tmp = new THREE.Color();
    this.tracers = this.tracers.filter((t) => (t.t += dt) < t.dur);
    for (const t of this.tracers) {
      tmp.set(t.color).lerp(new THREE.Color(0xffffff), 0.55);
      const o = n * 6;
      this.tracerPos[o] = t.x1; this.tracerPos[o + 1] = t.y1; this.tracerPos[o + 2] = t.z1;
      this.tracerPos[o + 3] = t.x2; this.tracerPos[o + 4] = t.y2; this.tracerPos[o + 5] = t.z2;
      for (let k = 0; k < 2; k++) {
        this.tracerCol[o + k * 3] = tmp.r;
        this.tracerCol[o + k * 3 + 1] = tmp.g;
        this.tracerCol[o + k * 3 + 2] = tmp.b;
      }
      n++;
    }
    this.tracerLines.geometry.setDrawRange(0, n * 2);
    this.tracerLines.geometry.attributes.position.needsUpdate = true;
    this.tracerLines.geometry.attributes.color.needsUpdate = true;

    this.fxs.update(dt);

    // Proyectiles
    const used = { rocket: 0, shell: 0, torpedo: 0, missile: 0, bomb: 0, nuke: 0 };
    for (const p of this.game.projectiles) {
      const list = this.projMeshes[p.kind];
      if (!list) continue;
      let mesh = list[used[p.kind]];
      if (!mesh) {
        mesh = new THREE.Mesh(this.projGeo[p.kind], this.projMat[p.kind]);
        this.scene.add(mesh);
        list.push(mesh);
      }
      used[p.kind]++;
      const k = Math.min(1, p.time / p.dur);
      const arc = p.kind === 'shell' ? Math.sin(Math.PI * k) * 1.1 : p.kind === 'rocket' ? Math.sin(Math.PI * k) * 0.35
        : p.kind === 'missile' ? Math.sin(Math.PI * k) * 2.6 : p.kind === 'nuke' ? Math.sin(Math.PI * k) * 14 : 0;
      const y = p.kind === 'torpedo' ? -0.02 : p.sy + (p.ty - p.sy) * k + arc;
      mesh.position.set(p.sx + (p.tx - p.sx) * k, y, p.sz + (p.tz - p.sz) * k);
      mesh.lookAt(p.tx, p.kind === 'torpedo' ? -0.02 : p.ty, p.tz);
      mesh.visible = true;
      // Estela de humo detrás de cohetes y misiles (más gruesa en el misil nuclear).
      if ((p.kind === 'rocket' || p.kind === 'missile') && Math.random() < dt * 45 * this.fxs.scale) {
        this.fxs.trail(mesh.position.x, mesh.position.y, mesh.position.z);
      }
      if (p.kind === 'nuke') {
        for (let i = 0; i < 3; i++) this.fxs.trail(mesh.position.x, mesh.position.y, mesh.position.z);
        this.fxs.flash(mesh.position.x, mesh.position.y, mesh.position.z, 0.5);
      }
    }
    for (const kind of Object.keys(this.projMeshes)) {
      const list = this.projMeshes[kind];
      for (let i = used[kind]; i < list.length; i++) list[i].visible = false;
    }

    this.markers = this.markers.filter((mk) => {
      mk.t += dt;
      const k = mk.t / 0.6;
      if (k >= 1) {
        this.scene.remove(mk.mesh);
        return false;
      }
      mk.mesh.scale.setScalar(1.6 - k);
      mk.mesh.material.opacity = 1 - k;
      return true;
    });
  }

  addPing(x, z, kind = 'bad') {
    this.pings.push({ x, z, kind, t: performance.now() });
    if (this.pings.length > 12) this.pings.shift();
  }

  addMoveMarker(ci, color = 0x7dffa8) {
    const c = this.game.cells[ci];
    const mesh = new THREE.Mesh(this.ringGeo, this.markerMat.clone());
    mesh.material.color.set(color);
    mesh.position.set(c.x, Math.max(c.h, 0) + 0.06, c.z);
    mesh.renderOrder = 5;
    this.scene.add(mesh);
    this.markers.push({ mesh, t: 0 });
  }

  // ---------- Faro, Banderas, Cúpulas y escudos ----------
  buildPoints() {
    this.pointVis = this.game.points.map((pt) => {
      const v = createPointVisual(pt.type);
```

```
      v.group.position.set(pt.x, pt.y, pt.z);
      this.scene.add(v.group);
      return v;
    });
  }

  updatePoints(time) {
    const tmp = new THREE.Color();
    this.game.points.forEach((pt, i) => {
      const v = this.pointVis[i];
      tmp.set(pt.owner >= 0 ? PLAYER_COLORS[pt.owner] : NEUTRAL);
      if (!v.own.color.equals(tmp)) v.own.color.copy(tmp);
      if (v.beam) v.beam.rotation.y = time * 0.9;
      if (v.cloth) v.cloth.rotation.y = Math.sin(time * 2.4 + i) * 0.25;
    });
  }

  buildShields() {
    this.shieldVis = this.game.players.map((p) => {
      const r = DOME.range * SQ3 + 0.5;
      const dome = new THREE.Mesh(new THREE.SphereGeometry(r, 32, 14, 0, Math.PI * 2, 0, Math.PI / 2),
        new THREE.MeshBasicMaterial({ color: 0x4aa8ff, transparent: true, opacity: 0.08, depthWrite: false }));
      const rim = new THREE.Mesh(new THREE.TorusGeometry(r, 0.05, 6, 48).rotateX(Math.PI / 2),
        new THREE.MeshBasicMaterial({ color: 0x4aa8ff, transparent: true, opacity: 0.7, depthWrite: false }));
      dome.add(rim);
      dome.renderOrder = 6;
      dome.visible = false;
      const bubble = new THREE.Mesh(new THREE.SphereGeometry(1.7, 24, 14),
        new THREE.MeshBasicMaterial({ color: 0xffb63a, transparent: true, opacity: 0.3, depthWrite: false }));
      bubble.renderOrder = 6;
      bubble.visible = false;
      this.scene.add(dome);
      this.scene.add(bubble);
      return { dome, rim, bubble };
    });
  }

  updateShields(time) {
    const g = this.game;
    g.players.forEach((p, i) => {
      const v = this.shieldVis[i];
      const core = p.core;
      const domeOn = p.alive && core.alive && p.dome && p.dome.hp > 0;
      v.dome.visible = !!domeOn;
      if (domeOn) {
        const k = p.dome.hp / p.dome.maxHp;
        const col = k > 0.6 ? 0x4aa8ff : k > 0.3 ? 0xf2c94c : 0xef5350;
        v.dome.material.color.set(col);
        v.rim.material.color.set(col);
        const hit = g.time - p.dome.lastHit < 0.25;
        v.dome.material.opacity = hit ? 0.22 : 0.08;
        v.dome.position.set(core.x, core.y, core.z);
      }
      const shield = p.alive && core.alive && core.shieldUntil > g.time;
      v.bubble.visible = !!shield;
      if (shield) {
        v.bubble.position.set(core.x, core.y + 0.6, core.z);
        v.bubble.material.opacity = 0.22 + Math.sin(time * 6) * 0.08;
      }
    });
  }

  // ---------- vista previa de construcción ----------
  setGhost(type, ci, rot, valid) {
    if (this.ghost && (this.ghost.type !== type)) {
      this.scene.remove(this.ghost.group);
      this.ghost = null;
    }
    if (!type || ci < 0) {
      if (this.ghost) this.ghost.group.visible = false;
      return;
    }
    if (!this.ghost) {
      const m = getModel(type === 'wall' && GLB.wall_post ? 'wall_post' : type, PLAYER_COLORS[0]);
      // Holograma ámbar (documento pág. 7); rojo cuando no se puede.
      const okMat = new THREE.MeshBasicMaterial({ color: 0xffb63a, transparent: true, opacity: 0.55, depthWrite: false
});
      const badMat = new THREE.MeshBasicMaterial({ color: 0xff5a5a, transparent: true, opacity: 0.55, depthWrite: false
});
      const group = new THREE.Group();
      const parts = [];
      const pp = m.partPos || {};
      for (const key of ['main', 'turret', 'rotor', 'rocker', 'blades', 'crystal']) {
        if (!m[key]) continue;
        const mesh = new THREE.Mesh(m[key], okMat);
        if (pp[key]) mesh.position.copy(pp[key]);
        else if (key === 'turret') mesh.position.set(0, m.turretY || 0, m.turretZ || 0);
        else if (key === 'rocker') mesh.position.set(0, m.rockerY, m.rockerZ || 0);
```

```
        else if (key === 'blades') mesh.position.set(0, m.bladesY, m.bladesZ);
        else if (key === 'crystal') mesh.position.y = m.crystalY;
        mesh.renderOrder = 7;
        group.add(mesh);
        parts.push(mesh);
      }
      this.scene.add(group);
      this.ghost = { type, group, parts, okMat, badMat };
    }
    const c = this.game.cells[ci];
    const gh = this.ghost;
    gh.group.visible = true;
    gh.group.position.set(c.x, c.h, c.z);
    gh.group.rotation.y = rot;
    for (const mesh of gh.parts) mesh.material = valid ? gh.okMat : gh.badMat;
    this.hoverMesh.material.color.set(valid ? 0x8dffb0 : 0xff6b6b);
    this.setHover(ci);
  }

  // Aspecto equipado en el Inventario: solo para tus tropas y tu Núcleo.
  lookFor(e) {
    const looks = this.game.opts.looks;
    if (!looks || e.owner !== 0) return null;
    return looks[skinSlotOf(e.kind === 'bld' ? 'bld' : 'unit', e.type)] || null;
  }

  // ---------- entidades ----------
  syncEntities(dt, time) {
    const g = this.game;
    let smoking = 0;
    for (const e of g.removed) {
      const v = this.vis.get(e.id);
      if (v) {
        this.scene.remove(v.group);
        this.vis.delete(e.id);
      }
    }
    g.removed.length = 0;

    const myTeam = g.players[0].team;
    for (const b of g.buildings) {
      // Núcleo, Torreta y Enfermería tienen un modelo distinto por nivel.
      const mkey = GLB[b.type + '_' + b.level] ? b.type + '_' + b.level : b.type;
      let v = this.vis.get(b.id);
      if (v && v.mkey !== mkey) {
        this.scene.remove(v.group);
        this.vis.delete(b.id);
        v = null;
      }
      if (!v) {
        v = b.type === 'wall' ? this.createWall(b) : createVisual(mkey, PLAYER_COLORS[b.owner], this.lookFor(b));
        v.mkey = mkey;
        v.group.rotation.y = b.type === 'wall' ? 0 : b.rot;
        v.group.position.set(b.x, b.y, b.z);
        this.scene.add(v.group);
        this.vis.set(b.id, v);
      }
      if (b.type === 'wall') this.updateWall(b, v);
      // Humo de daño: un edificio con menos del 40 % de vida echa humo hasta que lo reparan.
      if (b.built && b.hp < b.maxHp * 0.4) {
        smoking++;
        v.smokeT = (v.smokeT || 0) - dt;
        if (v.smokeT <= 0) {
          v.smokeT = this.quality === 'high' ? 0.32 : 0.65;
          this.fxs.smoke(b.x, b.y + (b.type === 'core' ? 1.1 : 0.6), b.z, b.type === 'core' ? 0.8 : 0.55);
        }
      }
      const k = b.built ? 1 : 0.2 + 0.8 * b.progress;
      v.main.scale.y = k;
      // Andamio mientras se construye o se mejora.
      const working = !b.built || !!b.upgrading;
      if (working && !v.scaffold && GLB.scaffold) {
        v.scaffold = new THREE.Mesh(getModel('scaffold', PLAYER_COLORS[b.owner]).main, fastMat(GLB_MAT, FAST_MATS));
        v.scaffold.rotation.y = -v.group.rotation.y;
        if (b.type === 'core') v.scaffold.scale.set(1.35, 1.6, 1.35);
        v.group.add(v.scaffold);
      }
      if (v.scaffold) v.scaffold.visible = working;
      if (v.turret) {
        v.turret.position.y = v.turretBaseY * k;
        v.turret.rotation.y = b.aimYaw - b.rot;
      }
      if (v.blades && b.built) {
        if (b.type === 'hydro') v.blades.rotation.x += dt * 1.6;
        else v.blades.rotation.z += dt * 2.2;
      }
      if (v.rocker) {
        v.rocker.visible = b.built;
```

```
        v.rocker.rotation.x = b.built ? Math.sin(time * 2.2 + b.id) * (b.type === 'wave' ? 0.12 : 0.28) : 0;
      }
      if (v.rotor && b.built) v.rotor.rotation.y += dt * 1.6;
      if (v.crystal) {
        v.crystal.rotation.y = time * 0.8;
        v.crystal.position.y = (v.crystalBaseY || 1.35) + Math.sin(time * 1.6) * 0.08;
        // v1.5: aro de luz del color del jugador que gira alrededor del cristal del Núcleo.
        if (b.type === 'core') {
          if (!v.halo) {
            const hm = new THREE.MeshBasicMaterial({ color: PLAYER_COLORS[b.owner], transparent: true, opacity: 0.75,
depthWrite: false, blending: THREE.AdditiveBlending });
            v.halo = new THREE.Mesh(new THREE.TorusGeometry(0.36 + 0.05 * b.level, 0.022, 6, 32), hm);
            v.halo2 = new THREE.Mesh(new THREE.TorusGeometry(0.3 + 0.05 * b.level, 0.016, 6, 32), hm);
            v.group.add(v.halo, v.halo2);
          }
          v.halo.position.y = v.crystal.position.y;
          v.halo2.position.y = v.crystal.position.y;
          v.halo.rotation.set(1.15, time * 1.4, 0);
          v.halo2.rotation.set(-1.0, -time * 1.1, 0.4);
        }
      }
    }
    this.smokingCount = smoking;
    // Movimiento de CMU solo en Alta calidad y con hasta 120 soldados a la vista (si no, balanceo simple).
    const gaitFull = !!CMU_GAIT && this.quality === 'high' && (this.soldiersSeen || 0) <= 120;
    let soldiers = 0;
    for (const u of g.units) {
      let v = this.vis.get(u.id);
      if (!v) {
        v = createVisual(u.type, PLAYER_COLORS[u.owner], this.lookFor(u));
        this.scene.add(v.group);
        this.vis.set(u.id, v);
      }
      const seen = !u.inside && (u.visible || g.players[u.owner].team === myTeam);
      v.group.visible = seen;
      if (!seen) continue;
      let y = u.y;
      if (u.mover === 'sea') y += Math.sin(time * 2.2 + u.id) * 0.025;
      if (u.mover === 'air') y += Math.sin(time * 1.7 + u.id) * 0.06;
      if (u.def.plane) {
        this.updatePlane(u, v, dt, time, y);
      } else {
        v.group.position.set(u.x, y, u.z);
        v.group.rotation.y = u.yaw;
        if (u.mover === 'air') v.group.rotation.x = u.moving ? 0.12 : 0;
      }
      if (v.turret) {
        const aiming = u.autoTarget && u.autoTarget.alive;
        const want = aiming ? u.aimYaw - u.yaw : 0;
        v.turret.rotation.y = lerpAngle(v.turret.rotation.y, want, Math.min(1, dt * 6));
      }
      if (v.rotor) v.rotor.rotation.y += dt * 22;
      if (v.legl) {
        soldiers++;
        this.animateLegs(u, v, dt, time, gaitFull);
        // v1.0: el constructor martilla mientras trabaja en la obra.
        if (u.workingAt !== undefined && g.time - u.workingAt < 0.25) v.main.rotation.x = 0.08 + Math.sin(time * 11 +
u.id) * 0.12;
      }
    }
    this.soldiersSeen = soldiers;
    this.updateUnitShields(time);
    this.updateSkyBeams(time);
  }

  // Aviones (v1.0): quietos no flotan en el aire, dan vueltas sobre su punto; se inclinan al girar
  // y dejan estela. Es solo la vista: la posición del juego sigue siendo el centro de la vuelta.
  updatePlane(u, v, dt, time, y) {
    const k = Math.min(1, dt * 1.5);
    v.orbitA = (v.orbitA || 0) + ((u.moving ? 0 : 1) - (v.orbitA || 0)) * k;
    const R = 0.75 * v.orbitA;
    const a = time * 0.8 + u.id * 1.7;
    const x = u.x + Math.cos(a) * R;
    const z = u.z + Math.sin(a) * R;
    // Rumbo: el de la vuelta cuando está quieto, el de su marcha cuando vuela a un destino.
    const orbitYaw = Math.atan2(-Math.sin(a), Math.cos(a));
    const yaw = v.orbitA > 0.5 ? orbitYaw : u.yaw;
    const prev = v.lastYaw === undefined ? yaw : v.lastYaw;
    let turn = yaw - prev;
    while (turn > Math.PI) turn -= Math.PI * 2;
    while (turn < -Math.PI) turn += Math.PI * 2;
    v.lastYaw = yaw;
    const wantBank = clamp(-(turn / Math.max(dt, 1e-3)) * 0.35, -0.6, 0.6);
    v.bank = (v.bank || 0) + (wantBank - (v.bank || 0)) * Math.min(1, dt * 4);
    v.group.position.set(x, y + 0.15, z);
    v.group.rotation.set(u.moving ? 0.06 : 0, yaw, v.bank);
    // Estela detrás de las toberas.
    v.trailT = (v.trailT || 0) - dt;
```

```
    if (v.trailT <= 0) {
      v.trailT = this.quality === 'high' ? 0.05 : 0.15;
      const s = v.group.scale.x || 1;
      this.fxs.trail(x - Math.sin(yaw) * 0.55 * s, y + 0.3 * s, z - Math.cos(yaw) * 0.55 * s);
    }
  }

  // Piernas de los soldados. Alta calidad: ciclo de caminar o correr de CMU (cadera, rodilla,
  // inclinación y rebote), al ritmo de la velocidad real para que los pies no patinen.
  // Más fluido o muchísimas tropas: el balanceo simple de antes.
  animateLegs(u, v, dt, time, full) {
    const moving = u.moving && !u.inside;
    const k = Math.min(1, dt * 8);
    v.gaitA = (v.gaitA || 0) + ((moving ? 1 : 0) - (v.gaitA || 0)) * k;
    const a = v.gaitA;
    if (!full || !v.shinl) {
      const sw = moving ? Math.sin(time * 10 + u.id) * 0.55 : 0;
      v.legl.rotation.x += (sw - v.legl.rotation.x) * Math.min(1, dt * 12);
      v.legr.rotation.x = -v.legl.rotation.x;
      if (v.shinl) { v.shinl.rotation.x = 0; v.shinr.rotation.x = 0; }
      v.main.rotation.x = 0;
      v.main.position.y = 0;
      v.gaitClip = null;
      return;
    }
    const cell = this.game.cells[u.cell];
    const slow = TERRAIN[cell.t].slow || 1;
    // Constructores, Ingenieros y el bosque: caminan. Las tropas de combate corren.
    const clip = u.def.builds || slow > 1 ? CMU_GAIT.walk : CMU_GAIT.run;
    v.gaitClip = clip === CMU_GAIT.walk ? 'walk' : 'run';
    const leg = 0.19 * (v.group.scale.x || 1);
    if (moving) {
      const speed = (u.def.speed * SQ3) / slow;
      v.gaitPhase = ((v.gaitPhase === undefined ? (u.id * 0.37) % 1 : v.gaitPhase) + dt * (speed / (clip.stride *
leg))) % 1;
    }
    const n = clip.hipL.length;
    const t = (v.gaitPhase || 0) * n;
    const i0 = Math.floor(t) % n;
    const i1 = (i0 + 1) % n;
    const f = t - Math.floor(t);
    const at = (arr) => arr[i0] + (arr[i1] - arr[i0]) * f;
    v.legl.rotation.x = -at(clip.hipL) * a;
    v.legr.rotation.x = -at(clip.hipR) * a;
    v.shinl.rotation.x = at(clip.kneeL) * a;
    v.shinr.rotation.x = at(clip.kneeR) * a;
    v.main.rotation.x = at(clip.pitch) * a;
    // Rebote al paso; quieto, respira despacio.
    v.group.position.y += at(clip.bob) * leg * a;
    v.main.position.y = Math.sin(time * 2.4 + u.id) * 0.004 * (1 - a);
  }

  // ---------- muralla: torreón + tramos hacia los muros vecinos ----------
  createWall(b) {
    const color = PLAYER_COLORS[b.owner];
    const v = createVisual(GLB.wall_post ? 'wall_post' : 'wall', color);
    v.segs = new THREE.Group();
    v.group.add(v.segs);
    v.mask = -1;
    return v;
  }

  updateWall(b, v) {
    const g = this.game;
    const c = g.cells[b.cell];
    let mask = 0;
    HEX_DIRS.forEach(([dq, dr], i) => {
      const n = g.map.get(c.q + dq, c.r + dr);
      if (n && n.bld && n.bld.alive && n.bld.type === 'wall' && n.bld.owner === b.owner) mask |= 1 << i;
    });
    if (mask === v.mask) return;
    v.mask = mask;
    while (v.segs.children.length) v.segs.remove(v.segs.children[0]);
    if (!GLB.wall_seg) return;
    const geo = getModel('wall_seg', PLAYER_COLORS[b.owner]).main;
    HEX_DIRS.forEach(([dq, dr], i) => {
      if (!(mask & (1 << i))) return;
      const w = hexToWorld(dq, dr);
      const seg = new THREE.Mesh(geo, fastMat(GLB_MAT, FAST_MATS));
      seg.castShadow = true;
      seg.receiveShadow = true;
      seg.rotation.y = Math.atan2(w.x, w.z);
      v.segs.add(seg);
    });
  }

  // ---------- escudo personal de las tropas (Cúpula nivel 3) ----------
  updateUnitShields(time) {
```

```
    const g = this.game;
    if (!this.shieldMesh) {
      const geo = new THREE.IcosahedronGeometry(1, 1);
      const mat = new THREE.MeshBasicMaterial({ color: 0xffffff, transparent: true, opacity: 0.26, wireframe: true, depthWrite: false });
      this.shieldMesh = new THREE.InstancedMesh(geo, mat, 200);
      this._shCol = new THREE.Color();
      this.shieldMesh.count = 0;
      this.shieldMesh.renderOrder = 6;
      this.shieldMesh.frustumCulled = false;
      this.scene.add(this.shieldMesh);
    }
    const m = new THREE.Matrix4();
    let n = 0;
    const myTeam = g.players[0].team;
    for (const u of g.units) {
      if (n >= 200) break;
      if (!u.alive || u.inside || !(u.shield > 0)) continue;
      if (!u.visible && g.players[u.owner].team !== myTeam) continue;
      const s = (u.cls === 'inf' ? 0.42 : u.cls === 'ship' || u.cls === 'sub' ? 0.85 : 0.6) * (0.96 + Math.sin(time * 3 + u.id) * 0.04);
      m.makeScale(s, s, s).setPosition(u.x, u.y + s * 0.6, u.z);
      this.shieldMesh.setColorAt(n, this._shCol.set(SHIELD_HEX(u.shield / u.shieldMax)));
      this.shieldMesh.setMatrixAt(n++, m);
    }
    this.shieldMesh.count = n;
    this.shieldMesh.instanceMatrix.needsUpdate = true;
    if (this.shieldMesh.instanceColor) this.shieldMesh.instanceColor.needsUpdate = true;
    this.shieldCount = n;
  }

  // ---------- rayo ámbar al cielo sobre cada Núcleo (documento pág. 13) ----------
  // ---------- v1.5: de quién es cada cosa y qué nivel tiene ----------
  // Banderín del color del jugador y marcas de nivel (doradas) en cada edificio; anillo del color del
  // jugador bajo cada tropa. Todo con mallas instanciadas: pocas llamadas de dibujo aunque haya cientos.
  buildMarks() {
    const N = 700;
    const pole = new THREE.CylinderGeometry(0.018, 0.022, 0.66, 5);
    pole.translate(0, 0.33, 0);
    this.flagPoles = new THREE.InstancedMesh(pole, new THREE.MeshLambertMaterial({ color: 0x59616b }), N);
    const flag = new THREE.PlaneGeometry(0.3, 0.18, 3, 1);
    flag.translate(0.15, 0.55, 0);
    this.flags = new THREE.InstancedMesh(flag, new THREE.MeshBasicMaterial({ color: 0xffffff, side: THREE.DoubleSide }), N);
    const pip = new THREE.OctahedronGeometry(0.075, 0);
    pip.scale(1, 0.6, 1);
    this.levelPips = new THREE.InstancedMesh(pip, new THREE.MeshLambertMaterial({ color: 0xffcf3a, emissive: 0x4a3200 }), N * 2);
    const ring = new THREE.RingGeometry(0.74, 1, 22);
    ring.rotateX(-Math.PI / 2);
    this.unitRings = new THREE.InstancedMesh(ring, new THREE.MeshBasicMaterial({ color: 0xffffff, transparent: true, opacity: 0.8, depthWrite: false }), 1200);
    this.unitRings.renderOrder = 2;
    // El color por instancia se crea con el tamaño que tiene la malla en ese momento: antes de vaciarla.
    this.flags.setColorAt(0, new THREE.Color(1, 1, 1));
    this.unitRings.setColorAt(0, new THREE.Color(1, 1, 1));
    for (const m of [this.flagPoles, this.flags, this.levelPips, this.unitRings]) {
      m.frustumCulled = false;
      m.count = 0;
      this.scene.add(m);
    }
    this.flags.castShadow = true;
    this.markColor = new THREE.Color();
    this.markMat = new THREE.Matrix4();
    this.markQuat = new THREE.Quaternion();
    this.markPos = new THREE.Vector3();
    this.markScale = new THREE.Vector3();
    this.markEuler = new THREE.Euler();
  }

  updateMarks(time) {
    if (!this.flags) this.buildMarks();
    const g = this.game;
    const M = this.markMat;
    const Q = this.markQuat;
    const P = this.markPos;
    const S = this.markScale;
    const E = this.markEuler;
    const col = this.markColor;
    let nf = 0;
    let np = 0;
    for (const b of g.buildings) {
      if (!b.alive || b.type === 'wall' || b.type === 'core') continue;
      const v = this.vis.get(b.id);
      if (!v || !v.group.visible) continue;
      if (nf >= 700) break;
      // Esquina de la casilla (detrás y a la derecha, mirando desde la cámara de siempre).
      const a = Math.PI / 6 + Math.PI / 3 * 4;
```

```
      const cx = b.x + Math.sin(a) * 0.74;
      const cz = b.z + Math.cos(a) * 0.74;
      const k = b.built ? 1 : 0.6;
      P.set(cx, b.y, cz);
      S.set(1, k, 1);
      Q.identity();
      M.compose(P, Q, S);
      this.flagPoles.setMatrixAt(nf, M);
      E.set(0, 0.6 + Math.sin(time * 2.6 + b.id) * 0.18, 0);
      Q.setFromEuler(E);
      S.set(1, k, 1);
      M.compose(P, Q, S);
      this.flags.setMatrixAt(nf, M);
      col.setHex(PLAYER_COLORS[b.owner]);
      this.flags.setColorAt(nf, col);
      nf++;
      // Marcas de nivel: una por nivel, al pie del banderín (solo en lo que se puede mejorar).
      if (b.def.maxLevel > 1 && b.built) {
        for (let i = 0; i < b.level && np < 1400; i++) {
          const aa = a - 0.42 - i * 0.2;
          P.set(b.x + Math.sin(aa) * 0.8, b.y + 0.07, b.z + Math.cos(aa) * 0.8);
          E.set(0, time * 0.8 + i, 0);
          Q.setFromEuler(E);
          S.set(1, 1, 1);
          M.compose(P, Q, S);
          this.levelPips.setMatrixAt(np++, M);
        }
      }
    }
    this.flagPoles.count = nf;
    this.flags.count = nf;
    this.levelPips.count = np;
    this.flagPoles.instanceMatrix.needsUpdate = true;
    this.flags.instanceMatrix.needsUpdate = true;
    if (this.flags.instanceColor) this.flags.instanceColor.needsUpdate = true;
    this.levelPips.instanceMatrix.needsUpdate = true;
    // Anillo del color del jugador bajo cada tropa (los aviones, en el suelo debajo de ellos).
    let nr = 0;
    Q.identity();
    for (const u of g.units) {
      if (!u.alive || u.inside || nr >= 1200) continue;
      const v = this.vis.get(u.id);
      if (!v || !v.group.visible) continue;
      const r = u.cls === 'inf' ? 0.27 : u.mover === 'sea' ? 0.56 : u.mover === 'air' ? 0.36 : 0.44;
      const c = g.cells[u.cell];
      const gy = u.mover === 'sea' ? 0.04 : u.mover === 'air' ? (c ? Math.max(0.04, c.h + 0.03) : 0.04) : (c ? c.h +
0.03 : u.y + 0.03);
      const x = u.def.plane ? v.group.position.x : u.x;
      const z = u.def.plane ? v.group.position.z : u.z;
      P.set(x, gy, z);
      S.set(r, 1, r);
      M.compose(P, Q, S);
      this.unitRings.setMatrixAt(nr, M);
      col.setHex(PLAYER_COLORS[u.owner]);
      this.unitRings.setColorAt(nr, col);
      nr++;
    }
    this.unitRings.count = nr;
    this.unitRings.instanceMatrix.needsUpdate = true;
    if (this.unitRings.instanceColor) this.unitRings.instanceColor.needsUpdate = true;
    this.marksShown = { flags: nf, pips: np, rings: nr };
  }

  // v1.5: icono del tipo de tropa cuando la cámara está lejos (las tropas se ven chicas).
  drawUnitIcons(ctx) {
    const g = this.game;
    const myTeam = g.players[0].team;
    let n = 0;
    for (const u of g.units) {
      if (!u.alive || u.inside || n >= 260) continue;
      if (!u.visible && g.players[u.owner].team !== myTeam) continue;
      const s = this.project(u.x, u.y + (u.mover === 'air' ? 0.9 : 0.75), u.z);
      if (!s.ok || s.x < -12 || s.y < -12 || s.x > this.width + 12 || s.y > this.height + 12) continue;
      n++;
      const x = s.x;
      const y = s.y;
      ctx.fillStyle = hexColor(PLAYER_COLORS[u.owner]);
      ctx.strokeStyle = 'rgba(12,16,22,0.85)';
      ctx.lineWidth = 1.5;
      ctx.beginPath();
      ctx.arc(x, y, 7, 0, Math.PI * 2);
      ctx.fill();
      ctx.stroke();
      ctx.fillStyle = '#ffffff';
      ctx.beginPath();
      if (u.cls === 'inf') {
        ctx.arc(x, y - 2.6, 1.7, 0, Math.PI * 2);
        ctx.rect(x - 1.6, y - 0.6, 3.2, 4.2);
```

```
      } else if (u.mover === 'air') {
        ctx.moveTo(x, y - 4.5); ctx.lineTo(x + 4.5, y + 2); ctx.lineTo(x, y + 0.5); ctx.lineTo(x - 4.5, y + 2);
ctx.closePath();
      } else if (u.mover === 'sea') {
        ctx.moveTo(x - 4.5, y - 0.5); ctx.lineTo(x + 4.5, y - 0.5); ctx.lineTo(x + 2.8, y + 3); ctx.lineTo(x - 2.8, y +
3); ctx.closePath();
        ctx.rect(x - 1, y - 3.5, 2.5, 3);
      } else {
        ctx.rect(x - 3.8, y - 1.2, 7.6, 3.8);
        ctx.rect(x - 1.6, y - 3.6, 3.2, 2.4);
        ctx.rect(x + 1.6, y - 2.9, 3.4, 1.1);
      }
      ctx.fill();
    }
    this.unitIcons = n;
  }

  updateSkyBeams(time) {
    const g = this.game;
    if (!this.skyBeams) {
      const geo = new THREE.CylinderGeometry(0.09, 0.22, 26, 10, 1, true);
      geo.translate(0, 13, 0);
      this.skyBeams = g.players.map(() => {
        const mesh = new THREE.Mesh(geo, new THREE.MeshBasicMaterial({
          color: 0xffb63a, transparent: true, opacity: 0.28, depthWrite: false, blending: THREE.AdditiveBlending, side:
THREE.DoubleSide,
        }));
        mesh.renderOrder = 4;
        this.scene.add(mesh);
        return mesh;
      });
    }
    g.players.forEach((p, i) => {
      const beam = this.skyBeams[i];
      const c = p.core;
      beam.visible = p.alive && c.alive;
      if (!beam.visible) return;
      beam.position.set(c.x, c.y + 1.2, c.z);
      beam.material.opacity = 0.18 + Math.sin(time * 2 + i) * 0.05;
      // v1.5: el rayo tiene el color del jugador.
      if (beam.userData.col !== p.color) {
        beam.userData.col = p.color;
        beam.material.color.setHex(p.color).lerp(new THREE.Color(0xffffff), 0.25);
      }
    });
  }

  updateSelectionRings(selection) {
    const items = selection.filter((e) => e.alive);
    while (this.rings.length < items.length) {
      const r = new THREE.Mesh(this.ringGeo, this.ringMatOwn);
      r.renderOrder = 5;
      this.scene.add(r);
      this.rings.push(r);
    }
    this.rings.forEach((r, i) => {
      const e = items[i];
      if (!e) {
        r.visible = false;
        return;
      }
      r.visible = true;
      r.material = e.owner === 0 || (e.kind === 'point' && e.owner < 0) ? this.ringMatOwn : this.ringMatEnemy;
      const s = e.kind === 'bld' || e.kind === 'point' ? 1.85 : e.cls === 'inf' ? 0.75 : e.cls === 'ship' || e.cls ===
'sub' ? 1.4 : 1.1;
      r.scale.setScalar(s);
      const y = e.kind === 'bld' || e.kind === 'point' ? e.y + 0.05 : e.mover === 'air' ?
Math.max(this.game.cells[e.cell].h, 0) + 0.05 : Math.max(e.y, 0) + 0.04;
      r.position.set(e.x, y, e.z);
    });
    // Círculo de alcance de la tropa o defensa seleccionada.
    const one = items.length === 1 ? items[0] : null;
    const w = one && one.def && one.def.weapon;
    if (w && !one.inside) {
      const R = w.range * SQ3 + (one.kind === 'bld' ? 0.75 : 0.35);
      this.rangeRing.visible = true;
      this.rangeRing.scale.setScalar(R);
      this.rangeRing.userData.radius = R;
      const gy = one.kind === 'bld' ? one.y : Math.max(this.game.cells[one.cell].h, 0);
      this.rangeRing.position.set(one.x, gy + 0.06, one.z);
      this.rangeRing.material.color.set(one.owner === 0 ? 0xffffff : 0xff7070);
    } else {
      this.rangeRing.visible = false;
    }
    const b = items.length === 1 && items[0].kind === 'bld' && items[0].owner === 0 ? items[0] : null;
    if (b && b.rally >= 0) {
      const c = this.game.cells[b.rally];
      this.rallyFlag.visible = true;
```

```
      this.rallyFlag.position.set(c.x, Math.max(c.h, 0), c.z);
    } else this.rallyFlag.visible = false;
  }

  setPlacement(cells) {
    const m = new THREE.Matrix4();
    let n = 0;
    if (cells) {
      for (const ci of cells) {
        const c = this.game.cells[ci];
        m.makeTranslation(c.x, Math.max(c.h, 0) + 0.05, c.z);
        this.placeMesh.setMatrixAt(n++, m);
      }
    }
    this.placeMesh.count = n;
    this.placeMesh.instanceMatrix.needsUpdate = true;
  }

  setHover(ci) {
    if (ci < 0) {
      this.hoverMesh.visible = false;
      return;
    }
    const c = this.game.cells[ci];
    this.hoverMesh.visible = true;
    this.hoverMesh.position.set(c.x, Math.max(c.h, 0) + 0.07, c.z);
  }

  // ---------- cámara ----------
  updateCamera() {
    const c = this.cam;
    const R = this.game.map.R * 1.5;
    const len = Math.hypot(c.tx, c.tz);
    if (len > R) {
      c.tx *= R / len;
      c.tz *= R / len;
    }
    c.dist = clamp(c.dist, 9, 70);
    const zt = (c.dist - 9) / 61;
    c.pitch = 0.78 + zt * 0.42;
    const cp = Math.cos(c.pitch);
    this.camera.position.set(
      c.tx + Math.sin(c.yaw) * cp * c.dist,
      Math.sin(c.pitch) * c.dist,
      c.tz + Math.cos(c.yaw) * cp * c.dist,
    );
    this.camera.lookAt(c.tx, 0, c.tz);
    this.camera.updateMatrixWorld();
    this.sun.position.set(c.tx - 16, 34, c.tz + 10);
    this.sun.target.position.set(c.tx, 0, c.tz);
    const span = clamp(c.dist * 1.1, 22, 60);
    const sc = this.sun.shadow.camera;
    if (Math.abs(sc.right - span) > 2) {
      sc.left = -span; sc.right = span; sc.top = span; sc.bottom = -span;
      sc.updateProjectionMatrix();
    }
  }

  screenToWorld(sx, sy, planeY = 0) {
    this._ndc.set((sx / this.width) * 2 - 1, -(sy / this.height) * 2 + 1);
    this.raycaster.setFromCamera(this._ndc, this.camera);
    this._plane.constant = -planeY;
    const hit = this.raycaster.ray.intersectPlane(this._plane, this._p);
    return hit ? { x: hit.x, z: hit.z } : null;
  }

  pickCell(sx, sy) {
    const map = this.game.map;
    let p = this.screenToWorld(sx, sy, 0.3);
    if (!p) return null;
    let c = map.atWorld(p.x, p.z);
    if (!c) return null;
    p = this.screenToWorld(sx, sy, Math.max(c.h, 0));
    const c2 = p && map.atWorld(p.x, p.z);
    return c2 || c;
  }

  project(x, y, z) {
    this._p.set(x, y, z).project(this.camera);
    return { x: (this._p.x + 1) * 0.5 * this.width, y: (1 - this._p.y) * 0.5 * this.height, ok: this._p.z < 1 &&
this._p.z > -1 };
  }

  pickUnit(sx, sy, filter) {
    let best = null;
    let bd = Math.max(16, 520 / this.cam.dist);
    for (const u of this.game.units) {
      if (!u.alive || u.inside || (filter && !filter(u))) continue;
```

```
      if (!u.visible && this.game.players[u.owner].team !== this.game.players[0].team) continue;
      const s = this.project(u.x, u.y + 0.25, u.z);
      if (!s.ok) continue;
      const d = Math.hypot(s.x - sx, s.y - sy);
      if (d < bd) { bd = d; best = u; }
    }
    return best;
  }

  // v1.2: tocar el cuerpo o el techo de un edificio también lo elige (antes solo valía su casilla del suelo).
  // Solo cuenta la zona entre la base y el techo, para no robar los toques al suelo de delante.
  pickBuilding(sx, sy) {
    let best = null;
    let bd = Infinity;
    const half = Math.max(16, 380 / this.cam.dist);
    for (const b of this.game.buildings) {
      if (!b.alive) continue;
      const base = this.project(b.x, b.y, b.z);
      const top = this.project(b.x, b.y + (b.type === 'wall' ? 0.6 : 1.15), b.z);
      if (!base.ok || !top.ok) continue;
      if (sy > base.y + 4 || sy < top.y - 8 || Math.abs(sx - base.x) > half) continue;
      const d = Math.abs(sx - base.x) + Math.abs(sy - (base.y + top.y) / 2) * 0.5;
      if (d < bd) { bd = d; best = b; }
    }
    return best;
  }

  unitsInRect(x1, y1, x2, y2, filter) {
    const lx = Math.min(x1, x2);
    const hx = Math.max(x1, x2);
    const ly = Math.min(y1, y2);
    const hy = Math.max(y1, y2);
    return this.game.units.filter((u) => {
      if (!u.alive || u.inside || (filter && !filter(u))) return false;
      const s = this.project(u.x, u.y + 0.2, u.z);
      return s.ok && s.x >= lx && s.x <= hx && s.y >= ly && s.y <= hy;
    });
  }

  // ---------- capa 2D: barras de vida ----------
  drawOverlay(selection) {
    const ctx = this.octx;
    ctx.clearRect(0, 0, this.width, this.height);
    const g = this.game;
    const sel = new Set(selection);
    const myTeam = g.players[0].team;
    this.upBars = 0;
    const bar = (e, w, hTop) => {
      const s = this.project(e.x, e.y + hTop, e.z);
      if (!s.ok || s.x < -40 || s.y < -40 || s.x > this.width + 40 || s.y > this.height + 40) return;
      const x = Math.round(s.x - w / 2);
      const y = Math.round(s.y);
      ctx.fillStyle = 'rgba(12,16,22,0.75)';
      ctx.fillRect(x - 1, y - 1, w + 2, 6);
      if (e.kind === 'bld' && !e.built) {
        ctx.fillStyle = '#f5c542';
        ctx.fillRect(x, y, w * e.progress, 4);
        return;
      }
      // v1.5: barra de la mejora (celeste) debajo de la de vida.
      if (e.kind === 'bld' && e.upgrading) {
        this.upBars++;
        ctx.fillStyle = 'rgba(12,16,22,0.75)';
        ctx.fillRect(x - 1, y + 5, w + 2, 4);
        ctx.fillStyle = '#7dd3ff';
        ctx.fillRect(x, y + 6, w * clamp(e.upgrading.progress, 0, 1), 2);
      }
      const k = clamp(e.hp / e.maxHp, 0, 1);
      ctx.fillStyle = k > 0.55 ? '#5ee07a' : k > 0.28 ? '#f2c94c' : '#ef5350';
      ctx.fillRect(x, y, w * k, 4);
      ctx.fillStyle = hexColor(PLAYER_COLORS[e.owner]);
      ctx.fillRect(x - 1, y - 1, 3, 6);
      // Escudo (v0.9): barra encima en azul (bien), amarillo (a medias) o rojo (débil).
      if (e.shieldMax > 0) {
        const ks = clamp(e.shield / e.shieldMax, 0, 1);
        ctx.fillStyle = 'rgba(12,16,22,0.75)';
        ctx.fillRect(x - 1, y - 5, w + 2, 4);
        ctx.fillStyle = SHIELD_CSS(ks);
        ctx.fillRect(x, y - 4, w * ks, 2);
      }
      // v1.4: estrellas de las tropas veteranas, encima de la barra.
      if (e.vet) {
        ctx.font = '700 10px system-ui, sans-serif';
        ctx.textAlign = 'center';
        ctx.textBaseline = 'bottom';
        const txt = '★'.repeat(e.vet);
        const ty = y - (e.shieldMax > 0 ? 6 : 2);
        ctx.lineWidth = 3;
```

```
        ctx.strokeStyle = 'rgba(12,16,22,0.85)';
        ctx.strokeText(txt, s.x, ty);
        ctx.fillStyle = '#ffd36a';
        ctx.fillText(txt, s.x, ty);
        ctx.textBaseline = 'alphabetic';
      }
    };
    // Progreso de captura sobre el Faro y las Banderas.
    for (const pt of g.points) {
      const busy = pt.contested || (pt.prog > 0 && pt.prog < 1) || (pt.owner >= 0 && pt.prog < 1);
      if (!busy && !sel.has(pt)) continue;
      const s = this.project(pt.x, pt.y + (pt.type === 'faro' ? 3.3 : 1.7), pt.z);
      if (!s.ok) continue;
      const R = pt.type === 'faro' ? 15 : 12;
      ctx.lineWidth = 4;
      ctx.strokeStyle = 'rgba(12,16,22,0.7)';
      ctx.beginPath();
      ctx.arc(s.x, s.y, R, 0, Math.PI * 2);
      ctx.stroke();
      const capColor = pt.owner >= 0 ? PLAYER_COLORS[pt.owner]
        : pt.capTeam >= 0 ? PLAYER_COLORS[g.players.findIndex((p) => p.team === pt.capTeam)] : NEUTRAL;
      ctx.strokeStyle = hexColor(capColor);
      ctx.beginPath();
      ctx.arc(s.x, s.y, R, -Math.PI / 2, -Math.PI / 2 + Math.PI * 2 * clamp(pt.prog, 0, 1));
      ctx.stroke();
      if (pt.contested) {
        ctx.fillStyle = '#ffd36a';
        ctx.font = '600 11px system-ui, sans-serif';
        ctx.textAlign = 'center';
        ctx.fillText('En disputa', s.x, s.y + R + 14);
      }
    }
    // Constructor en camino (v0.9): línea punteada desde él hasta su obra.
    ctx.setLineDash([5, 5]);
    ctx.lineWidth = 2;
    ctx.strokeStyle = 'rgba(245,197,66,0.85)';
    for (const u of g.units) {
      const b = u.buildTarget;
      if (!u.alive || u.owner !== 0 || u.state !== 'build' || !b || !b.alive) continue;
      const near = Math.hypot(u.x - b.x, u.z - b.z) < 2.2;
      if (near) continue;
      const a = this.project(u.x, u.y + 0.3, u.z);
      const c = this.project(b.x, b.y + 0.3, b.z);
      if (!a.ok || !c.ok) continue;
      ctx.beginPath();
      ctx.moveTo(a.x, a.y);
      ctx.lineTo(c.x, c.y);
      ctx.stroke();
    }
    ctx.setLineDash([]);
    // Radiación (v1.0): cartel con la cuenta regresiva sobre cada zona.
    for (const z of g.radZones || []) {
      const s = this.project(z.x, z.y + 0.9, z.z);
      if (!s.ok || s.x < -60 || s.y < -30 || s.x > this.width + 60 || s.y > this.height + 30) continue;
      const left = Math.max(0, Math.ceil(z.until - g.time));
      const text = `☢ Radiación · ${Math.floor(left / 60)}:${String(left % 60).padStart(2, '0')}`;
      ctx.font = '700 12px system-ui, sans-serif';
      const w = ctx.measureText(text).width + 16;
      ctx.fillStyle = 'rgba(12,22,10,0.85)';
      ctx.fillRect(s.x - w / 2, s.y - 20, w, 20);
      ctx.fillStyle = '#7cff4a';
      ctx.fillRect(s.x - w / 2, s.y - 3, w * clamp(left / NUKE.rad.time, 0, 1), 3);
      ctx.textAlign = 'center';
      ctx.fillStyle = '#c9ffb0';
      ctx.fillText(text, s.x, s.y - 7);
    }
    // AIR DEFENSE (v0.9): el radar marca la aeronave que sigue (amarillo) y la que ya fijó (rojo).
    // Si un radar enemigo fija una aeronave tuya, aparece un aro rojo de alerta sobre ella.
    for (const bl of g.buildings) {
      const t = bl.lockOn;
      if (!bl.alive || !t || !t.alive || t.cls !== 'air' || !bl.def.weapon || !bl.def.weapon.lock) continue;
      const mine = bl.owner === 0;
      if (!mine && t.owner !== 0) continue;
      if (!g.inRange(bl, g.weaponOf(bl), t)) continue;
      const s = this.project(t.x, t.y + 0.2, t.z);
      if (!s.ok || s.x < -30 || s.y < -30 || s.x > this.width + 30 || s.y > this.height + 30) continue;
      const locked = bl.lockT <= 0;
      ctx.lineWidth = 2;
      if (mine) {
        const r = locked ? 13 : 13 + Math.max(0, bl.lockT) * 30;
        ctx.strokeStyle = locked ? '#ef5350' : '#f2c94c';
        for (const [sx, sy] of [[-1, -1], [1, -1], [1, 1], [-1, 1]]) {
          ctx.beginPath();
          ctx.moveTo(s.x + sx * r, s.y + sy * (r - 6));
          ctx.lineTo(s.x + sx * r, s.y + sy * r);
          ctx.lineTo(s.x + sx * (r - 6), s.y + sy * r);
          ctx.stroke();
        }
```

```
      } else {
        const pulse = 0.5 + 0.5 * Math.sin(g.time * 12);
        ctx.strokeStyle = `rgba(239,83,80,${0.45 + pulse * 0.5})`;
        ctx.beginPath();
        ctx.arc(s.x, s.y, 15 + pulse * 3, 0, Math.PI * 2);
        ctx.stroke();
      }
      ctx.lineWidth = 1;
    }
    // Obras: «En construcción» o «Mejorando», con progreso y aviso si no hay constructor.
    for (const bl of g.buildings) {
      if (!bl.alive || g.players[bl.owner].team !== myTeam) continue;
      const working = !bl.built || bl.upgrading;
      if (!working && bl.level < 2) continue;
      const s = this.project(bl.x, bl.y + (bl.type === 'core' ? 2.4 : 1.45), bl.z);
      if (!s.ok || s.x < -60 || s.y < -30 || s.x > this.width + 60 || s.y > this.height + 30) continue;
      if (working) {
        const k = bl.upgrading ? bl.upgrading.progress : bl.progress;
        const idle = g.time - bl.lastWork > 1.5 && bl.type !== 'outpost';
        // v0.9: «Esperando constructor» mientras nadie trabaja; «Construyendo» cuando llega.
        const coming = idle && g.units.some((u) => u.alive && u.buildTarget === bl && u.state === 'build');
        const verb = bl.upgrading ? `Mejorando a nivel ${bl.upgrading.level}` : idle ? 'Esperando constructor' :
'Construyendo';
        const text = `${verb} · ${Math.floor(k * 100)} %`;
        const note = coming ? 'Constructor en camino' : 'Sin constructor';
        ctx.font = '600 11px system-ui, sans-serif';
        const w = Math.max(ctx.measureText(text).width, idle ? ctx.measureText(note).width : 0) + 14;
        const h = idle ? 32 : 20;
        ctx.fillStyle = 'rgba(12,16,22,0.85)';
        ctx.fillRect(s.x - w / 2, s.y - h, w, h);
        ctx.fillStyle = '#f2ab38';
        ctx.fillRect(s.x - w / 2, s.y - 3, w * clamp(k, 0, 1), 3);
        ctx.textAlign = 'center';
        ctx.fillStyle = '#edf3f7';
        ctx.fillText(text, s.x, s.y - h + 13);
        if (idle) {
          ctx.fillStyle = coming ? '#f5c542' : '#ff9a9b';
          ctx.fillText(note, s.x, s.y - h + 26);
        }
      } else if (bl.owner === 0) {
        // Nivel 2 o 3: marcas doradas.
        for (let i = 0; i < bl.level - 1; i++) {
          const x = s.x - (bl.level - 2) * 5 + i * 10;
          ctx.fillStyle = '#f2ab38';
          ctx.beginPath();
          ctx.moveTo(x, s.y - 12);
          ctx.lineTo(x + 4, s.y - 8);
          ctx.lineTo(x, s.y - 4);
          ctx.lineTo(x - 4, s.y - 8);
          ctx.closePath();
          ctx.fill();
        }
      }
    }
    // Icono de combate donde hay peleas.
    for (const f of g.fights) {
      const age = g.time - f.t;
      if (age > 1.6) continue;
      const s = this.project(f.x, (f.y || 0) + 1.1, f.z);
      if (!s.ok) continue;
      const c = g.cells[f.cell];
      if (g.fog && c && c.owner !== 0 && !g.units.some((u) => u.owner === 0 && Math.hypot(u.x - f.x, u.z - f.z) < 7))
continue;
      const r = 8 + Math.sin(age * 12) * 1.5;
      ctx.globalAlpha = 1 - age / 1.6;
      ctx.fillStyle = '#ff8a30';
      ctx.beginPath();
      for (let i = 0; i < 16; i++) {
        const a = (i / 16) * Math.PI * 2;
        const rr = i % 2 ? r * 0.45 : r;
        ctx.lineTo(s.x + Math.cos(a) * rr, s.y + Math.sin(a) * rr);
      }
      ctx.closePath();
      ctx.fill();
      ctx.fillStyle = '#ffe08a';
      ctx.beginPath();
      ctx.arc(s.x, s.y, r * 0.3, 0, Math.PI * 2);
      ctx.fill();
      ctx.globalAlpha = 1;
    }
    this.fightIcons = g.fights.filter((f) => g.time - f.t <= 1.6).length;
    // Producción a la vista: progreso y cola sobre tus edificios que entrenan.
    for (const bl of g.buildings) {
      if (!bl.alive || bl.owner !== 0 || !bl.queue.length) continue;
      const s = this.project(bl.x, bl.y + 1.55, bl.z);
      if (!s.ok || s.x < -30 || s.y < -30 || s.x > this.width + 30 || s.y > this.height + 30) continue;
      const q = bl.queue[0];
      const k = clamp(q.t / UNITS[q.type].build, 0, 1);
```

```
      ctx.fillStyle = 'rgba(12,16,22,0.82)';
      ctx.beginPath();
      ctx.arc(s.x, s.y, 11, 0, Math.PI * 2);
      ctx.fill();
      ctx.lineWidth = 3;
      ctx.strokeStyle = '#f2ab38';
      ctx.beginPath();
      ctx.arc(s.x, s.y, 11, -Math.PI / 2, -Math.PI / 2 + Math.PI * 2 * k);
      ctx.stroke();
      ctx.fillStyle = '#edf3f7';
      ctx.font = '600 11px system-ui, sans-serif';
      ctx.textAlign = 'center';
      ctx.textBaseline = 'middle';
      ctx.fillText(String(bl.queue.length), s.x, s.y + 0.5);
      ctx.textBaseline = 'alphabetic';
    }
    this.prodBadges = g.buildings.filter((bl) => bl.alive && bl.owner === 0 && bl.queue.length).length;
    // v1.5: aviso de «parado» sobre tus edificios que entrenan y llevan un rato sin nada en cola.
    let idle = 0;
    for (const bl of g.buildings) {
      const idleMap = this.idleSince || (this.idleSince = new Map());
      if (!bl.alive || bl.owner !== 0 || bl.type === 'core' || !bl.def.produces || !bl.built || bl.queue.length ||
bl.loop) { idleMap.delete(bl.id); continue; }
      if (!idleMap.has(bl.id)) idleMap.set(bl.id, g.time);
      if (g.time - idleMap.get(bl.id) < 6) continue;
      const s = this.project(bl.x, bl.y + 1.55, bl.z);
      if (!s.ok || s.x < -30 || s.y < -30 || s.x > this.width + 30 || s.y > this.height + 30) continue;
      idle++;
      ctx.fillStyle = 'rgba(12,16,22,0.78)';
      ctx.beginPath();
      ctx.arc(s.x, s.y, 10, 0, Math.PI * 2);
      ctx.fill();
      ctx.strokeStyle = 'rgba(255,255,255,0.55)';
      ctx.lineWidth = 1.5;
      ctx.stroke();
      ctx.fillStyle = '#cfd8e0';
      ctx.font = '700 9px system-ui, sans-serif';
      ctx.textAlign = 'center';
      ctx.textBaseline = 'middle';
      ctx.fillText('z', s.x - 3, s.y + 2);
      ctx.font = '700 11px system-ui, sans-serif';
      ctx.fillText('z', s.x + 2.5, s.y - 2);
      ctx.textBaseline = 'alphabetic';
    }
    this.idleBadges = idle;
    const recent = g.time - 4;
    for (const b of g.buildings) {
      if (!b.alive) continue;
      if (b.hp < b.maxHp || !b.built || b.upgrading || sel.has(b) || b.lastHit > recent || b.shield < b.shieldMax)
bar(b, b.type === 'core' ? 56 : 40, b.type === 'core' ? 1.75 + 0.3 * b.level : 1.15);
    }
    for (const u of g.units) {
      if (!u.alive || u.inside) continue;
      if (!u.visible && g.players[u.owner].team !== myTeam) continue;
      if (u.hp < u.maxHp || sel.has(u) || u.vet || (u.shieldMax > 0 && u.shield < u.shieldMax)) bar(u, u.cls === 'inf'
? 22 : 30, u.mover === 'air' ? 0.7 : u.cls === 'inf' ? 0.8 : 0.75);
      // Helicópteros y aviones (v0.9): icono mientras disparan o lanzan misiles.
      if (u.mover === 'air' && g.time - (u.lastShot || -99) < 0.6) {
        const s = this.project(u.x, u.y + 1.15, u.z);
        if (!s.ok) continue;
        const missile = u.lastShotKind === 'missile' || u.lastShotKind === 'rocket' || u.lastShotKind === 'bomb';
        ctx.fillStyle = 'rgba(12,16,22,0.8)';
        ctx.beginPath();
        ctx.arc(s.x, s.y, 9, 0, Math.PI * 2);
        ctx.fill();
        ctx.strokeStyle = missile ? '#ffb63a' : '#ff6b6b';
        ctx.lineWidth = 2;
        ctx.beginPath();
        if (missile) {
          ctx.moveTo(s.x - 5, s.y + 4); ctx.lineTo(s.x + 5, s.y - 4); ctx.moveTo(s.x + 1, s.y - 5); ctx.lineTo(s.x + 5,
s.y - 4); ctx.lineTo(s.x + 4, s.y);
        } else {
          ctx.arc(s.x, s.y, 5, 0, Math.PI * 2);
          ctx.moveTo(s.x - 8, s.y); ctx.lineTo(s.x + 8, s.y); ctx.moveTo(s.x, s.y - 8); ctx.lineTo(s.x, s.y + 8);
        }
        ctx.stroke();
        this.airShotIcons = (this.airShotIcons || 0) + 1;
      }
    }
    // v1.5: con la cámara lejos, un icono por tropa (persona, vehículo, barco o avión) con su color.
    this.unitIcons = 0;
    if (this.cam.dist >= UNIT_ICON_DIST) this.drawUnitIcons(ctx);
    if (this.nukeFlashT > 0) {
      ctx.fillStyle = `rgba(255,248,225,${Math.min(0.75, this.nukeFlashT * 0.6).toFixed(3)})`;
      ctx.fillRect(0, 0, this.width, this.height);
    }
  }

```

```
  // ---------- minimapa ----------
  minimapTransform(size) {
    const R = this.game.map.R;
    const ext = SQ3 * R + 1.2;
    const s = size / (ext * 2);
    return { s, ox: size / 2, oz: size / 2 };
  }

  drawMinimap(canvas) {
    const g = this.game;
    const size = canvas.width;
    const ctx = canvas.getContext('2d');
    const T = this.minimapTransform(size);
    if (!this.miniBase || this.miniBase.width !== size || this.miniDirty) {
      this.miniDirty = false;
      const base = this.miniBase && this.miniBase.width === size ? this.miniBase : document.createElement('canvas');
      base.width = size;
      base.height = size;
      const b = base.getContext('2d');
      b.fillStyle = '#16405a';
      b.fillRect(0, 0, size, size);
      const r = T.s * 1.02;
      const tmp = new THREE.Color();
      for (const c of g.cells) {
        tmp.set(terrainColor(this.game.map.kind, c.t));
        if (c.t === 'deep') continue;
        if (c.owner >= 0) tmp.lerp(new THREE.Color(PLAYER_COLORS[c.owner]), TERRAIN[c.t].water ? 0.35 : 0.45);
        b.fillStyle = '#' + tmp.getHexString();
        const cx = T.ox + c.x * T.s;
        const cy = T.oz + c.z * T.s;
        b.beginPath();
        for (let k = 0; k < 6; k++) {
          const a = (k * Math.PI) / 3;
          const px = cx + Math.cos(a) * r;
          const py = cy + Math.sin(a) * r;
          if (k === 0) b.moveTo(px, py);
          else b.lineTo(px, py);
        }
        b.fill();
      }
      this.miniBase = base;
    }
    ctx.drawImage(this.miniBase, 0, 0);
    const myTeam = g.players[0].team;
    for (const pt of g.points) {
      const x = T.ox + pt.x * T.s;
      const y = T.oz + pt.z * T.s;
      ctx.fillStyle = pt.owner >= 0 ? hexColor(PLAYER_COLORS[pt.owner]) : '#e6e8ea';
      ctx.strokeStyle = pt.type === 'faro' ? '#ffb63a' : '#1b2630';
      ctx.lineWidth = pt.type === 'faro' ? 2 : 1;
      ctx.beginPath();
      ctx.moveTo(x, y - 5);
      ctx.lineTo(x + 4, y);
      ctx.lineTo(x, y + 5);
      ctx.lineTo(x - 4, y);
      ctx.closePath();
      ctx.fill();
      ctx.stroke();
    }
    // v1.4: con «Colores para daltonismo», cada jugador tiene además su forma (Núcleo y tropas).
    const cvd = !!(window.FARO && window.FARO.cvd);
    for (const bl of g.buildings) {
      ctx.fillStyle = hexColor(PLAYER_COLORS[bl.owner]);
      const x = T.ox + bl.x * T.s;
      const y = T.oz + bl.z * T.s;
      if (cvd && bl.type === 'core') {
        ctx.strokeStyle = '#ffffff';
        ctx.lineWidth = 1.5;
        minimapShape(ctx, PLAYER_SHAPES[bl.owner], x, y, 6.5);
        continue;
      }
      const s = bl.type === 'core' ? 7 : 4;
      ctx.fillRect(x - s / 2, y - s / 2, s, s);
      if (bl.type === 'core') {
        ctx.strokeStyle = '#ffffff';
        ctx.lineWidth = 1;
        ctx.strokeRect(x - s / 2, y - s / 2, s, s);
      }
    }
    for (const u of g.units) {
      if (u.inside || (!u.visible && g.players[u.owner].team !== myTeam)) continue;
      ctx.fillStyle = hexColor(PLAYER_COLORS[u.owner]);
      if (cvd) {
        ctx.strokeStyle = 'rgba(10,14,20,0.8)';
        ctx.lineWidth = 0.8;
        minimapShape(ctx, PLAYER_SHAPES[u.owner], T.ox + u.x * T.s, T.oz + u.z * T.s, 2.6);
        continue;
      }
```

```
      ctx.beginPath();
      ctx.arc(T.ox + u.x * T.s, T.oz + u.z * T.s, 1.8, 0, Math.PI * 2);
      ctx.fill();
    }
    // Pulsos de alerta
    const now = performance.now();
    this.pings = this.pings.filter((pg) => now - pg.t < 5000);
    for (const pg of this.pings) {
      const k = ((now - pg.t) % 1000) / 1000;
      ctx.strokeStyle = pg.kind === 'bad' ? `rgba(255,90,90,${1 - k})` : `rgba(242,171,56,${1 - k})`;
      ctx.lineWidth = 2.5;
      ctx.beginPath();
      ctx.arc(T.ox + pg.x * T.s, T.oz + pg.z * T.s, 4 + k * 14, 0, Math.PI * 2);
      ctx.stroke();
    }
    // Vista de la cámara
    const corners = [[0, 0], [this.width, 0], [this.width, this.height], [0, this.height]];
    const pts = corners.map(([x, y]) => this.screenToWorld(x, y, 0));
    if (pts.every(Boolean)) {
      ctx.strokeStyle = 'rgba(255,255,255,0.9)';
      ctx.lineWidth = 1.2;
      ctx.beginPath();
      pts.forEach((p, i) => {
        const x = T.ox + p.x * T.s;
        const y = T.oz + p.z * T.s;
        if (i === 0) ctx.moveTo(x, y);
        else ctx.lineTo(x, y);
      });
      ctx.closePath();
      ctx.stroke();
    }
  }

  worldToMinimap(canvas, x, z) {
    const T = this.minimapTransform(canvas.width);
    const k = canvas.width / canvas.clientWidth;
    return { x: (x * T.s + T.ox) / k, y: (z * T.s + T.oz) / k };
  }

  minimapToWorld(canvas, mx, my) {
    const T = this.minimapTransform(canvas.width);
    const k = canvas.width / canvas.clientWidth;
    return { x: (mx * k - T.ox) / T.s, z: (my * k - T.oz) / T.s };
  }

  dispose() {
    if (this.fxs) this.fxs.dispose();
    this.scene.traverse((o) => {
      if (o.isInstancedMesh || o === this.borderMesh || o === this.tracerLines) {
        if (o.geometry) o.geometry.dispose();
        if (o.material && o.material !== MODEL_MAT) o.material.dispose();
      }
    });
  }

  // ---------- cuadro ----------
  frame(dt, time, selection) {
    const g = this.game;
    if (this.nukeFlashT > 0) this.nukeFlashT -= dt;
    if (g.territoryVersion !== this.lastTerritory) {
      this.lastTerritory = g.territoryVersion;
      this.tintTerritory();
      this.rebuildBorders();
      this.miniDirty = true;
    }
    const bkey = g.buildings.length * 1e6 + (g.buildings.length ? g.buildings[g.buildings.length - 1].id : 0);
    if (bkey !== this.lastBuildCount) {
      this.lastBuildCount = bkey;
      this.refreshDecor();
    }
    if (this.placeMesh.count) this.placeMesh.material.opacity = 0.32 + Math.sin(time * 4) * 0.14;
    this.consumeFx();
    this.syncEntities(dt, time);
    this.updateMarks(time);
    this.updateFx(dt);
    this.updateLasers(dt);
    this.updatePoints(time);
    this.updateShields(time);
    this.updateRadZones(time);
    this.updateSelectionRings(selection);
    this.updateCamera();
    this.renderer.render(this.scene, this.camera);
    // Con muchas tropas, las barras de vida y los iconos se dibujan un cuadro sí y otro no.
    this.ovFrame = (this.ovFrame || 0) + 1;
    // v0.9: en Más fluido las barras y avisos se dibujan cada dos cuadros con más de 40 tropas.
    this.overlayHalf = g.units.length > (this.quality === 'high' ? 120 : 40);
    if (!this.overlayHalf || this.ovFrame % 2 === 0) this.drawOverlay(selection);
  }
```

```
}

// v1.4: forma de cada jugador en el minimapa (modo para daltonismo).
function minimapShape(ctx, shape, x, y, r) {
  ctx.beginPath();
  if (shape === 'square') ctx.rect(x - r * 0.85, y - r * 0.85, r * 1.7, r * 1.7);
  else if (shape === 'triangle') { ctx.moveTo(x, y - r); ctx.lineTo(x + r, y + r * 0.8); ctx.lineTo(x - r, y + r * 0.8); ctx.closePath(); }
  else if (shape === 'diamond') { ctx.moveTo(x, y - r); ctx.lineTo(x + r, y); ctx.lineTo(x, y + r); ctx.lineTo(x - r, y); ctx.closePath(); }
  else if (shape === 'star') {
    for (let i = 0; i < 10; i++) {
      const a = -Math.PI / 2 + (i * Math.PI) / 5;
      const rr = i % 2 ? r * 0.45 : r;
      if (i) ctx.lineTo(x + Math.cos(a) * rr, y + Math.sin(a) * rr); else ctx.moveTo(x + Math.cos(a) * rr, y + Math.sin(a) * rr);
    }
    ctx.closePath();
  } else if (shape === 'cross') {
    const t = r * 0.38;
    ctx.moveTo(x - t, y - r); ctx.lineTo(x + t, y - r); ctx.lineTo(x + t, y - t); ctx.lineTo(x + r, y - t); ctx.lineTo(x + r, y + t);
    ctx.lineTo(x + t, y + t); ctx.lineTo(x + t, y + r); ctx.lineTo(x - t, y + r); ctx.lineTo(x - t, y + t); ctx.lineTo(x - r, y + t);
    ctx.lineTo(x - r, y - t); ctx.lineTo(x - t, y - t); ctx.closePath();
  } else ctx.arc(x, y, r, 0, Math.PI * 2);
  ctx.fill();
  ctx.stroke();
}
```

faro/src/js/061-fx.js · 232 líneas

```
'use strict';
// Efectos con partículas (v0.6): fuego, humo, restos, chispas, fogonazos, estelas y salpicaduras.
// Todas las partículas de un tipo de mezcla se dibujan en una sola pasada (THREE.Points), con un
// tope fijo: cuando se llena, la partícula más vieja se reutiliza. Así los efectos no ponen lento
// al iPad aunque haya muchas peleas.

const FX_VERT = `
attribute float psize;
attribute float palpha;
attribute vec3 pcolor;
uniform float scale;
varying vec3 vColor;
varying float vAlpha;
void main() {
  vColor = pcolor;
  vAlpha = palpha;
  vec4 mv = modelViewMatrix * vec4(position, 1.0);
  gl_PointSize = psize * scale / max(0.1, -mv.z);
  gl_Position = projectionMatrix * mv;
}`;

const FX_FRAG = `
varying vec3 vColor;
varying float vAlpha;
void main() {
  vec2 c = gl_PointCoord - 0.5;
  float d = length(c);
  if (d > 0.5) discard;
  gl_FragColor = vec4(vColor, vAlpha * smoothstep(0.5, 0.12, d));
}`;

class Particles {
  constructor(scene, max, additive) {
    this.max = max;
    this.head = 0;
    this.count = 0;
    this.pos = new Float32Array(max * 3);
    this.col = new Float32Array(max * 3);
    this.alpha = new Float32Array(max);
    this.size = new Float32Array(max);
    const g = new THREE.BufferGeometry();
    g.setAttribute('position', new THREE.BufferAttribute(this.pos, 3));
    g.setAttribute('pcolor', new THREE.BufferAttribute(this.col, 3));
    g.setAttribute('palpha', new THREE.BufferAttribute(this.alpha, 1));
    g.setAttribute('psize', new THREE.BufferAttribute(this.size, 1));
    g.setDrawRange(0, 0);
    this.mat = new THREE.ShaderMaterial({
      uniforms: { scale: { value: 400 } },
      vertexShader: FX_VERT,
      fragmentShader: FX_FRAG,
      transparent: true,
      depthWrite: false,
      blending: additive ? THREE.AdditiveBlending : THREE.NormalBlending,
    });
    this.points = new THREE.Points(g, this.mat);
    this.points.frustumCulled = false;
    this.points.renderOrder = additive ? 9 : 8;
    scene.add(this.points);
    this.p = [];
    for (let i = 0; i < max; i++) {
      this.p.push({ t: 1, life: 0, x: 0, y: 0, z: 0, vx: 0, vy: 0, vz: 0, s0: 0, s1: 0, r: 1, g: 1, b: 1, a: 1, grav: 0,
drag: 0, fade: 1 });
    }
  }

  spawn(o) {
    const p = this.p[this.head];
    this.head = (this.head + 1) % this.max;
    p.t = 0;
    p.life = o.life;
    p.x = o.x; p.y = o.y; p.z = o.z;
    p.vx = o.vx || 0; p.vy = o.vy || 0; p.vz = o.vz || 0;
    p.s0 = o.s0; p.s1 = o.s1 === undefined ? o.s0 : o.s1;
    p.r = o.r; p.g = o.g; p.b = o.b;
    p.a = o.a === undefined ? 1 : o.a;
    p.grav = o.grav || 0;
    p.drag = o.drag || 0;
    p.fade = o.fade || 1;
  }

  update(dt) {
    let n = 0;
    for (const p of this.p) {
      if (p.t >= p.life) continue;
```

```
      p.t += dt;
      if (p.t >= p.life) continue;
      p.vy -= p.grav * dt;
      if (p.drag) {
        const k = Math.max(0, 1 - p.drag * dt);
        p.vx *= k; p.vy *= k; p.vz *= k;
      }
      p.x += p.vx * dt; p.y += p.vy * dt; p.z += p.vz * dt;
      if (p.grav && p.y < 0) { p.y = 0; p.vy *= -0.3; p.vx *= 0.5; p.vz *= 0.5; }
      const k = p.t / p.life;
      const o = n * 3;
      this.pos[o] = p.x; this.pos[o + 1] = p.y; this.pos[o + 2] = p.z;
      this.col[o] = p.r; this.col[o + 1] = p.g; this.col[o + 2] = p.b;
      this.size[n] = p.s0 + (p.s1 - p.s0) * k;
      this.alpha[n] = p.a * Math.pow(1 - k, p.fade);
      n++;
    }
    this.count = n;
    const g = this.points.geometry;
    g.setDrawRange(0, n);
    if (n) {
      g.attributes.position.needsUpdate = true;
      g.attributes.pcolor.needsUpdate = true;
      g.attributes.palpha.needsUpdate = true;
      g.attributes.psize.needsUpdate = true;
    }
  }

  setScale(px) {
    this.mat.uniforms.scale.value = px;
  }

  dispose() {
    this.points.geometry.dispose();
    this.mat.dispose();
  }
}

const rnd = (a, b) => a + Math.random() * (b - a);

// Efectos compuestos. glow = mezcla aditiva (fuego, fogonazo, chispas); soft = normal (humo, restos, agua).
class FxSystem {
  constructor(scene) {
    this.glow = new Particles(scene, 520, true);
    this.soft = new Particles(scene, 720, false);
    this.scale = 1; // 0,5 en «Más fluido»: la mitad de partículas.
    this.spawned = 0;
  }

  n(k) {
    // Cantidad según la calidad: en «Más fluido» se reparte la mitad (redondeando al azar).
    const v = k * this.scale;
    return Math.floor(v) + (Math.random() < v - Math.floor(v) ? 1 : 0);
  }

  add(sys, o) {
    sys.spawn(o);
    this.spawned++;
  }

  explosion(x, y, z, s, bld) {
    const big = bld ? 1.3 : 1;
    this.add(this.glow, { x, y: y + 0.1, z, life: 0.16, s0: s * 1.4 * big, s1: s * 1.9 * big, r: 1, g: 0.95, b: 0.75, a:
1 });
    for (let i = this.n(bld ? 10 : 6); i > 0; i--) {
      const a = Math.random() * Math.PI * 2;
      const sp = rnd(0.4, 1.4) * s;
      this.add(this.glow, { x, y: y + rnd(0, 0.3) * s, z, vx: Math.cos(a) * sp, vy: rnd(0.4, 1.2) * s, vz: Math.sin(a) *
sp, drag: 3,
        life: rnd(0.35, 0.6), s0: s * rnd(0.5, 0.8) * big, s1: s * rnd(0.9, 1.3) * big, r: 1, g: rnd(0.45, 0.7), b:
0.15, a: 0.95, fade: 1.4 });
    }
    for (let i = this.n(bld ? 9 : 4); i > 0; i--) {
      const a = Math.random() * Math.PI * 2;
      this.add(this.soft, { x: x + Math.cos(a) * 0.2 * s, y: y + 0.2, z: z + Math.sin(a) * 0.2 * s, vx: Math.cos(a) *
0.25, vy: rnd(0.5, 1.1), vz: Math.sin(a) * 0.25,
        drag: 0.8, life: rnd(1.4, 2.4), s0: s * 0.55 * big, s1: s * 1.35 * big, r: 0.3, g: 0.31, b: 0.33, a: 0.5, fade:
1.2 });
    }
    for (let i = this.n(bld ? 12 : 6); i > 0; i--) {
      const a = Math.random() * Math.PI * 2;
      const sp = rnd(1.2, 2.8) * s;
      this.add(this.soft, { x, y: y + 0.2, z, vx: Math.cos(a) * sp, vy: rnd(1.5, 3.2) * s, vz: Math.sin(a) * sp, grav:
6,
        life: rnd(0.7, 1.2), s0: rnd(0.07, 0.13) * big, r: 0.16, g: 0.15, b: 0.14, a: 1, fade: 0.4 });
    }
    this.sparks(x, y + 0.2, z, s, bld ? 8 : 5);
  }
```

```
166
167   sparks(x, y, z, s, k = 4) {
168     for (let i = this.n(k); i > 0; i--) {
169       const a = Math.random() * Math.PI * 2;
170       const sp = rnd(1.5, 3.5) * Math.max(0.5, s);
171       this.add(this.glow, { x, y, z, vx: Math.cos(a) * sp, vy: rnd(0.8, 2.6), vz: Math.sin(a) * sp, grav: 7,
172         life: rnd(0.2, 0.42), s0: 0.08, s1: 0.03, r: 1, g: 0.85, b: 0.4, a: 1 });
173     }
174   }
175
176   flash(x, y, z, s = 0.28) {
177     this.add(this.glow, { x, y, z, life: 0.07, s0: s, s1: s * 1.4, r: 1, g: 0.9, b: 0.55, a: 1 });
178   }
179
180   impact(x, y, z, s) {
181     this.add(this.glow, { x, y, z, life: 0.22, s0: s * 1.2, s1: s * 1.8, r: 1, g: 0.75, b: 0.3, a: 0.9 });
182     this.sparks(x, y, z, s, 3);
183     if (this.n(1)) this.add(this.soft, { x, y, z, vy: 0.5, life: 0.9, s0: s * 0.7, s1: s * 1.6, r: 0.35, g: 0.35, b:
0.37, a: 0.5 });
184   }
185
186   splash(x, z, s) {
187     for (let i = this.n(9); i > 0; i--) {
188       const a = Math.random() * Math.PI * 2;
189       const sp = rnd(0.3, 1.1) * s;
190       this.add(this.soft, { x, y: 0.02, z, vx: Math.cos(a) * sp, vy: rnd(2, 3.6) * s, vz: Math.sin(a) * sp, grav: 8,
191         life: rnd(0.5, 0.8), s0: rnd(0.1, 0.18), s1: 0.05, r: 0.86, g: 0.95, b: 1, a: 0.9, fade: 0.6 });
192     }
193     this.add(this.soft, { x, y: 0.03, z, life: 0.6, s0: s * 0.6, s1: s * 1.8, r: 0.9, g: 0.97, b: 1, a: 0.55 });
194   }
195
196   trail(x, y, z) {
197     this.add(this.soft, { x, y, z, vy: 0.15, life: 0.55, s0: 0.1, s1: 0.32, r: 0.78, g: 0.78, b: 0.8, a: 0.55, drag: 1
});
198   }
199
200   smoke(x, y, z, s) {
201     this.add(this.soft, { x: x + rnd(-0.12, 0.12), y, z: z + rnd(-0.12, 0.12), vx: rnd(-0.1, 0.1), vy: rnd(0.45, 0.8),
vz: rnd(-0.1, 0.1),
202       drag: 0.4, life: rnd(1.8, 2.8), s0: s * 0.5, s1: s * 1.9, r: 0.24, g: 0.25, b: 0.27, a: 0.5, fade: 1.3 });
203     if (this.n(0.5)) this.add(this.glow, { x, y: y - 0.05, z, vy: 0.3, life: 0.4, s0: s * 0.45, s1: s * 0.2, r: 1, g:
0.5, b: 0.12, a: 0.7 });
204   }
205
206   heal(x, y, z) {
207     this.add(this.glow, { x, y, z, vy: 0.7, life: 0.8, s0: 0.14, s1: 0.06, r: 0.42, g: 1, b: 0.54, a: 0.9 });
208   }
209
210   update(dt) {
211     this.glow.update(dt);
212     this.soft.update(dt);
213   }
214
215   get count() {
216     return this.glow.count + this.soft.count;
217   }
218
219   get capacity() {
220     return this.glow.max + this.soft.max;
221   }
222
223   setScale(px) {
224     this.glow.setScale(px);
225     this.soft.setScale(px);
226   }
227
228   dispose() {
229     this.glow.dispose();
230     this.soft.dispose();
231   }
232 }
```

faro/src/js/07-input.js · 616 líneas

```
'use strict';
// Controles: toque (iPad/celular), ratón y teclado.
// Un dedo arrastra el mapa, dos dedos hacen zoom y giran, un toque selecciona u ordena.

class Input {
  constructor(app) {
    this.app = app;
    this.canvas = app.canvas;
    this.pointers = new Map();
    this.drag = null;
    this.keys = new Set();
    this.boxMode = false;
    this.amoveArm = false; // «Avanzar atacando»: el siguiente toque en el mapa da la orden
    this.placement = null;
    this.lastTap = { t: 0, x: 0, y: 0, id: -1 };
    this.lastRot = {};
    this.bind();
  }

  get game() {
    return this.app.game;
  }

  get r() {
    return this.app.renderer;
  }

  bind() {
    const c = this.canvas;
    c.addEventListener('pointerdown', (e) => this.onDown(e));
    c.addEventListener('pointermove', (e) => this.onMove(e));
    c.addEventListener('pointerup', (e) => this.onUp(e));
    c.addEventListener('pointercancel', (e) => this.onUp(e, true));
    c.addEventListener('pointerleave', () => { if (!this.pointers.size && this.r) this.r.setHover(-1); });
    c.addEventListener('wheel', (e) => this.onWheel(e), { passive: false });
    c.addEventListener('contextmenu', (e) => e.preventDefault());
    window.addEventListener('keydown', (e) => this.onKey(e, true));
    window.addEventListener('keyup', (e) => this.onKey(e, false));
    window.addEventListener('blur', () => this.keys.clear());
  }

  active() {
    return this.app.state === 'play' && this.game && this.r;
  }

  pos(e) {
    const rect = this.canvas.getBoundingClientRect();
    return { x: e.clientX - rect.left, y: e.clientY - rect.top };
  }

  onDown(e) {
    if (!this.active()) return;
    e.preventDefault();
    try { this.canvas.setPointerCapture(e.pointerId); } catch (err) { /* sin captura */ }
    const p = this.pos(e);
    this.pointers.set(e.pointerId, p);
    const cam = this.r.cam;
    if (this.pointers.size === 1) {
      this.drag = {
        mode: null, sx: p.x, sy: p.y, x: p.x, y: p.y, moved: false,
        right: e.button === 2, shift: e.shiftKey, grab: this.r.screenToWorld(p.x, p.y, 0),
      };
      // v1.2: mantener pulsado el mapa (sin moverse) abre la rueda de selección del ejército.
      clearTimeout(this.holdTimer);
      if (e.button !== 2 && !this.placement && !this.amoveArm && !this.boxMode && !e.shiftKey) {
        const d0 = this.drag;
        this.holdTimer = setTimeout(() => {
          if (this.drag === d0 && !d0.moved && this.pointers.size === 1) {
            d0.mode = 'wheel';
            d0.wheelAt = performance.now();
            this.app.ui.showWheel(d0.sx, d0.sy);
          }
        }, WHEEL_HOLD_MS);
      }
    } else if (this.pointers.size === 2) {
      clearTimeout(this.holdTimer);
      const [a, b] = [...this.pointers.values()];
      const mid = { x: (a.x + b.x) / 2, y: (a.y + b.y) / 2 };
      this.drag = {
        mode: 'pinch', moved: true, d0: Math.hypot(a.x - b.x, a.y - b.y) || 1,
        ang0: Math.atan2(b.y - a.y, b.x - a.x), dist0: cam.dist, yaw0: cam.yaw,
        grab: this.r.screenToWorld(mid.x, mid.y, 0),
      };
      this.app.ui.hideBox();
```

```
    }
  }

  onMove(e) {
    if (!this.active()) return;
    const p = this.pos(e);
    if (!this.pointers.has(e.pointerId)) {
      if (e.pointerType === 'mouse' && this.placement) this.hover(p.x, p.y);
      return;
    }
    this.pointers.set(e.pointerId, p);
    const d = this.drag;
    if (!d) return;
    const cam = this.r.cam;
    if (d.mode === 'pinch' && this.pointers.size >= 2) {
      const [a, b] = [...this.pointers.values()];
      const dist = Math.hypot(a.x - b.x, a.y - b.y) || 1;
      const ang = Math.atan2(b.y - a.y, b.x - a.x);
      cam.dist = clamp(d.dist0 * (d.d0 / dist), 9, 70);
      cam.yaw = d.yaw0 + (ang - d.ang0);
      this.r.updateCamera();
      const mid = { x: (a.x + b.x) / 2, y: (a.y + b.y) / 2 };
      const w = this.r.screenToWorld(mid.x, mid.y, 0);
      if (w && d.grab) {
        cam.tx += d.grab.x - w.x;
        cam.tz += d.grab.z - w.z;
        this.r.updateCamera();
      }
      return;
    }
    if (this.pointers.size !== 1) return;
    if (d.mode === 'wheel') {
      // Si el dedo ya se movía antes de que se abriera la rueda (el aviso llegó tarde en un cuadro lento),
      // era un arrastre del mapa: se cierra la rueda y se sigue moviendo la cámara.
      const early = d.wheelAt && e.timeStamp && e.timeStamp < d.wheelAt;
      if (early && Math.hypot(p.x - d.sx, p.y - d.sy) > 9) {
        this.app.ui.hideWheel();
        d.mode = null;
      } else {
        this.app.ui.hoverWheel(p.x, p.y);
        return;
      }
    }
    if (!d.moved && Math.hypot(p.x - d.sx, p.y - d.sy) > 9) {
      d.moved = true;
      clearTimeout(this.holdTimer);
      d.mode = this.boxMode || d.shift ? 'box' : 'pan';
    }
    if (d.mode === 'pan') {
      const w = this.r.screenToWorld(p.x, p.y, 0);
      if (w && d.grab) {
        cam.tx += d.grab.x - w.x;
        cam.tz += d.grab.z - w.z;
        this.r.updateCamera();
      }
    } else if (d.mode === 'box') {
      this.app.ui.showBox(d.sx, d.sy, p.x, p.y);
    }
    d.x = p.x;
    d.y = p.y;
    if (this.placement && e.pointerType === 'mouse') this.hover(p.x, p.y);
  }

  onUp(e, cancelled = false) {
    if (!this.pointers.has(e.pointerId)) return;
    const p = this.pos(e);
    this.pointers.delete(e.pointerId);
    const d = this.drag;
    if (!d) return;
    if (d.mode === 'pinch') {
      if (this.pointers.size === 0) this.drag = null;
      else {
        const [rest] = [...this.pointers.values()];
        this.drag = { mode: 'pan', sx: rest.x, sy: rest.y, moved: true, grab: this.r.screenToWorld(rest.x, rest.y, 0) };
      }
      return;
    }
    if (this.pointers.size) return;
    this.drag = null;
    clearTimeout(this.holdTimer);
    if (d.mode === 'wheel') {
      // Si suelta el dedo sobre una opción, la elige; si no, la rueda queda abierta para tocar.
      if (!cancelled) this.app.ui.releaseWheel(p.x, p.y);
      return;
    }
    if (cancelled || !this.active()) return;
    if (d.mode === 'box') {
      this.app.ui.hideBox();
```

```
      const list = this.r.unitsInRect(d.sx, d.sy, p.x, p.y, (u) => u.owner === 0);
      this.app.setSelection(list);
      if (!list.length) this.app.ui.toast('No hay tropas tuyas en el recuadro');
      return;
    }
    if (!d.moved) this.tap(p.x, p.y, d.right);
  }

  onWheel(e) {
    if (!this.active()) return;
    e.preventDefault();
    const cam = this.r.cam;
    const p = this.pos(e);
    const before = this.r.screenToWorld(p.x, p.y, 0);
    cam.dist = clamp(cam.dist * (1 + clamp(e.deltaY, -120, 120) * 0.0012), 9, 70);
    this.r.updateCamera();
    const after = this.r.screenToWorld(p.x, p.y, 0);
    if (before && after) {
      cam.tx += before.x - after.x;
      cam.tz += before.z - after.z;
    }
  }

  onKey(e, down) {
    if (e.target && (e.target.tagName === 'INPUT' || e.target.tagName === 'TEXTAREA')) return;
    const k = e.key.toLowerCase();
    if (down) this.keys.add(k);
    else this.keys.delete(k);
    if (!down || !this.active()) return;
    // Grupos (v0.7): número elige; Ctrl, Cmd o Mayús + número guarda. Se usa e.code para que
    // Mayús + 1 funcione aunque el teclado escriba «!».
    const digit = /^Digit([1-3])$/.exec(e.code || '');
    if (digit) {
      const n = Number(digit[1]);
      if (e.ctrlKey || e.metaKey || e.shiftKey) {
        e.preventDefault();
        this.app.saveGroup(n);
      } else {
        this.app.selectGroup(n);
      }
      return;
    }
    if (k === 'tab') {
      e.preventDefault();
      if (this.app.ui.wheelOpen()) this.app.ui.hideWheel();
      else this.app.ui.showWheel(this.r.width / 2, this.r.height / 2);
      return;
    }
    if (k === 'escape' && this.app.ui.wheelOpen()) {
      this.app.ui.hideWheel();
      return;
    }
    if (k === 'escape') {
      if (this.amoveArm) this.disarmAttackMove();
      else if (this.placement) this.cancelPlacement();
      else if (this.app.selection.length) this.app.setSelection([]);
      else this.app.togglePause(true);
    } else if (k === 'r' && this.placement && this.placement.kind === 'build') this.rotatePlacement();
    else if (k === 'enter' && this.placement && this.placement.kind === 'build') this.confirmPlacement();
    else if (k === 'b') this.app.ui.toggleBuild();
    else if (k === ' ') {
      e.preventDefault();
      this.app.focusCore();
    } else if (k === 'delete' || k === 'backspace') {
      const s = this.app.selection;
      if (s.length === 1 && s[0].kind === 'bld' && s[0].owner === 0) this.app.ui.demolishSelected();
    } else if (k === 't') this.app.quickSelect('army');
    else if (k === 'p') this.app.togglePause(true);
    else if (k === 'f') this.armAttackMove();
    else if (k === 'c') this.app.selectBuilders();
  }

  update(dt) {
    if (!this.active()) return;
    const cam = this.r.cam;
    const sp = cam.dist * 1.1 * dt;
    let fx = 0;
    let fz = 0;
    if (this.keys.has('w') || this.keys.has('arrowup')) fz -= 1;
    if (this.keys.has('s') || this.keys.has('arrowdown')) fz += 1;
    if (this.keys.has('a') || this.keys.has('arrowleft')) fx -= 1;
    if (this.keys.has('d') || this.keys.has('arrowright')) fx += 1;
    if (fx || fz) {
      const s = Math.sin(cam.yaw);
      const c = Math.cos(cam.yaw);
      cam.tx += (fx * c + fz * s) * sp;
      cam.tz += (-fx * s + fz * c) * sp;
    }
```

```
    if (this.keys.has('q')) cam.yaw += dt * 1.6;
    if (this.keys.has('e')) cam.yaw -= dt * 1.6;
  }

  hover(x, y) {
    const c = this.r.pickCell(x, y);
    this.r.setHover(c ? c.i : -1);
  }

  // ---------- avanzar atacando ----------
  // mode: true = «Avanzar atacando»; 'patrol' = «Patrullar» (v1.4).
  armAttackMove(mode = true) {
    const armed = this.app.selection.filter((e) => e.kind === 'unit' && e.owner === 0 && e.alive && e.def.weapon);
    if (!armed.length) {
      this.app.ui.toast('Elige primero tropas que puedan pelear', 'bad');
      return false;
    }
    this.amoveArm = mode;
    this.app.ui.selSig = '';
    this.app.ui.toast(mode === 'patrol' ? 'Toca el otro punto: irán y volverán entre aquí y allí, peleando en el camino'
      : 'Toca el destino: avanzarán peleando con todo lo que encuentren');
    return true;
  }

  disarmAttackMove() {
    if (!this.amoveArm) return;
    this.amoveArm = false;
    if (this.app.ui) this.app.ui.selSig = '';
  }

  // ---------- toques ----------
  tap(x, y, right) {
    const g = this.game;
    const app = this.app;
    if (this.placement && !right) {
      this.placementTap(x, y);
      return;
    }
    const amove = this.amoveArm;
    this.disarmAttackMove();
    const mine = app.selection.filter((e) => e.kind === 'unit' && e.owner === 0 && e.alive);
    const unit = this.r.pickUnit(x, y);
    const cell = this.r.pickCell(x, y);
    const bld = cell && cell.bld && cell.bld.alive ? cell.bld : (!unit ? this.r.pickBuilding(x, y) : null);
    const point = !unit && !bld && cell && cell.point ? cell.point : null;
    const target = unit || bld;
    const now = performance.now();
    const dbl = now - this.lastTap.t < 330 && Math.hypot(x - this.lastTap.x, y - this.lastTap.y) < 30;
    this.lastTap = { t: now, x, y, id: target ? target.id : -1 };

    // Avanzar atacando: a ese lugar, peleando en el camino (sobre un enemigo, se ataca directo).
    if (amove === 'patrol' && mine.length && cell) {
      const n = g.orderPatrol(mine, cell.i);
      this.r.addMoveMarker(cell.i, 0xffd166);
      app.ui.toast(n ? `${n === 1 ? 'Patrulla' : `${n} tropas patrullan`} entre aquí y allí` : 'Esas tropas no pueden
patrullar', n ? 'info' : 'bad');
      app.sound.play('order');
      return;
    }
    if (amove && mine.length && cell && !(target && g.isEnemy(target.owner, 0))) {
      g.orderMove(mine, cell.i, true, app.formation);
      this.r.addMoveMarker(cell.i, 0xff6060);
      app.ui.toast(`${mine.length === 1 ? 'Avanza' : `${mine.length} tropas avanzan`} atacando`);
      app.sound.play('order');
      return;
    }

    if (target && g.isEnemy(target.owner, 0)) {
      if (mine.length) {
        const armed = mine.filter((u) => u.def.weapon && g.canAttack(u, u.def.weapon, target));
        if (armed.length) {
          g.orderAttack(armed, target);
          this.r.addMoveMarker(target.kind === 'bld' ? target.cell : target.cell, 0xff6060);
          app.sound.play('order');
          const rest = mine.filter((u) => !armed.includes(u));
          if (rest.length) g.orderMove(rest, target.cell);
        } else {
          app.ui.toast('Tus tropas seleccionadas no pueden atacar eso', 'bad');
        }
        return;
      }
      app.setSelection([target]);
      return;
    }
    if (right) {
      if (mine.length && cell) this.moveTo(mine, cell.i);
      return;
    }
```

```
    // Constructores seleccionados + toque en una obra tuya = construir o mejorar.
    if (bld && bld.owner === 0 && g.needsWork(bld)) {
      const workers = mine.filter((u) => g.canBuildSite(u, bld));
      if (workers.length) {
        g.orderBuild(workers, bld);
        this.r.addMoveMarker(bld.cell, 0xf2ab38);
        app.ui.toast(`${workers.length === 1 ? 'Construyendo' : `${workers.length} constructores en`} ${bld.def.name}`);
        app.sound.play('order');
        return;
      }
    }
    // Ingeniero seleccionado + toque en un edificio tuyo dañado = reparar.
    if (bld && bld.owner === 0 && bld.hp < bld.maxHp && bld.built) {
      const fixers = mine.filter((u) => u.def.repairs);
      if (fixers.length) {
        g.orderRepair(fixers, bld);
        const rest = mine.filter((u) => !u.def.repairs);
        if (rest.length) g.orderMove(rest, bld.cell);
        this.r.addMoveMarker(bld.cell, 0x6dff8a);
        app.ui.toast(`Reparando ${bld.def.name}`);
        app.sound.play('order');
        return;
      }
    }
    // Toque en el Faro o una Bandera: con tropas seleccionadas, ir a capturarlo.
    if (point) {
      if (mine.length) {
        g.orderMove(mine, cell.i, true);
        this.r.addMoveMarker(cell.i, 0xffb63a);
        const inf = mine.some((u) => u.cls === 'inf');
        app.ui.toast(inf ? 'Quédate junto al punto sin enemigos cerca para capturarlo'
          : 'Solo la infantería captura puntos', inf ? 'info' : 'bad');
        app.sound.play('order');
      } else {
        app.setSelection([point]);
        app.sound.play('click');
      }
      return;
    }
    // Tropas seleccionadas + toque en tu transporte (Barcaza, Lancha, Jeep, Transporte, Portaaviones) = subir.
    if (target && target.owner === 0 && (target.def.capacity || target.def.hangar)) {
      const landers = mine.filter((u) => g.canBoard(u, target));
      if (landers.length) {
        g.orderBoard(landers, target);
        this.r.addMoveMarker(target.cell, 0x7dd3ff);
        app.ui.toast(`Subiendo ${landers.length} ${landers.length === 1 ? 'tropa' : 'tropas'} a ${target.def.name}`);
        app.sound.play('order');
        return;
      }
    }
    if (target && target.owner === 0) {
      if (target.kind === 'unit' && dbl) {
        const same = this.r.unitsInRect(0, 0, this.r.width, this.r.height, (u) => u.owner === 0 && u.type ===
target.type);
        app.setSelection(same.length ? same : [target]);
      } else {
        app.setSelection([target]);
      }
      app.sound.play('click');
      return;
    }
    if (target) {
      app.setSelection([target]);
      return;
    }
    if (mine.length && cell) {
      this.moveTo(mine, cell.i);
      return;
    }
    app.setSelection([]);
  }

  moveTo(units, ci) {
    const g = this.game;
    const founders = units.filter((u) => u.state === 'found');
    g.orderMove(units, ci, false, this.app.formation);
    for (const u of founders) u.foundCell = -1;
    this.r.addMoveMarker(ci);
    this.app.sound.play('order');
  }

  // ---------- colocación ----------
  startPlacement(p) {
    if (p.kind === 'build') {
      p.rot = this.lastRot[p.btype] || 0;
      p.cell = -1;
      p.err = null;
      p.placed = 0;
```

```
    }
    this.placement = p;
    this.refreshPlacement();
    this.app.ui.showPlaceBar(p);
  }

  previewAt(ci) {
    const p = this.placement;
    p.cell = ci;
    p.err = this.game.placeInvalid(0, p.btype, ci);
    this.r.setGhost(p.btype, ci, p.rot, !p.err);
    this.app.ui.showPlaceBar(p);
  }

  rotatePlacement() {
    const p = this.placement;
    if (!p || p.kind !== 'build') return;
    p.rot = (p.rot + Math.PI / 3) % (Math.PI * 2);
    this.lastRot[p.btype] = p.rot;
    if (p.cell >= 0) this.r.setGhost(p.btype, p.cell, p.rot, !p.err);
    this.app.sound.play('click');
  }

  // «Construir» (v1.0): tocar una casilla solo muestra la obra fantasma; este botón la pone.
  // Después se puede elegir otra casilla y volver a construir; «Terminar» sale.
  confirmPlacement() {
    const p = this.placement;
    if (!p || p.kind !== 'build') return;
    if (p.cell < 0) {
      this.app.ui.toast('Toca primero una casilla dorada', 'bad');
      this.app.sound.play('error');
      return;
    }
    this.buildAt(p.cell);
  }

  // Pone la obra en la casilla elegida y sigue en modo construir para poner más.
  buildAt(ci) {
    const g = this.game;
    const p = this.placement;
    p.cell = ci;
    const err = g.place(0, p.btype, p.cell, p.rot);
    if (err) {
      p.err = err;
      this.r.setGhost(p.btype, p.cell, p.rot, false);
      this.app.ui.showPlaceBar(p);
      this.app.ui.toast(err, 'bad');
      this.app.sound.play('error');
      return;
    }
    this.lastRot[p.btype] = p.rot;
    this.r.addMoveMarker(p.cell, 0xf5c542);
    this.app.ui.refreshBuild(true);
    p.placed++;
    p.cell = -1;
    p.err = null;
    this.r.setGhost(null, -1);
    this.r.setHover(-1);
    this.refreshPlacement();
    this.app.ui.showPlaceBar(p);
  }

  refreshPlacement() {
    const p = this.placement;
    if (!p || !this.game) return;
    const g = this.game;
    let cells = [];
    if (p.kind === 'build') {
      cells = g.cells.filter((c) => !g.placeInvalid(0, p.btype, c.i, true)).map((c) => c.i);
    } else if (p.kind === 'found') {
      const u = p.unit;
      const comp = g.cells[u.cell].comp;
      cells = g.cells.filter((c) => !g.foundInvalid(0, c.i, u.def.founds) &&
        (u.mover !== 'land' || c.comp === comp)).map((c) => c.i);
    } else if (p.kind === 'rally' || p.kind === 'bombzone' || p.kind === 'allyattack') {
      cells = [];
    } else if (p.kind === 'nuke') {
      cells = g.cells.filter((c) => !g.nukeInvalid(p.building, c.i) && g.map.dist(c, g.cells[p.building.cell]) <=
NUKE.range).map((c) => c.i);
    } else if (p.kind === 'nuke3') {
      const silos = g.readySilos(0).slice(0, 3);
      cells = silos.length < 3 || g.salvoInvalid(0) ? [] : g.cells.filter((c) => silos.every((b) => !g.nukeInvalid(b,
c.i))).map((c) => c.i);
    }
    p.valid = new Set(cells);
    this.r.setPlacement(cells);
    if (p.kind === 'build' && p.cell >= 0) {
      const err = g.placeInvalid(0, p.btype, p.cell);
```

```
      if (err !== p.err) {
        p.err = err;
        this.r.setGhost(p.btype, p.cell, p.rot, !err);
        this.app.ui.showPlaceBar(p);
      }
    }
  }

  cancelPlacement() {
    this.placement = null;
    if (this.r) {
      this.r.setPlacement(null);
      this.r.setGhost(null, -1);
      this.r.hoverMesh.material.color.set(0xffffff);
      this.r.setHover(-1);
    }
    this.app.ui.hidePlaceBar();
  }

  placementTap(x, y) {
    const g = this.game;
    const p = this.placement;
    const cell = this.r.pickCell(x, y);
    if (!cell) return;
    if (p.kind === 'build') {
      this.previewAt(cell.i);
      this.app.sound.play('click');
    } else if (p.kind === 'found') {
      if (!p.unit.alive) {
        this.cancelPlacement();
        return;
      }
      const err = g.orderFound(p.unit, cell.i);
      if (err) {
        this.app.ui.toast(err, 'bad');
        this.app.sound.play('error');
        return;
      }
      this.r.addMoveMarker(cell.i, 0xf5c542);
      this.app.ui.toast('El ingeniero va en camino (cuesta 800 al llegar)');
      this.cancelPlacement();
    } else if (p.kind === 'nuke') {
      const err = g.launchNuke(p.building, cell.i);
      if (err) {
        this.app.ui.toast(err, 'bad');
        this.app.sound.play('error');
        return;
      }
      this.r.addMoveMarker(cell.i, 0xff4040);
      this.cancelPlacement();
    } else if (p.kind === 'allyattack') {
      // El rival es el dueño de lo que tocaste: edificio, tropa cercana o territorio.
      let pid = cell.bld && cell.bld.alive ? cell.bld.owner : -1;
      if (pid < 0) {
        const u = g.units.find((x) => x.alive && Math.hypot(x.x - cell.x, x.z - cell.z) < 1.2 && g.isEnemy(0, x.owner));
        if (u) pid = u.owner;
      }
      if (pid < 0 && cell.owner >= 0) pid = cell.owner;
      if (pid < 0 || !g.isEnemy(0, pid)) {
        this.app.ui.toast('Toca algo de un rival', 'bad');
        this.app.sound.play('error');
        return;
      }
      const bots = g.bots.filter((b) => b.p.alive && b.p.team === g.players[0].team && b.pid !== 0);
      const n = bots.reduce((a, b) => a + b.answerAttack(pid), 0);
      this.r.addMoveMarker(cell.i, 0xff4040);
      this.app.ui.toast(`Tus aliados atacan a ${g.players[pid].name}${n ? ` con ${n} tropas` : ''}`, 'good');
      this.app.sound.play('order');
      this.cancelPlacement();
    } else if (p.kind === 'bombzone') {
      const bombers = p.units.filter((u) => u.alive);
      if (bombers.length) {
        g.orderMove(bombers, cell.i, true);
        this.r.addMoveMarker(cell.i, 0xff4040);
        this.app.ui.toast(`${bombers.length === 1 ? 'El Bombardero va' : 'Los Bombarderos van'} a bombardear la zona`,
'info');
        this.app.sound.play('order');
      }
      this.cancelPlacement();
    } else if (p.kind === 'nuke3') {
      const res = g.launchSalvo(0, cell.i);
      if (typeof res === 'string') {
        this.app.ui.toast(res, 'bad');
        this.app.sound.play('error');
        return;
      }
      this.r.addMoveMarker(cell.i, 0xff4040);
      this.app.ui.toast(`¡${res} misiles nucleares lanzados a la vez!`, 'info');
```

```
608       this.cancelPlacement();
609     } else if (p.kind === 'rally') {
610       p.building.rally = cell.i;
611       this.r.addMoveMarker(cell.i, 0xffffff);
612       this.cancelPlacement();
613       this.app.ui.toast('Punto de reunión fijado');
614     }
615   }
616 }
```

faro/src/js/08-ui.js · 2,024 líneas

```
'use strict';
// Interfaz HTML: menú, barra superior, panel de construcción, panel de selección,
// avisos, objetivos, pausa y pantalla final. También el sonido sintetizado.

const $ = (id) => document.getElementById(id);

const ICON_UP = '↑';
const ICON = {
  soundOn: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M4 9v6h4l5 4V5L8 9z"/><path d="M16.5 8.5a5 5 0 0 1 0 7M19 6a8.5 8.5 0 0 1 0 12"/></svg>',
  soundOff: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2"><path d="M4 9v6h4l5 4V5L8 9z"/><path d="M17 9l5 6M22 9l-5 6"/></svg>',
  close: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.2"><path d="M6 6l12 12M18 6 6 18"/></svg>',
  pop: '<svg viewBox="0 0 24 24" width="12" height="12" fill="none" stroke="currentColor" stroke-width="2.4"><circle cx="12" cy="8" r="3.5"/><path d="M5 21c0-4 3-7 7-7s7 3 7 7"/></svg>',
  bolt: '<svg viewBox="0 0 24 24" width="11" height="11" fill="currentColor"><path d="M13 2 4.5 13.5H11L10 22l8.5-11.5H12z"/></svg>',
  arrow: '<svg viewBox="0 0 24 24" fill="currentColor"><path d="M4 10h10V5l7 7-7 7v-5H4z"/></svg>',
};
// Iconos de los botones de acción (v0.8). bi(nombre) devuelve el icono listo para poner antes del texto.
const BTN_ICON = {
  amove: '<circle cx="12" cy="12" r="7"/><path d="M12 2v5M12 17v5M2 12h5M17 12h5"/>',
  formation: '<path d="M4 8h3M10.5 8h3M17 8h3M4 15h3M10.5 15h3M17 15h3"/>',
  stop: '<rect x="6" y="6" width="12" height="12" rx="1.5"/>',
  // v1.3: «Más detalles» (lista) y «Menos» (flecha arriba).
  more: '<path d="M5 7h14M5 12h14M5 17h9"/>',
  // v1.4: patrullar (ida y vuelta), mantener posición (ancla), reparar, siempre (bucle).
  patrol: '<path d="M4 9h13l-3-3M20 15H7l3 3"/>',
  hold: '<circle cx="12" cy="5" r="2"/><path d="M12 7v13M6 14a6 6 0 0 0 12 0M4 12h3M17 12h3"/>',
  wrench: '<path d="M14.5 5.5a4 4 0 0 0-5 5L4 16l4 4 5.5-5.5a4 4 0 0 0 5-5l-2.5 2.5-2.5-.5-.5-2.5z"/>',
  less: '<path d="M6 15l6-6 6 6"/>',
  upgrade: '<path d="M12 19V6M6 11l6-6 6 6"/><path d="M6 20h12"/>',
  demolish: '<path d="M5 5l14 14M19 5 5 19"/>',
  rally: '<path d="M6 21V4M6 4h11l-2.5 4L17 12H6"/>',
  repeat: '<path d="M4 12a8 8 0 0 1 13.7-5.6M20 4v5h-5M20 12a8 8 0 0 1-13.7 5.6M4 20v-5h5"/>',
  group: '<path d="M7 3h10v18l-5-4-5 4z"/>',
  unload: '<path d="M12 4v11M7 10l5 5 5-5M5 20h14"/>',
  found: '<path d="M6 21V4h10l-2 3.5L16 11H6M18 15v6M15 18h6"/>',
  dome: '<path d="M3 18a9 9 0 0 1 18 0zM2 18h20"/>',
};
const bi = (k) => `<span class="bi" aria-hidden="true"><svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2.2" stroke-linecap="round" stroke-linejoin="round">${BTN_ICON[k]}</svg></span>`;

function esc(s) {
  return String(s).replace(/[&<>"]/g, (c) => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '"': '&quot;' }[c]));
}

class UI {
  constructor(app) {
    this.app = app;
    this.tab = 'eco';
    this.selSig = '';
    this.buildSig = '';
    this.slowT = 0;
    this.lastToast = {};
    this.demolishArm = -1;
    this.playersSig = '';
    this.edgeAlerts = [];
    this.initHud();
  }

  showScreen(name) {
    $('menu').hidden = name !== 'menu';
    $('hud').hidden = name !== 'play';
    $('pause').hidden = true;
    $('end').hidden = true;
    $('brief').hidden = true;
    $('modal').hidden = true;
    if (this.app.lobby) {
      this.app.lobby.modal = null;
      if (name === 'menu') this.app.lobby.render();
    }
  }

  // Objetivos de la partida: en el Tutorial son sus 3 pasos.
  objList() {
    const g = this.app.game;
    return g && g.mode.tutorial ? TUTORIAL_STEPS : OBJECTIVES;
  }

  // ---------- campaña ----------
  // Historia y metas al empezar una misión. La partida espera en pausa hasta «¡A jugar!».
  showBrief() {
```

```
    const app = this.app;
    const m = Campaign.def(app.game.camp.id);
    app.paused = true;
    const el = $('brief');
    el.innerHTML = `<div class="dialog panel wide brief" role="dialog" aria-labelledby="brief-title">
      <small class="lbl">Campaña · Misión ${m.id} de ${CAMPAIGN.length}</small>
      <h2 id="brief-title">${esc(m.name)}</h2>
      <p class="story">${esc(m.story)}</p>
      <div class="brief-grid">
        <div><h3 class="lbl">Meta</h3><p><b>${esc(Campaign.goalText(m))}</b></p></div>
        <div><h3 class="lbl">Estrellas</h3><ul class="starlist"><li><b>★ Termina la misión</b></li>
          ${m.stars.map((s) => `<li><b>★ ${esc(Campaign.starText(s))}</b></li>`).join('')}</ul></div>
      </div>
      <p class="tip"><b>Consejo:</b> ${esc(m.tip)}</p>
      <div class="row brief-go"><button class="btn gold" type="button" id="btn-brief-go">¡A jugar!</button>
        <button class="btn" type="button" id="btn-brief-menu">Vestíbulo</button></div>
    </div>`;
    el.hidden = false;
    $('btn-brief-go').addEventListener('click', () => {
      app.sound.unlock();
      app.sound.play('click');
      this.hideBrief();
    });
    $('btn-brief-menu').addEventListener('click', () => {
      this.hideBrief();
      app.toMenu();
      app.abandonGame();
    });
  }

  hideBrief() {
    $('brief').hidden = true;
    this.app.paused = false;
  }

  // Panel de la misión: meta con su progreso y las estrellas (con su estado en vivo).
  renderMission(force) {
    const g = this.app.game;
    if (!g || !g.camp) return;
    const lines = Campaign.goalLines(g).map((l) => `<li class="${l.done ? 'done' : ''}">${esc(l.text)}</li>`);
    const stars = Campaign.stars(g).slice(1).map((s) =>
      `<li class="star ${s.live ? '' : 'lost'}"><span>${esc(s.text)}<small>${esc(s.live ? s.now : 'ya no se puede')}
</small></span></li>`);
    const html = lines.join('') + stars.join('');
    if (!force && html === this.missionSig) return;
    this.missionSig = html;
    $('objlist').innerHTML = html;
  }

  // ---------- HUD ----------
  initHud() {
    const app = this.app;
    $('btn-pause').addEventListener('click', () => app.togglePause(true));
    $('btn-resume').addEventListener('click', () => app.togglePause(false));
    $('btn-restart').addEventListener('click', () => app.startGame());
    $('btn-spec-exit').addEventListener('click', () => app.toMenu());
    // v1.2: tocar un mensaje abre el panel de Mensajes.
    $('feed').addEventListener('click', (e) => { if (!e.target.closest('button')) { this.chatKey = null; if (this.pop
!== 'chat') this.togglePop('chat'); } });
    $('btn-quit').addEventListener('click', () => app.toMenu());
    $('btn-fps').addEventListener('click', () => app.toggleFps());
    app.toggleFps(app.showFps);
    document.querySelectorAll('[data-quality]').forEach((b) => {
      b.addEventListener('click', () => app.setQuality(b.dataset.quality));
    });
    // Música (v1.1): volumen en la pausa y botón en el vestíbulo.
    document.querySelectorAll('[data-music]').forEach((b) => {
      b.addEventListener('click', () => { app.sound.unlock(); app.music.setLevel(Number(b.dataset.music));
this.updateMusicBtns(); });
    });
    $('btn-music-lobby').addEventListener('click', () => {
      app.sound.unlock();
      const lv = app.music.level > 0 ? 0 : 0.6;
      app.music.setLevel(lv);
      if (lv > 0) app.music.play(app.state === 'play' ? 'battle' : 'lobby');
      this.updateMusicBtns();
    });
    this.updateMusicBtns();
    // v1.4: ahorro de batería, daltonismo y letra grande.
    $('btn-battery').addEventListener('click', () => app.setBattery(!app.battery));
    $('btn-cvd').addEventListener('click', () => app.setCvd(!app.cvd));
    $('btn-bigtext').addEventListener('click', () => app.setBigText(!app.bigText));
    this.updateA11yBtns();
    for (const id of ['btn-sound', 'btn-sound2']) {
      $(id).addEventListener('click', () => {
        app.sound.toggle();
        this.updateSoundBtn();
      });
```

```
    }
    this.updateSoundBtn();
    $('btn-obj').addEventListener('click', () => {
      const el = $('objectives');
      el.classList.toggle('collapsed');
      $('btn-obj').textContent = el.classList.contains('collapsed') ? 'Mostrar' : 'Ocultar';
    });
    $('btn-build').addEventListener('click', () => this.toggleBuild());
    $('btn-build-close').addEventListener('click', () => {
      if (window.innerWidth <= 860) this.toggleBuild(false);
      else $('buildpanel').classList.toggle('min');
    });
    $('btn-place-cancel').addEventListener('click', () => app.input.cancelPlacement());
    $('btn-place-rot').addEventListener('click', () => app.input.rotatePlacement());
    $('btn-place-ok').addEventListener('click', () => app.input.confirmPlacement());
    $('btn-mini').addEventListener('click', () => {
      const w = $('miniwrap');
      w.classList.toggle('min');
      $('btn-mini').setAttribute('aria-label', w.classList.contains('min') ? 'Mostrar minimapa' : 'Plegar minimapa');
    });
    $('btn-home').addEventListener('click', () => app.focusCore());
    $('btn-box').addEventListener('click', () => {
      app.input.boxMode = !app.input.boxMode;
      $('btn-box').classList.toggle('on', app.input.boxMode);
      if (app.input.boxMode) this.toast('Arrastra para seleccionar varias tropas');
    });
    document.querySelectorAll('[data-quick]').forEach((b) => {
      b.addEventListener('click', () => app.quickSelect(b.dataset.quick));
    });
    document.querySelectorAll('[data-group]').forEach((b) => {
      b.addEventListener('click', () => app.selectGroup(Number(b.dataset.group)));
    });
    $('btn-builders').addEventListener('click', () => app.selectBuilders());
    $('prodbar').addEventListener('click', (e) => {
      const b = e.target.closest('[data-prod]');
      if (!b || !app.game) return;
      const bld = app.game.buildings.find((x) => x.id === Number(b.dataset.prod) && x.alive);
      if (!bld) return;
      app.sound.play('click');
      app.setSelection([bld]);
      app.focusAt(bld.x, bld.z);
    });

    $('btabs').innerHTML = BUILD_TABS.map((t) =>
      `<button class="tab" role="tab" type="button" data-tab="${t.id}" aria-selected="${t.id ===
this.tab}">${esc(t.name)}</button>`).join('');
    $('btabs').addEventListener('click', (e) => {
      const b = e.target.closest('[data-tab]');
      if (!b) return;
      this.tab = b.dataset.tab;
      $('btabs').querySelectorAll('.tab').forEach((x) => x.setAttribute('aria-selected', String(x.dataset.tab ===
this.tab)));
      this.renderBuildList();
    });
    $('blist').addEventListener('click', (e) => {
      const b = e.target.closest('[data-b]');
      if (!b || !app.game) return;
      app.sound.play('click');
      const type = b.dataset.b;
      const g = app.game;
      const def = BUILDINGS[type];
      if (def.limit && g.count(0, type) >= def.limit) {
        this.toast(`Límite alcanzado (${def.limit})`, 'bad');
        return;
      }
      app.setSelection([]);
      app.input.startPlacement({ kind: 'build', btype: type });
      if (window.innerWidth <= 860) this.toggleBuild(false);
    });

    $('selpanel').addEventListener('click', (e) => this.onSelClick(e));
    // Fichas de unidad: mantener pulsada una tarjeta (o pasar el ratón) muestra sus datos.
    const sp = $('selpanel');
    sp.addEventListener('pointerdown', (e) => {
      const card = e.target.closest('.card[data-type]');
      if (!card) return;
      clearTimeout(this.pressT);
      this.suppressClick = false;
      this.pressT = setTimeout(() => {
        this.suppressClick = true;
        this.showCardInfo(card.dataset.type);
      }, 450);
    });
    const endPress = () => clearTimeout(this.pressT);
    sp.addEventListener('pointerup', endPress);
    sp.addEventListener('pointercancel', endPress);
    sp.addEventListener('pointerover', (e) => {
      if (e.pointerType !== 'mouse') return;
```

```
      const card = e.target.closest('.card[data-type]');
      if (card) this.showCardInfo(card.dataset.type);
    });
    sp.addEventListener('contextmenu', (e) => { if (e.target.closest('.card')) e.preventDefault(); });
    // Desglose de economía y marcador.
    document.querySelectorAll('[data-econ]').forEach((el) => {
      const open = () => this.togglePop('econ', el.dataset.econ);
      el.addEventListener('click', open);
      el.addEventListener('keydown', (e) => { if (e.key === 'Enter' || e.key === ' ') { e.preventDefault(); open(); }
});
    });
    $('players').addEventListener('click', () => this.togglePop('score'));
    $('players').addEventListener('keydown', (e) => { if (e.key === 'Enter' || e.key === ' ') { e.preventDefault();
this.togglePop('score'); } });
    for (const id of ['econ', 'score', 'chatpop']) {
      $(id).addEventListener('click', (e) => {
        if (e.target.closest('[data-close]')) this.togglePop(null);
        const ab = e.target.closest('[data-ally]');
        if (ab) this.allyAction(ab.dataset.ally, Number(ab.dataset.pid));
        const ct = e.target.closest('[data-chattab]');
        if (ct) { this.chatTab = ct.dataset.chattab; this.chatKey = null; this.renderPop(); }
        const qm = e.target.closest('[data-qmsg]');
        if (qm) this.quickMsg(qm.dataset.qmsg);
        if (e.target.closest('[data-openchat]')) { this.chatKey = null; this.togglePop('chat'); }
      });
    }

    const mm = $('minimap');
    const moveCam = (e) => {
      if (!app.renderer || app.state !== 'play') return;
      const rect = mm.getBoundingClientRect();
      const w = app.renderer.minimapToWorld(mm, e.clientX - rect.left, e.clientY - rect.top);
      app.renderer.cam.tx = w.x;
      app.renderer.cam.tz = w.z;
    };
    mm.addEventListener('pointerdown', (e) => {
      e.preventDefault();
      try { mm.setPointerCapture(e.pointerId); } catch (err) { /* sin captura */ }
      this.mmDrag = true;
      moveCam(e);
    });
    mm.addEventListener('pointermove', (e) => { if (this.mmDrag) moveCam(e); });
    mm.addEventListener('pointerup', () => { this.mmDrag = false; });
    mm.addEventListener('pointercancel', () => { this.mmDrag = false; });
  }

  updateA11yBtns() {
    const app = this.app;
    const set = (id, on, label) => {
      const b = $(id);
      if (!b) return;
      b.classList.toggle('gold', !!on);
      b.setAttribute('aria-pressed', String(!!on));
      b.textContent = `${label}: ${on ? 'sí' : 'no'}`;
    };
    set('btn-battery', app.battery, 'Ahorro de batería');
    set('btn-cvd', app.cvd, 'Colores para daltonismo');
    set('btn-bigtext', app.bigText, 'Letra grande');
  }

  updateMusicBtns() {
    const lv = this.app.music.level;
    document.querySelectorAll('[data-music]').forEach((b) => b.classList.toggle('gold', Number(b.dataset.music) ===
lv));
    const lb = $('btn-music-lobby');
    lb.classList.toggle('off', lv === 0);
    lb.setAttribute('aria-pressed', String(lv > 0));
    lb.title = lv > 0 ? 'Música: sí (toca para apagarla)' : 'Música: no (toca para encenderla)';
  }

  updateSoundBtn() {
    $('btn-sound2').textContent = this.app.sound.on ? 'Sonido: sí' : 'Sonido: no';
    $('btn-sound').innerHTML = this.app.sound.on ? ICON.soundOn : ICON.soundOff;
    $('btn-sound').setAttribute('aria-label', this.app.sound.on ? 'Silenciar' : 'Activar sonido');
  }

  startHud() {
    this.selSig = '';
    this.buildSig = '';
    this.playersSig = '';
    $('toasts').innerHTML = '';
    $('objectives').hidden = false;
    const camp = this.app.game.camp;
    // En pantallas chicas el panel de la misión empieza plegado (se abre con «Mostrar»).
    const small = window.innerWidth <= 860 || window.innerHeight <= 520;
    $('objectives').classList.toggle('collapsed', !!camp && small);
    $('btn-obj').textContent = camp && small ? 'Mostrar' : 'Ocultar';
    $('leftcol').classList.toggle('camp', !!camp);
```

```
    document.body.classList.toggle('camp-on', !!camp);
    $('objtitle').textContent = camp ? Campaign.mode(camp.id).name : 'Primeros pasos';
    if (camp) this.renderMission(true);
    else $('objlist').innerHTML = this.objList().map((o) => `<li data-obj="${o.id}"
class="${this.app.game.objDone[o.id] ? 'done' : ''}">${esc(o.text)}</li>`).join('');
    $('btn-box').classList.remove('on');
    this.groupsSig = '';
    this.renderGroups(true);
    this.prodSig = '';
    for (const a of this.edgeAlerts || []) if (a.el) a.el.remove();
    this.edgeAlerts = [];
    $('king').hidden = !this.app.game.mode.king;
    $('feed').innerHTML = '';
    this.togglePop(null);
    this.cardInfoType = null;
    this.renderBuildList();
    this.hidePlaceBar();
    $('selpanel').hidden = true;
  }

  toggleBuild(force) {
    const el = $('buildpanel');
    const open = force === undefined ? !el.classList.contains('open') : force;
    el.classList.toggle('open', open);
  }

  markObjective(id) {
    if (this.app.game.camp) return;
    const li = document.querySelector(`[data-obj="${id}"]`);
    if (li) li.classList.add('done');
    const all = this.objList().every((o) => this.app.game.objDone[o.id]);
    if (all) setTimeout(() => { $('objectives').hidden = true; }, 4000);
  }

  // Tarjeta «Qué mejoró» (v0.9): sale al terminar cada mejora de un edificio tuyo.
  upgradeCard(b, level, lines) {
    const box = $('toasts');
    const el = document.createElement('div');
    el.className = 'toast upcard good';
    const name = b.def.levelNames ? b.def.levelNames[level] : `${b.def.name} · nivel ${level}`;
    el.innerHTML = `<b>${ICON_UP} ${esc(name)}: qué mejoró</b><ul>${lines.map((l) => `<li>${esc(l)}</li>`).join('')}
</ul>`;
    el.style.pointerEvents = 'auto';
    el.addEventListener('click', () => el.remove());
    box.appendChild(el);
    while (box.children.length > 3) box.removeChild(box.firstChild);
    setTimeout(() => el.remove(), 7000);
  }

  toast(text, kind = 'info', at = null) {
    const now = performance.now();
    if (this.lastToast[text] && now - this.lastToast[text] < 1200) return;
    this.lastToast[text] = now;
    const box = $('toasts');
    const el = document.createElement('div');
    el.className = 'toast ' + kind;
    const span = document.createElement('span');
    span.textContent = text;
    el.appendChild(span);
    if (at && at.x !== undefined) {
      el.classList.add('has-btn');
      const b = document.createElement('button');
      b.type = 'button';
      b.className = 'mini-btn';
      b.textContent = 'Ver';
      b.addEventListener('click', () => {
        this.app.focusAt(at.x, at.z);
        el.remove();
      });
      el.appendChild(b);
    }
    box.appendChild(el);
    while (box.children.length > 3) box.removeChild(box.firstChild);
    setTimeout(() => el.remove(), at ? 5000 : 2800);
  }

  // Mensaje de un bot: al panel de mensajes (pantallas anchas) o como aviso (estrechas).
  chat(msg) {
    const g = this.app.game;
    if (!msg.toMe && !msg.ally && !msg.public) return;
    const p = g.players[msg.pid];
    const who = msg.ally ? `${p.name} (aliado)` : p.name;
    const at = msg.x !== undefined ? { x: msg.x, z: msg.z } : null;
    const feed = $('feed');
    const wide = feed.offsetParent !== null;
    if (wide) {
      const el = document.createElement('div');
      el.className = 'msg' + (msg.toMe ? ' bad' : '');
```

```
      el.innerHTML = `<i style="background:${hexColor(p.color)}"></i><div><b>${esc(who)}:</b> ${esc(msg.text)}</div>`;
      if (at) {
        const b = document.createElement('button');
        b.type = 'button';
        b.className = 'mini-btn';
        b.textContent = 'Ver';
        b.addEventListener('click', () => this.app.focusAt(at.x, at.z));
        el.appendChild(b);
      }
      feed.appendChild(el);
      while (feed.children.length > 4) feed.removeChild(feed.firstChild);
      setTimeout(() => el.classList.add('old'), 15000);
      setTimeout(() => el.remove(), 40000);
    }
    if (msg.toMe || !wide) this.toast(`${who}: ${msg.text}`, msg.toMe ? 'bad' : 'info', msg.toMe ? at : null);
    if (msg.toMe) this.app.sound.play('alarm');
  }

  // ---------- tutorial que señala qué tocar (v1.2) ----------
  // Si sigues en modo construir (la obra ya está puesta y el modo sigue para poner más), primero se sale de él:
  // si no, tocar un edificio o una tropa no la elige.
  coachStep(g) {
    const step = this.coachStepRaw(g);
    const p = this.app.input.placement;
    if (step && p && !step.place) {
      return { sel: '#btn-place-cancel', text: p.placed ? 'Toca «Terminar» para salir del modo construir' : 'Toca
«Cancelar» para salir de este modo' };
    }
    return step;
  }

  coachStepRaw(g) {
    const app = this.app;
    const me = g.players[0];
    const mine = g.units.filter((u) => u.owner === 0 && u.alive);
    const blds = g.buildings.filter((b) => b.owner === 0 && b.alive);
    const sel = app.selection;
    const core = me.core;
    const p = app.input.placement;
    const bp = $('buildpanel');
    // (offsetParent no sirve con paneles «position: fixed»; se mira si ocupa lugar en la pantalla.)
    const shown = (el) => !!el && el.getClientRects().length > 0 && getComputedStyle(el).visibility !== 'hidden';
    const panelOpen = shown(bp) && !bp.classList.contains('min') && (window.innerWidth > 860 ||
bp.classList.contains('open'));
    const buildFlow = (tab, type, name, tabName) => {
      if (p && p.kind === 'build' && p.btype === type) {
        if (p.cell < 0) {
          const c = [...(p.valid || [])].map((i) => g.cells[i]).sort((a, b) => g.map.dist(a, g.cells[core.cell]) -
g.map.dist(b, g.cells[core.cell]))[0];
          return c ? { world: { x: c.x, y: c.h, z: c.z }, text: 'Toca una casilla dorada (por ejemplo, esta)', place:
true } : { text: 'No quedan casillas libres', place: true };
        }
        return { sel: '#btn-place-ok', text: 'Toca «Construir» para poner la obra', place: true };
      }
      if (!panelOpen) return { sel: '#btn-build', text: 'Abre «Construir»' };
      if (this.tab !== tab) return { sel: `#btabs [data-tab="${tab}"]`, text: `Abre la pestaña «${tabName}»` };
      return { sel: `.bitem[data-b="${type}"]`, text: `Elige «${name}»` };
    };
    const hasBuilder = mine.some((u) => u.def.builds) || core.queue.some((q) => q.type === 'builder');
    if (!hasBuilder) {
      if (!(sel.length === 1 && sel[0] === core)) return { world: { x: core.x, y: core.y + 1, z: core.z }, text: 'Toca
tu Núcleo' };
      return { sel: '[data-act="train"][data-type="builder"]', text: 'Entrena un Constructor' };
    }
    // v1.3: los 3 pasos de tu documento. 1 Construí: otra Granja y una energía.
    const at = (e, dy = 0.5) => ({ x: e.x, y: (e.y || 0) + dy, z: e.z });
    const isSel = (e) => sel.length === 1 && sel[0] === e;
    if (blds.filter((b) => b.type === 'farm').length < 2) return buildFlow('eco', 'farm', 'Granja', 'Economía');
    if (!blds.some((b) => ['solar', 'wind', 'hydro', 'tidal', 'wave'].includes(b.type))) return buildFlow('energy',
'solar', 'Solar', 'Energía');
    // 2 Producí: un Cuartel y 3 soldados.
    const bar = blds.find((b) => b.type === 'barracks');
    if (!bar) return buildFlow('prod', 'barracks', 'Cuartel', 'Producción');
    const soldiers = mine.filter((u) => u.cls === 'inf' && u.def.weapon);
    const inQueue = bar.queue.filter((q) => UNITS[q.type].cls === 'inf' && UNITS[q.type].weapon).length;
    if (soldiers.length + inQueue < 3) {
      if (!bar.built) return { world: at(bar), text: 'El Constructor va a la obra: espera a que termine el Cuartel' };
      if (!isSel(bar)) return { sel: '[data-act="train"][data-type="rifle"]', text: `Toca tu Cuartel' };
      return { sel: '[data-act="train"][data-type="rifle"]', text: `Entrena Artilleros (tienes ${soldiers.length +
inQueue} de 3)` };
    }
    // 3 Atacá: Puerto, Barcaza, subir a los soldados, cruzar y tomar la isla del bot.
    const foe = g.players.find((q) => q.id !== 0 && q.alive && q.core && q.core.alive);
    if (!foe) return null;
    const foeComp = g.cells[foe.core.cell].comp;
    const landed = soldiers.filter((u) => !u.inside && g.cells[u.cell].comp === foeComp);
    if (landed.length) {
      if (!sel.some((u) => u.kind === 'unit' && u.def.weapon)) return { sel: '[data-quick="army"]', text: 'Toca
```

```
«Ejército» para elegir a tus soldados' };
      return { world: at(foe.core, 1), text: 'Toca el Núcleo del bot para tomar su isla' };
    }
    const harbor = blds.find((b) => b.type === 'harbor');
    if (!harbor) return buildFlow('prod', 'harbor', 'Puerto', 'Producción');
    const barge = mine.find((u) => u.type === 'barge');
    if (!barge) {
      if (!harbor.built) return { world: at(harbor), text: 'El Constructor va a la obra: espera a que termine el Puerto' };
      if (harbor.queue.some((q) => q.type === 'barge')) return { world: at(harbor), text: 'El Puerto fabrica la Barcaza: espera' };
      if (!isSel(harbor)) return { world: at(harbor), text: 'Toca tu Puerto' };
      return { sel: '[data-act="train"][data-type="barge"]', text: 'Entrena una Barcaza: lleva tropas por el mar' };
    }
    const ashore = soldiers.filter((u) => !u.inside);
    if (soldiers.length < 3) return { world: at(bar), text: `Espera a tus soldados (${soldiers.length} de 3)` };
    if (ashore.length) {
      if (ashore.every((u) => u.state === 'board')) return { world: at(barge, 0.3), text: 'Los soldados suben a la Barcaza: espera' };
      if (!sel.some((u) => u.kind === 'unit' && u.cls === 'inf' && u.def.weapon)) return { sel: '[data-quick="army"]', text: 'Toca «Ejército» para elegir a tus soldados' };
      return { world: at(barge, 0.3), text: 'Toca la Barcaza para subirlos' };
    }
    // Todos a bordo: llevar la Barcaza junto a la isla del bot y desembarcar.
    const c0 = g.cells[barge.cell];
    const nearFoe = g.cells.some((c) => c.comp === foeComp && TERRAIN[c.t].land && g.map.dist(c, c0) <= 2);
    if (!nearFoe) {
      if (barge.path && barge.pathI < barge.path.length) return { world: at(barge, 0.3), text: 'La Barcaza navega hacia la isla del bot: espera' };
      if (!isSel(barge)) return { world: at(barge, 0.3), text: 'Toca tu Barcaza' };
      const dest = this.landingCell(g, foe, foeComp);
      return dest ? { world: { x: dest.x, y: 0.1, z: dest.z }, text: 'Toca el mar junto a la isla del bot para llevarla' } : { text: 'No hay costa libre junto a la isla del bot' };
    }
    if (!isSel(barge)) return { world: at(barge, 0.3), text: 'Toca tu Barcaza' };
    return { sel: '[data-act="unload"]', text: 'Toca «Desembarcar»' };
  }

  // Casilla de mar junto a la isla del bot, lo más cerca posible de su Núcleo.
  landingCell(g, foe, foeComp) {
    if (this._landing && this._landing.g === g && this._landing.pid === foe.id) return this._landing.c;
    const core = g.cells[foe.core.cell];
    const c = g.cells.filter((x) => TERRAIN[x.t].water && !x.bld && x.nb.some((ni) => g.cells[ni].comp === foeComp && TERRAIN[g.cells[ni].t].land))
      .sort((a, b) => g.map.dist(a, core) - g.map.dist(b, core))[0] || null;
    this._landing = { g, pid: foe.id, c };
    return c;
  }

  updateCoach() {
    const app = this.app;
    const g = app.game;
    let el = $('coach');
    const step = g && g.mode.tutorial && !g.over && app.state === 'play' && !app.paused ? this.coachStep(g) : null;
    if (!step) {
      if (el) el.hidden = true;
      return;
    }
    if (!el) {
      el = document.createElement('div');
      el.id = 'coach';
      el.setAttribute('aria-live', 'polite');
      el.innerHTML = '<div class="coach-ring"></div><div class="coach-tip"></div>';
      document.body.appendChild(el);
      this._tipKey = null;
    }
    let rect = null;
    let text = step.text;
    if (step.sel) {
      const t = document.querySelector(step.sel);
      if (t && t.getClientRects().length > 0) {
        const r = t.getBoundingClientRect();
        rect = { x: r.left - 6, y: r.top - 6, w: r.width + 12, h: r.height + 12 };
      }
    } else if (step.world && app.renderer) {
      const sp = app.renderer.project(step.world.x, step.world.y, step.world.z);
      const onScreen = sp.ok && sp.x > 24 && sp.x < window.innerWidth - 24 && sp.y > 24 && sp.y < window.innerHeight - 24;
      // ¿Lo tapa un panel? (por ejemplo, el de «Construir»): tocar ahí tocaría el panel, no el mapa.
      const top = onScreen ? document.elementFromPoint(sp.x, sp.y) : null;
      const covered = onScreen && !!top && top.id !== 'gl' && top.id !== 'overlay';
      if (onScreen && !covered) rect = { x: sp.x - 30, y: sp.y - 30, w: 60, h: 60, round: true };
      else {
        // Fuera de la pantalla: el aro señala ese lugar en el minimapa (tocarlo lleva la cámara allí).
        const mm = $('minimap');
        if (mm && mm.getClientRects().length > 0) {
          const r = mm.getBoundingClientRect();
          const m = app.renderer.worldToMinimap(mm, step.world.x, step.world.z);
```

```
          rect = { x: r.left + clamp(m.x, 0, r.width) - 16, y: r.top + clamp(m.y, 0, r.height) - 16, w: 32, h: 32, round: true };
          text = covered ? 'Lo tapa un panel: toca este punto del minimapa para centrarlo'
            : 'Está fuera de la pantalla: toca este punto del minimapa para ir allí';
        }
      }
    }
    el.hidden = false;
    const ring = el.querySelector('.coach-ring');
    const tip = el.querySelector('.coach-tip');
    ring.hidden = !rect;
    if (rect) Object.assign(ring.style, { left: `${rect.x}px`, top: `${rect.y}px`, width: `${rect.w}px`, height: `${rect.h}px`, borderRadius: rect.round ? '50%' : '12px' });
    // v1.4: se mide el texto de verdad (con «Letra grande» es más ancho y más alto).
    const tipKey = text + document.body.classList.contains('bigtext');
    if (this._tipKey !== tipKey || tip.textContent !== text) {
      tip.textContent = text;
      this._tipKey = tipKey;
      this._tipSize = { w: tip.offsetWidth || 220, h: tip.offsetHeight || 44 };
    }
    const { w: tw, h: th } = this._tipSize;
    const W = window.innerWidth;
    const Hh = window.innerHeight;
    const tx = rect ? clamp(rect.x + rect.w / 2 - tw / 2, 8, W - tw - 8) : W / 2 - tw / 2;
    const below = rect ? rect.y + rect.h + 8 : 120;
    const ty = clamp(rect && below + th + 8 > Hh ? rect.y - th - 8 : below, 8, Math.max(8, Hh - th - 8));
    Object.assign(tip.style, { left: `${tx}px`, top: `${ty}px` });
    this.coachText = text;
  }

  // ---------- rueda de selección del ejército (v1.2) ----------
  wheelOpen() {
    return !!this.wheel;
  }

  showWheel(x, y) {
    this.hideWheel();
    const app = this.app;
    const g = app.game;
    if (!g || app.state !== 'play') return;
    const count = (f) => g.units.filter((u) => u.owner === 0 && u.alive && !u.inside && f(u)).length;
    const opts = [
      { k: 'army', label: 'Tierra', n: count((u) => u.def.weapon && u.mover === 'land') },
      { k: 'sea', label: 'Mar', n: count((u) => u.def.weapon && u.mover === 'sea') },
      { k: 'air', label: 'Aire', n: count((u) => u.def.weapon && u.mover === 'air') },
      { k: 'all', label: 'Todo', n: count((u) => u.def.weapon) },
      { k: 'builders', label: 'Constructores', n: count((u) => u.def.builds === 'land') },
      { k: 'g1', label: 'Grupo 1', n: (app.groups[1] || []).filter((u) => u.alive).length },
      { k: 'g2', label: 'Grupo 2', n: (app.groups[2] || []).filter((u) => u.alive).length },
      { k: 'g3', label: 'Grupo 3', n: (app.groups[3] || []).filter((u) => u.alive).length },
    ];
    // Radio para que las 8 opciones (84 px de ancho) no se pisen entre sí.
    const R = 122;
    const cx = clamp(x, R + 50, window.innerWidth - R - 50);
    const cy = clamp(y, R + 40, window.innerHeight - R - 40);
    const el = document.createElement('div');
    el.id = 'wheel';
    el.setAttribute('role', 'menu');
    el.setAttribute('aria-label', 'Rueda de selección');
    el.innerHTML = `<div class="wheel-hub" style="left:${cx}px;top:${cy}px">Elegir</div>` + opts.map((o, i) => {
      const a = -Math.PI / 2 + (i / opts.length) * Math.PI * 2;
      return `<button type="button" role="menuitem" class="wheel-opt" data-wk="${o.k}" style="left:${cx + Math.cos(a) * R}px;top:${cy + Math.sin(a) * R}px" ${o.n ? '' : 'disabled'}>
        <b>${o.label}</b><small>${o.n}</small></button>`;
    }).join('');
    el.addEventListener('click', (e) => {
      const b = e.target.closest('[data-wk]');
      if (b && !b.disabled) this.pickWheel(b.dataset.wk);
      else if (!e.target.closest('.wheel-opt')) this.hideWheel();
    });
    document.body.appendChild(el);
    this.wheel = el;
    app.sound.play('click');
  }

  hoverWheel(x, y) {
    if (!this.wheel) return;
    const hit = document.elementFromPoint(x, y);
    this.wheel.querySelectorAll('.wheel-opt').forEach((b) => b.classList.toggle('hot', b === (hit && hit.closest('.wheel-opt'))));
  }

  releaseWheel(x, y) {
    if (!this.wheel) return;
    const hit = document.elementFromPoint(x, y);
    const b = hit && hit.closest('.wheel-opt');
    if (b && !b.disabled) this.pickWheel(b.dataset.wk);
  }
```

```
  pickWheel(k) {
    const app = this.app;
    this.hideWheel();
    if (k === 'builders') app.selectBuilders();
    else if (k[0] === 'g') app.selectGroup(Number(k[1]));
    else app.quickSelect(k);
  }

  hideWheel() {
    if (this.wheel) this.wheel.remove();
    this.wheel = null;
  }

  // ---------- alianzas y órdenes a los aliados (v1.1) ----------
  allyAction(kind, pid) {
    const app = this.app;
    const g = app.game;
    if (!g) return;
    if (kind === 'ask') {
      const res = g.proposeAlliance(0, pid);
      this.toast(res.text, res.ok ? 'good' : 'bad');
      app.sound.play(res.ok ? 'ready' : 'error');
    } else if (kind === 'break') {
      if (this.breakArm !== pid) {
        this.breakArm = pid;
        this.toast(`¿Romper la alianza con ${g.players[pid].name}? Toca «Romper» otra vez`, 'bad');
        setTimeout(() => { if (this.breakArm === pid) this.breakArm = null; }, 4000);
        return;
      }
      this.breakArm = null;
      const err = g.breakAlliance(0);
      if (err) this.toast(err, 'bad');
      else {
        g.say(pid, 'Rompiste la alianza. Ahora somos rivales otra vez.', { force: true, toMe: true });
        this.toast(`Ya no eres aliado de ${g.players[pid].name}`, 'info');
      }
    } else if (kind === 'help' || kind === 'attack') {
      this.allyOrder(kind);
      return;
    }
    this.renderPop();
  }

  // ---------- mensajes (v1.2): pestañas Todos / Aliados y mensajes rápidos ----------
  chatHtml(g) {
    const tab = this.chatTab || 'all';
    const me = g.players[0];
    const list = g.chat.filter((m) => (tab === 'ally' ? m.ally : (m.ally || m.toMe || m.public))).slice(-30);
    const rows = list.length ? list.map((m) => {
      const p = g.players[m.pid];
      const who = m.mine ? 'Tú' : p.name + (m.ally ? ' (aliado)' : '');
      return `<p class="${m.mine ? 'mine' : ''}"><i style="background:${hexColor(p.color)}"></i><b>${esc(who)}:</b>
${esc(m.text)}<small>${fmtTime(m.time)}</small></p>`;
    }).join('') : `<p>${tab === 'ally' ? 'Todavía no hay mensajes de aliados.' : 'Todavía no hay mensajes.'}</p>`;
    const hasAllies = g.players.some((q) => q.id !== 0 && q.alive && !q.human && q.team === me.team) && me.alive;
    const quick = hasAllies ? `<div class="quickmsgs">${['¡Gracias!', '¡Bien hecho!', '¡Cuidado!'].map((t) => `<button
class="btn mini" type="button" data-qmsg="${t}">${t}</button>`).join('')}
      <button class="btn mini gold" type="button" data-ally="help" data-pid="0">¡Ayuda!</button>
      <button class="btn mini gold" type="button" data-ally="attack" data-pid="0">¡Ataquemos!</button></div>`
      : '<p class="allyhint">Cuando tengas aliados, aquí puedes mandarles mensajes rápidos.</p>';
    return `<header><h4>Mensajes</h4><button class="btn icon" type="button" data-close aria-
label="Cerrar">${ICON.close}</button></header>
      <div class="chattabs" role="tablist"><button class="btn mini" type="button" role="tab" data-chattab="all" aria-
selected="${tab === 'all'}">Todos</button>
      <button class="btn mini" type="button" role="tab" data-chattab="ally" aria-selected="${tab ===
'ally'}">Aliados</button></div>
      <div class="chatlog" id="chatlog">${rows}</div>${quick}`;
  }

  quickMsg(text) {
    const g = this.app.game;
    if (!g) return;
    const n = g.playerSay(text);
    this.chatKey = null;
    this.renderPop();
    this.app.sound.play('click');
    if (!n) this.toast('No tienes aliados que te lean', 'bad');
    const log = $('chatlog');
    if (log) log.scrollTop = log.scrollHeight;
  }

  allyOrder(kind) {
    const app = this.app;
    const g = app.game;
    const bots = g.bots.filter((b) => b.p.alive && b.p.team === g.players[0].team && b.pid !== 0);
    if (!bots.length) return;
    if (kind === 'help') {
```

```
      const n = bots.reduce((a, b) => a + b.answerHelp(0), 0);
      this.toast(n ? `Tus aliados mandan ${n} tropas a tu base` : 'Tus aliados no tienen tropas libres', n ? 'good' :
'bad');
      app.sound.play('order');
    } else {
      this.togglePop(null);
      app.input.startPlacement({ kind: 'allyattack' });
    }
  }

  // ---------- desglose de economía y marcador ----------
  togglePop(which, sub) {
    const same = this.pop === which && this.popSub === sub;
    this.pop = same ? null : which;
    this.popSub = sub;
    $('econ').hidden = this.pop !== 'econ';
    $('score').hidden = this.pop !== 'score';
    $('chatpop').hidden = this.pop !== 'chat';
    this.renderPop();
  }

  renderPop() {
    const g = this.app.game;
    if (!g || !this.pop) return;
    if (this.pop === 'econ') $('econ').innerHTML = this.popSub === 'energy' ? this.energyHtml(g) : this.econHtml(g);
    else if (this.pop === 'chat') {
      // Solo se vuelve a dibujar si llegó un mensaje nuevo (para no cortar el desplazamiento).
      const key = `${g.chat.length}:${g.chat.length ? g.chat[g.chat.length - 1].time : 0}:${this.chatTab}`;
      if (key !== this.chatKey) { this.chatKey = key; $('chatpop').innerHTML = this.chatHtml(g); }
    } else $('score').innerHTML = this.scoreHtml(g);
  }

  econHtml(g) {
    const p = g.players[0];
    const rows = [];
    const groups = new Map();
    for (const b of g.buildings) {
      if (!b.alive || b.owner !== 0 || !b.built || !b.def.income) continue;
      const k = groups.get(b.type) || { n: 0, sum: 0 };
      k.n++;
      k.sum += b.def.income;
      groups.set(b.type, k);
    }
    for (const [type, k] of groups) rows.push([`${BUILDINGS[type].name}${k.n > 1 ? ` ×${k.n}` : ''}`, k.sum]);
    const flags = g.points.filter((pt) => pt.owner === 0 && pt.def.income);
    if (flags.length) rows.push([`Banderas ×${flags.length}`, flags.reduce((s, pt) => s + pt.def.income, 0)]);
    rows.sort((a, b) => b[1] - a[1]);
    const bonus = p.income - p.baseIncome;
    let html = '<header><h4>Monedas por minuto</h4><button class="btn icon" type="button" data-close aria-
label="Cerrar">' + ICON.close + '</button></header><table><tbody>';
    html += rows.map(([n, v]) => `<tr><td>${esc(n)}</td><td class="n">+${fmt(v)}</td></tr>`).join('');
    if (bonus > 0) {
      html += `<tr><td>Subtotal</td><td class="n">${fmt(p.baseIncome)}</td></tr><tr><td>Faro (+${Math.round(p.bonus *
100)} %)</td><td class="n">+${fmt(bonus)}</td></tr>`;
    }
    html += `<tr class="total"><td>Total</td><td class="n" id="econ-total">+${fmt(p.income)}</td></tr></tbody>
</table>`;
    if (!groups.has('farm') && !groups.has('mine')) html += '<p class="hint-press" style="margin:0">Construye Granjas y
Minas para ganar más.</p>';
    return html;
  }

  energyHtml(g) {
    const p = g.players[0];
    const groups = new Map();
    for (const b of g.buildings) {
      if (!b.alive || b.owner !== 0 || !b.built || !b.def.energy) continue;
      const k = groups.get(b.type) || { n: 0, sum: 0 };
      k.n++;
      k.sum += g.bEnergy(b);
      groups.set(b.type, k);
    }
    let html = '<header><h4>Energía</h4><button class="btn icon" type="button" data-close aria-label="Cerrar">' +
ICON.close + '</button></header><table><tbody>';
    if (!groups.size) html += '<tr><td>Sin energías</td><td class="n">+0 / 10 s</td></tr>';
    for (const [type, k] of groups) html += `<tr><td>${esc(BUILDINGS[type].name)} ×${k.n}</td><td class="n">+${k.sum} /
10 s</td></tr>`;
    html += `<tr class="total"><td>Disponible</td><td class="n">${fmt(p.energy)}</td></tr></tbody></table>`;
    html += '<p class="hint-press" style="margin:0">La producen Solar, Eólica, Hidráulica, Mareomotriz y Undimotriz. La
gastan vehículos, buques de guerra, helicópteros, la Fábrica y el Helipuerto.</p>';
    return html;
  }

  scoreHtml(g) {
    const land = g.cells.filter((c) => TERRAIN[c.t].land).length || 1;
    const terr = new Map();
    for (const c of g.cells) if (c.owner >= 0 && TERRAIN[c.t].land) terr.set(c.owner, (terr.get(c.owner) || 0) + 1);
    const myTeam = g.players[0].team;
```

```
    const rows = g.players.map((p) => {
      const army = g.units.filter((u) => u.alive && u.owner === p.id && u.def.weapon).length;
      const blds = g.buildings.filter((b) => b.alive && b.owner === p.id).length;
      const pts = g.points.filter((pt) => pt.owner === p.id).length;
      const role = p.human ? '—' : PERSONALITIES[p.persona].name;
      const rel = p.id === 0 ? '' : p.team === myTeam ? ' · aliado' : '';
      // v1.1: alianzas en los modos de todos contra todos.
      const canAlly = g.alliancesAllowed() && p.id !== 0 && p.alive && g.players[0].alive;
      const allyBtn = !canAlly ? '' : p.team === myTeam
        ? `<button class="btn mini danger" type="button" data-ally="break" data-pid="${p.id}">Romper</button>`
        : `<button class="btn mini" type="button" data-ally="ask" data-pid="${p.id}">Aliarse</button>`;
      return `<tr class="${p.alive ? '' : 'dead'}" data-pid="${p.id}"><td><i class="dot"
style="background:${hexColor(p.color)}"></i>${esc(p.name)}</td>
        <td>${esc(role)}${rel}${allyBtn}</td><td class="n">${Math.round(((terr.get(p.id) || 0) / land) * 100)} %</td>
        <td class="n">${army}</td><td class="n">${blds}</td><td class="n">${p.alive ? '+' + fmt(p.income) : '—'}</td>
<td class="n">${pts}</td></tr>`;
    }).join('');
    const hasAllies = g.players.some((q) => q.id !== 0 && q.alive && !q.human && q.team === myTeam) &&
g.players[0].alive;
    const orders = hasAllies ? `<div class="allyorders"><span>A tus aliados:</span>
      <button class="btn mini gold" type="button" data-ally="help" data-pid="0" id="btn-ally-help">¡Ayuda!</button>
      <button class="btn mini gold" type="button" data-ally="attack" data-pid="0" id="btn-ally-attack">¡Ataquemos!
</button></div>` : '';
    const hint = g.alliancesAllowed() ? '<p class="allyhint">Toca «Aliarse» para proponer una alianza: los aliados no
se atacan y comparten lo que ven. Al final solo puede ganar uno.</p>' : '';
    return `<header><h4>Marcador · ${fmtTime(g.time)}</h4><span class="row" style="gap:6px"><button class="btn mini"
type="button" data-openchat id="btn-open-chat">Mensajes</button><button class="btn icon" type="button" data-close aria-
label="Cerrar">${ICON.close}</button></span></header>${orders}${hint}
      <div class="wrap"><table><thead><tr><th>Imperio</th><th>Estilo</th><th>Territorio</th><th>Ejército</th>
<th>Edificios</th><th>Monedas/min</th><th>Puntos</th></tr></thead>
      <tbody>${rows}</tbody></table></div>`;
  }

  // ---------- Aviones y Aeropuerto (v1.0) ----------
  planeState(u) {
    if (u.repairing) return 'Vuelve a repararse';
    if (u.autoTarget && u.autoTarget.alive) return 'En combate';
    if (u.moving) return 'En vuelo';
    return 'Patrulla en círculo';
  }

  planeHtml(planes) {
    const g = this.app.game;
    const rows = [];
    for (const type of ['fighter', 'bomber']) {
      const list = planes.filter((u) => u.type === type);
      if (!list.length) continue;
      const d = UNITS[type];
      const w = d.weapon;
      const u = list[0];
      const hp = Math.round((list.reduce((a, x) => a + x.hp / x.maxHp, 0) / list.length) * 100);
      const cells = [`<span>Estado</span><b>${esc(this.planeState(u))}</b>`, `<span>Vida</span><b>${hp} %</b>`];
      if (type === 'fighter') {
        cells.push(`<span>Misiles aire-aire</span><b>${w.dmg} · ×${w.mult.air} a aviones</b>`, `<span>Alcance</span>
<b>${w.range} casillas</b>`);
      } else {
        const cd = Math.max(0, u.cd || 0);
        const seen = g.time - (u.lastShot || -99) < 3;
        cells.push(`<span>Bombas</span><b class="${cd > 0.2 ? 'radar-track' : 'radar-idle'}">${cd > 0.2 ? `Recargando
${Math.ceil(cd)} s` : 'Listas'}</b>`,
          `<span>Furtivo</span><b class="${seen ? 'radar-lock' : ''}">${seen ? 'Visible (acaba de soltar bombas)' :
'Oculto: solo se ve a 3 casillas'}</b>`);
      }
      rows.push(`<div class="planecard"><img src="${Thumbs.get(type, PLAYER_COLORS[0])}" alt="">
        <div><div class="radar-status"><b>${esc(d.name)}</b>${list.length > 1 ? ` ×${list.length}` : ''}</div><div
class="numbers">${cells.join('')}</div></div></div>`);
    }
    const tip = planes.some((u) => u.type === 'bomber')
      ? '«Bombardear zona»: toca un lugar y los Bombarderos van atacando los edificios que encuentren.'
      : 'Los Cazas buscan solos aviones y helicópteros enemigos; con poca vida vuelven al Aeropuerto o al Helipuerto.';
    return rows.join('') + `<div class="queue">${tip}</div>`;
  }

  airportHtml(b) {
    const g = this.app.game;
    const heal = g.healOf(b);
    const R = heal.range * SQ3 + 0.6;
    const near = g.units.filter((u) => u.alive && !u.inside && u.owner === b.owner && u.cls === 'air' && Math.hypot(u.x
- b.x, u.z - b.z) <= R);
    const mine = g.units.filter((u) => u.alive && u.owner === b.owner && u.def.plane);
    const nF = mine.filter((u) => u.type === 'fighter').length;
    const nB = mine.filter((u) => u.type === 'bomber').length;
    const pads = near.length
      ? near.map((u) => `<span class="padslot"><img src="${Thumbs.get(u.type, PLAYER_COLORS[u.owner])}" alt="">
<b>${Math.round((u.hp / u.maxHp) * 100)} %</b></span>`).join('')
      : '<span class="queue">Ningún avión en la pista ahora.</span>';
    return `<div class="radar airport"><div class="radar-scope" aria-hidden="true"><i></i></div><div class="radar-
body">
```

```
      <div class="radar-status">Torre de control · <b>${nF} ${nF === 1 ? 'Caza' : 'Cazas'} · ${nB} ${nB === 1 ? 'Bombardero' : 'Bombarderos'}</b></div>
      <div class="pads">${pads}</div>
      <div class="queue">Repara ${Math.round(heal.rate)} de vida por segundo a los aviones y helicópteros cercanos; los dañados vuelven solos aquí.</div></div></div>`;
  }

  // ---------- AIR DEFENSE y Torreta (v0.9): alcance, recarga, contra qué sirve y qué sigue el radar ----------
  radarHtml(b) {
    const g = this.app.game;
    const w = g.weaponOf(b);
    const air = (w.mult.air || 0) > 0;
    const rows = [`<span>Alcance</span><b>${w.range} casillas</b>`,
      `<span>Recarga</span><b>${w.cd.toLocaleString('es-ES')} s${w.salvo ? ` · ráfaga de ${w.salvo}` : ''}</b>`,
      `<span>${w.salvo ? 'Daño a aeronaves por ráfaga' : 'Daño a aeronaves por tiro'}</span><b>${air ? fmt(w.dmg * w.mult.air * (w.salvo || 1)) : 'no'}</b>`];
    if (b.type === 'aadef' || (b.def.aaLevel && b.level >= b.def.aaLevel)) rows.push('<span>Detecta furtivos</span><b>a 3 casillas</b>');
    const t = b.lockOn && b.lockOn.alive && g.inRange(b, w, b.lockOn) ? b.lockOn : b.autoTarget && b.autoTarget.alive ? b.autoTarget : null;
    let status = '<b class="radar-idle">Sin blancos</b>';
    if (t) {
      const name = t.kind === 'bld' ? t.def.name : UNITS[t.type].name;
      const locking = w.lock && b.lockOn === t && b.lockT > 0;
      status = `<b class="${locking ? 'radar-track' : 'radar-lock'}">${locking ? 'Siguiendo' : 'Fijado'}: ${esc(name)}</b>`;
    }
    return `<div class="radar"><div class="radar-scope" aria-hidden="true"><i></i></div>
      <div class="radar-body"><div class="radar-status">Radar · ${status}</div><div class="numbers">${rows.join('')}</div></div></div>`;
  }

  // ---------- ficha de unidad ----------
  // v1.5: «la próxima en 3 s · todo en 12 s» (o por qué no avanza).
  queueTimeText(b) {
    const g = this.app.game;
    if (!b.queue.length) return '';
    const rate = g.prodRate(b);
    if (!(rate > 0)) return 'parada: falta energía';
    const q = b.queue[0];
    const first = Math.max(0, (UNITS[q.type].build - q.t) / rate);
    let total = first;
    for (let i = 1; i < b.queue.length; i++) total += UNITS[b.queue[i].type].build / rate;
    const f = (x) => (x < 60 ? `${Math.ceil(x)} s` : fmtTime(x));
    return b.queue.length > 1 ? `la próxima en ${f(first)} · todas en ${f(total)}` : `lista en ${f(first)}`;
  }

  // v1.5: ficha de una tropa: vida, daño, alcance, velocidad y contra qué es buena o mala.
  unitTilesHtml(u) {
    const d = u.def;
    const w = d.weapon;
    const tiles = [['Vida máxima', fmt(u.maxHp), u.vet ? 'gold' : '']];
    if (w) {
      const mult = 1 + (u.vet || 0) * VETERAN.dmg;
      tiles.push(['Daño/s', fmt((w.dmg * mult) / w.cd), u.vet ? 'gold' : '']);
      tiles.push(['Alcance', `${w.range} casillas`, '']);
    }
    tiles.push(['Velocidad', d.speed.toLocaleString('es-ES'), '']);
    let html = `<div class="coretiles unittiles">${tiles.map(([k, v, c]) => `<div class="ctile ${c}"><small>${k}</small><b>${v}</b></div>`).join('')}</div>`;
    if (w) {
      const cls = ['inf', 'veh', 'ship', 'sub', 'air', 'bld'];
      const strong = cls.filter((c) => (w.mult[c] || 0) >= 1.2).map((c) => CAT_NAMES[c]);
      const weak = cls.filter((c) => w.mult[c] > 0 && w.mult[c] <= 0.5).map((c) => CAT_NAMES[c]);
      const none = cls.filter((c) => !(w.mult[c] > 0)).map((c) => CAT_NAMES[c]);
      const line = [];
      if (strong.length) line.push(`<span class="vs good">Fuerte contra: <b>${strong.join(', ')}</b></span>`);
      if (weak.length) line.push(`<span class="vs bad">Débil contra: <b>${weak.join(', ')}</b></span>`);
      if (none.length) line.push(`<span class="vs muted">No ataca a: <b>${none.join(', ')}</b></span>`);
      if (line.length) html += `<div class="vsline">${line.join('')}</div>`;
    }
    return html;
  }

  statsHtml(type) {
    const u = UNITS[type];
    const w = u.weapon;
    const parts = [`<div class="title">${esc(u.name)}</div>`];
    parts.push(`<div><span>Vida</span> <b>${fmt(u.hp)}</b></div>`);
    if (w) {
      parts.push(`<div><span>Daño/s</span> <b>${fmt(w.dmg / w.cd)}</b></div>`);
      parts.push(`<div><span>Alcance</span> <b>${w.range}</b> casillas</div>`);
    }
    parts.push(`<div><span>Velocidad</span> <b>${u.speed.toLocaleString('es-ES')}</b></div>`);
    parts.push(`<div><span>Población</span> <b>${u.pop}</b>${u.energy ? ` · <span>Energía</span> <b>${u.energy}</b>` : ''}</div>`);
    if (w) {
      const cls = ['inf', 'veh', 'ship', 'sub', 'air', 'bld'];
```

```
      const strong = cls.filter((c) => (w.mult[c] || 0) >= 1.3).map((c) => CAT_NAMES[c]);
      const none = cls.filter((c) => !(w.mult[c] > 0)).map((c) => CAT_NAMES[c]);
      if (strong.length) parts.push(`<div class="wide">Fuerte contra: <b>${strong.join(', ')}</b></div>`);
      if (none.length) parts.push(`<div class="wide">No ataca a: <b>${none.join(', ')}</b></div>`);
    } else {
      parts.push(`<div class="wide">No combate. ${esc(u.desc)}</div>`);
    }
    return parts.join('');
  }

  showCardInfo(type) {
    this.cardInfoType = type;
    const el = $('cardinfo');
    if (!el) return;
    el.innerHTML = this.statsHtml(type);
    el.hidden = false;
  }

  showBox(x1, y1, x2, y2) {
    const el = $('boxsel');
    el.hidden = false;
    el.style.left = Math.min(x1, x2) + 'px';
    el.style.top = Math.min(y1, y2) + 'px';
    el.style.width = Math.abs(x2 - x1) + 'px';
    el.style.height = Math.abs(y2 - y1) + 'px';
  }

  hideBox() {
    $('boxsel').hidden = true;
  }

  showPlaceBar(p) {
    let text = '';
    const isBuild = p.kind === 'build';
    $('btn-place-rot').hidden = !isBuild;
    $('btn-place-ok').hidden = !isBuild;
    $('btn-place-cancel').hidden = false;
    $('btn-place-cancel').textContent = isBuild && p.placed ? 'Terminar' : 'Cancelar';
    if (isBuild) {
      // v1.0: tocar una casilla muestra la obra; «Construir» la pone; «Terminar» sale.
      const d = BUILDINGS[p.btype];
      const price = `<span class="cost">${fmt(d.cost)}</span>${d.energyCost ? ` <span
class="ecost">${ICON.bolt}${d.energyCost}</span>` : ''}`;
      const placed = p.placed ? ` · <b>${p.placed}</b> ${p.placed === 1 ? 'puesta' : 'puestas'}` : '';
      if (p.err && p.cell >= 0) text = `<b>${esc(d.name)}</b> · <span class="err">${esc(p.err)}</span>${placed}`;
      else if (p.cell >= 0) text = `<b>${esc(d.name)}</b> · ${price} · Toca «Construir» para poner la obra
aquí${placed}`;
      else if (p.valid && p.valid.size) text = `<b>${esc(d.name)}</b> · ${price} · Toca una casilla dorada y luego
«Construir»${placed}`;
      else text = `No quedan casillas para ${esc(d.name)} en tu territorio${placed}. ${esc(d.desc)}`;
      $('btn-place-ok').disabled = p.cell < 0 || !!p.err;
      $('btn-place-rot').disabled = false;
    } else if (p.kind === 'found') {
      text = p.valid && p.valid.size
        ? 'Toca dónde fundar el <b>Puesto avanzado</b> · <span class="cost">800</span> al llegar'
        : 'No hay sitio alcanzable para fundar un puesto';
    } else if (p.kind === 'rally') {
      text = 'Toca el <b>punto de reunión</b> para las tropas nuevas';
    } else if (p.kind === 'nuke') {
      text = `Toca el blanco del <b>misil nuclear</b> · <span class="cost">${fmt(NUKE.cost)}</span> · llega en
${NUKE.flight} s`;
    } else if (p.kind === 'allyattack') {
      text = 'Toca un edificio, una tropa o el territorio del <b>rival</b> que quieres atacar con tus aliados';
    } else if (p.kind === 'bombzone') {
      text = `Toca la <b>zona a bombardear</b>: ${p.units.length} ${p.units.length === 1 ? 'Bombardero va' :
'Bombarderos van'} atacando los edificios que encuentren`;
    } else if (p.kind === 'nuke3') {
      text = `Toca el blanco de los <b>3 misiles nucleares</b> · <span class="cost">${fmt(NUKE.cost * 3)}</span> ·
llegan en ${NUKE.flight} s`;
    }
    $('placetext').innerHTML = text;
    $('placebar').hidden = false;
    $('selpanel').hidden = true;
  }

  hidePlaceBar() {
    $('placebar').hidden = true;
    this.selSig = '';
  }

  // ---------- panel de construcción ----------
  renderBuildList() {
    const tab = BUILD_TABS.find((t) => t.id === this.tab);
    $('blist').innerHTML = tab.items.map((k) => {
      const d = BUILDINGS[k];
      return `<button class="bitem" type="button" data-b="${k}">
        <img src="${Thumbs.get(k, PLAYER_COLORS[0])}" alt="">
        <div class="top"><b>${esc(d.name)}</b><span class="costs"><span class="cost">${fmt(d.cost)}
```

```
</span>${d.energyCost ? `<span class="ecost">${ICON.bolt}${d.energyCost}</span>` : ''}</span></div>
        <div class="desc">${esc(d.desc)}<span data-count="${k}"></span></div>
      </button>`;
    }).join('');
    this.buildSig = '';
    this.refreshBuild(true);
  }

  refreshBuild(force) {
    const g = this.app.game;
    if (!g) return;
    const p = g.players[0];
    const placing = this.app.input.placement;
    const items = $('blist').querySelectorAll('[data-b]');
    let sig = '';
    const states = [];
    items.forEach((el) => {
      const k = el.dataset.b;
      const d = BUILDINGS[k];
      const n = g.count(0, k);
      const poor = p.coins < d.cost;
      const poorE = !!d.energyCost && p.energy < d.energyCost;
      const limit = g.limitFor(0, k);
      const maxed = limit && n >= limit;
      const act = placing && placing.kind === 'build' && placing.btype === k;
      states.push([el, k, d, n, poor, maxed, act, poorE, limit]);
      sig += `${k}${n}${poor}${maxed}${act}${poorE}${limit}|`;
    });
    if (!force && sig === this.buildSig) return;
    this.buildSig = sig;
    for (const [el, k, d, n, poor, maxed, act, poorE, limit] of states) {
      el.classList.toggle('poor', poor);
      el.classList.toggle('poorE', poorE);
      el.classList.toggle('maxed', !!maxed);
      el.classList.toggle('active', !!act);
      const cnt = el.querySelector(`[data-count="${k}"]`);
      if (cnt) cnt.textContent = limit ? ` (${n}/${limit})` : n ? ` (${n})` : '';
    }
  }

  // ---------- panel de selección ----------
  renderSel(force) {
    const app = this.app;
    const g = app.game;
    const panel = $('selpanel');
    const sel = app.selection.filter((e) => e.alive);
    if (!sel.length || app.input.placement) {
      panel.hidden = true;
      this.selSig = '';
      return;
    }
    const p = g.players[0];
    let sig = sel.map((e) => e.id).join(',');
    const first = sel[0];
    if (first.kind === 'point') {
      sig += `|${first.owner}|${first.contested}|${Math.round(first.prog * 20)}`;
    } else if (first.kind === 'bld') {
      sig += `|${first.built}|${first.queue.map((q) => q.type).join('.')}|${first.rally >=
0}|${this.demolishArm}|${first.level}` +
        `|${first.upgrading ? first.upgrading.level + ':' + Math.floor(first.upgrading.progress * 20) :
''}|${first.built ? '' : Math.floor(first.progress * 20)}` +
        `|${g.time - first.lastWork > 1.5}|${first.lastType || ''}|${first.loop ? 1 : 0}${first.loopWait || ''}`;
      if (first.def.produces) {
        sig += '|' + first.def.produces.map((t) => p.coins >= UNITS[t].cost && p.energy >= (UNITS[t].energy ||
0)).join('');
      }
      const uc = g.upgradeCost(first);
      if (uc) sig += '|u' + (p.coins >= uc);
      if (first.type === 'airport') sig += '|p' + g.units.filter((u) => u.alive && u.owner === 0 && u.cls === 'air' &&
Math.hypot(u.x - first.x, u.z - first.z) < 4.5).map((u) => u.id + ':' + Math.floor((u.hp / u.maxHp) * 10)).join(',') +
g.units.filter((u) => u.def.plane && u.owner === 0).length;
      if (first.type === 'aadef' || first.type === 'turret') sig += `|r${first.lockOn && first.lockOn.alive ?
first.lockOn.id + (first.lockT > 0 ? 't' : 'l') : first.autoTarget && first.autoTarget.alive ? 'a' + first.autoTarget.id :
'-'}`;
      if (first.type === 'aadef' || first.type === 'antinuke') sig += `|n${Math.ceil(Math.max(0, (first.nukeReadyAt ||
first.chargeAt || 0) - g.time))}${p.coins >= NUKE.cost}${first.type === 'aadef' && first.level >= 4 ? g.salvoInvalid(0) :
''}`;
      if (first.type === 'core' && first.owner === 0) {
        const d = p.dome;
        sig += `|d${d ? d.level + ':' + (d.hp > 0 ? Math.ceil((d.hp / d.maxHp) * 10) : 'x') : ''}${p.coins >=
g.domeCost(0)}|e${p.emergencyUsed}${first.shieldUntil > g.time}`;
        if (first.level >= AUTO_UP.coreLevel) {
          const cand = g.upgradeCandidates(0);
          sig += `|a${p.autoUpgrade ? 1 : 0}:${cand.length}:${cand.filter((x) => g.upgradeCost(x) <= p.coins).length}`;
        }
      }
    } else {
      sig += '|' + sel.map((u) => u.state === 'found').join('') + (p.coins >= 800) +
```

```
        sel.map((u) => (u.cargo ? u.cargo.length : '')).join('') + `|${this.app.formation}|${this.app.input.amoveArm}`
+
        `|${sel.map((u) => `${u.patrol ? 'p' : ''}${u.hold ? 'h' : ''}${u.vet || 0}:${u.kills ||
0}`).join(',')}|${g.players[0].autoRepair !== false}`;
      // Aviones (v1.0): estado, bombas y si el bombardero está oculto.
      for (const u of sel) if (u.def.plane) sig += `|${this.planeState(u)}${u.type === 'bomber' ? Math.ceil(Math.max(0,
u.cd || 0)) + (g.time - (u.lastShot || -99) < 3 ? 'v' : 'o') : ''}`;
    }
    if (force || sig !== this.selSig) {
      this.selSig = sig;
      panel.hidden = false;
      panel.classList.toggle('more', !!this.selMore);
      if (first.kind === 'point') panel.innerHTML = this.pointHtml(first);
      else if (first.kind === 'bld') panel.innerHTML = this.buildingHtml(first);
      else if (first.owner === 0) panel.innerHTML = this.unitsHtml(sel);
      else panel.innerHTML = this.inspectHtml(first);
    }
    this.updateSelLive(sel);
  }

  head(type, color, title, sub, hpId = true) {
    return `<div class="selhead">
      <img class="thumb" src="${Thumbs.get(type, color)}" alt="">
      <div style="min-width:0"><h3>${title}</h3><p class="sec">${sub}</p>
        ${hpId ? '<div class="hp" id="sel-hpbar"><i id="sel-hp"></i></div><div class="hptext" id="sel-hptext"></div>' :
''}
      </div>
      <div class="headbtns"><button class="btn seltoggle" type="button" data-act="seldetails" id="btn-seldetails" aria-
expanded="${!!this.selMore}">${this.selMore ? `${bi('less')}Menos` : `${bi('more')}Más detalles`}</button>
      <button class="btn icon" type="button" data-act="close" aria-label="Cerrar">${ICON.close}</button></div>
    </div>`;
  }

  buildingHtml(b) {
    const g = this.app.game;
    const own = b.owner === 0;
    const d = b.def;
    const extra = [];
    const inc = g.bIncome(b);
    if (inc) extra.push(`+${fmt(inc * g.incomeMult(g.players[b.owner]))} monedas/min`);
    if (g.bPop(b)) extra.push(`+${g.bPop(b)} de población`);
    if (d.energy) extra.push(`+${g.bEnergy(b)} energía cada 10 s`);
    if (d.healLevels) extra.push(`${d.healLevels[b.level].name}: cura ${Math.round(d.healLevels[b.level].pct * 100)}
%/s`);
    if (d.minigun) extra.push(b.level > 1 ? `${b.level - 1} miniguns${b.level > 2 ? 's' : ''}` : 'sin minigun');
    const sub = (own ? '' : `<b style="color:${hexColor(PLAYER_COLORS[b.owner])}">${esc(g.players[b.owner].name)}</b> ·
`) +
      esc(d.desc) + (extra.length ? ' · ' + extra.join(' · ') : '');
    const maxLv = d.maxLevel;
    const lv = maxLv > 1
      ? ` <span class="lvtag">Nivel ${b.level}</span><span class="levels" title="Nivel ${b.level} de
${maxLv}">${Array.from({ length: maxLv }, (_, i) => `<i class="${i < b.level ? 'on' : ''}"></i>`).join('')}</span>` : '';
    const mkey = GLB[b.type + '_' + b.level] ? b.type + '_' + b.level : b.type;
    let html = this.head(mkey, PLAYER_COLORS[b.owner], esc(d.levelNames ? d.levelNames[b.level] : d.name) + lv, sub);
    if (!own) return html + (g.isEnemy(b.owner, 0) ? this.counterHtml(b) : '');
    // Obra o mejora en curso.
    if (!b.built || b.upgrading) {
      const k = b.upgrading ? b.upgrading.progress : b.progress;
      const idle = g.time - b.lastWork > 1.5 && b.type !== 'outpost';
      const by = d.builtBy === 'sea' ? 'un Barco constructor' : d.builtBy === 'engineer' ? 'un Ingeniero' : 'un
Constructor o Ingeniero';
      html += `<div class="queue"><b>${b.upgrading ? `Mejorando a nivel ${b.upgrading.level}` : 'En construcción'}</b>
· ${Math.floor(k * 100)} % · ` +
        (idle ? `<span style="color:#ffb4b5">Sin constructor: selecciona ${by} y toca la obra.</span>` : 'Un
constructor está trabajando.') + '</div>';
    }
    if (b.type === 'core') {
      const p = g.players[0];
      const dome = p.dome;
      // v0.9: el estado del Núcleo en casillas claras.
      const L = d.levels[b.level];
      const emer = p.emergencyUsed
        ? (b.shieldUntil > g.time ? ['Activo', 'gold'] : ['Usado', 'muted']) : ['Listo', 'good'];
      const tiles = [
        ['Monedas', `+${fmt(g.bIncome(b) * g.incomeMult(p))}/min`, ''],
        ['Comunidad', `+${g.bPop(b)}`, ''],
        ['Láser', `${L.laser.beams > 1 ? '2 × ' : ''}${L.laser.dmg} · ${L.laser.range} cas.`, ''],
        ['Emergencia', emer[0], emer[1]],
        ['Cúpula', dome ? (dome.hp > 0 ? `Nivel ${dome.level} · ${Math.round((dome.hp / dome.maxHp) * 100)} %`
          : `Rota · ${Math.max(0, Math.ceil(DOME.recover - (g.time - dome.brokenAt)))} s`) : (b.level < 3 ? 'Núcleo
nivel 3' : 'Disponible'),
          dome ? (dome.hp > 0 ? 'good' : 'bad') : 'muted'],
      ];
      html += `<div class="coretiles sec">${tiles.map(([k, v, c]) => `<div class="ctile ${c}"><small>${k}</small>
<b>${v}</b></div>`).join('')}</div>`;
      if (dome && dome.level >= 3 && dome.hp > 0) html += '<div class="queue sec">La Cúpula nivel 3 da escudo a tus
tropas mientras están dentro de ella.</div>';
      const derr = g.domeInvalid(0);
```

```
      // v1.0: con el Núcleo en nivel 3, mejorar todo de un clic o dejar las mejoras automáticas.
      if (b.level >= AUTO_UP.coreLevel) {
        const cand = g.upgradeCandidates(0);
        let total = 0;
        let can = 0;
        let left = p.coins;
        for (const x of cand) {
          const c = g.upgradeCost(x);
          total += c;
          if (c <= left) { left -= c; can++; }
        }
        const auto = !!p.autoUpgrade;
        html += `<div class="upall"><div class="actions">
          <button class="btn gold" type="button" data-act="upall" id="btn-upall" ${can ? '' :
'disabled'}>${bi('upgrade')}Mejorar todo · ${can} de ${cand.length}</button>
          <button class="btn ${auto ? 'good' : ''}" type="button" data-act="autoup" id="btn-autoup" aria-
pressed="${auto}">${bi('repeat')}Mejoras automáticas: ${auto ? 'sí' : 'no'}</button></div>
          <span class="queue sec">${cand.length ? `Quedan ${cand.length} mejoras (${fmt(total)} en total). «Mejorar
todo» pone a mejorar las que alcanza tu dinero; las automáticas siguen hasta el máximo y guardan ${fmt(AUTO_UP.reserve)}
para tropas.` : 'Todo está al máximo.'}</span></div>`;
      }
      if (!dome || dome.level < 3) {
        const next = dome ? dome.level + 1 : 1;
        const label = dome ? `Mejorar Cúpula a nivel ${next}` : 'Comprar Cúpula';
        const why = b.level < 3 ? 'Necesita el Núcleo en nivel 3.' : next === 3 ? 'Nivel 3: escudo personal para todas
tus tropas.'
          : `Absorbe el daño de barcos, aeronaves y Lanzamisiles a ${DOME_LEVELS[next].range} casillas del Núcleo.`;
        html += `<div class="actions"><button class="btn gold" type="button" data-act="dome" ${derr && b.level < 3 ?
'disabled' : ''}>${bi('dome')}${label} · ${fmt(g.domeCost(0))}</button>
          <span class="queue sec" style="flex:1 1 200px">${why}</span></div>`;
      }
    }
    if (d.produces && b.built) {
      const pl = g.players[0];
      html += '<div class="cards">' + d.produces.map((t) => {
        const u = UNITS[t];
        const poor = pl.coins < u.cost || pl.energy < (u.energy || 0);
        return `<button class="card ${poor ? 'poor' : ''}" type="button" data-act="train" data-type="${t}"
title="${esc(u.desc)}">
          <img src="${Thumbs.get(t, PLAYER_COLORS[0])}" alt="">
          <b>${esc(u.name)}</b>
          <div class="meta"><span class="cost">${fmt(u.cost)}</span>${u.energy ? `<span
class="ecost">${ICON.bolt}${u.energy}</span>` : ''}<span>${ICON.pop}${u.pop}</span></div>
        </button>`;
      }).join('') + '</div>';
      if (b.lastType && d.produces.includes(b.lastType)) {
        // Repetir (v0.7): vuelve a poner la última tropa entrenada, una o cinco veces.
        const lu = UNITS[b.lastType];
        html += `<div class="actions repeat"><span class="queue">Última: <b>${esc(lu.name)}</b></span>
          <button class="btn" type="button" data-act="repeat" data-n="1" id="btn-repeat">${bi('repeat')}Repetir ·
${fmt(lu.cost)}</button>
          <button class="btn" type="button" data-act="repeat" data-n="5" id="btn-repeat5">${bi('repeat')}×5 ·
${fmt(lu.cost * 5)}</button>
          <button class="btn ${b.loop ? 'good' : ''}" type="button" data-act="loop" id="btn-loop" aria-
pressed="${!!b.loop}" title="Sigue entrenando la última tropa mientras alcance el dinero y la
comunidad">${bi('repeat')}Siempre: ${b.loop ? 'sí' : 'no'}</button></div>`;
        if (b.loop && b.loopWait && !b.queue.length) html += `<div class="queue" id="loop-wait">Esperando:
${esc(b.loopWait)}</div>`;
      }
      const info = this.cardInfoType && d.produces.includes(this.cardInfoType) ? this.statsHtml(this.cardInfoType) :
'';
      html += `<div class="cardinfo" id="cardinfo" ${info ? '' : 'hidden'}>${info}</div>`;
      // v1.5: la cola dice cuánto falta y cada tropa tiene su «×» para quitarla (devuelve el dinero).
      html += `<div class="queue ${b.queue.length ? '' : 'sec'}" id="sel-queue">` + (b.queue.length
        ? `<span class="qhead">En cola · <b id="sel-qtime">${this.queueTimeText(b)}</b></span>` + b.queue.map((q, i) =>
`<button class="qitem" type="button" data-act="cancel" data-idx="${i}" title="Quitar ${esc(UNITS[q.type].name)} (te devuelve
${fmt(UNITS[q.type].cost)})" aria-label="Quitar ${esc(UNITS[q.type].name)}">
            <img src="${Thumbs.get(q.type, PLAYER_COLORS[0])}" alt=""><i style="width:0"></i><span class="qx" aria-
hidden="true">×</span></button>`).join('') +
          '<span class="qnote">Toca una para quitarla: te devuelve el dinero.</span>'
        : 'Toca una tropa para entrenarla; mantenla pulsada para ver su ficha.') + '</div>';
    }
    if (b.built && d.weapon && (b.type === 'aadef' || b.type === 'turret')) html += this.radarHtml(b);
    if (b.type === 'aadef' && b.built && b.level >= 4) {
      // Misil nuclear (v0.9): muy caro, recarga de 3 min.
      const why = g.nukeInvalid(b);
      const wait = g.time < (b.nukeReadyAt || 0);
      html += `<div class="actions nukebox"><button class="btn danger" type="button" data-act="nuke" id="btn-nuke"
${why ? 'disabled' : ''}>☢ Misil nuclear · ${fmt(NUKE.cost)}</button>
        <span class="queue" style="flex:1 1 200px">${wait ? `Recargando: ${Math.ceil(b.nukeReadyAt - g.time)} s` : why
|| `Listo. Daña todo en ${NUKE.radius} casillas y deja ${NUKE.rad.time} s de radiación; un Escudo antimisiles lo puede
derribar.`}</span></div>`;
      // v1.0: los 3 a la vez (un Escudo antimisiles solo derriba uno por carga).
      const why3 = g.salvoInvalid(0);
      html += `<div class="actions nukebox"><button class="btn danger" type="button" data-act="nuke3" id="btn-nuke3"
${why3 ? 'disabled' : ''}>☢☢☢ Lanzar los 3 · ${fmt(NUKE.cost * 3)}</button>
        <span class="queue" style="flex:1 1 200px">${why3 || 'Tus 3 AIR DEFENSE de nivel 4 disparan juntos al mismo
blanco: un Escudo antimisiles solo puede derribar uno.'}</span></div>`;
```

```
    } else if (b.type === 'aadef' && b.built) {
      html += `<div class="queue">Nivel 4 (solo 3 de tus AIR DEFENSE): lanza el misil nuclear.</div>`;
    }
    if (b.type === 'airport' && b.built) html += this.airportHtml(b);
    if (b.type === 'antinuke' && b.built) {
      const ready = g.time >= (b.chargeAt || 0);
      html += `<div class="queue">${ready ? '<b style="color:var(--good)">Carga lista</b>: derriba un misil nuclear que caiga a 7 casillas.' : `Recargando: ${Math.ceil(b.chargeAt - g.time)} s`}</div>`;
      html += `<div class="queue">Defensa de punto: derriba misiles y cohetes enemigos a ${b.def.pointDefense.range} casillas · derribados: <b>${b.pdKills || 0}</b></div>`;
    }
    const acts = [];
    if (own && b.built && !b.upgrading && b.level < maxLv) {
      // v0.9: antes de pagar, qué vas a ganar con la mejora.
      const prev = g.upgradePreview(b).map((t) => t.replace(/^Ahora es /, 'Pasa a ser '));
      if (prev.length) html += `<div class="nextup"><small>Nivel ${b.level} → ${d.levelNames ? esc(d.levelNames[b.level + 1]) : `nivel ${b.level + 1}`}</small>${prev.map((t) => `<span>${esc(t)}</span>`).join('')}</div>`;
    }
    if (b.built && !b.upgrading && b.level < maxLv) {
      const nextName = d.levelNames ? d.levelNames[b.level + 1] : `nivel ${b.level + 1}`;
      acts.push(`<button class="btn gold" type="button" data-act="upgrade">${bi('upgrade')}Mejorar a ${nextName} · ${fmt(g.upgradeCost(b))}</button>`);
    }
    if (d.produces && b.built) acts.push(`<button class="btn" type="button" data-act="rally">${bi('rally')}${b.rally >= 0 ? 'Mover punto de reunión' : 'Punto de reunión'}</button>`);
    if (b.type !== 'core') {
      acts.push(`<button class="btn danger" type="button" data-act="demolish">${bi('demolish')}${this.demolishArm === b.id ? '¿Seguro? Toca otra vez' : 'Demoler'}</button>`);
    }
    if (acts.length) html += `<div class="actions mainacts">${acts.join('')}</div>`;
    return html;
  }

  unitsHtml(sel) {
    const counts = new Map();
    for (const u of sel) counts.set(u.type, (counts.get(u.type) || 0) + 1);
    const first = sel[0];
    const stars = (u) => (u.vet ? ` <span class="vetstars" title="Veterana: ${u.kills} eliminaciones">${'★'.repeat(u.vet)}</span>` : '');
    const vets = sel.filter((u) => u.vet).length;
    const title = sel.length === 1 ? esc(first.def.name) + stars(first) : `${sel.length} tropas${vets ? ` <span class="vetstars">★ ${vets}</span>` : ''}`;
    const sub = sel.length === 1 ? esc(first.def.desc) + (first.def.weapon ? ` · ${first.kills || 0} eliminaciones${first.vet < 3 ? ` (próxima estrella con ${VETERAN.kills[first.vet || 0]})` : ''}` : '') : 'Toca un destino para moverlas o un enemigo para atacarlo.';
    let html = this.head(first.type, PLAYER_COLORS[0], title, sub);
    if (sel.length === 1) html += this.unitTilesHtml(first);
    if (counts.size > 1) {
      // v1.5: tocar un tipo deja elegidas solo esas tropas.
      html += '<div class="chips">' + [...counts.entries()].map(([t, n]) =>
        `<button class="chip" type="button" data-act="seltype" data-type="${t}" title="Elegir solo ${esc(UNITS[t].name)}"><img src="${Thumbs.get(t, PLAYER_COLORS[0])}" alt="">${n}× ${esc(UNITS[t].name)}</button>`).join('') + '</div>';
    }
    const acts = [];
    // Transportes (Barcaza, Lancha, Jeep, Transporte blindado) y hangar del Portaaviones.
    const ship = sel.length === 1 && (first.def.capacity || first.def.hangar) ? first : null;
    if (ship) {
      const g = this.app.game;
      const hangar = !!ship.def.hangar;
      const cap = ship.def.capacity || ship.def.hangar;
      const how = hangar ? 'Selecciona Cazas y toca el Portaaviones para guardarlos.'
        : ship.def.carries === 'inf' ? `Selecciona soldados y toca el ${ship.def.name} para subirlos.`
          : `Selecciona tropas de tierra y toca la ${ship.def.name} para subirlas.`;
      html += `<div class="queue">${hangar ? 'Hangar' : 'Carga'}: ${g.cargoSlots(ship)}/${cap}` +
        (ship.cargo.length ? ' · ' + ship.cargo.map((u) => `<img class="qitem" style="cursor:default" src="${Thumbs.get(u.type, PLAYER_COLORS[0])}" alt="${esc(u.def.name)}">`).join('')
          : ' · ' + how) + '</div>';
      const label = hangar ? 'Lanzar Cazas' : ship.mover === 'sea' ? 'Desembarcar' : 'Bajar soldados';
      if (ship.cargo.length) acts.push(`<button class="btn gold" type="button" data-act="unload">${bi('unload')}${label}</button>`);
    }
    const planes = sel.filter((u) => u.def.plane);
    if (planes.length) {
      html += this.planeHtml(planes);
      if (planes.some((u) => u.type === 'bomber')) {
        acts.push(`<button class="btn gold" type="button" data-act="bombzone" id="btn-bombzone">${bi('amove')}Bombardear zona</button>`);
      }
    }
    if (sel.some((u) => u.def.builds)) {
      html += `<div class="queue">Construye solo las obras pendientes. Tócalo y luego una obra para mandarlo; toca un edificio dañado para repararlo.</div>`;
    }
    const founder = sel.find((u) => u.def.founds);
    if (founder) {
      acts.push(`<button class="btn gold" type="button" data-act="found">${bi('found')}Fundar puesto · 800</button>`);
    }
```

```
    const app = this.app;
    if (sel.some((u) => u.def.weapon)) {
      const arm = app.input.amoveArm;
      acts.push(`<button class="btn ${arm === true ? 'gold' : ''}" type="button" data-act="amove" id="btn-amove"
title="Avanzar atacando (F)">${bi('amove')}${arm === true ? 'Toca el destino…' : 'Avanzar atacando'}</button>`);
      // v1.4: patrullar y mantener posición.
      const patrolling = sel.some((u) => u.patrol);
      acts.push(`<button class="btn ${arm === 'patrol' || patrolling ? 'gold' : ''}" type="button" data-act="patrol"
id="btn-patrol" title="Van y vuelven entre aquí y el punto que toques, peleando en el camino">${bi('patrol')}${arm ===
'patrol' ? 'Toca el otro punto…' : patrolling ? 'Patrullando' : 'Patrullar'}</button>`);
      const holding = sel.filter((u) => u.def.weapon).every((u) => u.hold);
      acts.push(`<button class="btn ${holding ? 'gold' : ''}" type="button" data-act="hold" id="btn-hold" aria-
pressed="${holding}" title="Se quedan quietas y solo disparan a lo que tienen a tiro">${bi('hold')}Mantener posición:
${holding ? 'sí' : 'no'}</button>`);
    }
    if (sel.some((u) => u.def.repairs)) {
      // v1.4: los Constructores e Ingenieros libres reparan solos lo dañado cerca.
      const on = this.app.game.players[0].autoRepair !== false;
      acts.push(`<button class="btn ${on ? 'good' : ''}" type="button" data-act="autorep" id="btn-autorep" aria-
pressed="${on}" title="Los Constructores e Ingenieros sin trabajo reparan solos los edificios dañados a ${AUTO_REPAIR.range}
casillas">${bi('wrench')}Reparación sola: ${on ? 'sí' : 'no'}</button>`);
    }
    if (sel.length > 1) {
      acts.push(`<button class="btn" type="button" data-act="formation" id="btn-formation" title="Bloque: juntas ·
Línea: una al lado de otra, de frente al destino">${bi('formation')}Formación: ${app.formation === 'line' ? 'línea' :
'bloque'}</button>`);
    }
    acts.push(`<button class="btn" type="button" data-act="stop">${bi('stop')}Detener</button>`);
    html += `<div class="actions mainacts">${acts.join('')}</div>`;
    html += `<div class="actions groupsave"><span class="queue">${bi('group')}Guardar en</span>${[1, 2, 3].map((n) =>
      `<button class="btn" type="button" data-act="savegrp" data-v="${n}" title="Guardar en el grupo ${n} (Ctrl o Mayús
+ ${n})">${n}</button>`).join('')}</div>`;
    return html;
  }

  pointHtml(pt) {
    const g = this.app.game;
    const ownerTxt = pt.owner >= 0
      ? `<b style="color:${hexColor(PLAYER_COLORS[pt.owner])}">${esc(g.players[pt.owner].name)}</b>` : 'Sin dueño';
    const state = pt.contested ? 'En disputa: hay tropas de varios equipos cerca'
      : pt.owner < 0 && pt.prog > 0 ? `Capturando… ${Math.floor(pt.prog * 100)} %`
        : `Para capturarlo: infantería a 1 casilla y sin enemigos a ${CAPTURE.clear} casillas durante ${CAPTURE.time}
s.`;
    let html = this.head(pt.type, pt.owner >= 0 ? PLAYER_COLORS[pt.owner] : NEUTRAL, esc(pt.def.name),
      `${ownerTxt} · ${esc(pt.def.desc)}`, false);
    html += `<div class="queue">${state}</div>`;
    if (g.mode.king && pt.type === 'faro') {
      const parts = Object.entries(g.kingTime).map(([team, t]) => {
        const pid = g.players.findIndex((p) => String(p.team) === team);
        return `<b style="color:${hexColor(PLAYER_COLORS[pid])}">${esc(g.players[pid].name)}</b> ${fmtTime(t)}`;
      });
      html += `<div class="queue">Rey del Faro (meta ${fmtTime(KING_TIME)}): ${parts.join(' · ') || 'nadie lo ha
controlado aún'}</div>`;
    }
    return html;
  }

  inspectHtml(e) {
    const g = this.app.game;
    const owner = g.players[e.owner];
    const rel = g.isEnemy(e.owner, 0) ? 'Enemigo' : 'Aliado';
    const sub = `<b style="color:${hexColor(PLAYER_COLORS[e.owner])}">${esc(owner.name)}</b> · ${rel} ·
${esc(e.def.desc)}`;
    return this.head(e.type, PLAYER_COLORS[e.owner], esc(e.def.name), sub) + (g.isEnemy(e.owner, 0) ?
this.counterHtml(e) : '');
  }

  // Ficha del enemigo (v0.8): vida, nivel, contra qué es fuerte y qué conviene usar contra él.
  counterHtml(e) {
    const g = this.app.game;
    const cls = e.kind === 'bld' ? 'bld' : e.cls;
    const w = e.kind === 'bld' ? g.weaponOf(e) : e.def.weapon;
    const rows = [`<span>Vida</span><b id="enemy-hp">${fmt(Math.max(0, e.hp))} / ${fmt(e.maxHp)}
(${Math.round((Math.max(0, e.hp) / e.maxHp) * 100)} %)</b>`];
    if (e.kind === 'bld' && e.def.maxLevel > 1) rows.push(`<span>Nivel</span><b>${e.level} de ${e.def.maxLevel}</b>`);
    if (w && w.mult) {
      const strong = Object.entries(w.mult).filter(([k, v]) => v >= 1.2 && CAT_NAMES[k]).map(([k]) => CAT_NAMES[k]);
      const cant = Object.entries(w.mult).filter(([k, v]) => v === 0 && CAT_NAMES[k]).map(([k]) => CAT_NAMES[k]);
      rows.push(`<span>Fuerte contra</span><b>${strong.length ? strong.join(', ') : 'nada en especial'}</b>`);
      if (cant.length) rows.push(`<span>No ataca a</span><b>${cant.join(', ')}</b>`);
    } else {
      rows.push('<span>Ataque</span><b>no dispara</b>');
    }
    rows.push(`<span>Usa</span><b>${this.countersFor(cls).join(', ')}</b>`);
    return `<div class="cardinfo enemycard" id="enemycard">${rows.join('')}</div>`;
  }

  // Las 3 tropas que más daño hacen a esa clase por cada moneda (daño por segundo × multiplicador).
```

```
  countersFor(cls) {
    return Object.entries(UNITS)
      .filter(([, d]) => d.weapon && (d.weapon.mult[cls] || 0) > 0 && !d.limit)
      .map(([t, d]) => [t, ((d.weapon.dmg / d.weapon.cd) * d.weapon.mult[cls]) / Math.sqrt(d.cost)])
      .sort((a, b) => b[1] - a[1]).slice(0, 3).map(([t]) => UNITS[t].name);
  }

  updateSelLive(sel) {
    const ehp = $('enemy-hp');
    if (ehp && sel[0]) ehp.textContent = `${fmt(Math.max(0, sel[0].hp))} / ${fmt(sel[0].maxHp)}
(${Math.round((Math.max(0, sel[0].hp) / sel[0].maxHp) * 100)} %)`;
    const hp = $('sel-hp');
    if (!hp || sel[0].kind === 'point') return;
    const first = sel[0];
    const bar = $('sel-hpbar');
    if (first.kind === 'bld' && !first.built) {
      bar.classList.add('build');
      hp.style.width = (first.progress * 100).toFixed(1) + '%';
      $('sel-hptext').textContent = `Construyendo… ${Math.floor(first.progress * 100)}%`;
    } else {
      bar.classList.remove('build');
      let cur = 0;
      let max = 0;
      for (const e of sel) {
        cur += Math.max(0, e.hp);
        max += e.maxHp;
      }
      const k = max ? cur / max : 0;
      hp.style.width = (k * 100).toFixed(1) + '%';
      hp.style.background = k > 0.55 ? 'var(--good)' : k > 0.28 ? '#f2c94c' : 'var(--bad)';
      $('sel-hptext').textContent = `${fmt(cur)} / ${fmt(max)}`;
    }
    if (first.kind === 'bld' && first.queue.length) {
      const q = first.queue[0];
      const bar0 = document.querySelector('#sel-queue .qitem i');
      if (bar0) bar0.style.width = Math.min(100, (q.t / UNITS[q.type].build) * 100) + '%';
      const qt = $('sel-qtime');
      if (qt) qt.textContent = this.queueTimeText(first);
    }
  }

  onSelClick(e) {
    const btn = e.target.closest('[data-act]');
    if (!btn) return;
    const app = this.app;
    const g = app.game;
    const sel = app.selection.filter((x) => x.alive);
    const act = btn.dataset.act;
    if (act === 'train' && this.suppressClick) {
      this.suppressClick = false;
      return;
    }
    app.sound.play('click');
    if (act === 'seldetails') {
      // v1.3: las partes largas del panel se pliegan; este botón las muestra u oculta.
      this.selMore = !this.selMore;
      this.renderSel(true);
      return;
    }
    if (act === 'close') {
      app.setSelection([]);
      return;
    }
    if (!sel.length) return;
    const b = sel[0];
    if (act === 'train') {
      const err = g.enqueue(b, btn.dataset.type);
      if (err) {
        this.toast(err, 'bad');
        app.sound.play('error');
      }
      this.renderSel(true);
    } else if (act === 'seltype') {
      const keep = this.app.selection.filter((e) => e.kind === 'unit' && e.type === btn.dataset.type);
      if (keep.length) this.app.setSelection(keep);
    } else if (act === 'cancel') {
      g.cancelQueue(b, Number(btn.dataset.idx));
      this.renderSel(true);
    } else if (act === 'rally') {
      app.input.startPlacement({ kind: 'rally', building: b });
    } else if (act === 'demolish') {
      this.demolishSelected();
    } else if (act === 'found') {
      const u = sel.find((x) => x.def.founds);
      if (u) {
        app.setSelection([u]);
        app.input.startPlacement({ kind: 'found', unit: u });
      }
```

```
    } else if (act === 'upgrade') {
      const res = g.upgrade(b);
      if (typeof res === 'string') {
        this.toast(res, 'bad');
        app.sound.play('error');
      } else {
        app.setSelection([res]);
        this.toast(`Mejora de ${res.def.name} pagada: un constructor la hará`, 'good');
      }
    } else if (act === 'upall') {
      const res = g.upgradeAll(0);
      if (typeof res === 'string') {
        this.toast(res, 'bad');
        app.sound.play('error');
      } else if (!res.n) {
        this.toast('No alcanza el dinero para ninguna mejora', 'bad');
        app.sound.play('error');
      } else {
        this.toast(`Mejorando ${res.n} ${res.n === 1 ? 'edificio' : 'edificios'} (−${fmt(res.spent)}): tus
constructores van a cada obra`, 'good');
      }
      this.renderSel(true);
    } else if (act === 'autoup') {
      const pl = g.players[0];
      pl.autoUpgrade = !pl.autoUpgrade;
      this.toast(pl.autoUpgrade ? 'Mejoras automáticas encendidas: se mejora solo hasta el máximo' : 'Mejoras
automáticas apagadas', pl.autoUpgrade ? 'good' : 'info');
      this.renderSel(true);
    } else if (act === 'dome') {
      const err = g.buyDome(0);
      if (err) {
        this.toast(err, 'bad');
        app.sound.play('error');
      } else {
        this.toast(`Cúpula nivel ${g.players[0].dome.level}`, 'good');
      }
      this.renderSel(true);
    } else if (act === 'unload') {
      const res = g.unload(b);
      if (typeof res === 'string') {
        this.toast(res, 'bad');
        app.sound.play('error');
      } else {
        app.setSelection(res);
      }
    } else if (act === 'stop') {
      g.orderStop(sel.filter((x) => x.kind === 'unit'));
    } else if (act === 'nuke') {
      const err = g.nukeInvalid(b);
      if (err) {
        this.toast(err, 'bad');
        app.sound.play('error');
      } else {
        app.input.startPlacement({ kind: 'nuke', building: b });
      }
    } else if (act === 'bombzone') {
      const bombers = sel.filter((x) => x.kind === 'unit' && x.type === 'bomber');
      if (bombers.length) app.input.startPlacement({ kind: 'bombzone', units: bombers });
    } else if (act === 'nuke3') {
      const err = g.salvoInvalid(0);
      if (err) {
        this.toast(err, 'bad');
        app.sound.play('error');
      } else {
        app.input.startPlacement({ kind: 'nuke3', building: b });
      }
    } else if (act === 'repeat') {
      const n = Number(btn.dataset.n) || 1;
      let added = 0;
      let err = null;
      for (let k = 0; k < n; k++) {
        err = g.enqueue(b, b.lastType);
        if (err) break;
        added++;
      }
      if (!added) {
        this.toast(err, 'bad');
        app.sound.play('error');
      } else if (added < n) {
        this.toast(`${added} en cola · ${err}`, 'bad');
      }
      this.renderSel(true);
    } else if (act === 'amove') {
      if (app.input.amoveArm === true) app.input.disarmAttackMove();
      else app.input.armAttackMove();
      this.renderSel(true);
    } else if (act === 'patrol') {
      const us = sel.filter((x) => x.kind === 'unit' && x.def.weapon);
```

```
      if (app.input.amoveArm === 'patrol') app.input.disarmAttackMove();
      else if (us.some((u) => u.patrol)) { g.orderStop(us); this.toast('Dejaron de patrullar'); }
      else app.input.armAttackMove('patrol');
      this.renderSel(true);
    } else if (act === 'hold') {
      const us = sel.filter((x) => x.kind === 'unit' && x.def.weapon);
      const on = !us.every((u) => u.hold);
      g.setHold(us, on);
      this.toast(on ? 'Mantienen la posición: solo disparan a lo que tienen a tiro' : 'Vuelven a perseguir a los
enemigos cercanos');
      this.renderSel(true);
    } else if (act === 'autorep') {
      const p = g.players[0];
      p.autoRepair = p.autoRepair === false;
      this.toast(p.autoRepair ? 'Los constructores libres repararán solos' : 'Reparación sola apagada');
      this.renderSel(true);
    } else if (act === 'loop') {
      b.loop = !b.loop;
      b.loopT = 0;
      if (b.loop) this.toast(`${b.def.name}: sigue entrenando ${UNITS[b.lastType].name} mientras alcance el dinero`,
'good');
      this.renderSel(true);
    } else if (act === 'formation') {
      app.toggleFormation();
      this.renderSel(true);
    } else if (act === 'savegrp') {
      app.saveGroup(Number(btn.dataset.v));
    }
  }

  // ---------- v0.8: barra de producción ----------
  // Tus edificios que entrenan, con su cola y su progreso. Los parados (sin cola) en amarillo.
  renderProd() {
    const app = this.app;
    const g = app.game;
    const el = $('prodbar');
    if (!g) return;
    const order = { core: 0, barracks: 1, factory: 2, harbor: 3, helipad: 4 };
    const list = g.buildings.filter((b) => b.alive && b.owner === 0 && b.built && b.def.produces)
      .sort((a, b) => order[a.type] - order[b.type] || a.id - b.id);
    const sig = list.map((b) => `${b.id}:${b.queue.length}:${b.queue[0] ? b.queue[0].type : ''}`).join(',');
    el.hidden = list.length < 2;
    document.body.classList.toggle('has-prod', !el.hidden);
    if (el.hidden) return;
    if (sig !== this.prodSig) {
      this.prodSig = sig;
      el.innerHTML = '<span class="lbl">Producción</span>' + list.map((b) => {
        const idle = !b.queue.length && b.type !== 'core';
        const q = b.queue[0];
        const tip = `${b.def.name}: ${q ? `entrenando ${UNITS[q.type].name}${b.queue.length > 1 ? ` (+${b.queue.length
- 1})` : ''}` : 'parado'}`;
        return `<button class="pitem ${idle ? 'idle' : ''}" type="button" data-prod="${b.id}" title="${esc(tip)}" aria-
label="${esc(tip)}">
          <img src="${Thumbs.get(q ? q.type : b.type, PLAYER_COLORS[0])}" alt="">${b.queue.length ?
`<em>${b.queue.length}</em>` : ''}<i><s data-pbar="${b.id}"></s></i></button>`;
      }).join('');
    }
    for (const b of list) {
      const bar = el.querySelector(`[data-pbar="${b.id}"]`);
      const q = b.queue[0];
      if (bar) bar.style.width = q ? Math.min(100, (q.t / UNITS[q.type].build) * 100) + '%' : '0';
    }
  }

  // ---------- v0.8: avisos en el borde ----------
  // Si atacan algo tuyo fuera de la pantalla, una flecha en el borde apunta hacia allá.
  addEdgeAlert(text, x, z) {
    const now = performance.now();
    let label = text.replace(/^Atacan tu /, '').replace(/^Perdiste: /, 'Perdiste ').replace(/^Perdiste /, 'Perdiste ');
    if (/Núcleo/.test(text)) label = 'Núcleo';
    else if (/tropas/i.test(text)) label = 'Tropas';
    if (label.length > 18) label = label.slice(0, 17) + '…';
    const near = this.edgeAlerts.find((a) => Math.hypot(a.x - x, a.z - z) < 5);
    if (near) {
      near.until = now + 8000;
      near.label = label;
      near.x = x;
      near.z = z;
      if (near.el) near.el.querySelector('span').textContent = label;
      return;
    }
    this.edgeAlerts.push({ x, z, label, until: now + 8000, el: null });
    if (this.edgeAlerts.length > 4) {
      const old = this.edgeAlerts.shift();
      if (old.el) old.el.remove();
    }
  }

```

```
  updateEdges() {
    const r = this.app.renderer;
    if (!r) return;
    const now = performance.now();
    const box = $('edges');
    for (const a of this.edgeAlerts) {
      if (a.until > now) continue;
      if (a.el) a.el.remove();
      a.el = null;
    }
    this.edgeAlerts = this.edgeAlerts.filter((a) => a.until > now);
    const W = r.width;
    const H = r.height;
    const m = 34;
    for (const a of this.edgeAlerts) {
      if (!a.el) {
        a.el = document.createElement('button');
        a.el.type = 'button';
        a.el.className = 'edge';
        a.el.innerHTML = `<i>${ICON.arrow}</i><span></span>`;
        a.el.querySelector('span').textContent = a.label;
        a.el.addEventListener('click', () => {
          this.app.focusAt(a.x, a.z);
          a.until = 0;
        });
        box.appendChild(a.el);
      }
      const s = r.project(a.x, 0.3, a.z);
      const inside = s.ok && s.x > m && s.x < W - m && s.y > 70 && s.y < H - m;
      a.el.hidden = inside;
      if (inside) continue;
      let dx = s.x - W / 2;
      let dy = s.y - H / 2;
      if (!s.ok) { dx = -dx; dy = -dy; }
      const ang = Math.atan2(dy, dx);
      const hw = W / 2 - m - 40;
      const hh = H / 2 - m - 40;
      const c = Math.cos(ang);
      const sn = Math.sin(ang);
      const t = Math.min(Math.abs(c) > 1e-6 ? hw / Math.abs(c) : 1e9, Math.abs(sn) > 1e-6 ? hh / Math.abs(sn) : 1e9);
      a.el.style.transform = `translate(${Math.round(W / 2 + c * t)}px, ${Math.round(H / 2 + sn * t)}px)
translate(-50%, -50%)`;
      a.el.querySelector('i').style.transform = `rotate(${ang}rad)`;
    }
  }

  // Botones de grupo y de Constructores junto al minimapa: cuántas tropas tiene cada uno.
  renderGroups(force) {
    const app = this.app;
    const g = app.game;
    if (!g) return;
    const counts = [1, 2, 3].map((n) => app.groupUnits(n).length);
    const builders = g.units.filter((u) => u.owner === 0 && u.alive && u.def.builds === 'land');
    const idle = builders.filter((u) => u.state === 'idle' && !u.inside).length;
    const sig = counts.join(',') + '|' + idle + '/' + builders.length;
    if (!force && sig === this.groupsSig) return;
    this.groupsSig = sig;
    counts.forEach((k, i) => {
      const el = $('grp' + (i + 1));
      el.textContent = k ? String(k) : '–';
      el.parentElement.classList.toggle('has', k > 0);
    });
    $('bld-idle').textContent = builders.length ? `${idle}/${builders.length}` : '0';
    $('btn-builders').classList.toggle('alert', idle > 0);
  }

  demolishSelected() {
    const app = this.app;
    const b = app.selection[0];
    if (!b || b.kind !== 'bld' || b.owner !== 0 || b.type === 'core') return;
    if (this.demolishArm === b.id) {
      app.game.demolish(b);
      this.demolishArm = -1;
      app.setSelection([]);
      this.toast(`${b.def.name} ${b.def.fem ? 'demolida' : 'demolido'}`);
      return;
    }
    this.demolishArm = b.id;
    this.renderSel(true);
    setTimeout(() => {
      if (this.demolishArm === b.id) {
        this.demolishArm = -1;
        this.renderSel(true);
      }
    }, 3000);
  }

  // ---------- barra superior ----------
```

```
  updateTop() {
    const g = this.app.game;
    const p = g.players[0];
    $('coins').textContent = fmt(p.coins);
    $('income').textContent = `+${fmt(p.income)}/min`;
    $('pop').textContent = `${p.popUsed}/${p.popCap}`;
    $('energy').textContent = fmt(p.energy);
    $('energyRate').textContent = `+${p.energyRate}/10 s`;
    if (g.mode.king) {
      const faro = g.points.find((pt) => pt.type === 'faro');
      const ownerTeam = faro && faro.owner >= 0 ? g.players[faro.owner].team : -1;
      const myT = g.kingTime[p.team] || 0;
      let best = -1;
      let bestT = 0;
      for (const [team, t] of Object.entries(g.kingTime)) {
        if (Number(team) !== p.team && t > bestT) { bestT = t; best = Number(team); }
      }
      const rival = best >= 0 ? g.players.findIndex((pl) => pl.team === best) : -1;
      const bar = (t, color) => `<span class="bar"><i style="width:${Math.min(100, (t / KING_TIME) *
100)}%;background:${color}"></i></span>`;
      $('king').innerHTML = `Faro: ${ownerTeam < 0 ? 'libre' : esc(g.players[faro.owner].name)} · Tú ${bar(myT,
hexColor(PLAYER_COLORS[0]))}` +
        (rival >= 0 ? ` ${esc(g.players[rival].name)} ${bar(bestT, hexColor(PLAYER_COLORS[rival]))}` : '');
    }
    $('stat-pop').classList.toggle('full', p.popUsed >= p.popCap);
    $('clock').textContent = fmtTime(g.time);
    // v1.3: también cambia con las alianzas (equipos), no solo cuando alguien cae.
    const sig = g.players.map((pl) => `${pl.alive}:${pl.team}`).join(',');
    if (sig !== this.playersSig) {
      this.playersSig = sig;
      const myTeam = p.team;
      $('players').innerHTML = g.players.map((pl) => {
        const tag = pl.id === 0 ? '' : pl.team === myTeam ? '<em>aliado</em>' : '';
        const style = pl.human ? '' : ` · ${PERSONALITIES[pl.persona].name}`;
        return `<div class="pchip ${pl.alive ? '' : 'dead'}" title="${esc(pl.name + style)}"><i
style="background:${hexColor(pl.color)}"></i><span>${esc(pl.name)}</span>${tag}</div>`;
      }).join('');
    }
  }

  update(dt) {
    this.slowT -= dt;
    if (this.slowT > 0) return;
    this.slowT = 0.15;
    this.updateCoach();
    this.updateTop();
    this.refreshBuild();
    this.renderSel();
    this.renderMission();
    this.renderGroups();
    this.renderProd();
    this.popT = (this.popT || 0) - 0.15;
    if (this.pop && this.popT <= 0) {
      this.popT = 0.5;
      this.renderPop();
    }
  }

  showEnd() {
    const g = this.app.game;
    const win = g.result === 'victory';
    $('enddialog').className = 'dialog panel wide ' + (win ? 'result-victory' : 'result-defeat');
    $('enddialog').innerHTML = this.app.lobby.endHtml(g);
    $('end').hidden = false;
    $('btn-again').addEventListener('click', () => this.app.startGame());
    const next = $('btn-next');
    if (next) next.addEventListener('click', () => this.app.startMission(Number(next.dataset.v)));
    $('btn-menu').addEventListener('click', () => this.app.toMenu());
    $('btn-watch').addEventListener('click', () => { $('end').hidden = true; });
    const spec = $('btn-spectate');
    if (spec) spec.addEventListener('click', () => this.app.spectate());
    this.bindHistory(g);
  }

  // Gráficas del final: línea vertical y valores del momento al pasar el dedo o el ratón.
  bindHistory(g) {
    const H = g.history || [];
    document.querySelectorAll('#enddialog .histchart').forEach((fig) => {
      const key = fig.dataset.key;
      const svg = fig.querySelector('svg');
      const tip = fig.querySelector('.histtip');
      const cross = fig.querySelector('.hcross');
      const vb = svg.viewBox.baseVal;
      const L = 40;
      const R = 64;
      const tMax = Math.max(H[H.length - 1].t, 1);
      const show = (ev) => {
        const r = svg.getBoundingClientRect();
```

```
        const sx = ((ev.clientX - r.left) / r.width) * vb.width;
        const t = clamp((sx - L) / (vb.width - L - R), 0, 1) * tMax;
        let best = H[0];
        for (const h of H) if (Math.abs(h.t - t) < Math.abs(best.t - t)) best = h;
        const cx = L + (best.t / tMax) * (vb.width - L - R);
        cross.setAttribute('x1', cx);
        cross.setAttribute('x2', cx);
        cross.setAttribute('visibility', 'visible');
        const rows = g.players.map((p) => ({ p, v: best[key][p.id] })).sort((a, b) => b.v - a.v);
        tip.innerHTML = `<b>${fmtTime(best.t)}</b>` + rows.map((x) => `<span><i
style="background:${cssColor(x.p.color)}"></i>${esc(x.p.name)}<em>${fmt(x.v)}</em></span>`).join('');
        tip.hidden = false;
        const px = (cx / vb.width) * r.width;
        tip.style.left = `${Math.min(r.width - 150, Math.max(0, px + 10))}px`;
      };
      const hide = () => { tip.hidden = true; cross.setAttribute('visibility', 'hidden'); };
      svg.addEventListener('pointermove', show);
      svg.addEventListener('pointerdown', show);
      svg.addEventListener('pointerleave', hide);
    });
  }
}

// Sonido sintetizado con Web Audio (sin archivos externos).
class Sound {
  constructor() {
    this.ctx = null;
    this.on = true;
    this.last = {};
    try {
      this.on = localStorage.getItem('faro-sound') !== 'off';
    } catch (err) { /* sin almacenamiento */ }
  }

  unlock() {
    if (!this.ctx) {
      try {
        const AC = window.AudioContext || window.webkitAudioContext;
        if (AC) {
          this.ctx = new AC();
          const len = this.ctx.sampleRate;
          this.noise = this.ctx.createBuffer(1, len, this.ctx.sampleRate);
          const d = this.noise.getChannelData(0);
          for (let i = 0; i < len; i++) d[i] = Math.random() * 2 - 1;
          this.master = this.ctx.createGain();
          this.master.gain.value = 0.6;
          this.master.connect(this.ctx.destination);
        }
      } catch (err) {
        this.ctx = null;
      }
    }
    if (this.ctx && this.ctx.state === 'suspended') this.ctx.resume().catch(() => {});
    if (this.ctx && !this.unlocked) {
      this.unlocked = true;
      if (this.onUnlock && this.on) this.onUnlock();
    }
  }

  toggle() {
    this.on = !this.on;
    try {
      localStorage.setItem('faro-sound', this.on ? 'on' : 'off');
    } catch (err) { /* sin almacenamiento */ }
    if (this.on) this.unlock();
    if (this.onToggle) this.onToggle();
  }

  // Límite de voces: si ya suenan muchas, los sonidos nuevos menos importantes se saltan.
  voice(dur) {
    const now = this.ctx.currentTime;
    this.ends = (this.ends || []).filter((t) => t > now);
    if (this.ends.length >= 24) return false;
    this.ends.push(now + dur);
    this.voices = (this.voices || 0) + 1;
    return true;
  }

  tone(freq, dur, type = 'sine', vol = 0.08, slide = 0, delay = 0) {
    const c = this.ctx;
    const t0 = c.currentTime + delay;
    const o = c.createOscillator();
    const g = c.createGain();
    o.type = type;
    o.frequency.setValueAtTime(freq, t0);
    if (slide) o.frequency.exponentialRampToValueAtTime(Math.max(30, freq + slide), t0 + dur);
    g.gain.setValueAtTime(vol, t0);
    g.gain.exponentialRampToValueAtTime(0.0001, t0 + dur);
```

```
    o.connect(g);
    g.connect(this.master);
    o.start(t0);
    o.stop(t0 + dur + 0.02);
  }

  // Ruido filtrado; sweep cambia la frecuencia del filtro durante el sonido (zumbido de misil).
  burst(dur, filterType, freq, vol, delay = 0, sweep = 0, q = 0.8) {
    const c = this.ctx;
    const t0 = c.currentTime + delay;
    const src = c.createBufferSource();
    src.buffer = this.noise;
    const f = c.createBiquadFilter();
    f.type = filterType;
    f.frequency.setValueAtTime(freq, t0);
    if (sweep) f.frequency.exponentialRampToValueAtTime(Math.max(40, freq + sweep), t0 + dur);
    f.Q.value = q;
    const g = c.createGain();
    g.gain.setValueAtTime(0.0001, t0);
    g.gain.exponentialRampToValueAtTime(vol, t0 + Math.min(0.012, dur * 0.2));
    g.gain.exponentialRampToValueAtTime(0.0001, t0 + dur);
    src.connect(f);
    f.connect(g);
    g.connect(this.master);
    src.start(t0, Math.random() * 0.5);
    src.stop(t0 + dur + 0.02);
  }

  play(name, vol = 1) {
    if (!this.on || !this.ctx || this.ctx.state !== 'running') return;
    const now = this.ctx.currentTime;
    const gap = { bullet: 0.07, minigun: 0.05, flak: 0.08, bolt: 0.12, laser: 0.14, hit: 0.08, boom: 0.09, bigboom:
0.25,
      rocket: 0.1, missile: 0.2, shell: 0.12, torpedo: 0.2, build: 0.22, heal: 0.4 }[name] || 0.04;
    if (this.last[name] && now - this.last[name] < gap) return;
    const ui = ['click', 'order', 'place', 'done', 'ready', 'error', 'alarm', 'capture'].includes(name);
    if (!ui && !this.voice(0.4)) return;
    this.last[name] = now;
    const v = vol;
    switch (name) {
      case 'bullet': this.burst(0.06, 'highpass', 2400, 0.06 * v); this.tone(180, 0.04, 'square', 0.02 * v, -80);
break;
      case 'minigun':
        for (let i = 0; i < 4; i++) this.burst(0.03, 'bandpass', 2600 + i * 120, 0.045 * v, i * 0.028, 0, 2);
        break;
      case 'flak': this.burst(0.09, 'bandpass', 1400, 0.07 * v); this.tone(240, 0.08, 'triangle', 0.03 * v, -120);
break;
      case 'bolt': this.tone(520, 0.12, 'sawtooth', 0.04 * v, -300); break;
      case 'laser':
        this.tone(1500, 0.16, 'sawtooth', 0.025 * v, -1100);
        this.tone(760, 0.18, 'square', 0.018 * v, -500);
        this.burst(0.1, 'highpass', 5000, 0.02 * v);
        break;
      case 'rocket': this.burst(0.22, 'bandpass', 700, 0.07 * v, 0, 900, 1.2); this.tone(130, 0.12, 'sine', 0.05 * v,
-50); break;
      case 'missile': this.burst(0.5, 'bandpass', 500, 0.07 * v, 0, 1400, 1.5); this.tone(90, 0.25, 'sine', 0.06 * v,
-40); break;
      case 'shell':
        this.tone(110, 0.22, 'sine', 0.15 * v, -70);
        this.burst(0.1, 'lowpass', 1100, 0.07 * v);
        this.burst(0.35, 'lowpass', 300, 0.04 * v, 0.05);
        break;
      case 'torpedo': this.burst(0.35, 'lowpass', 420, 0.06 * v, 0, -200); break;
      case 'hit': this.burst(0.08, 'bandpass', 600, 0.05 * v); break;
      case 'boom':
        this.burst(0.55, 'lowpass', 520, 0.24 * v);
        this.tone(70, 0.4, 'sine', 0.12 * v, -40);
        this.burst(0.25, 'bandpass', 1800, 0.03 * v, 0.08, 0, 1.5);
        break;
      case 'bigboom':
        this.burst(1.1, 'lowpass', 380, 0.3 * v);
        this.tone(52, 0.9, 'sine', 0.18 * v, -22);
        this.burst(0.6, 'bandpass', 1500, 0.04 * v, 0.12, -900, 1.2);
        this.burst(0.4, 'lowpass', 900, 0.08 * v, 0.3);
        break;
      case 'build': this.tone(900 + Math.random() * 140, 0.05, 'triangle', 0.035 * v, -300); this.burst(0.04,
'highpass', 3000, 0.02 * v); break;
      case 'heal': this.tone(660, 0.2, 'sine', 0.025 * v); this.tone(990, 0.25, 'sine', 0.02 * v, 0, 0.08); break;
      case 'capture':
        [523, 659, 784, 1047].forEach((f, i) => this.tone(f, 0.18, 'triangle', 0.05, 0, i * 0.09));
        break;
      case 'click': this.tone(880, 0.04, 'triangle', 0.04); break;
      case 'order': this.tone(520, 0.08, 'triangle', 0.05, 180); break;
      case 'place': this.tone(300, 0.12, 'triangle', 0.07, 140); this.burst(0.06, 'lowpass', 900, 0.04); break;
      case 'done': this.tone(660, 0.12, 'triangle', 0.06); this.tone(990, 0.16, 'triangle', 0.06, 0, 0.1); break;
      case 'ready': this.tone(740, 0.08, 'triangle', 0.045); break;
      case 'error': this.tone(200, 0.14, 'square', 0.04, -40); break;
      case 'alarm': this.tone(560, 0.15, 'square', 0.04); this.tone(420, 0.15, 'square', 0.04, 0, 0.18); break;
```

```
      default: break;
    }
  }
}
```

faro/src/js/085-meta.js · 1,761 líneas

```
'use strict';
// Vestíbulo y progresión (documento págs. 14-27): jugar, modos, personalizado, tutorial, pase,
// misiones, premio diario, códigos, tienda, cofres, inventario, final de partida, volver a la
// partida (punto 16) y modo creador (punto 19). El progreso se guarda en este navegador.

const PROFILE_KEY = 'faro-profile-v1';
const XP_REWARD = { play: 40, win: 60, island: 10, core: 20, mission: 50 };
const LEVEL_XP = 1000;
const PASS_TIERS = 40;
const PASS_TIER_XP = 300;
const DAILY_REWARDS = [20, 20, 30, 30, 40, 40, 'faro'];
const GAME_CRYSTALS = { play: 10, win: 20 };
const MISSION_CRYSTALS = { daily: 25, weekly: 80 };
const CODES = {
  FARO2026: { text: '+100 cristales', crystals: 100 },
  PRIMERAISLA: { text: 'Un Cofre de Isla gratis', chest: 'isla' },
  AMBAR: { text: '+300 XP', xp: 300 },
};
const RARITY = {
  comun: { name: 'Común', price: 150, refund: 20 },
  raro: { name: 'Raro', price: 300, refund: 50 },
  epico: { name: 'Épico', price: 600, refund: 120 },
  legendario: { name: 'Legendario', price: 1200, refund: 300 },
};
const CHESTS = {
  isla: { name: 'Cofre de Isla', cost: 350, odds: { comun: 60, raro: 28, epico: 10, legendario: 2 } },
  faro: { name: 'Cofre del Faro', cost: 600, odds: { comun: 40, raro: 38, epico: 17, legendario: 5 } },
};
const CHEST_PITY = 30;
const SKIN_SLOTS = {
  inf: { name: 'Soldados', model: 'rifle' },
  veh: { name: 'Vehículos', model: 'tank' },
  sea: { name: 'Barcos', model: 'destroyer' },
  core: { name: 'Núcleo', model: 'core' },
};
const SKINS = [
  ['selva', 'inf', 'comun'], ['selva', 'veh', 'comun'], ['selva', 'sea', 'raro'], ['selva', 'core', 'epico'],
  ['artico', 'inf', 'comun'], ['artico', 'veh', 'raro'], ['artico', 'sea', 'raro'], ['artico', 'core', 'epico'],
  ['castillo', 'inf', 'raro'], ['castillo', 'veh', 'raro'], ['castillo', 'sea', 'epico'], ['castillo', 'core',
'legendario'],
  ['neon', 'inf', 'raro'], ['neon', 'veh', 'epico'], ['neon', 'sea', 'epico'], ['neon', 'core', 'legendario'],
  ['frente', 'inf', 'comun'], ['frente', 'veh', 'raro'], ['frente', 'sea', 'raro'], ['frente', 'core', 'epico'],
  ['marea', 'inf', 'raro'], ['marea', 'veh', 'raro'], ['marea', 'sea', 'epico'], ['marea', 'core', 'legendario'],
].map(([look, slot, rar]) => ({ id: look + '_' + slot, look, slot, rar, name: `${SKIN_SLOTS[slot].name}
${SKIN_LOOKS[look].name}` }));
SKINS.push({ id: 'toxico_inf', look: 'toxico', slot: 'inf', rar: 'legendario', name: 'Brigada Tóxica' });
SKINS.push({ id: 'dorado_inf', look: 'dorado', slot: 'inf', rar: 'legendario', name: 'Soldado Dorado', special: 'Premio
del Tutorial' });
const SKIN_BY_ID = Object.fromEntries(SKINS.map((s) => [s.id, s]));
// v1.2: planos y fragmentos. Un cofre da un aspecto (70 %), un plano (20 %) o fragmentos (10 %).
// Los repetidos se vuelven fragmentos; en el Taller, plano + fragmentos = aspecto.
const CHEST_SPLIT = { skin: 70, plan: 20 };
const FRAG_DUP = { comun: 15, raro: 30, epico: 60, legendario: 120 };
const FRAG_COST = { comun: 60, raro: 120, epico: 240, legendario: 480 };
const FRAG_PACK = [20, 40];
const FRAG_GAME = { play: 4, win: 8 };
const CRYSTAL_PACKS = [
  { name: 'Bolsa de cristales', amount: 100 },
  { name: 'Caja de cristales', amount: 550 },
  { name: 'Cofre de cristales', amount: 1200 },
  { name: 'Tesoro del Faro', amount: 2600 },
];
const MISSIONS = {
  daily: [
    { id: 'd_play', text: 'Juega 2 partidas', stat: 'games', goal: 2 },
    { id: 'd_win', text: 'Gana 1 partida', stat: 'wins', goal: 1 },
    { id: 'd_build', text: 'Construye 12 edificios', stat: 'built', goal: 12 },
    { id: 'd_train', text: 'Entrena 15 tropas', stat: 'trained', goal: 15 },
    { id: 'd_cap', text: 'Captura 2 Banderas o el Faro', stat: 'captures', goal: 2 },
    { id: 'd_kill', text: 'Destruye 25 enemigos', stat: 'kills', goal: 25 },
    { id: 'd_up', text: 'Mejora 3 edificios', stat: 'upgrades', goal: 3 },
  ],
  weekly: [
    { id: 'w_win', text: 'Gana 5 partidas', stat: 'wins', goal: 5 },
    { id: 'w_play', text: 'Juega 10 partidas', stat: 'games', goal: 10 },
    { id: 'w_core', text: 'Destruye 5 Núcleos', stat: 'cores', goal: 5 },
    { id: 'w_build', text: 'Construye 60 edificios', stat: 'built', goal: 60 },
    { id: 'w_kill', text: 'Destruye 150 enemigos', stat: 'kills', goal: 150 },
    { id: 'w_cap', text: 'Captura 12 Banderas o el Faro', stat: 'captures', goal: 12 },
  ],
};
const MISSION_BY_ID = Object.fromEntries([...MISSIONS.daily, ...MISSIONS.weekly].map((m) => [m.id, m]));
// v1.3: los 3 pasos de tu documento de juego v2 (pág. 18), con la Barcaza que pide el PRD 1.5.
const TUTORIAL_STEPS = [
```

```
  { id: 'tut_build', text: '1 · Construí: pon otra Granja y una energía (Solar o Eólica)' },
  { id: 'tut_army', text: '2 · Producí: construye un Cuartel y entrena 3 soldados' },
  { id: 'kill', text: '3 · Atacá: cruza el mar en una Barcaza y toma la isla del bot' },
];
const LOBBY_MAPS = ['archipelago', 'open', 'continent', 'bay', 'reef', 'mesa', 'dunes'];
// v1.2: miniatura de cada mapa (se dibuja una vez con una semilla fija y se guarda).
const MAP_THUMBS = {};
// v1.5: iconos dibujados del vestíbulo (antes eran letras sueltas).
const LOBBY_ICONS = (() => {
  const svg = (d) => `<svg viewBox="0 0 24 24" aria-hidden="true" fill="none" stroke="currentColor" stroke-width="2"
stroke-linecap="round" stroke-linejoin="round">${d}</svg>`;
  return {
    play: '<svg viewBox="0 0 24 24" aria-hidden="true"><path d="M7 4.5v15l12-7.5z" fill="currentColor"/></svg>',
    modes: svg('<rect x="3" y="4" width="7" height="7" rx="1.5"/><rect x="14" y="4" width="7" height="7" rx="1.5"/>
<rect x="3" y="14" width="7" height="7" rx="1.5"/><rect x="14" y="14" width="7" height="7" rx="1.5"/>'),
    map: svg('<path d="M9 4 3 6.5v13L9 17l6 2.5 6-2.5v-13L15 6.5 9 4z"/><path d="M9 4v13M15 6.5v13"/>'),
    dice: svg('<rect x="4" y="4" width="16" height="16" rx="3"/><circle cx="9" cy="9" r="1.2" fill="currentColor"/>
<circle cx="15" cy="15" r="1.2" fill="currentColor"/><circle cx="15" cy="9" r="1.2" fill="currentColor"/><circle cx="9"
cy="15" r="1.2" fill="currentColor"/>'),
    pass: svg('<path d="M10 21h4l-.8-11h-2.4z"/><path d="M9.5 10h5M10.5 6h3l.7 4h-4.4z"/><path d="M12 3v3M15 7.5l4.5-
2M9 7.5l-4.5-2"/>'),
    missions: svg('<rect x="5" y="4" width="14" height="17" rx="2"/><path d="M9 3h6v3H9z"/><path d="m8.5 12 2 2 4-4M8.5
17.5h7"/>'),
    daily: svg('<rect x="4" y="9" width="16" height="11" rx="1.5"/><path d="M3 9h18M12 9v11"/><path d="M12 9c-1.5-3.5-
5-4-5-1.5S10 9 12 9zm0 0c1.5-3.5 5-4 5-1.5S14 9 12 9z"/>'),
    codes: svg('<circle cx="8" cy="15" r="4"/><path d="m11 12 8-8M16 7l2.5 2.5M14 9l2 2"/>'),
    backup: svg('<path d="M4 8h13l-3-3M20 16H7l3 3"/>'),
    creator: svg('<path d="m12 3 2.6 5.6 6.1.7-4.5 4.2 1.2 6L12 16.6 6.6 19.5l1.2-6-4.5-4.2 6.1-.7z"/>'),
  };
})();

function mapThumb(kind, players = 6) {
  const key = kind + ':' + players;
  if (MAP_THUMBS[key]) return MAP_THUMBS[key];
  let url = '';
  try {
    const m = new GameMap(4321, kind, players, false);
    const S = 132;
    const cv = document.createElement('canvas');
    cv.width = S;
    cv.height = S;
    const ctx = cv.getContext('2d');
    ctx.fillStyle = '#' + terrainColor(kind, 'deep').toString(16).padStart(6, '0');
    ctx.fillRect(0, 0, S, S);
    const ext = Math.max(...m.cells.map((c) => Math.max(Math.abs(c.x), Math.abs(c.z)))) + 1;
    const k = (S / 2 - 3) / ext;
    for (const c of m.cells) {
      if (c.t === 'deep') continue;
      ctx.fillStyle = '#' + terrainColor(kind, c.t).toString(16).padStart(6, '0');
      ctx.beginPath();
      ctx.arc(S / 2 + c.x * k, S / 2 + c.z * k, k * 1.02, 0, Math.PI * 2);
      ctx.fill();
    }
    for (const hi of m.homes) {
      const c = m.cells[hi];
      ctx.fillStyle = '#ffffff';
      ctx.beginPath();
      ctx.arc(S / 2 + c.x * k, S / 2 + c.z * k, 3.2, 0, Math.PI * 2);
      ctx.fill();
    }
    const f = m.cells[m.points[0].cell];
    ctx.fillStyle = '#f2ab38';
    ctx.beginPath();
    ctx.arc(S / 2 + f.x * k, S / 2 + f.z * k, 4, 0, Math.PI * 2);
    ctx.fill();
    url = cv.toDataURL('image/png');
  } catch (err) {
    console.warn('[Vestíbulo] Sin miniatura de mapa:', err);
  }
  MAP_THUMBS[key] = url;
  return url;
}
const MONTHS = ['ene', 'feb', 'mar', 'abr', 'may', 'jun', 'jul', 'ago', 'sep', 'oct', 'nov', 'dic'];

function skinSlotOf(kind, type) {
  if (kind === 'bld') return type === 'core' ? 'core' : null;
  const d = UNITS[type];
  if (!d) return null;
  if (d.mover === 'sea') return 'sea';
  if (d.cls === 'inf') return 'inf';
  if (d.cls === 'veh') return 'veh';
  return null;
}

function dayKey(d = new Date()) {
  return `${d.getFullYear()}-${String(d.getMonth() + 1).padStart(2, '0')}-${String(d.getDate()).padStart(2, '0')}`;
}

```

```
function dayNum(key) {
  const [y, m, d] = key.split('-').map(Number);
  return Math.round(Date.UTC(y, m - 1, d) / 86400000);
}

function keyOfDay(n) {
  const d = new Date(n * 86400000);
  return `${d.getUTCFullYear()}-${String(d.getUTCMonth() + 1).padStart(2, '0')}-${String(d.getUTCDate()).padStart(2, '0')}`;
}

// Semana que empieza el lunes (el 1 de enero de 1970 fue jueves).
function weekKey(d = new Date()) {
  return 'w' + Math.floor((dayNum(dayKey(d)) + 3) / 7);
}

function hashStr(s) {
  let h = 2166136261;
  for (let i = 0; i < s.length; i++) h = Math.imul(h ^ s.charCodeAt(i), 16777619);
  return h >>> 0;
}

function prettyDate(key) {
  const [y, m, d] = key.split('-').map(Number);
  return `${d} ${MONTHS[m - 1]} ${y}`;
}

function cssColor(c) {
  return '#' + c.toString(16).padStart(6, '0');
}

// ---------- perfil del jugador ----------
// Trazo de cada imperio en las gráficas del final (además del color): Tú, línea continua.
const HIST_DASH = ['', '7 4', '2 4', '10 3 2 3', '4 3', '1 3'];

// Rangos (v1.3, como tu documento de juego v2): 6 rangos propios, solo en la Liga 3 vs 3 y desde el
// nivel 10. Ganar suma 25 y perder resta 15. Almirante del Faro: en Roblox serán los 50 mejores; aquí,
// sin servidores, desde 2.400 puntos.
const RANKS = ['Recluta', 'Cabo', 'Sargento', 'Capitán', 'Comandante', 'Almirante del Faro'];
const RANK_MIN = [0, 300, 700, 1200, 1800, 2400];
const RANK_ICONS = ['R', 'C', 'S', 'Cp', 'Cm', 'A'];
const RANK_WIN = 25;
const RANK_LOSS = 15;
const RANK_COLORS = ['#8a9aa6', '#b87333', '#c0c7cf', '#f2c94c', '#7fc8ff', '#f2ab38'];
const RANKED_MODE = 'league';
const RANKED_LEVEL = 10;
// Temporadas (v1.3): de unos 2 meses. La 1 es noviembre-diciembre de 2026. Al empezar una nueva,
// el pase vuelve a empezar y bajas 1 rango (no a cero).
const SEASON_START = { y: 2026, m: 10 };
const SEASON_MONTHS = 2;
function seasonOf(d = new Date()) {
  const months = (d.getFullYear() - SEASON_START.y) * 12 + d.getMonth() - SEASON_START.m;
  const no = Math.max(0, Math.floor(months / SEASON_MONTHS)) + 1;
  const m0 = SEASON_START.m + (no - 1) * SEASON_MONTHS;
  const start = new Date(SEASON_START.y, m0, 1);
  const end = new Date(SEASON_START.y, m0 + SEASON_MONTHS, 1);
  return { no, start, end };
}
// Tabla de la Liga 3 vs 3 (v1.2): 20 comandantes bot con nombres propios. Sus puntos salen de tu
// semilla y se mueven un poco con cada partida que juegas (no son jugadores reales).
const LADDER_NAMES = ['Capitana Brisa', 'Coronel Basalto', 'Almirante Coral', 'Sargento Pino', 'Mariscal Ceniza',
'Teniente Ámbar',
  'Comandante Niebla', 'Capitán Granito', 'General Duna', 'Cabo Relámpago', 'Comodoro Marea', 'Brigadier Roble',
'Capitana Aurora',
  'Coronela Cobre', 'Mayor Tormenta', 'Teniente Alba', 'Almirante Faro', 'Sargenta Lava', 'General Escarcha', 'Capitán
Bruma'];
function ladderRows(P) {
  const rng = mulberry32((P.seed || 1) + 4242);
  const games = (P.ranked && P.ranked.games) || 0;
  const rows = LADDER_NAMES.map((name, i) => {
    const base = Math.round(60 + rng() * 2550);
    const drift = Math.round(Math.sin((games + 1) * 0.7 + i) * 18 + games * (rng() * 6 - 1));
    return { name, points: Math.max(0, base + drift), bot: true };
  });
  rows.push({ name: 'Tú', points: (P.ranked && P.ranked.points) || 0, me: true });
  return rows.sort((a, b) => b.points - a.points || (a.me ? -1 : 1)).map((r, i) => ({ ...r, pos: i + 1 }));
}
function rankIndex(points) {
  let i = 0;
  while (i + 1 < RANK_MIN.length && Math.max(0, points) >= RANK_MIN[i + 1]) i++;
  return i;
}
function rankName(points) {
  return RANKS[rankIndex(points)];
}
// Desde Sargento, los rivales de la Liga 3 vs 3 juegan en Difícil.
function rankedDifficulty(points) {
  return rankIndex(points) >= 2 ? 'hard' : 'normal';
```

```
}

// v1.4: logros. Cada uno con su insignia (inicial y color propios) y cristales al lograrlo.
// done(c): c = { P, g, me, win } al terminar una partida. prog(P): [actual, meta] para los acumulados.
const ACHIEVEMENTS = [
  { id: 'primer_faro', name: 'Primer Faro', desc: 'Termina el Tutorial.', icon: 'PF', color: '#f2ab38', crystals: 0,
    done: (c) => c.P.tutorialDone },
  { id: 'intacto', name: 'Sin un rasguño', desc: 'Gana una partida sin perder ningún edificio.', icon: 'SR', color:
'#5ee07a', crystals: 100,
    done: (c) => c.win && !c.g.mode.tutorial && c.g.time >= 180 && !(c.me.stats.lostBld > 0) },
  { id: 'guardian', name: 'Guardián del Faro', desc: 'Captura el Faro 3 veces.', icon: 'GF', color: '#ffd36a',
crystals: 100,
    done: (c) => (c.P.stats.faro || 0) >= 3, prog: (P) => [P.stats.faro || 0, 3] },
  { id: 'lobo_mar', name: 'Lobo de mar', desc: 'Desembarca tropas en otra isla con una Barcaza.', icon: 'LM', color:
'#7fc8ff', crystals: 75,
    done: (c) => (c.me.stats.landings || 0) > 0 },
  { id: 'veterano', name: 'Veterano', desc: 'Consigue una tropa con 3 estrellas.', icon: '★', color: '#ffb347',
crystals: 100,
    done: (c) => (c.me.stats.bestVet || 0) >= 3 },
  { id: 'incansable', name: 'Constructor incansable', desc: 'Termina 25 edificios en una partida.', icon: 'CI', color:
'#c0c7cf', crystals: 75,
    done: (c) => c.me.stats.built >= 25 },
  { id: 'cien', name: 'Ejército de cien', desc: 'Entrena 100 tropas (en total).', icon: '100', color: '#b87333',
crystals: 100,
    done: (c) => (c.P.stats.trained || 0) >= 100, prog: (P) => [P.stats.trained || 0, 100] },
  { id: 'diez', name: 'Diez victorias', desc: 'Gana 10 partidas.', icon: '10', color: '#f2c94c', crystals: 150,
    done: (c) => c.P.stats.wins >= 10, prog: (P) => [P.stats.wins, 10] },
  { id: 'expres', name: 'Victoria exprés', desc: 'Gana una partida en menos de 10 minutos.', icon: 'VE', color:
'#ef8a5a', crystals: 100,
    done: (c) => c.win && !c.g.mode.tutorial && !c.g.camp && c.g.time < 600 },
  { id: 'cielo', name: 'Cielo despejado', desc: 'Derriba 10 aeronaves en una partida.', icon: 'CD', color: '#9fd3ff',
crystals: 100,
    done: (c) => (c.me.stats.airKills || 0) >= 10 },
  { id: 'sargento', name: 'Sargento de la Liga', desc: 'Llega a Sargento (700 puntos) en la Liga 3 vs 3.', icon: 'S',
color: '#c0c7cf', crystals: 150,
    done: (c) => (c.P.ranked.best || 0) >= 700, prog: (P) => [Math.min(P.ranked.best || 0, 700), 700] },
  { id: 'semanal', name: 'Desafío de la semana', desc: 'Gana un desafío semanal.', icon: 'DS', color: '#e88bd0',
crystals: 100,
    done: (c) => c.win && !!c.g.opts.weekly },
];
const ACH_BY_ID = Object.fromEntries(ACHIEVEMENTS.map((a) => [a.id, a]));

// v1.4: desafío semanal. Cambia los lunes; mapa, semilla y una regla fijos toda la semana.
const WEEKLY_CHALLENGES = [
  { id: 'nofactory', name: 'Sin fábricas', desc: 'No puedes construir la Fábrica: gana con soldados, barcos y
aeronaves.', mode: 'conquest', mapKind: 'continent', difficulty: 'normal', rule: 'nofactory' },
  { id: 'noair', name: 'Cielo cerrado', desc: 'Sin Helipuerto ni Aeropuerto: todo se decide en tierra y mar.', mode:
'conquest', mapKind: 'archipelago', difficulty: 'normal', rule: 'noair' },
  { id: 'poor', name: 'Bolsillos vacíos', desc: 'Empiezas con 800 monedas en vez de 2.000.', mode: 'conquest', mapKind:
'mesa', difficulty: 'normal', startCoins: 800 },
  { id: 'fog', name: 'Niebla de guerra', desc: 'Niebla y bots en Difícil.', mode: 'conquest', mapKind: 'bay',
difficulty: 'hard', fog: true },
  { id: 'sea', name: 'Todo por mar', desc: 'Mar abierto: sin puentes, cada isla se toma por mar o por aire.', mode:
'conquest', mapKind: 'open', difficulty: 'normal' },
  { id: 'duel', name: 'Duelo en las dunas', desc: 'Uno contra uno en Dunas Rojas, contra un bot en Difícil.', mode:
'duel', mapKind: 'dunes', difficulty: 'hard' },
];
const WEEKLY_CRYSTALS = 150;
function weekNumber(d = new Date()) {
  return Math.floor((dayNum(dayKey(d)) + 3) / 7);
}
function weeklyFor(d = new Date()) {
  const n = weekNumber(d);
  const ch = WEEKLY_CHALLENGES[n % WEEKLY_CHALLENGES.length];
  const next = new Date(d.getFullYear(), d.getMonth(), d.getDate());
  next.setDate(next.getDate() + ((8 - next.getDay()) % 7 || 7));
  return { ...ch, week: 'w' + n, seed: 90000 + n * 37, next };
}

// v1.4: Top jugadores (tu documento, pág. 20): tú y comandantes bot, por temporada.
const TOP_PREFIX = ['Brisa', 'Coral', 'Roca', 'Marea', 'Ancla', 'Bruma', 'Duna', 'Ámbar', 'Lince', 'Ola'];
const TOP_SUFFIX = ['Norte', 'Sur', 'Veloz', 'Real', 'Azul', 'Roja', 'Gris', 'Dorada', 'Firme', 'Alta'];
function topRows(P, tab) {
  const S = seasonOf(Profile.now());
  const rng = mulberry32((P.seed || 1) + 777 + S.no * 13);
  const games = (P.ranked && P.ranked.games) || 0;
  const liga = ladderRows(P).filter((r) => r.bot);
  const extra = [];
  for (let i = 0; i < 30; i++) {
    const name = `${TOP_PREFIX[Math.floor(rng() * 10)]}${TOP_SUFFIX[Math.floor(rng() * 10)]}_${10 + Math.floor(rng() *
89)}`;
    extra.push({ name, points: Math.max(0, Math.round(40 + rng() * 2500 + Math.sin(games + i) * 15)), bot: true });
  }
  const bots = [...liga, ...extra].map((r, i) => {
    const r2 = mulberry32((P.seed || 1) + 91 + i * 7 + S.no);
    const pct = 0.42 + Math.min(0.32, r.points / 8000) + r2() * 0.06;
    const ranked = Math.round(20 + r.points / 9 + r2() * 30);
    return { name: r.name, bot: true, points: r.points, pct, wins: Math.round(ranked * pct + r2() * 120), islands:
```

```
Math.round(r2() * 900 + r.points / 6) };
  318   });
  319   const R = P.ranked;
  320   const me = { name: 'Tú', me: true, points: R.points, pct: R.games ? R.wins / R.games : 0, wins: P.stats.wins,
islands: P.stats.islands };
  321   const key = tab === 'wins' ? 'wins' : tab === 'islands' ? 'islands' : 'points';
  322   return [...bots, me].sort((a, b) => b[key] - a[key] || (a.me ? -1 : 1)).map((r, i) => ({ ...r, pos: i + 1 }));
  323 }
  324
  325 const Profile = {
  326   data: null,
  327   memoryOnly: false,
  328
  329   defaults() {
  330     return {
  331       v: 1, seed: (Math.random() * 1e9) | 0, since: dayKey(), xp: 0, crystals: 0,
  332       stats: { games: 0, wins: 0, islands: 0, cores: 0, kills: 0, trained: 0, faro: 0 },
  333       achievements: {}, weekly: {},
  334       owned: {}, equipped: { inf: null, veh: null, sea: null, core: null }, badges: {},
  335       tutorialDone: false, pass: {}, daily: { idx: 0, last: null, lost: 0 }, codes: {},
  336       missions: { day: null, daily: [], rerolled: false, week: null, weekly: [] },
  337       chests: { pity: 0, opened: 0 },
  338       play: { mode: 'conquest', mapKind: 'archipelago', difficulty: 'normal' },
  339       custom: { kind: 'conquest', players: 4, mapKind: 'archipelago', coins: START_COINS, difficulty: 'normal', teams:
false, fog: false, turbo: false },
  340       creator: { on: true, instant: false },
  341       campaign: { done: {}, stars: {} },
  342       ranked: { points: 0, best: 0, games: 0, wins: 0, history: [], shield: false },
  343       fragments: 0, plans: {},
  344       season: { no: 0, passXp: 0, notice: null },
  345     };
  346   },
  347
  348   load() {
  349     const d = this.defaults();
  350     try {
  351       const saved = JSON.parse(localStorage.getItem(PROFILE_KEY) || 'null');
  352       if (saved && saved.v === 1) {
  353         for (const k of Object.keys(saved)) {
  354           if (d[k] && typeof d[k] === 'object' && !Array.isArray(d[k]) && saved[k] && typeof saved[k] === 'object')
Object.assign(d[k], saved[k]);
  355           else d[k] = saved[k];
  356         }
  357       }
  358     } catch (err) {
  359       this.memoryOnly = true;
  360     }
  361     // v1.4: logros; quien ya terminó el Tutorial tiene «Primer Faro».
  362     d.achievements = d.achievements || {};
  363     if (d.tutorialDone && !d.achievements.primer_faro) d.achievements.primer_faro = d.since || dayKey();
  364     // v1.3: la Clasificatoria aparte pasa a ser la Liga 3 vs 3.
  365     if (d.play.mode === 'ranked') d.play.mode = RANKED_MODE;
  366     if (!MODES[d.play.mode] || !MODES[d.play.mode].lobby) d.play.mode = 'conquest';
  367     if (!MAPS[d.play.mapKind]) d.play.mapKind = 'archipelago';
  368     if (!DIFFICULTY[d.play.difficulty]) d.play.difficulty = 'normal';
  369     this.data = d;
  370     this.refresh();
  371     return d;
  372   },
  373
  374   save() {
  375     try {
  376       localStorage.setItem(PROFILE_KEY, JSON.stringify(this.data));
  377     } catch (err) {
  378       this.memoryOnly = true;
  379     }
  380   },
  381
  382   level(xp = this.data.xp) {
  383     return 1 + Math.floor(xp / LEVEL_XP);
  384   },
  385
  386   // v1.3: copia del progreso para llevarlo a otro iPad o navegador (sin servidores, no hay cuentas).
  387   // Código = «FARO1.» + suma de control + «.» + el perfil en base64.
  388   exportCode() {
  389     const json = JSON.stringify(this.data);
  390     const bytes = new TextEncoder().encode(json);
  391     let bin = '';
  392     for (let i = 0; i < bytes.length; i += 0x8000) bin += String.fromCharCode.apply(null, bytes.subarray(i, i +
0x8000));
  393     return `FARO1.${hashStr(json).toString(36)}.${btoa(bin)}`;
  394   },
  395
  396   importCode(code) {
  397     const m = String(code || '').trim().replace(/\s+/g, '').match(/^FARO1\.([0-9a-z]+)\.([A-Za-z0-9+/=]+)$/);
  398     if (!m) return { err: 'Ese texto no es una copia del progreso de Faro de Ámbar' };
  399     let json;
  400     try {
```

```
      const bin = atob(m[2]);
      const bytes = new Uint8Array(bin.length);
      for (let i = 0; i < bin.length; i++) bytes[i] = bin.charCodeAt(i);
      json = new TextDecoder().decode(bytes);
    } catch (err) {
      return { err: 'La copia está incompleta: cópiala entera otra vez' };
    }
    if (hashStr(json).toString(36) !== m[1]) return { err: 'La copia está incompleta o cambiada: cópiala entera otra vez' };
    let data;
    try { data = JSON.parse(json); } catch (err) { return { err: 'La copia no se puede leer' }; }
    if (!data || data.v !== 1 || typeof data.xp !== 'number') return { err: 'La copia no es de este juego' };
    return { data };
  },

  restore(data) {
    try { localStorage.setItem(PROFILE_KEY, JSON.stringify(data)); } catch (err) { this.memoryOnly = true; }
    if (this.memoryOnly) {
      this.data = data;
      return;
    }
    this.load();
  },

  // XP: el nivel cuenta toda la XP; el pase, solo la de esta temporada.
  addXp(n) {
    this.data.xp += n;
    this.data.season.passXp = (this.data.season.passXp || 0) + n;
  },

  now() {
    return new Date();
  },

  // Al empezar una temporada nueva: el pase vuelve a empezar y la Liga baja 1 rango (no a cero).
  checkSeason() {
    const P = this.data;
    const cur = seasonOf(this.now()).no;
    if (!P.season.no) {
      // Perfiles de antes de la v1.3: se quedan con lo que llevaban en el pase.
      P.season.no = cur;
      P.season.passXp = P.xp;
      return false;
    }
    if (P.season.no >= cur) return false;
    const R = P.ranked;
    const from = rankName(R.points);
    const idx = rankIndex(R.points);
    R.points = RANK_MIN[Math.max(0, idx - 1)];
    R.shield = false;
    P.pass = {};
    P.season.passXp = 0;
    P.season.notice = { no: cur, from, to: rankName(R.points) };
    P.season.no = cur;
    return true;
  },

  passTier() {
    return Math.min(PASS_TIERS, Math.floor((this.data.season.passXp || 0) / PASS_TIER_XP));
  },

  // Cambio de día o de semana: misiones nuevas y racha del premio diario.
  refresh() {
    const P = this.data;
    this.checkSeason();
    const today = dayKey();
    const M = P.missions;
    if (M.day !== today) {
      M.day = today;
      M.daily = this.pick(MISSIONS.daily, 3, 'd' + today).map((m) => ({ id: m.id, prog: 0, done: false }));
      M.rerolled = false;
    }
    const wk = weekKey();
    if (M.week !== wk) {
      M.week = wk;
      M.weekly = this.pick(MISSIONS.weekly, 3, wk).map((m) => ({ id: m.id, prog: 0, done: false }));
    }
    // Premio diario: si faltas un día, la racha baja 1 (por cada día que faltes).
    const D = P.daily;
    if (D.last) {
      const gap = dayNum(today) - dayNum(D.last);
      if (gap > 1) {
        const missed = gap - 1;
        D.lost = Math.min(D.idx, missed);
        D.idx = Math.max(0, D.idx - missed);
        D.last = keyOfDay(dayNum(today) - 1);
      }
    }
```

```
    this.save();
  },

  pick(pool, n, key) {
    const rng = mulberry32(hashStr(key) ^ this.data.seed);
    const list = pool.slice();
    for (let i = list.length - 1; i > 0; i--) {
      const j = Math.floor(rng() * (i + 1));
      [list[i], list[j]] = [list[j], list[i]];
    }
    return list.slice(0, n);
  },

  rerollDaily(i) {
    const M = this.data.missions;
    const cur = M.daily[i];
    if (!cur || cur.done || M.rerolled) return false;
    const used = new Set(M.daily.map((m) => m.id));
    const options = MISSIONS.daily.filter((m) => !used.has(m.id));
    if (!options.length) return false;
    const next = options[Math.floor(Math.random() * options.length)];
    M.daily[i] = { id: next.id, prog: 0, done: false };
    M.rerolled = true;
    this.save();
    return true;
  },

  dailyClaimable() {
    return this.data.daily.last !== dayKey();
  },

  passClaimable() {
    let n = 0;
    for (let t = 1; t <= this.passTier(); t++) if (!this.data.pass[t]) n++;
    return n;
  },

  owns(id) {
    return !!this.data.owned[id] || (Creator.active() && !!SKIN_BY_ID[id]);
  },

  looks() {
    const out = {};
    for (const [slot, id] of Object.entries(this.data.equipped)) {
      if (id && SKIN_BY_ID[id] && this.owns(id)) out[slot] = SKIN_BY_ID[id].look;
    }
    return out;
  },
};

function passReward(t) {
  if (t % 10 === 0) return { chest: 'faro', text: 'Cofre del Faro' };
  if (t % 5 === 0) return { chest: 'isla', text: 'Cofre de Isla' };
  const n = 20 + 5 * Math.floor((t - 1) / 10);
  return { crystals: n, text: `${n} cristales` };
}

// ---------- modo creador (solo la cuenta dueña del juego) ----------
const Creator = {
  owner: false,
  checked: false,

  // La plataforma de Claude dice si quien abre el enlace es el dueño. Sin plataforma, nadie lo es.
  async detect() {
    try {
      if (window.claude && typeof window.claude.use === 'function') {
        const user = await window.claude.use('user');
        this.owner = !!(user && (await user.isOwner()));
      }
    } catch (err) {
      this.owner = false;
    }
    this.checked = true;
    return this.owner;
  },

  active() {
    return this.owner && !!(Profile.data && Profile.data.creator.on);
  },
};

// ---------- vestíbulo ----------
class Lobby {
  constructor(app) {
    this.app = app;
    this.view = 'play';
    this.sub = 'main';
    this.shopTab = 'skins';
```

```
    this.modal = null;
    $('lobby-tabs').addEventListener('click', (e) => {
      const b = e.target.closest('[data-view]');
      if (!b) return;
      this.view = b.dataset.view;
      if (this.view === 'play') this.syncOpts();
      this.render();
    });
    $('menu').addEventListener('click', (e) => this.onClick(e));
    $('modal').addEventListener('click', (e) => {
      if (e.target === $('modal')) this.closeModal();
      else this.onClick(e);
    });
    $('modal').addEventListener('change', (e) => {
      if (e.target.id !== 'bk-filein' || !e.target.files || !e.target.files[0]) return;
      e.target.files[0].text().then((txt) => {
        const box = $('bk-in');
        if (box) box.value = txt.trim();
        this.loadBackup(txt, $('btn-bk-load'));
      });
    });
    $('modal').addEventListener('submit', (e) => {
      e.preventDefault();
      if (e.target.id === 'code-form') this.redeem($('code-input').value);
    });
    this.syncOpts(true);
    this.render();
  }

  get P() {
    return Profile.data;
  }

  // Opciones de la partida según la vista (Jugar, Personalizado o Tutorial).
  optsFor(sub = this.sub) {
    const P = this.P;
    if (sub === 'weekly') {
      const W = weeklyFor(Profile.now());
      return { mode: W.mode, mapKind: W.mapKind, difficulty: W.difficulty, seed: W.seed, fog: !!W.fog, rule: W.rule ||
null, startCoins: W.startCoins || 0, weekly: W.week, weeklyName: W.name };
    }
    if (sub === 'custom') {
      const c = P.custom;
      return {
        mode: c.kind === 'king' ? 'king' : 'conquest', mapKind: c.mapKind, difficulty: c.difficulty,
        custom: { players: c.players, teams: c.teams, coins: c.coins, fog: c.fog, turbo: c.turbo, king: c.kind ===
'king' },
      };
    }
    if (sub === 'tutorial') return { mode: 'tutorial', mapKind: 'open', difficulty: 'easy' };
    // Campaña: el fondo del vestíbulo sigue con la isla de «Jugar»; cada misión trae su propio mapa.
    // Liga 3 vs 3 (con rango): la dificultad la pone tu rango.
    if (MODES[P.play.mode] && MODES[P.play.mode].ranked) return { mode: P.play.mode, mapKind: P.play.mapKind,
difficulty: rankedDifficulty(P.ranked.points) };
    return { mode: P.play.mode, mapKind: P.play.mapKind, difficulty: P.play.difficulty };
  }

  syncOpts(first) {
    const next = this.optsFor();
    const o = this.app.opts;
    const players = (x) => (x.custom ? x.custom.players + ':' + x.custom.teams : MODES[x.mode].players);
    const changed = first || o.mode !== next.mode || o.mapKind !== next.mapKind || players(o) !== players(next);
    // El desafío semanal trae su propia semilla (el mismo mapa toda la semana).
    this.app.opts = { ...next, seed: next.seed || o.seed || ((Math.random() * 1e9) | 0) };
    if (changed && !first && this.app.loaded) this.app.makePreview();
  }

  setPlay(key, value) {
    if (this.sub === 'custom') this.P.custom[key] = value;
    else this.P.play[key] = value;
    Profile.save();
    this.syncOpts();
    this.render();
  }

  // ---------- dibujo ----------
  render() {
    if (!Profile.data) return;
    const P = this.P;
    $('crystals').textContent = Creator.active() ? '∞' : fmt(P.crystals);
    $('lvl-badge').textContent = `Nivel ${Profile.level()}`;
    $('creator-badge').hidden = !Creator.active();
    document.querySelectorAll('#lobby-tabs [data-view]').forEach((b) => b.setAttribute('aria-selected',
String(b.dataset.view === this.view)));
    const body = $('lobby-body');
    body.classList.toggle('wide', this.view !== 'play');
    if (this.view === 'shop') body.innerHTML = this.shopHtml();
    else if (this.view === 'inv') body.innerHTML = this.invHtml();
```

```
    else body.innerHTML = this.leftHtml() + `<div class="lmain">${this.mainHtml()}</div>` + this.sideHtml();
    // v1.3: aviso (una vez) de que empezó una temporada nueva.
    const note = P.season && P.season.notice;
    if (note && !this.modal) {
      P.season.notice = null;
      Profile.save();
      this.openModal('info', { title: `Empieza la Temporada ${note.no}`,
        text: `El Pase del Faro vuelve a empezar desde el nivel 1. En la Liga 3 vs 3 bajas 1 rango: de ${note.from} a
${note.to}.` });
      return;
    }
    if (this.modal) this.renderModal();
  }

  leftHtml() {
    const item = (sub, label, hint) => `<button type="button" data-act="sub" data-sub="${sub}" class="${this.sub ===
sub ? 'on' : ''}"><span class="lm-label">${label}</span>${hint ? `<small>${hint}</small>` : ''}</button>`;
    const off = (label) => `<button type="button" disabled title="Necesita servidores"><span class="lm-label">${label}
</span><small>Roblox</small></button>`;
    return `<nav class="lmenu panel" aria-label="Menú del vestíbulo">
      ${item('main', 'Jugar')}
      ${item('campaign', 'Campaña', `${this.campaignStars()} de ${CAMPAIGN.length * 3} ★`)}
      ${item('custom', 'Personalizado', 'mitad de XP')}
      ${item('tutorial', 'Tutorial', this.P.tutorialDone ? 'hecho' : 'premio')}
      ${item('ranks', 'Rangos', rankName(this.P.ranked.points))}
      ${item('top', 'Top jugadores', `T. ${seasonOf(Profile.now()).no}`)}
      ${item('weekly', 'Desafío semanal', ((this.P.weekly || {})[weeklyFor(Profile.now()).week] || {}).won ? 'ganado' :
'nuevo')}${off('Servidores')}
    </nav>`;
  }

  resumeHtml() {
    const app = this.app;
    if (app.suspended && !app.suspended.over) {
      const g = app.suspended;
      return `<div class="resume"><p><b>Partida en pausa</b> · ${esc(g.mode.name)} · ${fmtTime(g.time)}</p>
        <div class="row"><button class="btn gold" type="button" data-act="resume">Volver a la partida</button>
        <button class="btn" type="button" data-act="abandon">Abandonar</button></div></div>`;
    }
    const s = Save.peek();
    if (s) {
      return `<div class="resume"><p><b>Partida guardada</b> · ${esc(s.mode || '')} · ${esc(s.map || '')} ·
${fmtTime(s.time || 0)}</p>
        <div class="row"><button class="btn gold" type="button" data-act="load">Continuar partida guardada</button>
        <button class="btn" type="button" data-act="abandon">Borrar</button></div></div>`;
    }
    return '';
  }

  mainHtml() {
    const P = this.P;
    const resume = this.resumeHtml();
    if (this.sub === 'custom') return resume + this.customHtml();
    if (this.sub === 'tutorial') return resume + this.tutorialHtml();
    if (this.sub === 'campaign') return resume + this.campaignHtml();
    if (this.sub === 'ranks') return resume + this.ranksHtml();
    if (this.sub === 'top') return resume + this.topHtml();
    if (this.sub === 'weekly') return resume + this.weeklyHtml();
    const m = MODES[P.play.mode];
    const diffs = Object.entries(DIFFICULTY).map(([k, d]) => `<button type="button" data-act="diff" data-v="${k}"
class="${P.play.difficulty === k ? 'on' : ''}">${d.name}</button>`).join('');
    const st = P.stats;
    const pct = st.games ? Math.round((st.wins / st.games) * 100) : 0;
    const cover = Thumbs.cover();
    const ranked = MODES[P.play.mode] && MODES[P.play.mode].ranked;
    const diffName = ranked ? DIFFICULTY[rankedDifficulty(P.ranked.points)].name : DIFFICULTY[P.play.difficulty].name;
    const mapName = MAPS[P.play.mapKind] ? MAPS[P.play.mapKind].name : '';
    const thumb = LOBBY_MAPS.includes(P.play.mapKind) ? mapThumb(P.play.mapKind, Math.max(2, m.players || 6)) : '';
    const tile = (n, label) => `<div><b>${n}</b><span>${label}</span></div>`;
    // v1.5: portada nueva: la imagen grande con JUGAR adentro, el modo con la miniatura del mapa y tus números en
fichas.
    return `${resume}
      <section class="cover hero panel"${cover ? ` style="--cover:url('${cover}')"` : ''}>
        <div class="cover-img" aria-hidden="true"></div>
        <div class="cover-text">
          <div class="cover-brand"><svg viewBox="0 0 64 64" aria-hidden="true"><path d="M32 3 57 17.5v29L32 61 7 46.5v-
29z" fill="#1a2834" stroke="#f2ab38" stroke-width="3"/><path d="M26 50h12l-2-24h-8z" fill="#eef1f3"/><path d="M27.2
40h9.6l.5 5h-10.6zM28 31h8l.4 4h-8.8z" fill="#c0392b"/><rect x="27" y="20" width="10" height="6" rx="1" fill="#f2ab38"/>
<path d="M26 20l6-6 6 6z" fill="#eef1f3"/><path d="M37 21l15-6v10z" fill="#f2ab38" opacity=".55"/><path d="M27 21 12 15v10z"
fill="#f2ab38" opacity=".35"/></svg>
            <small>Versión ${GAME_VERSION}</small></div>
          <h2>Faro de <span>Ámbar</span></h2>
          <p>Construye tu isla, arma tu ejército de tierra, mar y aire y toma el Faro.</p>
          <ul class="cover-news" aria-label="Novedades">${['Núcleo nuevo', 'Edificios con tu color', 'Fichas de
tropas', 'Portada nueva', 'Errores arreglados'].map((t) => `<li>${t}</li>`).join('')}</ul>
        </div>
        <div class="hero-play">
          <button class="btn gold hero-btn" type="button" data-act="play" id="btn-play">${LOBBY_ICONS.play}
```

```
<span>Jugar</span></button>
          <span class="hero-mode">${esc(m.name)} · ${esc(mapName)} · ${esc(diffName)}</span>
        </div>
      </section>
      <section class="lcard panel modecard">
        ${thumb ? `<img class="modethumb" alt="" src="${thumb}">` : ''}
        <div class="mode-now"><small>Modo</small><b>${esc(m.name)}</b><span>${esc(m.desc)}</span>
          <span>Mapa: <b>${esc(mapName)}</b></span></div>
        <div class="modebtns">
          <button class="btn" type="button" data-act="modal" data-m="modes">${LOBBY_ICONS.modes}Cambiar modo</button>
          <button class="btn" type="button" data-act="modal" data-m="maps">${LOBBY_ICONS.map}Mapa</button>
          <button class="btn" type="button" data-act="reroll">${LOBBY_ICONS.dice}Otra isla</button>
        </div>
        ${ranked ? `<p class="queue modefoot">Liga 3 vs 3 · ${rankName(P.ranked.points)} · ${P.ranked.points} puntos.
La dificultad la pone tu rango (${DIFFICULTY[rankedDifficulty(P.ranked.points)].name}).</p>`
          : `<div class="field modefoot"><label>Dificultad de los bots</label><div class="seg">${diffs}</div></div>`}
      </section>
      <section class="lcard panel">
        <h3>Tus números</h3>
        <div class="stattiles numbers">
          ${tile(st.games, 'Partidas')}${tile(st.wins, 'Victorias')}${tile(`${pct} %`, 'De victorias')}
          ${tile(st.islands, 'Islas tomadas')}${tile(st.cores, 'Núcleos destruidos')}${tile(prettyDate(P.since),
'Jugando desde')}
        </div>
      </section>`;
  }

  customHtml() {
    const c = this.P.custom;
    const seg = (key, list) => `<div class="seg">${list.map(([v, label]) => `<button type="button" data-act="cset"
data-k="${key}" data-v="${v}" class="${String(c[key]) === String(v) ? 'on' : ''}">${label}</button>`).join('')}</div>`;
    return `<section class="lcard panel">
      <h2>Personalizado</h2>
      <p>Arma tu partida contra bots. Da la mitad de XP.</p>
      <div class="field"><label>Modo</label>${seg('kind', [['conquest', 'Conquista'], ['king', 'Rey del Faro']])}</div>
      <div class="field"><label>Jugadores</label>${seg('players', [[2, '2 (Duelo 1 vs 1)'], [3, '3'], [4, '4'], [5,
'5'], [6, '6']])}</div>
      <div class="field"><label>Mapa</label><div class="mapthumbs">${LOBBY_MAPS.map((k) => `<button type="button" data-
act="cset" data-k="mapKind" data-v="${k}" class="mapopt ${c.mapKind === k ? 'on' : ''}"><img alt="" src="${mapThumb(k,
Math.max(2, c.players))}"><span>${esc(MAPS[k].name)}</span></button>`).join('')}</div></div>
      <div class="field"><label>Monedas iniciales</label>${seg('coins', [[2000, '2.000'], [4000, '4.000'], [6000,
'6.000'], [10000, '10.000']])}</div>
      <div class="field"><label>Dificultad</label>${seg('difficulty', Object.entries(DIFFICULTY).map(([k, d]) => [k,
d.name]))}</div>
      <div class="field"><label>Equipos</label>${seg('teams', [[false, 'Todos contra todos'], [true, 'Dos equipos']])}
</div>
      <div class="field"><label>Extras</label>${seg('fog', [[false, 'Sin niebla'], [true, 'Niebla']])}${seg('turbo',
[[false, 'Ritmo normal'], [true, 'Turbo ×2']])}</div>
      <div class="row">
        <button class="btn gold" type="button" data-act="play" id="btn-play-custom">Jugar personalizado</button>
        <button class="btn" type="button" disabled>Unirse con código · Disponible en Roblox</button>
      </div>
    </section>`;
  }

  campaignStars() {
    const st = this.P.campaign.stars;
    return CAMPAIGN.reduce((n, m) => n + (st[m.id] || 0), 0);
  }

  // Campaña «La ruta del Faro»: 6 misiones; cada una se abre al terminar la anterior.
  campaignHtml() {
    const P = this.P;
    const creator = Creator.active();
    const cards = CAMPAIGN.map((m) => {
      const open = Campaign.unlocked(m.id, P, creator);
      const got = P.campaign.stars[m.id] || 0;
      const done = !!P.campaign.done[m.id];
      const stars = [0, 1, 2].map((k) => `<i class="${k < got ? 'on' : ''}" aria-hidden="true">★</i>`).join('');
      const extra = m.stars.map((s) => Campaign.starText(s)).join(' · ');
      return `<article class="mcard ${open ? '' : 'locked'} ${done ? 'done' : ''}">
        <div class="mnum">${m.id}</div>
        <div class="mbody"><b>${esc(m.name)}</b><span>${esc(Campaign.goalText(m))}</span>
          <small>★ ${esc(extra)}</small></div>
        <div class="mside"><span class="mstars" title="${got} de 3 estrellas">${stars}</span>
          ${open ? `<button class="btn ${done ? '' : 'gold'}" type="button" data-act="mission" data-v="${m.id}"
id="btn-mission-${m.id}">${done ? 'Repetir' : 'Jugar'}</button>`
            : `<span class="mlock">Termina la misión ${m.id - 1}</span>`}</div>
      </article>`;
    }).join('');
    return `<section class="lcard panel">
      <h2>La ruta del Faro</h2>
      <p>Seis misiones para volver a encender el Faro de Ámbar. Cada una se abre al terminar la anterior y da hasta 3
estrellas:
        una por terminarla y dos por las condiciones marcadas con ★.${creator ? ' Modo creador: todas abiertas.' : ''}
</p>
      <div class="missions">${cards}</div>
      <p class="small-note">Premio la primera vez: cristales (50 a 250) y XP. Cada estrella nueva:
```

```
+${CAMPAIGN_STAR_CRYSTALS} cristales. Tienes ${this.campaignStars()} de ${CAMPAIGN.length * 3} estrellas.</p>
    </section>`;
  }

  tutorialHtml() {
    const done = this.P.tutorialDone;
    return `<section class="lcard panel">
      <h2>Tutorial</h2>
      <p>Tres pasos contra un bot lento con un Núcleo débil, que está en otra isla: para llegar hace falta una Barcaza.
</p>
      <ol class="steps">${TUTORIAL_STEPS.map((s) => `<li>${esc(s.text)}</li>`).join('')}</ol>
      <div class="prizes">
        <div><b>Primer Faro</b><span>Insignia</span></div>
        <div><b>150 cristales</b><span>Una sola vez</span></div>
        <div><b>Soldado Dorado</b><span>Aspecto legendario</span></div>
      </div>
      <p>${done ? 'Ya recibiste el premio. Puedes repetir el tutorial cuando quieras.' : 'El premio se entrega la
primera vez que lo terminas.'}</p>
      <div class="row"><button class="btn gold" type="button" data-act="play" id="btn-play-tutorial">Empezar
tutorial</button></div>
    </section>`;
  }

  // Rangos (v1.3): tu rango, los 6 rangos de tu documento, la temporada, la tabla y tus partidas.
  ranksHtml() {
    const P = this.P;
    const R = P.ranked;
    const idx = rankIndex(R.points);
    const top = idx === RANKS.length - 1;
    const lo = RANK_MIN[idx];
    const hi = top ? lo : RANK_MIN[idx + 1];
    const pct = top ? 100 : Math.round(((R.points - lo) / (hi - lo)) * 100);
    const ladder = RANKS.map((n, i) => `<li class="${i === idx ? 'on' : i < idx ? 'past' : ''}"><i
style="background:${RANK_COLORS[i]}"></i>${n}<small>${fmt(RANK_MIN[i])}</small></li>`).join('');
    const hist = (R.history || []).slice(0, 8).map((h) => `<li class="${h.win ? 'win' : 'loss'}"><b>${h.win ?
'Victoria' : h.forfeit ? 'Abandono' : 'Derrota'}</b><span>${h.delta >= 0 ? '+' : ''}${h.delta}</span><small>${esc(h.rank)} ·
${prettyDate(h.day)}</small></li>`).join('');
    const diff = DIFFICULTY[rankedDifficulty(R.points)].name;
    const S = seasonOf(Profile.now());
    const ends = `${S.end.getDate()} de ${MONTHS[S.end.getMonth()]}`;
    const lv = Profile.level();
    const locked = lv < RANKED_LEVEL && !Creator.active();
    return `<section class="lcard panel ranked">
        <div class="rankhead"><div class="rankbadge" style="--rk:${RANK_COLORS[idx]}"><b>${RANK_ICONS[idx]}</b></div>
          <div><small>Tu rango · Temporada ${S.no}</small><h2 id="rank-name">${RANKS[idx]}</h2><span>${fmt(R.points)}
puntos · mejor: ${rankName(R.best || 0)}</span></div></div>
        <div class="rankbar" role="progressbar" aria-valuemin="0" aria-valuemax="100" aria-valuenow="${pct}"><i
style="width:${pct}%"></i></div>
        <p class="queue">${top ? 'Estás en el rango más alto.' : `${fmt(hi - R.points)} puntos para ${RANKS[idx +
1]}.`} Solo cuentan las partidas de la Liga 3 vs 3 (aliados bots; rivales en ${diff}). Ganar: +${RANK_WIN} · perder o
abandonar: −${RANK_LOSS}.</p>
        <p class="queue" id="rank-rules">${R.shield ? '<b>Protección activa:</b> acabas de subir, tu primera derrota no
te baja de rango. ' : 'Recién llegado a un rango, la primera derrota no te baja. '}La temporada termina el ${ends}: entonces
bajas 1 rango (no a cero) y el pase vuelve a empezar.</p>
        <div class="row"><button class="btn gold" type="button" data-act="ranked" id="btn-ranked" ${locked ? 'aria-
disabled="true"' : ''}>Jugar Liga 3 vs 3</button>${locked ? `<small class="lockmsg">Desde nivel ${RANKED_LEVEL} (tienes
${lv})</small>` : ''}</div>
      </section>
      <section class="lcard panel"><h3>Los 6 rangos</h3><ol class="ranklist">${ladder}</ol>
        <p class="queue">Almirante del Faro: en Roblox serán los 50 mejores; aquí, sin servidores, desde
${fmt(RANK_MIN[5])} puntos.</p></section>
      <section class="lcard panel"><h3>Tabla de la Liga 3 vs 3</h3>
        <p class="queue">Comandantes bot del juego (sin servidores no hay otros jugadores reales). Tu puesto cambia con
tus puntos.</p>
        <ol class="ladder" id="ladder">${ladderRows(P).map((r) => `<li class="${r.me ? 'me' : ''}"><b>${r.pos}.º</b>
<span>${esc(r.name)}${r.bot ? ' <small>bot</small>' : ''}</span><i style="background:${RANK_COLORS[rankIndex(r.points)]}">
</i><em>${rankName(r.points)}</em><strong>${r.points}</strong></li>`).join('')}</ol></section>
      <section class="lcard panel"><h3>Últimas partidas · ${R.wins || 0} de ${R.games || 0} ganadas</h3>${hist ? `<ul
class="rankhist">${hist}</ul>` : '<p class="queue">Todavía no jugaste la Liga 3 vs 3.</p>'}</section>`;
  }

  sideHtml() {
    const P = this.P;
    const lv = Profile.level();
    const into = P.xp % LEVEL_XP;
    const tier = Profile.passTier();
    const passN = Profile.passClaimable();
    const missionsLeft = [...P.missions.daily, ...P.missions.weekly].filter((m) => !m.done).length;
    const btn = (m, ic, title, sub, badge) => `<button class="sidebtn" type="button" data-act="modal" data-m="${m}">
      <span class="ic">${ic}</span><span><b>${title}</b><small>${sub}</small></span>${badge ? `<span
class="badge">${badge}</span>` : '<span></span>'}</button>`;
    return `<aside class="lside">
      <div class="lcard panel"><h3>Nivel ${lv}</h3><div class="xpbar"><i style="width:${(into / LEVEL_XP) * 100}%"></i>
</div>
        <p>${fmt(into)} / ${fmt(LEVEL_XP)} XP</p></div>
      ${btn('pass', LOBBY_ICONS.pass, 'Pase del Faro', `Nivel ${tier} de ${PASS_TIERS}`, passN || '')}
      ${btn('missions', LOBBY_ICONS.missions, 'Misiones', `${missionsLeft} por hacer`, '')}
      ${btn('daily', LOBBY_ICONS.daily, 'Premio diario', Profile.dailyClaimable() ? '¡Listo para reclamar!' : 'Vuelve
```

```
mañana', Profile.dailyClaimable() ? '1' : '')}
 876       ${btn('codes', LOBBY_ICONS.codes, 'Códigos', 'Canjea un código', '')}
 877       ${btn('backup', LOBBY_ICONS.backup, 'Copia de tu progreso', 'Llévalo a otro iPad', '')}
 878       ${Creator.owner ? btn('creator', LOBBY_ICONS.creator, 'Modo creador', Creator.active() ? 'Activado' : 'Apagado',
'') : ''}
 879     </aside>`;
 880   }
 881
 882   shopHtml() {
 883     const P = this.P;
 884     const tabs = [['skins', 'Apariencias'], ['chests', 'Cofres'], ['workshop', 'Taller'], ['crystals', 'Cristales']]
 885       .map(([k, l]) => `<button type="button" data-act="shoptab" data-v="${k}" class="${this.shopTab === k ? 'on' :
''}">${l}</button>`).join('');
 886     let inner = '';
 887     if (this.shopTab === 'skins') {
 888       inner = `<p>Solo cambian cómo se ven tus tropas y tu Núcleo: vida, daño y velocidad = iguales.</p>
 889         <div class="shopgrid">${SKINS.filter((s) => !s.special).map((s) => {
 890     const own = Profile.owns(s.id);
 891     const price = RARITY[s.rar].price;
 892     return `<div class="item ${own ? 'owned' : ''}">
 893             <img alt="" src="${Thumbs.get(SKIN_SLOTS[s.slot].model, PLAYER_COLORS[0], s.look)}">
 894             <b>${esc(s.name)}</b><span class="rar ${s.rar}">${RARITY[s.rar].name}</span>
 895             ${own ? '<small>Ya es tuyo</small>' : `<button class="btn" type="button" data-act="buy" data-id="${s.id}"
${P.crystals < price ? 'disabled' : ''}>${fmt(price)} cristales</button>`}
 896           </div>`;
 897   }).join('')}</div>`;
 898     } else if (this.shopTab === 'chests') {
 899       inner = `<div class="mode-cards">${Object.entries(CHESTS).map(([k, C]) => `<div class="chest">
 900           <b>${C.name}</b>
 901           <div class="probs">${Object.entries(C.odds).map(([r, pct]) => `<span class="rar ${r}">${RARITY[r].name}
</span><div class="xpbar"><i style="width:${pct}%"></i></div><span>${pct} %</span>`).join('')}</div>
 902           <button class="btn gold" type="button" data-act="chest" data-v="${k}" ${P.crystals < C.cost &&
!Creator.active() ? 'disabled' : ''}>Abrir · ${fmt(C.cost)} cristales</button>
 903         </div>`).join('')}</div>
 904         <p>Cada cofre da un aspecto (${CHEST_SPLIT.skin} %), un plano (${CHEST_SPLIT.plan} %) o fragmentos (${100 -
CHEST_SPLIT.skin - CHEST_SPLIT.plan} %). Garantía: 1 legendario cada ${CHEST_PITY} cofres (te faltan ${CHEST_PITY -
P.chests.pity}). Un aspecto repetido se convierte en fragmentos (${Object.entries(FRAG_DUP).map(([r, n]) =>
`${RARITY[r].name} ${n}`).join(', ')}).</p>`;
 905     } else if (this.shopTab === 'workshop') {
 906       // v1.2: Taller: plano + fragmentos = aspecto.
 907       const plans = SKINS.filter((s) => (P.plans || {})[s.id] && !P.owned[s.id]);
 908       inner = `<p>Tienes <b id="frag-count">${P.fragments || 0}</b> fragmentos. Ganas fragmentos con los aspectos
repetidos, en algunos cofres y jugando (+${FRAG_GAME.play} por partida, +${FRAG_GAME.win} si ganas).</p>
 909         ${plans.length ? `<div class="shopgrid">${plans.map((s) => `<div class="item plan">
 910           <img alt="" src="${Thumbs.get(SKIN_SLOTS[s.slot].model, PLAYER_COLORS[0], s.look)}">
 911           <b>${esc(s.name)}</b><span class="rar ${s.rar}">${RARITY[s.rar].name}</span>
 912           <button class="btn gold" type="button" data-act="craft" data-id="${s.id}" ${(P.fragments || 0) <
FRAG_COST[s.rar] ? 'disabled' : ''}>Fabricar · ${FRAG_COST[s.rar]} fragmentos</button></div>`).join('')}</div>`
 913         : '<p class="queue">Todavía no tienes planos: salen en los cofres.</p>'}`;
 914     } else {
 915       inner = `<p>Los cristales se ganan jugando: partidas, misiones, pase y premio diario. Comprarlos con dinero real
solo estará en Roblox.</p>
 916         <div class="shopgrid">${CRYSTAL_PACKS.map((c) => `<div class="item"><b>${esc(c.name)}</b><span>${fmt(c.amount)}
cristales</span>
 917           <button class="btn" type="button" disabled>Solo en Roblox</button></div>`).join('')}</div>`;
 918     }
 919     return `<section class="lcard panel"><header class="rowx"><h2>Tienda</h2><span class="crystals"><i></i>
<b>${Creator.active() ? '∞' : fmt(P.crystals)}</b></span></header>
 920       <div class="subtabs">${tabs}</div>${inner}</section>`;
 921   }
 922
 923   invHtml() {
 924     const P = this.P;
 925     const rows = Object.entries(SKIN_SLOTS).map(([slot, S]) => {
 926       const mine = SKINS.filter((s) => s.slot === slot && Profile.owns(s.id));
 927       const eq = P.equipped[slot];
 928       const none = `<button type="button" class="item ${!eq ? 'equipped' : ''}" data-act="equip" data-slot="${slot}"
data-id="">
 929         <img alt="" src="${Thumbs.get(S.model, PLAYER_COLORS[0])}"><b>Normal</b><small>${!eq ? 'Equipado' : 'Equipar'}
</small></button>`;
 930       return `<div class="field"><label>${S.name}</label><div class="shopgrid">${none}${mine.map((s) => `
 931         <button type="button" class="item ${eq === s.id ? 'equipped' : 'owned'}" data-act="equip" data-slot="${slot}"
data-id="${s.id}">
 932           <img alt="" src="${Thumbs.get(S.model, PLAYER_COLORS[0], s.look)}"><b>${esc(s.name)}</b>
 933           <span class="rar ${s.rar}">${RARITY[s.rar].name}</span><small>${eq === s.id ? 'Equipado' : 'Equipar'}</small>
</button>`).join('')}
 934         </div></div>`;
 935     }).join('');
 936     return `<section class="lcard panel"><h2>Inventario</h2>
 937       <p>Toca un aspecto para equiparlo. Se ve en tus tropas y tu Núcleo en la próxima partida.</p>
 938       ${rows}</section>` + this.achievementsHtml();
 939   }
 940
 941   // ---------- ventanas ----------
 942   // Descarga la copia como archivo. En el enlace publicado se usa el permiso de descargas del visor
 943   // (te pide confirmar); abierto como archivo suelto, la descarga normal del navegador.
 944   async saveBackupFile() {
 945     const filename = `faro-de-ambar-progreso-${dayKey()}.txt`;
```

```
    const data = Profile.exportCode();
    if (window.claude && typeof window.claude.use === 'function') {
      let dl = null;
      try { dl = await window.claude.use('downloads'); } catch (err) { dl = null; }
      if (!dl) {
        this.openModal('backup', { msg: 'Aquí no se puede descargar el archivo: usa «Copiar» y guarda el texto.' });
        return;
      }
      try {
        await dl.save({ filename, data });
        this.openModal('backup', { msg: 'Archivo guardado. En el otro iPad, usa «Elegir archivo».' });
      } catch (err) {
        const code = err && err.code;
        if (code === 'declined') return;
        this.openModal('backup', { msg: code === 'rate_limited' ? 'Espera un momento y vuelve a tocar «Descargar archivo».'
          : 'Aquí no se puede descargar el archivo: usa «Copiar» y guarda el texto.' });
      }
      return;
    }
    const a = document.createElement('a');
    a.href = URL.createObjectURL(new Blob([data], { type: 'text/plain' }));
    a.download = filename;
    document.body.appendChild(a);
    a.click();
    setTimeout(() => { URL.revokeObjectURL(a.href); a.remove(); }, 1000);
  }

  // Carga una copia: primero avisa que reemplaza el progreso (toca otra vez para confirmar).
  loadBackup(code, btn) {
    const r = Profile.importCode(code);
    if (r.err) {
      this.openModal('backup', { msg: r.err });
      return;
    }
    if (!this.bkArm || this.bkArm !== code) {
      this.bkArm = code;
      if (btn) btn.textContent = `¿Reemplazar? Nivel ${Profile.level(r.data.xp)} · toca otra vez`;
      return;
    }
    this.bkArm = null;
    Profile.restore(r.data);
    this.openModal('backup', { msg: `Listo: cargaste tu progreso (nivel ${Profile.level()}, ${fmt(Profile.data.crystals)} cristales).` });
    this.render();
  }

  openModal(name, data) {
    this.modal = name;
    this.modalData = data || null;
    $('modal').hidden = false;
    this.renderModal();
  }

  closeModal() {
    this.modal = null;
    $('modal').hidden = true;
  }

  renderModal() {
    const P = this.P;
    const head = (t) => `<header><h2>${t}</h2><button class="btn icon" type="button" data-act="close" aria-label="Cerrar">${ICON.close}</button></header>`;
    let html = '';
    const n = this.modal;
    if (n === 'modes') {
      const lv = Profile.level();
      html = head('Elige un modo') + `<div class="mode-cards">${Object.entries(MODES).filter(([, m]) => m.lobby).map(([k, m]) => {
        const locked = m.minLevel && lv < m.minLevel && !Creator.active();
        return `<button type="button" class="mode-card ${P.play.mode === k ? 'on' : ''}" data-act="pickmode" data-v="${k}" ${locked ? 'disabled' : ''}>
          <b>${esc(m.name)}</b><small>${esc(m.desc)}</small>${locked ? `<em>Desde nivel ${m.minLevel}</em>` : ''}
        </button>`;
      }).join('')}</div>`;
    } else if (n === 'maps') {
      html = head('Elige un mapa') + `<div class="mode-cards">${LOBBY_MAPS.map((k) => `<button type="button" class="mode-card mapcard ${P.play.mapKind === k ? 'on' : ''}" data-act="pickmap" data-v="${k}">
        <img class="mapthumb" alt="" src="${mapThumb(k)}"><b>${esc(MAPS[k].name)}</b><small>${esc(MAPS[k].desc)}</small></button>`).join('')}</div>`;
    } else if (n === 'pass') {
      const tier = Profile.passTier();
      const tiers = [];
      for (let t = 1; t <= PASS_TIERS; t++) {
        const r = passReward(t);
        const reached = t <= tier;
        const claimed = !!P.pass[t];
        tiers.push(`<div class="tier ${reached ? 'reached' : ''} ${claimed ? 'claimed' : ''}"><b>${t}</b>
```

```
<span>${r.text}</span>
1026           ${claimed ? '<small>Reclamado</small>' : reached ? `<button class="mini-btn" type="button" data-
act="passclaim" data-v="${t}">Reclamar</button>` : `<small>${fmt(t * PASS_TIER_XP)} XP</small>`}</div>`);
1027       }
1028       const S = seasonOf(Profile.now());
1029       html = head(`Pase del Faro · Temporada ${S.no}`) + `<p>Pista gratis de ${PASS_TIERS} niveles. Cada
${PASS_TIER_XP} XP subes un nivel del pase. Esta temporada llevas ${fmt(P.season.passXp || 0)} XP (nivel ${tier}). Termina
el ${S.end.getDate()} de ${MONTHS[S.end.getMonth()]}: entonces el pase vuelve a empezar.</p>
1030         ${Profile.passClaimable() ? '<div class="row"><button class="btn gold" type="button" data-
act="passall">Reclamar todo</button></div>' : ''}
1031         <div class="tiers">${tiers.join('')}</div>`;
1032     } else if (n === 'missions') {
1033       const row = (m, i, kind) => {
1034         const d = MISSION_BY_ID[m.id];
1035         const prog = Math.min(d.goal, m.prog);
1036         return `<div class="mission ${m.done ? 'done' : ''}"><span>${esc(d.text)}</span>
1037           <span>${m.done ? 'Hecha' : `${prog} / ${d.goal}`}${kind === 'daily' && !m.done && !P.missions.rerolled ? `
<button class="mini-btn" type="button" data-act="reroll-m" data-v="${i}">Cambiar</button>` : ''}</span>
1038           <div class="xpbar"><i style="width:${(prog / d.goal) * 100}%"></i></div></div>`;
1039       };
1040       html = head('Misiones') + `<p>Cada misión da ${XP_REWARD.mission} XP y cristales (${MISSION_CRYSTALS.daily} las
diarias, ${MISSION_CRYSTALS.weekly} las semanales). Se completan al terminar una partida.</p>
1041         <h3>Diarias${P.missions.rerolled ? ' · ya cambiaste una hoy' : ' · puedes cambiar 1 al día'}
</h3>${P.missions.daily.map((m, i) => row(m, i, 'daily')).join('')}
1042         <h3>Semanales</h3>${P.missions.weekly.map((m, i) => row(m, i, 'weekly')).join('')}`;
1043     } else if (n === 'daily') {
1044       const D = P.daily;
1045       const can = Profile.dailyClaimable();
1046       html = head('Premio diario') + `<p>Entra cada día para avanzar. Si faltas un día, la racha baja 1.${D.lost ? `
Faltaste y bajaste ${D.lost}.` : ''}</p>
1047         <div class="daygrid">${DAILY_REWARDS.map((r, i) => `<div class="day ${i < D.idx ? 'done' : ''} ${i === D.idx &&
can ? 'today' : ''} ${r === 'faro' ? 'big' : ''}">
1048           <span>Día ${i + 1}</span><b>${r === 'faro' ? 'Cofre' : r}</b><small>${r === 'faro' ? 'del Faro' :
'cristales'}</small></div>`).join('')}</div>
1049         <div class="row"><button class="btn gold" type="button" data-act="dailyclaim" ${can ? '' : 'disabled'}>${can ?
`Reclamar día ${D.idx + 1}` : 'Ya reclamaste hoy'}</button></div>`;
1050     } else if (n === 'codes') {
1051       html = head('Códigos') + `<form id="code-form" class="field"><label for="code-input">Escribe un código</label>
1052         <input type="text" id="code-input" autocomplete="off" autocapitalize="characters" spellcheck="false"
maxlength="20" placeholder="FARO2026">
1053         <button class="btn gold" type="submit">Canjear</button></form>
1054         ${this.modalData && this.modalData.msg ? `<div class="advice">${esc(this.modalData.msg)}</div>` : ''}
1055         <p>Cada código se usa una vez. Los códigos nuevos se anuncian en las actualizaciones.</p>`;
1056     } else if (n === 'backup') {
1057       const D = this.modalData || {};
1058       html = head('Copia de tu progreso') + `<p>Tu nivel, cristales, aspectos y rango se guardan solo en este
navegador. Con esta copia los llevas a otro iPad o navegador (en Roblox se guardarán en tu cuenta).</p>
1059         <div class="field"><label for="bk-out">1 · Guardar: copia este texto o descárgalo</label>
1060           <textarea id="bk-out" readonly rows="3" spellcheck="false">${esc(Profile.exportCode())}</textarea>
1061           <div class="row"><button class="btn gold" type="button" data-act="bk-copy" id="btn-bk-copy">Copiar</button>
1062           <button class="btn" type="button" data-act="bk-file" id="btn-bk-file">Descargar archivo</button></div></div>
1063         <div class="field"><label for="bk-in">2 · Cargar: pega aquí una copia (reemplaza tu progreso de este navegador)
</label>
1064           <textarea id="bk-in" rows="3" spellcheck="false" placeholder="FARO1...."></textarea>
1065           <div class="row"><button class="btn gold" type="button" data-act="bk-load" id="btn-bk-load">Cargar
copia</button>
1066           <label class="btn" for="bk-filein">Elegir archivo</label><input type="file" id="bk-filein"
accept=".txt,.json,text/plain,application/json" hidden></div></div>
1067         ${D.msg ? `<div class="advice" id="bk-msg">${esc(D.msg)}</div>` : ''}`;
1068     } else if (n === 'creator') {
1069       const C = P.creator;
1070       html = head('Modo creador') + `<p>Solo lo ves tú: la plataforma confirma que eres el dueño de este juego. Los
demás jugadores no pueden abrirlo.</p>
1071         <div class="field"><label>Monedas y energía infinitas · todo desbloqueado</label><div class="seg">
1072           <button type="button" data-act="creator" data-k="on" data-v="true" class="${C.on ? 'on' :
''}">Activado</button>
1073           <button type="button" data-act="creator" data-k="on" data-v="false" class="${!C.on ? 'on' :
''}">Apagado</button></div></div>
1074         <div class="field"><label>Construcción rápida</label><div class="seg">
1075           <button type="button" data-act="creator" data-k="instant" data-v="true" class="${C.instant ? 'on' :
''}">Sí</button>
1076           <button type="button" data-act="creator" data-k="instant" data-v="false" class="${!C.instant ? 'on' :
''}">No</button></div></div>
1077         <p>Las partidas en modo creador no dan XP ni cristales. En Roblox se hará igual, con tu UserId comprobado en el
servidor.</p>`;
1078     } else if (n === 'chestopen') {
1079       const r = this.modalData;
1080       if (r.kind === 'frags') {
1081         html = head(esc(r.chest)) + `<div class="item equipped" style="cursor:default"><div class="fragicon" aria-
hidden="true"></div>
1082           <b>${r.frags} fragmentos</b><small>Ahora tienes ${this.P.fragments}. Úsalos en el Taller con un plano.
</small></div>
1083           <div class="row"><button class="btn gold" type="button" data-act="close">Genial</button></div>`;
1084       } else if (r.kind === 'plan') {
1085         html = head(esc(r.chest)) + `<div class="item equipped plan" style="cursor:default">
1086           <img alt="" src="${Thumbs.get(SKIN_SLOTS[r.item.slot].model, PLAYER_COLORS[0], r.item.look)}">
1087           <b>Plano: ${esc(r.item.name)}</b><span class="rar ${r.item.rar}">${RARITY[r.item.rar].name}</span>
1088           <small>Fabrícalo en el Taller con ${FRAG_COST[r.item.rar]} fragmentos (tienes ${this.P.fragments}).</small>
```

```
</div>
          <div class="row"><button class="btn gold" type="button" data-act="close">Genial</button>
          <button class="btn" type="button" data-act="shoptab" data-v="workshop">Ir al Taller</button></div>`;
      } else {
        html = head(esc(r.chest)) + `<div class="item ${r.dup ? 'owned' : 'equipped'}" style="cursor:default">
          <img alt="" src="${Thumbs.get(SKIN_SLOTS[r.item.slot].model, PLAYER_COLORS[0], r.item.look)}">
          <b>${esc(r.item.name)}</b><span class="rar ${r.item.rar}">${RARITY[r.item.rar].name}</span>
          <small>${r.dup ? `Repetido: se convierte en ${r.frags} fragmentos` : '¡Nuevo! Ya está en tu Inventario'}
</small></div>
          <div class="row"><button class="btn gold" type="button" data-act="close">Genial</button>
          ${r.dup ? '' : `<button class="btn" type="button" data-act="equip" data-slot="${r.item.slot}" data-
id="${r.item.id}">Equipar</button>`}</div>`;
      }
    } else if (n === 'info') {
      html = head(esc(this.modalData.title)) + `<p>${esc(this.modalData.text)}</p><div class="row"><button class="btn
gold" type="button" data-act="close">Aceptar</button></div>`;
    }
    $('modal-body').innerHTML = html;
  }

  // ---------- acciones ----------
  onClick(e) {
    const b = e.target.closest('[data-act]');
    if (!b || b.disabled) return;
    const P = this.P;
    const act = b.dataset.act;
    const app = this.app;
    app.sound.unlock();
    app.sound.play('click');
    if (act === 'sub') {
      this.sub = b.dataset.sub;
      this.syncOpts();
      this.render();
    } else if (act === 'play') {
      this.syncOpts();
      app.startGame();
    } else if (act === 'toptab') {
      this.topTab = b.dataset.v;
      this.render();
    } else if (act === 'weekly') {
      this.sub = 'weekly';
      this.syncOpts();
      app.startGame();
    } else if (act === 'ranked') {
      if (Profile.level() < RANKED_LEVEL && !Creator.active()) {
        this.openModal('info', { title: 'Liga 3 vs 3', text: `La Liga 3 vs 3 se abre en el nivel ${RANKED_LEVEL}. Ahora
estás en el nivel ${Profile.level()}: juega otras partidas para subir.` });
        return;
      }
      this.P.play.mode = RANKED_MODE;
      Profile.save();
      this.sub = 'main';
      this.syncOpts();
      app.startGame();
    } else if (act === 'mission') {
      app.startMission(Number(b.dataset.v));
    } else if (act === 'reroll') {
      app.reroll();
    } else if (act === 'diff') {
      this.setPlay('difficulty', b.dataset.v);
    } else if (act === 'cset') {
      let v = b.dataset.v;
      if (v === 'true' || v === 'false') v = v === 'true';
      else if (/^\d+$/.test(v)) v = Number(v);
      this.setPlay(b.dataset.k, v);
    } else if (act === 'modal') {
      this.openModal(b.dataset.m);
    } else if (act === 'close') {
      this.closeModal();
    } else if (act === 'bk-copy') {
      const t = $('bk-out');
      t.select();
      const done = () => this.openModal('backup', { msg: 'Copiado. Pégalo en «Cargar» en el otro iPad o navegador.' });
      if (navigator.clipboard && navigator.clipboard.writeText) navigator.clipboard.writeText(t.value).then(done, () =>
{ try { document.execCommand('copy'); } catch (err) { /* nada */ } done(); });
      else { try { document.execCommand('copy'); } catch (err) { /* nada */ } done(); }
    } else if (act === 'bk-file') {
      this.saveBackupFile();
    } else if (act === 'bk-load') {
      this.loadBackup($('bk-in').value, b);
    } else if (act === 'pickmode') {
      P.play.mode = b.dataset.v;
      Profile.save();
      this.closeModal();
      this.syncOpts();
      this.render();
    } else if (act === 'pickmap') {
      P.play.mapKind = b.dataset.v;
```

```
      Profile.save();
      this.closeModal();
      this.syncOpts();
      this.render();
    } else if (act === 'resume') {
      app.resumeGame();
    } else if (act === 'load') {
      app.loadSavedGame();
    } else if (act === 'abandon') {
      app.abandonGame();
    } else if (act === 'passclaim') {
      this.claimTier(Number(b.dataset.v));
    } else if (act === 'passall') {
      for (let t = 1; t <= Profile.passTier(); t++) if (!P.pass[t]) this.claimTier(t, true);
      this.render();
    } else if (act === 'reroll-m') {
      Profile.rerollDaily(Number(b.dataset.v));
      this.render();
    } else if (act === 'dailyclaim') {
      this.claimDaily();
    } else if (act === 'shoptab') {
      this.shopTab = b.dataset.v;
      this.view = 'shop';
      if (this.modal) this.closeModal();
      this.render();
    } else if (act === 'craft') {
      const err = this.craft(b.dataset.id);
      if (err) this.openModal('info', { title: 'Taller', text: err });
      else this.openModal('info', { title: 'Taller', text: `¡Fabricaste ${SKIN_BY_ID[b.dataset.id].name}! Ya está en tu
Inventario.` });
      this.render();
    } else if (act === 'buy') {
      this.buySkin(b.dataset.id);
    } else if (act === 'chest') {
      this.openChest(b.dataset.v, false);
    } else if (act === 'equip') {
      P.equipped[b.dataset.slot] = b.dataset.id || null;
      Profile.save();
      if (this.modal === 'chestopen') this.closeModal();
      this.render();
    } else if (act === 'creator') {
      if (!Creator.owner) return;
      P.creator[b.dataset.k] = b.dataset.v === 'true';
      Profile.save();
      this.render();
    }
  }

  give(r) {
    const P = this.P;
    if (r.crystals) P.crystals += r.crystals;
    if (r.xp) Profile.addXp(r.xp);
    if (r.chest) return this.openChest(r.chest, true);
    return null;
  }

  claimTier(t, quiet) {
    const P = this.P;
    if (t < 1 || t > Profile.passTier() || P.pass[t]) return;
    P.pass[t] = true;
    const r = passReward(t);
    if (r.chest) this.openChest(r.chest, true, quiet);
    else this.give(r);
    Profile.save();
    if (!quiet) this.render();
  }

  claimDaily() {
    const P = this.P;
    if (!Profile.dailyClaimable()) return;
    const D = P.daily;
    const r = DAILY_REWARDS[D.idx];
    D.last = dayKey();
    D.lost = 0;
    D.idx = (D.idx + 1) % DAILY_REWARDS.length;
    if (r === 'faro') this.openChest('faro', true);
    else {
      P.crystals += r;
      this.app.ui.toast(`+${r} cristales`, 'good');
    }
    Profile.save();
    this.render();
  }

  redeem(raw) {
    const P = this.P;
    const code = String(raw || '').trim().toUpperCase().replace(/\s+/g, '');
    let msg;
```

```
    if (!code) msg = 'Escribe un código.';
    else if (!CODES[code]) msg = 'Ese código no existe.';
    else if (P.codes[code]) msg = 'Ya usaste este código.';
    else {
      P.codes[code] = dayKey();
      msg = `¡Listo! ${CODES[code].text}.`;
      const c = CODES[code];
      if (c.chest) {
        Profile.save();
        this.openChest(c.chest, true);
        return;
      }
      this.give(c);
    }
    Profile.save();
    this.modalData = { msg };
    this.render();
  }

  buySkin(id) {
    const P = this.P;
    const s = SKIN_BY_ID[id];
    if (!s || Profile.owns(id) || s.special) return;
    const price = RARITY[s.rar].price;
    if (P.crystals < price) return;
    P.crystals -= price;
    P.owned[id] = true;
    Profile.save();
    this.app.ui.toast(`Compraste ${s.name}`, 'good');
    this.render();
  }

  // Taller (v1.2): fabrica un aspecto con su plano y fragmentos.
  craft(id) {
    const P = this.P;
    const s = SKIN_BY_ID[id];
    if (!s || !(P.plans || {})[id]) return 'No tienes el plano de ese aspecto';
    if (P.owned[id]) return 'Ya tienes ese aspecto';
    const cost = FRAG_COST[s.rar];
    if ((P.fragments || 0) < cost) return `Te faltan ${cost - (P.fragments || 0)} fragmentos`;
    P.fragments -= cost;
    delete P.plans[id];
    P.owned[id] = true;
    Profile.save();
    return null;
  }

  openChest(kind, free, quiet) {
    const P = this.P;
    const C = CHESTS[kind];
    if (!C) return null;
    if (!free && !Creator.active()) {
      if (P.crystals < C.cost) return null;
      P.crystals -= C.cost;
    }
    P.chests.opened++;
    P.chests.pity++;
    let roll = Math.random() * 100;
    let rar = 'comun';
    for (const [r, pct] of Object.entries(C.odds)) {
      if (roll < pct) {
        rar = r;
        break;
      }
      roll -= pct;
    }
    // Garantía: si llevas 29 cofres sin legendario, el siguiente lo es.
    if (P.chests.pity >= CHEST_PITY) rar = 'legendario';
    if (rar === 'legendario') P.chests.pity = 0;
    const pool = SKINS.filter((s) => s.rar === rar && !s.special);
    const item = pool[Math.floor(Math.random() * pool.length)];
    // v1.2: aspecto, plano o fragmentos (la garantía del legendario siempre da el aspecto).
    P.fragments = P.fragments || 0;
    P.plans = P.plans || {};
    const split = rar === 'legendario' && P.chests.pity === 0 ? 0 : Math.random() * 100;
    let kindOut = split < CHEST_SPLIT.skin ? 'skin' : split < CHEST_SPLIT.skin + CHEST_SPLIT.plan ? 'plan' : 'frags';
    let planItem = null;
    if (kindOut === 'plan') {
      const cand = SKINS.filter((s) => !s.special && !P.owned[s.id] && !P.plans[s.id]);
      if (cand.length) planItem = cand[Math.floor(Math.random() * cand.length)];
      else kindOut = 'frags';
    }
    let frags = 0;
    let dup = false;
    if (kindOut === 'skin') {
      dup = !!P.owned[item.id];
      if (dup) frags = FRAG_DUP[rar];
      else P.owned[item.id] = true;
```

```
    } else if (kindOut === 'plan') {
      P.plans[planItem.id] = true;
    } else {
      frags = FRAG_PACK[0] + Math.floor(Math.random() * (FRAG_PACK[1] - FRAG_PACK[0] + 1));
    }
    P.fragments += frags;
    Profile.save();
    const result = { chest: C.name, item: kindOut === 'plan' ? planItem : item, dup, frags, kind: kindOut };
    if (!quiet) {
      this.openModal('chestopen', result);
      this.render();
    }
    return result;
  }

  // ---------- logros (v1.4) ----------
  checkAchievements(c) {
    const P = c.P;
    P.achievements = P.achievements || {};
    const got = [];
    for (const a of ACHIEVEMENTS) {
      if (P.achievements[a.id]) continue;
      let ok = false;
      try { ok = !!a.done(c); } catch (err) { ok = false; }
      if (ok) {
        P.achievements[a.id] = dayKey();
        got.push(a);
      }
    }
    return got;
  }

  achievementsHtml() {
    const P = this.P;
    const A = P.achievements || {};
    const n = ACHIEVEMENTS.filter((a) => A[a.id]).length;
    return `<section class="lcard panel" id="logros"><h3>Logros · ${n} de ${ACHIEVEMENTS.length}</h3><div
class="achs">${ACHIEVEMENTS.map((a) => {
      const on = !!A[a.id];
      const pr = !on && a.prog ? a.prog(P) : null;
      return `<div class="ach ${on ? 'on' : ''}" data-ach="${a.id}"><span class="achbadge" style="--
ac:${a.color}">${esc(a.icon)}</span>
        <div><b>${esc(a.name)}</b><small>${esc(a.desc)}</small>
        ${pr ? `<div class="xpbar"><i style="width:${Math.min(100, (pr[0] / pr[1]) * 100)}%"></i></div>
<small>${fmt(pr[0])} / ${fmt(pr[1])}</small>` : ''}
        <em>${on ? `Logrado el ${prettyDate(A[a.id])}` : a.crystals ? `${a.crystals} cristales` : 'Insignia'}</em>
</div></div>`;
    }).join('')}</div></section>`;
  }

  // ---------- Top jugadores (v1.4, tu documento pág. 20) ----------
  topHtml() {
    const P = this.P;
    const tab = this.topTab || 'liga';
    const rows = topRows(P, tab);
    const me = rows.find((r) => r.me);
    const S = seasonOf(Profile.now());
    const tabs = [['liga', 'Liga 3 vs 3'], ['wins', 'Victorias'], ['islands', 'Islas tomadas']]
      .map(([k, l]) => `<button type="button" data-act="toptab" data-v="${k}" class="${tab === k ? 'on' : ''}" aria-
pressed="${tab === k}">${l}</button>`).join('');
    const head = tab === 'liga' ? '<th>Rango</th><th>Puntos</th><th>% victorias</th>' : tab === 'wins' ?
'<th>Victorias</th>' : '<th>Islas tomadas</th>';
    const cells = (r) => (tab === 'liga' ? `<td>${rankName(r.points)}</td><td>${fmt(r.points)}</td>
<td>${Math.round(r.pct * 100)} %</td>`
      : tab === 'wins' ? `<td>${fmt(r.wins)}</td>` : `<td>${fmt(r.islands)}</td>`);
    const row = (r) => `<tr class="${r.me ? 'me' : ''}"><td>${r.pos <= 3 ? `<span class="medal m${r.pos}" aria-
label="${['Oro', 'Plata', 'Bronce'][r.pos - 1]}">${r.pos}</span>` : r.pos}</td>
      <td>${esc(r.name)}${r.bot ? ' <small>bot</small>' : ' <small>(tú)</small>'}</td>${cells(r)}</tr>`;
    const top = rows.slice(0, 10);
    const body = top.map(row).join('') + (me.pos > 10 ? `<tr class="gap"><td colspan="5">···</td></tr>${row(me)}` :
'');
    return `<section class="lcard panel"><h2>Top jugadores · Temporada ${S.no}</h2>
      <p class="queue">${tab === 'liga' ? 'Los mejores de la Liga 3 vs 3 en esta temporada.' : tab === 'wins' ? 'Más
victorias (todos los modos).' : 'Más islas tomadas (todos los modos).'} Sin servidores, los demás son comandantes bot; en
Roblox serán jugadores reales. Tu fila siempre se ve.</p>
      <div class="subtabs">${tabs}</div>
      <table class="toptable" id="toptable"><thead><tr><th>#</th><th>Jugador</th>${head}</tr></thead><tbody>${body}
</tbody></table></section>`;
  }

  // ---------- desafío semanal (v1.4) ----------
  weeklyHtml() {
    const W = weeklyFor(Profile.now());
    const rec = (this.P.weekly || {})[W.week] || { tries: 0, won: false, best: null };
    const days = Math.max(1, Math.round((W.next - new Date(Profile.now().getFullYear(), Profile.now().getMonth(),
Profile.now().getDate())) / 86400000));
    return `<section class="lcard panel weekly"><h2>Desafío semanal: ${esc(W.name)}</h2>
      <div class="weeklybox"><img class="mapthumb" alt="" src="${mapThumb(W.mapKind, MODES[W.mode].players)}">
```

```
      <div><p id="weekly-desc">${esc(W.desc)}</p>
      <p class="queue">${esc(MODES[W.mode].name)} · ${esc(MAPS[W.mapKind].name)} · bots en ${DIFFICULTY[W.difficulty].name}. Es el mismo mapa toda la semana; cambia en ${days} ${days === 1 ? 'día' : 'días'} (el lunes).</p>
      <p>${rec.won ? `<b>Ganado.</b> Tu mejor tiempo: <b id="weekly-best">${fmtTime(rec.best)}</b>. Intenta bajarlo.` : `Premio la primera vez que lo ganas: <b>${WEEKLY_CRYSTALS} cristales</b>.`} Intentos: ${rec.tries}.</p>
      <div class="row"><button class="btn gold" type="button" data-act="weekly" id="btn-weekly">Jugar desafío</button></div></div></div></section>`;
  }

  // ---------- Liga 3 vs 3 (v1.3) ----------
  // Gana +25, pierde −15. Recién llegado a un rango, la primera derrota no te baja de él.
  rankedResult(win, forfeit = false) {
    const R = this.P.ranked;
    const before = R.points;
    const idx0 = rankIndex(before);
    let delta = win ? RANK_WIN : -Math.min(RANK_LOSS, before);
    let saved = false;
    if (!win && R.shield && rankIndex(before + delta) < idx0) {
      delta = RANK_MIN[idx0] - before;
      saved = true;
    }
    if (!win) R.shield = false;
    R.points = Math.max(0, before + delta);
    if (rankIndex(R.points) > idx0) R.shield = true;
    R.best = Math.max(R.best || 0, R.points);
    R.games = (R.games || 0) + 1;
    if (win) R.wins = (R.wins || 0) + 1;
    R.history = [{ day: dayKey(), win, delta, rank: rankName(R.points), forfeit }, ...(R.history || [])].slice(0, 12);
    Profile.save();
    return { before, after: R.points, delta, from: rankName(before), to: rankName(R.points), saved, forfeit };
  }

  rankedLine(r) {
    return `Liga 3 vs 3${r.forfeit ? ' (abandonada)' : ''}: ${r.delta >= 0 ? '+' : ''}${r.delta} puntos · ${r.from === r.to ? r.to : `${r.from} → ${r.to}`}${r.saved ? ' · protección de rango: no bajaste' : ''}`;
  }

  // Abandonar una partida de la Liga (o empezar otra encima) cuenta como derrota.
  forfeitRanked() {
    const r = this.rankedResult(false, true);
    this.app.ui.toast(this.rankedLine(r), 'bad');
    return r;
  }

  // ---------- final de partida ----------
  finish(game) {
    if (game.rewarded) return game.reward || null;
    game.rewarded = true;
    const P = this.P;
    const me = game.players[0];
    const win = game.result === 'victory';
    const out = { xp: 0, crystals: 0, lines: [], missions: [], levelUp: false, creator: !!game.creator, tutorial: null };
    if (game.creator) {
      game.reward = out;
      return out;
    }
    Profile.refresh();
    const lv0 = Profile.level();
    const half = game.opts.custom ? 0.5 : 1;
    const parts = [
      ['Jugar', XP_REWARD.play],
      ['Ganar', win ? XP_REWARD.win : 0],
      [`Islas tomadas (${me.stats.captures})`, me.stats.captures * XP_REWARD.island],
      [`Núcleos destruidos (${me.coresKilled})`, me.coresKilled * XP_REWARD.core],
    ];
    for (const [label, n] of parts) {
      const v = Math.round(n * half);
      if (v > 0) {
        out.xp += v;
        out.lines.push(`${label}: +${v} XP`);
      }
    }
    if (half < 1) out.lines.push('Personalizado: mitad de XP');
    out.crystals = Math.round((GAME_CRYSTALS.play + (win ? GAME_CRYSTALS.win : 0)) * half);
    // v1.2: fragmentos por jugar.
    out.frags = Math.round((FRAG_GAME.play + (win ? FRAG_GAME.win : 0)) * half);
    P.fragments = (P.fragments || 0) + out.frags;
    if (out.frags) out.lines.push(`Fragmentos: +${out.frags}`);
    P.stats.games++;
    if (win) P.stats.wins++;
    P.stats.islands += me.stats.captures;
    P.stats.cores += me.coresKilled;
    P.stats.kills += me.stats.kills;
    P.stats.trained = (P.stats.trained || 0) + me.stats.trained;
    P.stats.faro = (P.stats.faro || 0) + (me.stats.faro || 0);
    // v1.4: desafío semanal: mejor tiempo y premio la primera vez que lo ganas.
```

```
    if (game.opts.weekly) {
      const W = P.weekly[game.opts.weekly] || (P.weekly[game.opts.weekly] = { tries: 0, won: false, best: null });
      W.tries++;
      if (win) {
        if (!W.won) {
          out.crystals += WEEKLY_CRYSTALS;
          out.lines.push(`Desafío semanal ganado: +${WEEKLY_CRYSTALS} cristales`);
        }
        W.won = true;
        W.best = W.best === null ? Math.round(game.time) : Math.min(W.best, Math.round(game.time));
        out.weekly = W.best;
      }
    }
    // Misiones: suman lo que hiciste en esta partida.
    const delta = { games: 1, wins: win ? 1 : 0, built: me.stats.built, trained: me.stats.trained, captures:
me.stats.captures,
      kills: me.stats.kills, upgrades: me.stats.upgrades, cores: me.coresKilled };
    for (const kind of ['daily', 'weekly']) {
      for (const m of P.missions[kind]) {
        if (m.done) continue;
        const d = MISSION_BY_ID[m.id];
        m.prog += delta[d.stat] || 0;
        if (m.prog >= d.goal) {
          m.done = true;
          out.xp += XP_REWARD.mission;
          out.crystals += MISSION_CRYSTALS[kind];
          out.missions.push(d.text);
        }
      }
    }
    // Tutorial: premio único al terminarlo por primera vez.
    if (game.mode.tutorial && win && !P.tutorialDone) {
      P.tutorialDone = true;
      P.badges.primerFaro = true;
      P.owned.dorado_inf = true;
      out.crystals += 150;
      out.tutorial = 'Insignia «Primer Faro», 150 cristales y el aspecto Soldado Dorado';
    }
    // Campaña: premio la primera vez que terminas la misión y cristales por cada estrella nueva.
    if (game.camp) {
      const m = Campaign.def(game.camp.id);
      const got = Campaign.stars(game).filter((s) => s.ok).length;
      const prev = P.campaign.stars[m.id] || 0;
      out.camp = { id: m.id, got, prev, first: 0, newStars: 0 };
      if (win && !P.campaign.done[m.id]) {
        P.campaign.done[m.id] = true;
        out.camp.first = m.crystals;
        out.crystals += m.crystals;
      }
      if (got > prev) {
        out.camp.newStars = got - prev;
        out.crystals += out.camp.newStars * CAMPAIGN_STAR_CRYSTALS;
        P.campaign.stars[m.id] = got;
      }
    }
    // Liga 3 vs 3: puntos y rango.
    if (game.mode.ranked && !game.opts.custom) {
      out.ranked = this.rankedResult(win);
      out.lines.push(this.rankedLine(out.ranked));
    }
    // v1.4: logros nuevos de esta partida.
    out.achievements = this.checkAchievements({ P, g: game, me, win });
    for (const a of out.achievements) {
      out.crystals += a.crystals;
      out.lines.push(`Logro: ${a.name}${a.crystals ? ` (+${a.crystals} cristales)` : ''}`);
    }
    Profile.addXp(out.xp);
    P.crystals += out.crystals;
    out.level = Profile.level();
    out.levelUp = out.level > lv0;
    Profile.save();
    game.reward = out;
    return out;
  }

  positions(game) {
    // Solo hay «Ganador» si queda un equipo (o en Rey del Faro); si siguen varios, todos «Seguía en pie».
    const aliveTeams = new Set(game.players.filter((p) => p.alive).map((p) => p.team));
    const winTeam = game.kingWinner !== undefined ? game.kingWinner : aliveTeams.size === 1 ? [...aliveTeams][0] :
-999;
    const alive = game.players.filter((p) => p.alive).sort((a, b) => (b.team === winTeam) - (a.team === winTeam));
    const list = [];
    let rank = 0;
    let lastWin = null;
    alive.forEach((p, i) => {
      const w = p.team === winTeam;
      if (!(w && lastWin)) rank = i + 1;
      lastWin = w;
```

```
      list.push({ p, rank, note: w ? 'Ganador' : 'Seguía en pie' });
    });
    for (let i = game.elimOrder.length - 1; i >= 0; i--) {
      const e = game.elimOrder[i];
      list.push({ p: game.players[e.pid], rank: list.length + 1, note: `Cayó a los ${fmtTime(e.time)}` });
    }
    return list;
  }

  advice(game) {
    const hb = game.players[0].stats.hitBy || {};
    const groups = {};
    for (const [k, v] of Object.entries(hb)) {
      const d = UNITS[k];
      const g = k === 'bld' ? 'bld' : !d ? 'other' : k === 'arty' ? 'arty' : d.cls === 'inf' ? 'inf' : d.cls === 'veh'
? 'veh' : d.cls === 'air' ? 'air' : 'sea';
      groups[g] = (groups[g] || 0) + v;
    }
    const top = Object.entries(groups).sort((a, b) => b[1] - a[1])[0];
    const tips = {
      bld: 'Las torretas y defensas enemigas te hicieron mucho daño. Atácalas desde lejos con Tanques o Lanzamisiles.',
      inf: 'La infantería enemiga te desgastó. Prueba Escopeteros, Jeeps o Torretas mejoradas con miniguns.',
      veh: 'Los vehículos enemigos te superaron. Los Cazadores y Coheteros hacen más daño a tanques.',
      air: 'Te atacaron desde el aire. Construye AIR DEFENSE y Camiones antiaéreos cerca del Núcleo.',
      sea: 'Los barcos atacaron tu costa. Pon Torretas en la costa, ten Destructores y mejora el Núcleo para la
Cúpula.',
      arty: 'Los Lanzamisiles te atacaron de lejos. Sube el Núcleo a nivel 3 y compra la Cúpula.',
    };
    if (top && tips[top[0]]) return tips[top[0]];
    return 'Entrena Constructores pronto y construye más Granjas y energía: con más monedas tendrás un ejército más
grande.';
  }

  // Final de una misión: estrellas, por qué faltó cada una y «Siguiente misión».
  missionEndHtml(game) {
    const r = this.finish(game);
    const m = Campaign.def(game.camp.id);
    const me = game.players[0];
    const win = game.result === 'victory';
    const stars = Campaign.stars(game);
    if (!win) stars[0].why = 'Tu Núcleo cayó';
    const got = stars.filter((s) => s.ok).length;
    const next = Campaign.def(m.id + 1);
    const nextOpen = !!next && Campaign.unlocked(next.id, this.P, Creator.active());
    const c = r.camp;
    const opened = nextOpen && !!c && c.first > 0;
    const text = !win ? 'Tu Núcleo cayó. Puedes repetir la misión cuando quieras.'
      : next ? `Misión cumplida.${opened ? ` Se abrió la misión ${next.id}: ${next.name}.` : ''}`
        : '¡Terminaste la campaña! El Faro de Ámbar vuelve a brillar.';
    const why = c ? [c.first ? `Primera vez: ${c.first}` : '', c.newStars ? `${c.newStars === 1 ? '1 estrella nueva' :
`${c.newStars} estrellas nuevas`}: ${c.newStars * CAMPAIGN_STAR_CRYSTALS}` : '',
      r.missions.length ? 'misiones' : ''].filter(Boolean).join(' · ') : '';
    const prizes = r.creator
      ? '<div><b>Modo creador</b><span>Sin XP ni cristales</span></div>'
      : `<div><b>+${fmt(r.xp)} XP</b><span>${r.lines.map(esc).join(' · ')}</span></div>
         <div><b>+${fmt(r.crystals)} cristales</b><span>${esc(why || 'Por jugar')}</span></div>
         <div><b>Nivel ${r.level}</b><span>${r.levelUp ? '¡Subiste de nivel!' : `${fmt(this.P.xp % LEVEL_XP)} /
${fmt(LEVEL_XP)} XP`}</span></div>`;
    const core = me.core;
    return `
      <div class="endhero"><div><small class="lbl">Campaña · Misión ${m.id} de ${CAMPAIGN.length}</small>
        <h2>${win ? '¡Misión cumplida!' : 'Misión fallida'}</h2><p>${esc(m.name)} · ${esc(text)}</p></div>
        <div class="bigstars" role="img" aria-label="${got} de 3 estrellas">${stars.map((s) => `<i class="${s.ok ? 'on'
: ''}">★</i>`).join('')}</div></div>
      <ol class="starlist">${stars.map((s) => `<li class="${s.ok ? 'ok' : 'no'}"><b>${esc(s.text)}</b><span>${s.ok ?
'Conseguida' : esc(s.why)}</span></li>`).join('')}</ol>
      <div class="endstats">
        <div><b>${fmtTime(game.time)}</b><span>Tiempo</span></div>
        <div><b>${me.stats.lost}</b><span>Tropas perdidas</span></div>
        <div><b>${game.camp.lostBld}</b><span>Edificios perdidos</span></div>
        <div><b>${core.alive ? Math.round((core.hp / core.maxHp) * 100) : 0} %</b><span>Vida del Núcleo</span></div>
      </div>
      <h3 class="lbl">Premios</h3>
      <div class="prizes">${prizes}</div>
      ${r.missions.length ? `<div class="advice">Misiones completadas: ${r.missions.map(esc).join(', ')}</div>` : ''}
      ${win ? '' : `<div class="advice"><b>Consejo:</b> ${esc(m.tip)}</div>`}
      <div class="row">
        <button class="btn" type="button" id="btn-watch">Mirar el final</button>
        ${win && next ? `<button class="btn gold" type="button" id="btn-next" data-v="${next.id}">Siguiente
misión</button>` : ''}
        <button class="btn ${win && next ? '' : 'gold'}" type="button" id="btn-again">Repetir</button>
        <button class="btn" type="button" id="btn-menu">Vestíbulo</button>
      </div>`;
  }

  // ---------- gráficas del final (v1.1) ----------
  // Dos gráficas (no un solo eje doble): monedas por minuto y tropas de combate de cada imperio.
  // Cada línea tiene el color de su imperio y además su propio trazo y su nombre al final.
```

```
  historyHtml(game) {
    const H = game.history || [];
    if (H.length < 2) return '';
    const ids = game.players.map((p) => p.id);
    return `<h3 class="lbl">Cómo fue la partida</h3>
      <div class="histcharts">${this.historyChart(game, 'inc', 'Monedas por minuto')}${this.historyChart(game, 'army', 'Tropas de combate')}</div>
      <div class="histlegend">${ids.map((i) => `<span><svg width="26" height="8" aria-hidden="true"><line x1="1" y1="4" x2="25" y2="4" stroke="${cssColor(game.players[i].color)}" stroke-width="2" stroke-dasharray="${HIST_DASH[i]}" stroke-linecap="round"/></svg>${esc(game.players[i].name)}</span>`).join('')}</div>
      <details class="histtable"><summary>Ver como tabla</summary><table><thead><tr><th>Imperio</th><th>Monedas/min (final)</th><th>Monedas/min (máx.)</th><th>Tropas (máx.)</th></tr></thead><tbody>
        ${ids.map((i) => `<tr><td>${esc(game.players[i].name)}</td><td>${fmt(H[H.length - 1].inc[i])}</td><td>${fmt(Math.max(...H.map((h) => h.inc[i])))}</td><td>${Math.max(...H.map((h) => h.army[i]))}</td></tr>`).join('')}
      </tbody></table></details>`;
  }

  historyChart(game, key, title) {
    const H = game.history;
    const W = 560;
    const Ht = 170;
    const L = 40;
    const R = 64;
    const T = 10;
    const B = 22;
    const tMax = Math.max(H[H.length - 1].t, 1);
    const vMax = Math.max(1, ...H.map((h) => Math.max(...h[key])));
    const nice = Math.pow(10, Math.floor(Math.log10(vMax)));
    const yMax = Math.ceil(vMax / nice) * nice;
    const x = (t) => L + (t / tMax) * (W - L - R);
    const y = (v) => T + (1 - v / yMax) * (Ht - T - B);
    const grid = [0, 0.5, 1].map((k) => `<line x1="${L}" x2="${W - R}" y1="${y(yMax * k)}" y2="${y(yMax * k)}" class="hgrid"/>
      <text x="${L - 6}" y="${y(yMax * k) + 4}" text-anchor="end" class="haxis">${fmt(Math.round(yMax * k))}</text>`).join('');
    const mins = Math.max(1, Math.round(tMax / 60));
    const step = mins > 20 ? 10 : mins > 8 ? 5 : mins > 3 ? 2 : 1;
    let xt = '';
    for (let m = 0; m * 60 <= tMax; m += step) xt += `<text x="${x(m * 60)}" y="${Ht - 6}" text-anchor="middle" class="haxis">${m} min</text>`;
    // Nombres al final de cada línea; si se tocan, se separan un poco.
    const ends = game.players.map((p) => ({ id: p.id, v: H[H.length - 1][key][p.id] })).sort((a, b) => b.v - a.v);
    let lastY = -99;
    const labels = ends.map((e) => {
      let ly = y(e.v) + 4;
      if (ly - lastY < 11) ly = lastY + 11;
      lastY = ly;
      return `<text x="${W - R + 6}" y="${ly}" class="hend">${esc(game.players[e.id].name)}</text>`;
    }).join('');
    const lines = game.players.map((p) => {
      const pts = H.map((h) => `${x(h.t).toFixed(1)},${y(h[key][p.id]).toFixed(1)}`).join(' ');
      return `<polyline points="${pts}" fill="none" stroke="${cssColor(p.color)}" stroke-width="${p.id === 0 ? 2.5 : 2}" stroke-dasharray="${HIST_DASH[p.id]}" stroke-linejoin="round" stroke-linecap="round"/>`;
    }).join('');
    return `<figure class="histchart" data-key="${key}"><figcaption>${title}</figcaption>
      <div class="histwrap"><svg viewBox="0 0 ${W} ${Ht}" role="img" aria-label="${title} de cada imperio a lo largo de la partida">
        ${grid}${xt}${lines}${labels}<line class="hcross" x1="0" x2="0" y1="${T}" y2="${Ht - B}" visibility="hidden"/>
        <rect class="hhit" x="${L}" y="${T}" width="${W - L - R}" height="${Ht - T - B}" fill="transparent"/></svg>
        <div class="histtip" hidden></div></div></figure>`;
  }

  endHtml(game) {
    if (game.camp) return this.missionEndHtml(game);
    const r = this.finish(game);
    const me = game.players[0];
    const s = me.stats;
    const win = game.result === 'victory';
    const pos = this.positions(game);
    const winnerName = game.spectateWinner >= 0 && game.players[game.spectateWinner] ? game.players[game.spectateWinner].name : null;
    const text = game.kingWinner !== undefined
      ? (win ? `Tu equipo mantuvo el Faro ${fmtTime(KING_TIME)}.` : 'Otro imperio mantuvo el Faro el tiempo necesario.')
      : (win ? 'Todos los Núcleos rivales cayeron. El archipiélago es tuyo.' : winnerName ? `Tu Núcleo fue destruido. Seguiste mirando: ganó ${winnerName}.` : 'Tu Núcleo fue destruido.');
    const prizes = r.creator
      ? '<div><b>Modo creador</b><span>Sin XP ni cristales</span></div>'
      : `<div><b>+${fmt(r.xp)} XP</b><span>${r.lines.map(esc).join(' · ')}</span></div>
         <div><b>+${fmt(r.crystals)} cristales</b><span>${r.missions.length ? 'Incluye misiones' : 'Por jugar' + (win ? ' y ganar' : '')}</span></div>
         <div><b>Nivel ${r.level}</b><span>${r.levelUp ? '¡Subiste de nivel!' : `${fmt(this.P.xp % LEVEL_XP)} / ${fmt(LEVEL_XP)} XP`}</span></div>`;
    return `
      <div class="endhero"><div><h2>${win ? '¡Victoria!' : 'Derrota'}</h2><p>${esc(text)}</p></div></div>
      <div class="endstats">
        <div><b>${fmtTime(game.time)}</b><span>Tiempo</span></div>
        <div><b>${s.captures}</b><span>Islas tomadas</span></div>
```

```
        <div><b>${s.trained}</b><span>Tropas entrenadas</span></div>
        <div><b>${me.coresKilled}</b><span>Núcleos destruidos</span></div>
      </div>
      <h3 class="lbl">Posiciones</h3>
      <div class="positions">${pos.map((x) => `<div class="${x.p.id === 0 ? 'me' : ''}"><b>${x.rank}.º</b><i
style="background:${cssColor(x.p.color)}"></i>
        <span>${esc(x.p.name)}</span><small>${esc(x.note)}</small></div>`).join('')}</div>
      <h3 class="lbl">Premios</h3>
      <div class="prizes">${prizes}</div>
      ${r.missions.length ? `<div class="advice">Misiones completadas: ${r.missions.map(esc).join(', ')}</div>` : ''}
      ${r.tutorial ? `<div class="advice">Premio del Tutorial: ${esc(r.tutorial)}</div>` : ''}
      ${win ? '' : `<div class="advice"><b>Consejo:</b> ${esc(this.advice(game))}</div>`}
      ${this.historyHtml(game)}
      <div class="row">
        ${game.canSpectate() ? '<button class="btn" type="button" id="btn-spectate">Seguir mirando</button>' : ''}
        <button class="btn" type="button" id="btn-watch">Mirar el mapa</button>
        <button class="btn gold" type="button" id="btn-again">Revancha</button>
        <button class="btn" type="button" id="btn-menu">Vestíbulo</button>
      </div>`;
  }
}
```

faro/src/js/086-music.js · 190 líneas

```
'use strict';
// Música propia (v1.1): melodías generadas por el juego con Web Audio, sin archivos.
// Vestíbulo: tranquila (acordes suaves y arpegio). Partida: más rápida, con bajo; cuando hay
// combate se suman bombo, caja y platillos. Volumen guardado en el aparato.

const MUSIC_THEMES = {
  // Acordes por compás (notas MIDI) y tempo.
  lobby: { bpm: 84, chords: [[57, 60, 64], [53, 57, 60], [48, 52, 55], [55, 59, 62]], bass: false, arp: [0, 1, 2, 1, 0, 1, 2, 1] },
  battle: { bpm: 112, chords: [[50, 53, 57], [46, 50, 53], [48, 52, 55], [45, 49, 52]], bass: true, arp: [0, 2, 1, 2, 0, 2, 1, 2] },
};
const MUSIC_LEVELS = [0, 0.3, 0.6, 1];

class Music {
  constructor(sound) {
    this.sound = sound;
    this.theme = null;
    this.intensity = 0;
    this.step = 0;
    this.nextT = 0;
    this.timer = null;
    this.level = 0.6;
    try {
      const v = Number(localStorage.getItem('faro-music'));
      if (localStorage.getItem('faro-music') !== null && MUSIC_LEVELS.includes(v)) this.level = v;
    } catch (err) { /* sin almacenamiento */ }
    this.notes = 0;
  }

  get playing() {
    return !!this.timer;
  }

  setLevel(v) {
    this.level = v;
    try { localStorage.setItem('faro-music', String(v)); } catch (err) { /* sin almacenamiento */ }
    if (this.gain) this.gain.gain.setTargetAtTime(this.volume(), this.sound.ctx.currentTime, 0.2);
    if (v > 0 && this.theme && !this.timer) this.start(this.theme);
    if (v === 0) this.stop();
  }

  volume() {
    return this.sound.on ? this.level * 0.35 : 0;
  }

  // Elige la melodía (lobby o battle) y empieza si el audio ya está desbloqueado.
  play(theme) {
    this.theme = theme;
    if (!this.sound.ctx || this.level === 0) return;
    if (this.timer && this.current === theme) return;
    this.start(theme);
  }

  start(theme) {
    const c = this.sound.ctx;
    if (!c) return;
    if (!this.gain) {
      this.gain = c.createGain();
      this.gain.connect(this.sound.master);
    }
    this.gain.gain.setTargetAtTime(this.volume(), c.currentTime, 0.3);
    this.current = theme;
    this.step = 0;
    this.nextT = c.currentTime + 0.1;
    clearInterval(this.timer);
    this.timer = setInterval(() => this.tick(), 90);
  }

  stop() {
    clearInterval(this.timer);
    this.timer = null;
    this.current = null;
  }

  // 0 = calma, 1 = combate: suma percusión en la partida. Sube o baja con el tiempo real (no con los cuadros).
  setIntensity(k) {
    this.target = Math.max(0, Math.min(1, k));
  }

  refreshVolume() {
    if (this.gain && this.sound.ctx) this.gain.gain.setTargetAtTime(this.volume(), this.sound.ctx.currentTime, 0.2);
  }

```

```
  midi(n) {
    return 440 * Math.pow(2, (n - 69) / 12);
  }

  tick() {
    const c = this.sound.ctx;
    if (!c || !this.current) return;
    if (c.state !== 'running') return;
    this.intensity += ((this.target || 0) - this.intensity) * 0.12;
    const T = MUSIC_THEMES[this.current];
    const eighth = 60 / T.bpm / 2;
    while (this.nextT < c.currentTime + 0.35) {
      this.playStep(T, this.step, this.nextT, eighth);
      this.step = (this.step + 1) % 64;
      this.nextT += eighth;
    }
  }

  playStep(T, step, t, eighth) {
    const bar = Math.floor(step / 16) % T.chords.length;
    const inBar = step % 16;
    const chord = T.chords[bar];
    // Acorde de fondo al empezar cada compás.
    if (inBar === 0) for (const n of chord) this.pad(this.midi(n), t, eighth * 16);
    // Arpegio una octava arriba en corcheas.
    if (inBar % 2 === 0 || this.current === 'battle') {
      const n = chord[T.arp[inBar % 8]] + 12;
      this.pluck(this.midi(n), t, eighth * 1.6, this.current === 'battle' ? 0.05 : 0.06);
    }
    if (T.bass && inBar % 4 === 0) this.bass(this.midi(chord[0] - 12), t, eighth * 3.2);
    if (this.current === 'battle') {
      const k = this.intensity;
      if (k > 0.25 && inBar % 8 === 0) this.kick(t);
      if (k > 0.25 && inBar % 8 === 4) this.snare(t);
      if (k > 0.55 && inBar % 2 === 1) this.hat(t);
    }
  }

  env(t, a, d, peak) {
    const c = this.sound.ctx;
    const g = c.createGain();
    g.gain.setValueAtTime(0.0001, t);
    g.gain.linearRampToValueAtTime(peak, t + a);
    g.gain.exponentialRampToValueAtTime(0.0001, t + a + d);
    g.connect(this.gain);
    return g;
  }

  osc(type, f, t, dur, g) {
    const o = this.sound.ctx.createOscillator();
    o.type = type;
    o.frequency.setValueAtTime(f, t);
    o.connect(g);
    o.start(t);
    o.stop(t + dur + 0.05);
    this.notes++;
    return o;
  }

  pad(f, t, dur) {
    const g = this.env(t, dur * 0.3, dur * 0.7, 0.035);
    this.osc('triangle', f, t, dur, g);
    this.osc('triangle', f * 1.004, t, dur, g);
  }

  pluck(f, t, dur, vol) {
    this.osc('sine', f, t, dur, this.env(t, 0.01, dur, vol));
  }

  bass(f, t, dur) {
    const c = this.sound.ctx;
    const g = this.env(t, 0.02, dur, 0.09);
    const lp = c.createBiquadFilter();
    lp.type = 'lowpass';
    lp.frequency.value = 420;
    lp.connect(g);
    this.osc('sawtooth', f, t, dur, lp);
  }

  kick(t) {
    this.drums = (this.drums || 0) + 1;
    const g = this.env(t, 0.005, 0.25, 0.22);
    const o = this.osc('sine', 120, t, 0.3, g);
    o.frequency.exponentialRampToValueAtTime(42, t + 0.22);
  }

  noise(t, dur, type, freq, vol) {
    const c = this.sound.ctx;
```

```
    if (!this.sound.noise) return;
    const src = c.createBufferSource();
    src.buffer = this.sound.noise;
    const f = c.createBiquadFilter();
    f.type = type;
    f.frequency.value = freq;
    src.connect(f);
    f.connect(this.env(t, 0.003, dur, vol));
    src.start(t, Math.random() * 0.5, dur + 0.05);
    this.notes++;
  }

  snare(t) {
    this.noise(t, 0.16, 'bandpass', 1800, 0.12);
  }

  hat(t) {
    this.noise(t, 0.05, 'highpass', 7000, 0.05);
  }
}
```

faro/src/js/09-main.js · 595 líneas

```
'use strict';
// Arranque y bucle principal.

const App = {
  state: 'menu',
  game: null,
  renderer: null,
  gl: null,
  input: null,
  ui: null,
  sound: null,
  selection: [],
  paused: false,
  quality: 'high', // calidad que se usa ahora: 'high' o 'low'
  qualityPref: 'auto', // la elegida en la pausa: 'auto', 'high' o 'low'
  showFps: false,
  fps: 0,
  fpsFrames: 0,
  fpsT: 0,
  lowFpsSecs: 0,
  opts: { mode: 'conquest', mapKind: 'archipelago', difficulty: 'normal', seed: 0 },
  suspended: null,
  lobby: null,
  // Controles v0.7: grupos 1-3 (tropas guardadas) y formación al mover.
  groups: { 1: [], 2: [], 3: [] },
  groupTap: { n: 0, t: 0 },
  formation: 'block',
  // v1.3: guardado automático cada 60 s (antes 15) y al pausar, salir o cambiar de app.
  saveT: 60,
  last: 0,
  time: 0,
  miniT: 0,
  endTimer: null,

  init(hotData) {
    window.FARO = this;
    this.canvas = $('gl');
    this.overlay = $('overlay');
    try {
      const q = localStorage.getItem('faro-quality');
      if (q === 'high' || q === 'low' || q === 'auto') this.qualityPref = q;
      this.showFps = localStorage.getItem('faro-fps') === 'on';
      if (localStorage.getItem('faro-formation') === 'line') this.formation = 'line';
      // v1.4: ahorro de batería, colores para daltonismo y letra grande.
      this.battery = localStorage.getItem('faro-battery') === '1';
      this.cvd = localStorage.getItem('faro-cvd') === '1';
      this.bigText = localStorage.getItem('faro-bigtext') === '1';
      // v1.5: ya no hay decoraciones de temporada (Calabazas, Invierno).
      localStorage.removeItem('faro-season');
    } catch (err) { /* sin almacenamiento */ }
    applyPalette(this.cvd);
    document.body.classList.toggle('bigtext', !!this.bigText);
    Profile.load();
    this.quality = this.qualityPref === 'low' ? 'low' : 'high';
    if (hotData && hotData.opts && MODES[hotData.opts.mode]) Object.assign(this.opts, hotData.opts);
    this.opts.seed = (Math.random() * 1e9) | 0;
    // Al republicar la página, conserva las opciones elegidas en el menú.
    if (window.claude && window.claude.hot && window.claude.hot.snapshot) {
      window.claude.hot.snapshot(() => ({ opts: { ...this.opts } }));
    }

    try {
      this.gl = new THREE.WebGLRenderer({ canvas: this.canvas, antialias: true, powerPreference: 'high-performance' });
    } catch (err) {
      document.body.insertAdjacentHTML('beforeend',
        '<div class="screen"><div class="dialog panel"><h2>Sin 3D</h2><p>Tu navegador no pudo iniciar WebGL. Prueba con Safari o Chrome actualizados.</p></div></div>');
      return;
    }
    Thumbs.init();
    this.sound = new Sound();
    // Música propia (v1.1): empieza con el primer toque (los navegadores no dejan antes).
    this.music = new Music(this.sound);
    this.sound.onUnlock = () => this.music.play(this.state === 'play' ? 'battle' : 'lobby');
    this.sound.onToggle = () => { if (this.sound.on) this.music.play(this.state === 'play' ? 'battle' : 'lobby'); else this.music.stop(); this.music.refreshVolume(); };
    // Primero los modelos de Blender; si fallan, se usan los modelos de código.
    this.ready = Promise.all([loadGLBModels().catch(() => 0), loadEnvMap(this.gl).catch(() => null)]).then(([n]) => {
      this.modelCount = n;
      this.ui = new UI(this);
      this.input = new Input(this);
      this.lobby = new Lobby(this);
      this.makePreview();
      this.ui.showScreen('menu');
```

```
    this.loaded = true;
    // Modo creador: solo si la plataforma confirma que quien abre el juego es su dueño.
    Creator.detect().then(() => this.lobby.render());
  });

  window.addEventListener('resize', () => this.renderer && this.renderer.resize());
  // Safari en iPad: evita que el gesto de pellizco haga zoom a toda la página.
  document.addEventListener('gesturestart', (e) => e.preventDefault());
  document.addEventListener('dblclick', (e) => e.preventDefault());
  document.addEventListener('visibilitychange', () => {
    if (document.hidden && this.state === 'play' && !this.game.over && !this.game.spectating) {
      this.togglePause(true);
      Save.write(this.game);
    }
  });
  // Si se cierra la pestaña sin querer, la partida queda guardada.
  window.addEventListener('pagehide', () => {
    if (this.state === 'play' && this.game && !this.game.over && !this.game.spectating) Save.write(this.game);
  });
  this.last = performance.now();
  requestAnimationFrame((t) => this.loop(t));
},

reroll() {
  this.opts.seed = (Math.random() * 1e9) | 0;
  this.makePreview();
},

newRenderer(game) {
  if (this.renderer) this.renderer.dispose();
  this.renderer = new Renderer(this.gl, this.canvas, this.overlay, game, this.battery ? 'low' : this.quality);
  this.renderer.tintTerritory();
  if (this.battery && this.renderer.fxs) this.renderer.fxs.scale = BATTERY.fx;
},

// v1.4: ahorro de batería: 30 cuadros por segundo, calidad «más fluido» y menos partículas.
setBattery(on) {
  this.battery = !!on;
  try { localStorage.setItem('faro-battery', on ? '1' : '0'); } catch (err) { /* sin almacenamiento */ }
  if (this.renderer) {
    this.renderer.applyQuality(on ? 'low' : this.quality);
    if (this.renderer.fxs) this.renderer.fxs.scale = on ? BATTERY.fx : this.quality === 'high' ? 1 : 0.5;
  }
  this.ui.updateA11yBtns();
},

// v1.4: colores para daltonismo. Cambia los colores (y los nombres que dicen el color) de los
// jugadores; en una partida, vuelve a dibujar todo con los colores nuevos.
setCvd(on) {
  this.cvd = !!on;
  try { localStorage.setItem('faro-cvd', on ? '1' : '0'); } catch (err) { /* sin almacenamiento */ }
  const oldNames = PLAYER_NAMES.slice();
  applyPalette(this.cvd);
  for (const g of [this.game, this.suspended]) {
    if (!g || !g.players) continue;
    for (const p of g.players) {
      p.color = PLAYER_COLORS[p.id];
      if (p.name === oldNames[p.id]) p.name = PLAYER_NAMES[p.id];
    }
  }
  if (this.game && this.renderer) this.newRenderer(this.game);
  this.ui.playersSig = '';
  this.ui.updateA11yBtns();
},

setBigText(on) {
  this.bigText = !!on;
  try { localStorage.setItem('faro-bigtext', on ? '1' : '0'); } catch (err) { /* sin almacenamiento */ }
  document.body.classList.toggle('bigtext', this.bigText);
  this.ui.updateA11yBtns();
},

makePreview() {
  this.game = new Game({ ...this.opts });
  this.game.preview = true;
  this.newRenderer(this.game);
  const r = this.renderer;
  r.cam.tx = 0;
  r.cam.tz = 0;
  r.cam.dist = this.game.map.R * 2.3;
  this.selection = [];
},

// Opciones de la partida: las del vestíbulo, el aspecto equipado y el modo creador.
gameOpts() {
  const o = { ...this.opts, looks: Profile.looks() };
  delete o.creator;
  delete o.instantBuild;
```

```
    if (Creator.active()) {
      o.creator = true;
      o.instantBuild = !!Profile.data.creator.instant;
    }
    return o;
  },

  // Campaña: empieza la misión con su mapa fijo (si está abierta).
  startMission(id) {
    if (!Campaign.unlocked(id, Profile.data, Creator.active())) {
      this.ui.toast(`Termina primero la misión ${id - 1}`, 'bad');
      return;
    }
    this.opts = { ...Campaign.opts(id) };
    this.startGame();
  },

  // v1.3: una partida de la Liga 3 vs 3 que se deja sin terminar (abandonarla o empezar otra encima)
  // cuenta como derrota.
  dropUnfinished() {
    const g = this.suspended || (this.game && !this.game.preview && !this.game.over && this.state === 'play' ? this.game
: null);
    let ranked = false;
    if (g) ranked = !g.over && !g.spectating && g.mode.ranked && !g.opts.custom;
    else {
      const meta = Save.peek();
      ranked = !!(meta && meta.ranked);
    }
    if (ranked) this.lobby.forfeitRanked();
    this.suspended = null;
    Save.clear();
  },

  startGame() {
    clearTimeout(this.endTimer);
    if (this.game && (this.game.over || this.game.spectating) && !this.game.preview) this.lobby.finish(this.game);
    this.input.cancelPlacement();
    this.input.boxMode = false;
    this.dropUnfinished();
    this.game = new Game(this.gameOpts());
    this.enterGame();
    // v1.4: el desafío semanal avisa su regla al empezar.
    if (this.game.opts.weekly) {
      const W = weeklyFor(Profile.now());
      this.ui.toast(`Desafío semanal: ${W.name}. ${W.desc}`, 'info');
    }
    if (this.game.camp) {
      this.focusCore();
      this.ui.showBrief();
    }
  },

  enterGame(keepRenderer) {
    if (!keepRenderer) this.newRenderer(this.game);
    // Los grupos son de esta partida: al cambiar de partida empiezan vacíos.
    if (this.groupsGame !== this.game) {
      this.groups = { 1: [], 2: [], 3: [] };
      this.groupsGame = this.game;
    }
    this.input.disarmAttackMove();
    this.selection = [];
    this.paused = false;
    this.state = 'play';
    if (this.music) this.music.play('battle');
    this.saveT = 60;
    this.ui.showScreen('play');
    this.ui.startHud();
    this.sound.unlock();
    window.FARO = this;
  },

  // Salir al menú no borra la partida (punto 16): queda en pausa para «Volver a la partida».
  // v1.1: después de perder, sigue la partida hasta el final para mirarla.
  spectate() {
    const g = this.game;
    if (!g || !g.startSpectating()) return;
    $('end').hidden = true;
    this.setSelection([]);
    this.input.cancelPlacement();
    document.body.classList.add('spectating');
    $('specbar').hidden = false;
    this.ui.toast('Modo espectador: ves el resto de la partida', 'info');
  },

  toMenu() {
    clearTimeout(this.endTimer);
    this.input.cancelPlacement();
    document.body.classList.remove('spectating');
```

```
    $('specbar').hidden = true;
    const g = this.game;
    if (g && !g.preview) {
      if (g.over || g.spectating) {
        this.lobby.finish(g);
        this.suspended = null;
        Save.clear();
      } else {
        this.suspended = g;
        Save.write(g);
      }
    }
    this.state = 'menu';
    this.paused = false;
    $('pause').hidden = true;
    if (this.music) this.music.play('lobby');
    if (!this.suspended) {
      // Después de una misión, el fondo del vestíbulo vuelve a la isla de «Jugar».
      if (this.opts.mission) this.opts = { ...this.lobby.optsFor('main') };
      this.opts.seed = (Math.random() * 1e9) | 0;
      this.makePreview();
    }
    this.ui.showScreen('menu');
  },

  resumeGame() {
    const g = this.suspended;
    if (!g || g.over) return;
    this.suspended = null;
    const same = this.game === g;
    this.game = g;
    this.enterGame(same);
    this.focusCore();
  },

  loadSavedGame() {
    const g = Save.read();
    if (!g) {
      Save.clear();
      this.ui.toast('No se pudo abrir la partida guardada', 'bad');
      this.lobby.render();
      return;
    }
    // El modo creador solo sigue activo para el dueño del juego.
    if (g.creator && !Creator.owner) {
      g.creator = false;
      g.opts.creator = false;
      g.opts.instantBuild = false;
    }
    this.suspended = null;
    this.game = g;
    this.enterGame(false);
    this.focusCore();
  },

  abandonGame() {
    this.dropUnfinished();
    if (this.opts.mission) this.opts = { ...this.lobby.optsFor('main') };
    this.opts.seed = (Math.random() * 1e9) | 0;
    this.makePreview();
    this.lobby.render();
  },

  togglePause(on) {
    if (this.state !== 'play' || !$('brief').hidden) return;
    this.paused = on;
    $('pause').hidden = !on;
    // Al pausar se guarda (con la partida quieta no se nota el momento del guardado).
    if (on && this.game && !this.game.over && !this.game.spectating && !this.game.preview) {
      Save.write(this.game);
      this.saveT = 60;
    }
    this.markQuality();
  },

  markQuality() {
    document.querySelectorAll('[data-quality]').forEach((b) => b.classList.toggle('gold', b.dataset.quality ===
this.qualityPref));
    const note = $('quality-note');
    if (note) {
      note.textContent = this.qualityPref === 'auto'
        ? `Ahora: ${this.quality === 'high' ? 'alta calidad' : 'más fluido'}${this.fps ? ` · ${this.fps} cuadros por
segundo` : ''}`
        : '';
    }
  },

  // Gráficos: «Automática» empieza en alta calidad y baja sola si el iPad no llega a 40 cuadros por segundo.
```

```
  setQuality(q) {
    this.qualityPref = q;
    try { localStorage.setItem('faro-quality', q); } catch (err) { /* sin almacenamiento */ }
    this.quality = q === 'low' ? 'low' : 'high';
    this.lowFpsSecs = 0;
    if (this.renderer) this.renderer.applyQuality(this.quality);
    this.markQuality();
  },

  toggleFps(on = !this.showFps) {
    this.showFps = on;
    try { localStorage.setItem('faro-fps', on ? 'on' : 'off'); } catch (err) { /* sin almacenamiento */ }
    $('fps').hidden = !on;
    $('btn-fps').textContent = `Contador de cuadros: ${on ? 'sí' : 'no'}`;
  },

  // Se llama una vez por segundo con los cuadros por segundo medidos.
  autoQuality(fps) {
    this.fps = fps;
    if (this.showFps) $('fps').textContent = `${fps} fps`;
    if (this.qualityPref !== 'auto' || this.quality !== 'high' || this.state !== 'play' || this.paused || this.battery)
{
      this.lowFpsSecs = 0;
      return false;
    }
    this.lowFpsSecs = fps < 40 ? this.lowFpsSecs + 1 : 0;
    if (this.lowFpsSecs < 4) return false;
    this.quality = 'low';
    this.lowFpsSecs = 0;
    if (this.renderer) this.renderer.applyQuality('low');
    this.ui.toast(`Gráficos: más fluido (automático, ${fps} cuadros por segundo)`, 'info');
    return true;
  },

  setSelection(list) {
    const before = this.selection;
    this.selection = list.filter((e) => e && e.alive && !e.inside);
    if (this.input.amoveArm && (this.selection.length !== before.length || this.selection.some((e, i) => e !==
before[i]))) {
      this.input.disarmAttackMove();
    }
    this.ui.demolishArm = -1;
    if (this.input.placement && this.input.placement.kind === 'found' &&
!this.selection.includes(this.input.placement.unit)) {
      this.input.cancelPlacement();
    }
    this.ui.renderSel(true);
  },

  // ---------- grupos, constructores y formación (v0.7) ----------
  saveGroup(n) {
    if (this.state !== 'play') return;
    const list = this.selection.filter((e) => e.kind === 'unit' && e.owner === 0 && e.alive);
    if (!list.length) {
      this.ui.toast('Elige tropas y luego guárdalas en un grupo', 'bad');
      return;
    }
    this.groups[n] = list.slice();
    this.ui.toast(`Grupo ${n}: ${list.length} ${list.length === 1 ? 'tropa' : 'tropas'}`, 'good');
    this.sound.play('click');
    this.ui.renderGroups(true);
  },

  groupUnits(n) {
    const list = (this.groups[n] || []).filter((u) => u.alive && u.owner === 0);
    this.groups[n] = list;
    return list;
  },

  // Tocar un grupo lo elige; tocarlo dos veces seguidas lleva la cámara a sus tropas.
  selectGroup(n) {
    if (this.state !== 'play') return;
    const list = this.groupUnits(n).filter((u) => !u.inside);
    const now = performance.now();
    const twice = this.groupTap.n === n && now - this.groupTap.t < 450;
    this.groupTap = { n, t: now };
    if (!list.length) {
      this.ui.toast(`Grupo ${n} vacío: elige tropas y toca «Guardar en ${n}»`);
      return;
    }
    this.setSelection(list);
    this.sound.play('click');
    if (twice) {
      let x = 0;
      let z = 0;
      for (const u of list) { x += u.x; z += u.z; }
      this.focusAt(x / list.length, z / list.length);
    }
```

```
  },

  // Constructores e Ingenieros: primero los que no trabajan; si ya los tienes elegidos, todos.
  selectBuilders() {
    const g = this.game;
    if (!g || this.state !== 'play') return;
    const all = g.units.filter((u) => u.owner === 0 && u.alive && !u.inside && u.def.builds === 'land');
    if (!all.length) {
      this.ui.toast('No tienes Constructores: entrénalos en el Núcleo o el Cuartel', 'bad');
      return;
    }
    const idle = all.filter((u) => u.state === 'idle');
    const same = (a, b) => a.length === b.length && a.every((u) => b.includes(u));
    const pick = idle.length && !same(idle, this.selection) ? idle : all;
    this.setSelection(pick);
    this.sound.play('click');
    this.ui.toast(pick === idle ? `${idle.length} ${idle.length === 1 ? 'constructor libre' : 'constructores libres'}`
      : `${all.length} ${all.length === 1 ? 'constructor' : 'constructores'} (${idle.length} sin trabajo)`);
  },

  toggleFormation() {
    this.formation = this.formation === 'line' ? 'block' : 'line';
    try { localStorage.setItem('faro-formation', this.formation); } catch (err) { /* sin almacenamiento */ }
    this.ui.toast(this.formation === 'line' ? 'Formación: línea (de frente al destino)' : 'Formación: bloque');
    this.ui.selSig = '';
  },

  quickSelect(kind) {
    const g = this.game;
    if (!g) return;
    const list = g.units.filter((u) => u.owner === 0 && u.alive && !u.inside && u.def.weapon && (
      kind === 'all' ? true : kind === 'army' ? u.mover === 'land' : kind === 'sea' ? u.mover === 'sea' : u.mover ===
'air'));
    if (!list.length) {
      this.ui.toast(kind === 'all' ? 'No tienes tropas de combate' : kind === 'army' ? 'No tienes tropas de tierra' :
kind === 'sea' ? 'No tienes barcos de combate' : 'No tienes unidades aéreas');
      return;
    }
    this.setSelection(list);
    this.sound.play('click');
  },

  focusAt(x, z) {
    if (!this.renderer) return;
    this.renderer.cam.tx = x;
    this.renderer.cam.tz = z;
    this.renderer.addPing(x, z, 'look');
  },

  focusCore() {
    const g = this.game;
    if (!g || !this.renderer) return;
    const core = g.players[0].core;
    this.renderer.cam.tx = core.x;
    this.renderer.cam.tz = core.z;
  },

  drainEvents() {
    const g = this.game;
    for (const ev of g.events) {
      if (ev.type === 'toast') {
        this.ui.toast(ev.text, ev.kind, ev.x !== undefined ? { x: ev.x, z: ev.z } : null);
        if (ev.kind === 'bad' && ev.x !== undefined) this.ui.addEdgeAlert(ev.text, ev.x, ev.z);
      }
      else if (ev.type === 'ping') this.renderer.addPing(ev.x, ev.z, ev.kind);
      else if (ev.type === 'chat') {
        this.ui.chat(ev.msg);
        if (ev.msg.toMe && ev.msg.x !== undefined) this.renderer.addPing(ev.msg.x, ev.msg.z, 'bad');
      }
      else if (ev.type === 'sound') {
        let vol = 1;
        if (ev.x !== undefined && this.renderer) {
          const d = Math.hypot(ev.x - this.renderer.cam.tx, ev.z - this.renderer.cam.tz);
          const reach = this.renderer.cam.dist * 1.2;
          if (d > reach) continue;
          vol = 1 - d / reach;
        }
        if (ev.pid !== undefined && ev.pid !== 0) continue;
        this.sound.play(ev.name, vol);
      } else if (ev.type === 'objective') this.ui.markObjective(ev.id);
      else if (ev.type === 'built' || ev.type === 'spawned') this.ui.selSig = '';
      else if (ev.type === 'upgraded' && ev.lines && ev.lines.length) this.ui.upgradeCard(ev.b, ev.level, ev.lines);
      else if (ev.type === 'over') {
        document.body.classList.remove('spectating');
        $('specbar').hidden = true;
        Save.clear();
        this.lobby.finish(g);
        this.endTimer = setTimeout(() => this.ui.showEnd(ev.result), 1600);
```

```
        this.sound.play(ev.result === 'victory' ? 'done' : 'alarm');
      }
    }
    g.events.length = 0;
  },

  loop(now) {
    if (!this.loaded) {
      this.last = now;
      requestAnimationFrame((t) => this.loop(t));
      return;
    }
    // v1.4: ahorro de batería: se salta cuadros para quedarse en unos 30 por segundo.
    if (this.battery && now - (this.lastDraw || 0) < BATTERY.minFrameMs) {
      requestAnimationFrame((t) => this.loop(t));
      return;
    }
    this.lastDraw = now;
    const raw = Math.max(0, (now - this.last) / 1000);
    const dt = Math.min(0.1, raw);
    this.last = now;
    // Cuadros por segundo reales (para el contador y la calidad automática).
    this.fpsFrames++;
    this.fpsT += raw;
    if (this.fpsT >= 1) {
      this.autoQuality(Math.round(this.fpsFrames / this.fpsT));
      this.fpsFrames = 0;
      this.fpsT = 0;
    }
    this.time += dt;
    try {
      if (this.state === 'play' && this.game) {
        if (!this.paused) this.game.update(dt);
        this.drainEvents();
        this.input.update(dt);
        if (this.selection.some((e) => !e.alive || e.inside)) this.setSelection(this.selection);
        if (this.input.placement && this.input.placement.kind === 'build') {
          this.input.placeT = (this.input.placeT || 0) - dt;
          if (this.input.placeT <= 0) {
            this.input.placeT = 1;
            this.input.refreshPlacement();
          }
        }
        this.renderer.frame(this.paused ? 0 : dt, this.time, this.selection);
        this.ui.updateEdges();
        // Música: más intensa si te atacan o si tus tropas pelean.
        this.musicT = (this.musicT || 0) - dt;
        if (this.musicT <= 0 && this.music) {
          this.musicT = 0.5;
          const g = this.game;
          const me = g.players[0];
          const fighting = g.time - (me.underAttackAt ?? -99) < 8 || g.units.some((u) => u.owner === 0 && u.alive &&
u.autoTarget && u.autoTarget.alive);
          this.musicFight = fighting ? 1 : 0;
        }
        if (this.music) this.music.setIntensity(this.musicFight || 0);
        // Guardado automático cada 15 s.
        this.saveT -= dt;
        if (this.saveT <= 0) {
          this.saveT = 60;
          if (!this.game.over && !this.game.spectating) Save.write(this.game);
        }
        this.ui.update(dt);
        this.miniT -= dt;
        if (this.miniT <= 0) {
          this.miniT = 0.2;
          this.renderer.drawMinimap($('minimap'));
        }
      } else if (this.renderer) {
        this.renderer.cam.yaw += dt * 0.05;
        this.renderer.frame(dt, this.time, []);
      }
    } catch (err) {
      console.error('[Faro] Error en el bucle:', err);
    }
    requestAnimationFrame((t) => this.loop(t));
  },
};

const hot = window.claude && window.claude.hot;
if (hot && hot.ready) hot.ready((data) => App.init(data || {}));
else App.init((hot && hot.data) || {});
```

faro/build.mjs · 62 líneas

```
// Une la plantilla HTML y los scripts de src/js en un único archivo jugable.
// Uso: node faro/build.mjs [ruta-extra-sin-envoltorio.html]
import { readFileSync, readdirSync, writeFileSync } from 'fs';
import { dirname, join } from 'path';
import { fileURLToPath } from 'url';

const ROOT = dirname(fileURLToPath(import.meta.url));
const JS_DIR = join(ROOT, 'src', 'js');

const files = readdirSync(JS_DIR).filter((f) => f.endsWith('.js')).sort();
// Modelos 3D hechos con Blender (faro/blender/generate_models.py), incrustados en base64.
const MODEL_DIR = join(ROOT, 'assets', 'models');
let models = {};
try {
  for (const f of readdirSync(MODEL_DIR).filter((x) => x.endsWith('.glb')).sort()) {
    models[f.replace(/\.glb$/, '')] = readFileSync(join(MODEL_DIR, f)).toString('base64');
  }
} catch (err) {
  console.warn('[Faro] Sin modelos GLB:', err.message);
  models = {};
}
// Cielo de Poly Haven (CC0, «Kloofendal 48d Partly Cloudy», Greg Zaal) reducido a 256×128 para la luz de ambiente.
let envUri = '';
try {
  envUri = '' + readFileSync(join(ROOT, 'assets', 'env', 'sky_env.jpg')).toString('base64');
} catch (err) {
  console.warn('[Faro] Sin cielo de ambiente:', err.message);
}
// Ciclos de caminar y correr sacados de la captura de movimiento de CMU (blender/cmu_gait.py).
let anim = null;
try {
  anim = JSON.parse(readFileSync(join(ROOT, 'assets', 'anim', 'cmu_gait.json'), 'utf8'));
} catch (err) {
  console.warn('[Faro] Sin animaciones de CMU:', err.message);
}
const modelJs = `// ---- modelos GLB (Blender) ----\nconst FARO_GLB = ${JSON.stringify(models)};\nconst FARO_ENV =
${JSON.stringify(envUri)};\n` +
  `const FARO_ANIM = ${JSON.stringify(anim)};\n`;
const js = modelJs + files.map((f) => `// ---- ${f} ----\n${readFileSync(join(JS_DIR, f), 'utf8')}`).join('\n');
const template = readFileSync(join(ROOT, 'src', 'index.template.html'), 'utf8');
if (!template.includes('/*__SCRIPTS__*/')) throw new Error('Falta el marcador /*__SCRIPTS__*/ en la plantilla');
// Función de reemplazo para que los "$" del código no se interpreten como patrones.
const body = template.replace('/*__SCRIPTS__*/', () => js);

const full = [
  '<!doctype html>',
  '<html lang="es">',
  '<head>',
  '<meta charset="utf-8">',
  '<meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1, viewport-fit=cover">',
  body,
  '</html>',
  '',
].join('\n');

writeFileSync(join(ROOT, 'faro.html'), full);
console.log(`[Faro] faro.html listo (${(full.length / 1024).toFixed(0)} KB, ${files.length} scripts,
${Object.keys(models).length} modelos)`);

const extra = process.argv[2];
if (extra) {
  writeFileSync(extra, body);
  console.log(`[Faro] Versión sin envoltorio: ${extra}`);
}
```

faro/blender/cmu_gait.py · 125 líneas

```
"""Saca un ciclo de paso (caminar y correr) de la base de datos de captura de movimiento de CMU.

Uso: python faro/blender/cmu_gait.py <carpeta_con_bvh> faro/assets/anim/cmu_gait.json

Datos: CMU Graphics Lab Motion Capture Database (mocap.cs.cmu.edu), conversión BVH de B. Hahne
(cgspeed). Uso libre según su READMEFIRST. Archivos: 02_01 (caminar) y 35_17 (correr).

Para cada cuadro se miden, en el plano de la marcha, los ángulos de cadera y rodilla de cada pierna,
la inclinación del torso y la altura de la cadera. Se toma un ciclo completo (de un paso adelante de
la pierna izquierda al siguiente) y se reparte en 24 muestras. Los ángulos van en radianes; el rebote
y la zancada van en largos de pierna, para escalarlos a cualquier soldado.
"""
import json
import math
import os
import sys

import bpy  # primero bpy: agrega la carpeta de módulos de Blender (addon_utils)
import addon_utils  # noqa: E402
from mathutils import Vector  # noqa: E402

CLIPS = {'walk': '02_01.bvh', 'run': '35_17.bvh'}
SAMPLES = 24


def load(path):
    bpy.ops.wm.read_factory_settings(use_empty=True)
    addon_utils.enable('io_anim_bvh', default_set=True)
    bpy.ops.import_anim.bvh(filepath=path, global_scale=1.0, rotate_mode='NATIVE', axis_forward='-Z', axis_up='Y',
                            update_scene_fps=True, update_scene_duration=True)
    arm = [o for o in bpy.data.objects if o.type == 'ARMATURE'][0]
    return arm


def joints(arm, frame):
    bpy.context.scene.frame_set(frame)
    mw = arm.matrix_world
    pb = arm.pose.bones
    head = lambda n: mw @ pb[n].head
    return {
        'hips': head('Hips'), 'chest': head('Spine1'),
        'hipL': head('LeftUpLeg'), 'kneeL': head('LeftLeg'), 'ankleL': head('LeftFoot'),
        'hipR': head('RightUpLeg'), 'kneeR': head('RightLeg'), 'ankleR': head('RightFoot'),
    }


def angle_from_down(d, fwd, up):
    """Ángulo de un segmento respecto de la vertical, en el plano de la marcha (+ = hacia adelante)."""
    return math.atan2(d.dot(fwd), -d.dot(up))


def extract(path):
    arm = load(path)
    scene = bpy.context.scene
    f0, f1 = int(scene.frame_start), int(scene.frame_end)
    frames = [joints(arm, f) for f in range(f0, f1 + 1)]
    up = Vector((0, 0, 1))
    move = frames[-1]['hips'] - frames[0]['hips']
    move.z = 0
    fwd = move.normalized()
    leg = sum(((j['hipL'] - j['kneeL']).length + (j['kneeL'] - j['ankleL']).length) for j in frames) / len(frames)
    rows = []
    for j in frames:
        r = {}
        for s in ('L', 'R'):
            th = angle_from_down(j['knee' + s] - j['hip' + s], fwd, up)
            sh = angle_from_down(j['ankle' + s] - j['knee' + s], fwd, up)
            r['hip' + s] = th
            r['knee' + s] = th - sh  # flexión de rodilla (+ = el pie va hacia atrás)
        # Inclinación del torso (+ = hacia adelante).
        r['pitch'] = math.atan2((j['chest'] - j['hips']).dot(fwd), (j['chest'] - j['hips']).dot(up))
        r['y'] = j['hips'].z
        r['h'] = j['hips'].dot(fwd)
        rows.append(r)
    # Ciclo: entre dos máximos seguidos del ángulo de la cadera izquierda (pierna más adelante).
    hl = [r['hipL'] for r in rows]
    # Máximo dentro de ±20 cuadros (1/6 s): así no se cuentan los temblores dentro de un mismo paso.
    W = 20
    peaks = [i for i in range(W, len(hl) - W) if hl[i] >= max(hl[i - W:i + W + 1])]
    clean = []
    for p in peaks:
        if not clean or p - clean[-1] > 30:
            clean.append(p)
    if len(clean) < 2:
```

```
        raise RuntimeError(f'No hay un ciclo completo en {path}: picos {clean}')
    a, b = clean[0], clean[1]
    fps = 1.0 / bpy.context.scene.render.fps_base * bpy.context.scene.render.fps
    frame_time = 1.0 / fps
    # El importador ajusta los fps de la escena al archivo; si no, se usa 120 (CMU).
    if abs(fps - 24) < 0.01:
        frame_time = 1.0 / 120
    out = {k: [] for k in ('hipL', 'kneeL', 'hipR', 'kneeR', 'pitch', 'bob')}
    ys = [rows[i]['y'] for i in range(a, b + 1)]
    ymean = sum(ys) / len(ys)
    pmean = sum(rows[i]['pitch'] for i in range(a, b + 1)) / (b - a + 1)
    for k in range(SAMPLES):
        t = a + (b - a) * k / SAMPLES
        i = int(t)
        f = t - i
        lerp = lambda key: rows[i][key] * (1 - f) + rows[min(i + 1, len(rows) - 1)][key] * f
        for key in ('hipL', 'kneeL', 'hipR', 'kneeR'):
            out[key].append(round(lerp(key), 4))
        out['pitch'].append(round(lerp('pitch') - pmean * 0.5, 4))
        out['bob'].append(round((lerp('y') - ymean) / leg, 4))
    stride = (rows[b]['h'] - rows[a]['h']) / leg
    return {'dur': round((b - a) * frame_time, 4), 'stride': round(stride, 4), 'frames': [a, b], **out}


def main():
    src, dst = sys.argv[-2], sys.argv[-1]
    data = {'source': 'CMU Graphics Lab Motion Capture Database (mocap.cs.cmu.edu), conversión BVH de B. Hahne',
            'files': CLIPS, 'samples': SAMPLES}
    for name, f in CLIPS.items():
        data[name] = extract(os.path.join(src, f))
        d = data[name]
        print(name, 'ciclo', d['frames'], 'dur', d['dur'], 's · zancada', d['stride'], 'piernas · cadera',
              round(min(d['hipL']), 2), round(max(d['hipL']), 2), '· rodilla max', round(max(d['kneeL']), 2),
              '· rebote', round(min(d['bob']), 3), round(max(d['bob']), 3))
    os.makedirs(os.path.dirname(dst), exist_ok=True)
    with open(dst, 'w') as fh:
        json.dump(data, fh, separators=(',', ':'))
    print('guardado', dst, os.path.getsize(dst), 'bytes')


main()
```

faro/blender/gen_core15.py · 17 líneas

```
"""Exporta solo el Núcleo nuevo de la v1.5 (core_1, core_2 y core_3)."""
import os
import sys

import bpy

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from models_v15 import core15  # noqa: E402

out_dir = os.path.abspath(sys.argv[1] if len(sys.argv) > 1 else 'models')
os.makedirs(out_dir, exist_ok=True)
bpy.ops.wm.read_factory_settings(use_empty=True)
for lv in (1, 2, 3):
    m = core15(lv)
    path = os.path.join(out_dir, m.name + '.glb')
    m.export(path)
    print(f'[Blender] ✅ {m.name}: {os.path.getsize(path) / 1024:.0f} KB')
```

faro/blender/generate_models.py · 155 líneas

```
"""Genera los modelos 3D de Faro de Ámbar con Blender y los exporta como GLB.

Uso:  python generate_models.py <carpeta_salida>     (con el módulo bpy de Blender 5)
      blender -b -P generate_models.py -- <carpeta_salida>

Los GLB sirven para el juego web y se pueden importar en Roblox Studio (3D Importer).
"""
import math
import os
import sys

import bpy

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from lib import Model  # noqa: E402
# Modelos rehechos según tus referencias del 7 de noviembre.
from models_ref import (arty, barge, boat_engineer, core, destroyer, farm, helicopter, helipad,  # noqa: E402
                        jeep, mine, rig, soldier, speedboat, tank, turret, wheels)
# Modelos de la v0.6.
from models_ref2 import (aa_truck, apache, barracks, factory, harbor, hospital, house, hydro, miniturret,  # noqa: E402
                         outpost, solar, submarine, tidal, wave, wind)
# Modelos nuevos de la v0.9 (vehículos, barcos, aviones, Aeropuerto, Antinuclear y Defensa aérea nivel 4).
from models_ref3 import (aadef, airport, antinuke, apartment, apc, bomber, carrier, fighter, frigate, heavy_tank,  # noqa: E402
                         lander, launcher, light_tank, missile_truck)

PI = math.pi
H = PI / 2


# ============================================================================
# Soldados: cuerpo de bloques; cada uno con casco, equipo y arma propios
# ============================================================================


# ============================================================================
# Vehículos
# ============================================================================


# ============================================================================
# Barcos: cascos con forma, cubierta, puente y detalles
# ============================================================================


# ============================================================================
# Helicópteros
# ============================================================================


# ============================================================================
# Edificios
# ============================================================================


def pump():
    m = Model('pump')
    m.box('concreto', (1.1, 0.8, 0.05), (0, 0.05, 0), bevel=0.02)
    m.cyl('negro', 0.18, 0.012, (0.32, 0.25, 0.05), seg=18)
    m.box('metal', (0.06, 0.06, 0.62), (0, -0.12, 0.05), rot=(0.28, 0, 0), bevel=0.01)
    m.box('metal', (0.06, 0.06, 0.62), (0, 0.06, 0.05), rot=(-0.28, 0, 0), bevel=0.01)
    m.box('EQUIPO', (0.2, 0.22, 0.16), (0, 0.32, 0.05), bevel=0.03)
    m.cyl('blanco', 0.16, 0.4, (-0.38, -0.25, 0.05), seg=16)
    m.cyl('EQUIPO_OSC', 0.17, 0.03, (-0.38, -0.25, 0.45), seg=16)
    m.cyl('oscuro', 0.035, 0.28, (0, -0.42, 0.05), seg=10)
    m.pivot('rocker', 0, -0.03, 0.66)
    m.box('amarillo', (0.07, 0.9, 0.07), (0, 0.02, 0.62), bevel=0.015, part='rocker')
    m.box('amarillo', (0.09, 0.14, 0.24), (0, -0.46, 0.5), rot=(0.35, 0, 0), bevel=0.02, part='rocker')
    m.box('oscuro', (0.14, 0.16, 0.14), (0, 0.4, 0.58), bevel=0.02, part='rocker')
    return m


def wall_post():
    m = Model('wall_post')
    m.box('piedra', (0.4, 0.4, 0.62), (0, 0, 0), bevel=0.03)
    m.box('piedra_clara', (0.46, 0.46, 0.06), (0, 0, 0.56), bevel=0.02)
    for (x, y) in [(-0.15, -0.15), (0.15, -0.15), (-0.15, 0.15), (0.15, 0.15)]:
        m.box('piedra_clara', (0.12, 0.12, 0.12), (x, y, 0.62), bevel=0.015)
    m.box('EQUIPO', (0.42, 0.42, 0.04), (0, 0, 0.38), bevel=0.008)
    m.box('oscuro', (0.06, 0.012, 0.12), (0, -0.2, 0.42), bevel=0.0)
    return m


def wall_segment():
```

```
    """Tramo de muralla que va del centro de la casilla hacia -Y (el frente): mide medio paso de casilla."""
    m = Model('wall_seg')
    L = 0.9
    m.box('piedra', (0.26, L, 0.46), (0, -L / 2 - 0.05, 0), bevel=0.02)
    m.box('piedra_clara', (0.3, L, 0.04), (0, -L / 2 - 0.05, 0.46), bevel=0.01)
    for k in range(3):
        m.box('piedra_clara', (0.1, 0.12, 0.1), (-0.08, -0.22 - k * 0.28, 0.5), bevel=0.012)
        m.box('piedra_clara', (0.1, 0.12, 0.1), (0.08, -0.22 - k * 0.28, 0.5), bevel=0.012)
    m.box('EQUIPO', (0.27, L, 0.03), (0, -L / 2 - 0.05, 0.3), bevel=0.004)
    return m


def scaffold():
    """Andamio que rodea las obras en construcción."""
    m = Model('scaffold')
    R = 0.62
    corners = [(-R, -R), (R, -R), (R, R), (-R, R)]
    for (x, y) in corners:
        m.box('madera_clara', (0.04, 0.04, 0.95), (x, y, 0), bevel=0.0)
    for z in (0.32, 0.64, 0.92):
        for i in range(4):
            (x1, y1), (x2, y2) = corners[i], corners[(i + 1) % 4]
            mx, my = (x1 + x2) / 2, (y1 + y2) / 2
            rotz = 0 if y1 == y2 else H
            m.box('madera', (2 * R, 0.03, 0.03), (mx, my, z), rot=(0, 0, rotz), bevel=0.0)
    for i in range(4):
        (x1, y1), (x2, y2) = corners[i], corners[(i + 1) % 4]
        mx, my = (x1 + x2) / 2, (y1 + y2) / 2
        rotz = 0 if y1 == y2 else H
        m.box('madera', (2 * R * 1.1, 0.025, 0.025), (mx, my, 0.02), rot=(0.6, 0, rotz), bevel=0.0)
        m.box('amarillo', (2 * R, 0.12, 0.02), (mx * 1.04, my * 1.04, 0.33), rot=(0, 0, rotz), bevel=0.0)
    m.box('amarillo', (0.36, 0.012, 0.12), (0, -R - 0.02, 0.5), bevel=0.0)
    m.box('negro', (0.06, 0.014, 0.12), (-0.1, -R - 0.03, 0.5), rot=(0, 0.6, 0), bevel=0.0)
    m.box('negro', (0.06, 0.014, 0.12), (0.1, -R - 0.03, 0.5), rot=(0, 0.6, 0), bevel=0.0)
    return m


def all_models():
    out = []
    infantry = ['builder', 'engineer', 'medic', 'rifle', 'shotgun', 'ranger', 'sniper', 'breaker', 'hunter', 'rocket']
    for k in infantry:
        out.append(soldier(k))
    # Skin Brigada Tóxica: los mismos soldados con capucha y máscara de gas.
    for k in infantry:
        out.append(soldier(k, 'toxico'))
    out += [jeep(), tank(), aa_truck(), arty(), speedboat(), boat_engineer(), barge(), destroyer(), submarine(),
            helicopter(), apache(), light_tank(), heavy_tank(), missile_truck(), apc(), launcher(), frigate(),
            lander(), carrier(), fighter(), bomber()]
    out += [core(1), core(2), core(3), farm(), pump(), mine(), rig(), house(), apartment(), solar(), wind(), hydro(),
            tidal(), wave(),
            barracks(), factory(), harbor(), helipad(), turret(1), turret(2), turret(3), turret(4), miniturret(),
aadef(1),
            aadef(4), airport(), antinuke(),
            wall_post(), wall_segment(), hospital(1), hospital(2), hospital(3), outpost(), scaffold()]
    return out


def main():
    args = sys.argv[sys.argv.index('--') + 1:] if '--' in sys.argv else sys.argv[1:]
    out_dir = os.path.abspath(args[0] if args else 'models')
    os.makedirs(out_dir, exist_ok=True)
    bpy.ops.wm.read_factory_settings(use_empty=True)
    total = 0
    models = all_models()
    for m in models:
        path = os.path.join(out_dir, m.name + '.glb')
        m.export(path)
        total += os.path.getsize(path)
    print(f'[Blender] ✅ {len(models)} modelos · {total / 1024:.0f} KB en {out_dir}')


if __name__ == '__main__':
    main()
```

faro/blender/lib.py · 215 líneas

```
"""Herramientas para modelar Faro de Ámbar con Python dentro de Blender.

Cada modelo se arma con piezas simples (cajas biseladas, cilindros, conos, cascos de barco) que se
agrupan en partes: 'main' y, si el modelo se anima, 'turret', 'rotor', 'blades', 'rocker' o
'crystal'. Cada parte se exporta como un objeto con su origen en el pivote de giro.

Convenciones (Blender, Z arriba): el frente del modelo mira hacia -Y, que en el juego (glTF, Y
arriba) es +Z. Una casilla mide 1 de radio. Los materiales llamados EQUIPO y EQUIPO_OSC se pintan
en el juego con el color de cada jugador.
"""
import math

import bmesh
import bpy
from mathutils import Euler, Matrix, Vector

MATS = {}


def mat(name, rgb, rough=0.75, metal=0.0):
    """Material con color base (valores 0-1 tal cual, como la paleta del juego)."""
    if name in MATS:
        return name
    m = bpy.data.materials.new(name)
    m.use_nodes = True
    bsdf = m.node_tree.nodes.get('Principled BSDF')
    bsdf.inputs['Base Color'].default_value = (rgb[0], rgb[1], rgb[2], 1.0)
    bsdf.inputs['Roughness'].default_value = rough
    bsdf.inputs['Metallic'].default_value = metal
    MATS[name] = m
    return name


def hexrgb(h):
    return ((h >> 16 & 255) / 255, (h >> 8 & 255) / 255, (h & 255) / 255)


# Paleta (misma familia de colores que el juego).
P = {
    'EQUIPO': 0xffffff, 'EQUIPO_OSC': 0x888888,
    'piedra': 0x8f8b82, 'piedra_clara': 0xa9a59b, 'oscuro': 0x3b4048, 'metal': 0x6e7782,
    'metal_claro': 0x9aa3ad, 'madera': 0x8b5d3c, 'madera_clara': 0xb07d52, 'blanco': 0xefece4,
    'oliva': 0x5d6d3b, 'oliva_osc': 0x46532c, 'piel': 0xe0b089, 'amarillo': 0xf1c232, 'trigo': 0xc9a54b,
    'cultivo': 0x8dbf3d, 'ladrillo': 0xb3653b, 'rojo': 0xd94141, 'gris': 0x8b939c, 'negro': 0x24272c,
    'naranja': 0xf08a24, 'concreto': 0xb5b2aa, 'vidrio': 0x9fd6ee, 'oro': 0xffcf3a, 'agua': 0x4aa3c8,
    'panel': 0x1f3a63, 'arena': 0xc9b27a, 'tierra': 0x7a5a3a, 'goma': 0x1b1d20, 'cuerda': 0xc9b47a,
    'luz': 0xffd76a, 'camo': 0x2f3438, 'camo2': 0x4a5056, 'mascara': 0xf2c230, 'blanco_sucio': 0xd8d4c8,
}


def m_(name):
    return mat(name, hexrgb(P[name]), rough=0.45 if name in ('metal', 'metal_claro', 'vidrio') else 0.75,
               metal=0.3 if name in ('metal', 'metal_claro') else 0.0)


class Model:
    def __init__(self, name):
        self.name = name
        self.parts = {}
        self.pivots = {}
        self.mats = {}

    def _bm(self, part):
        if part not in self.parts:
            self.parts[part] = bmesh.new()
            self.mats[part] = []
            self.pivots.setdefault(part, Vector((0, 0, 0)))
        return self.parts[part]

    def pivot(self, part, x, y, z):
        self.pivots[part] = Vector((x, y, z))
        self._bm(part)

    def _mi(self, part, matname):
        m_(matname) if matname in P else None
        lst = self.mats[part]
        if matname not in lst:
            lst.append(matname)
        return lst.index(matname)

    def _merge(self, part, src, matname, matrix, smooth):
        dst = self._bm(part)
        mi = self._mi(part, matname)
        piv = self.pivots[part]
```

```
        full = Matrix.Translation(-piv) @ matrix
        vmap = {}
        for v in src.verts:
            vmap[v] = dst.verts.new(full @ v.co)
        for f in src.faces:
            nf = dst.faces.new([vmap[v] for v in f.verts])
            nf.material_index = mi
            nf.smooth = smooth
        src.free()

    @staticmethod
    def _xf(loc, rot, scale=(1, 1, 1)):
        return Matrix.LocRotScale(Vector(loc), Euler(rot).to_quaternion(), Vector(scale))

    def box(self, matname, size, loc=(0, 0, 0), rot=(0, 0, 0), bevel=0.02, part='main', base=True):
        """Caja biselada. Con base=True, loc[2] es la cara de abajo."""
        sx, sy, sz = size
        t = bmesh.new()
        bmesh.ops.create_cube(t, size=1.0)
        bmesh.ops.scale(t, vec=Vector((sx, sy, sz)), verts=t.verts)
        b = min(bevel, sx * 0.45, sy * 0.45, sz * 0.45)
        # Las piezas chicas van sin bisel: casi no se nota y el archivo pesa mucho menos.
        if b > 0.001 and min(sx, sy, sz) >= 0.05:
            bmesh.ops.bevel(t, geom=list(t.edges) + list(t.verts), offset=b, segments=1, affect='EDGES', profile=0.5)
        x, y, z = loc
        self._merge(part, t, matname, self._xf((x, y, z + (sz / 2 if base else 0)), rot), False)

    def cyl(self, matname, r, h, loc=(0, 0, 0), rot=(0, 0, 0), seg=16, r2=None, part='main', base=True, smooth=True):
        """Cilindro (o tronco de cono con r2). Con base=True, loc[2] es la base."""
        t = bmesh.new()
        bmesh.ops.create_cone(t, cap_ends=True, cap_tris=False, segments=seg, radius1=r,
                              radius2=r if r2 is None else r2, depth=h)
        x, y, z = loc
        off = Matrix.Translation((0, 0, h / 2 if base else 0))
        self._merge(part, t, matname, self._xf((x, y, z), rot) @ off, smooth)

    def cone(self, matname, r, h, loc=(0, 0, 0), rot=(0, 0, 0), seg=16, part='main', smooth=False):
        self.cyl(matname, r, h, loc, rot, seg, r2=0.0001, part=part, smooth=smooth)

    def sphere(self, matname, r, loc=(0, 0, 0), scale=(1, 1, 1), part='main', seg=14):
        t = bmesh.new()
        bmesh.ops.create_uvsphere(t, u_segments=seg, v_segments=max(6, seg // 2), radius=r)
        self._merge(part, t, matname, self._xf(loc, (0, 0, 0), scale), True)

    def ico(self, matname, r, loc=(0, 0, 0), scale=(1, 1, 1), part='main', sub=1):
        t = bmesh.new()
        bmesh.ops.create_icosphere(t, subdivisions=sub, radius=r)
        self._merge(part, t, matname, self._xf(loc, (0, 0, 0), scale), False)

    def hexprism(self, matname, r, h, loc=(0, 0, 0), part='main', bevel=0.02):
        """Prisma hexagonal alineado con las casillas del juego."""
        t = bmesh.new()
        bmesh.ops.create_cone(t, cap_ends=True, cap_tris=False, segments=6, radius1=r, radius2=r, depth=h)
        if bevel > 0:
            bmesh.ops.bevel(t, geom=list(t.edges) + list(t.verts), offset=bevel, segments=1, affect='EDGES',
profile=0.5)
        x, y, z = loc
        self._merge(part, t, matname, self._xf((x, y, z + h / 2), (0, 0, math.pi / 6)), False)

    def wedge(self, matname, size, loc=(0, 0, 0), rot=(0, 0, 0), part='main'):
        """Tejado a dos aguas: largo en X, ancho en Y, alto en Z, base en loc[2]."""
        lx, ly, lz = size
        t = bmesh.new()
        hx, hy = lx / 2, ly / 2
        vs = [t.verts.new(c) for c in [(-hx, -hy, 0), (hx, -hy, 0), (hx, hy, 0), (-hx, hy, 0), (-hx, 0, lz), (hx, 0,
lz)]]
        for f in [(0, 1, 2, 3), (0, 4, 5, 1), (3, 2, 5, 4), (0, 3, 4), (1, 5, 2)]:
            t.faces.new([vs[i] for i in f])
        bmesh.ops.recalc_face_normals(t, faces=t.faces)
        self._merge(part, t, matname, self._xf(loc, rot), False)

    @staticmethod
    def _between(p1, p2):
        a, b = Vector(p1), Vector(p2)
        d = b - a
        q = Vector((0, 0, 1)).rotation_difference(d.normalized())
        return (a + b) / 2, q, d.length

    def beam(self, matname, p1, p2, t=0.02, part='main'):
        """Barra de sección cuadrada entre dos puntos (barandas, grúas, celosías, patines)."""
        c, q, length = self._between(p1, p2)
        tm = bmesh.new()
        bmesh.ops.create_cube(tm, size=1.0)
        bmesh.ops.scale(tm, vec=Vector((t, t, length)), verts=tm.verts)
        self._merge(part, tm, matname, Matrix.LocRotScale(c, q, Vector((1, 1, 1))), False)

    def tube(self, matname, p1, p2, r=0.02, seg=10, part='main', r2=None):
        """Cilindro entre dos puntos (tubos de lancha, antenas, cañones)."""
```

```
        c, q, length = self._between(p1, p2)
        tm = bmesh.new()
        bmesh.ops.create_cone(tm, cap_ends=True, cap_tris=False, segments=seg, radius1=r,
                              radius2=r if r2 is None else r2, depth=length)
        self._merge(part, tm, matname, Matrix.LocRotScale(c, q, Vector((1, 1, 1))), True)

    def hull(self, matname, sections, loc=(0, 0, 0), part='main', smooth=False):
        """Casco de barco a partir de secciones (y, medio_ancho_cubierta, medio_ancho_fondo, z_fondo, z_cubierta).

        Las secciones van de popa (y mayor) a proa (y menor, el frente)."""
        t = bmesh.new()
        rings = []
        for (y, wt, wb, zb, zt) in sections:
            rings.append([t.verts.new(c) for c in [(-wb, y, zb), (wb, y, zb), (wt, y, zt), (-wt, y, zt)]])
        for a, b in zip(rings, rings[1:]):
            for i in range(4):
                j = (i + 1) % 4
                try:
                    t.faces.new([a[i], a[j], b[j], b[i]])
                except ValueError:
                    pass
        t.faces.new(list(reversed(rings[0])))
        t.faces.new(rings[-1])
        bmesh.ops.recalc_face_normals(t, faces=t.faces)
        self._merge(part, t, matname, self._xf(loc, (0, 0, 0)), smooth)

    # ---------- exportación ----------
    def export(self, path):
        for o in list(bpy.data.objects):
            bpy.data.objects.remove(o, do_unlink=True)
        for me in list(bpy.data.meshes):
            bpy.data.meshes.remove(me)
        scene = bpy.context.scene
        for part, bm in self.parts.items():
            me = bpy.data.meshes.new(f'{self.name}_{part}')
            bm.to_mesh(me)
            bm.free()
            for mname in self.mats[part]:
                me.materials.append(MATS[mname])
            ob = bpy.data.objects.new(part, me)
            ob.location = self.pivots[part]
            scene.collection.objects.link(ob)
        bpy.ops.export_scene.gltf(filepath=path, export_format='GLB', export_apply=True, export_yup=True,
                                  use_selection=False, export_materials='EXPORT', export_texcoords=False,
                                  export_normals=False, export_cameras=False, export_lights=False)
```

faro/blender/models_ref.py · 781 líneas

```
"""Modelos rehechos según las referencias del 7 de noviembre (diseño propio, estilo del juego).

Torreta naval con miniguns, jeep con artillero, Núcleo fortaleza, barcos, helicóptero, helipuerto,
AIR DEFENSE, lanzamisiles, granja, mina, plataforma, soldados con piernas animables y el skin
Brigada Tóxica (capucha y máscara de gas). Blender: Z arriba, el frente mira a -Y.
"""
import math

from lib import Model

PI = math.pi
H = PI / 2


# ---------------------------------------------------------------------
# Soldados (piernas como partes 'legl' y 'legr' para animarlas al caminar)
# ---------------------------------------------------------------------
def soldier(kind, look=None):
    """Soldado (v0.9, más realista): cabeza chica con cuello, uniforme oliva, chaleco táctico con el color
    del imperio, bolsillos, cinturón, mochila, rodilleras y brazos que sostienen el arma."""
    m = Model(kind if not look else f'{kind}@{look}')
    tox = look == 'toxico'
    pant = 'camo' if tox else 'oliva_osc'
    # Piernas en dos piezas (v0.8): muslo desde la cadera y pierna desde la rodilla, para el
    # movimiento de caminar y correr de CMU (blender/cmu_gait.py).
    for part, sx in (('legl', -1), ('legr', 1)):
        shin = 'shin' + part[-1]
        m.pivot(part, sx * 0.05, 0, 0.19)
        m.pivot(shin, sx * 0.05, 0, 0.105)
        m.box(pant, (0.075, 0.09, 0.095), (sx * 0.05, 0, 0.098), bevel=0.012, part=part)
        m.box(pant, (0.07, 0.085, 0.08), (sx * 0.05, 0, 0.03), bevel=0.012, part=shin)
        m.box('negro', (0.08, 0.115, 0.04), (sx * 0.05, -0.012, 0), bevel=0.01, part=shin)
        m.box('oscuro' if not tox else 'camo2', (0.076, 0.03, 0.035), (sx * 0.05, -0.043, 0.085), bevel=0.006,
part=shin)
    hx, hy, hz = 0.09, -0.13, 0.27
    head_z = 0.425
    armed = kind not in ('builder', 'engineer', 'medic')
    sleeve = 'camo' if tox else 'oliva'

    def arm(shoulder, elbow, hand, glove='piel'):
        m.beam(sleeve, shoulder, elbow, t=0.052)
        m.beam(sleeve, elbow, hand, t=0.046)
        m.box(glove, (0.038, 0.04, 0.038), hand, bevel=0.01)

    if tox:
        m.box('camo', (0.21, 0.14, 0.21), (0, 0, 0.19), bevel=0.025)
        for (x, z) in [(-0.06, 0.3), (0.05, 0.22), (0.07, 0.33), (-0.03, 0.21)]:
            m.box('camo2', (0.05, 0.144, 0.05), (x, 0, z), bevel=0.004)
        m.box('EQUIPO', (0.22, 0.15, 0.03), (0, 0, 0.19), bevel=0.008)
        m.box('EQUIPO', (0.2, 0.15, 0.03), (0, 0, 0.36), bevel=0.006)
        m.box('piel', (0.1, 0.105, 0.11), (0, 0, head_z), bevel=0.035)
        # Capucha oscura y máscara de gas amarilla con dos visores y filtro.
        m.sphere('negro', 0.068, (0, 0.012, 0.47), scale=(1.06, 1.08, 1.12))
        m.box('negro', (0.15, 0.07, 0.08), (0, 0.04, 0.39), bevel=0.025)
        m.box('mascara', (0.1, 0.03, 0.08), (0, -0.06, 0.44), bevel=0.018)
        for sx in (-1, 1):
            m.cyl('negro', 0.021, 0.012, (sx * 0.026, -0.075, 0.48), rot=(H, 0, 0), seg=12, base=False)
            m.cyl('vidrio', 0.015, 0.014, (sx * 0.026, -0.079, 0.48), rot=(H, 0, 0), seg=12, base=False)
        m.cyl('negro', 0.026, 0.045, (0, -0.095, 0.435), rot=(H, 0, 0), seg=12, base=False)
        m.cyl('mascara', 0.029, 0.012, (0, -0.12, 0.435), rot=(H, 0, 0), seg=12, base=False)
    else:
        # Uniforme, chaleco con bolsillos, cinturón y mochila.
        m.box('oliva', (0.2, 0.13, 0.2), (0, 0, 0.19), bevel=0.025)
        m.box('EQUIPO', (0.212, 0.148, 0.15), (0, 0, 0.235), bevel=0.02)
        for x in (-0.06, 0.0, 0.06):
            m.box('oliva_osc', (0.045, 0.024, 0.05), (x, -0.08, 0.245), bevel=0.006)
        m.box('oscuro', (0.212, 0.142, 0.03), (0, 0, 0.19), bevel=0.006)
        m.box('oliva_osc', (0.15, 0.07, 0.16), (0, 0.1, 0.22), bevel=0.02)
        m.box('oliva_osc', (0.13, 0.04, 0.03), (0, 0.1, 0.38), bevel=0.008)
        m.box('piel', (0.055, 0.055, 0.03), (0, 0, 0.398), bevel=0.008)
        m.box('piel', (0.1, 0.105, 0.11), (0, 0, head_z), bevel=0.035)
        m.box('negro', (0.016, 0.008, 0.016), (-0.025, -0.054, 0.48), bevel=0.003)
        m.box('negro', (0.016, 0.008, 0.016), (0.025, -0.054, 0.48), bevel=0.003)
    # Brazos: con arma, las dos manos la sostienen; sin arma, cuelgan junto al cuerpo.
    if kind == 'rocket':
        arm((0.11, 0, 0.37), (0.15, -0.05, 0.3), (0.12, -0.1, 0.36))
        arm((-0.11, 0, 0.37), (-0.04, -0.12, 0.31), (0.11, -0.19, 0.36))
    elif armed:
        arm((0.11, 0, 0.37), (0.14, -0.02, 0.27), (hx, hy + 0.06, hz - 0.01))
        arm((-0.11, 0, 0.37), (-0.05, -0.13, 0.29), (hx - 0.01, hy - 0.08, hz + 0.005))
    else:
        for sx in (-1, 1):
            arm((sx * 0.11, 0, 0.37), (sx * 0.13, -0.01, 0.28), (sx * 0.13, -0.04, 0.2))

```

```
    def helmet(color, brim=False):
        if tox:
            return
        m.sphere(color, 0.074, (0, 0.004, 0.52), scale=(1.06, 1.1, 0.72))
        m.cyl(color, 0.077, 0.024, (0, 0.004, 0.495), seg=18)
        m.box('oscuro', (0.11, 0.012, 0.018), (0, -0.07, 0.535), bevel=0.003)
        if brim:
            m.cyl(color, 0.09, 0.01, (0, -0.005, 0.495), seg=18)

    def rifle(length, color='negro', scope=False, stock='madera', barrel_r=0.012):
        m.box(color, (0.045, length * 0.55, 0.055), (hx, hy - length * 0.1, hz), bevel=0.008)
        m.box(stock, (0.04, length * 0.25, 0.05), (hx, hy + length * 0.28, hz - 0.01), bevel=0.008)
        m.cyl('metal', barrel_r, length * 0.45, (hx, hy - length * 0.38, hz + 0.028), rot=(H, 0, 0), seg=8, base=False)
        m.box('negro', (0.02, 0.04, 0.05), (hx, hy - length * 0.02, hz - 0.045), rot=(0.3, 0, 0), bevel=0.004)
        if scope:
            m.cyl('oscuro', 0.018, length * 0.22, (hx, hy - length * 0.08, hz + 0.075), rot=(H, 0, 0), seg=10,
base=False)

    if kind == 'builder':
        if not tox:
            m.cyl('naranja', 0.07, 0.05, (0, 0, 0.505), seg=18)
            m.cyl('naranja', 0.092, 0.012, (0, -0.01, 0.505), seg=18)
            m.box('naranja', (0.018, 0.1, 0.026), (0, 0, 0.552), bevel=0.006)
        # Chaleco reflectivo, viga de acero al hombro y taladro.
        m.box('naranja', (0.22, 0.152, 0.15), (0, 0, 0.225), bevel=0.02)
        for z in (0.26, 0.32):
            m.box('luz', (0.224, 0.156, 0.016), (0, 0, z), bevel=0.003)
        bx, bz = 0.13, 0.42
        m.box('metal_claro', (0.012, 0.56, 0.05), (bx, 0.02, bz), bevel=0.0)
        m.box('metal_claro', (0.05, 0.56, 0.01), (bx, 0.02, bz), bevel=0.0)
        m.box('metal_claro', (0.05, 0.56, 0.01), (bx, 0.02, bz + 0.05), bevel=0.0)
        m.box('naranja', (0.04, 0.09, 0.055), (-0.13, -0.08, 0.2), bevel=0.01)
        m.cyl('metal', 0.008, 0.06, (-0.13, -0.15, 0.225), rot=(H, 0, 0), seg=8, base=False)
    elif kind == 'engineer':
        if not tox:
            m.cyl('amarillo', 0.07, 0.05, (0, 0, 0.505), seg=18)
            m.cyl('amarillo', 0.092, 0.012, (0, -0.01, 0.505), seg=18)
            m.box('oscuro', (0.1, 0.026, 0.03), (0, -0.056, 0.52), bevel=0.008)
            m.box('vidrio', (0.08, 0.01, 0.018), (0, -0.07, 0.522), bevel=0.003)
        m.box('metal', (0.16, 0.08, 0.18), (0, 0.11, 0.21), bevel=0.02)
        m.box('naranja', (0.045, 0.09, 0.045), (0.05, 0.12, 0.38), bevel=0.008)
        m.box('metal_claro', (0.025, 0.16, 0.025), (0.13, -0.08, 0.2), bevel=0.006)
        m.cyl('metal_claro', 0.03, 0.03, (0.13, -0.17, 0.2), rot=(0, H, 0), seg=10, base=False)
    elif kind == 'medic':
        # Casco blanco con cruz roja, brazalete, botiquín en la mano y mochila médica.
        if not tox:
            m.sphere('blanco', 0.074, (0, 0.004, 0.52), scale=(1.06, 1.1, 0.72))
            m.cyl('blanco', 0.077, 0.024, (0, 0.004, 0.495), seg=18)
            m.box('rojo', (0.04, 0.01, 0.012), (0, -0.07, 0.53), bevel=0.0)
            m.box('rojo', (0.012, 0.01, 0.04), (0, -0.07, 0.53), bevel=0.0)
        m.box('blanco', (0.15, 0.075, 0.16), (0, 0.105, 0.22), bevel=0.02)
        m.box('rojo', (0.06, 0.006, 0.018), (0, 0.145, 0.26), bevel=0.0)
        m.box('rojo', (0.018, 0.006, 0.06), (0, 0.145, 0.26), bevel=0.0)
        m.box('blanco', (0.056, 0.06, 0.03), (-0.13, -0.01, 0.33), bevel=0.004)
        m.box('rojo', (0.03, 0.062, 0.01), (-0.13, -0.01, 0.335), bevel=0.0)
        m.box('blanco', (0.1, 0.05, 0.07), (0.13, -0.05, 0.14), bevel=0.012)
        m.box('rojo', (0.04, 0.052, 0.012), (0.13, -0.05, 0.17), bevel=0.0)
    elif kind == 'rifle':
        helmet('oliva')
        rifle(0.34)
        m.box('oliva_osc', (0.05, 0.05, 0.045), (hx, hy - 0.02, hz - 0.06), bevel=0.01)
    elif kind == 'shotgun':
        if not tox:
            m.cyl('oliva_osc', 0.068, 0.045, (0, 0, 0.5), seg=16)
            m.box('oliva_osc', (0.09, 0.07, 0.01), (0, -0.07, 0.5), bevel=0.004)
        m.box('madera', (0.05, 0.16, 0.065), (hx, hy + 0.02, hz), bevel=0.01)
        m.cyl('metal', 0.02, 0.18, (hx, hy - 0.13, hz + 0.02), rot=(H, 0, 0), seg=10, base=False)
        m.cyl('metal', 0.014, 0.16, (hx, hy - 0.12, hz - 0.012), rot=(H, 0, 0), seg=8, base=False)
        for k in range(4):
            m.box('rojo', (0.01, 0.018, 0.028), (-0.04 + k * 0.022, -0.082, 0.31), rot=(0, 0.5, 0), bevel=0.0)
    elif kind == 'ranger':
        helmet('oliva')
        if not tox:
            m.box('negro', (0.15, 0.016, 0.026), (0, -0.075, 0.535), bevel=0.006)
        m.box('arena', (0.222, 0.155, 0.14), (0, 0, 0.235), bevel=0.02)
        for sx in (-1, 0, 1):
            m.box('oliva_osc', (0.045, 0.02, 0.055), (sx * 0.065, -0.082, 0.25), bevel=0.006)
        rifle(0.38, scope=True)
    elif kind == 'sniper':
        if not tox:
            m.cyl('oliva', 0.11, 0.01, (0, 0, 0.51), seg=18)
            m.cyl('oliva', 0.066, 0.05, (0, 0, 0.51), seg=16, r2=0.058)
        rifle(0.62, scope=True, barrel_r=0.01)
        for sx in (-1, 1):
            m.beam('metal', (hx, hy - 0.3, hz), (hx + sx * 0.03, hy - 0.32, hz - 0.1), t=0.008)
    elif kind == 'breaker':
        helmet('tierra')
        m.box('negro', (0.055, 0.2, 0.07), (hx, hy, hz), bevel=0.012)
```

```
        m.cyl('oscuro', 0.045, 0.05, (hx, hy + 0.01, hz - 0.06), rot=(0, H, 0), seg=14, base=False)
        m.cyl('metal', 0.018, 0.14, (hx, hy - 0.15, hz + 0.02), rot=(H, 0, 0), seg=10, base=False)
    elif kind == 'hunter':
        if not tox:
            m.sphere('oscuro', 0.074, (0.008, 0, 0.51), scale=(1.1, 1.1, 0.4))
        rifle(0.64, color='oscuro', scope=True, stock='oscuro', barrel_r=0.014)
        m.cyl('oscuro', 0.026, 0.12, (hx, hy - 0.08, hz + 0.09), rot=(H, 0, 0), seg=12, base=False)
        for sx in (-1, 1):
            m.box('metal', (0.008, 0.008, 0.1), (hx + sx * 0.02, hy - 0.3, hz - 0.05), rot=(0, sx * 0.4, 0), bevel=0.0)
    elif kind == 'rocket':
        helmet('oliva')
        m.cyl('oliva', 0.044, 0.5, (0.11, -0.02, 0.41), rot=(H, 0, 0), seg=14, base=False)
        m.cyl('oscuro', 0.05, 0.05, (0.11, -0.27, 0.41), rot=(H, 0, 0), seg=14, base=False)
        m.cone('rojo', 0.038, 0.09, (0.11, -0.31, 0.41), rot=(H, 0, 0), seg=12)
        m.box('oscuro', (0.03, 0.06, 0.05), (0.11, -0.08, 0.36), bevel=0.006)
    return m


def wheels(m, positions, r=0.085, w=0.07, part='main'):
    for (x, y) in positions:
        m.cyl('goma', r, w, (x, y, r), rot=(0, H, 0), seg=16, base=False, part=part)
        m.cyl('metal', r * 0.55, w + 0.01, (x, y, r), rot=(0, H, 0), seg=10, base=False, part=part)


def tracks(m, x, length=0.92, n=6, r=0.048):
    for sx in (-1, 1):
        m.box('goma', (0.13, length, 0.14), (sx * x, 0, 0.02), bevel=0.05)
        for k in range(n):
            m.cyl('metal', r, 0.135, (sx * x, -length * 0.4 + k * length * 0.8 / (n - 1), 0.075), rot=(0, H, 0),
                  seg=12, base=False)


def launcher_face(m, cx, cy, cz, half_y, th, cols, rows, dx, dz, part):
    """Celdas de misiles con una X blanca en la cara delantera de una caja inclinada th (rot X)."""
    c, s = math.cos(th), math.sin(th)

    def fp(x, zc, dy):
        y0, z0 = -half_y - dy, zc
        return (cx + x, cy + y0 * c - z0 * s, cz + y0 * s + z0 * c)
    for ix in range(cols):
        for iz in range(rows):
            x = (ix - (cols - 1) / 2) * dx
            z = (iz - (rows - 1) / 2) * dz
            m.beam('oscuro', fp(x, z, -0.004), fp(x, z, 0.012), t=dx * 0.82, part=part)
            for sgn in (-1, 1):
                m.beam('blanco', fp(x - dx * 0.28, z - sgn * dz * 0.28, 0.014), fp(x + dx * 0.28, z + sgn * dz * 0.28,
0.014),
                       t=0.011, part=part)


# ---------------------------------------------------------------------
# Vehículos
# ---------------------------------------------------------------------
def jeep():
    """Vehículo ancho tipo Humvee con un artillero en la torreta del techo."""
    m = Model('jeep')
    wheels(m, [(-0.25, -0.27), (0.25, -0.27), (-0.25, 0.27), (0.25, 0.27)], r=0.1, w=0.09)
    m.box('EQUIPO_OSC', (0.42, 0.86, 0.1), (0, 0.01, 0.09), bevel=0.03)
    for sx in (-1, 1):
        for y in (-0.27, 0.27):
            m.box('EQUIPO', (0.13, 0.27, 0.045), (sx * 0.25, y, 0.2), bevel=0.015)
    m.box('EQUIPO', (0.5, 0.3, 0.09), (0, -0.27, 0.19), bevel=0.025)
    m.box('oscuro', (0.36, 0.02, 0.1), (0, -0.425, 0.12), bevel=0.005)
    for k in range(6):
        m.box('negro', (0.012, 0.024, 0.07), (-0.125 + k * 0.05, -0.437, 0.135), bevel=0.0)
    for sx in (-1, 1):
        m.box('luz', (0.06, 0.02, 0.04), (sx * 0.19, -0.428, 0.2), bevel=0.006)
        m.box('luz', (0.03, 0.02, 0.02), (sx * 0.12, -0.428, 0.25), bevel=0.004)
    m.box('negro', (0.46, 0.04, 0.05), (0, -0.44, 0.07), bevel=0.01)
    m.box('EQUIPO', (0.48, 0.42, 0.16), (0, 0.11, 0.22), bevel=0.03)
    m.box('luz', (0.42, 0.02, 0.1), (0, -0.1, 0.25), rot=(-0.28, 0, 0), bevel=0.005)
    m.box('oscuro', (0.03, 0.02, 0.11), (0, -0.103, 0.25), rot=(-0.28, 0, 0), bevel=0.0)
    for sx in (-1, 1):
        m.box('luz', (0.012, 0.13, 0.08), (sx * 0.242, 0.0, 0.27), bevel=0.003)
        m.box('luz', (0.012, 0.12, 0.08), (sx * 0.242, 0.2, 0.27), bevel=0.003)
    m.box('EQUIPO_OSC', (0.46, 0.44, 0.03), (0, 0.11, 0.38), bevel=0.01)
    m.box('EQUIPO_OSC', (0.46, 0.14, 0.13), (0, 0.38, 0.19), bevel=0.02)
    m.box('oscuro', (0.1, 0.03, 0.08), (0.15, 0.455, 0.22), bevel=0.01)
    # Torreta con artillero (gira hacia el enemigo).
    m.pivot('turret', 0, 0.12, 0.41)
    m.cyl('oscuro', 0.12, 0.035, (0, 0.12, 0.41), seg=16, part='turret')
    m.box('EQUIPO_OSC', (0.22, 0.02, 0.06), (0, 0.02, 0.44), bevel=0.006, part='turret')
    for sx in (-1, 1):
        m.box('EQUIPO_OSC', (0.02, 0.1, 0.06), (sx * 0.11, 0.07, 0.44), bevel=0.006, part='turret')
    m.box('oliva_osc', (0.13, 0.09, 0.11), (0, 0.13, 0.43), bevel=0.02, part='turret')
    m.box('piel', (0.085, 0.08, 0.085), (0, 0.13, 0.54), bevel=0.02, part='turret')
    m.sphere('oliva', 0.056, (0, 0.13, 0.62), scale=(1, 1.05, 0.65), part='turret')
    m.cyl('oliva', 0.058, 0.02, (0, 0.13, 0.6), seg=14, part='turret')
```

```
    for sx in (-1, 1):
        m.beam('oliva_osc', (sx * 0.07, 0.12, 0.52), (sx * 0.03, 0.02, 0.5), t=0.035, part='turret')
    m.cyl('oscuro', 0.012, 0.06, (0, 0.03, 0.46), seg=8, part='turret')
    m.box('negro', (0.04, 0.16, 0.045), (0, -0.02, 0.5), bevel=0.008, part='turret')
    m.box('oliva_osc', (0.05, 0.05, 0.05), (0.04, 0.03, 0.49), bevel=0.008, part='turret')
    m.tube('metal', (0, -0.1, 0.523), (0, -0.28, 0.523), r=0.009, part='turret')
    return m


def tank():
    m = Model('tank')
    tracks(m, 0.27, 0.92, 6, 0.05)
    for sx in (-1, 1):
        m.box('EQUIPO_OSC', (0.03, 0.8, 0.09), (sx * 0.345, 0.0, 0.08), bevel=0.01)
        m.box('oliva', (0.16, 0.88, 0.03), (sx * 0.27, 0, 0.17), bevel=0.01)
    m.box('oliva', (0.48, 0.84, 0.16), (0, 0.02, 0.09), bevel=0.04)
    m.box('oliva_osc', (0.46, 0.2, 0.1), (0, -0.39, 0.12), rot=(0.55, 0, 0), bevel=0.03)
    for sx in (-1, 1):
        m.box('luz', (0.05, 0.02, 0.025), (sx * 0.17, -0.47, 0.18), bevel=0.004)
    m.box('EQUIPO', (0.5, 0.18, 0.025), (0, 0.29, 0.25), bevel=0.008)
    m.box('oscuro', (0.22, 0.12, 0.06), (0, 0.35, 0.25), bevel=0.015)
    m.pivot('turret', 0, 0.04, 0.25)
    m.box('EQUIPO', (0.36, 0.44, 0.13), (0, 0.07, 0.25), bevel=0.05, part='turret')
    m.box('EQUIPO_OSC', (0.3, 0.14, 0.1), (0, -0.16, 0.26), rot=(0.25, 0, 0), bevel=0.03, part='turret')
    m.box('oscuro', (0.38, 0.12, 0.08), (0, 0.27, 0.26), bevel=0.02, part='turret')
    m.cyl('oscuro', 0.035, 0.6, (0, -0.47, 0.31), rot=(H, 0, 0), seg=12, base=False, part='turret')
    m.cyl('oscuro', 0.045, 0.08, (0, -0.74, 0.31), rot=(H, 0, 0), seg=12, base=False, part='turret')
    m.cyl('oliva_osc', 0.06, 0.04, (0.09, 0.12, 0.38), seg=14, part='turret')
    m.box('luz', (0.05, 0.02, 0.025), (-0.1, -0.1, 0.36), bevel=0.004, part='turret')
    for sx in (-1, 1):
        m.tube('negro', (sx * 0.13, 0.25, 0.38), (sx * 0.15, 0.3, 0.85), r=0.006, seg=6, part='turret')
    return m


def arty():
    """Lanzamisiles sobre orugas: cabina adelante y caja lanzadora que gira."""
    m = Model('arty')
    tracks(m, 0.24, 0.94, 6, 0.046)
    m.box('oliva_osc', (0.4, 0.92, 0.1), (0, 0, 0.12), bevel=0.02)
    m.box('EQUIPO', (0.42, 0.26, 0.2), (0, -0.33, 0.2), bevel=0.04)
    m.box('vidrio', (0.36, 0.02, 0.08), (0, -0.462, 0.31), rot=(-0.2, 0, 0), bevel=0.005)
    m.box('oscuro', (0.3, 0.02, 0.07), (0, -0.462, 0.225), bevel=0.005)
    for k in range(4):
        m.box('luz', (0.24, 0.012, 0.008), (0, -0.473, 0.205 + k * 0.016), bevel=0.0)
    for sx in (-1, 1):
        m.box('luz', (0.04, 0.02, 0.03), (sx * 0.17, -0.462, 0.27), bevel=0.004)
    m.pivot('turret', 0, 0.16, 0.22)
    m.box('oliva_osc', (0.3, 0.32, 0.06), (0, 0.16, 0.22), bevel=0.02, part='turret')
    th = -0.38
    m.box('EQUIPO', (0.38, 0.5, 0.22), (0, 0.13, 0.27), rot=(th, 0, 0), bevel=0.03, part='turret')
    launcher_face(m, 0, 0.13, 0.38, 0.25, th, 3, 2, 0.11, 0.09, 'turret')
    for sx in (-1, 1):
        m.beam('oscuro', (sx * 0.12, 0.3, 0.25), (sx * 0.12, 0.28, 0.42), t=0.03, part='turret')
    return m


# ---------------------------------------------------------------------
# Barcos
# ---------------------------------------------------------------------
def speedboat():
    """Bote neumático rápido con consola, asientos, arco trasero y ametralladora en proa."""
    m = Model('speedboat')
    m.hull('oscuro', [(0.42, 0.16, 0.1, -0.04, 0.08), (0.1, 0.17, 0.1, -0.06, 0.08), (-0.2, 0.15, 0.06, -0.05, 0.09),
                      (-0.42, 0.06, 0.02, -0.01, 0.1), (-0.5, 0.01, 0.01, 0.04, 0.1)])
    pts = [(0.2, 0.44), (0.2, 0.1), (0.18, -0.2), (0.1, -0.42), (0.0, -0.53)]
    for sx in (-1, 1):
        for (x1, y1), (x2, y2) in zip(pts, pts[1:]):
            m.tube('gris', (sx * x1, y1, 0.1), (sx * x2, y2, 0.11), r=0.062, seg=12)
        m.sphere('gris', 0.062, (sx * 0.2, 0.44, 0.1))
        m.tube('EQUIPO', (sx * 0.21, 0.42, 0.158), (sx * 0.2, -0.18, 0.165), r=0.013, seg=8)
    m.box('concreto', (0.3, 0.8, 0.012), (0, 0.0, 0.08), bevel=0.0)
    m.box('blanco_sucio', (0.16, 0.12, 0.12), (0, 0.0, 0.09), bevel=0.02)
    m.box('vidrio', (0.15, 0.012, 0.07), (0, -0.066, 0.2), rot=(-0.5, 0, 0), bevel=0.003)
    for (x, y) in [(-0.07, 0.16), (0.07, 0.16), (0, 0.3)]:
        m.box('EQUIPO', (0.1, 0.1, 0.07), (x, y, 0.09), bevel=0.02)
    for sx in (-1, 1):
        m.tube('metal_claro', (sx * 0.15, 0.36, 0.1), (sx * 0.13, 0.32, 0.42), r=0.012, seg=8)
    m.tube('metal_claro', (-0.13, 0.32, 0.42), (0.13, 0.32, 0.42), r=0.012, seg=8)
    m.tube('metal_claro', (0, 0.32, 0.42), (0, 0.32, 0.55), r=0.006, seg=6)
    m.sphere('luz', 0.018, (0, 0.32, 0.56))
    m.box('negro', (0.08, 0.08, 0.12), (0, 0.5, 0.05), bevel=0.02)
    m.box('negro', (0.03, 0.03, 0.12), (0, 0.5, -0.06), bevel=0.0)
    m.cyl('oscuro', 0.02, 0.06, (0, -0.25, 0.09), seg=8)
    m.box('negro', (0.035, 0.12, 0.04), (0, -0.27, 0.15), bevel=0.008)
    m.tube('metal', (0, -0.33, 0.17), (0, -0.46, 0.17), r=0.008, seg=6)
    return m


```

```
def boat_engineer():
    """Barco constructor tipo buque de apoyo: puente adelante, cubierta larga y grúa."""
    m = Model('boatEng')
    m.hull('amarillo', [(0.56, 0.24, 0.18, -0.05, 0.12), (0.0, 0.25, 0.18, -0.07, 0.13), (-0.34, 0.22, 0.12, -0.06,
0.15),
                        (-0.58, 0.05, 0.02, 0.0, 0.2)])
    for sx in (-1, 1):
        m.box('EQUIPO', (0.012, 1.0, 0.035), (sx * 0.25, 0.0, 0.08), rot=(0, 0, sx * 0.04), bevel=0.0)
    m.box('concreto', (0.46, 0.62, 0.012), (0, 0.22, 0.125), bevel=0.0)
    m.box('amarillo', (0.42, 0.3, 0.16), (0, -0.26, 0.13), bevel=0.03)
    m.box('amarillo', (0.36, 0.24, 0.12), (0, -0.25, 0.29), bevel=0.03)
    m.box('luz', (0.34, 0.012, 0.05), (0, -0.372, 0.33), bevel=0.003)
    m.box('luz', (0.38, 0.012, 0.04), (0, -0.412, 0.2), bevel=0.003)
    for sx in (-1, 1):
        m.box('luz', (0.012, 0.18, 0.04), (sx * 0.182, -0.25, 0.34), bevel=0.003)
    m.box('EQUIPO', (0.38, 0.26, 0.025), (0, -0.25, 0.41), bevel=0.008)
    m.tube('blanco', (0, -0.22, 0.43), (0, -0.22, 0.66), r=0.012, seg=8)
    m.box('blanco', (0.16, 0.02, 0.02), (0, -0.22, 0.58), bevel=0.0)
    m.sphere('blanco', 0.032, (0.1, -0.18, 0.46))
    m.sphere('blanco', 0.026, (-0.1, -0.18, 0.46))
    m.cyl('naranja', 0.05, 0.1, (0.13, 0.08, 0.13), seg=12)
    m.tube('naranja', (0.13, 0.08, 0.23), (0.13, 0.46, 0.58), r=0.022, seg=8)
    m.tube('oscuro', (0.13, 0.46, 0.58), (0.13, 0.46, 0.33), r=0.004, seg=6)
    m.box('metal', (0.05, 0.05, 0.04), (0.13, 0.46, 0.29), bevel=0.008)
    for (x, y, mat) in [(-0.12, 0.13, 'EQUIPO_OSC'), (-0.12, 0.31, 'blanco_sucio'), (0.0, 0.43, 'EQUIPO')]:
        m.box(mat, (0.12, 0.15, 0.09), (x, y, 0.13), bevel=0.01)
    return m


def barge():
    """Barcaza tipo aerodeslizador: faldón de goma, rampa adelante y dos hélices grandes atrás."""
    m = Model('barge')
    m.box('goma', (0.8, 1.32, 0.12), (0, 0, -0.04), bevel=0.06)
    m.box('gris', (0.72, 1.22, 0.08), (0, 0, 0.06), bevel=0.03)
    m.box('oscuro', (0.5, 0.86, 0.012), (0, -0.06, 0.14), bevel=0.0)
    for k in range(4):
        m.box('amarillo', (0.44, 0.03, 0.004), (0, -0.4 + k * 0.22, 0.152), bevel=0.0)
    for sx in (-1, 1):
        m.box('gris', (0.09, 1.1, 0.13), (sx * 0.31, -0.02, 0.14), bevel=0.02)
        m.box('EQUIPO', (0.095, 1.0, 0.03), (sx * 0.31, -0.02, 0.25), bevel=0.006)
        for k in range(3):
            m.box('luz', (0.012, 0.05, 0.02), (sx * 0.357, -0.4 + k * 0.35, 0.2), bevel=0.0)
    m.box('gris', (0.5, 0.06, 0.13), (0, -0.62, 0.11), rot=(0.45, 0, 0), bevel=0.01)
    m.box('blanco_sucio', (0.15, 0.22, 0.15), (0.29, -0.2, 0.27), bevel=0.02)
    m.box('luz', (0.13, 0.012, 0.04), (0.29, -0.312, 0.36), bevel=0.0)
    m.box('EQUIPO', (0.16, 0.23, 0.02), (0.29, -0.2, 0.42), bevel=0.005)
    for sx in (-1, 1):
        x, y, z = sx * 0.18, 0.52, 0.38
        m.cyl('gris', 0.16, 0.09, (x, y, z), rot=(H, 0, 0), seg=22, base=False)
        m.cyl('oscuro', 0.13, 0.095, (x, y, z), rot=(H, 0, 0), seg=22, base=False)
        for k in range(3):
            a = k * PI / 3
            m.beam('metal_claro', (x + math.cos(a) * 0.12, y - 0.05, z + math.sin(a) * 0.12),
                   (x - math.cos(a) * 0.12, y - 0.05, z - math.sin(a) * 0.12), t=0.025)
        m.cyl('metal', 0.03, 0.03, (x, y - 0.06, z), rot=(H, 0, 0), seg=10, base=False)
        m.beam('gris', (x, y, 0.14), (x, y, z - 0.15), t=0.04)
        m.box('EQUIPO_OSC', (0.015, 0.08, 0.2), (x, y + 0.1, 0.24), bevel=0.0)
    return m


def destroyer():
    m = Model('destroyer')
    m.hull('gris', [(0.88, 0.22, 0.15, -0.06, 0.15), (0.4, 0.26, 0.17, -0.09, 0.16), (-0.3, 0.25, 0.14, -0.09, 0.17),
                    (-0.7, 0.14, 0.05, -0.05, 0.19), (-0.98, 0.015, 0.01, 0.04, 0.23)])
    for sx in (-1, 1):
        m.box('EQUIPO', (0.012, 1.4, 0.03), (sx * 0.245, -0.03, 0.12), rot=(0, 0, sx * 0.05), bevel=0.0)
    m.box('metal_claro', (0.44, 1.5, 0.015), (0, -0.05, 0.16), bevel=0.0)
    m.box('gris', (0.34, 0.54, 0.14), (0, 0.12, 0.17), bevel=0.02)
    m.box('metal_claro', (0.28, 0.36, 0.14), (0, 0.04, 0.31), bevel=0.02)
    m.box('gris', (0.22, 0.2, 0.1), (0, -0.0, 0.45), bevel=0.02)
    m.box('luz', (0.24, 0.012, 0.03), (0, -0.142, 0.39), bevel=0.0)
    for sx in (-1, 1):
        m.box('luz', (0.012, 0.28, 0.02), (sx * 0.172, 0.08, 0.27), bevel=0.0)
        m.box('oscuro', (0.012, 0.12, 0.1), (sx * 0.142, -0.06, 0.34), rot=(0, sx * 0.25, 0), bevel=0.0)
    base, top = 0.55, 0.98
    for (x, y) in [(-0.06, -0.06), (0.06, -0.06), (0.06, 0.06), (-0.06, 0.06)]:
        m.beam('oscuro', (x, y + 0.0, base), (x * 0.25, y * 0.25, top), t=0.014)
    for z in (0.66, 0.78, 0.88):
        k = 1 - (z - base) / (top - base) * 0.75
        m.box('oscuro', (0.12 * k + 0.014, 0.012, 0.012), (0, -0.06 * k, z), bevel=0.0)
        m.box('oscuro', (0.12 * k + 0.014, 0.012, 0.012), (0, 0.06 * k, z), bevel=0.0)
    m.box('oscuro', (0.24, 0.012, 0.012), (0, 0, 0.84), bevel=0.0)
    m.sphere('blanco', 0.04, (0, 0, 1.01))
    m.box('gris', (0.14, 0.16, 0.18), (0, 0.36, 0.31), bevel=0.03)
    m.box('negro', (0.12, 0.14, 0.02), (0, 0.36, 0.49), bevel=0.0)
    m.cyl('blanco', 0.04, 0.06, (0, 0.5, 0.31), seg=12)
    m.sphere('blanco', 0.04, (0, 0.5, 0.37))
    for ix in range(4):
```

```
        for iy in range(2):
            m.box('oscuro', (0.05, 0.05, 0.012), (-0.09 + ix * 0.06, -0.22 + iy * 0.06, 0.163), bevel=0.0)
    m.box('oscuro', (0.36, 0.28, 0.01), (0, 0.62, 0.162), bevel=0.0)
    m.box('blanco', (0.03, 0.14, 0.004), (-0.05, 0.62, 0.17), bevel=0.0)
    m.box('blanco', (0.03, 0.14, 0.004), (0.05, 0.62, 0.17), bevel=0.0)
    m.box('blanco', (0.1, 0.03, 0.004), (0, 0.62, 0.17), bevel=0.0)
    m.pivot('turret', 0, -0.46, 0.17)
    m.cyl('gris', 0.1, 0.04, (0, -0.46, 0.17), seg=16, part='turret')
    m.box('metal_claro', (0.18, 0.2, 0.1), (0, -0.47, 0.2), bevel=0.05, part='turret')
    m.box('metal_claro', (0.14, 0.08, 0.08), (0, -0.58, 0.2), rot=(0.4, 0, 0), bevel=0.02, part='turret')
    m.tube('oscuro', (0, -0.6, 0.26), (0, -0.92, 0.27), r=0.022, seg=10, part='turret')
    return m


# ---------------------------------------------------------------------
# Helicóptero
# ---------------------------------------------------------------------
def helicopter():
    """Helicóptero con cabina grande, patines y una minigun en cada ala corta."""
    m = Model('heli')
    m.box('EQUIPO', (0.27, 0.5, 0.24), (0, 0.03, 0.1), bevel=0.08)
    m.sphere('vidrio', 0.155, (0, -0.21, 0.23), scale=(0.92, 1.05, 0.9))
    m.box('EQUIPO_OSC', (0.2, 0.22, 0.08), (0, 0.06, 0.33), bevel=0.03)
    m.box('oscuro', (0.06, 0.08, 0.04), (0.08, 0.13, 0.4), bevel=0.01)
    m.tube('EQUIPO_OSC', (0, 0.26, 0.24), (0, 0.84, 0.3), r=0.045, r2=0.024, seg=10)
    m.box('EQUIPO_OSC', (0.02, 0.14, 0.18), (0, 0.82, 0.34), rot=(0.3, 0, 0), bevel=0.01)
    m.box('EQUIPO_OSC', (0.22, 0.06, 0.015), (0, 0.76, 0.29), bevel=0.005)
    m.cyl('oscuro', 0.07, 0.01, (0.03, 0.86, 0.35), rot=(0, H, 0), seg=14, base=False)
    for sx in (-1, 1):
        m.tube('oscuro', (sx * 0.17, -0.3, 0.0), (sx * 0.17, 0.3, 0.0), r=0.015, seg=8)
        m.tube('oscuro', (sx * 0.17, -0.3, 0.0), (sx * 0.17, -0.36, 0.05), r=0.015, seg=8)
        m.tube('oscuro', (sx * 0.17, -0.13, 0.0), (sx * 0.1, -0.12, 0.11), r=0.01, seg=6)
        m.tube('oscuro', (sx * 0.17, 0.16, 0.0), (sx * 0.1, 0.15, 0.11), r=0.01, seg=6)
        m.box('oliva_osc', (0.16, 0.07, 0.02), (sx * 0.2, 0.03, 0.19), bevel=0.006)
        x, y, z = sx * 0.29, 0.03, 0.16
        m.cyl('oscuro', 0.036, 0.1, (x, y + 0.02, z), rot=(H, 0, 0), seg=10, base=False)
        for k in range(6):
            a = k * PI / 3
            m.tube('negro', (x + math.cos(a) * 0.018, y - 0.03, z + math.sin(a) * 0.018),
                   (x + math.cos(a) * 0.018, y - 0.22, z + math.sin(a) * 0.018), r=0.006, seg=6)
        m.cyl('metal', 0.03, 0.015, (x, y - 0.2, z), rot=(H, 0, 0), seg=10, base=False)
        # v0.9: lleva el equipo del Apache viejo: cohetes bajo el ala y un artillero en cada puerta.
        m.cyl('oliva_osc', 0.034, 0.17, (sx * 0.2, 0.06, 0.135), rot=(H, 0, 0), seg=10, base=False)
        for k in range(3):
            a = k * 2 * PI / 3 + PI / 6
            m.cyl('blanco', 0.01, 0.01, (sx * 0.2 + math.cos(a) * 0.017, -0.03, 0.135 + math.sin(a) * 0.017),
                  rot=(H, 0, 0), seg=6, base=False)
        m.box('negro', (0.012, 0.16, 0.13), (sx * 0.136, 0.08, 0.13), bevel=0.0)
        m.box('oliva', (0.06, 0.06, 0.08), (sx * 0.1, 0.08, 0.14), bevel=0.015)
        m.sphere('oliva', 0.032, (sx * 0.1, 0.08, 0.21), scale=(1, 1, 0.85))
        m.box('piel', (0.03, 0.02, 0.025), (sx * 0.112, 0.06, 0.19), bevel=0.005)
        m.tube('negro', (sx * 0.11, 0.06, 0.16), (sx * 0.22, 0.0, 0.13), r=0.008, seg=6)
        m.box('oscuro', (0.03, 0.05, 0.03), (sx * 0.13, 0.06, 0.155), bevel=0.005)
    m.box('luz', (0.03, 0.02, 0.02), (0, -0.36, 0.08), bevel=0.004)
    m.cyl('oscuro', 0.035, 0.06, (0, 0.05, 0.41), seg=10)
    m.pivot('rotor', 0, 0.05, 0.48)
    m.cyl('oscuro', 0.045, 0.04, (0, 0.05, 0.47), seg=12, part='rotor')
    m.box('oscuro', (1.34, 0.07, 0.012), (0, 0.05, 0.48), bevel=0.004, part='rotor')
    m.box('oscuro', (0.07, 1.0, 0.012), (0, 0.05, 0.482), bevel=0.004, part='rotor')
    return m


# ---------------------------------------------------------------------
# Edificios
# ---------------------------------------------------------------------
def crystal(m, z, size=0.2):
    m.pivot('crystal', 0, 0, z)
    m.cone('EQUIPO', size, size * 1.35, (0, 0, z), seg=4, part='crystal')
    m.cone('EQUIPO', size, size * 1.35, (0, 0, z), rot=(PI, 0, 0), seg=4, part='crystal')


def core(level):
    """Núcleo: fortaleza hexagonal con muros, torres y el cristal ámbar en el centro."""
    m = Model(f'core_{level}')
    m.hexprism('oscuro', 0.98, 0.12)
    m.hexprism('piedra', 0.92, 0.08, (0, 0, 0.12))
    wall_h = (0.16, 0.22, 0.28)[level - 1]
    rw = 0.76
    z0 = 0.2
    cs = [(math.cos(PI / 6 + k * PI / 3) * rw, math.sin(PI / 6 + k * PI / 3) * rw) for k in range(6)]
    for k in range(6):
        (x1, y1), (x2, y2) = cs[k], cs[(k + 1) % 6]
        ang = math.atan2(y2 - y1, x2 - x1)
        gate = k in (3, 4)
        pieces = [(0.0, 0.36), (0.64, 1.0)] if gate else [(0.0, 1.0)]
        for (a, b) in pieces:
            px1, py1 = x1 + (x2 - x1) * a, y1 + (y2 - y1) * a
            px2, py2 = x1 + (x2 - x1) * b, y1 + (y2 - y1) * b
```

```
            ln = math.hypot(px2 - px1, py2 - py1)
            cx, cy = (px1 + px2) / 2, (py1 + py2) / 2
            m.box('piedra_clara', (ln, 0.08, wall_h), (cx, cy, z0), rot=(0, 0, ang), bevel=0.012)
            m.box('EQUIPO', (ln, 0.086, 0.03), (cx, cy, z0 + wall_h * 0.45), rot=(0, 0, ang), bevel=0.004)
            n = max(1, int(ln / 0.09))
            for i in range(n):
                t = (i + 0.5) / n
                mx, my = px1 + (px2 - px1) * t, py1 + (py2 - py1) * t
                if i % 2 == 0:
                    m.box('piedra_clara', (0.045, 0.085, 0.04), (mx, my, z0 + wall_h), rot=(0, 0, ang), bevel=0.006)
        if level == 3:
            m.box('oro', (math.hypot(x2 - x1, y2 - y1) * 0.98, 0.09, 0.015), ((x1 + x2) / 2, (y1 + y2) / 2, z0 + wall_h
- 0.015),
                  rot=(0, 0, ang), bevel=0.0)
    towers = [0, 2, 4] if level == 1 else range(6)
    th = (0.32, 0.4, 0.52)[level - 1]
    for k in towers:
        x, y = cs[k]
        m.cyl('piedra_clara', 0.12, th, (x, y, z0), seg=6, smooth=False)
        m.cyl('oscuro', 0.135, 0.04, (x, y, z0 + th), seg=6, smooth=False)
        if level == 3:
            m.cone('EQUIPO', 0.15, 0.2, (x, y, z0 + th + 0.04), seg=6)
        else:
            for j in range(3):
                a = j * 2 * PI / 3
                m.box('piedra_clara', (0.05, 0.05, 0.05), (x + math.cos(a) * 0.09, y + math.sin(a) * 0.09, z0 + th +
0.04), bevel=0.006)
        m.box('luz', (0.03, 0.012, 0.05), (x, y - 0.118, z0 + th * 0.6), bevel=0.0)
    m.hexprism('oscuro', 0.44, 0.1, (0, 0, z0))
    m.cyl('luz', 0.32, 0.012, (0, 0, z0 + 0.1), seg=6, smooth=False)
    ped = (0.32, 0.55, 0.8)[level - 1]
    m.cyl('EQUIPO_OSC', 0.2, ped, (0, 0, z0 + 0.11), seg=6, r2=0.14, smooth=False)
    if level >= 2:
        for k in range(3):
            a = k * 2 * PI / 3 + PI / 6
            m.beam('piedra_clara', (math.cos(a) * 0.3, math.sin(a) * 0.3, z0 + 0.1), (math.cos(a) * 0.12, math.sin(a) *
0.12, z0 + ped * 0.8), t=0.06)
    if level == 3:
        m.cyl('oro', 0.18, 0.04, (0, 0, z0 + ped * 0.55), seg=6, smooth=False)
        m.cyl('oro', 0.16, 0.04, (0, 0, z0 + ped + 0.06), seg=6, smooth=False)
    # v0.9: banderas del imperio en las torres y portón en la entrada.
    for k in towers:
        x, y = cs[k]
        zt = z0 + th + (0.24 if level == 3 else 0.09)
        m.beam('metal', (x, y, zt - 0.06), (x, y, zt + 0.16), t=0.012)
        m.box('EQUIPO', (0.01, 0.11, 0.065), (x, y + 0.058, zt + 0.12), bevel=0.0)
    (gx1, gy1), (gx2, gy2) = cs[3], cs[4]
    gang = math.atan2(gy2 - gy1, gx2 - gx1)
    m.box('madera', (0.2, 0.03, wall_h * 0.85), ((gx1 + gx2) / 2, (gy1 + gy2) / 2, z0), rot=(0, 0, gang), bevel=0.006)
    m.box('metal', (0.21, 0.035, 0.018), ((gx1 + gx2) / 2, (gy1 + gy2) / 2, z0 + wall_h * 0.6), rot=(0, 0, gang),
bevel=0.0)
    size = (0.2, 0.24, 0.28)[level - 1]
    crystal(m, z0 + ped + 0.11 + size * 1.35, size)
    return m


def farm():
    """Granja: establo rojo, silo y parcelas de trigo dentro de una cerca."""
    m = Model('farm')
    m.box('tierra', (1.2, 0.92, 0.04), (0.08, 0.12, 0), bevel=0.01)
    for k in range(4):
        m.box('trigo', (0.86, 0.12, 0.08), (0.14, -0.15 + k * 0.19, 0.04), bevel=0.035)
    x1, x2, y1, y2 = -0.36, 0.64, -0.3, 0.6
    posts = []
    for i in range(7):
        posts.append((x1 + (x2 - x1) * i / 6, y1))
        posts.append((x1 + (x2 - x1) * i / 6, y2))
    for i in range(1, 5):
        posts.append((x2, y1 + (y2 - y1) * i / 5))
        posts.append((x1, y1 + (y2 - y1) * i / 5))
    for (x, y) in posts:
        m.box('madera', (0.025, 0.025, 0.13), (x, y, 0.0), bevel=0.0)
    for z in (0.06, 0.11):
        m.beam('madera_clara', (x1, y1, z), (x2, y1, z), t=0.015)
        m.beam('madera_clara', (x1, y2, z), (x2, y2, z), t=0.015)
        m.beam('madera_clara', (x2, y1, z), (x2, y2, z), t=0.015)
        m.beam('madera_clara', (x1, y1 + 0.2, z), (x1, y2, z), t=0.015)
    m.box('rojo', (0.4, 0.44, 0.3), (-0.5, -0.42, 0), bevel=0.02)
    m.wedge('EQUIPO', (0.46, 0.5, 0.2), (-0.5, -0.42, 0.3), rot=(0, 0, H))
    m.box('blanco', (0.16, 0.012, 0.2), (-0.5, -0.642, 0.0), bevel=0.003)
    m.box('oscuro', (0.12, 0.014, 0.17), (-0.5, -0.645, 0.0), bevel=0.003)
    for sx in (-1, 1):
        m.box('luz', (0.06, 0.012, 0.06), (-0.5 + sx * 0.13, -0.642, 0.15), bevel=0.002)
    m.box('ladrillo', (0.06, 0.06, 0.14), (-0.42, -0.32, 0.38), bevel=0.006)
    m.cyl('blanco', 0.11, 0.55, (-0.12, -0.54, 0), seg=14)
    m.cone('EQUIPO_OSC', 0.12, 0.14, (-0.12, -0.54, 0.55), seg=14, smooth=True)
    m.box('trigo', (0.12, 0.08, 0.08), (-0.25, -0.2, 0.0), bevel=0.02)
    return m
```

```
605
606
607 def mine():
608     """Mina de oro: rocas, entrada de madera iluminada, rieles y vagoneta con oro."""
609     m = Model('mine')
610     m.ico('piedra', 0.6, (0.06, 0.2, 0.1), scale=(1.15, 0.9, 0.75), sub=1)
611     m.ico('piedra_clara', 0.3, (-0.42, 0.3, 0.05), scale=(1, 1, 0.85), sub=1)
612     m.ico('piedra', 0.24, (0.48, -0.05, 0.04), scale=(1, 1, 0.8), sub=1)
613     m.box('negro', (0.32, 0.2, 0.3), (0.0, -0.36, 0.0), bevel=0.02)
614     m.box('luz', (0.2, 0.012, 0.2), (0.0, -0.463, 0.03), bevel=0.0)
615     m.box('madera', (0.05, 0.05, 0.36), (-0.18, -0.47, 0.0), bevel=0.01)
616     m.box('madera', (0.05, 0.05, 0.36), (0.18, -0.47, 0.0), bevel=0.01)
617     m.box('madera', (0.46, 0.07, 0.07), (0.0, -0.47, 0.33), bevel=0.01)
618     for sx in (-1, 1):
619         m.beam('metal', (sx * 0.06, -0.47, 0.012), (sx * 0.06, -0.92, 0.012), t=0.015)
620     for k in range(5):
621         m.box('madera', (0.18, 0.03, 0.012), (0, -0.5 - k * 0.1, 0.0), bevel=0.0)
622     m.box('metal', (0.17, 0.22, 0.1), (0, -0.74, 0.04), bevel=0.02)
623     for sx in (-1, 1):
624         for y in (-0.67, -0.81):
625             m.cyl('oscuro', 0.025, 0.02, (sx * 0.08, y, 0.03), rot=(0, H, 0), seg=10, base=False)
626     for k, (x, y) in enumerate([(-0.04, -0.76), (0.04, -0.72), (0.0, -0.8), (0.02, -0.74)]):
627         m.ico('oro', 0.045, (x, y, 0.15 + (k % 2) * 0.02), sub=0)
628     for (x, y, z) in [(0.3, 0.22, 0.42), (-0.1, 0.36, 0.5), (0.42, -0.05, 0.22), (-0.38, 0.22, 0.28)]:
629         m.ico('oro', 0.06, (x, y, z), sub=0)
630     m.tube('oscuro', (0.3, -0.48, 0.0), (0.3, -0.48, 0.38), r=0.012, seg=6)
631     m.sphere('luz', 0.03, (0.3, -0.48, 0.4))
632     return m
633
634
635 def rig():
636     """Plataforma petrolera: patas, torre de perforación, grúas, helipuerto y módulo de vivienda."""
637     m = Model('rig')
638     legs = [(-0.42, -0.42), (0.42, -0.42), (-0.42, 0.42), (0.42, 0.42)]
639     for (x, y) in legs:
640         m.cyl('amarillo', 0.08, 0.76, (x, y, -0.36), seg=14)
641         m.cyl('blanco', 0.083, 0.05, (x, y, 0.0), seg=14)
642     for i in range(4):
643         (xa, ya), (xb, yb) = legs[[0, 1, 3, 2][i]], legs[[1, 3, 2, 0][i]]
644         m.beam('metal', (xa, ya, 0.12), (xb, yb, 0.12), t=0.03)
645         m.beam('metal', (xa, ya, -0.1), (xb, yb, 0.3), t=0.02)
646     m.box('metal', (1.18, 1.18, 0.1), (0, 0, 0.38), bevel=0.03)
647     m.box('oscuro', (1.2, 1.2, 0.03), (0, 0, 0.36), bevel=0.0)
648     bx, by, bz, top = -0.22, -0.18, 0.48, 1.42
649     base = [(bx - 0.15, by - 0.15), (bx + 0.15, by - 0.15), (bx + 0.15, by + 0.15), (bx - 0.15, by + 0.15)]
650     m.box('oscuro', (0.38, 0.38, 0.06), (bx, by, bz), bevel=0.01)
651     for (x, y) in base:
652         m.beam('metal_claro', (x, y, bz + 0.06), (bx + (x - bx) * 0.2, by + (y - by) * 0.2, top), t=0.024)
653     for z in (0.75, 1.0, 1.22):
654         k = 1 - (z - bz) / (top - bz) * 0.8
655         pts = [(bx + (x - bx) * k, by + (y - by) * k) for (x, y) in base]
656         for i in range(4):
657             (xa, ya), (xb, yb) = pts[i], pts[(i + 1) % 4]
658             m.beam('metal_claro', (xa, ya, z), (xb, yb, z), t=0.014)
659     m.box('EQUIPO', (0.1, 0.1, 0.06), (bx, by, top), bevel=0.01)
660     m.box('EQUIPO', (0.42, 0.34, 0.26), (0.28, 0.3, 0.48), bevel=0.03)
661     for z in (0.56, 0.66):
662         m.box('luz', (0.38, 0.012, 0.03), (0.28, 0.128, z), bevel=0.0)
663     m.cyl('oscuro', 0.21, 0.025, (0.28, 0.3, 0.74), seg=20)
664     m.box('blanco', (0.025, 0.14, 0.006), (0.23, 0.3, 0.765), bevel=0.0)
665     m.box('blanco', (0.025, 0.14, 0.006), (0.33, 0.3, 0.765), bevel=0.0)
666     m.box('blanco', (0.1, 0.025, 0.006), (0.28, 0.3, 0.765), bevel=0.0)
667     for (px, py, ex, ey) in [(0.46, -0.4, 0.8, -0.7), (-0.46, 0.42, -0.78, 0.72)]:
668         m.cyl('amarillo', 0.05, 0.22, (px, py, 0.48), seg=12)
669         m.tube('amarillo', (px, py, 0.68), (ex, ey, 1.0), r=0.02, seg=8)
670         m.tube('oscuro', (ex, ey, 1.0), (ex, ey, 0.62), r=0.004, seg=6)
671         m.box('metal', (0.05, 0.05, 0.04), (ex, ey, 0.58), bevel=0.008)
672     for (x, y, mat) in [(0.25, -0.38, 'EQUIPO_OSC'), (0.25, -0.22, 'blanco_sucio'), (-0.4, 0.15, 'EQUIPO')]:
673         m.box(mat, (0.14, 0.12, 0.1), (x, y, 0.48), bevel=0.01)
674     m.tube('metal', (0.55, 0.0, 0.5), (0.85, 0.0, 0.72), r=0.015, seg=6)
675     m.cone('naranja', 0.04, 0.12, (0.86, 0.0, 0.72), seg=8)
676     return m
677
678
679 def helipad():
680     """Helipuerto redondo con la H, anillo amarillo, marcas y baranda."""
681     m = Model('helipad')
682     m.cyl('concreto', 0.88, 0.06, (0, 0, 0), seg=32)
683     m.cyl('EQUIPO', 0.82, 0.055, (0, 0, 0.06), seg=32)
684     m.cyl('oscuro', 0.75, 0.06, (0, 0, 0.06), seg=32)
685     m.cyl('amarillo', 0.44, 0.008, (0, 0, 0.12), seg=32)
686     m.cyl('oscuro', 0.37, 0.01, (0, 0, 0.12), seg=32)
687     m.box('blanco', (0.06, 0.34, 0.006), (-0.11, 0, 0.13), bevel=0.0)
688     m.box('blanco', (0.06, 0.34, 0.006), (0.11, 0, 0.13), bevel=0.0)
689     m.box('blanco', (0.2, 0.06, 0.006), (0, 0, 0.13), bevel=0.0)
690     for k in range(10):
691         a = k * PI / 5 + 0.2
692         m.box('blanco', (0.16, 0.03, 0.006), (math.cos(a) * 0.58, math.sin(a) * 0.58, 0.12), rot=(0, 0, a + H),
```

```
bevel=0.0)
    n = 16
    pts = [(math.cos(k * 2 * PI / n) * 0.85, math.sin(k * 2 * PI / n) * 0.85) for k in range(n)]
    for k, (x, y) in enumerate(pts):
        m.beam('EQUIPO_OSC', (x, y, 0.06), (x, y, 0.22), t=0.016)
        if k in (12, 13):
            continue
        (x2, y2) = pts[(k + 1) % n]
        m.beam('EQUIPO_OSC', (x, y, 0.22), (x2, y2, 0.22), t=0.014)
        m.beam('metal_claro', (x, y, 0.14), (x2, y2, 0.14), t=0.008)
    for k in range(8):
        a = k * PI / 4 + PI / 8
        m.sphere('luz', 0.022, (math.cos(a) * 0.79, math.sin(a) * 0.79, 0.125))
    return m


def turret(level):
    """Torreta tipo naval: base blindada, cañón largo y sensor arriba; miniguns en nivel 2 y 3."""
    m = Model(f'turret_{level}')
    m.hexprism('piedra', 0.5, 0.1)
    hb = 0.22 + (level - 1) * 0.05
    m.cyl('metal_claro', 0.42, hb, (0, 0, 0.1), seg=6, r2=0.36, rot=(0, 0, PI / 6), smooth=False)
    m.cyl('oscuro', 0.37, 0.02, (0, 0, 0.1 + hb), seg=6, rot=(0, 0, PI / 6), smooth=False)
    for sx in (-1, 1):
        m.box('metal', (0.12, 0.012, hb * 0.6), (sx * 0.09, -0.36, 0.14), rot=(-0.2, 0, 0), bevel=0.004)
        m.box('oscuro', (0.03, 0.014, 0.02), (sx * 0.09, -0.368, 0.14 + hb * 0.35), rot=(-0.2, 0, 0), bevel=0.0)
    top = 0.12 + hb
    m.pivot('turret', 0, 0, top)
    m.box('metal_claro', (0.42, 0.46, 0.2), (0, 0.05, top), bevel=0.04, part='turret')
    m.box('metal_claro', (0.36, 0.16, 0.16), (0, -0.2, top), rot=(0.5, 0, 0), bevel=0.03, part='turret')
    m.box('EQUIPO', (0.43, 0.2, 0.035), (0, 0.12, top + 0.12), bevel=0.006, part='turret')
    m.box('metal', (0.3, 0.012, 0.09), (0.0, 0.284, top + 0.06), bevel=0.003, part='turret')
    m.box('oscuro', (0.14, 0.1, 0.11), (0, -0.28, top + 0.06), bevel=0.02, part='turret')
    m.tube('metal', (0, -0.32, top + 0.11), (0, -0.82, top + 0.11), r=0.028, seg=12, part='turret')
    m.cyl('negro', 0.04, 0.07, (0, -0.84, top + 0.11), rot=(H, 0, 0), seg=12, base=False, part='turret')
    m.box('metal', (0.12, 0.08, 0.06), (0, 0.12, top + 0.2), bevel=0.01, part='turret')
    m.box('metal_claro', (0.13, 0.11, 0.1), (0, 0.12, top + 0.26), bevel=0.02, part='turret')
    m.box('vidrio', (0.08, 0.01, 0.06), (0, 0.064, top + 0.28), bevel=0.003, part='turret')
    m.tube('oscuro', (0.05, 0.16, top + 0.36), (0.05, 0.16, top + 0.48), r=0.005, seg=6, part='turret')
    sides = [1] if level == 2 else [1, -1] if level >= 3 else []
    if level >= 4:
        # Nivel 4: tercera minigun arriba y radar antiaéreo que gira.
        x, z = 0.0, top + 0.38
        m.box('metal', (0.12, 0.16, 0.06), (x, -0.04, top + 0.3), bevel=0.015, part='turret')
        m.cyl('oscuro', 0.042, 0.08, (x, 0.02, z), rot=(H, 0, 0), seg=12, base=False, part='turret')
        for k in range(6):
            a = k * PI / 3
            m.tube('negro', (x + math.cos(a) * 0.024, -0.06, z + math.sin(a) * 0.024),
                   (x + math.cos(a) * 0.024, -0.4, z + math.sin(a) * 0.024), r=0.008, seg=6, part='turret')
        m.cyl('metal', 0.04, 0.02, (x, -0.36, z), rot=(H, 0, 0), seg=12, base=False, part='turret')
        m.pivot('rotor', 0.14, 0.22, top + 0.44)
        m.box('oscuro', (0.2, 0.025, 0.1), (0.14, 0.22, top + 0.44), bevel=0.006, part='rotor')
        m.box('EQUIPO', (0.18, 0.01, 0.08), (0.14, 0.208, top + 0.445), bevel=0.0, part='rotor')
    for sx in sides:
        x = sx * 0.3
        m.box('metal', (0.1, 0.18, 0.1), (x, -0.02, top + 0.04), bevel=0.02, part='turret')
        m.cyl('oscuro', 0.042, 0.08, (x, 0.06, top + 0.1), rot=(H, 0, 0), seg=12, base=False, part='turret')
        for k in range(6):
            a = k * PI / 3
            m.tube('negro', (x + math.cos(a) * 0.024, -0.04, top + 0.1 + math.sin(a) * 0.024),
                   (x + math.cos(a) * 0.024, -0.38, top + 0.1 + math.sin(a) * 0.024), r=0.008, seg=6, part='turret')
        m.cyl('metal', 0.04, 0.02, (x, -0.33, top + 0.1), rot=(H, 0, 0), seg=12, base=False, part='turret')
        m.cyl('metal', 0.04, 0.02, (x, -0.16, top + 0.1), rot=(H, 0, 0), seg=12, base=False, part='turret')
        m.box('oliva_osc', (0.07, 0.1, 0.07), (x + sx * 0.08, 0.06, top + 0.02), bevel=0.01, part='turret')
    return m


def aadef():
    """AIR DEFENSE: caja de misiles con celdas marcadas y un radar aparte que gira."""
    m = Model('aadef')
    m.box('concreto', (0.92, 0.92, 0.08), (0, 0, 0), bevel=0.02)
    for (x, y) in [(-0.4, -0.4), (0.4, -0.4), (-0.4, 0.4), (0.4, 0.4)]:
        m.box('oscuro', (0.08, 0.08, 0.1), (x, y, -0.02), bevel=0.01)
    m.box('amarillo', (0.3, 0.02, 0.004), (-0.1, -0.44, 0.08), bevel=0.0)
    m.box('oliva_osc', (0.26, 0.24, 0.08), (0.27, 0.27, 0.08), bevel=0.02)
    for k in range(3):
        m.box('amarillo', (0.03, 0.12, 0.012), (0.21 + k * 0.05, 0.12, 0.08), rot=(0.5, 0, 0), bevel=0.0)
    m.cyl('oscuro', 0.03, 0.12, (0.27, 0.27, 0.16), seg=8)
    m.pivot('rotor', 0.27, 0.27, 0.28)
    m.box('EQUIPO', (0.26, 0.03, 0.17), (0.27, 0.27, 0.26), bevel=0.01, part='rotor')
    m.box('blanco_sucio', (0.23, 0.012, 0.14), (0.27, 0.252, 0.275), bevel=0.0, part='rotor')
    m.cyl('oscuro', 0.17, 0.1, (-0.1, -0.08, 0.08), seg=16)
    m.pivot('turret', -0.1, -0.08, 0.18)
    m.box('oliva_osc', (0.3, 0.28, 0.06), (-0.1, -0.08, 0.18), bevel=0.02, part='turret')
    for sx in (-1, 1):
        m.beam('oscuro', (-0.1 + sx * 0.13, -0.02, 0.22), (-0.1 + sx * 0.13, -0.04, 0.36), t=0.035, part='turret')
    th = -0.55
    m.box('EQUIPO', (0.4, 0.42, 0.26), (-0.1, -0.06, 0.24), rot=(th, 0, 0), bevel=0.03, part='turret')
```

```
    launcher_face(m, -0.1, -0.06, 0.37, 0.21, th, 3, 2, 0.115, 0.1, 'turret')
    return m
```

faro/blender/models_ref2.py · 436 líneas

```
"""Modelos de la v0.6 (diseño propio, estilo de las referencias del 7 de noviembre).

Cuartel, Fábrica, Puerto, Casa, cinco energías, Submarino, Apache Guardián, Camión antiaéreo,
Torreta chica, Puesto avanzado y Enfermería por niveles. Blender: Z arriba, el frente mira a -Y.
"""
import math

from lib import Model
from models_ref import wheels

PI = math.pi
H = PI / 2


def sandbags(m, cx, cy, r, a0, a1, n, z=0.0, part='main'):
    """Fila de sacos de arena en arco (de a0 a a1 radianes)."""
    for i in range(n):
        a = a0 + (a1 - a0) * (i + 0.5) / n
        m.box('arena', (0.12, 0.07, 0.06), (cx + math.cos(a) * r, cy + math.sin(a) * r, z), rot=(0, 0, a + H),
bevel=0.02, part=part)


def flag(m, x, y, z, h=0.6):
    m.tube('metal_claro', (x, y, z), (x, y, z + h), r=0.01, seg=6)
    m.box('EQUIPO', (0.18, 0.012, 0.11), (x + 0.09, y, z + h - 0.13), bevel=0.0)


def barracks():
    """Cuartel: nave con techo de lona, puerta grande, sacos de arena y bandera."""
    m = Model('barracks')
    m.box('concreto', (1.2, 0.9, 0.06), (0, 0.04, 0), bevel=0.02)
    m.box('oliva', (0.9, 0.56, 0.24), (0, 0.08, 0.06), bevel=0.02)
    m.cyl('oliva_osc', 0.28, 0.92, (0, 0.08, 0.3), rot=(0, H, 0), seg=16, base=False)
    for x in (-0.3, 0.0, 0.3):
        m.cyl('oscuro', 0.285, 0.025, (x, 0.08, 0.3), rot=(0, H, 0), seg=16, base=False)
    m.box('EQUIPO', (0.94, 0.08, 0.03), (0, 0.08, 0.57), bevel=0.0)
    m.box('oscuro', (0.3, 0.02, 0.2), (0, -0.205, 0.06), bevel=0.0)
    for k in range(4):
        m.box('negro', (0.3, 0.022, 0.008), (0, -0.208, 0.09 + k * 0.045), bevel=0.0)
    m.box('luz', (0.08, 0.02, 0.03), (0, -0.21, 0.29), bevel=0.0)
    for sx in (-1, 1):
        m.box('luz', (0.1, 0.012, 0.06), (sx * 0.3, -0.206, 0.17), bevel=0.0)
    sandbags(m, -0.38, -0.32, 0.16, PI * 0.6, PI * 1.4, 4, 0.06)
    sandbags(m, 0.38, -0.32, 0.16, -PI * 0.4, PI * 0.4, 4, 0.06)
    flag(m, 0.5, 0.42, 0.06, 0.75)
    for (x, y) in [(-0.5, 0.36), (-0.38, 0.4)]:
        m.box('madera', (0.1, 0.1, 0.09), (x, y, 0.06), bevel=0.01)
    return m


def factory():
    """Fábrica: nave con techo en sierra, chimeneas, portón y cinta con un vehículo saliendo."""
    m = Model('factory')
    m.box('concreto', (1.3, 1.0, 0.06), (0, 0.04, 0), bevel=0.02)
    m.box('concreto', (1.0, 0.66, 0.36), (0, 0.12, 0.06), bevel=0.02)
    m.box('EQUIPO', (1.02, 0.68, 0.04), (0, 0.12, 0.3), bevel=0.0)
    for k in range(4):
        x = -0.375 + k * 0.25
        m.wedge('metal', (0.25, 0.66, 0.16), (x, 0.12, 0.42), rot=(0, 0, H))
        m.box('vidrio', (0.012, 0.6, 0.11), (x - 0.06, 0.12, 0.45), rot=(0, -0.55, 0), bevel=0.0)
    for (x, y) in [(0.34, 0.34), (0.18, 0.4)]:
        m.cyl('ladrillo', 0.06, 0.62, (x, y, 0.42), seg=12)
        m.cyl('EQUIPO', 0.065, 0.04, (x, y, 0.88), seg=12)
        m.cyl('negro', 0.05, 0.02, (x, y, 1.04), seg=12)
    m.box('oscuro', (0.42, 0.02, 0.26), (-0.1, -0.215, 0.06), bevel=0.0)
    for k in range(6):
        m.box('amarillo' if k % 2 == 0 else 'negro', (0.07, 0.024, 0.03), (-0.275 + k * 0.07, -0.22, 0.33), bevel=0.0)
    m.box('negro', (0.26, 0.42, 0.04), (-0.1, -0.42, 0.06), bevel=0.01)
    for k in range(5):
        m.cyl('metal', 0.025, 0.27, (-0.1, -0.25 - k * 0.085, 0.08), rot=(0, H, 0), seg=8, base=False)
    m.box('EQUIPO', (0.16, 0.22, 0.08), (-0.1, -0.46, 0.13), bevel=0.02)
    m.box('vidrio', (0.14, 0.02, 0.05), (-0.1, -0.572, 0.17), bevel=0.0)
    for sx in (-1, 1):
        for y in (-0.39, -0.53):
            m.cyl('goma', 0.035, 0.03, (-0.1 + sx * 0.085, y, 0.135), rot=(0, H, 0), seg=10, base=False)
    m.box('luz', (0.3, 0.012, 0.05), (0.3, -0.216, 0.22), bevel=0.0)
    for (x, y, mat) in [(0.48, -0.32, 'madera'), (0.48, -0.2, 'EQUIPO_OSC')]:
        m.box(mat, (0.1, 0.1, 0.09), (x, y, 0.06), bevel=0.01)
    return m


def harbor():
    """Puerto: muelle de madera sobre pilotes, bodega, grúa de pórtico y contenedores."""
    m = Model('harbor')
```

```
    for x in (-0.55, -0.18, 0.18, 0.55):
        for y in (-0.45, 0.0, 0.45):
            m.cyl('madera', 0.045, 0.3, (x, y, -0.28), seg=8)
    m.box('madera', (1.3, 1.04, 0.08), (0, 0, 0.0), bevel=0.01)
    for k in range(9):
        m.box('madera_clara', (1.3, 0.012, 0.004), (0, -0.46 + k * 0.115, 0.08), bevel=0.0)
    m.box('EQUIPO_OSC', (0.56, 0.44, 0.32), (-0.3, 0.24, 0.08), bevel=0.02)
    m.wedge('EQUIPO', (0.62, 0.5, 0.14), (-0.3, 0.24, 0.4))
    m.box('oscuro', (0.22, 0.012, 0.2), (-0.3, 0.015, 0.08), bevel=0.0)
    m.box('luz', (0.1, 0.012, 0.05), (-0.48, 0.015, 0.26), bevel=0.0)
    for sx in (-1, 1):
        m.beam('amarillo', (0.4 + sx * 0.2, -0.05, 0.08), (0.4 + sx * 0.2, -0.05, 0.85), t=0.05)
        m.beam('amarillo', (0.4 + sx * 0.2, -0.05, 0.4), (0.4 - sx * 0.2, -0.05, 0.75), t=0.025)
    m.beam('amarillo', (0.16, -0.05, 0.85), (0.64, -0.05, 0.85), t=0.06)
    m.beam('amarillo', (0.4, -0.05, 0.85), (0.4, -0.62, 0.85), t=0.05)
    m.box('oscuro', (0.08, 0.08, 0.06), (0.4, -0.5, 0.82), bevel=0.01)
    m.tube('oscuro', (0.4, -0.5, 0.82), (0.4, -0.5, 0.42), r=0.004, seg=6)
    m.box('EQUIPO', (0.2, 0.12, 0.1), (0.4, -0.5, 0.32), bevel=0.01)
    for (x, y, z, mat) in [(0.36, 0.28, 0.08, 'EQUIPO'), (0.36, 0.28, 0.2, 'rojo'), (0.56, 0.28, 0.08, 'blanco_sucio')]:
        m.box(mat, (0.18, 0.12, 0.12), (x, y, z), bevel=0.01)
    for x in (-0.58, -0.3, 0.0, 0.3, 0.58):
        m.cyl('oscuro', 0.025, 0.05, (x, -0.49, 0.08), seg=10)
    m.tube('oscuro', (-0.6, -0.4, 0.08), (-0.6, -0.4, 0.42), r=0.01, seg=6)
    m.sphere('luz', 0.03, (-0.6, -0.4, 0.44))
    return m


def house():
    """Casa: techo a dos aguas, ventanas encendidas, chimenea y cerca."""
    m = Model('house')
    m.box('cultivo', (0.95, 0.9, 0.03), (0, 0.0, 0), bevel=0.01)
    m.box('blanco', (0.56, 0.46, 0.3), (0, 0.06, 0.03), bevel=0.02)
    m.wedge('EQUIPO', (0.66, 0.58, 0.26), (0, 0.06, 0.33))
    m.box('ladrillo', (0.08, 0.08, 0.2), (0.16, 0.16, 0.42), bevel=0.01)
    m.box('madera', (0.12, 0.012, 0.2), (0, -0.175, 0.03), bevel=0.0)
    for sx in (-1, 1):
        m.box('luz', (0.1, 0.012, 0.08), (sx * 0.17, -0.175, 0.15), bevel=0.0)
        m.box('blanco', (0.13, 0.016, 0.012), (sx * 0.17, -0.178, 0.105), bevel=0.0)
        m.box('luz', (0.012, 0.1, 0.08), (sx * 0.282, 0.06, 0.15), bevel=0.0)
    m.box('piedra_clara', (0.14, 0.2, 0.012), (0, -0.28, 0.03), bevel=0.0)
    for k in range(7):
        x = -0.42 + k * 0.14
        if abs(x) < 0.09:
            continue
        m.box('madera', (0.02, 0.02, 0.1), (x, -0.4, 0.03), bevel=0.0)
    m.beam('madera_clara', (-0.42, -0.4, 0.09), (-0.09, -0.4, 0.09), t=0.014)
    m.beam('madera_clara', (0.09, -0.4, 0.09), (0.42, -0.4, 0.09), t=0.014)
    m.sphere('oliva', 0.08, (-0.35, 0.3, 0.08), scale=(1, 1, 0.8))
    m.sphere('cultivo', 0.07, (0.36, 0.32, 0.07), scale=(1, 1, 0.8))
    return m


def solar():
    """Solar: dos filas de paneles inclinados sobre patas y un inversor."""
    m = Model('solar')
    m.box('concreto', (1.1, 0.95, 0.04), (0, 0, 0), bevel=0.01)
    for row in range(2):
        y = -0.22 + row * 0.42
        for col in range(3):
            x = -0.32 + col * 0.32
            m.beam('metal', (x, y + 0.08, 0.04), (x, y + 0.08, 0.26), t=0.025)
            m.beam('metal', (x, y - 0.08, 0.04), (x, y - 0.08, 0.14), t=0.025)
            m.box('metal_claro', (0.3, 0.28, 0.03), (x, y, 0.17), rot=(0.5, 0, 0), bevel=0.005)
            m.box('panel', (0.27, 0.25, 0.012), (x, y - 0.006, 0.19), rot=(0.5, 0, 0), bevel=0.0)
            for k in range(2):
                m.box('metal_claro', (0.27, 0.006, 0.004), (x, y - 0.04 + k * 0.08, 0.2 + k * 0.04), rot=(0.5, 0, 0),
bevel=0.0)
    m.box('EQUIPO', (0.14, 0.1, 0.16), (0.46, 0.38, 0.04), bevel=0.01)
    m.box('luz', (0.03, 0.012, 0.03), (0.46, 0.328, 0.15), bevel=0.0)
    return m


def wind():
    """Eólica: torre alta y fina, góndola con franja del equipo y tres aspas que giran."""
    m = Model('wind')
    m.cyl('concreto', 0.2, 0.06, (0, 0, 0), seg=12)
    m.cyl('blanco', 0.065, 1.48, (0, 0, 0.06), seg=12, r2=0.035)
    m.cyl('EQUIPO', 0.06, 0.05, (0, 0, 0.3), seg=12)
    m.box('blanco', (0.11, 0.28, 0.11), (0, 0.02, 1.51), bevel=0.04)
    m.box('EQUIPO', (0.115, 0.1, 0.03), (0, 0.08, 1.58), bevel=0.0)
    m.pivot('blades', 0, -0.13, 1.565)
    m.sphere('blanco', 0.055, (0, -0.15, 1.565), scale=(1, 1.4, 1), part='blades')
    for k in range(3):
        a = k * 2 * PI / 3
        m.box('blanco', (0.06, 0.018, 0.62), (math.sin(a) * 0.33, -0.15, 1.565 + math.cos(a) * 0.33),
              rot=(0, a, 0), bevel=0.006, part='blades', base=False)
        m.box('rojo', (0.062, 0.02, 0.06), (math.sin(a) * 0.6, -0.15, 1.565 + math.cos(a) * 0.6),
              rot=(0, a, 0), bevel=0.0, part='blades', base=False)
```

```
    return m


def hydro():
    """Hidráulica: casa de piedra con canal de agua y una rueda de madera que gira."""
    m = Model('hydro')
    m.box('piedra', (0.9, 0.8, 0.12), (-0.05, 0.05, 0), bevel=0.02)
    m.box('agua', (0.16, 0.8, 0.02), (0.38, 0.05, 0.1), bevel=0.0)
    for sx in (-1, 1):
        m.box('piedra_clara', (0.04, 0.8, 0.1), (0.38 + sx * 0.1, 0.05, 0.1), bevel=0.01)
    m.box('blanco', (0.46, 0.4, 0.3), (-0.12, 0.12, 0.12), bevel=0.02)
    m.wedge('EQUIPO', (0.54, 0.48, 0.18), (-0.12, 0.12, 0.42))
    m.box('madera', (0.12, 0.012, 0.18), (-0.12, -0.085, 0.12), bevel=0.0)
    m.box('luz', (0.08, 0.012, 0.06), (-0.27, -0.085, 0.26), bevel=0.0)
    m.cyl('metal', 0.03, 0.26, (0.15, -0.25, 0.22), rot=(0, H, 0), seg=10, base=False)
    m.pivot('blades', 0.38, -0.25, 0.22)
    m.cyl('metal', 0.04, 0.16, (0.38, -0.25, 0.22), rot=(0, H, 0), seg=10, base=False, part='blades')
    for sx in (-1, 1):
        m.cyl('madera', 0.2, 0.02, (0.38 + sx * 0.06, -0.25, 0.22), rot=(0, H, 0), seg=18, base=False, part='blades')
        m.cyl('madera_clara', 0.17, 0.024, (0.38 + sx * 0.06, -0.25, 0.22), rot=(0, H, 0), seg=18, base=False,
part='blades')
    for k in range(8):
        a = k * PI / 4
        m.box('madera', (0.14, 0.012, 0.07), (0.38, -0.25 + math.cos(a) * 0.19, 0.22 + math.sin(a) * 0.19),
              rot=(a, 0, 0), bevel=0.0, part='blades', base=False)
    return m


def tidal():
    """Mareomotriz: plataforma sobre pilotes con caseta y turbina bajo el agua."""
    m = Model('tidal')
    for (x, y) in [(-0.3, -0.25), (0.3, -0.25), (-0.3, 0.3), (0.3, 0.3)]:
        m.cyl('amarillo', 0.045, 0.5, (x, y, -0.3), seg=10)
    m.box('metal', (0.8, 0.7, 0.06), (0, 0.03, 0.18), bevel=0.01)
    for k in range(4):
        m.beam('metal_claro', (-0.4 + k * 0.266, -0.32, 0.24), (-0.4 + k * 0.266, -0.32, 0.36), t=0.012)
    m.beam('metal_claro', (-0.4, -0.32, 0.36), (0.4, -0.32, 0.36), t=0.012)
    m.box('EQUIPO', (0.3, 0.26, 0.22), (0.15, 0.15, 0.24), bevel=0.02)
    m.box('luz', (0.24, 0.012, 0.05), (0.15, 0.018, 0.36), bevel=0.0)
    m.box('blanco_sucio', (0.32, 0.28, 0.03), (0.15, 0.15, 0.46), bevel=0.005)
    m.beam('metal', (0, -0.26, 0.18), (0, -0.26, 0.0), t=0.05)
    m.cyl('metal_claro', 0.07, 0.24, (0, -0.14, 0.05), rot=(H, 0, 0), seg=12, base=False)
    m.pivot('blades', 0, -0.26, 0.05)
    m.cone('metal_claro', 0.07, 0.12, (0, -0.31, 0.05), rot=(H, 0, 0), seg=12, part='blades', smooth=True)
    for k in range(3):
        a = k * 2 * PI / 3
        m.box('blanco', (0.05, 0.016, 0.28), (math.sin(a) * 0.15, -0.27, 0.05 + math.cos(a) * 0.15),
              rot=(0, a, 0), bevel=0.005, part='blades', base=False)
    return m


def wave():
    """Undimotriz: base flotante con boyas amarillas en brazos que suben y bajan con las olas."""
    m = Model('wave')
    m.cyl('gris', 0.24, 0.16, (0, 0, -0.08), seg=16)
    m.cyl('EQUIPO', 0.25, 0.04, (0, 0, 0.06), seg=16)
    m.box('blanco_sucio', (0.2, 0.2, 0.16), (0, 0.0, 0.08), bevel=0.02)
    m.box('luz', (0.16, 0.012, 0.04), (0, -0.102, 0.18), bevel=0.0)
    m.tube('oscuro', (0, 0, 0.24), (0, 0, 0.5), r=0.008, seg=6)
    m.sphere('luz', 0.025, (0, 0, 0.51))
    m.pivot('rocker', 0, 0, 0.04)
    for (dx, dy) in [(0, -1), (0.87, 0.5), (-0.87, 0.5)]:
        x, y = dx * 0.55, dy * 0.55
        m.beam('metal', (dx * 0.2, dy * 0.2, 0.06), (x, y, 0.04), t=0.035, part='rocker')
        m.cyl('amarillo', 0.12, 0.12, (x, y, -0.04), seg=14, part='rocker')
        m.cyl('naranja', 0.122, 0.025, (x, y, 0.06), seg=14, part='rocker')
    return m


def submarine():
    """Submarino: casco largo con torre, timones y hélice."""
    m = Model('sub')
    m.cyl('oscuro', 0.15, 0.9, (0, 0.02, 0.0), rot=(H, 0, 0), seg=18, base=False)
    m.sphere('oscuro', 0.15, (0, -0.43, 0.0), scale=(1, 1.4, 1))
    m.cone('oscuro', 0.15, 0.4, (0, 0.67, 0.0), rot=(-H, 0, 0), seg=18, smooth=True)
    m.box('EQUIPO', (0.012, 0.6, 0.03), (0.15, 0.0, 0.03), bevel=0.0)
    m.box('EQUIPO', (0.012, 0.6, 0.03), (-0.15, 0.0, 0.03), bevel=0.0)
    m.box('oscuro', (0.1, 0.28, 0.2), (0, -0.1, 0.1), bevel=0.04)
    m.box('EQUIPO', (0.104, 0.1, 0.04), (0, -0.1, 0.24), bevel=0.0)
    m.box('oscuro', (0.34, 0.07, 0.014), (0, -0.12, 0.2), bevel=0.0)
    m.tube('metal', (0.02, -0.15, 0.3), (0.02, -0.15, 0.46), r=0.008, seg=6)
    m.tube('metal', (-0.02, -0.07, 0.3), (-0.02, -0.07, 0.4), r=0.008, seg=6)
    m.box('luz', (0.06, 0.012, 0.02), (0, -0.242, 0.24), bevel=0.0)
    m.box('oscuro', (0.42, 0.07, 0.014), (0, 0.64, 0.0), bevel=0.0)
    m.box('oscuro', (0.014, 0.07, 0.3), (0, 0.64, -0.15), bevel=0.0)
    m.cyl('metal', 0.035, 0.03, (0, 0.88, 0.0), rot=(H, 0, 0), seg=10, base=False)
    for k in range(4):
        a = k * H + PI / 4
```

```
        m.box('metal', (0.12, 0.012, 0.03), (math.cos(a) * 0.05, 0.9, math.sin(a) * 0.05), rot=(0, -a, 0), bevel=0.0,
base=False)
    return m


def apache():
    """Apache Guardián: cabina en tándem, alas con cohetes y misiles, cañón bajo la nariz."""
    m = Model('apache')
    m.box('EQUIPO', (0.2, 0.62, 0.2), (0, -0.02, 0.12), bevel=0.05)
    m.box('EQUIPO_OSC', (0.16, 0.26, 0.14), (0, -0.37, 0.13), rot=(0.12, 0, 0), bevel=0.04)
    m.box('vidrio', (0.14, 0.15, 0.09), (0, -0.34, 0.26), rot=(0.25, 0, 0), bevel=0.02)
    m.box('vidrio', (0.14, 0.15, 0.11), (0, -0.15, 0.3), rot=(0.2, 0, 0), bevel=0.02)
    m.box('oscuro', (0.06, 0.08, 0.06), (0, -0.5, 0.12), bevel=0.01)
    m.tube('EQUIPO_OSC', (0, 0.28, 0.22), (0, 0.86, 0.26), r=0.045, r2=0.028, seg=10)
    m.box('EQUIPO_OSC', (0.02, 0.16, 0.22), (0, 0.85, 0.32), rot=(0.25, 0, 0), bevel=0.01)
    m.box('EQUIPO_OSC', (0.26, 0.07, 0.015), (0, 0.8, 0.24), bevel=0.0)
    m.cyl('oscuro', 0.07, 0.01, (0.03, 0.88, 0.32), rot=(0, H, 0), seg=14, base=False)
    for sx in (-1, 1):
        m.box('oscuro', (0.32, 0.1, 0.025), (sx * 0.22, 0.0, 0.2), bevel=0.008)
        m.box('oscuro', (0.06, 0.14, 0.1), (sx * 0.13, 0.08, 0.16), bevel=0.02)
        m.cyl('oliva_osc', 0.045, 0.22, (sx * 0.3, -0.02, 0.15), rot=(H, 0, 0), seg=10, base=False)
        for k in range(4):
            a = k * H + PI / 4
            m.cyl('blanco', 0.012, 0.012, (sx * 0.3 + math.cos(a) * 0.022, -0.135, 0.15 + math.sin(a) * 0.022), rot=(H,
0, 0), seg=6, base=False)
        for k in range(2):
            m.tube('blanco', (sx * (0.17 + k * 0.05), 0.06, 0.15), (sx * (0.17 + k * 0.05), -0.12, 0.15), r=0.018,
seg=8)
            m.cone('rojo', 0.018, 0.04, (sx * (0.17 + k * 0.05), -0.14, 0.15), rot=(H, 0, 0), seg=8)
        m.beam('oscuro', (sx * 0.1, -0.2, 0.05), (sx * 0.1, -0.2, -0.04), t=0.02)
        m.beam('oscuro', (sx * 0.1, 0.25, 0.05), (sx * 0.1, 0.25, -0.04), t=0.02)
        m.cyl('goma', 0.035, 0.03, (sx * 0.1, -0.2, -0.03), rot=(0, H, 0), seg=10, base=False)
    m.cyl('oscuro', 0.03, 0.06, (0, -0.42, 0.02), seg=10)
    m.tube('metal', (0, -0.44, 0.03), (0, -0.62, 0.03), r=0.012, seg=8)
    m.box('luz', (0.03, 0.02, 0.02), (0, -0.505, 0.15), bevel=0.0)
    # v0.9: motores a los lados, bola de sensores en la nariz, más misiles y radar sobre el rotor.
    for sx in (-1, 1):
        m.cyl('EQUIPO_OSC', 0.045, 0.2, (sx * 0.12, 0.12, 0.27), rot=(H, 0, 0), seg=12, base=False)
        m.cyl('negro', 0.035, 0.02, (sx * 0.12, 0.225, 0.27), rot=(H, 0, 0), seg=12, base=False)
        for k in range(2):
            m.tube('metal_claro', (sx * (0.15 + k * 0.05), 0.05, 0.12), (sx * (0.15 + k * 0.05), -0.1, 0.12), r=0.014,
seg=8)
            m.cone('rojo', 0.014, 0.03, (sx * (0.15 + k * 0.05), -0.115, 0.12), rot=(H, 0, 0), seg=8)
    m.sphere('oscuro', 0.045, (0, -0.55, 0.1), scale=(1, 1, 1))
    m.cyl('vidrio', 0.02, 0.01, (0, -0.592, 0.1), rot=(H, 0, 0), seg=10, base=False)
    m.cyl('oscuro', 0.04, 0.1, (0, 0.0, 0.32), seg=10)
    m.pivot('rotor', 0, 0.0, 0.44)
    m.cyl('oscuro', 0.045, 0.04, (0, 0, 0.43), seg=12, part='rotor')
    m.cyl('oscuro', 0.018, 0.06, (0, 0, 0.45), seg=8, part='rotor')
    m.sphere('EQUIPO_OSC', 0.075, (0, 0, 0.53), scale=(1, 1, 0.55), part='rotor')
    m.box('oscuro', (1.36, 0.07, 0.012), (0, 0, 0.44), bevel=0.004, part='rotor')
    m.box('oscuro', (0.07, 1.36, 0.012), (0, 0, 0.44), bevel=0.004, part='rotor')
    return m


def aa_truck():
    """Camión antiaéreo: seis ruedas, cabina, cañones dobles y radar en la torreta."""
    m = Model('aa')
    wheels(m, [(-0.22, -0.3), (0.22, -0.3), (-0.22, 0.1), (0.22, 0.1), (-0.22, 0.3), (0.22, 0.3)], r=0.08)
    m.box('oliva_osc', (0.44, 0.86, 0.08), (0, 0.02, 0.1), bevel=0.02)
    m.box('EQUIPO', (0.42, 0.24, 0.2), (0, -0.3, 0.17), bevel=0.04)
    m.box('vidrio', (0.36, 0.02, 0.09), (0, -0.425, 0.27), rot=(-0.2, 0, 0), bevel=0.005)
    m.box('oscuro', (0.32, 0.02, 0.06), (0, -0.425, 0.19), bevel=0.0)
    for sx in (-1, 1):
        m.box('luz', (0.04, 0.02, 0.03), (sx * 0.16, -0.425, 0.24), bevel=0.0)
    m.box('oliva', (0.44, 0.5, 0.08), (0, 0.15, 0.18), bevel=0.02)
    m.pivot('turret', 0, 0.15, 0.26)
    m.cyl('oscuro', 0.15, 0.05, (0, 0.15, 0.26), seg=16, part='turret')
    m.box('EQUIPO', (0.26, 0.24, 0.12), (0, 0.16, 0.31), bevel=0.04, part='turret')
    for sx in (-1, 1):
        m.box('oscuro', (0.05, 0.14, 0.07), (sx * 0.08, 0.06, 0.38), rot=(-0.5, 0, 0), bevel=0.01, part='turret')
        m.tube('negro', (sx * 0.08, 0.02, 0.41), (sx * 0.08, -0.32, 0.6), r=0.014, seg=8, part='turret')
        m.cyl('metal', 0.02, 0.03, (sx * 0.08, -0.3, 0.59), rot=(H - 0.5, 0, 0), seg=8, base=False, part='turret')
    m.tube('oscuro', (0, 0.26, 0.43), (0, 0.26, 0.52), r=0.012, seg=6, part='turret')
    m.box('metal_claro', (0.18, 0.025, 0.11), (0, 0.27, 0.56), rot=(0.25, 0, 0), bevel=0.0, part='turret')
    return m


def miniturret():
    """Torreta chica: nido de ametralladora con sacos de arena."""
    m = Model('miniturret')
    m.cyl('tierra', 0.32, 0.04, (0, 0, 0), seg=16)
    sandbags(m, 0, 0, 0.27, -PI * 0.15, PI * 1.15, 9, 0.04)
    sandbags(m, 0, 0, 0.27, -PI * 0.05, PI * 1.05, 8, 0.1)
    m.pivot('turret', 0, 0, 0.12)
    m.cyl('oscuro', 0.07, 0.06, (0, 0, 0.1), seg=10, part='turret')
    for k in range(3):
        a = k * 2 * PI / 3
```

```
        m.beam('oscuro', (0, 0, 0.16), (math.cos(a) * 0.08, math.sin(a) * 0.08, 0.04), t=0.012, part='turret')
    m.box('negro', (0.05, 0.2, 0.06), (0, -0.06, 0.18), bevel=0.01, part='turret')
    m.tube('metal', (0, -0.16, 0.21), (0, -0.34, 0.21), r=0.01, seg=6, part='turret')
    m.box('EQUIPO', (0.18, 0.015, 0.1), (0, -0.12, 0.17), bevel=0.0, part='turret')
    m.box('oliva_osc', (0.05, 0.06, 0.05), (0.05, 0.0, 0.16), bevel=0.01, part='turret')
    return m


def outpost():
    """Puesto avanzado: torre de vigía de madera con techo del equipo, bandera y tienda."""
    m = Model('outpost')
    m.cyl('tierra', 0.7, 0.03, (0, 0, 0), seg=18)
    c = [(-0.12, -0.12), (0.12, -0.12), (0.12, 0.12), (-0.12, 0.12)]
    for (x, y) in c:
        m.beam('madera', (x * 1.5, y * 1.5, 0.03), (x, y, 0.62), t=0.035)
    for i in range(4):
        (xa, ya), (xb, yb) = c[i], c[(i + 1) % 4]
        m.beam('madera_clara', (xa * 1.32, ya * 1.32, 0.3), (xb * 1.32, yb * 1.32, 0.3), t=0.02)
    m.box('madera', (0.34, 0.34, 0.04), (0, 0, 0.62), bevel=0.01)
    for i in range(4):
        (xa, ya), (xb, yb) = c[i], c[(i + 1) % 4]
        m.beam('madera_clara', (xa * 1.35, ya * 1.35, 0.78), (xb * 1.35, yb * 1.35, 0.78), t=0.018)
    for (x, y) in c:
        m.beam('madera', (x * 1.35, y * 1.35, 0.66), (x * 1.35, y * 1.35, 0.92), t=0.02)
    m.cone('EQUIPO', 0.3, 0.2, (0, 0, 0.92), seg=4, rot=(0, 0, PI / 4))
    m.sphere('luz', 0.025, (0.0, -0.16, 0.85))
    for k in range(6):
        m.beam('madera_clara', (-0.05, -0.2 - k * 0.0, 0.1 + k * 0.1), (0.05, -0.2, 0.1 + k * 0.1), t=0.012)
    flag(m, 0.18, 0.18, 0.92, 0.35)
    m.wedge('oliva', (0.36, 0.3, 0.2), (0.38, 0.3, 0.03), rot=(0, 0, H))
    m.box('oscuro', (0.1, 0.012, 0.14), (0.38, 0.148, 0.03), bevel=0.0)
    for (x, y) in [(-0.36, 0.2), (-0.42, 0.06)]:
        m.box('madera', (0.1, 0.1, 0.09), (x, y, 0.03), bevel=0.01)
    return m


def hospital(level):
    """Enfermería: tienda (1), clínica (2) y hospital grande con helipuerto (3)."""
    m = Model(f'hospital_{level}')
    m.box('concreto', (1.0, 0.86, 0.04), (0, 0.02, 0), bevel=0.01)

    def cross(x, y, z, s, rot=0.0):
        if rot:
            m.box('rojo', (0.016, s, s * 0.34), (x, y, z), bevel=0.0)
            m.box('rojo', (0.016, s * 0.34, s), (x, y, z - s * 0.33), bevel=0.0)
        else:
            m.box('rojo', (s, 0.016, s * 0.34), (x, y, z), bevel=0.0)
            m.box('rojo', (s * 0.34, 0.016, s), (x, y, z - s * 0.33), bevel=0.0)
    if level == 1:
        m.wedge('blanco', (0.7, 0.5, 0.36), (0, 0.06, 0.04), rot=(0, 0, 0))
        m.box('oscuro', (0.14, 0.012, 0.18), (-0.36, 0.06, 0.04), rot=(0, 0, H), bevel=0.0)
        # Cruz roja apoyada en la lona inclinada (la cara delantera sube de y=-0,25 a la cumbrera).
        t = math.atan2(0.25, 0.36)
        cy, cz = -0.25 * (1 - 0.2 / 0.36) - 0.012 * math.cos(t), 0.04 + 0.2 + 0.012 * math.sin(t)
        m.box('rojo', (0.22, 0.012, 0.07), (0, cy, cz), rot=(-t, 0, 0), bevel=0.0, base=False)
        m.box('rojo', (0.07, 0.012, 0.22), (0, cy, cz), rot=(-t, 0, 0), bevel=0.0, base=False)
        for (x, y) in [(0.3, -0.32), (0.18, -0.32)]:
            m.box('blanco_sucio', (0.1, 0.18, 0.05), (x, y, 0.04), bevel=0.01)
        flag(m, -0.4, -0.3, 0.04, 0.5)
        return m
    if level == 2:
        m.box('blanco', (0.7, 0.5, 0.32), (0, 0.08, 0.04), bevel=0.02)
        m.box('EQUIPO', (0.72, 0.52, 0.04), (0, 0.08, 0.36), bevel=0.0)
        m.box('blanco_sucio', (0.3, 0.16, 0.04), (0, -0.24, 0.24), bevel=0.0)
        m.beam('metal_claro', (-0.13, -0.3, 0.04), (-0.13, -0.3, 0.24), t=0.02)
        m.beam('metal_claro', (0.13, -0.3, 0.04), (0.13, -0.3, 0.24), t=0.02)
        m.box('vidrio', (0.16, 0.012, 0.18), (0, -0.172, 0.04), bevel=0.0)
        for sx in (-1, 1):
            m.box('luz', (0.12, 0.012, 0.1), (sx * 0.24, -0.172, 0.18), bevel=0.0)
        m.box('blanco', (0.2, 0.02, 0.2), (0, -0.16, 0.38), bevel=0.0)
        cross(0, -0.172, 0.53, 0.14)
        return m
    m.box('blanco', (0.86, 0.6, 0.5), (0, 0.08, 0.04), bevel=0.02)
    m.box('EQUIPO', (0.88, 0.62, 0.035), (0, 0.08, 0.3), bevel=0.0)
    m.box('EQUIPO', (0.88, 0.62, 0.04), (0, 0.08, 0.54), bevel=0.0)
    for row in range(2):
        for col in range(4):
            if row == 0 and col in (1, 2):
                continue
            m.box('luz', (0.12, 0.012, 0.1), (-0.3 + col * 0.2, -0.222, 0.12 + row * 0.24), bevel=0.0)
    m.box('vidrio', (0.26, 0.012, 0.2), (0, -0.222, 0.04), bevel=0.0)
    m.box('blanco_sucio', (0.4, 0.2, 0.03), (0, -0.32, 0.25), bevel=0.0)
    for sx in (-1, 1):
        m.beam('metal_claro', (sx * 0.18, -0.4, 0.04), (sx * 0.18, -0.4, 0.25), t=0.02)
    m.box('blanco', (0.24, 0.02, 0.24), (-0.3, -0.21, 0.6), bevel=0.0)
    cross(-0.3, -0.222, 0.76, 0.18)
    m.cyl('oscuro', 0.22, 0.02, (0.12, 0.12, 0.58), seg=20)
    m.cyl('amarillo', 0.2, 0.006, (0.12, 0.12, 0.6), seg=20)
```

```
    m.cyl('oscuro', 0.17, 0.008, (0.12, 0.12, 0.6), seg=20)
    m.box('blanco', (0.022, 0.12, 0.005), (0.08, 0.12, 0.605), bevel=0.0)
    m.box('blanco', (0.022, 0.12, 0.005), (0.16, 0.12, 0.605), bevel=0.0)
    m.box('blanco', (0.08, 0.022, 0.005), (0.12, 0.12, 0.605), bevel=0.0)
    m.box('blanco', (0.2, 0.3, 0.12), (0.5, 0.22, 0.04), bevel=0.01)
    m.box('rojo', (0.21, 0.04, 0.03), (0.5, 0.22, 0.12), bevel=0.0)
    return m
```

faro/blender/models_ref3.py · 428 líneas

```
"""Modelos de la v0.9 de Faro de Ámbar: vehículos y barcos nuevos, aviones, Aeropuerto, defensas y
AIR DEFENSE rehecha. Los vehículos reales solo inspiran la forma general; los modelos son propios.

Convención: -Y es el frente, Z hacia arriba, 1 unidad ≈ una casilla. EQUIPO y EQUIPO_OSC toman el
color del imperio en el juego.
"""
import math

from lib import Model
from models_ref import launcher_face, tracks, wheels

PI = math.pi
H = PI / 2


# ---------------------------------------------------------------------
# Vehículos
# ---------------------------------------------------------------------
def light_tank():
    """Tanque ligero: casco bajo y anguloso, torreta chica y cañón corto."""
    m = Model('lighttank')
    tracks(m, 0.23, 0.82, 5, 0.044)
    m.box('oliva', (0.42, 0.78, 0.13), (0, 0.02, 0.08), bevel=0.035)
    m.box('oliva_osc', (0.4, 0.18, 0.08), (0, -0.36, 0.11), rot=(0.6, 0, 0), bevel=0.025)
    for sx in (-1, 1):
        m.box('EQUIPO_OSC', (0.03, 0.7, 0.07), (sx * 0.29, 0.02, 0.07), bevel=0.01)
        m.box('luz', (0.04, 0.02, 0.02), (sx * 0.15, -0.43, 0.16), bevel=0.004)
    m.box('EQUIPO', (0.44, 0.14, 0.02), (0, 0.26, 0.21), bevel=0.006)
    m.pivot('turret', 0, 0.06, 0.21)
    m.box('EQUIPO', (0.28, 0.32, 0.1), (0, 0.08, 0.21), bevel=0.04, part='turret')
    m.box('EQUIPO_OSC', (0.24, 0.1, 0.08), (0, -0.09, 0.22), rot=(0.3, 0, 0), bevel=0.02, part='turret')
    m.tube('oscuro', (0, -0.12, 0.27), (0, -0.48, 0.27), r=0.024, seg=10, part='turret')
    m.cyl('oscuro', 0.032, 0.05, (0, -0.5, 0.27), rot=(H, 0, 0), seg=10, base=False, part='turret')
    m.cyl('oliva_osc', 0.045, 0.03, (0.07, 0.12, 0.31), seg=12, part='turret')
    m.tube('negro', (-0.1, 0.2, 0.31), (-0.11, 0.24, 0.62), r=0.005, seg=6, part='turret')
    return m


def heavy_tank():
    """Tanque pesado: casco ancho, faldones laterales, torreta grande en cuña y cañón largo."""
    m = Model('heavytank')
    tracks(m, 0.3, 1.0, 7, 0.054)
    for sx in (-1, 1):
        m.box('EQUIPO_OSC', (0.05, 0.92, 0.12), (sx * 0.38, 0.0, 0.07), bevel=0.012)
        for k in range(5):
            m.box('oscuro', (0.052, 0.012, 0.1), (sx * 0.385, -0.36 + k * 0.18, 0.08), bevel=0.0)
        m.box('oliva', (0.18, 0.96, 0.035), (sx * 0.3, 0, 0.19), bevel=0.01)
    m.box('oliva', (0.56, 0.94, 0.17), (0, 0.02, 0.09), bevel=0.045)
    m.box('oliva_osc', (0.54, 0.22, 0.12), (0, -0.44, 0.12), rot=(0.62, 0, 0), bevel=0.035)
    for sx in (-1, 1):
        m.box('luz', (0.05, 0.02, 0.025), (sx * 0.2, -0.53, 0.2), bevel=0.004)
    m.box('oscuro', (0.3, 0.14, 0.07), (0, 0.4, 0.26), bevel=0.015)
    m.box('EQUIPO', (0.56, 0.2, 0.025), (0, 0.32, 0.27), bevel=0.008)
    m.pivot('turret', 0, 0.06, 0.27)
    m.box('EQUIPO', (0.44, 0.48, 0.16), (0, 0.1, 0.27), bevel=0.05, part='turret')
    m.box('EQUIPO_OSC', (0.18, 0.2, 0.13), (0.15, -0.16, 0.29), rot=(0.3, 0, -0.35), bevel=0.03, part='turret')
    m.box('EQUIPO_OSC', (0.18, 0.2, 0.13), (-0.15, -0.16, 0.29), rot=(0.3, 0, 0.35), bevel=0.03, part='turret')
    m.box('oscuro', (0.46, 0.14, 0.1), (0, 0.33, 0.29), bevel=0.02, part='turret')
    m.cyl('oscuro', 0.045, 0.72, (0, -0.56, 0.35), rot=(H, 0, 0), seg=12, base=False, part='turret')
    m.cyl('oscuro', 0.058, 0.12, (0, -0.42, 0.35), rot=(H, 0, 0), seg=12, base=False, part='turret')
    m.cyl('oscuro', 0.054, 0.08, (0, -0.9, 0.35), rot=(H, 0, 0), seg=12, base=False, part='turret')
    m.cyl('oliva_osc', 0.07, 0.05, (0.12, 0.16, 0.43), seg=14, part='turret')
    m.box('oscuro', (0.12, 0.08, 0.07), (-0.12, 0.12, 0.45), bevel=0.015, part='turret')
    m.box('vidrio', (0.08, 0.01, 0.04), (-0.12, 0.078, 0.47), bevel=0.003, part='turret')
    for sx in (-1, 1):
        m.tube('negro', (sx * 0.16, 0.3, 0.43), (sx * 0.18, 0.35, 0.92), r=0.006, seg=6, part='turret')
    return m


def missile_truck():
    """Camión de misiles: cabina adelante y lanzador inclinado de cuatro tubos atrás."""
    m = Model('missiletruck')
    wheels(m, [(-0.21, -0.3), (0.21, -0.3), (-0.21, 0.1), (0.21, 0.1), (-0.21, 0.3), (0.21, 0.3)], r=0.085, w=0.08)
    m.box('oscuro', (0.36, 0.9, 0.06), (0, 0.0, 0.1), bevel=0.015)
    m.box('EQUIPO', (0.4, 0.26, 0.2), (0, -0.32, 0.16), bevel=0.035)
    m.box('vidrio', (0.34, 0.02, 0.08), (0, -0.452, 0.27), rot=(-0.2, 0, 0), bevel=0.004)
    for sx in (-1, 1):
        m.box('luz', (0.04, 0.02, 0.03), (sx * 0.15, -0.454, 0.2), bevel=0.004)
    m.box('oliva_osc', (0.38, 0.6, 0.06), (0, 0.15, 0.16), bevel=0.015)
    m.pivot('turret', 0, 0.2, 0.22)
    m.box('oscuro', (0.22, 0.14, 0.06), (0, 0.3, 0.22), bevel=0.015, part='turret')
    for ix in range(2):
        for iz in range(2):
            x = -0.07 + ix * 0.14
```

```
            m.tube('EQUIPO', (x, 0.36, 0.25 + iz * 0.12), (x, -0.12, 0.47 + iz * 0.12), r=0.055, seg=12, part='turret')
            m.cyl('blanco', 0.045, 0.012, (x, -0.125, 0.472 + iz * 0.12), rot=(H + 0.43, 0, 0), seg=12, base=False,
part='turret')
    m.beam('oscuro', (0, 0.3, 0.22), (0, 0.18, 0.4), t=0.04, part='turret')
    return m


def apc():
    """Transporte blindado: caja con ruedas, rampa trasera y ametralladora en el techo."""
    m = Model('apc')
    wheels(m, [(-0.24, -0.3), (0.24, -0.3), (-0.24, 0.0), (0.24, 0.0), (-0.24, 0.3), (0.24, 0.3)], r=0.09, w=0.08)
    m.box('EQUIPO', (0.44, 0.84, 0.18), (0, 0.02, 0.1), bevel=0.04)
    m.box('EQUIPO_OSC', (0.42, 0.22, 0.12), (0, -0.38, 0.13), rot=(0.7, 0, 0), bevel=0.03)
    m.box('EQUIPO', (0.38, 0.6, 0.08), (0, 0.08, 0.28), bevel=0.03)
    for sx in (-1, 1):
        for k in range(3):
            m.box('vidrio', (0.012, 0.05, 0.03), (sx * 0.225, -0.12 + k * 0.17, 0.22), bevel=0.0)
        m.box('luz', (0.04, 0.02, 0.025), (sx * 0.15, -0.46, 0.19), bevel=0.004)
    m.box('oscuro', (0.3, 0.03, 0.14), (0, 0.44, 0.13), rot=(0.2, 0, 0), bevel=0.01)
    m.pivot('turret', 0, 0.02, 0.36)
    m.cyl('oscuro', 0.07, 0.04, (0, 0.02, 0.36), seg=14, part='turret')
    m.box('EQUIPO_OSC', (0.12, 0.1, 0.06), (0, 0.0, 0.4), bevel=0.015, part='turret')
    m.tube('negro', (0, -0.04, 0.43), (0, -0.26, 0.43), r=0.012, seg=8, part='turret')
    return m


def launcher():
    """Lanzador de misiles de largo alcance: camión de ocho ejes con un contenedor enorme encima."""
    m = Model('launcher')
    xs = [-0.55, -0.4, -0.22, -0.07, 0.1, 0.25, 0.42, 0.57]
    wheels(m, [(sx * 0.22, y) for y in xs for sx in (-1, 1)], r=0.085, w=0.08)
    m.box('oscuro', (0.36, 1.32, 0.07), (0, 0.02, 0.11), bevel=0.015)
    # Cabina dividida adelante.
    for sx in (-1, 1):
        m.box('EQUIPO', (0.16, 0.22, 0.2), (sx * 0.1, -0.62, 0.15), bevel=0.03)
        m.box('vidrio', (0.12, 0.02, 0.07), (sx * 0.1, -0.733, 0.27), rot=(-0.15, 0, 0), bevel=0.004)
        m.box('luz', (0.04, 0.02, 0.025), (sx * 0.12, -0.735, 0.19), bevel=0.004)
    m.box('EQUIPO_OSC', (0.42, 1.02, 0.08), (0, 0.12, 0.17), bevel=0.02)
    # Contenedor del misil (tubo grande con tapa redonda adelante) y sus soportes.
    m.pivot('turret', 0, 0.12, 0.25)
    m.tube('EQUIPO', (0, 0.66, 0.36), (0, -0.48, 0.36), r=0.13, seg=16, part='turret')
    m.sphere('EQUIPO_OSC', 0.13, (0, -0.48, 0.36), scale=(1, 0.55, 1), part='turret')
    for k in range(4):
        m.cyl('oscuro', 0.135, 0.025, (0, -0.3 + k * 0.28, 0.36), rot=(H, 0, 0), seg=16, base=False, part='turret')
    for y in (-0.25, 0.4):
        m.beam('oscuro', (0, y, 0.21), (0, y, 0.25), t=0.08, part='turret')
    return m


# ---------------------------------------------------------------------
# Barcos
# ---------------------------------------------------------------------
def frigate():
    """Fragata antiaérea: casco fino, celdas de misiles, mástil con radares planos."""
    m = Model('frigate')
    m.hull('gris', [(0.8, 0.2, 0.13, -0.06, 0.14), (0.35, 0.23, 0.15, -0.08, 0.15), (-0.3, 0.22, 0.12, -0.08, 0.16),
                    (-0.66, 0.12, 0.04, -0.04, 0.18), (-0.92, 0.012, 0.01, 0.04, 0.22)])
    m.box('metal_claro', (0.4, 1.36, 0.014), (0, -0.04, 0.15), bevel=0.0)
    for sx in (-1, 1):
        m.box('EQUIPO', (0.012, 1.24, 0.03), (sx * 0.225, -0.02, 0.11), rot=(0, 0, sx * 0.05), bevel=0.0)
    m.box('gris', (0.3, 0.46, 0.16), (0, 0.1, 0.16), bevel=0.02)
    m.box('metal_claro', (0.24, 0.24, 0.14), (0, 0.02, 0.32), bevel=0.02)
    m.box('luz', (0.22, 0.012, 0.03), (0, -0.102, 0.4), bevel=0.0)
    # Radares planos del mástil (en las cuatro caras).
    m.box('gris', (0.12, 0.12, 0.26), (0, 0.04, 0.46), bevel=0.01)
    for (dx, dy, rz) in [(0, -0.062, 0), (0, 0.142, 0), (0.062, 0.04, H), (-0.062, 0.04, H)]:
        m.box('oscuro', (0.1, 0.01, 0.12), (dx, dy, 0.56), rot=(0, 0, rz), bevel=0.0)
    m.tube('oscuro', (0, 0.04, 0.72), (0, 0.04, 0.9), r=0.008, seg=6)
    # Celdas de misiles adelante.
    for ix in range(4):
        for iy in range(3):
            m.box('oscuro', (0.045, 0.045, 0.012), (-0.08 + ix * 0.054, -0.3 + iy * 0.054, 0.155), bevel=0.0)
    m.box('oscuro', (0.3, 0.24, 0.01), (0, 0.56, 0.155), bevel=0.0)
    m.box('blanco', (0.03, 0.12, 0.004), (0, 0.56, 0.162), bevel=0.0)
    # Lanzador giratorio (dispara los misiles antiaéreos).
    m.pivot('turret', 0, -0.52, 0.16)
    m.cyl('gris', 0.08, 0.04, (0, -0.52, 0.16), seg=14, part='turret')
    m.box('metal_claro', (0.16, 0.16, 0.08), (0, -0.52, 0.2), bevel=0.03, part='turret')
    for sx in (-1, 1):
        m.tube('blanco', (sx * 0.05, -0.44, 0.27), (sx * 0.05, -0.66, 0.33), r=0.025, seg=8, part='turret')
    return m


def lander():
    """Lancha de desembarco sobre colchón de aire: faldón negro, cubierta abierta y dos ventiladores atrás."""
    m = Model('lander')
    m.box('goma', (0.62, 1.1, 0.1), (0, 0.0, -0.02), bevel=0.05)
    m.box('EQUIPO_OSC', (0.56, 1.02, 0.05), (0, 0.0, 0.08), bevel=0.02)
```

```
    m.box('metal', (0.36, 0.86, 0.012), (0, -0.04, 0.13), bevel=0.0)
    for sx in (-1, 1):
        m.box('EQUIPO', (0.1, 0.86, 0.12), (sx * 0.23, -0.02, 0.13), bevel=0.02)
        m.box('vidrio', (0.012, 0.12, 0.04), (sx * 0.282, -0.2, 0.21), bevel=0.0)
        # Ventiladores con aro.
        m.cyl('oscuro', 0.15, 0.06, (sx * 0.15, 0.48, 0.32), rot=(H, 0, 0), seg=18, base=False)
        m.cyl('metal', 0.03, 0.08, (sx * 0.15, 0.48, 0.32), rot=(H, 0, 0), seg=10)
        m.beam('oscuro', (sx * 0.15, 0.45, 0.13), (sx * 0.15, 0.45, 0.2), t=0.04)
        m.box('EQUIPO_OSC', (0.012, 0.16, 0.16), (sx * 0.15, 0.6, 0.32), bevel=0.0)
    m.box('amarillo', (0.3, 0.02, 0.004), (0, -0.45, 0.136), bevel=0.0)
    m.box('EQUIPO_OSC', (0.38, 0.04, 0.1), (0, -0.5, 0.12), rot=(-0.4, 0, 0), bevel=0.01)
    return m


def carrier():
    """Portaaviones: casco largo, cubierta plana con pista en ángulo, isla con radar y cazas en cubierta."""
    m = Model('carrier')
    m.hull('gris', [(1.0, 0.3, 0.2, -0.08, 0.16), (0.4, 0.34, 0.22, -0.11, 0.17), (-0.5, 0.32, 0.2, -0.11, 0.18),
                    (-0.88, 0.18, 0.06, -0.06, 0.2), (-1.08, 0.02, 0.01, 0.02, 0.22)])
    # Cubierta de vuelo (más ancha que el casco) y pista en ángulo.
    m.box('oscuro', (0.7, 2.0, 0.025), (0.06, -0.02, 0.19), bevel=0.0)
    m.box('oscuro', (0.36, 0.9, 0.025), (-0.17, 0.4, 0.19), rot=(0, 0, 0.16), bevel=0.0)
    for k in range(9):
        m.box('blanco', (0.012, 0.07, 0.004), (0.02, -0.85 + k * 0.17, 0.217), bevel=0.0)
    m.box('blanco', (0.008, 0.8, 0.004), (0.33, -0.02, 0.217), bevel=0.0)
    m.box('blanco', (0.008, 0.8, 0.004), (-0.28, -0.02, 0.217), bevel=0.0)
    m.box('EQUIPO', (0.6, 0.06, 0.006), (0.06, -0.98, 0.217), bevel=0.0)
    for sx in (-1, 1):
        m.box('EQUIPO', (0.012, 1.7, 0.03), (sx * 0.335, 0.0, 0.13), rot=(0, 0, sx * 0.04), bevel=0.0)
    # Isla (puente) a estribor con radar y mástil.
    m.box('gris', (0.12, 0.34, 0.24), (0.28, 0.18, 0.21), bevel=0.015)
    m.box('metal_claro', (0.1, 0.24, 0.1), (0.28, 0.16, 0.45), bevel=0.01)
    m.box('luz', (0.012, 0.22, 0.03), (0.226, 0.16, 0.49), bevel=0.0)
    m.tube('oscuro', (0.28, 0.16, 0.55), (0.28, 0.16, 0.8), r=0.01, seg=6)
    m.pivot('rotor', 0.28, 0.16, 0.66)
    m.box('oscuro', (0.04, 0.16, 0.05), (0.28, 0.16, 0.66), bevel=0.0, part='rotor')
    # Cazas estacionados (siluetas simples).
    for (x, y, a) in [(-0.12, -0.55, 0.0), (0.06, -0.55, 0.0), (-0.2, 0.6, 0.6), (-0.04, 0.75, 0.6)]:
        m.box('metal_claro', (0.04, 0.2, 0.025), (x, y, 0.21), rot=(0, 0, a), bevel=0.006)
        m.box('metal_claro', (0.16, 0.06, 0.008), (x, y + 0.02, 0.215), rot=(0, 0, a), bevel=0.0)
    # Defensa de corto alcance (dispara a lo que se acerca).
    m.pivot('turret', -0.3, -0.68, 0.2)
    m.cyl('blanco', 0.05, 0.08, (-0.3, -0.68, 0.2), seg=12, part='turret')
    m.sphere('blanco', 0.05, (-0.3, -0.68, 0.3), scale=(1, 1, 1.1), part='turret')
    m.tube('negro', (-0.3, -0.72, 0.26), (-0.3, -0.84, 0.26), r=0.01, seg=6, part='turret')
    return m


# ---------------------------------------------------------------------
# Aviones
# ---------------------------------------------------------------------
def fighter():
    """Caza furtivo (v1.0): fuselaje facetado con quillas, alas en diamante, colas horizontales,
    dos derivas inclinadas con el color del imperio, tomas de aire laterales y toberas planas."""
    m = Model('fighter')
    # Fuselaje: nariz en punta hacia -Y.
    m.hull('gris', [(0.5, 0.06, 0.05, 0.03, 0.09), (0.38, 0.17, 0.14, 0.0, 0.12), (0.05, 0.21, 0.17, 0.0, 0.13),
                    (-0.2, 0.15, 0.1, 0.02, 0.13), (-0.42, 0.07, 0.04, 0.04, 0.11), (-0.6, 0.008, 0.006, 0.065, 0.075)],
           loc=(0, 0, 0.1))
    # Alas en diamante (un solo casco ancho y fino) y colas horizontales.
    m.hull('gris', [(-0.12, 0.2, 0.2, 0.0, 0.02), (0.2, 0.86, 0.86, 0.0, 0.016), (0.24, 0.84, 0.84, 0.0, 0.014),
                    (0.3, 0.34, 0.34, 0.0, 0.016)], loc=(0, 0, 0.15))
    m.hull('gris', [(0.32, 0.22, 0.22, 0.0, 0.014), (0.46, 0.56, 0.56, 0.0, 0.012), (0.5, 0.5, 0.5, 0.0, 0.012),
                    (0.53, 0.26, 0.26, 0.0, 0.012)], loc=(0, 0, 0.16))
    # Cabina dorada y quillas de la nariz.
    m.sphere('oro', 0.055, (0, -0.24, 0.235), scale=(0.75, 1.9, 0.62))
    m.box('oscuro', (0.1, 0.012, 0.02), (0, -0.15, 0.24), bevel=0.0)
    for sx in (-1, 1):
        m.box('gris', (0.012, 0.3, 0.02), (sx * 0.075, -0.36, 0.15), rot=(0, 0, -sx * 0.14), bevel=0.0)
        # Tomas de aire trapezoidales.
        m.box('gris', (0.07, 0.16, 0.08), (sx * 0.12, -0.04, 0.16), rot=(0, sx * 0.25, sx * 0.08), bevel=0.01)
        m.box('negro', (0.05, 0.01, 0.06), (sx * 0.125, -0.12, 0.16), rot=(0, sx * 0.25, 0), bevel=0.0)
        # Derivas inclinadas hacia afuera, en dos piezas, con el color del imperio.
        m.box('EQUIPO', (0.012, 0.17, 0.12), (sx * 0.11, 0.38, 0.26), rot=(0.2, sx * 0.48, 0), bevel=0.0)
        m.box('EQUIPO', (0.012, 0.09, 0.06), (sx * 0.135, 0.42, 0.33), rot=(0.35, sx * 0.48, 0), bevel=0.0)
        # Franja del imperio en la punta del ala y luces de posición.
        m.box('EQUIPO', (0.08, 0.05, 0.004), (sx * 0.36, 0.2, 0.168), rot=(0, 0, sx * 0.5), bevel=0.0)
        m.box('rojo' if sx < 0 else 'cultivo', (0.016, 0.016, 0.01), (sx * 0.43, 0.22, 0.17), bevel=0.0)
        # Toberas planas con el brillo del motor.
        m.box('oscuro', (0.075, 0.06, 0.045), (sx * 0.05, 0.52, 0.17), bevel=0.008)
        m.box('luz', (0.055, 0.01, 0.025), (sx * 0.05, 0.553, 0.17), bevel=0.0)
    return m


def bomber():
    """Bombardero furtivo de ala volante (v1.0): ala en flecha con borde trasero en dientes, joroba
    central con cabina y cuatro tomas de motor arriba."""
    m = Model('bomber')
```

```
    # Ala: casco ancho y fino; la forma en planta sale de los anchos de cada sección.
    m.hull('oscuro', [(-0.46, 0.06, 0.06, 0.0, 0.03), (-0.26, 0.62, 0.62, 0.0, 0.03), (-0.04, 1.28, 1.28, 0.0, 0.026),
                      (0.1, 1.62, 1.62, 0.0, 0.02), (0.16, 1.5, 1.5, 0.0, 0.02), (0.22, 1.02, 1.02, 0.0, 0.022),
                      (0.3, 0.86, 0.86, 0.0, 0.024), (0.36, 0.44, 0.44, 0.0, 0.026), (0.44, 0.3, 0.3, 0.0, 0.026),
                      (0.48, 0.06, 0.06, 0.0, 0.026)], loc=(0, 0, 0.15))
    # Joroba central (cabina y motores).
    m.hull('oscuro', [(-0.42, 0.04, 0.04, 0.0, 0.02), (-0.26, 0.24, 0.3, 0.0, 0.1), (-0.05, 0.36, 0.42, 0.0, 0.12),
                      (0.2, 0.3, 0.36, 0.0, 0.09), (0.38, 0.12, 0.16, 0.0, 0.04)], loc=(0, 0, 0.16))
    for sx in (-1, 1):
        m.box('vidrio', (0.05, 0.05, 0.02), (sx * 0.035, -0.27, 0.252), rot=(0.45, 0, sx * 0.25), bevel=0.004)
        for k, x in enumerate((0.1, 0.2)):
            m.box('camo2', (0.07, 0.14, 0.04), (sx * x, -0.02 + k * 0.01, 0.27), bevel=0.012)
            m.box('negro', (0.05, 0.012, 0.022), (sx * x, -0.09 + k * 0.01, 0.28), bevel=0.0)
            m.box('camo2', (0.05, 0.03, 0.01), (sx * x, 0.18, 0.2), bevel=0.0)
        # Franja del imperio en el borde de ataque y en la punta.
        m.box('EQUIPO', (0.42, 0.02, 0.006), (sx * 0.5, -0.02, 0.18), rot=(0, 0, sx * 0.62), bevel=0.0)
        m.box('EQUIPO', (0.12, 0.06, 0.006), (sx * 0.76, 0.13, 0.172), bevel=0.0)
        m.box('rojo' if sx < 0 else 'cultivo', (0.02, 0.02, 0.01), (sx * 0.8, 0.12, 0.175), bevel=0.0)
    m.box('EQUIPO', (0.04, 0.32, 0.008), (0, 0.02, 0.285), bevel=0.0)
    return m


# ---------------------------------------------------------------------
# Edificios
# ---------------------------------------------------------------------
def apartment():
    """Apartamento (Casa nivel 2): edificio de tres pisos con balcones, ventanas encendidas y azotea."""
    m = Model('house_2')
    m.box('cultivo', (0.95, 0.9, 0.03), (0, 0.0, 0), bevel=0.01)
    m.box('blanco', (0.62, 0.5, 0.66), (0, 0.04, 0.03), bevel=0.02)
    for z in (0.25, 0.47):
        m.box('piedra_clara', (0.635, 0.515, 0.02), (0, 0.04, z), bevel=0.0)
    for f in range(3):
        z = 0.1 + f * 0.22
        for sx in (-1, 0, 1):
            m.box('luz', (0.1, 0.012, 0.09), (sx * 0.19, -0.212, z), bevel=0.0)
        for sy in (-1, 1):
            m.box('luz', (0.012, 0.09, 0.09), (0.312, 0.04 + sy * 0.12, z), bevel=0.0)
            m.box('luz', (0.012, 0.09, 0.09), (-0.312, 0.04 + sy * 0.12, z), bevel=0.0)
        if f > 0:
            for sx in (-1, 1):
                m.box('EQUIPO', (0.18, 0.08, 0.015), (sx * 0.19, -0.25, z - 0.04), bevel=0.0)
                m.box('metal_claro', (0.18, 0.008, 0.05), (sx * 0.19, -0.29, z - 0.012), bevel=0.0)
    m.box('madera', (0.11, 0.012, 0.13), (0, -0.212, 0.03), bevel=0.0)
    m.box('EQUIPO', (0.2, 0.1, 0.015), (0, -0.25, 0.17), bevel=0.0)
    m.box('oscuro', (0.64, 0.52, 0.03), (0, 0.04, 0.69), bevel=0.006)
    m.box('EQUIPO', (0.66, 0.54, 0.03), (0, 0.04, 0.66), bevel=0.004)
    m.cyl('metal_claro', 0.06, 0.1, (0.18, 0.15, 0.72), seg=12)
    m.box('metal', (0.1, 0.08, 0.06), (-0.16, 0.12, 0.72), bevel=0.01)
    m.box('piedra_clara', (0.14, 0.2, 0.012), (0, -0.33, 0.03), bevel=0.0)
    m.sphere('oliva', 0.08, (-0.36, 0.32, 0.08), scale=(1, 1, 0.8))
    m.sphere('cultivo', 0.07, (0.37, 0.33, 0.07), scale=(1, 1, 0.8))
    return m


def airport():
    """Aeropuerto militar (v1.0): pista con cabeceras y luces, calle de rodaje, dos hangares en arco,
    torre de control con radar que gira, tanques de combustible, manga de viento y un caza en la plataforma."""
    m = Model('airport')
    m.hexprism('concreto', 0.95, 0.05)
    ang = 0.35
    ca, sa = math.cos(ang), math.sin(ang)

    def on_runway(u, v):
        # u: a lo ancho de la pista, v: a lo largo (girado).
        return (-0.2 + u * ca - v * sa, u * sa + v * ca)

    m.box('oscuro', (0.36, 1.66, 0.012), (-0.2, 0.0, 0.05), rot=(0, 0, ang), bevel=0.0)
    for k in range(7):
        v = -0.6 + k * 0.2
        x, y = on_runway(0, v)
        m.box('blanco', (0.018, 0.09, 0.004), (x, y, 0.063), rot=(0, 0, ang), bevel=0.0)
    for end in (-0.76, 0.76):
        for j in range(5):
            x, y = on_runway(-0.12 + j * 0.06, end)
            m.box('blanco', (0.03, 0.08, 0.004), (x, y, 0.063), rot=(0, 0, ang), bevel=0.0)
    for side in (-0.17, 0.17):
        for k in range(8):
            x, y = on_runway(side, -0.7 + k * 0.2)
            m.box('luz', (0.016, 0.016, 0.012), (x, y, 0.066), bevel=0.0)
    # Calle de rodaje hasta la plataforma.
    m.box('gris', (0.12, 0.5, 0.01), (0.12, 0.3, 0.05), rot=(0, 0, -0.7), bevel=0.0)
    m.box('amarillo', (0.008, 0.48, 0.004), (0.12, 0.3, 0.061), rot=(0, 0, -0.7), bevel=0.0)
    # Dos hangares en arco con portón del color del imperio.
    for hx, hy in ((0.42, 0.22), (0.5, -0.12)):
        m.cyl('metal', 0.15, 0.32, (hx, hy, 0.05), rot=(0, H, 0), seg=14, smooth=True)
        m.box('concreto', (0.34, 0.32, 0.02), (hx, hy, 0.04), bevel=0.0)
        m.box('EQUIPO', (0.012, 0.22, 0.12), (hx - 0.165, hy, 0.05), bevel=0.0)
```

```
        m.box('oscuro', (0.014, 0.01, 0.12), (hx - 0.166, hy, 0.05), bevel=0.0)
    # Torre de control con cabina de vidrio, antena y radar que gira.
    m.cyl('blanco', 0.06, 0.48, (0.18, -0.52, 0.05), seg=10)
    m.cyl('vidrio', 0.11, 0.08, (0.18, -0.52, 0.53), seg=8, smooth=False, r2=0.13)
    m.cyl('oscuro', 0.135, 0.02, (0.18, -0.52, 0.61), seg=8, smooth=False)
    m.cyl('metal', 0.006, 0.14, (0.22, -0.5, 0.63), seg=6, base=False)
    m.box('rojo', (0.014, 0.014, 0.014), (0.22, -0.5, 0.775), bevel=0.0)
    m.cyl('metal', 0.012, 0.12, (0.46, -0.48, 0.05), seg=6, base=False)
    m.pivot('rotor', 0.46, -0.48, 0.2)
    m.box('metal_claro', (0.18, 0.02, 0.07), (0.46, -0.48, 0.2), rot=(0.3, 0, 0), bevel=0.0, part='rotor')
    # Tanques de combustible y manga de viento.
    for k in range(2):
        m.cyl('blanco', 0.05, 0.1, (-0.62 + k * 0.12, -0.42, 0.05), seg=12)
        m.cyl('EQUIPO', 0.051, 0.015, (-0.62 + k * 0.12, -0.42, 0.12), seg=12)
    m.cyl('metal', 0.005, 0.18, (-0.62, 0.46, 0.05), seg=6, base=False)
    m.cone('naranja', 0.025, 0.08, (-0.58, 0.46, 0.21), rot=(0, -H, 0), seg=8)
    # Caza estacionado en la plataforma (copia pequeña del Caza furtivo).
    px, py = 0.08, 0.6
    m.box('gris', (0.05, 0.26, 0.03), (px, py, 0.07), rot=(0, 0, -0.7), bevel=0.006)
    m.box('gris', (0.24, 0.08, 0.01), (px + 0.01, py + 0.01, 0.075), rot=(0, 0, -0.7), bevel=0.0)
    m.box('EQUIPO', (0.008, 0.05, 0.04), (px + 0.05, py + 0.06, 0.1), rot=(0, 0.4, -0.7), bevel=0.0)
    m.box('oro', (0.024, 0.05, 0.016), (px - 0.03, py - 0.04, 0.09), rot=(0, 0, -0.7), bevel=0.004)
    return m


def antinuke():
    """Escudo antimisiles: radar en cúpula que gira y celdas verticales de interceptores."""
    m = Model('antinuke')
    m.hexprism('concreto', 0.6, 0.08)
    m.box('oliva_osc', (0.5, 0.36, 0.2), (-0.1, 0.12, 0.08), bevel=0.03)
    for ix in range(3):
        for iy in range(2):
            m.cyl('blanco', 0.05, 0.04, (-0.24 + ix * 0.14, 0.06 + iy * 0.13, 0.28), seg=10)
            m.cyl('oscuro', 0.038, 0.004, (-0.24 + ix * 0.14, 0.06 + iy * 0.13, 0.32), seg=10, base=False)
    m.box('EQUIPO', (0.5, 0.04, 0.03), (-0.1, -0.06, 0.26), bevel=0.0)
    m.cyl('metal_claro', 0.1, 0.16, (0.28, -0.24, 0.08), seg=12)
    m.pivot('rotor', 0.28, -0.24, 0.34)
    m.sphere('blanco', 0.16, (0.28, -0.24, 0.34), scale=(1, 1, 0.9), part='rotor')
    m.box('EQUIPO_OSC', (0.18, 0.012, 0.06), (0.28, -0.4, 0.36), bevel=0.0, part='rotor')
    return m


def aadef(level=1):
    """AIR DEFENSE rehecha: remolque con cuatro contenedores de misiles que apuntan al cielo y radar
    plano aparte. Nivel 4: además, un misil nuclear blanco sobre su erector."""
    m = Model('aadef' if level < 4 else 'aadef_4')
    m.hexprism('concreto', 0.55, 0.08)
    for (x, y) in [(-0.42, 0.28), (0.42, 0.28), (-0.42, -0.28), (0.42, -0.28)]:
        m.box('amarillo', (0.06, 0.06, 0.012), (x, y, 0.08), bevel=0.0)
    # Generador y cables.
    m.box('oliva_osc', (0.2, 0.26, 0.14), (0.3, 0.32, 0.08), bevel=0.02)
    m.box('oscuro', (0.16, 0.012, 0.08), (0.3, 0.188, 0.12), bevel=0.0)
    # Radar plano (gira).
    m.cyl('oscuro', 0.035, 0.16, (-0.32, 0.3, 0.08), seg=8)
    m.pivot('rotor', -0.32, 0.3, 0.28)
    m.box('EQUIPO_OSC', (0.3, 0.04, 0.22), (-0.32, 0.3, 0.28), rot=(-0.3, 0, 0), bevel=0.01, part='rotor')
    m.box('oscuro', (0.26, 0.012, 0.18), (-0.32, 0.276, 0.29), rot=(-0.3, 0, 0), bevel=0.0, part='rotor')
    # Lanzador giratorio: cuatro contenedores cuadrados inclinados.
    m.cyl('oscuro', 0.16, 0.08, (0.0, -0.12, 0.08), seg=14)
    m.pivot('turret', 0.0, -0.12, 0.16)
    m.box('oliva_osc', (0.34, 0.3, 0.06), (0.0, -0.12, 0.16), bevel=0.02, part='turret')
    th = -0.7
    for ix in range(2):
        for iz in range(2):
            x = -0.07 + ix * 0.14
            m.box('EQUIPO', (0.13, 0.5, 0.12), (x, -0.08, 0.26 + iz * 0.13), rot=(th, 0, 0), bevel=0.012, part='turret')
            m.cyl('blanco', 0.04, 0.006, (x, -0.29, 0.43 + iz * 0.13), rot=(H + th, 0, 0), seg=10, base=False,
part='turret')
    for sx in (-1, 1):
        m.beam('oscuro', (sx * 0.15, 0.0, 0.18), (sx * 0.15, -0.02, 0.36), t=0.03, part='turret')
    if level >= 4:
        # Misil nuclear sobre su erector, con franja de aviso.
        m.beam('oscuro', (0.36, -0.3, 0.08), (0.36, -0.3, 0.2), t=0.06)
        m.tube('blanco', (0.36, -0.12, 0.22), (0.36, -0.12, 0.86), r=0.06, seg=14)
        m.cone('rojo', 0.06, 0.14, (0.36, -0.12, 0.86), seg=14)
        m.cyl('amarillo', 0.062, 0.05, (0.36, -0.12, 0.6), seg=14, base=False)
        m.cyl('negro', 0.062, 0.02, (0.36, -0.12, 0.66), seg=14, base=False)
        for k in range(4):
            a = k * H
            m.box('oscuro', (0.012, 0.08, 0.1), (0.36 + math.cos(a) * 0.07, -0.12 + math.sin(a) * 0.07, 0.24), rot=(0,
0, a), bevel=0.0)
        m.beam('oscuro', (0.45, -0.12, 0.08), (0.45, -0.12, 0.7), t=0.03)
    return m
```

faro/blender/models_v15.py · 110 líneas

```
"""Modelos de la v1.5: Núcleo nuevo (ciudadela con torre central que crece en cada nivel).

El Faro del mapa es una torre blanca y roja; para no confundirlos, el Núcleo es una ciudadela con
una torre de piedra, franjas y techos del color del jugador y el cristal en lo alto.
"""
import math

from lib import Model
from models_ref import crystal

PI = math.pi


def core15(level):
    """Núcleo v1.5: muros con torres, torre central que crece por nivel y cristal del color del jugador."""
    m = Model(f'core_{level}')
    # Base: zócalo oscuro con un borde del color del jugador.
    m.hexprism('EQUIPO_OSC', 0.995, 0.05, (0, 0, 0.05))
    m.hexprism('oscuro', 0.98, 0.12)
    m.hexprism('piedra', 0.92, 0.08, (0, 0, 0.12))
    wall_h = (0.17, 0.23, 0.29)[level - 1]
    rw = 0.76
    z0 = 0.2
    cs = [(math.cos(PI / 6 + k * PI / 3) * rw, math.sin(PI / 6 + k * PI / 3) * rw) for k in range(6)]
    # Muros con almenas y una franja del color del jugador; la entrada mira al frente.
    for k in range(6):
        (x1, y1), (x2, y2) = cs[k], cs[(k + 1) % 6]
        ang = math.atan2(y2 - y1, x2 - x1)
        gate = k in (3, 4)
        pieces = [(0.0, 0.36), (0.64, 1.0)] if gate else [(0.0, 1.0)]
        for (a, b) in pieces:
            px1, py1 = x1 + (x2 - x1) * a, y1 + (y2 - y1) * a
            px2, py2 = x1 + (x2 - x1) * b, y1 + (y2 - y1) * b
            ln = math.hypot(px2 - px1, py2 - py1)
            cx, cy = (px1 + px2) / 2, (py1 + py2) / 2
            m.box('piedra_clara', (ln, 0.085, wall_h), (cx, cy, z0), rot=(0, 0, ang), bevel=0.012)
            m.box('EQUIPO', (ln, 0.092, 0.04), (cx, cy, z0 + wall_h * 0.42), rot=(0, 0, ang), bevel=0.004)
            n = max(1, int(ln / 0.09))
            for i in range(n):
                t = (i + 0.5) / n
                mx, my = px1 + (px2 - px1) * t, py1 + (py2 - py1) * t
                if i % 2 == 0:
                    m.box('piedra_clara', (0.045, 0.09, 0.045), (mx, my, z0 + wall_h), rot=(0, 0, ang), bevel=0.006)
        if level == 3:
            m.box('oro', (math.hypot(x2 - x1, y2 - y1) * 0.98, 0.095, 0.016), ((x1 + x2) / 2, (y1 + y2) / 2, z0 + wall_h
- 0.016),
                  rot=(0, 0, ang), bevel=0.0)
    # Torres de las esquinas con techo del color del jugador (3 en nivel 1, 6 desde el 2).
    towers = [0, 2, 4] if level == 1 else range(6)
    th = (0.34, 0.42, 0.52)[level - 1]
    for k in towers:
        x, y = cs[k]
        m.cyl('piedra_clara', 0.125, th, (x, y, z0), seg=6, smooth=False)
        m.cyl('oscuro', 0.14, 0.04, (x, y, z0 + th), seg=6, smooth=False)
        m.cone('EQUIPO', 0.15, 0.2 + 0.04 * level, (x, y, z0 + th + 0.04), seg=6)
        m.box('luz', (0.035, 0.012, 0.055), (x, y - 0.122, z0 + th * 0.6), bevel=0.0)
        # Banderín en lo alto de cada torre.
        zt = z0 + th + 0.24 + 0.04 * level
        m.beam('metal', (x, y, zt - 0.06), (x, y, zt + 0.16), t=0.012)
        m.box('EQUIPO', (0.01, 0.12, 0.07), (x, y + 0.062, zt + 0.12), bevel=0.0)
    # Portón de madera en la entrada.
    (gx1, gy1), (gx2, gy2) = cs[3], cs[4]
    gang = math.atan2(gy2 - gy1, gx2 - gx1)
    m.box('madera', (0.2, 0.03, wall_h * 0.85), ((gx1 + gx2) / 2, (gy1 + gy2) / 2, z0), rot=(0, 0, gang), bevel=0.006)
    m.box('metal', (0.21, 0.035, 0.018), ((gx1 + gx2) / 2, (gy1 + gy2) / 2, z0 + wall_h * 0.6), rot=(0, 0, gang),
bevel=0.0)

    # Torre central: más alta en cada nivel, con franjas del color del jugador y ventanas encendidas.
    H = (0.62, 0.88, 1.14)[level - 1]
    r0, r1 = 0.31, 0.22
    m.hexprism('oscuro', 0.45, 0.08, (0, 0, z0))
    m.cyl('piedra_clara', r0, H, (0, 0, z0 + 0.08), seg=6, r2=r1, smooth=False)
    bands = level + 1
    for i in range(bands):
        t = (i + 1) / (bands + 1)
        r = r0 + (r1 - r0) * t
        m.cyl('EQUIPO', r + 0.014, 0.05, (0, 0, z0 + 0.08 + H * t - 0.025), seg=6, smooth=False)
    for k in range(6):
        a = PI / 6 + k * PI / 3
        for j in range(level + 1):
            t = (j + 0.5) / (bands + 1)
            r = (r0 + (r1 - r0) * t) * math.cos(PI / 6) + 0.006
            m.box('luz', (0.05, 0.012, 0.075), (math.cos(a) * r, math.sin(a) * r, z0 + 0.08 + H * t - 0.01),
                  rot=(0, 0, a + PI / 2), bevel=0.0)
```

```
    # Balcón con almenas en lo alto.
    zt = z0 + 0.08 + H
    m.cyl('oscuro', 0.31, 0.05, (0, 0, zt), seg=6, smooth=False)
    for k in range(6):
        a = k * PI / 3
        m.box('piedra_clara', (0.08, 0.05, 0.065), (math.cos(a) * 0.27, math.sin(a) * 0.27, zt + 0.05),
              rot=(0, 0, a + PI / 2), bevel=0.006)
    # Soporte del cristal (dorado en nivel 3).
    prong = 'oro' if level == 3 else 'metal'
    for k in range(3):
        a = k * 2 * PI / 3 + PI / 6
        m.beam(prong, (math.cos(a) * 0.17, math.sin(a) * 0.17, zt + 0.05), (math.cos(a) * 0.07, math.sin(a) * 0.07, zt +
0.26), t=0.035)
    m.cyl('EQUIPO_OSC', 0.12, 0.05, (0, 0, zt + 0.05), seg=6, smooth=False)
    # Nivel 2: arbotantes de los muros a la torre. Nivel 3: anillos dorados y 3 agujas.
    if level >= 2:
        for k in range(3):
            a = k * 2 * PI / 3 + PI / 6
            m.beam('piedra_clara', (math.cos(a) * 0.62, math.sin(a) * 0.62, z0 + wall_h), (math.cos(a) * 0.28,
math.sin(a) * 0.28, z0 + 0.08 + H * 0.45), t=0.055)
    if level == 3:
        m.cyl('oro', r0 + 0.02, 0.035, (0, 0, z0 + 0.08), seg=6, smooth=False)
        m.cyl('oro', 0.33, 0.03, (0, 0, zt + 0.045), seg=6, smooth=False)
        for k in range(3):
            a = k * 2 * PI / 3 - PI / 6
            x, y = math.cos(a) * 0.27, math.sin(a) * 0.27
            m.cone('oro', 0.04, 0.2, (x, y, zt + 0.1), seg=6)
    size = (0.22, 0.27, 0.32)[level - 1]
    crystal(m, zt + 0.14 + size * 1.35, size)
    return m
```

faro/docs/pdf.cjs · 43 líneas

```
// Convierte un PRD en HTML a PDF A4 con número de página.
// Uso: node faro/docs/pdf.cjs faro/docs/prd/PRD_web_v0.3.html [salida.pdf]
// Requiere Playwright con Chromium instalado.
const path = require('path');
const { chromium } = require('playwright');

(async () => {
  const input = path.resolve(process.argv[2] || '');
  if (!input.endsWith('.html')) {
    console.error('[PDF] Indica un archivo .html');
    process.exit(1);
  }
  const output = path.resolve(process.argv[3] || input.replace(/\.html$/, '.pdf'));
  const browser = await chromium.launch();
  try {
    const page = await browser.newPage();
    await page.goto('file://' + input, { timeout: 300000 });
    page.setDefaultTimeout(600000);
    const title = await page.title();
    // Fecha del pie: <meta name="fecha" content="8 NOV 2026"> en el PRD (las versiones viejas no la tienen).
    const fecha = await page.evaluate(() => (document.querySelector('meta[name="fecha"]') || {}).content || '7 NOV
2026');
    // Texto del pie (v1.6): «Prototipo web» salvo que el documento diga otro (<meta name="pie">).
    const pie = await page.evaluate(() => (document.querySelector('meta[name="pie"]') || {}).content || 'Prototipo web');
    const small = 'font-family: DejaVu Sans, Arial, sans-serif; font-size: 7.5pt; color: #5d6b70; width: 100%; padding: 0
18mm;';
    await page.pdf({
      path: output,
      format: 'A4',
      printBackground: true,
      displayHeaderFooter: true,
      headerTemplate: `<div style="${small} display:flex; justify-content:space-between; border-top: 3mm solid #d99a2b;
padding-top: 4mm;">
        <span style="color:#1b6b63; font-weight:700;">FARO DE ÁMBAR</span>
<span>${title.split('·').slice(1).join('·').trim().toUpperCase()}</span></div>`,
      footerTemplate: `<div style="${small} display:flex; justify-content:space-between;">
        <span>${fecha} · ${pie}</span><span><span class="pageNumber"></span> / <span class="totalPages"></span></span>
</div>`,
      margin: { top: '24mm', bottom: '18mm', left: '18mm', right: '18mm' },
    });
    console.log(`[PDF] ✅ ${output}`);
  } finally {
    await browser.close();
  }
})().catch((err) => {
  console.error('[PDF] ❌', err.message);
  process.exit(1);
});
```

faro/docs/doc_style.py · 57 líneas

```
"""Estilo común de los documentos de Faro de Ámbar (PRD, documento completo)."""

CSS = r'''
@page { size: A4; }
:root { --ink: #17313a; --teal: #1b6b63; --amber: #d99a2b; --soft: #f1f3f1; --line: #d9dfdc; --muted: #5d6b70; }
* { box-sizing: border-box; }
body { margin: 0; font-family: 'DejaVu Sans', 'Helvetica Neue', Arial, sans-serif; color: var(--ink); font-size: 10.5pt; line-height: 1.5; }
h1 { font-size: 30pt; margin: 0 0 4px; line-height: 1.1; }
h2 { font-size: 19pt; margin: 18px 0 10px; line-height: 1.2; page-break-after: avoid; }
h3 { color: var(--teal); font-size: 12pt; margin: 16px 0 6px; page-break-after: avoid; }
h4 { font-size: 10.5pt; margin: 12px 0 4px; page-break-after: avoid; }
p { margin: 0 0 8px; }
a { color: var(--teal); }
.eyebrow { font-size: 8.5pt; letter-spacing: 0.06em; color: var(--muted); text-transform: uppercase; margin-bottom: 2px; }
.sub { color: var(--teal); font-size: 15pt; margin: 0 0 12px; }
.meta { color: var(--muted); font-size: 9pt; margin-bottom: 16px; }
.tiles { display: grid; grid-template-columns: repeat(3, 1fr); gap: 10px; margin: 0 0 18px; }
.tile { background: var(--soft); border-radius: 8px; padding: 10px 12px; }
.tile b { display: block; color: var(--teal); font-size: 11pt; }
.tile span { font-size: 9pt; }
section { page-break-before: always; }
section.first { page-break-before: auto; }
table { width: 100%; border-collapse: collapse; margin: 6px 0 12px; font-size: 9pt; page-break-inside: auto; }
th { background: var(--teal); color: #fff; text-align: left; padding: 6px 8px; font-weight: 700; }
td { padding: 6px 8px; border-bottom: 1px solid var(--line); vertical-align: top; overflow-wrap: anywhere; }
tr { page-break-inside: avoid; }
tbody tr:nth-child(odd) td { background: var(--soft); }
ul, ol { margin: 0 0 8px; padding-left: 18px; }
li { margin-bottom: 3px; }
code { font-family: 'DejaVu Sans Mono', Menlo, monospace; font-size: 8.6pt; background: #eef1ef; padding: 0 3px; border-radius: 3px; }
pre { font-family: 'DejaVu Sans Mono', Menlo, monospace; font-size: 7.6pt; line-height: 1.35; background: #f6f7f6; border: 1px solid var(--line); border-radius: 6px; padding: 8px 10px; white-space: pre-wrap; overflow-wrap: anywhere; word-break: break-word; }
pre code { background: none; padding: 0; font-size: inherit; }
.chip { white-space: nowrap; display: inline-block; padding: 1px 7px; border-radius: 10px; font-size: 8pt; font-weight: 700; }
.pend { background: #e3e7e5; color: #3d4a4f; }
.ok { background: #d5f0dd; color: #165a2b; }
.fail { background: #f8d6d6; color: #8a1f1f; }
.note { border-left: 3px solid var(--amber); padding: 6px 10px; background: #fbf5e8; margin: 8px 0 12px; font-size: 9.5pt; }
.small { font-size: 8.5pt; color: var(--muted); }
figure { margin: 10px 0 14px; page-break-inside: avoid; }
figure img, figure svg { width: 100%; height: auto; border-radius: 8px; border: 1px solid var(--line); display: block; }
figcaption { font-size: 8.5pt; color: var(--muted); margin-top: 4px; }
.figgrid { display: grid; grid-template-columns: 1fr 1fr; gap: 10px; }
'''


def head(title, fecha, pie='Prototipo web'):
    return f'''<!doctype html>
<html lang="es">
<head>
<meta charset="utf-8">
<meta name="fecha" content="{fecha}">
<meta name="pie" content="{pie}">
<title>{title}</title>
<style>{CSS}</style>
</head>
<body>
'''
```

faro/docs/roblox/build_prd_roblox.py · 95 líneas

```
"""PRD Roblox v1.6 (escrito por el usuario) → HTML listo para PDF, con las imágenes en su lugar.

Uso: python3 faro/docs/roblox/build_prd_roblox.py   (luego: node faro/docs/pdf.cjs faro/docs/roblox/PRD_Roblox_v1.6.html)
El texto del usuario no se cambia: solo se ponen las capturas en <!-- REFERENCIAS_VISUALES --> y el
esquema de la interfaz en <!-- ESQUEMA_HUD -->.
"""
import os
import sys

import markdown

HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, os.path.dirname(HERE))
from doc_style import head  # noqa: E402

SRC = os.path.join(HERE, 'PRD_Faro_de_Ambar_v1_6.md')
OUT = os.path.join(HERE, 'PRD_Roblox_v1.6.html')

REFS = '''
<div class="figgrid">
<figure><img src="../img/ref-isla-propia.png" alt="Isla propia de cerca: Núcleo, granjas, Cuartel, fábrica, molino y puerto sobre hexágonos verdes y arena, rodeados de mar">
<figcaption><b>Referencia 1 · Isla propia de cerca</b> (captura enviada por el usuario). Hexágonos verdes con borde de arena, mar azul con reflejos, Núcleo al centro con su rayo, granjas, Cuartel, fábrica, molino y puerto; soldados verdes legibles.</figcaption></figure>
<figure><img src="../img/ref-archipielago.jpg" alt="Vista amplia del archipiélago: varias islas con bases de colores, islas pequeñas con energía y petróleo en el mar">
<figcaption><b>Referencia 2 · Archipiélago</b> (captura enviada por el usuario). Islas separadas por mar navegable, bases de otros jugadores con su color en el borde del territorio, islas chicas con recursos y manchas oscuras de petróleo en el mar.</figcaption></figure>
</div>
<p class="small">Las dos capturas son del prototipo web de Faro de Ámbar. Sirven de referencia de composición, color y legibilidad para la versión de Roblox.</p>
'''

HUD_SVG = '''
<figure><svg viewBox="0 0 1180 820" role="img" aria-label="Esquema del HUD en iPad horizontal" xmlns="http://www.w3.org/2000/svg" font-family="DejaVu Sans, Arial, sans-serif">
  <defs><linearGradient id="mar" x1="0" y1="0" x2="0" y2="1"><stop offset="0" stop-color="#4fb3d9"/><stop offset="1" stop-color="#2a8fbf"/></linearGradient></defs>
  <rect width="1180" height="820" fill="url(#mar)"/>
  <ellipse cx="590" cy="420" rx="330" ry="210" fill="#9ccb6b" stroke="#e8d59a" stroke-width="22"/>
  <text x="590" y="410" text-anchor="middle" font-size="30" font-weight="700" fill="#17313a">MAPA</text>
  <text x="590" y="446" text-anchor="middle" font-size="19" fill="#17313a">Centro libre para observar y dar órdenes</text>
  <g fill="#1d2a33" fill-opacity="0.93" stroke="#d99a2b" stroke-width="2">
    <rect x="12" y="12" width="560" height="64" rx="12"/>
    <rect x="586" y="12" width="450" height="64" rx="12"/>
    <rect x="1050" y="12" width="118" height="64" rx="12"/>
    <rect x="12" y="92" width="236" height="250" rx="12"/>
    <rect x="12" y="588" width="236" height="220" rx="12"/>
    <rect x="908" y="92" width="260" height="620" rx="12"/>
    <rect x="262" y="560" width="632" height="248" rx="12"/>
  </g>
  <g fill="#ffffff" font-size="17">
    <text x="30" y="40" font-weight="700" fill="#f2ab38">RECURSOS</text>
    <text x="30" y="64">Monedas · Tropas · Energía · Tiempo</text>
    <text x="604" y="40" font-weight="700" fill="#f2ab38">PARTICIPANTES</text>
    <text x="604" y="64">Colores de cada imperio · marcador</text>
    <text x="1068" y="50" font-weight="700" fill="#f2ab38">Pausa</text>
    <text x="30" y="122" font-weight="700" fill="#f2ab38">OBJETIVOS</text>
    <text x="30" y="148">Primeros pasos o misión</text>
    <text x="30" y="172">Avisos de la partida</text>
    <text x="30" y="618" font-weight="700" fill="#f2ab38">MINIMAPA (plegable)</text>
    <text x="30" y="644">Grupos 1-2-3</text>
    <text x="30" y="668">Ejército · Barcos · Aéreos</text>
    <text x="30" y="692">Recuadro · Núcleo</text>
    <text x="926" y="122" font-weight="700" fill="#f2ab38">CONSTRUCCIÓN (plegable)</text>
    <text x="926" y="150">Pestañas: Economía, Energía,</text>
    <text x="926" y="174">Producción, Defensa</text>
    <text x="926" y="208">Tarjeta: miniatura, coste en</text>
    <text x="926" y="232">amarillo, qué da y límite</text>
    <text x="280" y="590" font-weight="700" fill="#f2ab38">SELECCIÓN Y PRODUCCIÓN</text>
    <text x="280" y="618">Ficha: nombre, nivel, vida en verde</text>
    <text x="280" y="644">Botones: Mejorar · Punto de reunión · Demoler</text>
    <text x="280" y="670">Cola con tiempo y «×» · Repetir · Siempre</text>
    <text x="280" y="696">Tarjetas de tropas con coste y población</text>
  </g>
  <g fill="#f2ab38"><rect x="262" y="736" width="140" height="56" rx="10"/><rect x="414" y="736" width="140" height="56" rx="10" fill-opacity="0.35"/><rect x="566" y="736" width="140" height="56" rx="10" fill-opacity="0.35"/></g>
  <text x="332" y="770" text-anchor="middle" font-size="15" font-weight="700" fill="#1a1206">44 × 44 px mín.</text>
</svg>
<figcaption><b>Esquema del HUD en iPad horizontal</b> (sección 4): recursos y participantes arriba; objetivos y minimapa plegable a la izquierda; construcción plegable a la derecha; selección y producción abajo; el centro queda para el mapa.
```

```
Toda zona táctil mide 44 × 44 px o más.</figcaption></figure>
<figure><img src="../img/hud-v15-ipad.png" alt="Captura del prototipo web v1.5 en iPad horizontal con el Cuartel seleccionado">
<figcaption><b>Cómo se ve hoy en el prototipo web (v1.5)</b>: la misma distribución, con el Cuartel elegido (cola con tiempo, lo que cambia al mejorar y tarjetas de tropas).</figcaption></figure>
'''


def main():
    md = open(SRC, encoding='utf-8').read()
    assert '<!-- REFERENCIAS_VISUALES -->' in md and '<!-- ESQUEMA_HUD -->' in md
    md = md.replace('<!-- REFERENCIAS_VISUALES -->', '\n' + REFS + '\n')
    md = md.replace('<!-- ESQUEMA_HUD -->', '\n' + HUD_SVG + '\n')
    body = markdown.markdown(md, extensions=['tables', 'fenced_code', 'sane_lists'], output_format='html5')
    # Portada: el primer h1 y los dos párrafos que siguen forman la cabecera.
    body = body.replace('<h1>Faro de Ámbar</h1>\n<p>PRD y plan de inicio</p>',
                        '<div class="eyebrow">Faro de Ámbar · Plan para Roblox</div>\n<h1>Faro de Ámbar</h1>\n<p class="sub">PRD y plan de inicio · versión 1.6</p>', 1)
    body = body.replace('<p>Versión 1.6 · 13 de noviembre de 2026 · Diseño visual y desarrollo futuro pendiente</p>',
                        '<p class="meta">Versión 1.6 · 13 de noviembre de 2026 · Diseño visual y desarrollo futuro pendiente · Documento escrito por el usuario; imágenes y esquema añadidos en sus lugares marcados</p>', 1)
    html = head('Faro de Ámbar · PRD Roblox v1.6', '13 NOV 2026', 'Plan para Roblox') + body + '\n</body>\n</html>\n'
    open(OUT, 'w', encoding='utf-8').write(html)
    print('[PRD Roblox] ✅', OUT, f'{len(html) // 1024} KB')


if __name__ == '__main__':
    main()
```

faro/docs/documento/build_documento.py · 276 líneas

```
"""Documento completo de Faro de Ámbar: historia desde la v0.1, cómo está hecho, catálogo y TODO el código.

Uso (desde la raíz del repositorio):
    python3 faro/docs/documento/build_documento.py
    node faro/docs/pdf.cjs faro/docs/documento/Documento_Completo_Faro_de_Ambar.html

Se arma leyendo el proyecto, así que siempre está al día: cada versión nueva agrega sus mejoras en
`registro.md` y vuelve a correr este script (regla del proyecto, ver faro/AGENTS.md).
"""
import datetime
import glob
import html
import json
import os
import re
import subprocess
import sys

import markdown

HERE = os.path.dirname(os.path.abspath(__file__))
FARO = os.path.abspath(os.path.join(HERE, '..', '..'))
sys.path.insert(0, os.path.dirname(HERE))
from doc_style import head  # noqa: E402

OUT = os.path.join(HERE, 'Documento_Completo_Faro_de_Ambar.html')
MES = ['enero', 'febrero', 'marzo', 'abril', 'mayo', 'junio', 'julio', 'agosto', 'septiembre', 'octubre', 'noviembre',
'diciembre']
MES3 = ['ENE', 'FEB', 'MAR', 'ABR', 'MAY', 'JUN', 'JUL', 'AGO', 'SEP', 'OCT', 'NOV', 'DIC']
esc = html.escape


def md(text):
    return markdown.markdown(text, extensions=['tables', 'fenced_code', 'sane_lists'], output_format='html5')


def rel(p):
    return os.path.relpath(p, os.path.dirname(FARO))


# ---------- datos del juego (desde 00-config.js) ----------
def catalog():
    js = r'''
const fs = require('fs'); const vm = require('vm');
const dir = process.argv[1];
const ctx = { console, Math, JSON, Set, Map, Array, Object, Number, String, Infinity, isFinite };
vm.createContext(ctx);
vm.runInContext(fs.readFileSync(dir + '00-config.js', 'utf8') + ';globalThis.OUT=
{U:UNITS,B:BUILDINGS,M:MODES,P:MAPS,V:GAME_VERSION};', ctx);
const o = ctx.OUT;
const pick = (x) => JSON.parse(JSON.stringify(x, (k, v) => (typeof v === 'function' ? undefined : v)));
process.stdout.write(JSON.stringify({ U: pick(o.U), B: pick(o.B), M: pick(o.M), P: pick(o.P), V: o.V }));
'''
    out = subprocess.run(['node', '-e', js, os.path.join(FARO, 'src', 'js') + '/'], capture_output=True, text=True,
check=True).stdout
    return json.loads(out)


# ---------- PRD de cada versión ----------
def prds():
    rows = []
    for f in glob.glob(os.path.join(FARO, 'docs', 'prd', 'PRD_web_v*.html')):
        v = re.search(r'v(\d+\.\d+)\.html$', f).group(1)
        s = open(f, encoding='utf-8').read()
        fecha = (re.search(r'<meta name="fecha" content="([^"]+)"', s) or re.search(r'<p class="meta">([^·<]+)', s))
        estado = re.search(r'id="estado">([^<]+)<', s)
        tiles = re.findall(r'<div class="tile"><b>(.*?)</b>', s)
        chips = re.findall(r'class="chip (ok|fail|pend|np)"', s)
        rows.append({
            'v': v, 'key': tuple(int(x) for x in v.split('.')), 'file': os.path.basename(f),
            'fecha': html.unescape(fecha.group(1)).strip() if fecha else '',
            'estado': html.unescape(estado.group(1)) if estado else '',
            'tiles': [html.unescape(re.sub('<[^>]+>', '', t)) for t in tiles],
            'ok': chips.count('ok'), 'fail': chips.count('fail'), 'pend': chips.count('pend') + chips.count('np'),
            'pdf': os.path.exists(f.replace('.html', '.pdf')),
        })
    return sorted(rows, key=lambda r: r['key'])


# ---------- archivos ----------
CODE_FILES = (
    ['src/index.template.html'] + sorted(os.path.relpath(p, FARO) for p in glob.glob(os.path.join(FARO, 'src', 'js',
'*.js'))) +
    ['build.mjs'] + sorted(os.path.relpath(p, FARO) for p in glob.glob(os.path.join(FARO, 'blender', '*.py'))) +
```

```
    ['docs/pdf.cjs', 'docs/doc_style.py', 'docs/roblox/build_prd_roblox.py', 'docs/documento/build_documento.py', 'README.md', 'AGENTS.md']
)
DESC = {
    'src/index.template.html': 'Estructura de la página y todos los estilos de la interfaz.',
    'build.mjs': 'Une el código, los modelos y los recursos en un solo archivo: faro.html.',
    'README.md': 'Presentación del prototipo, controles, novedades de cada versión y créditos.',
    'AGENTS.md': 'Reglas de trabajo del proyecto (PRD y PDF en cada versión, documento completo).',
    'docs/pdf.cjs': 'Convierte un documento HTML en PDF A4 con encabezado y número de página.',
    'docs/doc_style.py': 'Estilo común de los documentos.',
    'docs/roblox/build_prd_roblox.py': 'Arma el PDF del PRD Roblox v1.6 con sus imágenes.',
    'docs/documento/build_documento.py': 'Arma este documento.',
}


def first_comment(path):
    for line in open(path, encoding='utf-8'):
        t = line.strip()
        if t.startswith('//'):
            return t.lstrip('/ ').strip()
        if t.startswith('"""') and len(t) > 6:
            return t.strip('"').strip()
        if t.startswith('#') and not t.startswith('#!'):
            return t.lstrip('# ').strip()
        if t and not t.startswith(('/*', '*', '"""', "'use strict'", '<!')):
            break
    return ''


def nlines(path):
    with open(path, encoding='utf-8') as fh:
        return sum(1 for _ in fh)


def main():
    cat = catalog()
    P = prds()
    hoy = datetime.date.today()
    fecha = f'{hoy.day} de {MES[hoy.month - 1]} de {hoy.year}'
    files = [(f, nlines(os.path.join(FARO, f))) for f in CODE_FILES if os.path.exists(os.path.join(FARO, f))]
    game_lines = sum(n for f, n in files if f.startswith('src/'))
    total_lines = sum(n for f, n in files)
    models = sorted(glob.glob(os.path.join(FARO, 'assets', 'models', '*.glb')))
    tests = sorted(glob.glob(os.path.join(FARO, 'tests', '*')))
    test_lines = sum(nlines(t) for t in tests if t.endswith(('.cjs', '.py', '.md')))
    ok_total = sum(p['ok'] for p in P)
    parts = []

    # ---------- portada ----------
    parts.append(f'''<section class="first">
  <div class="eyebrow">Faro de Ámbar · Documento completo del proyecto</div>
  <h1>Faro de Ámbar</h1>
  <p class="sub">Todo el juego, de la v0.1 a la v{esc(cat['V'])}</p>
  <p class="meta">{fecha} · Versión actual del prototipo web: <b>{esc(cat['V'])}</b> · Se vuelve a generar en cada versión</p>
  <div class="tiles">
    <div class="tile"><b>{len(P) + 2} versiones</b><span>v0.1 y v0.2 (antes de los PRD) y {len(P)} con su PRD, de la v{P[0]['v']} a la v{P[-1]['v']}.</span></div>
    <div class="tile"><b>{game_lines:,} líneas de juego</b><span>{total_lines:,} líneas con herramientas y documentos; {len(models)} modelos 3D hechos con Blender.</span></div>
    <div class="tile"><b>{ok_total} pruebas aprobadas</b><span>Sumando los PRD; además {len(tests)} archivos de prueba en <code>faro/tests/</code>.</span></div>
  </div>
  <h3>Qué hay en este documento</h3>
  <ol>
    <li>Qué es Faro de Ámbar y cómo se juega</li>
    <li>Línea de tiempo de todas las versiones</li>
    <li>Registro de mejoras (se amplía en cada versión)</li>
    <li>Novedades detalladas de cada versión</li>
    <li>Catálogo actual: tropas, edificios, modos y mapas</li>
    <li>Cómo está hecho: archivos, recursos y forma de construirlo</li>
    <li>Cómo se prueba</li>
    <li>Plan para Roblox (PRD v1.6 del usuario)</li>
    <li>Anexo: el código completo, archivo por archivo</li>
  </ol>
  <div class="note">Regla del proyecto: cada versión nueva agrega sus mejoras al registro (sección 3) y vuelve a generar este documento. Los PRD y PDF de cada versión siguen siendo documentos aparte, en <code>faro/docs/prd/</code>.</div>
</section>''')

    # ---------- 1. qué es ----------
    readme = open(os.path.join(FARO, 'README.md'), encoding='utf-8').read()
    intro = readme.split('## Novedades de la')[0]
    intro = re.sub(r'^# .*\n', '', intro)
    parts.append(f'<section><div class="eyebrow">01 · El juego</div><h2>Qué es Faro de Ámbar y cómo se juega</h2>{md(intro)}</section>')

    # ---------- 2. línea de tiempo ----------
    rows = ['<tr><td>0.1</td><td>6 nov 2026</td><td>Primera versión jugable, entonces llamada «HEXBOX».</td><td>—</td>
```

```
</tr>',
            '<tr><td>0.2</td><td>7 nov 2026</td><td>Transporte, Pozo petrolero, Batería antiaérea y Lanzamisiles.</td><td>—</td></tr>']
    for p in P:
        res = f"{p['ok']} aprobadas" + (f", {p['fail']} fallidas" if p['fail'] else '') + (f", {p['pend']} pendientes" if p['pend'] else '')
        rows.append(f"<tr><td>{p['v']}</td><td>{esc(p['fecha'])}</td><td>{esc(' · '.join(p['tiles'])) or esc(p['estado'])}</td><td>{res}</td></tr>")
    parts.append(f'''<section><div class="eyebrow">02 · Historia</div><h2>Línea de tiempo de todas las versiones</h2>
  <p>Cada versión desde la 0.3 empezó con un PRD (el plan) y terminó con su PDF con los resultados reales de las pruebas.</p>
  <table><thead><tr><th style="width:8%">Versión</th><th style="width:16%">Fecha</th><th>Qué trajo</th><th style="width:22%">Pruebas del PRD</th></tr></thead><tbody>{''.join(rows)}</tbody></table></section>''')

    # ---------- 3. registro ----------
    reg = open(os.path.join(HERE, 'registro.md'), encoding='utf-8').read()
    reg = re.sub(r'^# .*\n', '', reg)
    parts.append(f'<section><div class="eyebrow">03 · Registro</div><h2>Registro de mejoras</h2>{md(reg)}</section>')

    # ---------- 4. novedades detalladas ----------
    nov = '## Novedades de la' + readme.split('## Novedades de la', 1)[1]
    nov = nov.split('## Editar el juego')[0]
    nov = nov.replace('## ', '### ')
    parts.append(f'<section><div class="eyebrow">04 · Detalle</div><h2>Novedades detalladas de cada versión</h2>{md(nov)}</section>')

    # ---------- 5. catálogo ----------
    U = cat['U']
    B = cat['B']
    prod_of = {}
    for bk, b in B.items():
        for t in b.get('produces', []) or []:
            prod_of.setdefault(t, []).append(b.get('name', bk))
    cls_name = {'inf': 'Infantería', 'veh': 'Vehículo', 'ship': 'Barco', 'sub': 'Submarino', 'air': 'Aéreo'}
    urows = []
    for k, u in U.items():
        w = u.get('weapon') or {}
        dps = f"{w['dmg'] / w['cd']:.0f}" if w.get('dmg') and w.get('cd') else '—'
        urows.append(f"<tr><td>{esc(u.get('name', k))}</td><td>{cls_name.get(u.get('cls'), u.get('cls', ''))}</td><td>{u.get('cost', '')}</td><td>{u.get('hp', '')}</td><td>{dps}</td><td>{w.get('range', '—')}</td><td>{u.get('speed', '')}</td><td>{u.get('pop', '')}</td><td>{esc(', '.join(prod_of.get(k, [])))}</td></tr>")
    brows = []
    for k, b in B.items():
        brows.append(f"<tr><td>{esc(b.get('name', k))}</td><td>{b.get('cost', '')}</td><td>{b.get('hp', '')}</td><td>{b.get('build', '')}</td><td>{b.get('maxLevel', '')}</td><td>{esc(b.get('desc', ''))}</td></tr>")
    mrows = ''.join(f"<tr><td>{esc(m.get('name', k))}</td><td>{m.get('players', '')}</td><td>{esc(m.get('desc', ''))}</td></tr>" for k, m in cat['M'].items())
    prow = ''.join(f"<tr><td>{esc(m.get('name', k))}</td><td>{esc(m.get('desc', '') or '')}</td></tr>" for k, m in cat['P'].items())
    parts.append(f'''<section><div class="eyebrow">05 · Catálogo</div><h2>Catálogo actual (v{esc(cat['V'])})</h2>
  <p>Sacado directamente de <code>src/js/00-config.js</code>, así que siempre coincide con el juego. Daño/s = daño por disparo ÷ tiempo entre disparos (sin bonificaciones contra cada tipo).</p>
  <h3>Tropas ({len(U)})</h3>
  <table><thead><tr><th>Tropa</th><th>Tipo</th><th>Coste</th><th>Vida</th><th>Daño/s</th><th>Alcance</th><th>Velocidad</th><th>Comunidad</th><th>Se entrena en</th></tr></thead><tbody>{''.join(urows)}</tbody></table>
  <h3>Edificios ({len(B)})</h3>
  <table><thead><tr><th>Edificio</th><th>Coste</th><th>Vida</th><th>Obra (s)</th><th>Nivel máx.</th><th>Qué hace</th></tr></thead><tbody>{''.join(brows)}</tbody></table>
  <h3>Modos ({len(cat['M'])})</h3>
  <table><thead><tr><th>Modo</th><th>Jugadores</th><th>Reglas</th></tr></thead><tbody>{mrows}</tbody></table>
  <h3>Mapas ({len(cat['P'])})</h3>
  <table><thead><tr><th>Mapa</th><th>Cómo es</th></tr></thead><tbody>{prow}</tbody></table></section>''')

    # ---------- 6. cómo está hecho ----------
    frows = ''.join(f"<tr><td><code>faro/{esc(f)}</code></td><td>{n:,}</td><td>{esc(DESC.get(f) or first_comment(os.path.join(FARO, f)))}</td></tr>" for f, n in files)
    msize = sum(os.path.getsize(m) for m in models)
    mlist = ', '.join(os.path.basename(m)[:-4] for m in models)
    built = os.path.join(FARO, 'faro.html')
    bsize = os.path.getsize(built) / 1048576 if os.path.exists(built) else 0
    parts.append(f'''<section><div class="eyebrow">06 · Cómo está hecho</div><h2>Archivos, recursos y forma de construirlo</h2>
  <p>El juego es una sola página web (<code>faro/faro.html</code>, {bsize:.2f} MB) que <code>build.mjs</code> arma uniendo el código de <code>src/</code>, los modelos 3D, la animación de caminar y el cielo. Usa three.js r128 para el 3D. Funciona en Safari y Chrome, también en el iPad, sin instalar nada.</p>
  <h3>Archivos de código ({len(files)})</h3>
  <table><thead><tr><th style="width:36%">Archivo</th><th style="width:10%">Líneas</th><th>Qué contiene</th></tr></thead><tbody>{frows}</tbody></table>
  <h3>Recursos</h3>
  <ul>
    <li><b>{len(models)} modelos 3D</b> (GLB, {msize / 1024:.0f} KB) generados con Blender por los scripts de <code>faro/blender/</code>: {esc(mlist)}.</li>
    <li><b>Animación de caminar y correr</b> (<code>assets/anim/cmu_gait.json</code>) sacada de la base de captura de movimiento de CMU.</li>
    <li><b>Cielo de ambiente</b> (<code>assets/env/sky_env.jpg</code>) de Poly Haven, licencia CC0.</li>
  </ul>
  <h3>Cómo se construye</h3>
  <pre><code>node faro/build.mjs                                          # genera faro/faro.html
```

```
python3 faro/blender/generate_models.py faro/assets/models  # modelos 3D (módulo bpy de Blender)
node faro/docs/pdf.cjs faro/docs/prd/PRD_web_vX.Y.html      # PDF de un PRD
python3 faro/docs/documento/build_documento.py              # este documento (y luego pdf.cjs)</code></pre>
  </section>''')

    # ---------- 7. pruebas ----------
    trows = ''.join(f"<tr><td><code>{esc(os.path.basename(t))}</code></td><td>{nlines(t) if t.endswith(('.cjs', '.py', '.md')) else ''}</td><td>{esc(first_comment(t))}</td></tr>" for t in tests)
    parts.append(f'''<section><div class="eyebrow">07 · Pruebas</div><h2>Cómo se prueba</h2>
  <p>Cada versión se prueba con partidas simuladas (la lógica del juego corre en Node, sin gráficos) y en Chromium con tamaño de iPad y de celular, tocando la pantalla como lo haría una persona. Las pruebas de versiones anteriores se vuelven a correr en cada versión. Chromium dibuja sin tarjeta gráfica: sus cuadros por segundo solo sirven para comparar versiones, no para saber cómo va en un iPad.</p>
  <p>En los PRD hay <b>{ok_total} pruebas aprobadas</b> en total. Los {len(tests)} archivos de prueba ({test_lines:,} líneas) están en <code>faro/tests/</code>; por su tamaño, aquí se listan y su código queda en esa carpeta.</p>
  <table><thead><tr><th style="width:30%">Archivo</th><th style="width:9%">Líneas</th><th>Qué prueba</th></tr></thead><tbody>{trows}</tbody></table></section>''')

    # ---------- 8. Roblox ----------
    parts.append('''<section><div class="eyebrow">08 · Roblox</div><h2>Plan para Roblox (PRD v1.6 del usuario)</h2>
  <p>El usuario escribió el <b>PRD y plan de inicio v1.6</b> para llevar Faro de Ámbar a <b>Roblox Studio + Luau</b>. Está completo en <code>faro/docs/roblox/</code> (Markdown original y PDF con sus imágenes). El prototipo web sigue siendo el banco de pruebas de reglas y controles; el desarrollo en Roblox empieza cuando el usuario lo indique.</p>
  <table><thead><tr><th style="width:20%">Etapa</th><th style="width:34%">Herramientas</th><th>Para qué</th></tr></thead><tbody>
  <tr><td>1. Empezar</td><td>Roblox Studio + Luau + Rojo + Git</td><td>Base del juego, mecánicas, código organizado y versiones.</td></tr>
  <tr><td>2. Intermedio</td><td>Blender + Animation Editor + SurfaceAppearance</td><td>Personajes, vehículos, edificios, animaciones y materiales.</td></tr>
  <tr><td>3. Completar</td><td>Roblox Studio + Luau + DataStoreService</td><td>Menús, inventario, misiones, controles y guardado del progreso.</td></tr>
  <tr><td>4. Pulir</td><td>MicroProfiler + Developer Console + simuladores</td><td>Errores, rendimiento, multijugador y móvil.</td></tr>
  <tr><td>5. Publicar</td><td>Roblox Studio + Creator Hub + Analytics</td><td>Publicar, revisar y mejorar con los resultados.</td></tr>
  </tbody></table>
  <div class="note">Lo que el prototipo web ya probó (reglas, controles en iPad, bots, campaña, Barcaza, Cúpula, progreso) sirve de referencia para las pruebas CA-01 a CA-16 del plan de Roblox. Los modelos GLB de <code>faro/assets/models/</code> se pueden importar en Roblox Studio (3D Importer).</div>
  </section>''')

    # ---------- anexo: código ----------
    parts.append(f'''<section><div class="eyebrow">Anexo</div><h2>El código completo, archivo por archivo</h2>
  <p>{len(files)} archivos, {total_lines:,} líneas, en el orden en que se arma el juego: la página, la lógica y el dibujo (<code>src/js/</code>, que se ejecutan en orden de nombre), el armado, los modelos de Blender, las herramientas de documentos y las reglas del proyecto. El archivo final <code>faro.html</code> no se imprime porque es este mismo código unido a los modelos.</p>
  <ul>{''.join(f'<li><code>faro/{esc(f)}</code> · {n:,} líneas</li>' for f, n in files)}</ul></section>''')
    for f, n in files:
        text = open(os.path.join(FARO, f), encoding='utf-8').read().rstrip('\n').split('\n')
        width = len(str(len(text)))
        body = '\n'.join(f'<span class="ln">{i:>{width}}</span> {esc(line)}' for i, line in enumerate(text, 1))
        parts.append(f'<div class="codefile"><h3>faro/{esc(f)} <span class="small">· {n:,} líneas</span></h3><pre class="code">{body}</pre></div>')

    extra = '''<style>
  .codefile { page-break-before: always; }
  .codefile h3 { margin-top: 0; }
  pre.code { font-size: 6.4pt; line-height: 1.28; padding: 6px 8px; background: #fbfbfa; }
  pre.code .ln { color: #9aa6a3; user-select: none; }
</style>'''
    title = f'Faro de Ámbar · Documento completo v{cat["V"]}'
    doc = head(title, f'{hoy.day} {MES3[hoy.month - 1]} {hoy.year}', 'Documento completo') + extra + '\n'.join(parts) + '\n</body>\n</html>\n'
    open(OUT, 'w', encoding='utf-8').write(doc)
    print(f'[Documento] ✅ {rel(OUT)} · {len(doc) // 1024} KB · {len(files)} archivos, {total_lines} líneas de código')


if __name__ == '__main__':
    main()
```

faro/README.md · 256 líneas

```
# Faro de Ámbar · prototipo web

Juego de estrategia en tiempo real sobre un mapa de hexágonos, hecho para el navegador.
Funciona en iPad, celular y computadora (Safari o Chrome), sin instalar nada. Es el banco de
pruebas de las reglas del PRD 1.5 antes de llevarlas a Roblox Studio.

Empiezas con tu Núcleo, una Granja y un Constructor: entrena más, levanta tu base, conquista
islas y mantén el **Faro** (un 50 % más de monedas). Los rivales son bots que construyen,
mejoran, capturan, cruzan el mar y atacan.

**Versión actual:** 1.5 · PRD y PDF en `docs/prd/`.

## Cómo jugar

Abre `faro.html` en Safari o Chrome. En el iPad lo más cómodo es el enlace publicado.

| Acción | Toque (iPad / celular) | Computadora |
| --- | --- | --- |
| Mover el mapa | Arrastra con un dedo | Arrastra o WASD / flechas |
| Zoom y giro | Pellizca y gira con dos dedos | Rueda del ratón, Q y E |
| Seleccionar | Toca una tropa o un edificio | Clic |
| Todas las tropas de un tipo | Doble toque | Doble clic |
| Varias tropas | Botón «Recuadro» y arrastra | Mayús + arrastrar |
| Rueda de selección | Mantén pulsado el mapa y elige (Tierra, Mar, Aire, Todo, Constructores, grupos) | Tab |
| Patrullar / mantener posición | Botones «Patrullar» (y luego toca el otro punto) y «Mantener posición» en el panel de tropas | Igual |
| Grupos 1, 2 y 3 | «Guardar en 1/2/3» y botones 1-2-3 junto al minimapa (dos toques: ir) | Número elige; Ctrl, Cmd o Mayús + número guarda |
| Avanzar atacando | Botón en el panel de tropas y luego toca el destino | F y clic |
| Constructores | Botón junto al minimapa (primero los que no trabajan) | C |
| Mover / atacar | Toca un destino o un enemigo | Clic izquierdo o derecho |
| Construir | Elige edificio → toca una casilla dorada → «Construir» (puedes poner varias seguidas) → «Terminar» | Igual; R gira, Enter construye |
| Mejorar | Toca un edificio terminado → «Mejorar» | Igual |
| Capturar | Lleva infantería junto a una Bandera o al Faro | Igual |
| Pausa / salir | Botón de pausa → «Salir al menú» (la partida queda guardada) | Esc o P |

**Ganar:** destruye todos los Núcleos enemigos (en «Rey del Faro», también manteniendo el Faro
3 minutos). **Perder:** si cae tu Núcleo.

## Novedades de la v1.5

- **Sin decoraciones de temporada:** ya no hay calabazas ni nieve por la fecha. Las temporadas de la Liga y del Pase siguen.
- **Portada nueva:** la imagen grande con el botón JUGAR adentro, el modo con la miniatura del mapa, iconos dibujados y tus números en fichas.
- **Núcleo nuevo (Blender):** una ciudadela con una torre central que crece en cada nivel, franjas y techos del color del jugador y un aro de luz que gira alrededor del cristal. El rayo del cielo tiene el color del jugador.
- **Edificios:** banderín del color del dueño, marcas doradas de nivel, barra celeste mientras se mejoran y aviso «zz» si un edificio que entrena se queda sin cola.
- **Fichas:** la cola dice cuánto falta y cada tropa tiene su «×» para quitarla (devuelve el dinero); lo que cambia al mejorar se ve siempre; las tarjetas de tropas ya no se cortan.
- **Tropas:** anillo del color del jugador debajo de cada una, icono de su tipo al alejar la cámara, ficha con vida, daño, alcance, velocidad y contra qué es fuerte o débil; con varias elegidas, tocar un tipo deja solo esas.
- **Errores arreglados (caza de errores):**
  - Dúos en el mapa Arrecife Doble no se podía empezar.
  - Un Constructor o Ingeniero mandado a reparar un edificio rodeado se quedaba esperando para siempre.
  - «Avanzar atacando» podía quedar quieto viendo enemigos al otro lado del agua.
  - La fila de jugadores de arriba se partía en dos líneas en el iPad.

## Novedades de la v1.4

- **Mandar mejor a tu ejército:**
  - **Patrullar:** van y vuelven entre dos puntos, peleando en el camino.
  - **Mantener posición:** se quedan quietas y solo disparan a lo que tienen a tiro.
  - **Tropas veteranas:** 1, 2 y 3 estrellas con 3, 8 y 15 eliminaciones. Cada estrella da +10 % de vida y +8 % de daño. Los bots también las ganan.
  - **Reparación sola:** los Constructores e Ingenieros libres reparan lo dañado a 4 casillas. Se puede apagar.
  - **Producción continua:** el botón «Siempre» sigue entrenando la última tropa mientras alcance el dinero y la comunidad.
- **Más motivos para volver:**
  - **12 logros** con insignia y cristales; se ven en el Inventario.
  - **Top jugadores** (documento, pág. 20) con pestañas Liga 3 vs 3, Victorias e Islas tomadas. Tu fila siempre se ve. Los demás son comandantes bot.
  - **Desafío semanal:** mapa, semilla y regla fijos toda la semana. Cambia el lunes y la primera victoria da 150 cristales.
- **Para tu iPad** (en la pausa):
  - **Ahorro de batería:** unos 30 cuadros por segundo y menos partículas.
  - **Colores para daltonismo:** elegidos con simulación de los 3 tipos de daltonismo, con una forma por jugador en el minimapa.
  - **Letra grande.**
- **Arreglo:** si los soldados no pueden llegar a la orilla junto a la Barcaza (lejos de la costa o con la orilla ocupada por edificios), la Barcaza va a buscarlos. Antes esperaban para siempre.

```

```
## Novedades de la v1.3

Los 29 puntos de la revisión «Bugs y mejoras»:

- **Bugs arreglados:**
  - «Avanzar atacando» ya no se congela con enemigos inalcanzables.
  - Abandonar la Liga cuenta como derrota.
  - El misil nuclear no daña lo tuyo ni a tus aliados, y la Cúpula lo frena.
  - Ninguna tropa de tierra queda sobre el agua.
  - «Construye 12 edificios» cuenta obras terminadas.
  - Demoler devuelve la mejora en curso.
  - El primer objetivo, la fila «aliado» y los textos de la Casa y del Cuartel ya están bien.
- **Más fluido:**
  - El guardado es unas 6-8 veces más chico, se hace cada minuto y al pausar o cambiar de app.
  - Los bots piensan por turnos (nunca dos en el mismo paso).
- **Interfaz:**
  - Panel de selección compacto: los botones principales arriba y «Más detalles» para el resto.
  - En el celular el panel ya no tapa la barra de arriba, y en el iPad deja libre el centro.
  - Todos los botones del juego miden 44 × 44 px como mínimo.
- **Como el documento de juego v2:**
  - 6 rangos (Recluta, Cabo, Sargento, Capitán, Comandante y Almirante del Faro), solo en la Liga 3 vs 3 y desde el nivel 10.
  - Recién llegado a un rango, la primera derrota no te baja.
  - Temporadas de 2 meses: el pase vuelve a empezar y la Liga baja 1 rango.
  - Tutorial de 3 pasos con Puerto y Barcaza para tomar la isla del bot.
  - Muro a 30.
  - El modo «Relámpago» ahora se llama «Choque exprés».
- **Copia de tu progreso:** un código o un archivo para llevar tu nivel, cristales, aspectos y rango a otro iPad.

## Novedades de la v1.2

- **Rueda de selección:** mantén pulsado el mapa (o la tecla Tab) y elige Tierra, Mar, Aire, Todo,
  Constructores o un grupo; puedes soltar el dedo sobre la opción.
- **Mensajes:** ventana con pestañas Todos / Aliados y mensajes rápidos («¡Gracias!», «¡Bien hecho!»,
  «¡Cuidado!», «¡Ayuda!», «¡Ataquemos!»); los bots aliados contestan. Se abre desde el marcador o tocando
  los mensajes de la esquina.
- **Tabla de la Clasificatoria:** tú y 20 comandantes bot (marcados como bots), ordenados por puntos.
- **Colecciones nuevas:** «Frente 1940» y «Marea» (soldados, vehículos, barcos y Núcleo).
- **Planos, fragmentos y Taller:** un cofre da un aspecto, un plano o fragmentos; los repetidos dan
  fragmentos; en el Taller, plano + fragmentos = aspecto.
- **Mapas nuevos:** «Mesetas de Cobre» (desde una meseta se dispara y se ve 1 casilla más lejos) y
  «Dunas Rojas» (desierto con oasis). Todos los mapas muestran su miniatura al elegir.
- **Tutorial que señala:** un aro y un texto marcan qué tocar en cada paso; si el lugar está fuera de la
  pantalla o tapado por un panel, señala el punto en el minimapa.
- **Temporadas:** «Calabazas» (15 oct-5 nov) e «Invierno» (10 dic-6 ene, con nieve), solas según la fecha
  o elegidas en la pausa.
- **Toques más fáciles:** tocar el cuerpo o el techo de un edificio también lo elige.

## Novedades de la v1.1

- **Alianzas** en los modos de todos contra todos: desde el marcador, «Aliarse» con un rival (el bot acepta o
  no, y dice por qué) y «Romper». Los aliados no se atacan y comparten lo que ven; si solo quedan aliados, la
  alianza se rompe: gana uno solo.
- **Dúos:** 3 parejas (tú y un bot aliado), cada pareja en su zona del mapa.
- **Clasificatoria:** 3 vs 3 por puntos con 10 rangos (Bronce I a Ámbar); el panel «Rangos» del vestíbulo
  muestra tu rango, tus puntos y tus últimas partidas.
- **Órdenes a aliados:** «¡Ayuda!» y «¡Ataquemos!» en el marcador.
- **Escudo antimisiles:** además del misil nuclear, derriba misiles y cohetes enemigos a 3 casillas.
- **Música propia** generada por el juego (vestíbulo y batalla, más intensa en combate) con volumen en la pausa.
- **Espectador:** al perder, «Seguir mirando» hasta el final.
- **Resumen final** con gráficas de monedas por minuto y tropas de cada imperio.

## Novedades de la v1.0

- **Construir:** vuelve el botón «Construir» (tocar una casilla solo muestra la obra) y el constructor tiene
  que llegar y quedarse pegado a la obra para que avance; mientras trabaja, martilla.
- **Base grande:** comunidad hasta 240 (8 Casas). Con el Núcleo en nivel 3: «Mejorar todo» (un clic) y
  «Mejoras automáticas» hasta dejar todo al máximo, guardando una reserva de monedas.
- **Misil nuclear:** deja 1 minuto de radiación que quema a los enemigos de dentro (con cuenta regresiva), y
  «Lanzar los 3» dispara los tres AIR DEFENSE de nivel 4 a la vez (un Escudo antimisiles solo derriba uno).
- **Difícil de verdad:** más monedas, piensa y ataca antes, y tácticas propias (busca Constructores y
  economía, retira heridos antes, defensa antiaérea al ver aviones, mejora y usa el misil antes).
- **Aviones:** Caza, Bombardero y Aeropuerto rehechos en Blender; los aviones dan vueltas en el aire, se
  inclinan y dejan estela; fichas con misiles, bombas y si están ocultos, orden «Bombardear zona» y torre de
  control en el Aeropuerto.
- **Portada:** más grande, con cazas, Bombardero, Apache y Portaaviones, logo, versión y novedades.

## Novedades de la v0.9

- **Ejército nuevo:** Aeropuerto militar con Caza furtivo y Bombardero furtivo; Portaaviones, Fragata y
  Lancha de desembarco en el Puerto; Tanque ligero, Tanque pesado, Camión de misiles, Transporte blindado
  y Lanzacohetes en la Fábrica; Médico en el Cuartel. El Jeep y el Transporte llevan soldados.
- **Helicópteros:** el Helicóptero tiene ahora el equipo y el precio del Apache viejo (con artilleros en
  las puertas); el Apache cuesta 10 000 y es más fuerte (cañón, misiles y bengalas). Sin límite propio,
  vuelven solos a repararse al Helipuerto o al Aeropuerto. Se ve cuándo disparan y lanzan misiles.
- **Defensas:** 20 Torretas; la Torreta nivel 4 tiene 3 miniguns, más daño y tiro antiaéreo. La AIR
  DEFENSE fija el blanco con el radar, prioriza aeronaves y lanza ráfagas de 2 misiles, con ficha de
  radar. Nivel 4 (solo 3): misil nuclear carísimo con 3 minutos de recarga; el Escudo antimisiles lo derriba.
```

```
- **Reglas:** comunidad hasta 180 (y más por eliminar tropas y Núcleos), Casa nivel 2 = Apartamento
  (+20), 6 Helipuertos, cada edificio con su escudo, escudo de las tropas solo dentro de la Cúpula,
  barras de escudo en azul, amarillo y rojo, tarjeta «Qué mejoró» en cada mejora, tierra fértil más
  grande, construir de un toque y todas las tropas armadas dañan edificios.
- **Modelos y portada:** soldados más realistas (chaleco, mochila, brazos que sostienen el arma),
  Núcleo con banderas, Apartamento, aviones, barcos y vehículos nuevos hechos en Blender, y una portada
  con una escena 3D del juego.
- **Calidad:** «Más fluido» usa materiales más livianos y «Alta» se ve más nítida en pantallas Retina.
- **Suelo de la v0.6:** vuelven los colores lisos por terreno; se quitaron las texturas de la v0.8.

## Novedades de la v0.8

- **Interfaz:** suelo con texturas escaneadas de Poly Haven, flechas en el borde cuando atacan algo
  tuyo fuera de la pantalla, barra de producción (con los edificios parados en amarillo), iconos en
  los botones y ficha del enemigo (vida, nivel y qué usar contra él).
- **Bots más listos:** defensa en equipo con una Torreta del lado del ataque, se forman en línea antes
  de atacar, fuego concentrado en el enemigo más débil, hostigamiento a Granjas y Minas, aliados que
  atacan al mismo rival y se ayudan, y oleadas con todo el ejército pasados 15 minutos.
- **Soldados con movimiento real:** caminar y correr de la base de datos de captura de movimiento de
  CMU, procesados en Blender y aplicados a piernas con rodilla.

## Novedades de la v0.7

- **Campaña «La ruta del Faro»:** 6 misiones con historia, meta, progreso en pantalla y hasta 3
  estrellas cada una (Primera luz, Defender la costa, El Faro, Cruzar el mar, Asedio y La gran
  batalla). Cada misión se abre al terminar la anterior; premio de cristales la primera vez y por
  cada estrella nueva.
- **Mejores controles:** grupos 1-2-3, «Avanzar atacando», formación en bloque o en línea (de frente
  al destino), «Repetir» y «×5» en los edificios que entrenan, y botón Constructores.

## Novedades de la v0.6

- **Ritmo más rápido:** cada imperio empieza con 1 Constructor y los bots atacan antes (primera
  oleada hacia el minuto 4-5 en Normal).
- **Modo fluido para iPad:** gráficos «Automática» (baja la calidad sola si el iPad no llega a 40
  cuadros por segundo), «Alta calidad» y «Más fluido», y un contador de cuadros por segundo.
- **15 modelos más con Blender:** Cuartel, Fábrica, Puerto, Casa, las cinco energías, Submarino,
  Apache Guardián, Camión antiaéreo, Torreta chica, Puesto avanzado y Enfermería 1-3.
- **Efectos y sonidos:** explosiones con fuego, humo y restos, fogonazos, estelas, chispas,
  salpicaduras, humo en edificios dañados y sonidos sintetizados nuevos.

## Contenido de la v0.5

- **Constructores:** las obras quedan «En construcción» hasta que un Constructor, un Ingeniero
  (el doble de rápido) o un Barco constructor (obras de mar) las termina.
- **Niveles 1-3** en todos los edificios (Casa hasta 4). Núcleo con láser en todos los niveles y
  doble láser en el 3; la Cúpula se compra con el Núcleo en nivel 3 y, en su nivel 3, da escudo a
  tus tropas. Torreta con 1 minigun en nivel 2 y 2 en nivel 3. Enfermería: clínica, mini hospital
  y hospital grande.
- **Cinco energías:** Solar, Eólica, Hidráulica, Mareomotriz y Undimotriz, cada una en su lugar.
- **Lugares de recursos:** Granjas en tierra fértil, Minas en vetas de oro, Bombas en petróleo de
  tierra y Plataformas en petróleo del mar (sin aviso de territorio). Cada edificio tiene límite.
- **20 unidades** y **AIR DEFENSE**. 5 mapas: Archipiélago Ámbar, Mar abierto, Continente,
  Bahía Gris y Arrecife Doble.
- **Vestíbulo:** PLAY, CAMBIAR modo, Personalizado, Tutorial, Tus números, Pase de 40 niveles,
  misiones, premio diario, códigos, Tienda (apariencias y cofres) e Inventario.
- **Volver a la partida:** salir al menú la deja en pausa y se guarda sola cada 15 s.
- **Modo creador** (solo para la cuenta dueña del juego): monedas y energía infinitas.
- **Modelos 3D hechos con Blender** (`blender/`), incluido el skin Brigada Tóxica, y luz de
  ambiente con un cielo de [Poly Haven](https://polyhaven.com) (CC0, «Kloofendal 48d Partly
  Cloudy», Greg Zaal).

## Editar el juego

El código está en `src/` y se une en un solo archivo:

```bash
node faro/build.mjs                                  # genera faro/faro.html
node faro/docs/pdf.cjs faro/docs/prd/PRD_web_v0.7.html  # PDF del PRD (requiere Playwright)
python faro/blender/generate_models.py faro/assets/models  # modelos 3D (requiere el módulo bpy)
python faro/blender/cmu_gait.py <carpeta_bvh> faro/assets/anim/cmu_gait.json  # caminar y correr de CMU
```

Archivo	Qué contiene
`src/js/00-config.js`	Costos, niveles, energías, límites, unidades, modos, dificultad y misiones
`src/js/02-map.js`	Generación de islas, lugares de recursos, el Faro y las Banderas
`src/js/03-models.js`	Carga de los modelos GLB, aspectos y luz de ambiente
`src/js/04-world.js`	Reglas: obras, mejoras, economía, combate, captura, victoria
`src/js/045-save.js`	Guardar y cargar la partida
`src/js/046-campaign.js`	Campaña: preparación de cada misión, metas, progreso y estrellas
`src/js/05-ai.js`	Comportamiento de los bots
`src/js/06-render.js`	Gráficos, Cúpulas, minimapa, cámara y calidad
`src/js/061-fx.js`	Partículas: explosiones, humo, chispas, estelas y agua
`src/js/07-input.js`	Controles táctiles, ratón y teclado
`src/js/08-ui.js`	Barra superior, paneles, avisos y sonido
`src/js/085-meta.js`	Vestíbulo, progresión, tienda, final de partida y modo creador
`blender/`	Generador de modelos 3D y ciclos de paso de CMU (`cmu_gait.py`) para Blender
```

```
| `assets/anim/` | Ciclos de caminar y correr sacados de CMU |
| `src/index.template.html` | Estructura y estilos de la interfaz |

Cada versión empieza con su PRD y termina con su PDF (ver `AGENTS.md`).

Usa [three.js](https://threejs.org) r128 desde cdnjs y su cargador GLTF desde jsDelivr.

## Créditos

- Cielo de ambiente: [Poly Haven](https://polyhaven.com) (CC0).
- Captura de movimiento: CMU Graphics Lab Motion Capture Database (mocap.cs.cmu.edu), conversión
  BVH de B. Hahne (archivos 02_01 y 35_17), de uso libre.
```

faro/AGENTS.md · 45 líneas

```
# Faro de Ámbar · reglas de trabajo

Prototipo web del juego de estrategia **Faro de Ámbar** (nombre de trabajo). La ruta final es
Roblox Studio + Luau según el PRD 1.5 del usuario; esta versión web sirve para probar reglas y
controles en el iPad mientras tanto.

## Regla obligatoria: PRD y PDF en cada versión

Pedido explícito del usuario: **cada versión del juego empieza con un PRD y termina con su PDF.**

1. Antes de programar, escribe `docs/prd/PRD_web_vX.Y.html`: objetivo, alcance, reglas y números,
   controles, pruebas de aceptación (estado «Pendiente») y decisiones abiertas.
2. Envía al usuario el PDF del plan antes de implementar.
3. Al terminar, rellena el estado real de cada prueba (Aprobada / Fallida / No probada) con su
   evidencia y vuelve a generar el PDF: `node faro/docs/pdf.cjs faro/docs/prd/PRD_web_vX.Y.html`.
4. Nunca marques «Aprobada» una prueba que no se ejecutó. Las pruebas en navegador no sustituyen
   una prueba en un iPad físico: dilo.

## Regla obligatoria: documento completo en cada mejora (desde la v1.6)

Pedido explícito del usuario: **un documento completo del juego, desde la primera versión, con todos
los archivos y el código, que se amplía cada vez que hay una mejora.** Es aparte del PRD/PDF.

1. Agrega las mejoras y partes importantes de la versión en `docs/documento/registro.md`.
2. Vuelve a generarlo: `python3 faro/docs/documento/build_documento.py` y
   `node faro/docs/pdf.cjs faro/docs/documento/Documento_Completo_Faro_de_Ambar.html`.
3. Si agregas archivos de código o herramientas, súmalos a `CODE_FILES` en `build_documento.py`.
   Las pruebas van en `tests/` (el documento las lista).

## Fuentes del usuario

- `docs/roblox/PRD_Faro_de_Ambar_v1_6.md` (13 nov 2026): PRD y plan de inicio para Roblox escrito por
  el usuario (5 etapas). El desarrollo en Roblox empieza solo cuando el usuario lo indique.
  `docs/roblox/build_prd_roblox.py` arma su PDF con las capturas de `docs/img/`.

- `PRD_Faro_de_Ambar_v1_5.pdf` (6 nov 2026): PRD maestro para Roblox. Sus cifras marcadas como
  EJEMPLO no son precios aprobados.
- Las capturas de otros juegos son referencias de composición. No copies nombres, números, mapas
  ni arte de otros juegos.

## Código

- `src/js/*.js` se une con `node faro/build.mjs` en `faro/faro.html`.
- Balance en `src/js/00-config.js`; reglas en `04-world.js`; bots en `05-ai.js`.
- Interfaz en español, pensada para iPad horizontal; prueba también iPad vertical y celular.
```